Nmap: сканирование сетей

Русский перевод официального руководства Гордона Лайона по Nmap: обнаружению узлов, сканированию портов, определению служб и операционных систем, NSE и обходу сетевой защиты.

Больше крутых материалов в моём телеграм-канале @grogrigs

Nmap: сканирование сетей

Официальное руководство проекта Nmap по обнаружению сетевых узлов и сканированию безопасности

Гордон «Фёдор» Лайон

Эта книга, написанная автором и сопровождающим Nmap, охватывает всё - от основ сканирования портов для начинающих до конструирования пакетов, которым пользуются опытные взломщики, - и подходит специалистам по безопасности и сетям любого уровня. Вместо простого перечисления назначения каждой опции Nmap книга «Nmap: сканирование сетей» показывает, как применять эти возможности для решения реальных задач: тестирования на проникновение, инвентаризации сети, обнаружения посторонних беспроводных точек доступа или открытых прокси-серверов, подавления вспышек сетевых червей и вирусов и многого другого. Примеры и схемы демонстрируют реальный обмен данными в сети. Эта книга необходима каждому, кто хочет использовать Nmap с максимальной отдачей, особенно аудиторам безопасности, системным и сетевым администраторам.

Выходные данные

Nmap: сканирование сетей. Официальное руководство проекта Nmap по обнаружению сетевых узлов и сканированию безопасности
Гордон «Фёдор» Лайон

Адрес книги: http://nmap.org/book/
ISBN-13: 978-0-9799587-1-7
ISBN-10: 0-9799587-1-7
Контрольный номер Библиотеки Конгресса (LCCN): 2008940582

Тематические рубрики Библиотеки Конгресса:

1. Компьютерные сети - меры безопасности
2. Компьютерная безопасность

Издано Insecure.Com LLC. По вопросам оптовых закупок, специальных продаж, прав, распространения книги или переводов обращайтесь непосредственно к нам:

Insecure.Com LLC
370 Altair Way #113
Sunnyvale, CA 94086-6161
United States
Электронная почта: sales@insecure.com; телефон: +1-650-989-4206; факс: +1-650-989-4206

История редакций:

• Первое издание: декабрь 2008 года
• Предварительный выпуск к Defcon: август 2008 года
• Выпуск нулевого дня: май 2008 года

© 2008 Insecure.Com LLC. Все права защищены. За исключением случаев, отдельно оговорённых в этой работе, никакая её часть не может быть воспроизведена или передана в какой бы то ни было

форме и какими бы то ни было средствами - электронными или механическими, включая фотокопирование и запись, - либо посредством какой бы то ни было системы хранения или поиска информации без предварительного письменного разрешения правообладателя.

Nmap является зарегистрированным товарным знаком Insecure.Com LLC. Другие упомянутые здесь названия продуктов и компаний могут являться товарными знаками соответствующих владельцев. Там, где такие обозначения встречаются в этой книге и издателю было известно о притязании на товарный знак, они напечатаны с прописной начальной буквы либо целиком прописными буквами.

Авторы и издатель тщательно готовили эту книгу, однако не дают никаких явно выраженных или подразумеваемых гарантий и не несут ответственности за ошибки или упущения. Не принимается никакая ответственность за случайный или косвенный ущерб, связанный с использованием содержащихся здесь сведений или программ либо возникший вследствие такого использования.

Предисловие

1. Введение

1 сентября 1997 года я опубликовал сканер безопасности под названием Nmap в пятьдесят первом выпуске журнала Phrack. Моей целью было объединить разрозненную область специализированных сканеров портов в одном мощном, гибком и свободном инструменте с единообразным интерфейсом и эффективной реализацией всех практически полезных методов сканирования портов. В то время Nmap состоял из трёх файлов (всего около 2 000 строк кода) и поддерживал только операционную систему Linux. Я написал его для собственных нужд и выпустил в надежде, что он окажется полезен и другим.

От этих скромных истоков, благодаря силе разработки с открытым исходным кодом, Nmap вырос в самый популярный в мире сканер сетевой безопасности[1], которым пользуются миллионы людей по всему миру. С годами в Nmap продолжали появляться расширенные возможности: удалённое определение ОС, определение версий и служб, Idle-сканирование по IP ID, движок сценариев Nmap и быстрое ping-сканирование с несколькими пробами. Теперь он поддерживает все основные платформы Unix, Windows и Mac OS. Доступны как консольная, так и графическая версии. Такие издания, как Linux Journal, InfoWorld, LinuxQuestions.org и Codetalker Digest, признавали Nmap «инструментом безопасности года». Он даже появлялся в нескольких фильмах[2], в том числе «Матрица: Перезагрузка», «Ультиматум Борна» и «Крепкий орешек 4.0».

Nmap («Network Mapper», «сетевой картограф») - свободная утилита с открытым исходным кодом для исследования сетей и аудита безопасности. Многие системные и сетевые администраторы считают её полезной также для инвентаризации сети, управления расписаниями обновления служб и наблюдения за доступностью узлов или служб. Nmap необычным образом использует необработанные IP-пакеты, чтобы определять, какие узлы доступны в сети, какие службы (название и версию приложения) они предоставляют, какие операционные системы и их версии на них работают, какие фильтры пакетов и брандмауэры применяются, а также десятки других характеристик. Nmap проектировался для быстрого сканирования больших сетей, но прекрасно работает и с отдельными узлами.

Nmap чрезвычайно мощен, но вместе с тем сложен. Более ста опций командной строки дают специалистам по сетям широкие выразительные возможности, однако могут сбить с толку начинающих. Некоторые опции вообще никогда прежде не документировались. Эта книга описывает все возможности Nmap и, что ещё важнее, учит наиболее эффективным способам их применения. Работа над ней заняла почти четыре года, и по мере развития Nmap текст постоянно обновлялся.

Эта книга посвящается сообществу пользователей и разработчиков Nmap. Ваши увлечённость, идеи, исправления, запросы новых возможностей, жаркие споры, отчёты об ошибках и ночные тирады сформировали Nmap таким, каким он стал сегодня.

• Гордон «Фёдор» Лайон fyodor@insecure.org

2. Для кого предназначена книга и как она организована

Эта книга описывает свободный сканер безопасности Nmap - от основ сканирования портов для начинающих до видов конструирования пакетов, которыми пользуются опытные взломщики. Она должна быть полезна пользователям Nmap (и тем, кто только собирается им стать) с любым уровнем подготовки.

Начиная с основ, в главе 1 книга на примерах даёт обзор Nmap. Затем глава 2 рассказывает о получении, компиляции и установке Nmap. В главах 3-5 возможности рассматриваются в том порядке, в каком они могут применяться при тестировании на проникновение. Сначала идёт обнаружение узлов («ping-сканирование»), определяющее доступные узлы сети. Далее подробно рассматривается сканирование портов. В главе 5 описаны все методы сканирования Nmap, приведены рекомендации и примеры. Сканирование большой сети может занять много времени, поэтому глава 6 целиком посвящена советам по оптимизации производительности. В главе 7 подробно описывается определение версий служб и приложений: Nmap опрашивает порты, чтобы точно установить, что на них работает, а не просто делает предположение по номеру порта. Глава 8 посвящена одной из самых любимых возможностей Nmap - удалённому определению ОС. В главе 9 подробно рассматривается одна из новейших возможностей Nmap - движок сценариев Nmap. NSE позволяет пользователям и разработчикам легко расширять Nmap новыми возможностями, создавая простые сценарии, которые эффективно выполняются на целевых машинах. Моя любимая глава - десятая: «Обнаружение и обход брандмауэров и систем обнаружения вторжений». Для равновесия за ней следует глава о защите от сканирований Nmap. Затем в главе 12 полностью документируются многоплатформенный графический интерфейс Nmap Zenmap и его средство просмотра результатов. Следующие две главы посвящены форматам вывода и файлам данных. Последняя и самая длинная глава представляет собой справочное руководство Nmap - удобный ресурс для быстрого поиска отдельных опций Nmap.

По всей книге разбросаны подробные инструкции по выполнению распространённых задач, например по сканированию сети в поисках одного определённого открытого TCP-порта или по обнаружению беспроводных точек доступа посредством сканирования из проводного сегмента. Сначала описывается каждая задача, затем предлагается эффективное решение. В завершающем разделе с обсуждением решение разбирается подробнее; там же могут приводиться альтернативные решения и соображения по поводу сходных задач.

3. Условные обозначения

Во всей книге вывод Nmap используется для демонстрации принципов и возможностей. Его часто редактируют, удаляя строки, не относящиеся к рассматриваемой теме. Как правило, удаляются также даты, время и номера версий, выводимые Nmap, поскольку некоторых читателей они отвлекают. Конфиденциальные сведения, например имена узлов, IP-адреса и MAC-адреса, могут быть изменены или удалены. Другие сведения могут быть вырезаны, а строки - перенесены, чтобы они помещались на печатной странице. Аналогичным образом редактируется вывод других приложений. Пример 1 позволяет бегло познакомиться с возможностями Nmap и одновременно демонстрирует оформление вывода.

Пример 1. Типичное сканирование Nmap

```
# nmap -A -T4 scanme.nmap.org
Starting Nmap ( http://nmap.org
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain  ISC BIND 9.3.4
70/tcp  closed gopher
```

```
80/tcp  open   http    Apache httpd 2.2.2 ((Fedora))
|_ HTML title: Go ahead and ScanMe!
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.20-1 (Fedora Core 5)

TRACEROUTE (using port 80/tcp)
HOP RTT   ADDRESS
[Cut first seven hops for brevity]
8   10.59 so-4-2-0.mpr3.pao1.us.above.net (64.125.28.142)
9   11.00 metro0.sv.svcolo.com (208.185.168.173)
10  9.93  scanme.nmap.org (64.13.134.52)

Nmap done: 1 IP address (1 host up) scanned in 17.00 seconds
```

Для некоторых элементов, например имён файлов и команд приложений, предусмотрено специальное оформление. В таблице 1 показаны наиболее распространённые соглашения об оформлении.

Таблица 1. Соглашения о стиле оформления

Тип элемента — пример

• **Литеральная строка** — Меня гораздо больше радуют порты в состоянии `open`, чем порты, обозначенные как `closed` или `filtered`.
• **Опции командной строки** — Одна из самых замечательных, но наименее понятных опций Nmap - `--packet-trace`.
• **Имена файлов** — После опции `-iL` укажите имя входного файла, например `C:\net\dhcp-leases.txt` или `/home/h4x/hosts-to-pwn.lst`.
• **Выделение** — *Использовать Nmap с рабочего или учебного компьютера для атак на банки и военные объекты - плохая идея.*
• **Команды приложений** — Тринити просканировала Матрицу командой `nmap -v -sS -O 10.2.2.2`.
• **Заменяемые переменные** — Пусть `<source>` - машина, на которой работает Nmap, а `<target>` - `microsoft.com`.

4. Другие ресурсы

Хотя эта книга служит важным справочным материалом по Nmap, она далеко не единственная. Веб-сайт Nmap по адресу http://nmap.org предназначен не только для загрузки программы. На нём также размещён значительный объём документации от разработчиков Nmap и сторонних авторов. Например, там можно найти справочное руководство Nmap в переводе на десяток языков. Доступны также другие книги, видеоматериалы и статьи о Nmap.

Официальный сайт этой книги находится по адресу http://nmap.org/book/. Там публикуются исправления, обновления и множество примеров глав.

Каждому серьёзному пользователю Nmap следует подписаться на список рассылки `nmap-hackers`, в котором публикуются объявления о Nmap и Insecure.Org. Трафик там очень невелик (около шести сообщений в год), поскольку список предназначен только для самых важных объявлений. Разработчики и особенно преданные пользователи могут также подписаться на список `nmap-dev`. Трафик в нём значительно выше (сотни сообщений в месяц), но это прекрасное место, где можно узнать о новых возможностях и опробовать их до выпуска, а также перенять полезные приёмы у опытных пользователей. Сведения о подписке и архивы обоих списков доступны по адресу http://seclists.org.

Nmap может быть полезен, но он не решит всех ваших проблем безопасности. Раз в несколько лет я опрашиваю тысячи пользователей Nmap, чтобы выяснить, какие ещё инструменты им нравятся. Список публикуется по адресу http://sectools.org; этот сайт стал одним из самых популярных моих веб-сайтов. Просмотрите список, и вы наверняка обнаружите множество жемчужин, о которых прежде даже не слышали. Большинство инструментов свободны и имеют открытый исходный код.

5. Просьба присылать замечания

Я приложил все усилия, чтобы сделать эту книгу исчерпывающей, точной и современной, однако все мы ошибаемся. Если вы обнаружите какие-либо проблемы или просто захотите предложить, как улучшить книгу, сообщите мне об этом по электронной почте fyodor@insecure.org. Принцип открытого исходного кода, которому следуют многие читатели и участники проекта, столь же применим к документации, как и к программному обеспечению. Как подтверждает следующий раздел, десятки людей уже щедро вложили своё время и мастерство, чтобы эта книга состоялась.

Если у вас есть вопрос или замечание о Nmap, а не о самой книге, лучше всего направить его в список разработки Nmap, как описано в разделе 15.17 «Ошибки».

6. Благодарности

Когда я впервые предложил в списке рассылки `nmap-hackers` идею написать книгу о Nmap, меня завалили предложениями и готовностью помочь. Этот поток энтузиазма убедил меня взяться за работу. Свою роль сыграла и моя полная наивность в отношении предстоящего объёма труда. Работа оказалась грандиозной, но переходить от главы к главе мне помогала закрытая группа рецензентов под названием `nmap-writers`. На всём протяжении процесса они предоставляли бесценные отзывы, советы и подробные замечания. В особенности я хотел бы поблагодарить следующих людей:

• Дэвид Файфилд указан первым (все остальные перечислены по алфавиту), потому что оказал огромную помощь в процессе написания книги. Он решил ряд технических проблем DocBook, создал многие окончательные иллюстрации на основе моих ужасных набросков, значительно улучшил предметный указатель, помог с корректурой и даже написал главу 12 «Руководство пользователя графического интерфейса Zenmap».

• Мэтт Бакстер разрешил использовать свои прекрасные схемы заголовков TCP/IP (в разделе 7 «Справочник по TCP/IP» [xxvi]). Несколько других схем в этой книге выполнены в том же стиле, чтобы соответствовать им.

• Саурабх Бхасин регулярно присылал подробные отзывы.

• На полезный совет Марка Брьюиса всегда можно было рассчитывать.

• Эллен Коломбо очень помогала с самого начала.

• Патрик Доннелли помог улучшить главу 9 «Движок сценариев Nmap».

• Брэндон Энрайт распечатал всю книгу и проверил её главу за главой.

• Брайан Хэтч всегда оказывал большую помощь.

• Лорен Хил был постоянным источником идей.

• Дэн Бенедж давал советы и вычитал множество глав.

• Тор Хаутон проверил каждую главу и, вероятно, прислал мне больше замечаний, чем кто-либо другой.

• Дуг Хойт задокументировал множество добавленных им возможностей Nmap, а также выполнил большую часть работы над предметным указателем книги.

• Мариус Хусе Якобсен проверил многие главы и предоставил подробные отзывы.

• Крис Каттерджон тщательно проверил несколько глав.

• Эрик Кроснес прислал полезные замечания по технической части и к тому же регулярно донимал меня вопросами о ходе работы над книгой. Это помогало, поскольку у меня не было традиционного редактора, который занимался бы этим.

• Влад Алекса Манчини создал для обложки логотип Nmap в виде глаза (он же используется на веб-сайте Nmap).

• Майкл Нэф любезно проверил множество глав.

• Билл Поллок из No Starch Press всегда охотно давал советы и отвечал на вопросы о книгоиздании, опираясь на свой многолетний опыт.

• Дэвид Пайбас был одним из тех, кто чаще всего предлагал идеи и помогал с корректурой.

• Тайлер Регули помог, проверив несколько глав именно тогда, когда это было нужнее всего.

• Чак Стерлинг предоставил как общие рекомендации, так и подробную корректуру нескольких глав.

• Андерс Тулин подробно проверил многие главы.

• Беннетт Тодд прислал десятки предложений.

• Диман Тодоров написал первоначальный черновик главы 9 «Движок сценариев Nmap».

• Кэтрин Торнабене прочитала множество глав и прислала чрезвычайно подробные отзывы.

6.1. Технологии, использованные при создании этой книги

Будучи автором инструментов с открытым исходным кодом, я искренне верю в их силу и возможности. Поэтому при создании этой книги я старался использовать их везде, где только можно. Не собирался же я писать её в Microsoft Word, а затем выполнять вёрстку в Adobe FrameMaker!

Книга «Nmap: сканирование сетей» была написана в текстовом редакторе GNU Emacs в формате XML DocBook. Бесплатные главы для публикации в Интернете создаются из XML с помощью таблиц стилей XSL Нормана Уолша и XSL-процессора `xsltproc`.

В печатной версии также используются таблицы стилей Нормана и `xsltproc`, но вывод формируется в формате XSL-FO[3]. Затем процессор XSL-FO создаёт PDF. Я хотел бы применять для этого Apache FOP[4], однако мешает ошибка, связанная со сносками[5], поэтому я перешёл на движок RenderX XEP. XEP является проприетарным продуктом, но по крайней мере работает в Linux. Надеюсь вернуться к FOP после исправления ошибки со сносками.

Макет обложки был подготовлен в Scribus и, из-за требований типографии к формату, в Adobe InDesign. Растровая графика для обложки и внутренних иллюстраций создавалась в Gimp, а для векторной графики использовался Inkscape.

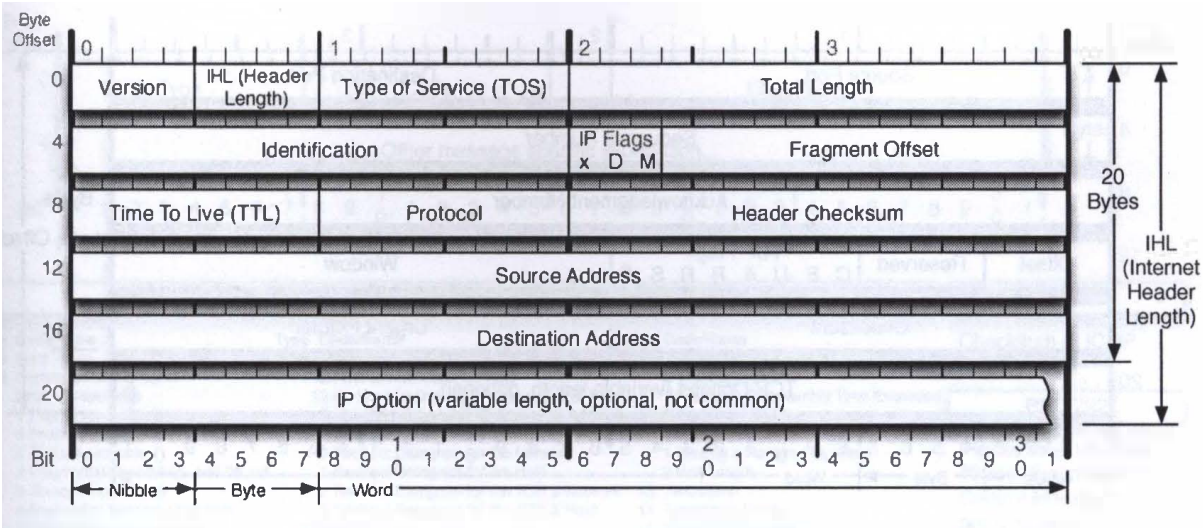
Для управления версиями применялся Subversion, а бесплатные веб-версии глав обслуживаются Apache httpd.

7. Справочник по TCP/IP

В этой книге предполагается знакомство читателя с основами TCP/IP и сетевыми понятиями. На её страницах вы не найдёте ни вводного курса по семиуровневой модели OSI, ни обзора API сокетов Беркли. В качестве исчерпывающего руководства по TCP/IP я рекомендую книгу Чарльза Козьерока «The TCP/IP Guide» или старую классику У. Ричарда Стивенса «TCP/IP Illustrated, Volume 1».

Хотя знание TCP/IP предполагается, даже лучшие из нас порой забывают байтовые смещения полей и флагов заголовков пакетов. В этом разделе приведены удобные справочные схемы и описания полей протоколов IPv4, TCP, UDP и ICMP. Эти прекрасные схемы с сайта http://www.fatpipe.org/~mjb/Drawings используются с разрешения их автора Мэтта Бакстера.

Рисунок 1. Заголовок IPv4



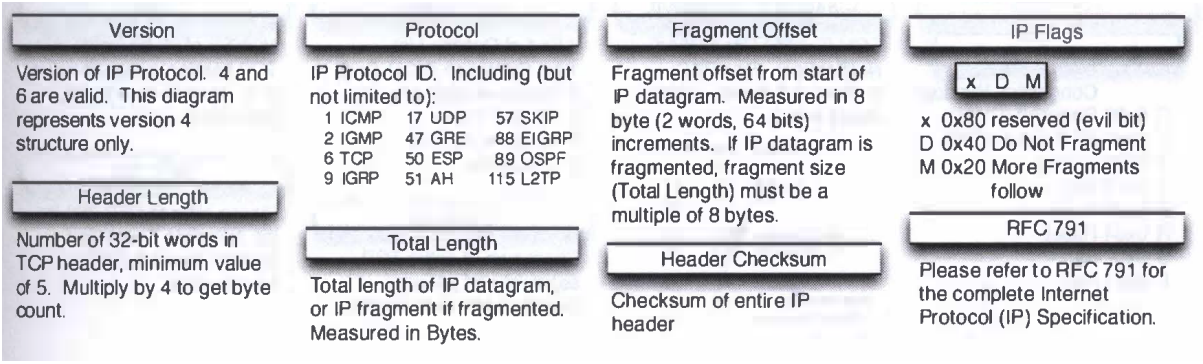


Рисунок 2. Заголовок TCP

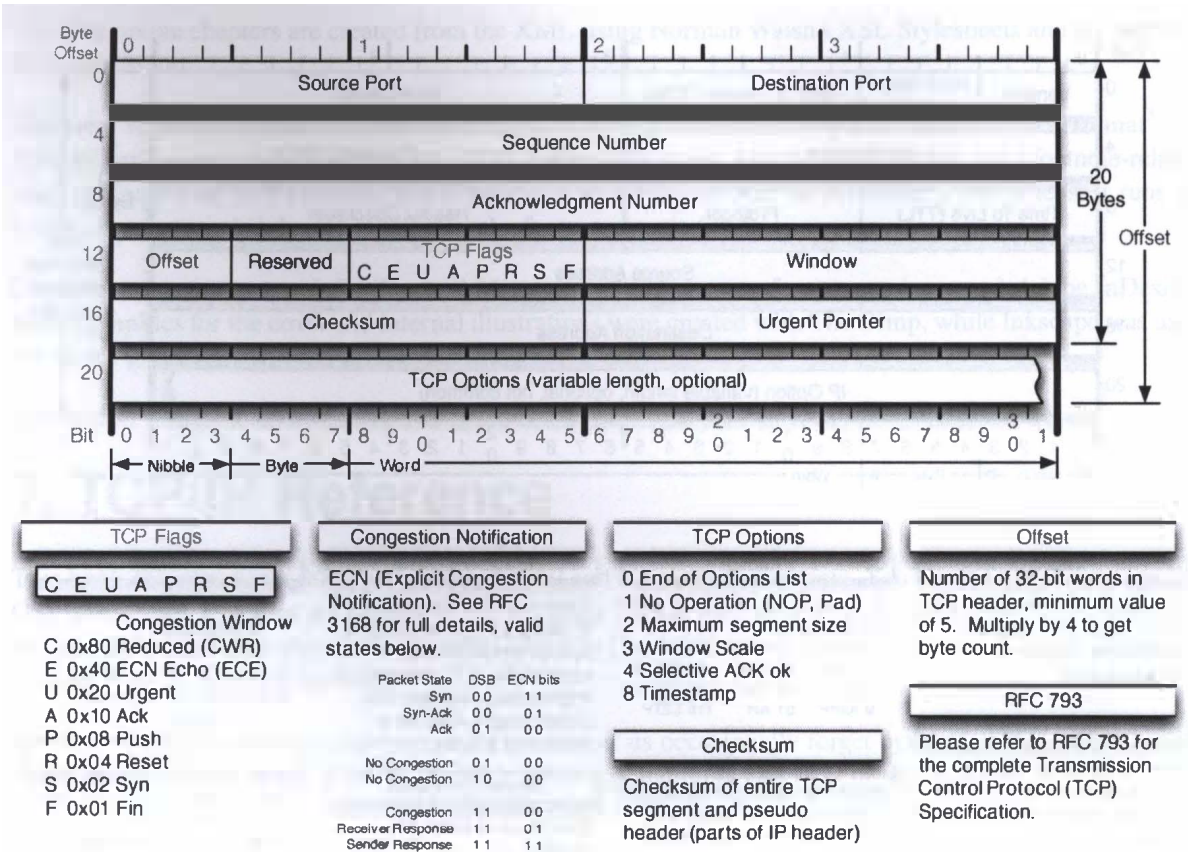


Рисунок 3. Заголовок UDP

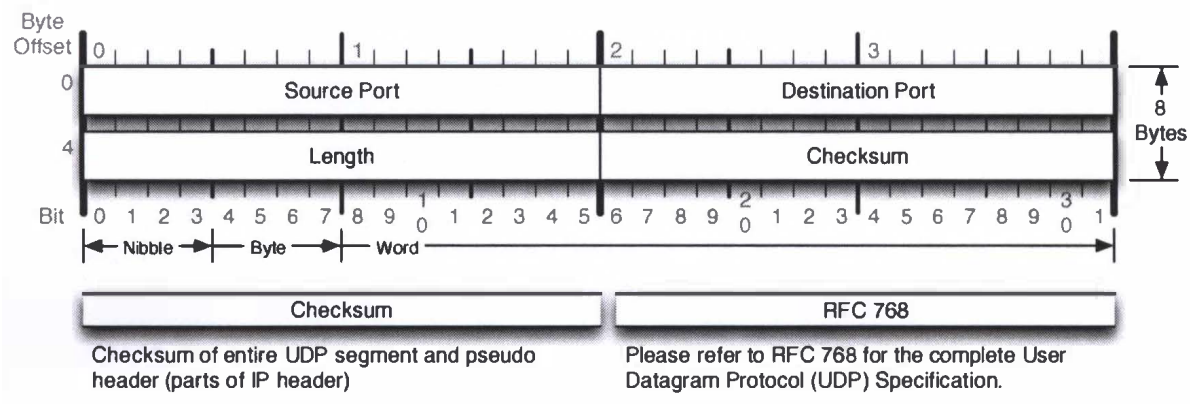
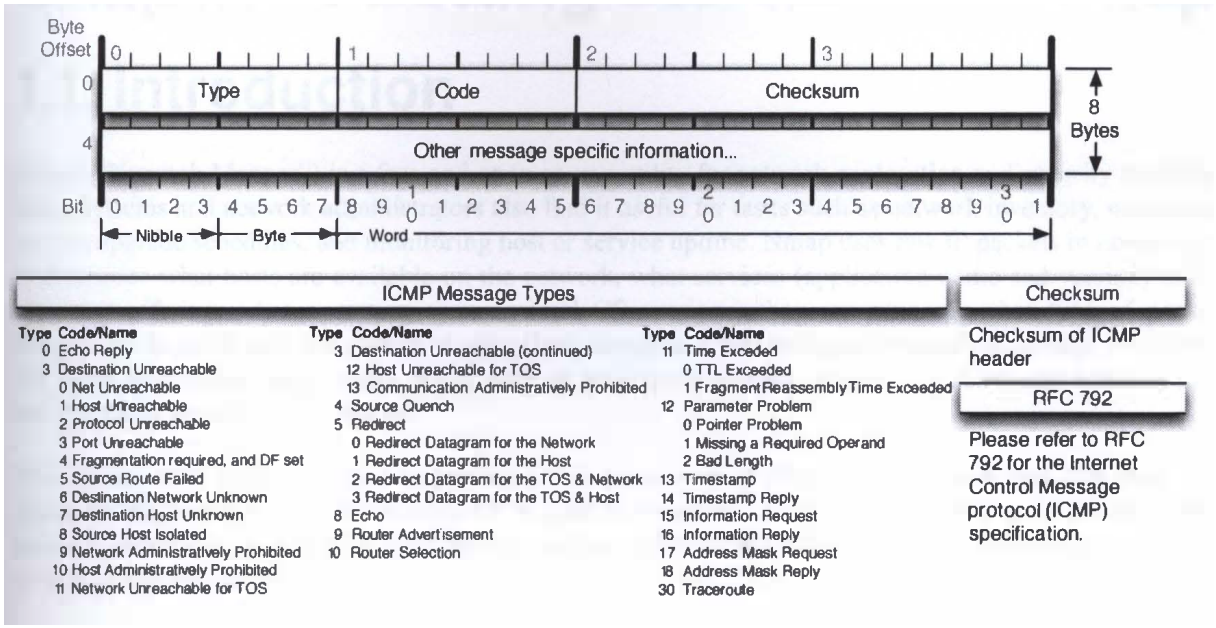


Рисунок 4. Заголовок ICMP



Сноски

[1] [назад] Вывод основан на частоте загрузок, количестве результатов Google и рейтинге «популярности» программ на Freshmeat.Net.
[2] [назад] http://nmap.org/movies.html
[3] [назад] http://en.wikipedia.org/wiki/XSL_Formatting_Objects
[4] [назад] http://xmlgraphics.apache.org/fop/
[5] [назад] https://issues.apache.org/bugzilla/show_bug.cgi?id=37579

Глава 1. Начало работы с Nmap

1.1. Введение

Nmap («Network Mapper», «сетевой картограф») - свободная утилита с открытым исходным кодом для исследования сетей и аудита безопасности. Многие системные и сетевые администраторы считают её полезной также для инвентаризации сети, управления расписаниями обновления служб и наблюдения за доступностью узлов или служб. Nmap необычным образом использует необработанные IP-пакеты, чтобы определять, какие узлы доступны в сети, какие службы (название и версию приложения) они предоставляют, какие операционные системы и их версии на них работают, какие фильтры пакетов и брандмауэры применяются, а также десятки других характеристик. Nmap проектировался для быстрого сканирования больших сетей, но прекрасно работает и с отдельными узлами. Он запускается во всех основных компьютерных операционных системах и доступен как в консольной, так и в графической версии.

В этой главе с помощью вымышленных историй даётся общий обзор Nmap и типичных способов его применения. Важный раздел о правовых вопросах помогает пользователям избегать спорных способов использования, способных привести к закрытию учётной записи интернет-провайдером или даже к гражданскому и уголовному преследованию, либо по крайней мере осознавать связанные с ними последствия. В нём также обсуждаются риски вызвать сбой удалённых машин и различные другие вопросы, в том числе лицензия Nmap (GNU GPL) и авторские права.

1.2. Обзор и демонстрация Nmap

Иногда лучший способ понять что-либо - увидеть это в действии. В этом разделе приведены примеры использования Nmap в ситуациях, которые в основном вымышлены, но вполне типичны. Новичкам в Nmap не следует рассчитывать, что они сразу поймут всё. Это всего лишь общий обзор возможностей, подробно описанных в последующих главах. «Решения», встречающиеся по всей книге, демонстрируют множество других распространённых задач Nmap, с которыми сталкиваются аудиторы безопасности и сетевые администраторы.

1.2.1. Avatar Online

15 декабря Феликс исправно приходит на работу, хотя и не ожидает каких-либо плановых заданий. В небольшой фирме из Сан-Франциско, занимающейся тестированием на проникновение, в преддверии праздников в последнее время было тихо. Рабочее время Феликс посвящает новому увлечению: собирает мощные антенны Wi-Fi для аудита беспроводных сетей и поездок в поисках точек доступа. И всё же он надеется, что заказов станет больше. Взлом был его увлечением и страстью с детства, которое он провёл, изучая всё, что мог, о сетях, безопасности, Unix и телефонных системах. Порой любопытство заводило его слишком далеко, и Феликс едва не попал под преследования в рамках операции Sundevil 1990 года. К счастью, он вышел из подросткового возраста без судимости, сохранив экспертные знания о слабостях безопасности. Став профессионалом, он может совершать те же виды сетевых вторжений, что и прежде, но теперь с дополнительным преимуществом: договор защищает его от судебного преследования, да ещё и платят зарплату! Вместо того чтобы держать изобретательные эксплойты в тайне, он может

хвастаться ими перед руководством заказчика во время представления отчётов. Поэтому Феликс не огорчился, когда начальник прервал пайку антенны сообщением о том, что отдел продаж наконец заключил договор на тестирование с компанией Avatar Online, выпускающей игры.

Avatar Online (AO) - небольшая компания, которая работает над следующим поколением массовых многопользовательских ролевых онлайн-игр (MMORPG). Её продукт, вдохновлённый Метавселенной из романа Нила Стивенсона «Лавина», увлекателен, но пока держится в строжайшей тайне. Увидев, какой громкий резонанс вызвала утечка исходного кода готовящейся игры Valve Software[1], AO поспешила нанять консультантов по безопасности. Задача Феликса - начать внешнюю, то есть проводимую из-за пределов брандмауэра, оценку уязвимостей, пока его партнёры занимаются физической безопасностью, аудитом исходного кода, социальной инженерией и прочим. Феликсу разрешено эксплуатировать любые найденные уязвимости.

Первый этап оценки уязвимостей - исследование сети. На этой стадии разведки определяются используемые целью диапазоны IP-адресов, доступные узлы, предоставляемые этими узлами службы, общие сведения о топологии сети и действующие правила брандмауэра и фильтрации.

Обычно определение сканируемых диапазонов IP-адресов представляет собой сложный процесс, включающий запросы к ARIN (или другому региональному реестру), запросы DNS и попытки передачи зон, различные методы поиска в Интернете и многое другое. Но в данном случае Avatar Online явно указала сети, которые следует проверить: корпоративную сеть `6.209.24.0/24`, а также рабочие системы и системы в DMZ, находящиеся в сети `6.207.0.0/22`. Феликс всё же проверяет записи ARIN о распределении IP-адресов и подтверждает, что эти диапазоны принадлежат AO[2]. Он машинально расшифровывает запись CIDR[3] и понимает, что речь идёт о 1 280 IP-адресах. Никаких проблем.

Будучи человеком осторожным, Феликс начинает с так называемого сканирования списка Nmap (опция `-sL`). Эта возможность просто перечисляет все IP-адреса в заданных целевых сетевых блоках и выполняет для каждого обратный DNS-запрос, если только не указана опция `-n`. Одна из причин начать именно с этого - скрытность. Имена узлов могут намекнуть на возможные уязвимости и помочь лучше понять целевую сеть, причём без поднятия тревоги[4]. Однако Феликс поступает так по другой причине: он хочет ещё раз проверить правильность диапазонов IP-адресов. Предоставивший их системный администратор мог ошибиться, а сканирование чужой компании обернулось бы катастрофой. Подписанный с Avatar Online договор может послужить Феликсу своего рода индульгенцией при проникновении в её сети, но ничем не поможет, если он случайно получит права суперпользователя на сервере другой компании! Использованная им команда и фрагмент результатов показаны в примере 1.1.

Пример 1.1. Сканирование списка IP-адресов Avatar Online с помощью Nmap

```
felix> nmap -sL 6.209.24.0/24 6.207.0.0/22

Starting Nmap ( http://nmap.org )
Host 6.209.24.0 not scanned
Host fw.corp.avataronline.com (6.209.24.1) not scanned
Host dev2.corp.avataronline.com (6.209.24.2) not scanned
Host 6.209.24.3 not scanned
Host 6.209.24.4 not scanned
Host 6.209.24.5 not scanned

Host dhcp-21.corp.avataronline.com (6.209.24.21) not scanned
Host dhcp-22.corp.avataronline.com (6.209.24.22) not scanned
Host dhcp-23.corp.avataronline.com (6.209.24.23) not scanned
Host dhcp-24.corp.avataronline.com (6.209.24.24) not scanned
Host dhcp-25.corp.avataronline.com (6.209.24.25) not scanned
Host dhcp-26.corp.avataronline.com (6.209.24.26) not scanned

Host 6.207.0.0 not scanned
Host gw.avataronline.com (6.207.0.1) not scanned
Host ns1.avataronline.com (6.207.0.2) not scanned
```

```
Host ns2.avataronline.com (6.207.0.3) not scanned
Host ftp.avataronline.com (6.207.0.4) not scanned
Host 6.207.0.5 not scanned
Host 6.207.0.6 not scanned
Host www.avataronline.com (6.207.0.7) not scanned
Host 6.207.0.8 not scanned

Host cluster-c120.avataronline.com (6.207.2.120) not scanned
Host cluster-c121.avataronline.com (6.207.2.121) not scanned
Host cluster-c122.avataronline.com (6.207.2.122) not scanned
Host cluster-c123.avataronline.com (6.207.2.123) not scanned
Host cluster-c124.avataronline.com (6.207.2.124) not scanned

Host 6.207.3.253 not scanned
Host 6.207.3.254 not scanned
Host 6.207.3.255 not scanned
Nmap done: 1280 IP addresses scanned in 331.49 seconds
felix>
```

Просматривая результаты, Феликс обнаруживает, что все машины, для которых существуют записи обратного DNS, разрешаются в имена Avatar Online. Судя по всему, никакие другие компании это адресное пространство не разделяют. Более того, результаты позволяют Феликсу примерно понять, сколько машин используется, и довольно хорошо представить назначение многих из них. Теперь он готов действовать чуть настойчивее и попробовать сканирование портов. Он применяет возможности Nmap, которые пытаются определить приложение и номер версии каждой службы, прослушивающей сеть. Кроме того, он просит Nmap попытаться угадать удалённую операционную систему с помощью последовательности низкоуровневых проб TCP/IP, известных как снятие отпечатка ОС. Такое сканирование совсем не скрытно, но Феликса это не беспокоит. Ему интересно, заметят ли администраторы АО хотя бы настолько явное сканирование. Немного подумав, Феликс останавливается на следующей команде:

```
nmap -sS -p- -PS22,80,113,33334 -PA80,113,21000 -PU19000 -PE -A -T4 -oA avatartcpscan-
    121503 6.209.24.0/24 6.207.0.0/22
```

Эти опции описаны в последующих главах, но здесь приведена их краткая сводка.

`-sS`

Включает эффективный метод сканирования TCP-портов, известный как SYN-сканирование. Если бы Феликс хотел выполнить ещё и UDP-сканирование, он добавил бы в конце `U`, но это он оставляет на потом. SYN-сканирование является типом сканирования по умолчанию, однако явно указать его не повредит.

`-p-`

Требует от Nmap просканировать все порты от 1 до 65535. По умолчанию сканируются только порты с первого по 1024-й и ещё около 600 портов, явно упомянутых в базе данных `nmap-services`. Такая форма опции `-` всего лишь сокращение для `-p1-65535`. Если бы Феликс хотел просканировать ещё и сомнительный с точки зрения допустимости нулевой порт, он мог бы указать `-p0-65535`. У опции `-p` очень гибкий синтаксис, позволяющий даже задавать разные наборы портов UDP и TCP.

`-PS22,80,113,33334 -PA80,113,21000 -PU19000 -PE`

Это всё типы ping-проверок, которые применяются совместно, чтобы определить, действительно ли узел доступен, и не тратить много времени на сканирование неиспользуемых IP-адресов. Эта конкретная комбинация отправляет пакет TCP SYN на порты 22, 80, 113 и 33334; пакет TCP ACK на порты 80, 113 и 21000; пакет UDP на порт 19000 и обычный пакет эхо-запроса ICMP. Если Nmap получает от самого целевого узла ответ на любую из этих проб, узел считается работающим и доступным для сканирования. Такая проверка полнее стандартной проверки Nmap, при которой отправляются только эхо-запрос и пакет ACK на порт 80. При тестировании на проникновение часто

требуется сканировать каждый узел, даже если он кажется недоступным. В конце концов, он может быть столь жёстко отфильтрован, что выбранные вами пробы игнорируются, хотя какой-либо другой малоизвестный порт может оказаться доступен. Чтобы просканировать каждый IP-адрес независимо от того, обнаружен ли на нём доступный узел, вместо всех перечисленных выше опций укажите `-PN`. Феликс запускает такое сканирование в фоновом режиме, хотя на его завершение может потребоваться целый день.

`-A`

Эта сокращённая опция включает расширенные и агрессивные возможности, например определение ОС и служб. На момент написания книги она равнозначна `-sV -sC -O --traceroute` (определение версий, движок сценариев Nmap, удалённое определение ОС и трассировка маршрута). В дальнейшем опция `-A` может включать дополнительные возможности.

`-T4`

Устанавливает агрессивный уровень времени (четвёртый из пяти). Это то же самое, что указать `-T aggressive`, но такую запись проще набирать, и в ней труднее ошибиться. В целом опция `-T4` рекомендуется, если соединение между вами и целевыми сетями быстрее модемного подключения.

`-oA avatartcpscan-121503`

Выводит результаты во всех форматах (обычном, XML и предназначенном для обработки `grep`) в файлы с именами `avatartcpscan-121503.<расширение>`, где расширениями служат соответственно `.nmap`, `.xml` и `.gnmap`. Все форматы вывода содержат дату и время начала, но Феликс предпочитает явно отмечать дату в имени файла. Обычный вывод и ошибки по-прежнему направляются также в `stdout`[5].

`6.209.24.0/24 6.207.0.0/22`

Это сетевые блоки Avatar Online, рассмотренные выше. Они заданы в нотации CIDR, но Nmap позволяет указывать их во множестве других форматов. Например, вместо `6.209.24.0/24` можно было бы написать `6.209.24.0-255`.

Поскольку столь полное сканирование более чем тысячи IP-адресов может занять некоторое время, Феликс просто запускает его и возвращается к работе над антенной Яги. Через пару часов он замечает, что сканирование завершилось, и заглядывает в результаты. В примере 1.2 показана одна из обнаруженных машин.

Пример 1.2. Результаты Nmap для брандмауэра АО

```
Interesting ports on fw.corp.avataronline.com (6.209.24.1):
(The 65530 ports scanned but not shown below are in state: filtered)
PORT     STATE  SERVICE    VERSION
22/tcp   open   ssh        OpenSSH 3.7.1p2 (protocol 1.99)
53/tcp   open   domain     ISC BIND 9.2.1
110/tcp  open   pop3       Courier pop3d
113/tcp  closed auth
143/tcp  open   imap       Courier Imap 1.6.X - 1.7.X
3128/tcp open   http-proxy Squid webproxy 2.2.STABLE5
Device type: general purpose
Running: Linux 2.4.X|2.5.X
OS details: Linux Kernel 2.4.0 - 2.5.20
Uptime 3.134 days
```

Опытному взгляду эти сведения немало говорят о состоянии безопасности АО. Первым делом Феликс замечает имя обратного DNS: по всей видимости, эта машина должна служить брандмауэром корпоративной сети. Следующая строка важна, однако её слишком часто оставляют без внимания. В ней сказано, что подавляющее большинство портов этой машины находится в состоянии `filtered`. Это означает, что Nmap не может добраться до порта, поскольку тот

заблокирован правилами брандмауэра. Состояние, при котором все порты, кроме нескольких специально выбранных, отфильтрованы, свидетельствует о грамотном подходе к безопасности. «Запрещать по умолчанию» - принцип безопасности, ставший мантрой не без причины: даже если кто-то случайно оставит на этой машине открытым SunRPC (порт 111), правила брандмауэра не позволят злоумышленникам с ним связаться.

Затем Феликс по очереди рассматривает строку каждого порта. Первый порт занят Secure Shell (OpenSSH). Версия 3.7.1p2 широко распространена, поскольку многие администраторы обновились до неё из-за потенциально эксплуатируемых ошибок управления буферами в предыдущих версиях. Nmap также отмечает версию протокола SSH 1.99, из чего следует, что поддерживается устаревший и менее совершенный протокол SSHv1. По-настоящему параноидальный системный администратор разрешил бы SSH-соединения только с определённых доверенных IP-адресов, но открытый доступ можно оправдать необходимостью экстренного подключения администратора, находящегося далеко от дома. Безопасность часто требует компромиссов, и этот, возможно, оправдан. Феликс записывает, что нужно испытать на сервере его программу перебора паролей и особенно собственный инструмент перечисления пользователей SSH на основе анализа времени.

Феликс отмечает и порт 53. На нём работает ISC BIND, за которым тянется долгая история удалённо эксплуатируемых уязвимостей. Дополнительные сведения можно найти на странице безопасности BIND[6]. Даже в BIND 9.2.1 есть потенциально эксплуатируемое переполнение буфера, хотя сборка по умолчанию не уязвима. Феликс проверяет сервер и выясняет, что проблема `libbind` его не затрагивает, однако суть не в этом. Почти наверняка на этом сервере не должен работать доступный извне сервер имён. Чтобы свести к минимуму риск катастрофического взлома, на брандмауэре должны работать только совершенно необходимые службы. Кроме того, этот сервер не является авторитетным ни для одного домена: настоящие серверы имён находятся в рабочей сети. Вероятно, администратор хотел, чтобы к этому серверу имён обращались только клиенты из-за брандмауэра, но не позаботился ограничить его работу одним внутренним интерфейсом. Позднее Феликс попытается получить с этого ненужного сервера важные сведения с помощью запросов передачи зон и агрессивных запросов. Возможно, он попробует и отравление кеша. Подменив IP-адрес `windowsupdate.microsoft.com` или другого важного сервера загрузок, Феликс, возможно, сумеет обманом заставить ничего не подозревающих внутренних пользователей запустить троянскую программу, которая предоставит ему полный доступ к сети за брандмауэром.

Следующие два открытых порта - 110 (POP3) и 143 (IMAP). Обратите внимание, что находящийся между ними порт 113 (`auth`) закрыт, а не открыт. POP3 и IMAP - службы получения почты, которым, как и BIND, нет законного места на этом сервере. Они представляют и угрозу безопасности, поскольку обычно передают почту и, что ещё хуже, учётные данные для аутентификации без шифрования. Вероятно, пользователям следует подключаться через VPN и проверять почту на внутреннем сервере. Эти порты можно было бы также защитить шифрованием SSL. Тогда Nmap обозначил бы службы как `ssl/pop3` и `ssl/imap`. Феликс испытает на них атаки перечисления пользователей и подбора паролей, которые, вероятно, окажутся значительно эффективнее, чем против SSH.

Последний открытый порт занят прокси-сервером Squid. Это ещё одна служба, которая, возможно, предназначалась для внутренних клиентов и не должна быть доступна извне, особенно на брандмауэре. Первоначально положительное мнение Феликса об администраторах безопасности AO продолжает ухудшаться. Он проверит, можно ли злоупотребить этим прокси для соединения с другими сайтами в Интернете. Спамеры и злоумышленники часто используют прокси таким образом, чтобы скрыть следы. Что ещё важнее, Феликс попытается через прокси попасть во внутреннюю сеть. Именно с помощью этой распространённой атаки Адриан Ламо в 2002 году проник во внутреннюю сеть New York Times[7]. Ламо поймали после того, как он позвонил журналистам, чтобы в подробностях похвастаться своими атаками на NY Times и другие

компании[8].

Следующие строки сообщают, что это машина Linux, а такая информация ценна при попытке эксплуатации. Небольшое время работы - всего три дня - было определено при снятии отпечатка ОС: Nmap отправил несколько проб, получил значения опции временной метки TCP и экстраполировал соответствующую линию обратно до нуля.

Затем Феликс изучает вывод Nmap для другой машины, показанный в примере 1.3.

Пример 1.3. Ещё одна интересная машина АО

```
Interesting ports on dhcp-23.corp.avataronline.com (6.209.24.23):
(The 65526 ports scanned but not shown below are in state: closed)
PORT      STATE    SERVICE      VERSION
135/tcp   filtered msrpc
136/tcp   filtered profile
137/tcp   filtered netbios-ns
138/tcp   filtered netbios-dgm
139/tcp   filtered netbios-ssn
445/tcp   open     microsoft-ds Microsoft Windows XP microsoft-ds
1002/tcp  open     windows-icfw?
1025/tcp  open     msrpc        Microsoft Windows msrpc
16552/tcp open     unknown
Device type: general purpose
Running: Microsoft Windows NT/2K/XP
OS details: Microsoft Windows XP Professional RC1+ through final release
```

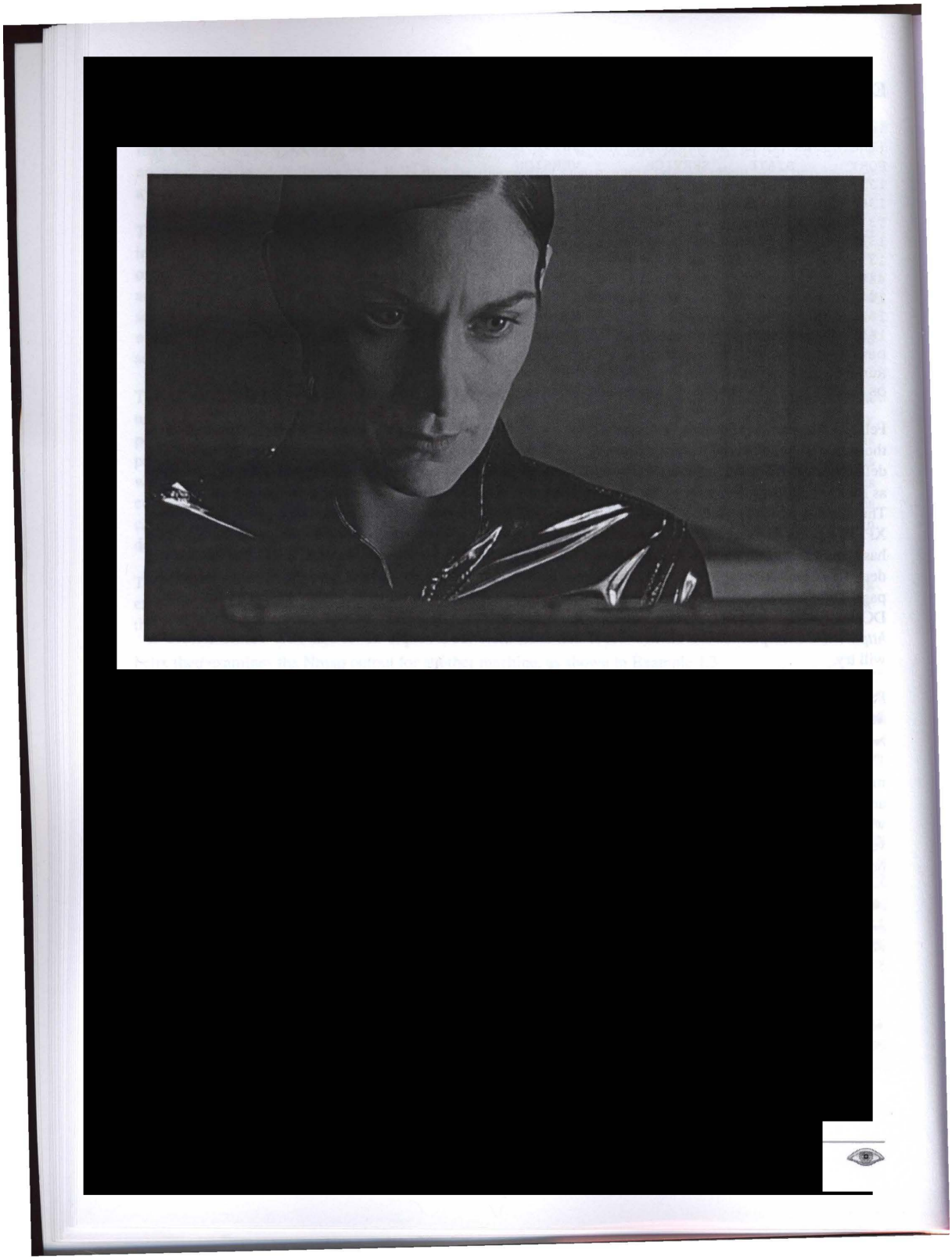
Феликс улыбается, заметив в сети эту машину Windows XP. Из-за целой череды уязвимостей MS RPC такие машины очень легко взломать, если исправления ОС не обновлены. Во второй строке указано, что состоянием по умолчанию является `closed`: для этой машины брандмауэр не применяет ту же политику запрета по умолчанию, что и для самого себя. Вместо этого администраторы попытались точечно заблокировать порты Windows 135-139, которые сочли опасными. Такой фильтр совершенно недостаточен, поскольку в Windows XP Microsoft предоставляет функции MS RPC через множество других портов. Два примера из этого сканирования - TCP-порты 445 и 1025. Хотя Nmap не сумел распознать порт 16552, Феликс видел такую картину достаточно часто и знает, что это, вероятно, служба MS Messenger. Если бы АО применяла фильтрацию с запретом по умолчанию, порт 16552 вообще не был бы доступен. Просматривая страницу результатов, Феликс замечает в этой DHCP-сети ещё несколько машин Windows. Ему не терпится испытать на них свой любимый эксплойт DCOM RPC. Эксплойт написал HD Moore, и он доступен по адресу http://www.metasploit.com/tools/dcom.c. Если тот не сработает, Феликс попробует ещё пару более новых уязвимостей MS RPC.

Феликс продолжает скрупулёзно изучать результаты в поисках уязвимостей, которыми можно воспользоваться для взлома сети. В рабочей сети он видит, что `gw.avataronline.com` - маршрутизатор Cisco, который также служит для систем простейшим брандмауэром. Администраторы попались в ловушку, блокируя только привилегированные порты, то есть порты ниже 1024; в результате в этой сети остаётся доступно множество уязвимых служб SunRPC и других служб. На каждой машине с именем вида `clust-*` открыты десятки портов, которые Nmap не распознаёт. Вероятно, это пользовательские демоны, на которых работает игровой движок АО. `www.avataronline.com` - машина Linux с открытым сервером Apache на портах HTTP и HTTPS. К сожалению, он скомпонован с уязвимой версией библиотеки OpenSSL. Вот незадача! Ещё до заката Феликс получает привилегированный доступ к узлам как в корпоративной, так и в рабочей сети.

Как показал Феликс, аудиторы безопасности и сетевые администраторы часто используют Nmap для поиска уязвимостей в сетях клиентов и компаний. В последующих главах значительно подробнее описываются применённые Феликсом методы и многие другие возможности Nmap.

1.2.2. Спасение человечества

Рисунок 1.1. Тринити начинает атаку



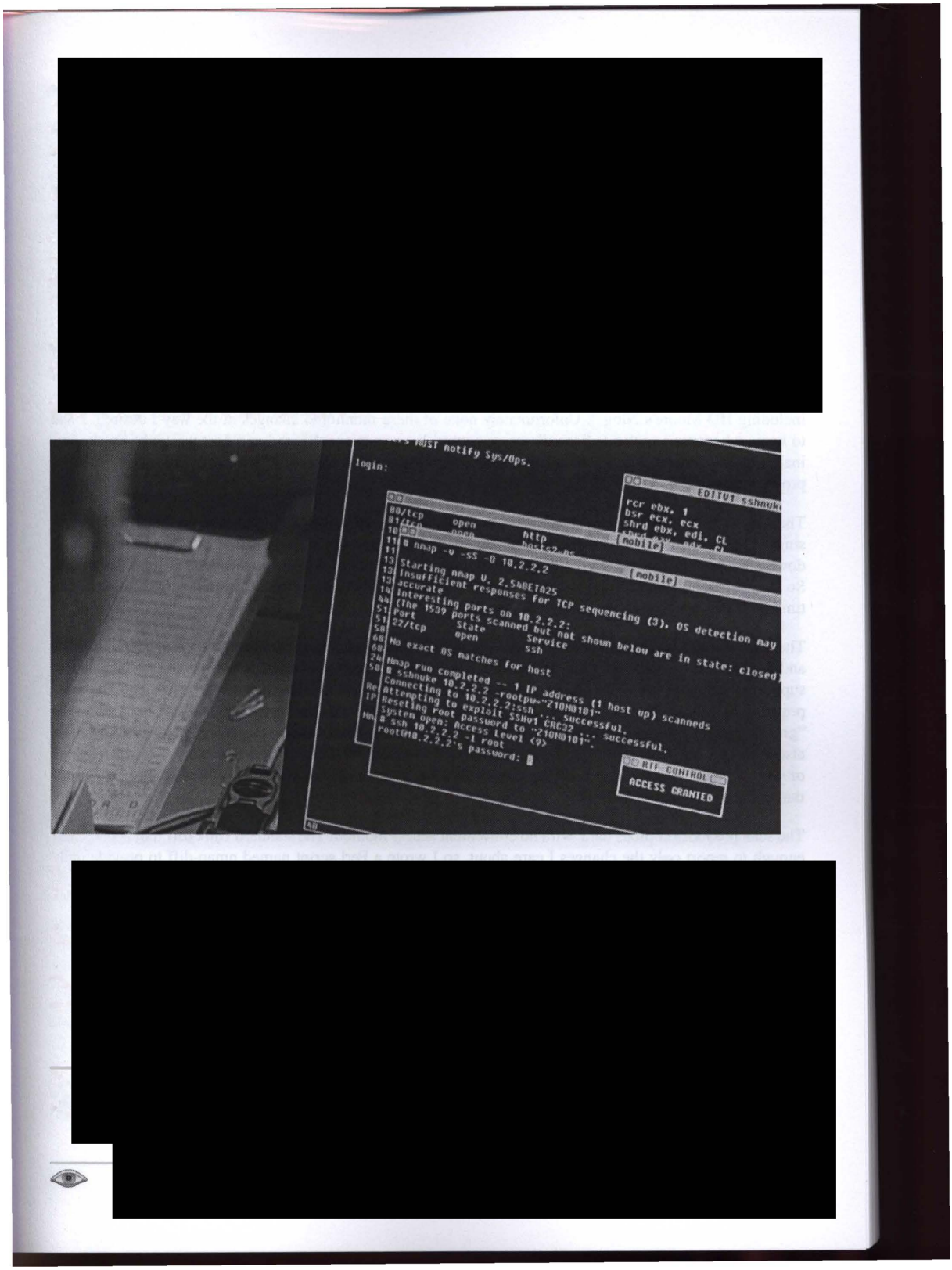
Тринити попала в серьёзную переделку! Узнав, что мир, который мы воспринимаем как данность, в действительности является виртуальной «Матрицей» под властью машин, она решает дать отпор и освободить человечество от ментального рабства. Положение усугубляется тем, что её подземную колонию освобождённых людей (Зион) атакуют 250 000 могучих машин-часовых. Единственная

надежда Тринити - менее чем за пять минут отключить систему аварийного электроснабжения 27 городских кварталов. Предыдущая команда погибла при попытке сделать это. К чему обратиться в самые мрачные минуты жизни, когда кажется, что надежды больше нет? Конечно же, к Nmap! Но пока ещё рано.

Сначала ей нужно преодолеть защиту периметра, которая во многих сетях включает брандмауэры и системы обнаружения вторжений (IDS). Тринити прекрасно знакома с передовыми методами обхода таких устройств, описанными далее в этой книге. К сожалению, администраторы аварийной энергосистемы догадались не подключать столь критически важную систему к Интернету даже косвенно. Ни маршрутизация от источника, ни сканирование с подменой IP ID не помогут Тринити преодолеть защиту посредством «воздушного зазора». Быстро соображая, она придумывает хитрый план: перепрыгнуть на мотоцикле с крыши соседнего здания, приземлиться на пост охраны электростанции, а затем избить всех охранников. Эта передовая методика не описана ни в одном руководстве по физической безопасности, но оказывается чрезвычайно эффективной. Так становится ясно, что изобретательные взломщики исследуют и разрабатывают собственные атаки, а не всегда следуют подходу скрипт-кидди и пользуются готовыми эксплойтами.

Пробившись в компьютерный зал, Тринити садится за терминал. Она быстро определяет, что в сети используется частное адресное пространство `10.0.0.0/8`. Ping-запрос к адресу сети получает ответы от десятков машин. Ping-сканирование Nmap дало бы более полный список доступных машин, но широковещательный приём сэкономил драгоценные секунды. Затем она достаёт Nmap[9]. На терминале установлена версия 2.54BETA25. Эта версия очень старая (2001 год) и менее эффективна, чем новые выпуски, но у Тринити нет времени устанавливать более совершенную версию из будущего. В любом случае работа не займёт много времени. Она выполняет команду `nmap -v -sS -O 10.2.1.3`. Команда запускает TCP SYN-сканирование и определение ОС для `10.2.1.3`, выводя подробные сведения. Узел выглядит сущей катастрофой в плане безопасности: AIX 3.2 и намного больше десятка открытых портов. К сожалению, это не та машина, которую ей нужно взломать. Поэтому она выполняет ту же команду для `10.2.2.2`. На этот раз целевая ОС не распознана (нужно было обновить Nmap!), и открыт только порт 22. Это зашифрованная служба администрирования Secure Shell. Любая сексуальная богиня-взломщица в костюме из ПВХ знает, что во многих SSH-серверах того времени (2001 год) присутствует эксплуатируемая уязвимость в детекторе компенсационной атаки CRC32. Тринити достаёт эксплойт, целиком написанный на ассемблере, и с его помощью меняет пароль `root` на целевой машине на `Z10N0101`. В обычных обстоятельствах Тринити пользуется куда более надёжными паролями. Она входит в систему как `root` и отдаёт команду отключить систему аварийного электроснабжения 27 городских кварталов, закончив точно в срок! Вот снимок происходящего: прищурьтесь как следует, и сможете прочесть текст.

Рисунок 1.2. Тринити сканирует Матрицу



Кроме того, в Интернете доступна видеозапись с видом терминала, показывающая весь взлом. По крайней мере, она будет там до тех пор, пока об этом не узнает MPAA и не пошлёт за владельцами сайта часовых или юристов[10].

1.2.3. MadHat в Стране чудес

Эта история отличается от предыдущих тем, что произошла на самом деле. Её написал активный пользователь и участник Nmap MadHat; он рассказывает, как расширил и приспособил Nmap для повседневной работы на крупном предприятии.

В полном соответствии с духом открытого исходного кода он опубликовал эти ценные сценарии на своём веб-сайте[11]. IP-адреса изменены, чтобы не раскрывать компанию. Остаток этого раздела изложен его собственными словами.

Потратив пару десятилетий на изучение компьютеров и пройдя путь от технической поддержки через системное администрирование до работы мечты - должности руководителя по информационной безопасности крупной интернет-компании, - я столкнулся с проблемой. На меня одного возложили ответственность за наблюдение за безопасностью всего нашего адресного пространства IP. Когда несколько лет назад я начинал, это было почти 50 000 узлов по всему миру, а с тех пор их число удвоилось.

Сканировать все эти машины в поисках возможных уязвимостей в рамках ежемесячных или ежеквартальных проверок было бы и без того непросто, но руководство хотело выполнять эту работу ежедневно. Злоумышленники не станут ждать неделю или месяц, прежде чем воспользоваться новой открывшейся уязвимостью, а значит, и я не могу столько ждать, чтобы найти и устранить её.

Изучив доступные инструменты, я быстро выбрал Nmap в качестве сканера портов. Его широко считают лучшим сканером, а я уже много лет применял его для устранения неполадок в сетях и проверки безопасности. Затем мне понадобилась программа, которая объединяла бы вывод Nmap и печатала различия между запусками. Я рассмотрел несколько существующих инструментов, включая Nlog HD Moore[12]. К сожалению, ни один из них не отслеживал изменения так, как мне требовалось. Мне нужно было узнавать о каждом случае неверной настройки маршрутизатора или списка управления доступом брандмауэра, а также о каждом узле, публично предоставлявшем недопустимое содержимое. Масштабируемость этих решений тоже вызывала сомнения, поэтому я решил взяться за задачу сам.

Первой возникла проблема скорости. Наши сети расположены по всему миру, но для сканирования мне предоставили только один узел в США. Во многих случаях брандмауэры между площадками значительно замедляли сканирование. Проверка всех 100 000 узлов занимала более 30 часов, что неприемлемо для ежедневного сканирования. Поэтому я написал сценарий `nmap-wrapper`, который параллельно запускает десятки процессов Nmap и сокращает время сканирования до пятнадцати часов, даже с учётом определения ОС.

Следующей проблемой стал столь огромный объём данных. База данных SQL казалась лучшим подходом с точки зрения масштабируемости и интеллектуального анализа данных, но из-за нехватки времени от этой идеи пришлось отказаться. Возможно, такая поддержка появится в будущей версии. Вместо этого я использовал обычный файл, в котором для каждого дня хранились результаты каждого диапазона адресов класса C. Самым мощным и расширяемым способом разбора и хранения этих сведений был формат XML Nmap, но я выбрал формат «для обработки `grep`» (опция `-oG`), поскольку его очень легко разбирать простыми сценариями. Для подготовки отчётов сохраняются также временные метки отдельных узлов. Они оказались весьма полезны, когда администраторы пытаются обвинить сканер в сбоях машин или служб. Нельзя правдоподобно утверждать, что служба отказала в 7:12, если у меня есть доказательство того, что сканирование выполнялось в 9:45.

Сканирование создаёт огромный объём данных, к которому нет удобного способа доступа. Стандартная утилита Unix `diff` недостаточно умна, чтобы сообщать только о важных для меня изменениях, поэтому я написал сценарий Perl под названием `nmap-diff`, формирующий ежедневные отчёты об изменениях. Типичный отчёт показан в примере 1.4.

Пример 1.4. Типичный вывод `nmap-diff`

```
> nmap-diff.pl -c3
  5 IPs showed changes

  10.12.4.8 (ftp-box.foocompany.biz)
       21/tcp open ftp
       80/tcp open http
      443/tcp open https
     1027/tcp open IIS
   + 1029/tcp open ms-lsa
    38292/tcp open landesk-cba
  OS: Microsoft Windows Millennium Edition (Me)
      Windows 2000 Professional or Advanced Server
      or Windows XP

  10.16.234.3 (media.foocompany.biz)
       80/tcp open http
   +  554/tcp open rtsp
   + 7070/tcp open realserver

  192.168.10.186 (testbox.foocompany.biz)
   + 8082/tcp open blackice-alerts
  OS: Linux Kernel 2.4.0 - 2.5.20

  172.24.12.58 (mta.foocompany.biz)
   +   25/tcp open smtp
  OS: FreeBSD 4.3 - 4.4PRERELEASE

  172.23.76.22 (media2.foocorp.biz)
       80/tcp open http
     1027/tcp open IIS
   + 1040/tcp open netsaint
     1755/tcp open wms
     3372/tcp open msdtc
     6666/tcp open irc-serv
     7007/tcp open afs3-bos
  OS: Microsoft Windows Millennium Edition (Me)
      Windows 2000 Professional or Advanced Server
      or Windows XP
```

Руководство и сотрудники были впечатлены, когда я продемонстрировал эту новую систему на внутреннем симпозиуме компании по безопасности. Но вместо того чтобы позволить мне почивать на лаврах, они начали просить новые возможности. Им требовалось подсчитывать почтовые и веб-серверы, оценивать рост и многое другое. Все эти данные содержались в результатах сканирования, однако получать к ним доступ было трудно. Поэтому я создал ещё один сценарий Perl - `nmap-report`, который значительно упростил запросы к данным. Он принимает условия, например открытые порты или операционные системы, и находит все системы, которые соответствовали им в заданный день.

У такого подхода к наблюдению за безопасностью есть одна проблема: сотрудники не всегда размещают службы на официальных портах, зарегистрированных IANA. Например, они могут поместить веб-сервер на порт 22 (SSH) или наоборот. Как раз когда я размышлял, как решить эту проблему, в Nmap появилась расширенная система определения служб и версий (см. главу 7 «Определение версий служб и приложений»). Теперь у `nmap-report` есть возможность повторного сканирования, которая с помощью определения версий сообщает настоящие службы, а не делает предположения по номеру порта. Я надеюсь ещё теснее интегрировать определение версий в будущих выпусках. В примере 1.5 показано, как `nmap-report` перечисляет FTP-серверы.

Пример 1.5. Выполнение `nmap-report`

```
> nmap-report -p21 -rV
[...]
172.21.199.76 (ftp1.foocorp.biz)
  21/tcp open ssl|ftp Serv-U ftpd 4.0
192.168.12.56 (ftp2.foocorp.biz)
```

```
    21/tcp open ftp     NcFTPd

192.168.13.130 (dropbox.foocorp.biz)
    21/tcp open ftp     WU-FTPD 6.00LS
```

Эти сценарии далеки от совершенства, но оказались весьма ценными для наблюдения за изменениями в больших сетях, способными повлиять на безопасность. Поскольку сам Nmap имеет открытый исходный код, мне показалось справедливым также опубликовать свои сценарии. Они находятся в свободном доступе по адресу http://www.unspecific.com/nmap.

1.3. Этапы сканирования Nmap

Теперь, когда мы познакомились с некоторыми применениями Nmap, посмотрим, что происходит во время сканирования. Сканирование выполняется поэтапно, причём каждый этап завершается до начала следующего. Как видно из приведённых ниже описаний, Nmap - это гораздо больше, чем просто сканирование портов.

Перечисление целей. На этом этапе Nmap исследует указанные пользователем спецификаторы узлов, которые могут представлять собой сочетание DNS-имён узлов, IP-адресов, сетевых обозначений CIDR и других форм. Можно даже попросить Nmap выбрать цели за вас с помощью `-iR`! Nmap преобразует эти спецификаторы в список адресов IPv4 или IPv6 для сканирования. Пропустить этот этап нельзя, поскольку он необходим для дальнейшего сканирования, однако обработку можно упростить, передав только IP-адреса, чтобы Nmap не выполнял прямое разрешение имён. Если передать опции `-sL -n` (сканирование списка без обратного разрешения DNS), Nmap напечатает цели и не станет выполнять дальнейшее сканирование. Этот этап рассматривается в разделе 3.2 «Задание целевых узлов и сетей» и разделе 3.5.1 «Сканирование списка (`-sL`)».

Обнаружение узлов (ping-сканирование). Сканирование сети обычно начинается с определения того, какие цели в сети работают и потому заслуживают более глубокого исследования. Этот процесс называется обнаружением узлов или ping-сканированием. Nmap предлагает множество методов обнаружения узлов - от быстрых запросов ARP до сложных сочетаний TCP-, ICMP- и других проб. Этот этап выполняется по умолчанию, однако его можно пропустить, просто считая все целевые IP-адреса работающими, с помощью опции `-PN` (без ping-проверки). Чтобы завершить работу после обнаружения узлов, укажите `-sP -n`. Обнаружению узлов посвящена глава 3.

Обратное разрешение DNS. Определив сканируемые узлы, Nmap ищет обратные DNS-имена всех узлов, которые ping-сканирование обнаружило работающими. Иногда имя узла подсказывает его назначение, а имена делают отчёты понятнее, чем одни номера IP. Этот этап можно пропустить с помощью опции `-n` (без разрешения) или расширить на все целевые IP-адреса, включая неработающие, с помощью `-R` (разрешать все). Разрешение имён рассматривается в разделе 3.4 «Разрешение имён DNS».

Сканирование портов. Это основная операция Nmap. Пробы отправляются, а ответы на них или отсутствие ответов используются для классификации удалённых портов по таким состояниям, как `open`, `closed` и `filtered`. Этот краткий рассказ даже не начинает охватывать многочисленные типы сканирования Nmap, возможности настройки сканирования и алгоритмы повышения скорости и точности. Обзор сканирования портов приведён в главе 4. Подробные сведения об алгоритмах и опциях командной строки находятся в главе 5. Сканирование портов выполняется по умолчанию, но его можно пропустить и всё же выполнить некоторые последующие этапы трассировки маршрута и части движка сценариев Nmap, указав соответствующие им опции командной строки (например, `--traceroute` и `--script`) вместе с ping-сканированием (`-sP`).

Определение версий. Если некоторые порты оказались открыты, Nmap может определить, какое серверное программное обеспечение работает в удалённой системе. Для этого он отправляет разнообразные пробы и сопоставляет ответы с базой данных, содержащей тысячи известных сигнатур служб. Определение версий включается опцией `-sV`. Оно полностью описано в главе 7.

Определение ОС. Если оно запрошено опцией `-O`, Nmap переходит к определению ОС. Разные операционные системы реализуют сетевые стандарты с едва заметными различиями. Измеряя эти различия, нередко удаётся определить операционную систему, работающую на удалённом узле. Nmap сопоставляет ответы на стандартный набор проб с базой данных, содержащей более тысячи известных вариантов ответов операционных систем. Определение ОС рассматривается в главе 8.

Трассировка маршрута. В Nmap имеется оптимизированная реализация traceroute, включаемая опцией `--traceroute`. Она способна параллельно находить сетевые маршруты ко множеству узлов, используя лучшие доступные пакеты-пробы, выбранные по результатам предыдущих этапов обнаружения Nmap. Обычно traceroute влечёт ещё один цикл обратного разрешения DNS для промежуточных узлов. Дополнительные сведения приведены в разделе 15.4 «Обнаружение узлов».

Сканирование сценариями. Движок сценариев Nmap (NSE) использует набор специализированных сценариев, чтобы получить ещё больше сведений об удалённых системах. В основе NSE лежат язык программирования Lua и стандартная библиотека, предназначенная для сбора сетевой информации. Среди предоставляемых возможностей - расширенное определение версий, уведомление об уязвимостях служб и обнаружение бэкдоров и другого вредоносного программного обеспечения. NSE - обширная тема, полностью рассмотренная в главе 9. NSE не выполняется, если его явно не запросить такими опциями, как `--script` или `-sC`.

Вывод. Наконец Nmap собирает все полученные сведения и записывает их на экран или в файл. Nmap может формировать вывод в нескольких форматах. В этой книге обычно представлен стандартный удобочитаемый формат (интерактивный формат). Помимо прочих, Nmap предлагает также формат вывода на основе XML. Все тонкости вывода рассматриваются в главе 13.

Как уже говорилось, Nmap предлагает множество опций, управляющих выполнением этих этапов. При сканировании больших сетей каждый этап повторяется много раз, поскольку Nmap обрабатывает узлы небольшими группами. Он полностью сканирует каждую группу и выводит её результаты, после чего переходит к следующей группе узлов.

1.4. Правовые вопросы

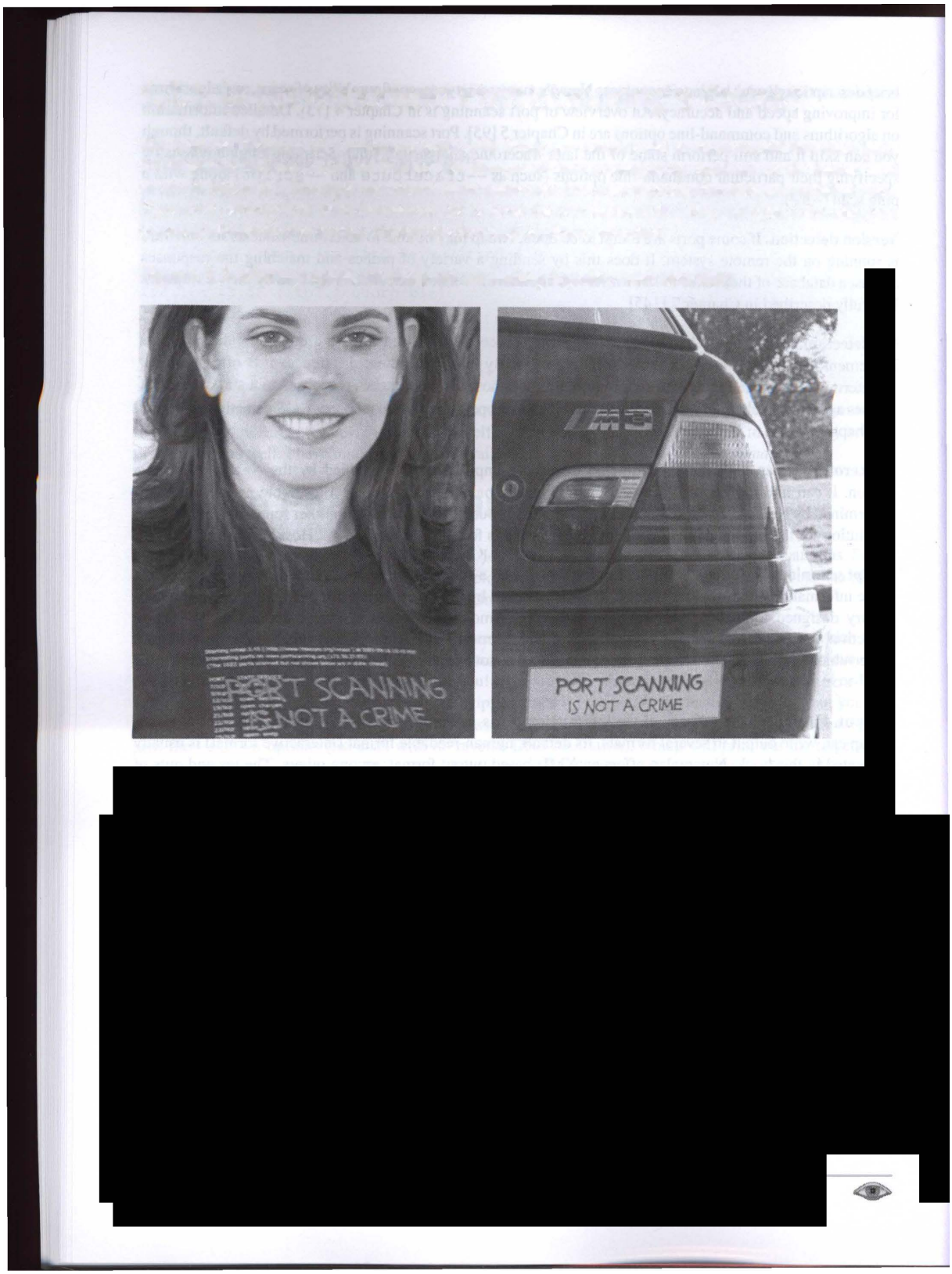
При правильном использовании Nmap помогает защитить сеть от нарушителей. Но при неправильном применении Nmap в редких случаях может привести вас в суд, стать причиной увольнения, исключения из учебного заведения, тюремного заключения или блокировки интернет-провайдером. Прежде чем запускать Nmap, прочитайте это руководство по правовым вопросам, чтобы снизить риск.

1.4.1. Является ли несанкционированное сканирование портов преступлением?

Правовые последствия сканирования сетей с помощью Nmap сложны и вызывают столько споров, что сторонние организации даже выпускали футболки и наклейки на бампер с выражением своих взглядов по этому вопросу, как показано на рисунке 1.3[13]. Эта тема также порождает множество жарких, но зачастую бесплодных дискуссий и словесных войн. Если вы когда-либо участвуете в таких обсуждениях, старайтесь избегать затёртых и плохо подходящих аналогий со стуком в чужую

дверь или проверкой того, заперты ли у человека двери и окна.

Рисунок 1.3. Категорические мнения о законности и нравственности сканирования портов



Хотя я согласен с мнением, что сканирование портов не должно быть незаконным, редко бывает разумно получать юридические советы с футболки. Впрочем, получать их от инженера-программиста и автора книги лишь немногим лучше. Чтобы лучше понять, как закон применяется к вашей конкретной ситуации, обратитесь к компетентному юристу в своей

юрисдикции. Сделав эту важную оговорку, приведу некоторые общие сведения, которые могут оказаться полезными.

Лучший способ избежать споров при использовании Nmap - всегда получать письменное разрешение представителей целевой сети до начала любого сканирования. Вероятность того, что ваш интернет-провайдер создаст проблемы, если заметит сканирование или если администраторы цели по ошибке направят ему жалобу на злоупотребление, всё равно остаётся, но обычно такую ситуацию легко разрешить. При выполнении теста на проникновение разрешение должно быть включено в техническое задание. Проверяя собственную компанию, убедитесь, что эта деятельность однозначно входит в ваши должностные обязанности. Консультантам по безопасности следует ознакомиться с превосходным «Руководством по методологии тестирования безопасности с открытым исходным кодом» (Open Source Security Testing Methodology Manual, OSSTMM)[14], в котором приведены лучшие практики для таких ситуаций.

Хотя гражданские и особенно уголовные судебные дела представляют собой кошмарный сценарий для пользователей Nmap, они крайне редки. В конце концов, ни один федеральный закон США прямо не объявляет сканирование портов незаконным. Гораздо чаще целевая сеть замечает сканирование и направляет жалобу поставщику сетевых услуг, из сети которого оно исходило, то есть вашему интернет-провайдеру. Похоже, большинство сетевых администраторов не обращает внимания или даже не замечает множества сканирований, ежедневно ударяющихся об их сети, но некоторые всё же жалуются. Интернет-провайдер, из сети которого велось сканирование, может разыскать пользователя по указанным в жалобе IP-адресу и времени, а затем отчитать его или даже отключить от услуги. Иногда сканирование портов без разрешения нарушает правила допустимого использования (Acceptable Use Policy, AUP) провайдера. Например, в тот момент AUP огромного провайдера кабельного Интернета Comcast гласили[15]:

> Инструменты исследования сети или сканирования портов разрешены только при использовании совместно с домашней сетью частного лица либо при наличии явного разрешения целевого узла и/или сети. Несанкционированное сканирование портов по любой причине строго запрещено.

Даже если интернет-провайдер прямо не запрещает несанкционированное сканирование портов, он может заявить, что применимо какое-либо положение «против взлома». Разумеется, от этого сканирование портов не становится незаконным. Интернет-провайдеры запрещают множество совершенно законных и, в США, защищённых конституцией действий. Например, приведённые выше AUP запрещают пользователям передавать, хранить или публиковать «любую информацию или материал, которые разумный человек мог бы счесть нежелательными, оскорбительными, непристойными, порнографическими, ... смущающими, тревожащими, вульгарными, исполненными ненависти, оскорбительными по расовому или этническому признаку либо иным образом неуместными, независимо от того, являются ли этот материал или его распространение незаконными». Иными словами, некоторые интернет-провайдеры запрещают любое поведение, которое потенциально может кого-либо оскорбить или раздражать. Беспорядочное сканирование чужих сетей и компьютеров вполне способно вызвать такую реакцию. Если вы всё же решите проводить столь спорное сканирование, никогда не делайте этого с работы, из учебного заведения или через иного поставщика услуг, обладающего существенным влиянием на ваше благополучие. Вместо этого используйте коммутируемое соединение или коммерческого поставщика широкополосного доступа. Потеря DSL-соединения и необходимость сменить провайдера - небольшое неудобство, несравнимо предпочтительнее исключения или увольнения.

Хотя судебные дела, связанные со сканированием портов без последующих взломов, редки, они всё же случаются. Одно из самых известных дел касалось Скотта Моултона, у которого был действующий договор на консультационное обслуживание системы экстренного вызова 911 округа Чероки, штат Джорджия. В декабре 1999 года ему поручили настроить маршрутизатор,

соединяющий полицейское управление города Кантон, штат Джорджия, с центром E911. Опасаясь, что это может поставить под угрозу безопасность центра E911, Скотт начал предварительное сканирование портов задействованных сетей. В процессе он просканировал веб-сервер округа Чероки, принадлежавший конкурирующей консультационной фирме VC3 и обслуживаемый ею. Сотрудники VC3 заметили сканирование и отправили Скотту письмо. Он ответил, что работает на центр 911 и проверяет безопасность. После этого VC3 сообщила о случившемся в полицию. Скотт потерял договор на обслуживание E911 и был арестован по обвинению в нарушении пункта 1030(a)(5)(B) американского Закона о компьютерном мошенничестве и злоупотреблениях[16]. Закон применяется к тому, кто «преднамеренно получает несанкционированный доступ к защищённому компьютеру и в результате такого поведения причиняет ущерб» при выполнении также других условий. Заявленный VC3 ущерб состоял во времени, затраченном на расследование сканирования портов и связанных действий. Скотт подал иск к VC3 о клевете, а VC3 подала встречный иск о нарушении Закона о компьютерном мошенничестве и злоупотреблениях и Закона штата Джорджия о защите компьютерных систем.

Гражданское дело против Скотта было прекращено до суда, что указывает на полное отсутствие оснований. Решение заставило многих пользователей Nmap улыбнуться:

> «Суд постановил, что выполненные истцом несанкционированное сканирование портов и проверка пропускной способности серверов ответчика не являются нарушением ни Закона штата Джорджия о защите компьютерных систем, ни Закона о компьютерном мошенничестве и злоупотреблениях». - Гражданское дело № 1:00-CV-434-TWT (Северный округ Джорджии, 6 ноября 2000 года)

Это была воодушевляющая победа в гражданском деле, но над Скоттом всё ещё висели уголовные обвинения. К счастью, он не падал духом и отправил в список рассылки `nmap-hackers` следующую заметку[17]:

> Я горжусь тем, что смог принести некоторую пользу компьютерному сообществу, защищая права специалистов в компьютерной области. Однако поддержка таких усилий обходится ЧРЕЗВЫЧАЙНО дорого, чему я совсем не рад. Но я продолжу бороться и доказывать, что в сканировании портов нет ничего незаконного, особенно когда я всего лишь выполнял свою работу.

В конце концов уголовный суд пришёл к тому же выводу, и все обвинения были сняты. Хотя в итоге Скотт был оправдан, ему пришлось оплатить шестизначные счета за юридические услуги и пережить несколько тяжёлых лет судебной борьбы. Положительной стороной стало то, что, потратив столько времени на обучение своих адвокатов техническим вопросам дела, Скотт основал успешную компанию, оказывающую услуги компьютерной криминалистики[18].

Хотя дело Моултона подаёт хороший пример, пусть и не создаёт судебного прецедента, другие суды или обстоятельства всё ещё могут привести к худшему исходу. Помните, что во многих штатах действуют собственные законы о компьютерных злоупотреблениях, некоторые из которых при определённом толковании могут объявлять незаконной даже ping-проверку удалённой машины без разрешения[19].

Законы других стран, разумеется, тоже отличаются. Например, в Финляндии 17-летний юноша был признан виновным в попытке компьютерного вторжения лишь за сканирование портов банка[20]. Его оштрафовали, чтобы покрыть расходы цели на расследование. Решение по делу Моултона могло оказаться иным, если бы машина VC3 действительно вышла из строя и компании удалось обосновать требуемый законом ущерб в размере 5 000 долларов.

На другом полюсе находится израильский судья, который в начале 2004 года оправдал Ави Мизрахи[21], сканировавшего уязвимости секретной службы «Моссад». Судья Авраам Тенненбаум

даже похвалил Ави следующими словами:

> В некотором смысле пользователи Интернета, проверяющие веб-сайты на уязвимости, действуют на благо общества. Если у них нет злого умысла и они не причиняют ущерба, их следует даже похвалить.

В 2007 и 2008 годах в Германии[22] и Англии[23] вступили в силу новые законы о киберпреступлениях с широкой сферой действия. Эти законы призваны запретить распространение, использование и даже хранение «инструментов взлома». Например, поправка Великобритании к Закону о неправомерном использовании компьютеров запрещает «предоставлять или предлагать предоставить [инструмент], полагая, что он, вероятно, будет использован для совершения [нарушения Закона о неправомерном использовании компьютеров] или содействия такому нарушению». Из-за этих законов некоторые авторы инструментов безопасности уже прекратили работу или перенесли проекты в другие страны. Проблема в том, что большинство инструментов безопасности могут использовать и этичные специалисты («белые шляпы») для защиты своих сетей, и «чёрные шляпы» для нападения. Эти опасные законы основываются на намерениях автора или пользователя инструмента, а намерения субъективны, и распознать их трудно. Nmap создавался, чтобы сделать Интернет безопаснее, но мне совсем не хотелось бы оказаться под арестом и быть вынужденным доказывать свои намерения судье и присяжным. Маловероятно, что эти законы затронут столь широко распространённые и популярные инструменты, как Nmap, однако они оказали сдерживающее воздействие на небольшие инструменты и на те, которыми чаще злоупотребляют компьютерные преступники (например, комплексы средств эксплуатации уязвимостей).

Каким бы ни был правовой статус сканирования портов, учётные записи у интернет-провайдеров будут по-прежнему закрывать, если поступит много жалоб. Лучший способ избежать сообщений провайдеру о злоупотреблении или гражданских и уголовных обвинений - прежде всего не раздражать администраторов целевой сети. Вот несколько практических советов:

• Вероятно, не менее 90% сетевого сканирования не вызывает споров. Вас едва ли станут беспокоить за сканирование собственной машины или сетей, которыми вы управляете. Споры возникают при сканировании чужих сетей. Для такого исследования сети существует множество причин, хороших и плохих. Возможно, вы сканируете другие системы в общежитии или отделе в поисках общедоступных файлов (FTP, SMB, WWW и так далее). Или просто пытаетесь найти IP-адрес определённого принтера. Возможно, вы просканировали любимый веб-сайт, чтобы узнать, предлагает ли он другие службы, или из любопытства к используемой операционной системе. Быть может, вы лишь проверяете связность или хотите быстро удостовериться в безопасности, прежде чем передавать данные своей кредитной карты компании электронной торговли. Возможно, вы проводите исследование Интернета. Или выполняете первоначальную разведку, готовясь к попытке взлома? Удалённые администраторы редко знают ваши истинные намерения и порой начинают подозревать неладное. Лучше всего сначала получить разрешение. Я видел, как несколько человек, не занимавших административных должностей, попадали в серьёзные неприятности, решив «доказать» небезопасность сети навязчивым сканированием всей компании или кампуса. Администраторы обычно охотнее сотрудничают, когда их спрашивают заранее, а не будят в три часа ночи сигналом IDS о массированной атаке. Поэтому по возможности получайте письменное разрешение до сканирования сети. Адриан Ламо, вероятно, избежал бы тюрьмы, если бы попросил *New York Times* разрешить ему проверить её безопасность, а не рассказал журналистам об уязвимостях постфактум. К сожалению, ему, скорее всего, отказали бы. Будьте готовы к такому ответу.

• Ограничивайте цель сканирования как можно точнее. Любую подключённую к Интернету машину сканируют достаточно регулярно, поэтому большинство администраторов игнорируют такой сетевой шум. Однако сканирование большого числа сетей или выполнение очень шумных и

навязчивых проверок повышает вероятность жалоб. Если вы ищете только веб-серверы, укажите `-p80`, а не сканируйте все 65 536 портов TCP каждой машины. Если нужно лишь найти доступные узлы, выполните ping-сканирование Nmap вместо полного сканирования портов. Не сканируйте сеть CIDR `/16` (65 тысяч узлов), когда достаточно блока `/24`. Теперь режим случайного сканирования принимает аргумент с числом узлов, а не работает бесконечно. Поэтому, если этого достаточно, выберите `-iR 1000` вместо `-iR 10000`. Используйте стандартные временные параметры (или даже `-T polite`), а не `-T insane`. Избегайте шумных и сравнительно навязчивых проверок, таких как определение версий (`-sV`). Аналогично, SYN-сканирование (`-sS`) тише сканирования с установлением соединения (`-sT`), хотя даёт те же сведения и часто работает быстрее.

• Как отмечалось ранее, не делайте ничего спорного через соединение на работе или в учебном заведении. Даже при благих намерениях вы слишком многим рискуете, если наделённый властью человек (например, начальник или декан) решит, что вы злонамеренный взломщик. Неужели вы хотите объяснять свои действия тому, кто, возможно, даже не понимает терминов «пакет» или «сканер портов»? Потратьте 40 долларов в месяц на коммутируемое соединение, учётную запись с оболочкой или домашний широкополосный доступ. Если вы кого-то обидите с такой учётной записи, последствия будут менее суровыми, а администраторы целевой сети с меньшей вероятностью вообще станут жаловаться массовому провайдеру. Прочитайте также соответствующую политику допустимого использования и выбирайте провайдера с её учётом. Если ваш провайдер, подобно рассмотренной выше Comcast, запрещает любое несанкционированное сканирование портов и публикацию «оскорбительных» материалов, не удивляйтесь отключению за такую деятельность. Как правило, чем больше вы платите поставщику услуг, тем сговорчивее он будет. Провайдер линии T1 едва ли отключит вас без предупреждения из-за сообщения о сканировании портов. Провайдер коммутируемого или домашнего DSL- либо кабельного доступа вполне может так поступить. Это способно произойти, даже когда сканирование подделал кто-то другой.

• Nmap предлагает много вариантов скрытного сканирования, включая подмену исходного IP-адреса, сканирование с ложными узлами и более новую технику холостого сканирования. Они рассматриваются в главе об обходе IDS. Но помните, что всегда приходится идти на компромисс. Вас труднее найти, если запускать сканирование через открытую беспроводную точку доступа вдали от дома, с 17 ложными узлами, а последующие пробы отправлять по цепочке из девяти открытых прокси. Но если вас всё же отследят, ваши намерения вызовут очень серьёзные подозрения.

• Всегда имейте законное основание для сканирования. Оскорблённый администратор может сначала написать вам (либо провайдер перешлёт вам его жалобу), ожидая какого-либо объяснения этой деятельности. В рассмотренном выше деле Скотта Моултона VC3 сначала отправила Скотту письмо с вопросом о происходящем. Если бы его ответ их удовлетворил, всё могло закончиться на этом, не перерастая в гражданское и уголовное разбирательство. Группы, сканирующие значительные части Интернета в исследовательских целях, часто используют обратное DNS-имя с описанием проекта и запускают веб-сервер с подробными сведениями и формой отказа от сканирования.

Помните также, что сопутствующие и последующие действия часто служат доказательством намерений. Само по себе сканирование портов не всегда означает атаку. Но сканирование портов, за которым почти сразу следует эксплуатация уязвимости IIS, заявляет о намерении громко и недвусмысленно. Это важно, поскольку решения о судебном преследовании (либо увольнении, отчислении, подаче жалобы и так далее) часто принимаются на основании события в целом, а не только одного его элемента, например сканирования портов.

Один примечательный случай произошёл с канадцем Уолтером Новаковски, который, по-видимому, стал первым человеком в Канаде, обвинённым в краже услуг связи (статья S.342.1 Уголовного кодекса Канады) за доступ в Интернет через чужую незащищённую сеть Wi-Fi. Тысячи канадских

«охотников за беспроводными сетями» делают это каждый день, так почему же выделили именно его? Из-за сопутствующих действий и намерений. Утверждалось, что его задержали, когда он ехал против движения по улице с односторонним движением, обнажённый ниже пояса, с ноутбуком в руке, загружая детскую порнографию через упомянутую незащищённую беспроводную точку доступа.[24] Полиция, по-видимому, сочла его поведение настолько вопиющим, что стала подбирать подходящие обвинения и добавила к многочисленным обвинениям, связанным с детской порнографией, кражу услуг связи.

Аналогично, обвинения, связанные со сканированием портов, обычно предъявляют в самых вопиющих случаях. Даже когда параноидальные администраторы сообщают полиции, что их просканировали, судебное преследование или любые последующие действия чрезвычайно редки. Вероятно, именно участие службы экстренной помощи 911 побудило прокуратуру действовать в деле Моултона. При написании этой книги её автор просканировал сотни тысяч узлов Интернета и не получил ни одной жалобы.

Подводя итог всему разделу, нельзя дать простой ответ на вопрос о законности сканирования портов. Как бы мне этого ни хотелось, я не могу безоговорочно сказать: «Сканирование портов никогда не является преступлением». Законы резко различаются в разных юрисдикциях, а исход дел зависит от конкретных подробностей. Даже при почти одинаковых фактах разные судьи и прокуроры не всегда толкуют их одинаково. Я могу лишь призвать к осторожности и повторить приведённые выше советы.

В целях тестирования вам разрешается сканировать узел `scanme.nmap.org`. Возможно, вы уже заметили, что он использовался в нескольких примерах. Обратите внимание: разрешается только сканирование с помощью Nmap, но не проверка эксплойтов и не атаки типа «отказ в обслуживании». Для экономии пропускной способности не запускайте против этого узла более дюжины сканирований в сутки. Если этой бесплатной целью для сканирования будут злоупотреблять, её отключат, и Nmap сообщит `Failed to resolve given hostname/IP: scanme.nmap.org`.

1.4.2. Может ли сканирование портов вывести из строя целевой компьютер или сеть?

В Nmap нет функций, предназначенных для вывода целевых сетей из строя. Обычно программа старается действовать осторожно. Например, Nmap обнаруживает потерянные пакеты и при их появлении замедляет работу, чтобы не перегружать сеть. Nmap также не отправляет повреждённых пакетов. Заголовки IP, TCP, UDP и ICMP всегда корректны, хотя узел назначения необязательно ожидает эти пакеты. По этим причинам ни приложение, ни узел, ни сетевой компонент не должны выходить из строя из-за сканирования Nmap. Если это происходит, в системе есть ошибка, которую должен исправить поставщик.

Сообщения о системах, выведенных из строя Nmap, редки, но такое случается. Вероятно, многие из этих систем изначально работали нестабильно, и Nmap либо стал последней каплей, либо система отказала одновременно со сканированием Nmap по чистой случайности. В других случаях удавалось воспроизводимо вывести из строя плохо написанные приложения, стеки TCP/IP и даже операционные системы определённой командой Nmap. Обычно это старые унаследованные устройства, поскольку новое оборудование редко выпускают с такими проблемами. Разумные компании проверяют устройства с помощью Nmap и многих других распространённых сетевых инструментов перед поставкой. Те, кто пропускает такое предварительное тестирование, часто узнают о проблеме во время ранних бета-испытаний, когда устройство впервые подключают к Интернету. Обычно проходит совсем немного времени, прежде чем данный IP-адрес просканируют как часть фонового сетевого шума. Обновление систем и устройств последними исправлениями и

микропрограммами поставщика должно уменьшить их подверженность таким проблемам, одновременно повысив безопасность и удобство использования сети.

Во многих случаях обнаружение того, что машина выходит из строя от определённого сканирования, даёт ценные сведения. В конце концов, злоумышленники могут сделать всё, что умеет Nmap, применяя сам Nmap или собственные сценарии. Устройства не должны выходить из строя от сканирования, а если это происходит, поставщиков следует вынуждать выпускать исправления. В некоторых сценариях использования нежелательно обнаруживать ненадёжные машины, выводя их из строя. Тогда можно проводить очень щадящее сканирование, чтобы снизить риск неблагоприятных последствий. Вот несколько советов:

• Используйте SYN-сканирование (`-sS`) вместо сканирования с установлением соединения (`-sT`). Приложения пользовательского режима, например веб-серверы, редко способны даже обнаружить первое, поскольку всё обрабатывается в пространстве ядра (исключение составляют некоторые старые ядра Linux), а потому у служб нет оправдания для отказа.
• Сканирование версий (`-sV`) может вывести из строя плохо написанные приложения. Аналогично, сообщалось, что некоторые жалкие операционные системы отказывают при определении ОС (`-O`). Не используйте эти опции в особенно чувствительной среде или там, где результаты вам не нужны.
• Применение временных режимов `-T2` или более медленных (`-T1`, `-T0`) может уменьшить вероятность того, что сканирование портов повредит системе, но резко замедляет проверку. В старых машинах Linux демон `identd` мог временно блокировать службы при слишком частом обращении к ним. Это могло произойти как во время сканирования портов, так и при нормальной высокой нагрузке. Более медленные временные параметры способны здесь помочь. Эти режимы следует применять лишь в крайнем случае, поскольку они могут замедлить сканирование на порядок и более.
• Ограничивайте число сканируемых портов и машин необходимым минимумом. У каждой просканированной машины есть ничтожная вероятность отказа, поэтому уменьшение числа машин улучшает ваши шансы. Сокращение числа портов снижает риск и для конечных узлов, и для сетевых устройств. Многие устройства NAT и межсетевые экраны хранят запись состояния для каждой пробы порта. Большинство удаляет старые записи, когда таблица заполняется, но отдельные жалкие реализации вместо этого выходят из строя. Сокращение числа сканируемых портов и узлов уменьшает количество записей состояния и тем самым может помочь этим скверным устройствам продолжить работу.

1.4.3. Авторские права на Nmap

Хотя Nmap имеет открытый исходный код, у него всё же есть лицензия, основанная на авторском праве, которую необходимо соблюдать. Как свободное программное обеспечение, Nmap также не предоставляет никаких гарантий. Эти вопросы намного подробнее рассматриваются в разделе 15.19 «Правовые уведомления». Компаниям, желающим включать и использовать Nmap в проприетарном программном обеспечении и устройствах, особенно рекомендуется прочитать этот раздел, чтобы случайно не нарушить лицензию Nmap. К счастью, проект Nmap продаёт лицензии на коммерческое распространение нуждающимся в них компаниям.

1.5. История и будущее Nmap

Многие старые и любимые инструменты безопасности, такие как Netcat, tcpdump и John the Ripper, почти не менялись с годами. Другие, включая Nessus, Wireshark, Cain and Abel и Snort, непрерывно развиваются со дня выпуска. Nmap относится ко второй категории. В 1997 году он вышел как

простой сканер портов только для Linux. За следующие десять с лишним лет у него появилось множество ценных возможностей, включая определение ОС, определение версий, механизм сценариев Nmap, версию для Windows, графический интерфейс пользователя и многое другое. В этом разделе приведена хронология важнейших событий более чем десятилетней истории Nmap, а затем краткие прогнозы его будущего. Обо всех существенных изменениях Nmap, которых были тысячи, читайте в журнале изменений Nmap.[25] Старые выпуски Nmap доступны по адресу http://nmap.org/dist/, а древние версии - по адресу http://nmap.org/dist-old/.

• **1 сентября 1997 года.** Nmap впервые опубликован в статье 11 выпуска 51 журнала *Phrack*.[26] Номера версии у него нет, поскольку новые выпуски не планируются. Nmap содержит около 2 000 строк, а для компиляции достаточно команды `gcc -O6 -o nmap nmap.c -lm`.
• **5 сентября 1997 года.** Из-за высокого спроса выходит слегка изменённая версия кода из *Phrack*, называющая себя версией 1.25. Сжатый gzip архив tar занимает 28 КБ. Версия 1.26 (48 КБ) выходит 19 дней спустя.
• **11 января 1998 года.** Регистрируется Insecure.Org, и Nmap переезжает туда из прежнего дома у провайдера DataHaven Project.[27]
• **14 марта 1998 года.** Рено Дерезон пишет мне, сообщая, что создаёт сканер безопасности, и спрашивает, можно ли использовать часть исходного кода Nmap. Разумеется, я отвечаю согласием. Через девять дней он присылает предварительную версию Nessus, отмечая, что она «предназначена для системных администраторов, а не для 3133t H4ck3rZ».
• **1 сентября 1998 года.** Вдохновлённый первой годовщиной Nmap, я начинаю добавлять удалённое определение ОС для будущего Nmap 2.00. 7 октября я выпускаю первую закрытую бета-версию для небольшой группы ведущих разработчиков Nmap. Несколько месяцев мы тихо работаем над ней.
• **12 декабря 1998 года.** Публично выходит Nmap версии 2.00, впервые представляющий определение ОС Nmap. Статья с описанием этих методов опубликована в *Phrack* 54, статья 9.[28] К этому моменту Nmap разделён на множество файлов, состоит примерно из 8 000 строк кода, хранится в закрытой системе управления версиями CVS, а архив tar занимает 275 КБ. Создаётся список рассылки `nmap-hackers`, впоследствии выросший до более чем 55 000 участников.

• **11 апреля 1999 года.** Выходит Nmap 2.11BETA1. Это первая версия с графическим интерфейсом пользователя как альтернативой традиционному применению из командной строки. Включённый в комплект интерфейс только для Unix под названием NmapFE первоначально написал Зак Смит. Некоторым он нравится, но большинство предпочитает запуск из командной строки.
• **28 апреля 2000 года.** Выходит Nmap 2.50.[29] К этому моменту архив tar вырос до 461 КБ. В выпуск вошли временные режимы, такие как `-T aggressive`, непосредственное сканирование SunRPC, а также методы сканирования Window и ACK.
• **28 мая 2000 года.** Герхард Ригер отправляет в список `nmap-dev` сообщение[30] с описанием разработанного им нового «сканирования протоколов» и даже прилагает исправление. Это настолько здорово, что менее чем через 12 часов я выпускаю[31] Nmap 2.54BETA1 с его исправлением.
• **7 декабря 2000 года.** Nmap 2.54BETA16 становится первой официальной версией,[32] которая компилируется и работает в Microsoft Windows. Перенос на Windows выполнили Райан Пермех и Энди Лютомирски.
• **9 июля 2001 года.** В Nmap 2.54BETA26 появляется холостое сканирование по идентификатору IP. Одновременно публикуется статья с описанием метода. Эта исключительно интересная, хотя и не всегда практичная, техника рассматривается в разделе 5.10 «Холостое сканирование TCP (`-sI`)».
• **25 июля 2002 года.** Я увольняюсь из Netscape/AOL и приступаю к работе мечты - занимаюсь Nmap полный рабочий день.
• **31 июля 2002 года.** Выходит Nmap 3.00.[33] Архив tar занимает 922 КБ. В выпуск вошли поддержка Mac OS X, вывод XML и определение времени работы системы.

• **28 августа 2002 года.** Nmap переводится с C на C++, а в выпуске Nmap 3.10ALPHA1[34] добавляется поддержка IPv6.
• **15 мая 2003 года.** Nmap появляется в фильме «Матрица: Перезагрузка», где Тринити использует его, а затем настоящий эксплойт SSH, чтобы взломать электростанцию и спасти мир. Это приносит Nmap больше известности, чем когда-либо прежде или впоследствии. Подробности и снимки экрана доступны по адресу http://nmap.org/movies.html.
• **21 июля 2003 года.** Я завершаю первую реализацию определения служб и версий Nmap (глава 7 «Определение версий служб и приложений») и выпускаю её для пары десятков ведущих разработчиков и пользователей Nmap под названием Nmap 3.40PVT1. За следующие пару месяцев следуют ещё 16 закрытых выпусков, пока мы совершенствуем систему и добавляем сигнатуры.
• **16 сентября 2003 года.** Определение служб Nmap наконец публикуется[35] в составе Nmap 3.45. Одновременно выходит подробная статья.
• **20 февраля 2004 года.** Выходит Nmap 3.50.[36] Теперь архив tar занимает 1 571 КБ. Корпорации SCO запрещается распространять Nmap, поскольку она отказывается соблюдать GPL. Ей приходится пересобрать образы ISO своего выпуска Caldera, чтобы удалить Nmap. В этот выпуск вошли трассировка пакетов и ping-проверка UDP. В нём также появилась система классификации ОС, которая распределяет каждую из сотен обнаруживаемых операционных систем по названию поставщика, названию операционной системы, поколению ОС и типу устройства.

• **31 августа 2004 года.** Для Nmap 3.70[37] переписано ядро сканирования портов Nmap. Новый механизм под названием `ultra_scan` получил значительно улучшенные алгоритмы и поддержку распараллеливания, повышающие как точность, так и скорость. Особенно заметна разница для узлов за строгими межсетевыми экранами.

• **25 июня 2005 года.** В рамках инициативы Google Summer of Code[38] компания Google оплачивает десяти студентам колледжей и аспирантам полный рабочий день над Nmap в течение лета. Среди проектов - система определения ОС второго поколения (Чжао Лэй), новый кроссплатформенный графический интерфейс Umit (Адриано Монтейро Маркес) и многие другие интересные проекты, описанные по адресу http://seclists.org/nmap-hackers/2005/0008.html.

• **8 сентября 2005 года.** С выпуском версии 3.90[39] Nmap получает поддержку отправки необработанных кадров Ethernet. Это позволяет выполнять ARP-сканирование (см. раздел 3.6.6 «ARP-сканирование (`-PR`)»), подменять MAC-адреса и обходить запрет необработанных IP-пакетов, введённый Microsoft в Windows XP SP2.

• **31 января 2006 года.** Выходит Nmap 4.00.[40] Теперь архив tar занимает 2 388 КБ. Выпуск включает взаимодействие во время работы для получения оценки оставшегося времени по запросу, исполняемый установщик Windows, обновления NmapFE с поддержкой GTK2 и многое другое.

• **24 мая 2006 года.** В рамках программы SoC Google оплачивает работу ещё десяти летних разработчиков Nmap. Чжао и Адриано возвращаются в SoC 2006 года, чтобы продолжить развитие своих проектов. Диман Тодоров получает поддержку для помощи в разработке механизма сценариев Nmap. Эти и ещё семь талантливых студентов со своими проектами описаны по адресу http://seclists.org/nmap-hackers/2006/0009.html.

• **24 июня 2006 года.** После двух лет разработки и тестирования система определения ОС второго поколения включается в Nmap 4.20ALPHA1.[41] Новая система основана на всём, чему мы научились, и новых идеях, появившихся за восемь лет после дебюта системы первого поколения. После некоторого времени, необходимого для роста базы данных, новая система оказывается гораздо точнее и подробнее прежней. Она описана в главе 8 «Удалённое определение ОС».

• **10 декабря 2006 года.** Механизм сценариев Nmap выходит[42] в составе Nmap 4.21ALPHA1. NSE позволяет пользователям писать простые сценарии и делиться ими, автоматизируя самые разнообразные сетевые задачи. Система достигает огромного успеха и описана в главе 9

«Механизм сценариев Nmap».

• **20 декабря 2006 года.** Репозиторий исходного кода Nmap в Subversion становится общедоступным.[43] До этого доступ к закрытому репозиторию исходного кода имела лишь горстка разработчиков. Всем остальным приходилось ждать выпусков. Теперь каждый может следить за разработкой Nmap изо дня в день. Существует даже список рассылки `nmap-svn`, в реальном времени уведомляющий об изменениях по электронной почте. Подробности приведены в разделе 2.1.5 «Получение Nmap из репозитория Subversion (SVN)».

• **28 мая 2007 года.** В рамках программы SoC Google оплачивает работу шести летних разработчиков Nmap. Тем временем графический интерфейс Umit для Nmap под руководством Адриано утверждается как самостоятельная программа, финансируемая SoC. Среди поддержанных студентов был Дэвид Файфилд, продолживший работу спустя долгое время после окончания лета и ставший одним из ведущих разработчиков Nmap. Студенты Nmap и их проекты перечислены по адресу http://seclists.org/nmap-hackers/2007/0003.html.

• **27 июня 2007 года.** В кинотеатрах выходит фильм «Крепкий орешек 4.0». В нём есть короткая сцена, где хакер Мэттью Фаррелл (Джастин Лонг) демонстрирует владение Nmap. Затем он оставляет компьютер и присоединяется к Брюсу Уиллису в борьбе с дьявольски хитрым главарём террористов. Через неделю выходит «Ультиматум Борна», в котором тоже есть сцена с Nmap! ЦРУ использует Nmap, чтобы взломать почтовый сервер газеты и прочитать письма убитого ими журналиста (хорошие ребята)! Снимки экрана всех эпизодических появлений Nmap в кино доступны на странице фильмов Nmap.[44]

• **8 июля 2007 года.** Графическая оболочка Umit совершенствуется и для тестирования включается в выпуск Nmap 4.22SOC1.[45] Позднее Umit переименовывают в Zenmap, а почтенный интерфейс NmapFE удаляют. Zenmap рассматривается в главе 12 «Руководство пользователя графического интерфейса Zenmap».

• **13 декабря 2007 года.** В честь десятилетия Nmap выходит Nmap 4.50![46]

• **1 июня 2008 года.** Выходит Nmap 4.65,[47] впервые включающий исполняемый установщик для Mac OS X. Архив tar с исходным кодом Nmap теперь занимает четыре мегабайта. В выпуск входят 41 сценарий NSE, 1 307 отпечатков ОС и 4 706 сигнатур определения версий.

• **18 августа 2008 года.** Проект Nmap завершает четвёртое участие в Summer of Code с самым высоким в своей истории процентом успеха: шесть из семи получивших поддержку студентов. Они значительно усовершенствовали Zenmap, механизм сценариев Nmap, определение ОС и Ncat, как описано по адресу http://seclists.org/nmap-dev/2008/q4/0193.html.

• **8 сентября 2008 года.** Выходит Nmap 4.75[48] почти со 100 существенными улучшениями по сравнению с 4.68. Среди них - топология сети и объединение результатов сканирования в Zenmap (см. главу 12 «Руководство пользователя графического интерфейса Zenmap»). В выпуск также входят данные о частоте портов из моего проекта Worldscan, который я представил[49] на конференциях Black Hat и Defcon в августе.

Историю Nmap легко изложить, но будущее остаётся неопределённым. Разработка Nmap начиналась без какого-либо грандиозного плана, и большинство вех предыдущей хронологии не планировались более чем на год вперёд. Вместо попыток предсказать далёкое будущее Интернета и сетевых технологий я внимательно изучаю их текущее состояние и решаю, что окажется наиболее полезным для Nmap сейчас и в ближайшем будущем. Поэтому я не представляю, каким будет Nmap через десять лет, хотя ожидаю, что он останется столь же популярным и активно развивающимся, как всегда. Сообщество Nmap достаточно велико, чтобы мы могли направлять Nmap туда, куда потребуется. Nmap и прежде сталкивался с неожиданностями: внезапным удалением поддержки необработанных пакетов в Windows XP SP2, резкими изменениями практик и технологий сетевой фильтрации и медленным распространением IPv6. Каждая из них требовала существенных изменений Nmap, и нам придётся делать то же самое, чтобы принимать будущие изменения сетевых технологий или хотя бы приспосабливаться к ним.

Хотя десятилетний план остаётся неясным, ближайший год предсказать проще. Какими бы захватывающими ни были крупные новые возможности, основное внимание будет уделено не им. Никто из нас не хочет, чтобы Nmap стал раздутым и неорганизованным. Поэтому этот год станет годом консолидации. Системы Zenmap и NSE не настолько зрелы, как остальная часть Nmap, и их совершенствование имеет высокий приоритет. Новые сценарии NSE замечательны тем, что расширяют возможности Nmap без угрозы стабильности, связанной с включением нового исходного кода непосредственно в Nmap. Тем временем Zenmap нуждается в повышении удобства и стабильности, а также в улучшении визуализации результатов. Ещё одним направлением станет веб-сайт Nmap, который будет полезнее и динамичнее. Планируются веб-система обсуждений, демонстрационный сайт Nmap и вики.

Возможности Nmap по сканированию веб-среды также могут расшириться. Когда Nmap только разрабатывался, разные службы часто предоставлялись отдельными демонами, которые определялись по номеру прослушиваемого ими порта. Теперь многие новые службы просто работают поверх HTTP и определяются путём URL, а не номером порта. Поиск известных путей URL во многом подобен сканированию портов, а также сканированию SunRPC, которое Nmap тоже выполняет уже много лет. Nmap уже осуществляет некоторое сканирование веб-среды с помощью механизма сценариев Nmap (см. главу 9 «Механизм сценариев Nmap»), но основная поддержка непосредственно в Nmap была бы быстрее и эффективнее.

Некоторые из самых замечательных прежних возможностей Nmap, такие как определение ОС и сканирование версий, разрабатывались тайно и выпускались неожиданно. В ближайшие годы можно ожидать новых таких возможностей, потому что создавать их очень весело!

Сноски

[1] [назад] http://www.smh.com.au/articles/2003/10/03/1064988378345.html
[2] [назад] В действительности эти IP-адреса зарегистрированы за Юмским испытательным полигоном армии США, где испытывают разнообразную артиллерию, ракеты, танки и другое смертоносное оружие. Вывод прост: будьте чрезвычайно внимательны к тому, кого сканируете, чтобы случайно не затронуть сеть с особо строгим режимом. Результаты сканирования в этой истории на самом деле получены не из данного диапазона IP-адресов.
[3] [назад] Бесклассовая междоменная маршрутизация (Classless Inter-Domain Routing, CIDR) - способ описания сетей с большей точностью, чем обозначения класса A (CIDR `/8`), класса B (CIDR `/16`) или класса C (CIDR `/24`). Прекрасное описание доступно по адресу http://public.pacbell.net/dedicated/cidr.html.
[4] [назад] Возможно, целевой сервер имён зарегистрирует подозрительную серию обратных DNS-запросов от сервера имён Феликса, но большинство организаций даже не хранит такие журналы, не говоря уже об их анализе.
[5] [назад] `stdout` - принятое в языке C обозначение стандартного механизма вывода системы, например терминала Unix `xterm` или командного окна Windows, из которого был запущен Nmap.
[6] [назад] http://www.isc.org/products/BIND/bind-security.html
[7] [назад] http://en.wikipedia.org/wiki/Adrian_Lamo
[8] [назад] http://www.securityfocus.com/news/340
[9] [назад] На самом деле сеанс начала сексуальная нападавшая из предыдущей команды, одетая в кожу. Неясно, в какой момент она погибла и оставила оставшиеся задачи Тринити.
[10] [назад] http://nmap.org/movies.html
[11] [назад] http://www.unspecific.com/nmap/
[12] [назад] http://www.securiteam.com/tools/3T5QMQONFK.html
[13] [назад] Эти изображения взяты с ныне не существующего сайта AmericanSushi.Com.
[14] [назад] http://www.osstmm.org/
[15] [назад] http://www.comcast.net/terms/use.jsp
[16] [назад] http://www4.law.cornell.edu/uscode/18/1030.html
[17] [назад] http://seclists.org/nmap-hackers/2001/0026.html
[18] [назад] http://www.forensicstrategy.com/
[19] [назад] Прекрасная статья адвоката Итана Престона по этой теме доступна по адресу http://grove.ufl.edu/~techlaw/vol6/issue1/preston.html. Он также написал замечательную работу о юридических рисках публикации сведений о безопасности и эксплойтов: http://www.mcandl.com/computer-security.html.
[20] [назад] http://insecure.org/stf/fin.html
[21] [назад] http://www.theregister.co.uk/2004/03/01/mossad_website_hacker_walks_free/
[22] [назад] http://www.beskerming.com/commentary/2007/08/12/249/German_Security_Professionals_in_the_Mist
[23] [назад] http://www.theregister.co.uk/2008/01/02/hacker_tool_ban_guidance/
[24] [назад] http://www.ctv.ca/servlet/ArticleNews/story/CTVNews/1069439746264_64848946/

[25] [назад] http://nmap.org/changelog.html
[26] [назад] http://nmap.org/p51-11.html
[27] [назад] http://www.dhp.com
[28] [назад] http://nmap.org/phrack54-09.txt
[29] [назад] http://seclists.org/nmap-hackers/2000/0140.html
[30] [назад] http://seclists.org/nmap-hackers/2000/0217.html
[31] [назад] http://seclists.org/nmap-hackers/2000/0219.html
[32] [назад] http://seclists.org/nmap-dev/2000/q4/0013.html
[33] [назад] http://insecure.org/stf/Nmap-3.00-Release.html
[34] [назад] http://seclists.org/nmap-dev/2002/q3/0041.html
[35] [назад] http://seclists.org/nmap-hackers/2003/0030.html
[36] [назад] http://insecure.org/stf/Nmap-3.50-Release.html
[37] [назад] http://seclists.org/nmap-hackers/2004/0010.html
[38] [назад] http://code.google.com/soc
[39] [назад] http://seclists.org/nmap-hackers/2005/0012.html
[40] [назад] http://insecure.org/stf/Nmap-4.00-Release.html
[41] [назад] http://seclists.org/nmap-dev/2006/q2/0444.html
[42] [назад] http://seclists.org/nmap-dev/2006/q4/0184.html
[43] [назад] http://seclists.org/nmap-dev/2006/q4/0253.html
[44] [назад] http://nmap.org/movies.html
[45] [назад] http://seclists.org/nmap-dev/2007/q3/0030.html
[46] [назад] http://insecure.org/stf/Nmap-4.50-Release.html
[47] [назад] http://seclists.org/nmap-dev/2008/q2/0558.html
[48] [назад] http://seclists.org/nmap-hackers/2008/0004.html
[49] [назад] http://insecure.org/presentations/

Глава 2. Получение, компиляция, установка и удаление Nmap

2.1. Введение

Nmap часто можно установить или обновить одной командой, поэтому пусть объём этой главы вас не пугает. Большинство читателей с помощью оглавления сразу перейдут к нужным им разделам. В этой главе описывается установка Nmap на множестве платформ, включая как компиляцию исходного кода, так и установку двоичных пакетов. Рассматриваются и сопоставляются графическая версия Nmap и версия для командной строки. На случай, если вы передумаете, приведены также инструкции по удалению Nmap.

2.1.1. Проверка наличия установленного Nmap

Первый шаг к получению Nmap - проверить, нет ли у вас уже установленной версии. Многие дистрибутивы свободных операционных систем, включая большинство систем Linux и BSD, поставляются с пакетами Nmap, хотя по умолчанию они могут быть не установлены. В системах Unix откройте окно терминала и попробуйте выполнить команду `nmap --version`. Если Nmap существует и находится в вашем PATH, вы увидите вывод, похожий на пример 2.1.

Пример 2.1. Проверка наличия Nmap и определение номера версии

```
felix~> nmap --version
Nmap version 4.76 ( http://nmap.org )
felix~>
```

Если Nmap отсутствует в системе или переменная PATH задана неверно, выводится сообщение об ошибке, например `nmap: Command not found`. Как показано выше, в ответ на команду Nmap печатает номер своей версии, в данном случае 4.76.

Даже если копия Nmap уже установлена, следует подумать об обновлении до последней версии, доступной по адресу http://nmap.org/download.html. Новые версии часто работают быстрее, исправляют важные ошибки и содержат обновлённые базы данных определения операционных систем и версий служб. Список изменений по сравнению с уже установленной версией находится по адресу http://nmap.org/changelog.html. Примеры вывода Nmap в этой книге могут не совпадать с выводом старых версий.

2.1.2. Командная строка и графические интерфейсы

Традиционно Nmap был инструментом командной строки, запускаемым из оболочки Unix или, с недавних пор, из командной строки Windows. Это позволяет специалистам быстро выполнять команду, которая делает именно то, что им нужно, не переходя между множеством панелей настройки и разрозненных полей параметров. Кроме того, Nmap легче использовать в сценариях, а полезными командами удобно обмениваться внутри сообщества пользователей.

Один недостаток работы из командной строки состоит в том, что она способна отпугнуть новых и нерегулярных пользователей. Nmap предлагает более сотни параметров командной строки, хотя многие из них представляют собой редко применяемые функции или средства отладки, которые

большинство пользователей может игнорировать. Для тех, кто предпочитает графический интерфейс, создано много графических оболочек. Традиционно в комплект Nmap входил простой графический интерфейс для Unix под названием NmapFE, но в 2007 году его заменил Zenmap, который мы разрабатываем с 2005 года. Zenmap намного мощнее и эффективнее NmapFE, особенно при просмотре результатов. Интерфейс Zenmap со вкладками позволяет искать и сортировать результаты, а также просматривать их несколькими способами: сведения об узлах, необработанный вывод Nmap и порты или узлы. Он работает в Linux, Windows, Mac OS X и на других платформах. Zenmap подробно рассматривается в главе 12 «Руководство пользователя графического интерфейса Zenmap». Остальная часть книги сосредоточена на запуске Nmap из командной строки. Разобравшись в работе параметров командной строки и научившись толковать вывод, легко пользоваться Zenmap или другими доступными графическими интерфейсами Nmap. Параметры Nmap действуют одинаково, независимо от того, выбираете вы их переключателями и меню или вводите в командной строке.

2.1.3. Загрузка Nmap

Nmap.Org является официальным источником для загрузки исходного кода Nmap и двоичных пакетов Nmap и Zenmap. Исходный код распространяется в архивах tar, сжатых bzip2 и gzip, а двоичные пакеты доступны для Linux (формат RPM), Windows (исполняемый установщик NSIS) и Mac OS X (образ диска `.dmg`). Всё это находится по адресу http://nmap.org/download.html.

2.1.4. Проверка целостности загруженных файлов Nmap

При проверке целостности файлов, загруженных из Интернета, паранойя часто оправданна. Популярные пакеты, такие как Sendmail (пример[1]), OpenSSH (пример[2]), tcpdump, Libpcap, BitchX, Fragrouter и многие другие, заражались вредоносными троянскими программами. Успешно взламывались также сайты распространения программного обеспечения Free Software Foundation, Debian и SourceForge. С Nmap такого никогда не случалось, но осторожность нужна всегда. Чтобы проверить подлинность выпуска Nmap, обратитесь к отделённым подписям PGP или криптографическим хешам, включая SHA1 и MD5, опубликованным для выпуска в каталоге подписей Nmap по адресу http://nmap.org/dist/sigs/?C=M&O=D.

Самый безопасный механизм проверки - отделённые подписи PGP. Поскольку ключ подписи никогда не хранится на рабочих серверах, даже успешно взломавший веб-сервер злоумышленник не сможет подделать и правильно подписать троянский выпуск. Проверять подписи PGP умеют многие приложения, но я рекомендую GNU Privacy Guard (GPG).[3]

Выпуски Nmap подписываются специальным ключом подписи проекта Nmap, который можно получить с основных серверов ключей или по адресу http://nmap.org/data/nmap_gpgkeys.txt. В этом файле находится и мой ключ. Ключи импортируются командой `gpg --import nmap_gpgkeys.txt`. Это требуется сделать только один раз, после чего с данной машины можно проверять все будущие выпуски Nmap. Прежде чем доверять ключам, убедитесь, что их отпечатки совпадают со значениями в примере 2.2.

Пример 2.2. Проверка отпечатков ключей PGP Nmap и Фёдора

```
flog~> gpg --fingerprint nmap fyodor
pub 1024D/33599B5F 2005-04-24
    Key fingerprint = BB61 D057 C0D7 DCEF E730 996C 1AF6 EC50 3359 9B5F
uid                  Fyodor <fyodor@insecure.org>
sub 2048g/D3C2241C 2005-04-24

pub 1024D/6B9355D0 2005-04-24
```

```
    Key fingerprint = 436D 66AB 9A79 8425 FDA0 E3F8 01AF 9F03 6B93 55D0
uid                  Nmap Project Signing Key (http://insecure.org/)
sub 2048g/A50A6A94 2005-04-24
```

Для каждого файла загружаемого пакета Nmap, например `nmap-4.76.tar.bz2` и `nmap-4.76-win32.zip`, в каталоге `sigs` находится соответствующий файл, к имени которого добавлено `.gpg.txt`, например `nmap-4.76.tar.bz2.gpg.txt`. Это файл отделённой подписи.

Если в связке ключей имеется правильный ключ PGP и загружен файл отделённой подписи, для проверки выпуска Nmap достаточно одной команды GPG, как показано в примере 2.3. Если файл был изменён, результат будет выглядеть как в примере 2.4.

Пример 2.3. Проверка отпечатков ключей PGP (успешная)

```
flog> gpg --verify nmap-4.76.tar.bz2.gpg.txt nmap-4.76.tar.bz2
gpg: Signature made Fri 12 Sep 2008 02:03:59 AM PDT using DSA key ID 6B9355D0
gpg: Good signature from "Nmap Project Signing Key (http://www.insecure.org/)"
```

Пример 2.4. Обнаружение поддельного файла

```
flog> gpg --verify nmap-4.76.tar.bz2.gpg.txt nmap-4.76-hacked.tar.bz2
gpg: Signature made Fri 12 Sep 2008 02:03:59 AM PDT using DSA key ID 6B9355D0
gpg: BAD signature from "Nmap Project Signing Key (http://www.insecure.org/)"
```

Хотя рекомендуемым способом проверки являются подписи PGP, для менее строгой проверки доступны также хеши SHA1, MD5 и другие. Злоумышленник, способный в реальном времени управлять вашим интернет-трафиком и обладающий исключительно высокой квалификацией, либо взломавший Nmap.Org и заменивший как файл дистрибутива, так и файл контрольных сумм, сможет обойти эту проверку. Тем не менее сверка с эталонными хешами Nmap.Org может быть полезна, если вы получили Nmap от третьей стороны или полагаете, что файл мог быть случайно повреждён. Для каждого файла загружаемого пакета Nmap в каталоге `sigs` находится соответствующий файл, к имени которого добавлено `.digest.txt`, например `nmap-4.76.tar.bz2.digest.txt`. Пример показан в примере 2.5. Это файл отделённой подписи. Хеши из файла контрольных сумм можно проверить распространёнными инструментами, такими как `sha1sum`, `md5sum` или gpg, как показано в примере 2.6 «Проверка хешей Nmap».

Пример 2.5. Типичный файл контрольных сумм выпуска Nmap

```
flog> cat sigs/nmap-4.76.tgz.digest.txt
nmap-4.76.tgz:    MD5    54 B5 C9 E3 F4 4C 1A DD  E1 7D F6 81 70 EB 7C FE
nmap-4.76.tgz:   SHA1    4374 CF9C A882 2C28 5DE9  D00E 8F67 06D0 BCFA A403
nmap-4.76.tgz: RMD160    AE7B 80EF 4CE6 DBAA 6E65  76F9 CA38 4A22 3B89 BD3A
nmap-4.76.tgz: SHA224    524D479E 717D98D0 2FB0A42B 9A4E6E52 4027C9B6 1D843F95
                         D419F87F
nmap-4.76.tgz: SHA256    0E960E05 53EB7647 0C8517A0 038092A3 969DB65C BE23C03F
                         D6DAEF1A CDCC9658
nmap-4.76.tgz: SHA384    D52917FD 9EE6EE62 F5F456BF E245675D B6EEEBC5 0A287B27
                         3CAA4F50 B171DC23 FE7808A8 C5E3A49A 4A78ACBE A5AEED33
nmap-4.76.tgz: SHA512    826CD89F 7930A765 C9FE9B41 1DAFD113 2C883857 2A3A9503
                         E4C1E690 20A37FC8 37564DC3 45FF0C97 EF45ABE6 6CEA49FF
                         E262B403 A52F4ECE C23333A0 48DEDA66
```

Пример 2.6. Проверка хешей Nmap

```
flog> sha1sum nmap-4.76.tgz
4374cf9ca8822c285de9d00e8f6706d0bcfaa403  nmap-4.76.tgz
flog> md5sum nmap-4.76.tgz
54b5c9e3f44c1adde17df68170eb7cfe  nmap-4.76.tgz
flog> gpg --print-md sha1 nmap-4.76.tgz
nmap-4.76.tgz: 4374 CF9C A882 2C28 5DE9  D00E 8F67 06D0 BCFA A403
```

Хотя выпуски Nmap.Org подписываются, как описано в этом разделе, некоторые дополнения Nmap, интерфейсы и двоичные пакеты для отдельных платформ разрабатываются и распространяются другими сторонами. Для подтверждения подлинности загружаемых файлов у них применяются другие механизмы.

2.1.5. Получение Nmap из репозитория Subversion (SVN)

Помимо регулярных стабильных выпусков и выпусков для разработки, новейший исходный код Nmap всегда доступен через систему управления версиями Subversion (SVN).[4] Это позволяет получать новые возможности и обновления баз данных определения версий и ОС сразу по мере их разработки. Недостаток состоит в том, что головные ревизии SVN не всегда настолько стабильны, как официальные выпуски. Поэтому SVN наиболее полезен разработчикам Nmap и пользователям, которым требуется ещё не выпущенное официально исправление.

Право записи в SVN строго ограничено ведущими разработчиками Nmap, но читать репозиторий может каждый. Получите новейший код командой:

```
svn co --username guest --password "" svn://svn.insecure.org/nmap/
```

В дальнейшем исходный код можно обновить, введя `svn up` в рабочем каталоге. Имя пользователя `guest` необходимо из-за ошибки авторизации `svnserve`.

Хотя большинство пользователей следят в SVN только за каталогом `/nmap`, который сам подтягивает `/nbase`, `/nsock` и `/zenmap`, существует ещё один интересный каталог: `/nmap-exp`. В нём находятся экспериментальные ветви Nmap, создаваемые разработчиками для проверки новых идей без нарушения стабильности основной версии Nmap. Когда разработчики считают экспериментальную ветвь готовой к более широкому тестированию, они обычно отправляют её адрес в список рассылки `nmap-dev`.

После получения рабочей копии Nmap его можно собрать из исходного кода точно так же, как из архива tar Nmap, что описано далее в этой главе.

Если вы хотите в реальном времени либо в виде сводок получать по электронной почте уведомления и различия при любых изменениях Nmap, подпишитесь на список рассылки `nmap-svn` по адресу http://cgi.insecure.org/mailman/listinfo/nmap-svn.

2.2. Компиляция и установка из исходного кода в Unix

Хотя для большинства платформ доступны двоичные пакеты, рассматриваемые в следующих разделах, компиляция и установка из исходного кода представляют собой традиционный и наиболее гибкий способ установки Nmap. Так гарантируется доступность новейшей версии, а Nmap может приспособиться к имеющимся в системе библиотекам и структуре каталогов. Например, при наличии криптографических библиотек OpenSSL Nmap использует их для определения версий, однако большинство двоичных пакетов не включает такую возможность. С другой стороны, двоичные пакеты обычно устанавливаются быстрее и проще и позволяют единообразно управлять всем пакетным программным обеспечением системы: устанавливать, удалять, обновлять и так далее.

Установка из исходного кода обычно проходит без затруднений: система сборки спроектирована так, чтобы автоматически определять как можно больше параметров. Для стандартной установки требуются следующие действия:

1. Загрузите новейшую версию Nmap в формате `.tar.bz2` со сжатием bzip2 или `.tgz` со сжатием gzip с адреса http://nmap.org/download.html.

2. Распакуйте загруженный архив tar командой вроде следующей:

```
bzip2 -cd nmap-<VERSION>.tar.bz2 | tar xvf -
```

При использовании GNU tar достаточно более простой команды `tar xvjf nmap-<VERSION>.tar.bz2`. Если загружена версия `.tgz`, замените в команде распаковки `bzip2` на `gzip`.

3. Перейдите в только что созданный каталог: `cd nmap-<VERSION>`.

4. Настройте систему сборки: `./configure`.

Если настройка завершится успешно, появится ASCII-дракон, который поздравит вас и предупредит о необходимости осторожности, как показано в примере 2.7.

Пример 2.7. Экран успешной настройки

```
flog~/nmap> ./configure
checking build system type... x86_64-unknown-linux-gnu
[hundreds of lines cut]
configure: creating ./config.status
config.status: creating Makefile
config.status: creating nsock_config.h
config.status: nsock_config.h is unchanged
   (  )   /\   _                (
    \ |  (  \ ( \.(              )                       _____
  \  \ \  `  `   ) \            (  ___                  / _  \
 (_`    \+   . x  ( .\           \/   \____-----------/ (o)   \_
- .-              \+  ;          (  O                          \____
(__                +- .( -'.- <.   \_____________  `             \  /
(_____             ._._: <_ - <- _-_  VVVVVVV VV V\                \/
  .    /./.+-  . .- /  +--  - .    (--_AAAAAAA__A_/                |
  (__ ' /x  / x _/ (                \_______________//_            \_______
 , x / ( '  . / .  /                                 \___'          \     /
    /  /  _/ /    +                                    |            \   /
   '  (__/                                            /              \/
                                                     /                \
             NMAP IS A POWERFUL TOOL -- USE CAREFULLY AND RESPONSIBLY
Configuration complete. Type make (gmake on some *BSD machines) to compile.
```

5. Соберите Nmap, а также графический интерфейс Zenmap, если выполнены его требования: `make`.

Обратите внимание, что требуется GNU Make. В Unix-системах семейства BSD он часто установлен под именем `gmake`. Поэтому, если `make` выдаёт множество ошибок наподобие `Makefile, line 1: Need an operator`, попробуйте вместо него выполнить `gmake`.

6. Станьте привилегированным пользователем для общесистемной установки: `su root`.

Этот шаг можно пропустить, если у вас есть лишь непривилегированная учётная запись оболочки в системе. В таком случае, вероятно, на четвёртом шаге потребуется передать `configure` параметр `--prefix`, как описано в следующем разделе.

7. Установите Nmap, вспомогательные файлы, документацию и прочее: `make install`.

Поздравляем! Теперь Nmap установлен как `/usr/local/bin/nmap`! Запустите его без аргументов, чтобы увидеть краткую справку.

Как видно выше, простая компиляция и установка из исходного кода сводится почти исключительно к выполнению `./configure; make; make install` от имени `root`. Однако у `configure` есть ряд параметров, влияющих на сборку Nmap.

2.2.1. Директивы configure

Большинством параметров сборки в Unix управляет сценарий `configure`, использованный на четвёртом шаге выше. На сборку Nmap влияют десятки параметров командной строки и переменных окружения. Выполните `./configure --help`, чтобы увидеть огромный список с краткими описаниями. К сборке Nmap в Windows они неприменимы. Ниже перечислены параметры,

относящиеся именно к Nmap или имеющие особую важность.

• `--prefix=<directoryname>` — Этот параметр, стандартный для сценариев `configure` большинства программ, определяет место установки Nmap и его компонентов. По умолчанию префикс имеет значение `/usr/local`: `nmap` устанавливается в `/usr/local/bin`, страница руководства `nmap.1` - в `/usr/local/man/man1`, а файлы данных (`nmap-os-db`, `nmap-services`, `nmap-service-probes` и другие) - в `/usr/local/share/nmap`. Если требуется изменить путь лишь для отдельных компонентов, используйте параметры `--bindir`, `--datadir` и (или) `--mandir`. Например, для установки Nmap в свою учётную запись без привилегий я выполнил бы `./configure --prefix=/home/fyodor`. На этапе установки Nmap создаёт отсутствующие подкаталоги, например `/home/fyodor/man/man1`.

• `--without-zenmap` — Этот параметр запрещает установку графической оболочки Zenmap. Обычно система сборки проверяет наличие в системе таких требований, как язык сценариев Python, и устанавливает Zenmap, если доступны все необходимые компоненты.

• `--with-openssl=<directoryname>` — Система определения версий и механизм сценариев Nmap способны проверять службы, зашифрованные SSL, с помощью свободных библиотек OpenSSL. Обычно система сборки Nmap ищет эти библиотеки в вашей системе и включает такую возможность, если находит их. Если библиотеки находятся в каталоге, который компилятор по умолчанию не просматривает, но вы всё же хотите их использовать, укажите `--with-openssl=<directoryname>`. Тогда Nmap будет искать сами библиотеки OpenSSL в `<directoryname>/libs`, а необходимые заголовочные файлы - в `<directoryname>/include`. Чтобы полностью отключить SSL, укажите `--without-openssl`.

• `--with-libpcap=<directoryname>` — Nmap использует библиотеку Libpcap[5] для захвата необработанных IP-пакетов. Обычно Nmap ищет установленную копию Libpcap в системе и применяет её, если подходят номер версии и платформа. В противном случае Nmap использует собственную свежую копию Libpcap, изменённую для улучшения работы в Linux. Конкретные изменения описаны в файле `libpcap/NMAP_MODIFICATIONS` каталога исходного кода Nmap. Из-за этих изменений для Linux Nmap по умолчанию всегда использует собственную Libpcap. Чтобы принудительно связать Nmap с вашей Libpcap, передайте `configure` параметр `--with-libpcap=<directoryname>`. Тогда Nmap ожидает библиотеку Libpcap в `<directoryname>/lib/libpcap.a`, а включаемые файлы - в `<directoryname>/include`. Если указать `--with-libpcap=included`, Nmap всегда будет использовать версию Libpcap из своего архива tar.

• `--with-libpcre=<directoryname>` — PCRE - библиотека совместимых с Perl регулярных выражений, доступная по адресу http://www.pcre.org. Обычно Nmap ищет её копию в системе, а при неудаче использует собственную. Если библиотека PCRE отсутствует в стандартном пути поиска компилятора, Nmap, вероятно, её не найдёт. В таком случае сообщите Nmap её местоположение параметром `--with-libpcre=<directoryname>` для `configure`. Тогда Nmap ожидает файлы библиотеки в `<directoryname>/lib`, а включаемые файлы - в `<directoryname>/include`. Иногда предпочтительно использовать библиотеки PCRE, входящие в Nmap, вместо уже установленных в системе. В таком случае укажите `--with-libpcre=included`.

• `--with-libdnet=<directoryname>` — Libdnet - превосходная сетевая библиотека, которую Nmap использует для отправки необработанных кадров Ethernet. Версия в дереве Nmap значительно изменена, особенно код Windows, поэтому по умолчанию применяется включённая версия. Чтобы вместо неё использовать версию, уже установленную в системе, укажите `--with-libdnet=<directoryname>`. Тогда Nmap ожидает файлы библиотеки в `<directoryname>/lib`, а включаемые файлы - в `<directoryname>/include`.

• `--with-localdirs` — Этот простой параметр предписывает Nmap искать важные библиотеки и заголовочные файлы в `/usr/local/lib` и `/usr/local/include`. Он никогда не должен быть

необходим, однако некоторые помещают такие библиотеки в `/usr/local`, не настроив компилятор на их поиск. Если вы относитесь к их числу, используйте этот параметр.

2.2.2. При возникновении проблем с компиляцией

В идеальном мире программное обеспечение всегда компилировалось бы безупречно и быстро в любой системе. К сожалению, общество ещё не достигло такого состояния нирваны. Несмотря на все наши усилия сделать Nmap переносимым, при компиляции иногда возникают проблемы. Ниже приведены советы на случай, если не удаётся скомпилировать дистрибутив исходного кода.

Обновитесь до последней версии Nmap

Проверьте по адресу http://nmap.org/download.html, что используете новейшую версию Nmap. Возможно, проблема уже исправлена.

Внимательно прочитайте сообщение об ошибке

Прокрутите экран вывода вверх и изучите сообщения об ошибках, появившиеся при сбое команд. Часто лучше всего найти первое сообщение, поскольку оно нередко вызывает каскад последующих ошибок. Прочитайте его внимательно: оно может указывать на системную проблему, например нехватку места на диске или неработающий компилятор. Пользователи с навыками программирования способны самостоятельно устранить более широкий круг проблем. Если для исправления вы изменили код, отправьте исправление, созданное командой `diff -uw <oldfile> <newfile>`, и сведения о проблеме и платформе в `nmap-dev`, как описано в разделе 15.17 «Ошибки». Включение изменения в основной дистрибутив Nmap принесёт пользу многим другим пользователям и избавит вас от необходимости повторять его для каждой новой версии Nmap.

Обратитесь к Google и другим интернет-ресурсам

Попробуйте найти точный текст сообщения об ошибке в Google или других поисковых системах. Возможно, стоит также просмотреть недавнюю активность в списке разработчиков Nmap (`nmap-dev`): архивы и интерфейс поиска доступны по адресу http://seclists.org.

Спросите в nmap-dev

Если исследования не привели к решению, попробуйте отправить отчёт в список рассылки разработчиков Nmap (`nmap-dev`), как описано в разделе 15.17 «Ошибки».

Рассмотрите двоичные пакеты

Двоичные пакеты Nmap доступны на большинстве платформ и обычно легко устанавливаются. Недостатки состоят в том, что они могут быть не самыми свежими и лишают вас части гибкости самостоятельной компиляции. Далее в этой главе описывается, как найти двоичные пакеты для многих платформ, а ещё больше можно обнаружить поиском в Интернете. Разумеется, устанавливать двоичные пакеты следует только из заслуживающих доверия источников.

2.3. Дистрибутивы Linux

Linux является самой популярной платформой для запуска Nmap. В одном опросе пользователей 86% ответили, что Linux входит по меньшей мере в число платформ, на которых они запускают Nmap. Первый выпуск Nmap 1997 года работал только в Linux.

Пользователи Linux могут выбрать установку из исходного кода или двоичные пакеты, предоставленные их дистрибутивом либо Insecure.Org. Двоичные пакеты обычно устанавливаются

быстрее и проще и часто немного приспособлены к стандартным путям каталогов дистрибутива и другим его особенностям. Эти пакеты также обеспечивают единообразное управление обновлением, удалением и учётом программного обеспечения в системе. Недостаток состоит в том, что пакеты, создаваемые дистрибутивами, неизбежно отстают от выпусков исходного кода Nmap.Org. Большинство дистрибутивов Linux, особенно Debian и Gentoo, поддерживают пакет Nmap сравнительно свежим, однако в некоторых он сильно устарел. Установка исходного кода даёт больше свободы при выборе способа сборки и оптимизации Nmap для вашей системы. Сборка Nmap из исходного кода описана в разделе 2.2 «Компиляция и установка из исходного кода в Unix». Ниже приведены простые инструкции по работе с пакетами для самых распространённых дистрибутивов.

2.3.1. Дистрибутивы на основе RPM (Red Hat, Mandrake, SUSE, Fedora)

Для каждого выпуска Nmap я собираю пакеты RPM и публикую их на странице загрузки Nmap по адресу http://nmap.org/download.html. Я создаю два пакета: `nmap` содержит только исполняемый файл командной строки и файлы данных, а `zenmap` - необязательную графическую оболочку Zenmap (см. главу 12 «Руководство пользователя графического интерфейса Zenmap»). Перед `zenmap` необходимо установить пакет `nmap`. Один из недостатков установки RPM вместо компиляции из исходного кода заключается в том, что RPM не поддерживают OpenSSL для определения версий и проверки служб SSL механизмом сценариев Nmap.

Установка через RPM очень проста: если передать правильные URL, он даже сам загрузит пакет. В следующем примере загружается и устанавливается Nmap 4.68 вместе с графической оболочкой. Разумеется, вместо него следует использовать новейшую версию с указанного выше сайта загрузки. Все уже установленные через RPM версии обновляются. Процесс показан в примере 2.8.

Пример 2.8. Установка Nmap из двоичных RPM

```
# rpm -vhU http://nmap.org/dist/nmap-4.68-1.i386.rpm
Retrieving http://nmap.org/dist/nmap-4.68-1.i386.rpm
Preparing...                ########################################### [100%]
   1:nmap                   ########################################### [100%]
# rpm -vhU http://nmap.org/dist/zenmap-4.68-1.noarch.rpm
Retrieving http://nmap.org/dist/zenmap-4.68-1.noarch.rpm
Preparing...                ########################################### [100%]
   1:zenmap                 ########################################### [100%]
```

Как следует из имён файлов, эти двоичные RPM созданы для обычных ПК с архитектурой x86. Я также распространяю двоичные файлы x86_64 для пользователей 64-разрядной Linux. Эти двоичные файлы не будут работать у сравнительно немногих пользователей Linux на других платформах, таких как SPARC, Alpha или PowerPC. Кроме того, они могут отказаться устанавливаться, если версии ваших библиотек существенно отличаются от тех, с которыми RPM первоначально собирались. В таких случаях можно найти двоичные RPM, подготовленные поставщиком Linux именно для вашего дистрибутива. Хорошей отправной точкой служат исходные установочные CD или DVD. К сожалению, они могут быть устаревшими или недоступными. Другой вариант - установить Nmap из исходного кода, как описано выше, хотя при этом теряются преимущества единообразного сопровождения двоичных пакетов. Третий вариант - собрать и установить собственные двоичные RPM из исходных RPM, распространяемых на указанной выше странице загрузки. В примере 2.9 этот способ показан для Nmap 4.68.

Пример 2.9. Сборка и установка Nmap из исходных RPM

```
> rpmbuild --rebuild http://nmap.org/dist/nmap-4.68-1.src.rpm
[ hundreds of lines cut ]
Wrote: /home/fyodor/rpmdir/RPMS/i386/nmap-4.68-1.i386.rpm
[ cut ]
> su
```

```
Password:
# rpm -vhU /home/fyodor/rpmdir/RPMS/i386/nmap-4.68-1.i386.rpm
Preparing...                ########################################### [100%]
   1:nmap                   ########################################### [100%]
#
```

Пересобирать Zenmap таким способом не требуется, поскольку RPM Zenmap не зависит от архитектуры (`noarch`). По этой причине исходных RPM Zenmap не существует.

Удалить пакеты RPM так же просто: `rpm -e nmap zenmap`.

2.3.2. Обновление Red Hat, Fedora, Mandrake и Yellow Dog Linux с помощью Yum

В дистрибутивах Red Hat, Fedora, Mandrake и Yellow Dog Linux есть приложение Yum, управляющее установкой и обновлением программ из центральных репозиториев RPM. Оно делает установку и обновление программ элементарными. Поскольку обычно используются репозитории Yum конкретного дистрибутива, можно быть уверенным, что программы уже проверены на совместимость именно с ним. Большинство дистрибутивов содержит Nmap в своём репозитории Yum, но не всегда своевременно его обновляет. Это особенно неприятно, если вы, как и большинство людей, не всегда быстро переходите на последний выпуск дистрибутива. В двухлетнем выпуске Linux Yum часто предоставит двухлетнюю версию Nmap. Даже новейшим версиям дистрибутивов нередко требуются месяцы, чтобы перейти на новый выпуск Nmap. Поэтому для получения последней версии Nmap в этих системах попробуйте распространяемые нами RPM, как описано в предыдущем разделе. Но если наши RPM несовместимы с вашей системой или вы очень спешите, для установки Nmap из Yum обычно достаточно выполнить `yum install nmap` либо `yum install nmap zenmap`, если нужен и графический интерфейс, хотя некоторые дистрибутивы пока не упаковывают Zenmap. Yum самостоятельно связывается с репозиторием в Интернете, находит подходящий пакет для вашей архитектуры и устанавливает его со всеми необходимыми зависимостями. Сокращённый вывод показан в примере 2.10. Позднее можно выполнить `yum update`, чтобы установить доступные обновления Nmap и других пакетов репозитория.

Пример 2.10. Установка Nmap из системного репозитория Yum

```
flog~#yum install nmap
Setting up Install Process
Parsing package install arguments
Resolving Dependencies
--> Running transaction check
---> Package nmap.x86_64 2:4.52-1.fc8 set to be updated
--> Finished Dependency Resolution
Dependencies Resolved

 Package                 Arch        Version          Repository          Size

Installing:
 nmap                    x86_64      2:4.52-1.fc8     updates             1.0 M

Transaction Summary

Install       1 Package(s)
Update        0 Package(s)
Remove        0 Package(s)

Total download size: 1.0 M
Is this ok [y/N]: y
Downloading Packages:
(1/1): nmap-4.52-1.fc8.x8 100% |=========================| 1.0 MB     00:02
Running Transaction Test
Transaction Test Succeeded
```

```
Running Transaction
  Installing: nmap                         ########################## [1/1]

Installed: nmap.x86_64 2:4.52-1.fc8
Complete!
```

2.3.3. Debian Linux и производные дистрибутивы, такие как Ubuntu

Ламонт Джонс превосходно сопровождает пакеты Nmap `.deb`, в том числе поддерживает их достаточно свежими. Правильная команда обновления или установки - `apt-get install nmap`. Она работает и в производных от Debian дистрибутивах, таких как Ubuntu. Сведения о последнем пакете Nmap для стабильной версии Debian доступны по адресу http://packages.debian.org/stable/nmap, а пакеты Nmap и Zenmap для разрабатываемой, то есть нестабильной, версии - по адресам http://packages.debian.org/unstable/nmap и http://packages.debian.org/unstable/zenmap.

2.3.4. Другие дистрибутивы Linux

Дистрибутивов Linux слишком много, чтобы перечислить их здесь, но даже многие малоизвестные содержат Nmap в дереве пакетов. Если его там нет, можно просто скомпилировать исходный код, как описано в разделе 2.2 «Компиляция и установка из исходного кода в Unix».

2.4. Windows

Когда-то Nmap был инструментом исключительно для Unix, но в 2000 году вышла версия для Windows, которая с тех пор стала второй по популярности платформой Nmap после Linux. Из-за этой популярности и отсутствия компилятора у многих пользователей Windows для каждого крупного выпуска Nmap распространяются двоичные исполняемые файлы. Хотя версия значительно усовершенствовалась, перенос на Windows всё ещё не настолько эффективен и стабилен, как Unix-версия. Известны следующие ограничения:

• Обычно нельзя просканировать собственную машину с неё самой, используя IP-адрес обратной петли вроде `127.0.0.1` или любой из её зарегистрированных IP-адресов. Это ограничение Windows, которое нам пока не удалось обойти. Если это действительно необходимо, используйте TCP-сканирование с установлением соединения без ping-проверки (`-sT -PN`), поскольку оно применяет высокоуровневый API сокетов, а не отправляет необработанные пакеты.
• Для сканирования необработанными пакетами Nmap поддерживает только интерфейсы Ethernet, включая большинство беспроводных карт 802.11 и многие клиенты VPN. Если не использовать параметры `-sT -PN`, соединения RAS, например коммутируемые PPP, и некоторые клиенты VPN не поддерживаются. Эта поддержка была прекращена, когда Microsoft удалила поддержку необработанных сокетов TCP/IP в Windows XP SP2. Теперь Nmap вместо них приходится отправлять кадры Ethernet более низкого уровня.

Скорость сканирования в Windows в целом сопоставима с Unix, хотя последняя часто имеет небольшое преимущество. Исключение составляет сканирование с установлением соединения (`-sT`), которое в Windows нередко намного медленнее из-за недостатков сетевого API Windows. Это досадно, поскольку именно такое сканирование TCP работает против локального узла и через все типы сетей, а не только Ethernet, как сканирование необработанными пакетами. Производительность сканирования с установлением соединения можно существенно повысить,

применив изменения реестра из входящего в комплект Nmap файла `nmap_performance.reg`. По умолчанию исполняемый установщик Nmap применяет эти изменения за вас. Этот файл реестра находится в каталоге `nmap-<version>` двоичного ZIP-файла Windows и в `nmap-<version>/mswin32` архива tar с исходным кодом, где `<version>` - номер конкретного выпуска. Изменения увеличивают число эфемерных портов, зарезервированных для пользовательских приложений, таких как Nmap, и сокращают задержку до повторного использования закрытого соединения. Большинство пользователей просто устанавливает флажок применения этих изменений в исполняемом установщике Nmap, но их можно применить также двойным щелчком по `nmap_performance.reg` или командой `regedit32 nmap_performance.reg`. Для ручного внесения изменений добавьте следующие три значения DWORD реестра в `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters`:

• `MaxUserPort` — Задайте большое значение, например 65534 (`0x0000fffe`). См. статью Microsoft KB Q196271.[6]

• `TCPTimedWaitDelay` — Задайте минимальное значение (`0x0000001e`). См. статью Microsoft KB Q149532.[7]

• `StrictTimeWaitSeqCheck` — Задайте значение 1, чтобы проверялся `TCPTimedWaitDelay`.

> **Примечание.** Я хотел бы поблагодарить Райана Пермеха из eEye, Энди Лютомирски и Йенса Фогта за их напряжённую работу над переносом Nmap в Windows. Много лет Nmap был инструментом исключительно для Unix и, вероятно, оставался бы им до сих пор, если бы не их усилия.

Пользователи Windows могут установить Nmap тремя способами; все необходимые файлы доступны на странице загрузки http://nmap.org/download.html.

2.4.1. Зависимости Windows 2000

Nmap поддерживает Windows 2000, но сначала необходимо установить две зависимости Microsoft: Windows Installer 3.1 (v2)[8] и обновление безопасности для Windows 2000 (KB835732).[9] После этого установите Nmap по общим инструкциям из следующих двух разделов.

2.4.2. Самоустанавливающийся пакет Windows

В каждый выпуск Nmap входит самоустанавливающийся пакет Windows под названием `nmap-<version>-setup.exe`, где `<version>` - номер конкретного выпуска. Большинство пользователей Nmap выбирают этот вариант из-за его простоты. Ещё одно преимущество самоустанавливающегося пакета - возможность установить графический интерфейс Zenmap. Просто запустите файл установки и следуйте панелям выбора пути установки и установки WinPcap. Установщик создан с помощью системы Nullsoft Scriptable Install System с открытым исходным кодом.[10] По завершении прочитайте раздел 2.4.5 «Выполнение Nmap в Windows», где описан запуск Nmap из командной строки и через Zenmap.

2.4.3. Двоичные ZIP-файлы для командной строки

> **Примечание.** Большинство пользователей предпочитает устанавливать Nmap с помощью рассмотренного выше самоустанавливающегося пакета.

Каждый стабильный выпуск Nmap поставляется с двоичными файлами командной строки Windows и сопутствующими файлами в ZIP-архиве. Графический интерфейс не включён, поэтому `nmap.exe` необходимо запускать из окна DOS или командной строки. Можно также загрузить и установить более совершенную командную оболочку, например входящую в свободную систему Cygwin, доступную по адресу http://www.cygwin.com. Ниже приведены пошаговые инструкции по установке и выполнению двоичных файлов Nmap из `.zip`.

Установка двоичных ZIP-файлов Nmap

1. Загрузите двоичные `.zip`-файлы с адреса http://nmap.org/download.html.
2. Распакуйте ZIP-файл в каталог, где должен находиться Nmap, например `C:\Program Files`. Должен появиться каталог `nmap-<version>`, содержащий исполняемый файл Nmap и файлы данных. Microsoft Windows XP и Vista умеют извлекать ZIP-файлы: просто щёлкните файл правой кнопкой мыши в Проводнике. Если у вас нет программы для распаковки ZIP, такая программа под названием `unzip` входит в описанный выше Cygwin; кроме того, можно загрузить свободную утилиту 7-Zip с открытым исходным кодом.[11] Коммерческие альтернативы - WinZip[12] и PKZIP.[13]
3. Для повышения производительности примените рассмотренные выше изменения реестра Nmap.
4. Nmap требует свободную библиотеку захвата пакетов WinPcap. Мы собираем собственный установщик WinPcap, находящийся в ZIP-файле под именем `winpcap-nmap-<version>.exe`, где `<version>` обозначает версию Nmap, а не WinPcap. В качестве альтернативы можно получить и установить последнюю версию с адреса http://www.winpcap.org. Необходимо установить версию 4.0 или более новую.
5. Из-за способа компиляции Nmap ему требуется пакет компонентов времени выполнения Microsoft Visual C++ 2008 Redistributable Package. Во многих системах он уже установлен вместе с другими пакетами, но на всякий случай следует запустить `vcredist_x86.exe` из ZIP-файла.
6. Инструкции по выполнению скомпилированного Nmap приведены в разделе 2.4.5 «Выполнение Nmap в Windows».

2.4.4. Компиляция из исходного кода

Большинство пользователей Windows предпочитает двоичный самоустанавливающийся пакет Nmap, однако можно выполнить компиляцию из исходного кода, особенно если вы собираетесь помогать в разработке Nmap. Для компиляции требуется Microsoft Visual C++ 2008, входящий в коммерческий комплект Visual Studio. Подойдёт любое издание Visual Studio, включая бесплатное Visual C++ 2008 Express.[14]

Компиляция Nmap из исходного кода в Windows

1. Загрузите новейший дистрибутив исходного кода Nmap с адреса http://nmap.org/download.html. Его имя имеет вид `nmap-<version>.tar.bz2` или `nmap-<version>.tgz`. Это один и тот же файл tar, сжатый соответственно gzip или bzip2. Версия, сжатая bzip2, меньше.
2. Распакуйте только что загруженный файл исходного кода. Последние выпуски свободного дистрибутива Cygwin[15] умеют работать и с `.tar.bz2`, и с `.tgz`. Используйте соответственно команду `tar xvjf nmap-<version>.tar.bz2` или `tar xvzf nmap-<version>.tgz`. В качестве альтернативы распространённое приложение WinZip может распаковать версию `.tgz`.
3. Откройте Visual Studio и файл решения Nmap (`nmap-<version>/mswin32/nmap.sln`).
4. Выберите **Build Solution** в меню **Build**. Должна начаться компиляция Nmap и завершиться строкой `-- Done --`, сообщающей, что все проекты успешно собраны и не было ни одного сбоя.
5. Исполняемый файл и файлы данных находятся в `nmap-<version>/mswin32/Release/`. Их можно скопировать в предпочтительный каталог при условии, что все они останутся вместе.

6. Убедитесь, что установлен WinPcap. Его можно получить, установив наш двоичный самоустанавливающийся пакет или выполнив `winpcap-nmap-<version>.exe` из нашего ZIP-пакета.

В качестве альтернативы официальный установщик доступен по адресу http://www.winpcap.org.
7. Инструкции по выполнению скомпилированного Nmap приведены в следующем разделе.

Многие спрашивали, можно ли скомпилировать Nmap с помощью `gcc` и `g++`, входящих в Cygwin, либо других компиляторов. Некоторые пользователи сообщали об успехе, но мы не сопровождаем инструкции по сборке Nmap в Cygwin.

2.4.5. Выполнение Nmap в Windows

Теперь выпуски Nmap включают графический интерфейс Zenmap. Если вы использовали установщик Nmap и оставили поле Zenmap отмеченным, на рабочем столе и в меню «Пуск» должен появиться новый пункт Zenmap. Чтобы начать работу, щёлкните его. Полная документация Zenmap приведена в главе 12 «Руководство пользователя графического интерфейса Zenmap». Многим пользователям нравится Zenmap, но другие предпочитают традиционный запуск Nmap из командной строки. Ниже приведены подробные инструкции для тех, кто не знаком с интерфейсами командной строки.

1. Убедитесь, что вошедший в систему пользователь имеет на компьютере административные привилегии, то есть состоит в группе администраторов.

2. Откройте окно командной строки или DOS. Его можно найти в дереве меню программ, но проще всего выбрать **Пуск** -> **Выполнить** и ввести `cmd<enter>`. Можно также открыть окно Cygwin, если он установлен, щёлкнув значок Cygwin на рабочем столе, хотя необходимые команды немного отличаются от приведённых здесь.

3. Перейдите в каталог установки Nmap. Если использован стандартный путь, введите следующие команды:

```
c:
cd "\Program Files\Nmap"
```

4. Выполните `nmap.exe`. На рисунке 2.1 показан простой пример.

Рисунок 2.1. Выполнение Nmap из командной оболочки Windows

Если вы часто выполняете Nmap, каталог Nmap, по умолчанию `C:\Program Files\Nmap`, можно добавить в путь выполнения команд. Точное место настройки зависит от платформы Windows. На своей машине с Windows XP я выполняю следующие действия:

1. На рабочем столе щёлкните правой кнопкой мыши **Мой компьютер**, затем выберите **Свойства**.
2. В окне **Свойства системы** откройте вкладку **Дополнительно**.
3. Нажмите кнопку **Переменные среды**.

4. Выберите `Path` в разделе системных переменных и нажмите **Изменить**.
5. Добавьте в конец значения точку с запятой, а затем каталог Nmap, по умолчанию `C:\Program Files\Nmap`.
6. Откройте новое окно DOS; теперь из любого каталога должна выполняться команда вроде `nmap scanme.nmap.org`.

2.5. Sun Solaris

Solaris уже давно хорошо поддерживается Nmap. Компания Sun даже пожертвовала проекту полноценную SPARCstation, которая до сих пор используется для проверки новых сборок Nmap. Поэтому многие пользователи Solaris компилируют и устанавливают программу из исходного кода, как описано в разделе 2.2 «Компиляция и установка из исходного кода в Unix».

Пользователям, предпочитающим собственные пакеты Solaris, будет приятно узнать, что Стивен Кристенсен превосходно сопровождает пакеты Nmap для всех современных версий и архитектур Solaris по адресу http://www.sunfreeware.com. Инструкции находятся на его сайте и обычно очень просты: загрузите подходящий пакет Nmap для своей версии Solaris, распакуйте его и выполните `pkgadd -d <packagename>`. Как и обычно для предоставленных сторонними авторами двоичных пакетов, эти пакеты Solaris устанавливаются просто и быстро. Преимущества компиляции из исходного кода состоят в возможности получить более новую версию и большей свободе управления процессом сборки.

2.6. Apple Mac OS X

Благодаря нескольким людям, любезно предоставившим учётные записи оболочки на своих машинах с Mac OS X, Nmap обычно компилируется на этой платформе без проблем. Поскольку не у всех есть инструменты разработки, необходимые для компиляции из исходного кода, доступен также исполняемый установщик. Кроме того, Nmap можно получить через такие системы упаковки программ Unix для Mac OS X, как MacPorts и Fink.

2.6.1. Исполняемый установщик

Проще всего установить Nmap и Zenmap в Mac OS X с помощью нашего установщика. В разделе Mac OS X страницы загрузки Nmap[16] находится файл `nmap-<version>.dmg`, где `<version>` - номер самого последнего выпуска. Файл `dmg` называется «образом диска». Ниже приведены инструкции по установке.

1. Загрузите файл `nmap-<version>.dmg`. Откройте его двойным щелчком по значку. В зависимости от способа загрузки он может открыться автоматически.
2. Появится содержимое образа диска. Среди файлов будет метапакет Mac под названием `nmap-<version>.mpkg`. Дважды щёлкните его, чтобы запустить установщик.
3. Следуйте инструкциям установщика. У вас запросят пароль, поскольку Nmap устанавливается в системный каталог.
4. По завершении установки извлеките образ диска, щёлкнув его значок с нажатой клавишей Control и выбрав **Eject**. Теперь образ диска можно переместить в корзину.

Справку по запуску установленных Nmap и Zenmap см. в разделе 2.6.4 «Выполнение Nmap в Mac OS X».

Установленные программы представляют собой универсальные двоичные файлы, работающие в Mac OS X 10.4 (Tiger) и более новых версиях. Пользователям более ранних версий придётся компилировать исходный код или применять сторонний пакет.

2.6.2. Компиляция из исходного кода

После подготовки надлежащей среды сборки компиляция Nmap из исходного кода в Mac OS X не сложнее, чем на других платформах.

Компиляция Nmap из исходного кода

Для компиляции Nmap в Mac OS X требуется Xcode[17] - инструменты разработчика Apple, включающие GCC и остальную часть обычной системы сборки. По умолчанию Xcode не устанавливается, но доступен как необязательный компонент на установочных дисках Mac OS X. Если установочных дисков нет или нужна более новая версия, Xcode можно бесплатно загрузить, выполнив следующие действия.

1. Apple разрешает загрузку Xcode только участникам Apple Developer Connection. Перейдите по адресу http://connect.apple.com и заполните формы для создания учётной записи. Если учётная запись уже есть, переходите к следующему шагу.
2. Вернитесь на http://connect.apple.com и войдите со своими учётными данными.
3. Нажмите ссылку **Download**, затем выберите **Developer Tools**.
4. Загрузите и установите новейший Xcode.

Конкретные шаги могут измениться, но будем надеяться, что общий подход продолжит работать.

После установки Xcode выполните инструкции по компиляции из раздела 2.2 «Компиляция и установка из исходного кода в Unix». Обратите внимание: в некоторых старых версиях Mac OS X команду `./configure` может потребоваться заменить на `./configure CPP=/usr/bin/cpp`.

Компиляция Zenmap из исходного кода

Zenmap зависит от нескольких внешних библиотек, отсутствующих в Mac OS X, включая GTK+ и PyGTK. У этих библиотек, в свою очередь, много собственных зависимостей. Удобный способ установить их все - воспользоваться сторонней системой пакетов, описанной в разделе 2.6.3. После установки зависимостей установите Zenmap обычным способом по инструкциям раздела 2.2 «Компиляция и установка из исходного кода в Unix».

2.6.3. Сторонние пакеты

Другой вариант установки Nmap - использовать систему упаковки программ Unix для Mac OS X. Здесь рассматриваются Fink[18] и MacPorts.[19] Инструкции по установке диспетчеров пакетов находятся на сайтах соответствующих проектов.

Для установки через Fink выполните `fink install nmap`. Nmap будет установлен как `/sw/bin/nmap`. Для удаления используйте `fink remove nmap`.

Для установки через MacPorts выполните `sudo port install nmap`. Nmap будет установлен как `/opt/local/bin/nmap`. Для удаления выполните `sudo port uninstall nmap`.

Эти системы устанавливают исполняемый файл `nmap` вне глобального `PATH`. Чтобы Zenmap мог его найти, задайте для переменной `nmap_command_path` в `zenmap.conf` значение `/sw/bin/nmap` или `/opt/local/bin/nmap`, как описано в разделе 12.10.1 «Исполняемый файл nmap».

2.6.4. Выполнение Nmap в Mac OS X

Эмулятор терминала в Mac OS X называется Terminal и находится в каталоге `/Applications/Utilities`. Откройте его, и появится окно терминала. В нём вы будете вводить команды.

По умолчанию пользователь `root` в Mac OS X отключён. Чтобы выполнить сканирование с правами `root`, поставьте перед именем команды `sudo`, например `sudo nmap -sS <target>`. Будет запрошен пароль - это ваш обычный пароль входа. Сделать это могут только пользователи с административными привилегиями.

Для Zenmap требуется приложение X11. Если оно не было установлено по умолчанию, то может быть доступно как необязательный компонент на установочных дисках Mac OS X.

При запуске Zenmap появляется диалоговое окно с просьбой ввести пароль. Пользователи с административными привилегиями могут указать пароль, разрешив Zenmap работать от имени `root` и выполнять более сложные сканирования. Чтобы запустить Zenmap в непривилегированном режиме, нажмите кнопку **Cancel** в этом диалоговом окне проверки подлинности.

2.7. FreeBSD / OpenBSD / NetBSD

Разновидности BSD хорошо поддерживаются Nmap, поэтому программу можно просто скомпилировать из исходного кода, как описано в разделе 2.2 «Компиляция и установка из исходного кода в Unix». Это даёт обычные преимущества: всегда новейшую версию и гибкий процесс сборки. Если вы предпочитаете двоичные пакеты, каждая из этих разновидностей BSD сопровождает собственные пакеты Nmap. Во многих системах BSD есть также дерево портов, стандартизирующее компиляцию популярных приложений. Ниже приведены инструкции по установке Nmap в самых распространённых разновидностях BSD.

2.7.1. Инструкции по двоичным пакетам и портам исходного кода OpenBSD

Согласно FAQ OpenBSD,[20] пользователям «НАСТОЯТЕЛЬНО рекомендуется применять пакеты вместо сборки приложения из портов. Команда портов OpenBSD считает целью своей работы пакеты, а не сами порты». В том же FAQ содержатся подробные инструкции для каждого способа. Ниже приведена сводка.

Установка с использованием двоичных пакетов

1. Выберите зеркало на странице http://www.openbsd.org/ftp.html, затем подключитесь к нему по FTP и получите пакет Nmap из `/pub/OpenBSD/<version>/packages/<platform>/nmap-<version>.tgz`. Либо возьмите его с компакт-диска дистрибутива OpenBSD.
2. От имени `root` выполните `pkg_add -v nmap-<version>.tgz`.

Установка с использованием дерева портов исходного кода

1. Если у вас ещё нет копии дерева портов, получите её через CVS по инструкциям на странице http://openbsd.org/faq/faq15.html.

2. От имени `root` выполните следующую команду, заменив `/usr/ports` путём к своему каталогу портов, если он отличается:

```
cd /usr/ports/net/nmap && make install clean
```

2.7.2. Инструкции по двоичному пакету и портам исходного кода FreeBSD

В руководстве проекта FreeBSD процессам установки пакетов и портов посвящена целая глава.[21] Ниже приведена краткая сводка.

Установка двоичного пакета

Проще всего установить двоичный пакет Nmap командой `pkg_add -r nmap`. Затем, если нужна оболочка для X Window, можно выполнить ту же команду с аргументом `zenmap`. Если вы предпочитаете получить пакет вручную, загрузите его с http://freshports.org/security/nmap и http://freshports.org/security/zenmap либо с CD-ROM, а затем выполните `pkg_add <packagename.tgz>`.

Установка с использованием дерева портов исходного кода

1. Дерево портов часто устанавливается вместе с самой системой, обычно в `/usr/ports`. Если его ещё нет, конкретные инструкции по установке приведены в упомянутой выше главе руководства FreeBSD.

2. От имени `root` выполните следующую команду, заменив `/usr/ports` путём к своему каталогу портов, если он отличается:

```
cd /usr/ports/security/nmap && make install clean
```

2.7.3. Инструкции по двоичным пакетам NetBSD

NetBSD упаковала Nmap для огромного числа платформ: от обычной i386 до PlayStation 2, PowerPC, VAX, SPARC, MIPS, Amiga, ARM и нескольких платформ, о которых даже я никогда не слышал! К сожалению, эти пакеты не слишком свежие. Список пакетов Nmap для NetBSD доступен по адресу ftp://ftp.netbsd.org/pub/NetBSD/packages/pkgsrc/net/nmap/README.html, а описание установки приложений с помощью системы пакетов - по адресу http://netbsd.org/Documentation/pkgsrc/using.html.

2.8. Amiga, HP-UX, IRIX и другие платформы

Одно из чудес разработки открытого исходного кода состоит в том, что ресурсы часто направляются на то, что люди считают увлекательным, а не исключительно на прибыль, как в большинстве корпораций. Именно так появился перенос на Amiga. Диего Касорран выполнил большую часть работы и прислал аккуратное исправление, включённое в основной дистрибутив Nmap. Как правило, пользователям AmigaOS достаточно следовать инструкциям по компиляции исходного кода из раздела 2.2 «Компиляция и установка из исходного кода в Unix». В некоторых системах могут встретиться препятствия, но, полагаю, для поклонников Amiga это часть удовольствия.

Nmap поддерживает множество проприетарных разновидностей Unix, таких как HP-UX и SGI IRIX. Проект Nmap зависит от помощи сообщества пользователей в поддержании надлежащей работы в этих системах. При возникновении проблем попробуйте отправить отчёт со всеми подробностями в список рассылки `nmap-dev`, как описано в разделе 15.17 «Ошибки». Если вы разработаете исправление, улучшающее поддержку вашей платформы, тоже сообщите нам, чтобы мы могли включить его в Nmap.

2.9. Удаление Nmap

Если вы удаляете Nmap лишь ради обновления до последней версии, обычно можно воспользоваться функцией обновления, которую предоставляют большинство диспетчеров двоичных пакетов. Аналогично, установка новейшего исходного кода, как описано в разделе 2.2 «Компиляция и установка из исходного кода в Unix», обычно перезаписывает все предыдущие установки из исходного кода. Удалить Nmap стоит при смене способа установки, например с исходного кода на RPM или наоборот, либо если программа больше не используется и вам важны занимаемые ею несколько мегабайт дискового пространства.

Способ удаления Nmap зависит от первоначального способа установки, рассмотренного в предыдущих разделах. Простота удаления и другого обслуживания - важное преимущество большинства двоичных пакетов. Например, если Nmap установлен с помощью распространённой в дистрибутивах Linux системы RPM, его можно удалить, выполнив от имени `root` команду `rpm -e nmap zenmap`. Аналогичные возможности предлагают большинство других диспетчеров пакетов; подробности ищите в их документации.

Если Nmap был установлен с помощью установщика Windows, просто откройте **Панель управления**, выберите **Установка и удаление программ**, а затем нажмите кнопку **Удалить** для Nmap. Можно также удалить WinPcap, если он не требуется другим приложениям, таким как Wireshark.

Если Nmap устанавливался из исходного кода, удаление немного сложнее. Если каталог сборки, где первоначально выполнялась команда `make install`, всё ещё доступен, Nmap можно удалить командой `make uninstall`. Если каталога сборки больше нет, введите `nmap -V`, чтобы получить номер версии Nmap. Затем загрузите архив tar с исходным кодом этой версии Nmap с адреса http://nmap.org/dist/ или http://nmap.org/dist-old/. Распакуйте архив и перейдите в созданный каталог (`nmap-<version>`). Выполните `./configure`, включив все параметры пути установки, которые задавались в первый раз, например `--prefix` или `--datadir`. Затем выполните `make uninstall`. В качестве альтернативы можно просто удалить все связанные с Nmap файлы. Если использовалась стандартная установка из исходного кода Nmap версии 4.50 или более новой, программа удаляется следующими командами.

```
# cd /usr/local
# rm -f bin/nmap bin/nmapfe bin/xnmap
# rm -f man/man1/nmap.1 man/man1/zenmap.1
# rm -rf share/nmap
# ./bin/uninstall_zenmap
```

Возможно, приведённые команды придётся немного изменить, если при первоначальной установке Nmap задавался `--prefix` или другой параметр пути установки. Файлы, относящиеся к `zenmap`, `nmapfe` и `xnmap`, отсутствуют, если графическая оболочка Zenmap не устанавливалась.

Сноски

[1] [назад] http://cert.org/advisories/CA-2002-28.html
[2] [назад] http://cert.org/advisories/CA-2002-24.html
[3] [назад] http://www.gnupg.org/
[4] [назад] http://subversion.tigris.org
[5] [назад] http://www.tcpdump.org
[6] [назад] http://support.microsoft.com/kb/Q196271
[7] [назад] http://support.microsoft.com/kb/Q149532
[8] [назад] http://microsoft.com/downloads/details.aspx?FamilyID=889482FC-5F56-4A38-83B8-DE776FD4138C
[9] [назад] http://microsoft.com/downloads/details.aspx?FamilyID=0692C27E-F63A-414C-B3EB-D2342FBB6C00
[10] [назад] http://nsis.sourceforge.net/Main_Page
[11] [назад] http://www.7-zip.org

[12] [назад] http://www.winzip.com
[13] [назад] http://www.pkware.com
[14] [назад] http://www.microsoft.com/express/vc
[15] [назад] http://www.cygwin.com/
[16] [назад] http://nmap.org/download.html#macosx
[17] [назад] http://developer.apple.com/tools/xcode/
[18] [назад] http://www.finkproject.org
[19] [назад] http://www.macports.org
[20] [назад] http://www.openbsd.org/faq/
[21] [назад] http://www.freebsd.org/doc/en_US.ISO8859-1/books/handbook/ports.html

Глава 3. Обнаружение узлов («ping-сканирование»)

3.1. Введение

Один из самых первых шагов любой сетевой разведки - сократить иногда огромное множество диапазонов IP до списка активных или представляющих интерес узлов. Сканировать каждый порт каждого IP-адреса медленно и обычно не требуется. Разумеется, критерий интереса к узлу сильно зависит от цели сканирования. Сетевых администраторов могут интересовать лишь узлы, на которых работает определённая служба, тогда как аудиторам безопасности важно каждое устройство с IP-адресом. Администратору может быть достаточно обычной ping-проверки ICMP для поиска узлов во внутренней сети, а внешний специалист по тестированию на проникновение способен применить разнообразный набор из десятков проб, пытаясь обойти ограничения межсетевого экрана.

Поскольку потребности в обнаружении узлов весьма разнообразны, Nmap предлагает широкий выбор параметров для настройки применяемых методов. Несмотря на название «ping-сканирование», возможности программы далеко выходят за пределы простых пакетов эхо-запроса ICMP, связанных с повсеместно распространённой утилитой `ping`. Пользователи могут полностью пропустить этап ping-проверки с помощью сканирования списка (`-sL`) или отключения ping (`-PN`), либо обращаться к сети произвольными сочетаниями многопортовых проб TCP SYN/ACK, UDP и ICMP. Цель таких проб - вызвать ответы, доказывающие, что IP-адрес действительно активен, то есть используется узлом или сетевым устройством. Во многих сетях в каждый конкретный момент активно лишь небольшое количество IP-адресов. Это особенно характерно для пространства частных адресов, например `10.0.0.0/8`. В этой сети 16,8 миллиона IP-адресов, но я видел, как её использовали компании, имевшие менее тысячи машин. Обнаружение узлов позволяет найти эти машины в редко заселённом море IP-адресов.

Сначала в этой главе в общих чертах рассматривается работа ping-сканирования Nmap и высокоуровневые параметры управления. Затем описываются конкретные методы, включая принципы их работы и наиболее подходящие случаи применения. Nmap предлагает множество методов ping-проверки, потому что для прохождения через цепочку межсетевых экранов и фильтров маршрутизаторов на пути к целевой сети часто требуется тщательно составленное сочетание. После обсуждения эффективных общих стратегий ping-сканирования алгоритмы рассматриваются на низком уровне.

3.2. Задание целевых узлов и сетей

Всё, что в командной строке Nmap не является параметром или аргументом параметра, считается спецификацией целевого узла. В простейшем случае для сканирования указывается целевой IP-адрес или имя узла.

Иногда требуется просканировать целую сеть соседних узлов. Для этого Nmap поддерживает адресацию в стиле CIDR. К IPv4-адресу или имени узла можно добавить `/<numbits>`, и Nmap просканирует все IP-адреса, первые `<numbits>` битов которых совпадают с заданным эталонным IP-адресом или именем узла. Например, `192.168.10.0/24` просканирует все 256 узлов от `192.168.10.0` (в двоичном виде `11000000 10101000 00001010 00000000`) до `192.168.10.255` (в двоичном виде `11000000 10101000 00001010 11111111`) включительно. Спецификация

`192.168.10.40/24` просканирует в точности те же цели. Поскольку узел `scanme.nmap.org` имеет IP-адрес `64.13.134.52`, спецификация `scanme.nmap.org/16` просканирует 65 536 IP-адресов от `64.13.0.0` до `64.13.255.255`. Наименьшее допустимое значение - `/0`, при котором сканируется весь Интернет. Наибольшее - `/32`, при котором сканируется только указанный узел или IP-адрес, поскольку фиксированы все биты адреса.

Нотация CIDR кратка, но не всегда достаточно гибка. Например, может потребоваться просканировать `192.168.0.0/16`, пропустив все IP-адреса, оканчивающиеся на `.0` или `.255`, поскольку они могут использоваться как адреса сети и широковещательные адреса подсети. Nmap поддерживает это с помощью адресации диапазонов октетов. Вместо обычного IP-адреса для каждого октета можно указать разделённый запятыми список чисел или диапазонов. Например, `192.168.0-255.1-254` пропустит все адреса диапазона, заканчивающиеся на `.0` или `.255`. Диапазоны необязательно ограничивать последними октетами: спецификация `0-255.0-255.13.37` выполнит сканирование всего Интернета для всех IP-адресов, оканчивающихся на `13.37`. Такая широкая выборка бывает полезна для обзоров и исследований Интернета.

Адреса IPv6 можно задавать только полностью определённым адресом IPv6 или именем узла. CIDR и диапазоны октетов для IPv6 не поддерживаются, поскольку редко приносят пользу.

Nmap принимает в командной строке несколько спецификаций узлов, и они необязательно должны относиться к одному типу. Команда `nmap scanme.nmap.org 192.168.0.0/8 10.0.0,1,3-7.0-255` делает именно то, чего от неё можно ожидать.

3.2.1. Ввод из списка (-iL)

Огромный список узлов часто неудобно передавать в командной строке, хотя такая потребность возникает регулярно. Например, DHCP-сервер может экспортировать список 10 000 текущих арендованных адресов, которые требуется просканировать. Или вы хотите просканировать все IP-адреса, кроме этих, чтобы найти узлы, использующие несанкционированные статические IP-адреса. Просто создайте список сканируемых узлов и передайте имя файла Nmap как аргумент параметра `-iL`. Записи могут быть представлены в любом формате, принимаемом Nmap в командной строке: IP-адрес, имя узла, CIDR, IPv6 или диапазоны октетов. Записи разделяются одним или несколькими пробелами, знаками табуляции или переводами строк. Если Nmap должен читать узлы из стандартного ввода, а не из файла, в качестве имени файла можно указать дефис (`-`).

3.2.2. Случайный выбор целей (-iR <numtargets>)

Для обзоров всего Интернета и других исследований может потребоваться случайный выбор целей. Он выполняется параметром `-iR`, аргумент которого задаёт число создаваемых IP-адресов. Nmap автоматически пропускает некоторые нежелательные IP-адреса, например из частных, многоадресных или нераспределённых диапазонов. Аргумент `0` задаёт бесконечное сканирование. Помните, что некоторые сетевые администраторы резко возражают против несанкционированного сканирования своих сетей. Перед применением `-iR` внимательно прочитайте раздел 1.4 «Правовые вопросы».

Если дождливым днём вам станет совсем скучно, попробуйте команду `nmap -sS -PS80 -iR 0 -p 80`, чтобы находить случайные веб-серверы для просмотра.

3.2.3. Исключение целей (--exclude, --excludefile <filename>)

Нередко существуют машины, которые нельзя сканировать ни при каких обстоятельствах. Они могут быть настолько критичными, что вы не готовы даже к малейшему риску неблагоприятной реакции. Вас могут обвинить в случайном сбое, даже если сканирование Nmap не имело к нему никакого отношения. Возможно, имеется унаследованное оборудование, которое, как известно, выходит из строя при сканировании, но исправить или заменить его пока не удалось. Либо некоторые диапазоны IP принадлежат дочерним компаниям, клиентам или партнёрам, на сканирование которых у вас нет разрешения. Консультанты часто не хотят включать собственную машину в сканирование сетей клиента. Какова бы ни была причина, узлы или целые сети можно исключить параметром `--exclude`. Просто передайте ему разделённый запятыми список исключаемых целей и сетевых блоков в обычном синтаксисе Nmap. В качестве альтернативы можно создать файл исключаемых узлов или сетей и передать его Nmap с параметром `--excludefile`.

3.2.4. Практические примеры

Некоторые инструменты имеют простые интерфейсы, разрешающие лишь список узлов или, возможно, начальный и конечный IP-адреса диапазона; Nmap намного мощнее и гибче. Но изучать Nmap тоже может быть сложнее, а сканирование неверных IP-адресов иногда приводит к катастрофе. К счастью, Nmap позволяет выполнить пробный запуск с помощью сканирования списка, параметр `-sL`. Просто выполните `nmap -sL -n <targets>`, чтобы увидеть предназначенные для сканирования IP-адреса до фактического запуска.

Возможно, примеры - самый эффективный способ объяснить синтаксис задания узлов Nmap. Ниже приведено несколько примеров, начиная с простейших.

• `nmap scanme.nmap.org`, `nmap scanme.nmap.org/32`, `nmap 64.13.134.52` — Все эти команды делают одно и то же, если `scanme.nmap.org` разрешается в `64.13.134.52`. Они сканируют один IP-адрес и завершают работу.

• `nmap scanme.nmap.org/24`, `nmap 64.13.134.52/24`, `nmap 64.13.134.0-255` — Все они предписывают Nmap просканировать 256 IP-адресов от `64.13.134.0` до `64.13.134.255`. Иными словами, сканируется окружающее `scanme.nmap.org` адресное пространство размером с сеть класса C.

• `nmap 64.13.134.52/24 --exclude scanme.nmap.org,insecure.org` — Предписывает Nmap просканировать сеть класса C вокруг `64.13.134.52`, но пропустить `scanme.nmap.org` и `insecure.org`, если они окажутся в этом диапазоне адресов.

• `nmap 10.0.0.0/8 --exclude 10.6.0.0/16,ultra-sensitive-host.company.com` — Предписывает Nmap просканировать весь частный диапазон 10, пропустив всё, что начинается с `10.6`, а также `ultra-sensitive-host.company.com`.

• `egrep '^lease' /var/lib/dhcp/dhcpd.leases | awk '{print $2}' | nmap -iL -` — Получает список выданных DHCP IP-адресов и передаёт их непосредственно Nmap для сканирования. Обратите внимание, что параметру `-iL` передаётся дефис для чтения из стандартного ввода.

• `nmap -6 2001:800:40:2a03::3` — Сканирует узел IPv6 по адресу `2001:800:40:2a03::3`.

3.3. Поиск IP-адресов организации

Nmap автоматизирует многие аспекты сетевого сканирования, но вам всё равно необходимо указать сканируемые сети. Можно было бы задать `-iR` и надеяться, что Nmap случайно попадёт в

целевую компанию, либо применить грубую силу, указав `0.0.0.0/0` для сканирования всего Интернета. Однако любой из этих вариантов может занять месяцы или годы и, возможно, навлечь неприятности. Поэтому перед сканированием важно тщательно исследовать целевые сетевые блоки. Даже если вы проводите законное тестирование на проникновение и клиент предоставил список своих сетевых блоков, его необходимо перепроверить. Иногда записи клиентов устаревают или просто записаны неверно. Подписанное клиентом письмо с разрешением не поможет, если вы случайно взломаете не ту компанию.

Во многих случаях вначале имеется лишь доменное имя компании. В этом разделе показаны несколько наиболее распространённых и эффективных способов преобразовать его в список сетевых блоков, принадлежащих целевой компании, управляемых ею или связанных с ней. Демонстрируются типичные утилиты командной строки Linux, но сходные инструменты доступны и для других платформ.

На конференции ShmooCon в 2006 году ко мне подошёл человек и пожаловался, что в документации Nmap приведено много примеров сканирования `target.com`. Он заметил, что для этой цели ICANN зарезервировала доменное имя `example.com`, и требовал соответствующим образом исправить страницу руководства. Технически он был прав, но такая одержимость казалась странной. Его мотив стал понятен, когда он протянул мне визитную карточку.

Рисунок 3.1. Визитная карточка всё объясняет

По-видимому, многие пользователи Nmap копировали примеры прямо со страницы руководства и запускали их, не меняя спецификатор цели. Поэтому `target.com` оказался завален сканированиями и соответствующими предупреждениями IDS. В память об этом случае цель данного раздела - определить диапазоны IP, выделенные корпорации Target и используемые ею.

3.3.1. Приёмы DNS

Основное назначение DNS - разрешать доменные имена в IP-адреса, поэтому логично начать именно с него. В примере 3.1 я использую команду Linux `host`, чтобы запросить несколько распространённых типов записей DNS.

Пример 3.1. Использование команды host для запроса распространённых типов записей DNS

```
> host -t ns target.com
target.com name server ns4.target.com.
target.com name server ns3.target.com.
target.com name server ns1-auth.sprintlink.net.
target.com name server ns2-auth.sprintlink.net.
target.com name server ns3-auth.sprintlink.net.
> host -t a target.com
target.com has address 161.225.130.163
target.com has address 161.225.136.0
> host -t aaaa target.com
target.com has no AAAA record
> host -t mx target.com
target.com mail is handled by 50 smtp02.target.com.
target.com mail is handled by 5 smtp01.target.com.
> host -t soa target.com
target.com has SOA record extdns02.target.com. hostmaster.target.com.
```

Затем я снова разрешаю с помощью host IP-адреса приведённых выше имён узлов и пробую несколько распространённых имён поддоменов, таких как www.target.com и ftp.target.com. Начиная с имён вроде ns3.target.com и smtp01.target.com, я меняю цифры, пытаясь найти новые машины. В результате получается следующий список имён и адресов target.com.

Таблица 3.1. Первая попытка перечислить IP-адреса target.com

Имя узла — IP-адрес

• ns3.target.com — 161.225.130.130
• ns4.target.com — 161.225.136.136
• ns5.target.com — 161.225.130.150
• target.com — 161.225.136.0, 161.225.130.163
• smtp01.target.com — 161.225.140.120
• smtp02.target.com — 198.70.53.234, 198.70.53.235
• extdns02.target.com — 172.17.14.69
• www.target.com — 207.171.166.49

Хотя таким способом можно создать значительный список имён узлов, настоящий кладезь имён даёт передача зоны. Сейчас большинство DNS-серверов отклоняет запросы передачи зоны, но попытаться стоит, потому что многие всё ещё разрешают её. Обязательно попробуйте каждый DNS-сервер, найденный по записям NS домена и в результате сканирования портов корпоративных диапазонов IP. Пока мы нашли семь серверов имён Target: ns3.target.com, ns4.target.com, ns5.target.com, ns1-auth.sprintlink.net, ns2-auth.sprintlink.net, ns3-auth.sprintlink.net и extdns02.target.com. К сожалению, все эти серверы либо отказали в передаче, либо не поддерживали соединения DNS через TCP, необходимые для передачи зоны. В примере 3.2 показана неудачная попытка передачи зоны target.com с помощью распространённого инструмента dig (domain information groper), а затем успешная передача для не связанной с ним организации (cpsr.org).[1] **Пример 3.2. Неудачная и успешная передача зоны**

```
> dig @ns2-auth.sprintlink.net -t AXFR target.com
; <<>> DiG 9.5.0b3 <<>> @ns2-auth.sprintlink.net -t AXFR target.com

; Transfer failed.

> dig @ns2.eppi.com -t AXFR cpsr.org
; <<>> DiG 9.5.0b1 <<>> @ns2.eppi.com -t AXFR cpsr.org

cpsr.org.            10800   IN      SOA   ns1.findpage.com. root.cpsr.org.
cpsr.org.            10800   IN      NS    ns.stimpy.net.
cpsr.org.            10800   IN      NS    ns1.findpage.com.
cpsr.org.            10800   IN      NS    ns2.eppi.com.
cpsr.org.            10800   IN      A     208.96.55.202
```

```
cpsr.org.            10800   IN      MX    0 smtp.electricembers.net.
diac.cpsr.org.       10800   IN      A     64.147.163.10
groups.cpsr.org.     10800   IN      NS    ns1.electricembers.net.
localhost.cpsr.org.  10800   IN      A     127.0.0.1
mail.cpsr.org.       10800   IN      A     209.209.81.73
peru.cpsr.org.       10800   IN      A     208.96.55.202
www.peru.cpsr.org.   10800   IN      A     208.96.55.202
[...]
```

Распространённая ошибка при сборе подобных результатов прямого DNS - предположение, будто все системы, найденные в домене, обязательно относятся к сети данной организации и безопасны для сканирования. В действительности ничто не мешает организации добавлять записи, указывающие на любое место в Интернете. Так часто поступают, передавая службы внешним исполнителям, но сохраняя исходное доменное имя как часть бренда. Например, `www.target.com` разрешается в `207.171.166.49`. Относится ли он к сети Target или управляется третьей стороной, которую мы не хотим сканировать? Три быстрые и простые проверки - обратное разрешение DNS, трассировка маршрута и запрос whois к соответствующему реестру IP-адресов. Первые две проверки может выполнить Nmap, а для третьей хорошо подходит команда Linux `whois`. Эти проверки для `target.com` показаны в примерах 3.3 и 3.4.

Пример 3.3. Обратное разрешение DNS и трассировка маршрута Nmap для www.target.com

```
# nmap -PN -T4 --traceroute www.target.com

Starting Nmap ( http://nmap.org )
Interesting ports on 166-49.amazon.com (207.171.166.49):
Not shown: 998 filtered ports
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https

TRACEROUTE (using port 80/tcp)
HOP RTT    ADDRESS
[cut]
9   84.94  ae-2.ebr4.NewYork1.Level3.net (4.69.135.186)
10  87.91  ae-3.ebr4.Washington1.Level3.net (4.69.132.93)
11  94.80  ae-94-94.csw4.Washington1.Level3.net (4.69.134.190)
12  86.40  ae-21-69.car1.Washington3.Level3.net (4.68.17.7)
13  185.10 AMAZONCOM.car1.Washington3.Level3.net (4.71.204.18)
14  84.70  72.21.209.38
15  85.73  72.21.193.37
16  85.68  166-49.amazon.com (207.171.166.49)

Nmap done: 1 IP address (1 host up) scanned in 20.57 seconds
```

Пример 3.4. Использование whois для поиска владельца IP-адреса www.target.com

```
> whois 207.171.166.49
[Querying whois.arin.net]
[whois.arin.net]

OrgName:    Amazon.com, Inc.
OrgID:      AMAZON-4
Address:    605 5th Ave S
City:       SEATTLE
StateProv:  WA
PostalCode: 98104
Country:    US
[...]
```

В примере 3.3 жирным шрифтом выделены обратное DNS-разрешение в двух местах и интересные результаты трассировки. Доменное имя Amazon.com с высокой вероятностью указывает, что веб-сайт обслуживает Amazon, а не сама Target. Затем результаты whois, где владельцем пространства IP названо `Amazon.com, Inc.`, устраняют все сомнения. Веб-сайт оформлен под брендом Target, но внизу отображает надпись `Powered by Amazon.com`. Если бы Target наняла нас проверить её безопасность, для работы с этим адресным пространством потребовалось бы

отдельное разрешение Amazon.

Для поиска имён узлов в заданном домене можно использовать и веб-базы данных. Например, у Netcraft есть средство поиска DNS веб-сайтов по адресу http://searchdns.netcraft.com/?host. Ввод `.target.com` в форму даёт 36 результатов, как показано на рисунке 3.2. В удобной таблице указан также владелец сетевого блока, что позволяет заметить случаи вроде обслуживания `www.target.com` компанией Amazon. О некоторых найденных узлах мы уже знали, но едва ли догадались бы о таких именах, как `sendasmoochie.target.com`.

Рисунок 3.2. Netcraft находит 36 веб-серверов Target

Найдено 36 сайтов. В исходном рисунке показаны первые девять:

№ · сайт · отчёт сайта · впервые замечен · сетевой блок · ОС

• **1.** `www.target.com` — отчёт: есть; впервые замечен: октябрь 1995; сетевой блок: Amazon.com, Inc.; ОС: неизвестна.
• **2.** `weeklyad.target.com` — отчёт: есть; впервые замечен: январь 2005; сетевой блок: Akamai Technologies; ОС: Linux.
• **3.** `sites.target.com` — отчёт: есть; впервые замечен: август 2005; сетевой блок: Target Corporation; ОС: F5 Big-IP.
• **4.** `redcard.target.com` — отчёт: есть; впервые замечен: ноябрь 2005; сетевой блок: Target Corporation; ОС: F5 Big-IP.
• **5.** `www.target.com.au` — отчёт: нет; впервые замечен: июнь 2000; сетевой блок: APNIC; ОС: Windows 2000.
• **6.** `targetrewards.target.com` — отчёт: нет; впервые замечен: август 2005; сетевой блок: Target Corporation; ОС: F5 Big-IP.
• **7.** `cinemared.target.com` — отчёт: есть; впервые замечен: август 2005; сетевой блок: Target Corporation; ОС: F5 Big-IP.
• **8.** `recipes.target.com` — отчёт: нет; впервые замечен: ноябрь 2005; сетевой блок: Allrecipes.com, Inc.; ОС: Windows Server 2003.
• **9.** `bookmarked.target.com` — отчёт: нет; впервые замечен: сентябрь; сетевой блок: Implex.net; ОС: Linux.

Для этой цели можно использовать и Google с запросами вроде `site:target.com`.

3.3.2. Запросы Whois к реестрам IP-адресов

После обнаружения начального набора «затравочных» IP-адресов их необходимо исследовать, чтобы убедиться в принадлежности ожидаемой компании и определить сетевые блоки, частью которых они являются. Небольшой компании может быть выделено всего от 1 до 16 IP-адресов, а крупные корпорации часто имеют тысячи. Эти сведения хранятся в региональных базах данных, таких как ARIN (American Registry for Internet Numbers) для Северной Америки и RIPE для Европы и Ближнего Востока. Современные инструменты whois принимают IP-адрес и автоматически запрашивают соответствующий реестр.

Малые и средние компании обычно не получают пространство IP непосредственно от организаций вроде ARIN. Вместо этого их провайдеры делегируют им сетевые блоки. Иногда запросы IP возвращают такие сведения о провайдере. Обычно в результате получается большой сетевой блок, но неизвестно, какая его часть выделена цели. К счастью, сейчас многие провайдеры субделегируют клиентские диапазоны с помощью Shared Whois (SWIP) или Referral Whois (RWhois). Если провайдер сделал это, можно узнать точный размер сетевого блока клиента.

Один из ранее обнаруженных IP-адресов `target.com` - `161.225.130.163`. В примере 3.5 показан запрос whois, автоматически направляемый в ARIN, для определения владельца и сведений о выделении этого IP-адреса.

Пример 3.5. Использование whois для поиска сетевого блока, содержащего 161.225.130.163

```
> whois 161.225.130.163
[Querying whois.arin.net]
[whois.arin.net]

OrgName:    Target Corporation
OrgID:      TARGET-14
Address:    1000 Nicollet TPS 3165
City:       Minneapolis
StateProv:  MN
PostalCode: 55403
Country:    US

NetRange:   161.225.0.0 - 161.225.255.255
CIDR:       161.225.0.0/16
NetName:    TARGETNET
NetHandle:  NET-161-225-0-0-1
Parent:     NET-161-0-0-0-0
NetType:    Direct Assignment
NameServer: NS3.TARGET.COM
NameServer: NS4.TARGET.COM
Comment:
RegDate:    1993-03-04
Updated:    2005-11-02

OrgTechHandle: DOMAI45-ARIN
OrgTechName:   Domainnames admin
OrgTechPhone:  +1-612-696-2525
OrgTechEmail:  Domainnames.admin@target.com
```

Неудивительно, что Target принадлежит огромный сетевой блок класса B, охватывающий все 65 536 IP-адресов от `161.225.0.0` до `161.225.255.255`. Поскольку в поле `OrgName` указана Target, это не тот случай, когда мы видим результаты её провайдера.

Следующий шаг - аналогично найти все обнаруженные ранее IP-адреса, не попадающие в этот диапазон. После этого можно перейти к более сложным запросам. Команда `whois -h whois.arin.net \?` показывает синтаксис запросов ARIN. Было бы удобно искать все сетевые блоки, соответствующие заданному адресу, `OrgID` или `OrgTechEmail`, но реестры IP обычно этого не разрешают. Тем не менее доступно много других полезных запросов. Например, `whois -h whois.arin.net @target.com` показывает все контакты ARIN с адресами электронной почты в `target.com`. Запрос `whois -h whois.arin.net "n target*"` выводит все дескрипторы сетевых блоков, начинающиеся с `target`, без учёта регистра. Аналогично, `whois -h whois.arin.net "o target*"` показывает все названия организаций, начинающиеся с `target`. Для каждой записи можно найти адрес, номер телефона и контактную электронную почту, чтобы определить, относится ли она к компании, которую требуется сканировать. Часто это третьи стороны, которым просто досталось похожее имя.

3.3.3. Сведения о маршрутизации в Интернете

Основным протоколом маршрутизации Интернета является Border Gateway Protocol (BGP). При сканировании средних и крупных организаций таблицы маршрутизации BGP помогают находить их IP-подсети по всему миру. Например, предположим, что нужно просканировать IP-адреса корпорации Microsoft. DNS-запрос для `microsoft.com` даёт IP-адрес `207.46.196.115`. Рассмотренный в предыдущем разделе запрос whois показывает, что весь блок `207.46.0.0/16` принадлежит Microsoft по вполне уместному адресу «One Microsoft Way» в Редмонде. Это даёт 65

536 IP-адресов для сканирования, но таблицы BGP раскрывают гораздо больше.

Таким организациям, как Microsoft, для маршрутизации назначаются номера автономных систем (AS). Удобный инструмент определения номера AS, объявляемого для заданного IP-адреса, доступен по адресу http://asn.cymru.com/. Ввод в форму адреса `207.46.0.0` даёт номер AS Microsoft 8075. Далее требуется найти все префиксы IP, маршрутизируемые в эту AS. Удобный инструмент для этого доступен по адресу http://www.robtex.com/as/. Ввод `AS8075` и нажатие **Go** приводит к сводному экрану, где показаны 42 найденных префикса. Они представляют 339 456 IP-адресов, которые можно перечислить, открыв вкладку **BGP**.

Получать сведения BGP из готовых веб-форм удобно, но получать данные маршрутизации от настоящих маршрутизаторов интереснее, а иногда это позволяет выполнять более мощные пользовательские запросы. Несколько организаций предоставляют такую службу. Например, подключитесь по telnet к `route-views.routeviews.org` или посетите http://routeviews.org. Разумеется, эти службы дают доступ к данным только для чтения. Если в рамках дьявольского плана захвата Интернета вам требуется изменять глобальные таблицы маршрутизации, это выходит за рамки книги.

3.4. Разрешение DNS

Основная задача обнаружения узлов Nmap - определить, какие узлы сети включены и отвечают. Это сужает множество целей, поскольку невозможно взломать несуществующий узел. Но на этом обнаружение не должно заканчиваться. Вы ведь не стали бы встречаться с девушками или парнями лишь потому, что они дышат; выбор сетевых машин для проникновения тоже заслуживает особой тщательности. Прекрасный источник сведений, о сетевых узлах, а не о потенциальных партнёрах, - система доменных имён DNS. Даже заботящиеся о безопасности организации часто назначают имена, раскрывающие назначение систем. Нередко беспроводные точки доступа называются `wap` или `wireless`, межсетевые экраны - `fw`, `firewall` или `fw-1`, а веб-серверы разработки с ещё не опубликованным содержимым - `dev`, `staging`, `www-int` или `beta`. Часто раскрываются также названия мест или отделов, как у компании, назвавшей межсетевой экран чикагского офиса `fw.chi`.

По умолчанию Nmap выполняет обратное разрешение DNS для каждого IP-адреса, ответившего на пробы обнаружения узлов, то есть работающего. Если обнаружение узлов пропущено параметром `-PN`, разрешение выполняется для всех IP-адресов. Вместо медленных стандартных библиотек разрешения DNS Nmap использует собственный минимальный резолвер, параллельно выполняющий десятки запросов.

Хотя параметры по умолчанию обычно работают хорошо, Nmap предлагает четыре средства управления разрешением DNS. Они способны существенно повлиять на скорость сканирования и объём собранных сведений.

• `-n` **(не выполнять разрешение DNS)** — Предписывает Nmap никогда не выполнять обратное разрешение DNS для найденных активных IP-адресов. Поскольку DNS даже со встроенным параллельным минимальным резолвером Nmap может работать медленно, этот параметр сокращает время сканирования.

• `-R` **(разрешение DNS для всех целей)** — Предписывает Nmap всегда выполнять обратное разрешение DNS для целевых IP-адресов. Обычно оно выполняется только для отвечающих, то есть работающих, узлов.

• `--system-dns` **(использовать системный резолвер DNS)** — По умолчанию Nmap разрешает IP-адреса, отправляя запросы непосредственно серверам имён, настроенным на вашем узле, и

ожидая ответы. Для повышения производительности много запросов, часто десятки, выполняются параллельно. Этот параметр предписывает вместо этого использовать системный резолвер, по одному IP-адресу за раз через вызов `getnameinfo`. Это медленно и редко полезно, если только вы не обнаружили ошибку в параллельном резолвере Nmap; в таком случае сообщите нам. Для сканирования IPv6 всегда используется системный резолвер.

• `--dns-servers <server1>[,<server2>[,...]]` **(серверы для обратных запросов DNS)** — По умолчанию Nmap определяет серверы DNS для разрешения rDNS из файла `resolv.conf` в Unix или реестра в Win32. Этот параметр позволяет указать другие серверы. Он игнорируется при применении `--system-dns` и при сканировании IPv6. Несколько DNS-серверов часто работают быстрее, особенно если выбрать полномочные серверы для целевого пространства IP. Параметр способен также повысить скрытность, поскольку запросы можно пересылать почти через любой рекурсивный DNS-сервер в Интернете.

Этот параметр полезен и при сканировании частных сетей. Иногда правильные сведения rDNS предоставляют лишь несколько серверов имён, а вы можете даже не знать, где они находятся. Можно просканировать в сети порт 53, возможно с определением версий, а затем поочерёдно выполнять сканирование списка Nmap (`-sL`), указывая каждый сервер имён с помощью `--dns-servers`, пока не найдётся работающий.

3.5. Управление обнаружением узлов

По умолчанию до более навязчивых проб, таких как сканирование портов, определение ОС, механизм сценариев Nmap или определение версий, Nmap включает этап ping-сканирования. Обычно навязчивые проверки выполняются только на машинах, доступность которых установлена на этапе ping-сканирования. Это значительно экономит время и пропускную способность по сравнению с полным сканированием каждого IP-адреса. Однако такой подход подходит не всегда. Иногда требуется сканировать каждый IP-адрес (`-PN`), иногда - выполнить только обнаружение узлов (`-sP`), а порой нужно вывести целевые узлы и завершить работу ещё до отправки ping-проб (`-sL`). Nmap предлагает несколько высокоуровневых параметров управления этим поведением.

3.5.1. Сканирование списка (-sL)

Сканирование списка - вырожденная форма обнаружения узлов, которая просто перечисляет каждый узел в заданных сетях, не отправляя никаких пакетов целевым узлам. По умолчанию Nmap всё же выполняет для узлов обратное разрешение DNS, чтобы узнать их имена. В конце Nmap также сообщает общее число IP-адресов. Сканирование списка служит хорошей проверкой здравого смысла, позволяющей удостовериться в правильности IP-адресов целей. Если узлы имеют незнакомые доменные имена, стоит провести дополнительное исследование, чтобы не просканировать сеть другой компании.

Целевые диапазоны IP могут оказаться неверными по многим причинам. Даже сетевые администраторы способны допустить опечатку в собственных сетевых блоках, а у специалистов по тестированию на проникновение поводов для беспокойства ещё больше. Иногда консультантам по безопасности дают неверные адреса. В других случаях они пытаются найти правильные диапазоны IP в таких источниках, как базы whois и таблицы маршрутизации. Базы могут устареть, либо компания может одалживать пространство IP другим организациям. Вопрос о сканировании материнских, родственных и дочерних компаний, поставщиков услуг следует заранее согласовать с клиентом. Предварительное сканирование списка помогает точно подтвердить предназначенные для сканирования цели.

Другая причина предварительного сканирования списка - скрытность. Иногда нежелательно сразу начинать полномасштабное наступление на целевую сеть, которое, вероятно, вызовет предупреждения IDS и привлечёт ненужное внимание. Сканирование списка ненавязчиво и предоставляет сведения, способные помочь при выборе отдельных целевых машин. Цель теоретически может заметить все запросы обратного DNS, хотя это крайне маловероятно. Если это вызывает опасения, запросы можно переправлять через анонимные рекурсивные DNS-серверы с помощью параметра `--dns-servers`, как описано в разделе «Проксирование DNS».

Сканирование списка задаётся параметром командной строки `-sL`. Поскольку задача состоит лишь в печати списка целевых узлов, с `-sL` нельзя сочетать параметры функций более высокого уровня, таких как сканирование портов, определение ОС или ping-сканирование. Если требуется отключить ping-сканирование, продолжая выполнять функции более высокого уровня, прочитайте о параметре `-PN`. В примере 3.6 сканирование списка используется для перечисления сети CIDR `/28`, то есть 16 IP-адресов, окружающей основной веб-сервер Стэнфордского университета.

Пример 3.6. Перечисление узлов вокруг www.stanford.edu с помощью сканирования списка

```
felix~> nmap -sL www.stanford.edu/28
Starting Nmap ( http://nmap.org )
Host www9.Stanford.EDU (171.67.16.80) not scanned
Host www10.Stanford.EDU (171.67.16.81) not scanned
Host scriptorium.Stanford.EDU (171.67.16.82) not scanned
Host coursework-a.Stanford.EDU (171.67.16.83) not scanned
Host coursework-e.Stanford.EDU (171.67.16.84) not scanned
Host www3.Stanford.EDU (171.67.16.85) not scanned
Host leland-dev.Stanford.EDU (171.67.16.86) not scanned
Host coursework-preprod.Stanford.EDU (171.67.16.87) not scanned
Host stanfordwho-dev.Stanford.EDU (171.67.16.88) not scanned
Host workgroup-dev.Stanford.EDU (171.67.16.89) not scanned
Host courseworkbeta.Stanford.EDU (171.67.16.90) not scanned
Host www4.Stanford.EDU (171.67.16.91) not scanned
Host coursework-i.Stanford.EDU (171.67.16.92) not scanned
Host leland2.Stanford.EDU (171.67.16.93) not scanned
Host coursework-j.Stanford.EDU (171.67.16.94) not scanned
Host 171.67.16.95 not scanned
Nmap done: 16 IP addresses (0 hosts up) scanned in 0.38 seconds
```

3.5.2. Ping-сканирование (-sP)

Этот параметр предписывает Nmap выполнить только ping-сканирование, затем вывести ответившие доступные узлы. Никаких дальнейших проверок, таких как сканирование портов или определение ОС, не выполняется, за исключением сценариев узлов механизма сценариев Nmap (`--script`) и трассировки (`--traceroute`), если эти параметры были указаны.

Это на один шаг навязчивее сканирования списка и часто применяется для тех же целей. Оно быстро и без привлечения особого внимания выполняет лёгкую разведку целевой сети. Знание числа работающих узлов ценнее для злоумышленников, чем список каждого IP-адреса и имени узла, предоставляемый сканированием списка.

Системные администраторы тоже часто считают этот параметр полезным. С его помощью легко подсчитывать доступные машины в сети или наблюдать за доступностью серверов. Это часто называют «ping sweep»; способ надёжнее ping-проверки широковещательного адреса, потому что многие узлы не отвечают на широковещательные запросы.

В примере 3.7 показано быстрое ping-сканирование диапазона CIDR `/24`, то есть 256 IP-адресов, вокруг одного из моих любимых веб-сайтов - Linux Weekly News.

Пример 3.7. Обнаружение узлов вокруг www.lwn.net с помощью ping-сканирования

```
# nmap -sP -T4 www.lwn.net/24

Starting Nmap ( http://nmap.org )
Host 66.216.68.0 seems to be a subnet broadcast address (returned 1 extra ping)
Host 66.216.68.1 appears to be up.
Host 66.216.68.2 appears to be up.
Host 66.216.68.3 appears to be up.
Host server1.camnetsec.com (66.216.68.10) appears to be up.
Host akqa.com (66.216.68.15) appears to be up.
Host asria.org (66.216.68.18) appears to be up.
Host webcubic.net (66.216.68.19) appears to be up.
Host dizzy.yellowdog.com (66.216.68.22) appears to be up.
Host www.outdoorwire.com (66.216.68.23) appears to be up.
Host www.inspectorhosting.com (66.216.68.24) appears to be up.
Host jwebmedia.com (66.216.68.25) appears to be up.
[...]
Host rs.lwn.net (66.216.68.48) appears to be up.
Host 66.216.68.52 appears to be up.
Host cuttlefish.laughingsquid.net (66.216.68.53) appears to be up.
[...]
Nmap done: 256 IP addresses (105 hosts up) scanned in 12.69 seconds
```

Этот пример занял всего 13 секунд, но дал ценные сведения. В диапазоне адресов размером с сеть класса C работает 105 узлов. По множеству не связанных друг с другом доменных имён, собранных в столь малом пространстве IP, ясно, что LWN использует услуги совместного размещения или поставщика выделенных серверов. Если машины LWN окажутся хорошо защищёнными, злоумышленник может атаковать одну из соседних машин, а затем выполнить локальную атаку Ethernet с помощью таких инструментов, как Ettercap или Dsniff. Эти сведения можно применить этично, например если сетевой администратор обдумывает перенос машин к этому поставщику. Прежде чем заключать долгосрочный договор или предпринимать дорогостоящий и нарушающий работу переезд центра обработки данных, он может написать нескольким перечисленным организациям и спросить их мнение об услуге.

По умолчанию параметр `-sP` отправляет эхо-запрос ICMP и пакет TCP ACK на порт 80. Поскольку непривилегированные пользователи Unix или пользователи Windows без установленного WinPcap не могут отправлять такие необработанные пакеты, в этих случаях вместо них посылается пакет SYN. Пакет SYN отправляется системным вызовом TCP `connect` на порт 80 целевого узла. Когда привилегированный пользователь сканирует цели в локальной сети Ethernet, используются запросы ARP (`-PR`), если не задан параметр `--send-ip`.

Параметр `-sP` можно сочетать с любыми методами из раздела 3.6 «Методы обнаружения узлов» для большей гибкости. Если указаны какие-либо параметры типов проб и номеров портов из того раздела, они заменяют стандартные пробы ACK и эхо-запроса. При наличии строгих межсетевых экранов между исходным узлом с Nmap и целевой сетью рекомендуется применять эти расширенные методы. Иначе узлы могут остаться незамеченными, когда межсетевой экран отбрасывает пробы или ответы на них.

3.5.3. Отключение ping (-PN)

Другой вариант - полностью пропустить этап обнаружения Nmap. Обычно Nmap использует его для определения активных машин перед более тяжёлым сканированием. По умолчанию тяжёлые проверки, такие как сканирование портов, определение версий или определение ОС, выполняются только для работающих узлов. Отключение обнаружения узлов параметром `-PN` заставляет Nmap пытаться выполнить запрошенные функции сканирования для каждого заданного целевого IP-адреса. Поэтому, если в командной строке указано целевое пространство адресов размером с класс B (`/16`), будут просканированы все 65 536 IP-адресов.

Полноценное обнаружение узлов пропускается, как при сканировании списка, но вместо остановки и печати списка целей Nmap продолжает выполнять запрошенные функции, считая активным каждый целевой IP-адрес.

Есть много причин отключать ping-проверки Nmap. Одна из самых распространённых - навязчивая оценка уязвимостей. Можно задать десятки разных ping-проб, пытаясь вызвать ответ всех доступных узлов, но активная машина за строгим межсетевым экраном всё равно может не ответить ни на одну из них. Поэтому, чтобы ничего не пропустить, аудиторы часто выполняют интенсивные проверки, например всех 65 536 портов TCP, для каждого IP-адреса целевой сети. Отправка сотен тысяч пакетов IP-адресам, за которыми, вероятно, нет слушающего узла, может казаться расточительной и замедлить сканирование на порядок или более. Nmap приходится повторно передавать пробы каждому порту на случай, если исходная проба потерялась в пути, а также долго ожидать ответов, поскольку для не отвечающих IP-адресов нет оценки времени прохождения туда и обратно (RTT). Но серьёзные специалисты по тестированию на проникновение готовы платить эту цену, чтобы избежать даже небольшого риска пропустить активную машину. Они могут параллельно выполнить быструю проверку, оставив огромное сканирование `-PN` работать в фоне. Дополнительные советы по настройке производительности приведены в главе 6 «Оптимизация производительности Nmap».

Ещё одна частая причина применения `-PN` - наличие у проверяющего списка машин, о которых уже известно, что они работают. Пользователь не видит смысла тратить время на этап обнаружения узлов, создаёт собственный список активных машин и передаёт его Nmap параметром `-iL`, то есть «ввод из списка». С точки зрения экономии времени такая стратегия редко полезна. Из-за повторных передач и оценки RTT, рассмотренных в предыдущем абзаце, даже один не отвечающий IP-адрес в большом списке часто потребует больше времени, чем целый этап ping-сканирования. Кроме того, этап ping позволяет Nmap собрать образцы RTT, способные ускорить последующее сканирование портов, особенно если на целевом узле действуют строгие правила межсетевого экрана. Хотя `-PN` редко экономит время, он важен, когда некоторые машины в списке блокируют все методы обнаружения, которые иначе были бы указаны. Пользователям приходится искать равновесие между скоростью сканирования и вероятностью пропустить тщательно скрытые машины.

3.6. Методы обнаружения узлов

Когда-то определить, зарегистрирован ли IP-адрес за активным узлом, было просто. Достаточно было отправить пакет эхо-запроса ICMP (`ping`) и дождаться ответа. Межсетевые экраны редко блокировали такие запросы, а подавляющее большинство узлов послушно отвечало. Такой ответ требуется с 1989 года стандартом RFC 1122, где недвусмысленно сказано: «Каждый узел ДОЛЖЕН реализовать функцию сервера ICMP Echo, принимающую эхо-запросы и отправляющую соответствующие эхо-ответы».

К несчастью для исследователей сетей, многие администраторы решили, что соображения безопасности важнее требований RFC, и заблокировали сообщения ICMP ping. В примере 3.8 выполняется ping-сканирование Nmap только по ICMP для шести популярных веб-сайтов, но ответы приходят лишь от двух. Это показывает, что отсутствие ответа на пробы ICMP ping больше не позволяет считать узел недоступным. Параметры `-sP -PE` в примере задают ping-сканирование только по ICMP. Параметр `-R` предписывает Nmap выполнять обратное разрешение DNS для всех узлов, даже неработающих.

Пример 3.8. Попытки ping-проверки популярных узлов Интернета

```
# nmap -sP -PE -R -v microsoft.com ebay.com citibank.com google.com \
```

```
                    slashdot.org yahoo.com
Starting Nmap ( http://nmap.org )
Host origin2.microsoft.com (207.46.250.252) appears to be down.
Host pages.ebay.com (66.135.192.87) appears to be down.
Host ldl-www.citicorp.com (192.193.195.132) appears to be down.
Host 216.239.57.99 appears to be up.
Host slashdot.org (66.35.250.150) appears to be down.
Host w3.rc.dcn.yahoo.com (216.109.127.30) appears to be up.
Nmap done: 6 IP addresses (2 hosts up) scanned in 3.76 seconds
```

К счастью, помимо стандартного эхо-запроса ICMP Nmap предлагает множество других методов обнаружения узлов. Они описаны в следующих разделах. Обратите внимание: любые параметры `-P`, рассматриваемые в этом разделе, заменяют стандартные пробы обнаружения, а не добавляются к ним.

3.6.1. TCP SYN Ping (-PS<port list>)

Параметр `-PS` отправляет пустой пакет TCP с установленным флагом SYN. Стандартный порт назначения - 80; во время компиляции его можно изменить, задав `DEFAULT_TCP_PROBE_PORT_SPEC` в `nmap.h`, но другой порт можно передать как параметр. Допускается список портов, например `-PS22-25,80,113,1050,35000`, и тогда пробы каждого порта выполняются параллельно.

Флаг SYN сообщает удалённой системе, что вы пытаетесь установить соединение. Обычно порт назначения закрыт и в ответ приходит пакет RST (сброс). Если порт открыт, цель выполняет второй шаг трёхэтапного рукопожатия TCP, отвечая пакетом TCP SYN/ACK. Затем машина с Nmap прерывает зарождающееся соединение, отвечая RST вместо пакета ACK, который завершил бы трёхэтапное рукопожатие и установил полноценное соединение.[2]

Для Nmap не имеет значения, открыт или закрыт порт. И RST, и рассмотренный выше ответ SYN/ACK сообщают Nmap, что узел доступен и отвечает.

В Unix обычно только привилегированный пользователь `root` может отправлять и принимать необработанные пакеты TCP. Для непривилегированных пользователей автоматически применяется обходной путь: для каждого целевого порта инициируется системный вызов `connect`. В результате узлу отправляется пакет SYN в попытке установить соединение. Если `connect` быстро возвращает успешный результат или ошибку `ECONNREFUSED`, нижележащий стек TCP должен был получить SYN/ACK либо RST, и узел отмечается доступным. Если попытка соединения остаётся незавершённой до истечения времени ожидания, узел отмечается неработающим. Этот обходной путь применяется и для соединений IPv6, поскольку Nmap ещё не поддерживает создание необработанных пакетов IPv6.

В примере 3.8 не удалось обнаружить четыре из шести машин, поскольку они не ответили на эхо-запросы ICMP. Повтор опыта с пробой SYN на порт 80 (HTTP) даёт ответы от всех шести, как показано в примере 3.9.

Пример 3.9. Повторное обнаружение узлов с помощью проб SYN порта 80

```
# nmap -sP -PS80 -R -v microsoft.com ebay.com citibank.com google.com \
                        slashdot.org yahoo.com

Starting Nmap ( http://nmap.org )
Host origin2.microsoft.com (207.46.249.252) appears to be up.
Host pages.ebay.com (66.135.192.87) appears to be up.
Host ldl-www.citicorp.com (192.193.195.132) appears to be up.
Host 216.239.57.99 appears to be up.
Host slashdot.org (66.35.250.150) appears to be up.
Host w3.rc.dcn.yahoo.com (216.109.127.30) appears to be up.
Nmap done: 6 IP addresses (6 hosts up) scanned in 0.48 seconds
```

Помимо обнаружения всех шести машин, второй запуск намного быстрее. Он занимает менее половины секунды, поскольку машины сканируются параллельно и проверка ни разу не ждёт ответа до истечения времени ожидания. Тест не вполне справедлив: все цели являются популярными веб-серверами и потому, как ожидается, слушают порт TCP 80. Тем не менее он показывает, что разные типы узлов отвечают на разные виды проб. Nmap поддерживает параллельное применение многих типов сканирования для эффективной проверки разнообразных сетей.

3.6.2. TCP ACK Ping (-PA<port list>)

Ping-проверка TCP ACK очень похожа на SYN. Отличие, как нетрудно догадаться, в том, что вместо SYN устанавливается флаг TCP ACK. Такой пакет ACK будто бы подтверждает данные в установленном соединении TCP, хотя никакого соединения нет. Поэтому удалённые узлы должны всегда отвечать пакетом RST, тем самым обнаруживая своё существование.

Параметр `-PA` использует тот же стандартный порт, что и проба SYN (80), и может принимать список портов назначения в том же формате. Если проверку выполняет непривилегированный пользователь или указана цель IPv6, применяется рассмотренный выше обходной путь через `connect`. Он несовершенен, поскольку `connect` в действительности отправляет пакет SYN, а не ACK.

Пробы SYN и ACK предлагаются одновременно, чтобы максимально повысить вероятность обхода межсетевых экранов. Многие администраторы настраивают маршрутизаторы и другие простые межсетевые экраны на блокирование входящих пакетов SYN, кроме направленных публичным службам вроде веб-сайта или почтового сервера компании. Это предотвращает другие входящие соединения с организацией, но позволяет пользователям беспрепятственно устанавливать исходящие соединения с Интернетом. Такой подход без учёта состояния потребляет мало ресурсов межсетевого экрана или маршрутизатора и широко поддерживается аппаратными и программными фильтрами. В качестве одного примера распространённости этого метода программное обеспечение межсетевого экрана Linux Netfilter/iptables предлагает удобный параметр `--syn`, который страница руководства описывает так:

> Соответствуют только пакеты TCP с установленным битом SYN и сброшенными битами ACK и RST. Такие пакеты используются для запроса начала соединения TCP; например, блокирование таких пакетов, поступающих в интерфейс, предотвратит входящие соединения TCP, но не повлияет на исходящие соединения TCP. Это равнозначно `--tcp-flags SYN,RST,ACK SYN`.

При наличии таких правил межсетевого экрана ping-пробы SYN (`-PS`), скорее всего, будут блокироваться при отправке закрытым целевым портам. В подобных случаях проба ACK отлично проходит прямо сквозь эти правила.

Другой распространённый тип межсетевого экрана использует правила с отслеживанием состояния, отбрасывающие неожиданные пакеты. Поначалу эта возможность встречалась преимущественно в дорогих межсетевых экранах, но с годами стала гораздо распространённее. Система Linux Netfilter/iptables поддерживает её с помощью параметра `--state`, который классифицирует пакеты по состоянию соединения, как описано в следующем отрывке со страницы руководства:

> Возможные состояния: `INVALID` означает, что пакет не связан ни с одним известным соединением; `ESTABLISHED` означает, что пакет связан с соединением, в котором пакеты уже передавались в обоих направлениях; `NEW` означает, что пакет начал новое соединение или иным образом связан с соединением, в котором пакеты ещё не передавались в обоих

направлениях; RELATED означает, что пакет начинает новое соединение, но связан с существующим соединением, например с передачей данных FTP или ошибкой ICMP.

Проба ACK вряд ли сработает против межсетевых экранов, применяющих такой подход, поскольку подобный неожиданный пакет будет отнесён к состоянию INVALID и, вероятно, отброшен. В примере 3.10 показана попытка ACK ping против Microsoft. Её межсетевой экран с отслеживанием состояния отбрасывает пакет, из-за чего Nmap ошибочно заключает, что узел не работает. В таких случаях у пробы SYN гораздо больше шансов сработать. Возникает вопрос: какой метод применять, когда правила межсетевых экранов целевых сетей неизвестны или неодинаковы? Обычно правильный ответ - оба. Nmap может параллельно отправлять пробы SYN и ACK на множество портов, одновременно выполняя другие методы обнаружения узлов. Подробнее это обсуждается в разделе 3.7 «Сведение всего воедино: стратегии обнаружения узлов».

Пример 3.10. Попытка ACK ping против Microsoft

```
# nmap -sP -PA www.microsoft.com

Starting Nmap ( http://nmap.org )
Warning: Hostname www.microsoft.com resolves to 5 IPs. Using 207.46.192.254.
Note: Host seems down. If it is really up, but blocking ping probes, try -PN
Nmap done: 1 IP address (0 hosts up) scanned in 2.22 seconds
```

3.6.3. UDP Ping (-PU<port list>)

Другой вариант обнаружения узлов - UDP ping, отправляющий пустой пакет UDP (если не задан --data-length) на указанные порты. Список портов имеет тот же формат, что у рассмотренных ранее параметров -PS и -PA. Если порты не указаны, по умолчанию используется 31 338. Это значение можно настроить во время компиляции, изменив DEFAULT_UDP_PROBE_PORT_SPEC в nmap.h. По умолчанию выбран крайне редко используемый порт, поскольку для этого типа сканирования отправка на открытые порты часто нежелательна.

При попадании в закрытый порт целевой машины проба UDP должна вызвать ответный пакет ICMP «порт недоступен». Для Nmap это означает, что машина работает и доступна. Многие другие типы ошибок ICMP, например «узел/сеть недоступны» или «превышен TTL», указывают на неработающий или недоступный узел. Отсутствие ответа трактуется так же. При попадании в открытый порт большинство служб просто игнорируют пустой пакет и ничего не возвращают. Поэтому стандартным портом пробы выбран 31 338, который почти наверняка не используется. Некоторые службы, например протокол Character Generator (chargen), отвечают на пустой пакет UDP и тем самым сообщают Nmap, что машина доступна.

Главное преимущество этого типа сканирования состоит в обходе межсетевых экранов и фильтров, проверяющих только TCP. Например, когда-то у меня был беспроводной широкополосный маршрутизатор Linksys BEFW11S4. Внешний интерфейс этого устройства по умолчанию фильтровал все порты TCP, но пробы UDP всё равно вызывали сообщения «порт недоступен» и тем самым выдавали устройство.

3.6.4. Типы ICMP Ping (-PE, -PP и -PM)

Помимо необычных методов обнаружения узлов через TCP и UDP, рассмотренных выше, Nmap может отправлять стандартные пакеты, используемые повсеместной программой ping. Nmap отправляет целевым IP-адресам пакет ICMP типа 8 (эхо-запрос), ожидая от доступных узлов ответ типа 0 (эхо-ответ). Как отмечалось в начале этой главы, многие узлы и межсетевые экраны теперь блокируют такие пакеты вместо ответа, требуемого RFC 1122. Поэтому сканирование только по

ICMP редко бывает достаточно надёжным для неизвестных целей в Интернете. Но для системных администраторов, контролирующих внутреннюю сеть, это может быть практичным и эффективным подходом. Включить такое поведение с эхо-запросами позволяет параметр `-PE`.

Хотя эхо-запрос является стандартным запросом ICMP ping, Nmap им не ограничивается. Стандарт ICMP (RFC 792) также определяет пакеты запроса временной метки, запроса информации и запроса маски адреса с кодами 13, 15 и 17 соответственно. Хотя заявленная цель этих запросов - получение таких сведений, как маска адреса и текущее время, их легко использовать для обнаружения узлов. В настоящее время Nmap не реализует пакеты запроса информации, поскольку они не имеют широкой поддержки (RFC 1122 настаивает, что «узел НЕ ДОЛЖЕН реализовывать эти сообщения»). Запросы временной метки и маски адреса можно отправить параметрами `-PP` и `-PM` соответственно. Ответ с временной меткой (код ICMP 14) или ответ с маской адреса (код 18) показывает, что узел доступен. Эти два запроса могут оказаться ценными, когда администраторы специально блокируют пакеты эхо-запроса, но забывают, что для той же цели можно использовать другие запросы ICMP.

3.6.5. IP Protocol Ping (-PO<protocol list>)

Новейший вариант обнаружения узлов - ping по протоколу IP, отправляющий пакеты IP с указанным номером протокола в заголовке IP. Список протоколов имеет тот же формат, что списки портов в рассмотренных выше вариантах обнаружения через TCP и UDP. Если протоколы не указаны, по умолчанию отправляется несколько пакетов IP для ICMP (протокол 1), IGMP (протокол 2) и IP-in-IP (протокол 4). Стандартные протоколы можно настроить во время компиляции, изменив `DEFAULT_PROTO_PROBE_PORT_SPEC` в `nmap.h`. Обратите внимание: для ICMP, IGMP, TCP (протокол 6) и UDP (протокол 17) пакеты отправляются с правильными заголовками соответствующих протоколов, а пакеты других протоколов не содержат дополнительных данных после заголовка IP, если только не задан параметр `--data-length`.

Этот метод обнаружения узлов ищет либо ответы по тому же протоколу, что и проба, либо сообщения ICMP «протокол недоступен», означающие, что заданный протокол не поддерживается целевым узлом. Ответ любого из этих типов означает, что целевой узел работает.

3.6.6. ARP-сканирование (-PR)

Один из наиболее распространённых сценариев применения Nmap - сканирование локальной сети Ethernet. В большинстве локальных сетей, особенно использующих диапазоны частных адресов, выделенные RFC 1918, подавляющее большинство IP-адресов в каждый данный момент не используется. Когда Nmap пытается отправить необработанный пакет IP, например эхо-запрос ICMP, операционная система должна определить аппаратный адрес назначения (ARP), соответствующий целевому IP, чтобы правильно адресовать кадр Ethernet. Для этого ей требуется отправить ряд запросов ARP. Это показано в примере 3.11, где предпринимается ping-сканирование локального узла Ethernet. Параметр `--send-ip` предписывает Nmap отправлять пакеты уровня IP, а не необработанные кадры Ethernet, хотя цель находится в локальной сети. В сеанс вставлен вывод Wireshark с тремя запросами ARP и временем их отправки.

Пример 3.11. Ping-сканирование необработанными пакетами IP неработающей цели

```
# nmap -n -sP --send-ip 192.168.33.37

Starting Nmap ( http://nmap.org )
  0.000000 00:01:29:f5:27:f2 -> ff:ff:ff:ff:ff:ff ARP Who has 192.168.33.37?
  0.999836 00:01:29:f5:27:f2 -> ff:ff:ff:ff:ff:ff ARP Who has 192.168.33.37?
  1.999684 00:01:29:f5:27:f2 -> ff:ff:ff:ff:ff:ff ARP Who has 192.168.33.37?
```

```
Note: Host seems down. If it is really up, but blocking ping probes, try -PN
Nmap done: 1 IP address (0 hosts up) scanned in 2.04 seconds
```

Выполнение этого примера заняло более двух секунд, потому что операционная система Linux отправила три запроса ARP с интервалом в одну секунду, прежде чем прекратить попытки обращения к узлу. Поскольку ответы ARP обычно приходят за пару миллисекунд, многосекундные ожидания чрезмерны. Сокращение этого времени ожидания не является приоритетом для разработчиков операционных систем, поскольку подавляющее большинство пакетов отправляется действительно существующим узлам. Nmap, напротив, должен отправить пакеты 16 миллионам IP-адресов, если ему передана такая цель, как `10.0.0.0/8`. Двухсекундное ожидание для каждого адреса превращается в огромную задержку, хотя множество целей проверяется параллельно.

У ping-сканирования необработанными пакетами IP в локальных сетях есть ещё одна проблема. Когда целевой узел, как в предыдущем примере, не отвечает, исходный узел обычно добавляет неполную запись для этого IP-адреса назначения в таблицу ARP ядра. Объём таблицы ARP конечен, и некоторые операционные системы плохо реагируют на её переполнение. При работе Nmap в режиме необработанных пакетов IP (`--send-ip`) ему иногда приходится ждать несколько минут, пока записи кэша ARP не устареют, и только после этого продолжать обнаружение узлов.

ARP-сканирование решает обе проблемы, передавая управление Nmap. Nmap самостоятельно отправляет необработанные запросы ARP и по своему усмотрению управляет повторными передачами и периодами ожидания. Системный кэш ARP обходится. Разница показана в примере 3.12. Это ARP-сканирование занимает немногим больше одной десятой времени, затраченного эквивалентным сканированием IP.

Пример 3.12. ARP ping-сканирование неработающей цели

```
# nmap -n -sP -PR --packet-trace --send-eth 192.168.33.37
Starting Nmap ( http://nmap.org )
SENT (0.0060s) ARP who-has 192.168.33.37 tell 192.168.0.100
SENT (0.1180s) ARP who-has 192.168.33.37 tell 192.168.0.100
Note: Host seems down. If it is really up, but blocking ping probes, try -PN
Nmap done: 1 IP address (0 hosts up) scanned in 0.23 seconds
```

В примере 3.12 ни параметр `-PR`, ни `--send-eth` не оказывают никакого влияния. Причина в том, что ARP является стандартным типом сканирования для узлов Ethernet, которые Nmap определяет как находящиеся в локальной сети Ethernet. Сюда входят как традиционные проводные сети Ethernet, так и беспроводные сети 802.11. ARP-сканирование не только эффективнее, как рассмотрено выше, но и точнее. Узлы часто блокируют ping-пакеты на основе IP, однако обычно не могут блокировать запросы или ответы ARP и при этом продолжать обмен данными в сети. Даже если указаны другие типы ping, например `-PE` или `-PS`, Nmap вместо них использует ARP для всех целей в той же локальной сети. Если вы совершенно не хотите выполнять ARP-сканирование, задайте `--send-ip`, как показано в примере 3.11 «Ping-сканирование необработанными пакетами IP неработающей цели».

Передача Nmap управления отправкой необработанных кадров Ethernet также позволяет ему управлять исходным MAC-адресом. Если на конференции по безопасности только у вас в зале есть PowerBook, а с зарегистрированного на Apple MAC-адреса начинается масштабное ARP-сканирование, присутствующие могут повернуть головы в вашу сторону. Подменить MAC-адрес можно параметром `--spoof-mac`, как описано в разделе 10.4.8 «Подмена MAC-адреса».

3.6.7. Стандартное сочетание

Если не выбран ни один из этих методов обнаружения узлов, Nmap использует стандартный набор, равнозначный параметрам `-PA -PE` для пользователей Windows или привилегированных пользователей Unix (`root`). Внимательные читатели знают, что это означает отправку каждой

машине пакета TCP ACK на порт 80 и эхо-запроса ICMP. Исключение состоит в том, что для всех целей в локальной сети Ethernet применяется ARP-сканирование. Для непривилегированных пользователей командной оболочки Unix стандартное поведение равнозначно `-PS`, то есть системному вызову TCP `connect` для порта 80 целевых узлов. Для аудита безопасности я рекомендую использовать более полный набор типов ping, например рассмотренные в разделе «Проектирование идеальных сочетаний проб».

3.7. Сведение всего воедино: стратегии обнаружения узлов

3.7.1. Связанные параметры

В предыдущих разделах описаны основные параметры управления этапом обнаружения узлов Nmap и настройки применяемых методов. Однако с ним связано гораздо больше общих параметров Nmap. В этом разделе кратко описывается отношение этих флагов к ping-сканированию. Полные описания каждого параметра приведены в главе 15 «Справочное руководство Nmap».

• `-v` **(то же, что** `--verbose`**)** — По умолчанию Nmap обычно выводит только активные, отвечающие узлы. В подробном режиме он выводит также неработающие узлы и дополнительные сведения об активных.

• `--source-port <portnum>` **(то же, что** `-g`**)** — Задание постоянного исходного порта работает для ping-сканирования (TCP и UDP), как и для других функций Nmap. Некоторые наивные администраторы межсетевых экранов делают в наборе правил исключение, чтобы обеспечить работу DNS (порт 53) или FTP-DATA (порт 20). Разумеется, этим они открывают дыру, достаточно большую, чтобы через неё проехало ping-сканирование Nmap. Дополнительные сведения об этом методе приведены в разделе 10.4.2 «Манипулирование исходным портом».

• `-n`, `-R` — Параметр `-n` отключает любое разрешение DNS, а `-R` включает запросы DNS для всех узлов, даже неработающих. Стандартное поведение ограничивает разрешение DNS активными узлами. Для ping-сканирования эти параметры особенно важны, поскольку разрешение DNS способно сильно повлиять на время сканирования.

• `--dns-servers <server1>[,<server2>[,...]]` **(серверы для обратных запросов DNS)** — По умолчанию Nmap пытается определить серверы DNS для разрешения rDNS из файла `resolv.conf` в Unix или реестра в Win32. Вместо этого с помощью данного параметра можно указать другие серверы. Параметр не учитывается при использовании `--system-dns` или сканирования IPv6. Несколько серверов DNS часто работают быстрее и скрытнее, чем запросы только к одному. Наилучшую производительность нередко даёт указание всех полномочных серверов для целевого пространства IP.

• `--data-length <length>` — Этот параметр добавляет к каждому пакету `<length>` случайных байтов данных и работает с типами ping-сканирования TCP, UDP и ICMP для привилегированных пользователей, сканирующих IPv4. Он помогает сделать сканирование менее заметным и больше похожим на пакеты, создаваемые повсеместной диагностической программой `ping`. В нескольких системах обнаружения вторжений (IDS), включая Snort, есть предупреждения для ping-пакетов с нулевым объёмом данных. Данный параметр обходит такие предупреждения. Значение 32 делает эхо-запрос больше похожим на запрос Windows, а 56 имитирует стандартный `ping` Linux.

• `--ttl <value>` — Установка исходящего TTL поддерживается для привилегированных пользователей, выполняющих ping-сканирование IPv4. Это может служить мерой

предосторожности, не позволяющей сканированию распространиться за пределы локальной сети. Так можно ещё убедительнее имитировать штатную программу ping. В некоторых корпоративных сетях существуют известные петли маршрутизации, которые нелегко исправить. Уменьшение исходящего TTL с помощью --ttl помогает снизить нагрузку на процессор маршрутизатора при возникновении петель.

• **Готовые параметры времени (**-T3**,** -T4**,** -T5 **и т. д.)** — Более высокие значения -T ускоряют ping-сканирование так же, как и другие функции Nmap. При умеренно быстром и надёжном соединении между исходной и целевой сетями, то есть любом соединении быстрее модемного, рекомендуется параметр -T4.

• --max-parallelism**,** --min-parallelism <value> — Эти параметры влияют на число проб, которые могут одновременно ожидать ответа. При стандартном типе ping из двух проб значение параллелизма приблизительно равно числу одновременно сканируемых машин. Сокращение числа методов ping до одной пробы на узел, например -PE, удваивает число одновременно сканируемых узлов при заданном уровне параллелизма, тогда как увеличение до четырёх проб на узел, например -PE -PS22,113,50000, уменьшает его вдвое. Большинство пользователей просто применяют готовые параметры времени, такие как -T4.

• --min-rtt-timeout**,** --max-rtt-timeout**,** --initial-rtt-timeout <time> — Эти параметры управляют тем, как долго Nmap ожидает ответ на ping.

• **Параметры ввода (**-iL <filename>**,** -iR <number>**)** — Параметры ввода узлов поддерживаются так же, как и в остальных частях Nmap. Пользователи часто сочетают ввод из списка (-iL) с -PN, чтобы не выполнять ping-сканирование узлов, о работе которых уже известно. Прежде чем делать это ради экономии времени, прочитайте раздел 3.5.3 «Отключение ping (-PN)». Параметр -iR выбирает случайные узлы из распределённого адресного пространства IP Интернета. В качестве аргумента он принимает число случайных узлов, которые требуется просканировать. Ноль задаёт бесконечное сканирование, пока вы не прервёте или не завершите процесс Nmap.

• **Параметры вывода (**-oA**,** -oN**,** -oG**,** -oX **и т. д.)** — Все типы вывода Nmap - обычный, пригодный для grep и XML - поддерживают ping-сканирование. Их работа подробнее описана в главе 13 «Форматы вывода Nmap».

• --randomize-hosts — Перемешивание порядка сканирования узлов с помощью этого параметра может сделать сканирование менее заметным, хотя одновременно способно немного затруднить чтение вывода.

• --reason — Обычный вывод Nmap показывает, работает узел или нет, но не сообщает, на какие проверки обнаружения он ответил. Для получения этих сведений добавьте параметр --reason. Результаты обнаружения узлов могут сбивать с толку, поскольку Nmap не всегда испробует каждую пробу: он останавливается сразу после получения первого ответа.

Таким образом, во время одного запуска Nmap может сообщить об эхо-ответе ICMP от узла, а во время второго первым может прийти ответ RST, и тогда Nmap укажет его.

• --packet-trace — Если требуется ещё больше подробностей, чем даёт --reason, попробуйте --packet-trace. Этот параметр показывает каждый отправленный и полученный Nmap пакет, включая такие сведения, как порядковые номера, значения TTL и флаги TCP.

• -D <decoy1,decoy2,...> — Ложные цели полностью поддерживаются при привилегированном ping-сканировании IPv4, маскируя настоящего атакующего.

• -6 — Ping-сканирование на основе TCP connect (-PS) поддерживает протокол IPv6, включая режим нескольких портов, например -PS22,80,113.

• -S <source IP address>**,** -e <sending device name> — Как и для других функций Nmap, этими параметрами можно указать исходный адрес и отправляющее устройство.

• **Общие параметры** — По умолчанию, если не заданы `-sP` или `-sL`, после этапа обнаружения узлов Nmap переходит к более навязчивому сканированию. Поэтому можно использовать многие десятки общих параметров сканирования портов, определения ОС и определения версий. Дополнительные сведения приведены в справочном руководстве и соответствующих главах.

3.7.2. Выбор и сочетание параметров ping

Для эффективного сканирования недостаточно знать все параметры, описанные в этом и предыдущих разделах. Пользователи должны понимать, как и когда применять их в соответствии с топологией целевой сети и целями сканирования.

Выбор проб и портов TCP

Параметры TCP ping относятся к наиболее мощным методам обнаружения Nmap. Администратор может без существенных последствий для большинства пользователей блокировать эхо-запросы ICMP, но сервер непременно должен отвечать на пакеты SYN, отправляемые предоставляемым им общедоступным службам. Тем временем пакеты ACK часто проходят через межсетевые экраны без отслеживания состояния. Я рекомендую применять и пробы SYN, и пробы ACK, выбирая списки портов с учётом имеющихся сведений о целевых сетях, а также общеупотребительные порты. Быстрое сканирование более 10 000 IP-адресов по всему Интернету показало особую ценность портов из таблицы 3.2. Для узлов с фильтром, по умолчанию отбрасывающим пакеты, то есть наиболее труднодоступного типа, это 14 портов с наибольшей вероятностью быть доступными - открытыми или закрытыми.

Таблица 3.2. Наиболее ценные порты для проб TCP в порядке убывания доступности

Номер порта / служба — обоснование

• `80/http` — Распространённость веб-серверов в Интернете заставляет многих новичков считать, что Интернет и есть Всемирная паутина.
• `25/smtp` — Почта - ещё одно важнейшее приложение Интернета, которое компании пропускают через свои межсетевые экраны.
• `22/ssh` — Похоже, SSH наконец превзошёл Telnet и стал стандартом удалённого администрирования через терминал.

• `443/https` — SSL - популярный способ защиты конфиденциальных сведений каталогов на веб-сайтах.
• `21/ftp` — Этот протокол передачи файлов продолжает жить, хотя многие администраторы межсетевых экранов не стали бы оплакивать его уход.
• `113/auth` — Служба auth (`identd`) позволяет серверам, обычно почтовым или IRC, запрашивать имя подключённого к ним пользователя клиента. Администраторы часто оставляют этот порт нефильтруемым, чтобы избежать долгих периодов ожидания, возникающих, когда правила межсетевого экрана не позволяют серверам подключаться обратно к порту 113. Применение этого порта для ping-сканирования иногда приводит к ложноположительным результатам: некоторые администраторы настраивают межсетевые экраны на подделку ответных пакетов RST для запросов auth к любому IP-адресу своей сети, даже если машины с таким адресом не существует. Они делают это, чтобы избежать периодов ожидания сервера, одновременно не допуская доступа к портам.
• `23/telnet` — Многие устройства до сих пор предоставляют этот интерфейс администрирования, хотя с точки зрения безопасности он кошмарен.
• `53/domain` — Серверы доменных имён чрезвычайно распространены.

• `554/rtsp` — Real Time Stream Control Protocol используется мультимедийными серверами, включая QuickTime и RealServer.
• `3389/ms-term-server` — Microsoft Terminal Services позволяет пользователям, а иногда и взломщикам, получать доступ к приложениям и данным на удалённом компьютере.
• `1723/pptp` — Point-to-Point Tunneling Protocol часто применяется для реализации решений VPN в Microsoft Windows.
• `389/ldap` — Lightweight Directory Access Protocol часто используется для хранения каталогов контактов и подобных сведений.
• `636/ldapssl` — LDAP поверх SSL популярен для доступа к конфиденциальной информации.
• `256/FW1-secureremote` — На устройствах Checkpoint Firewall-1 часто открыт этот порт администрирования.

Помимо популярных портов из приведённого списка рекомендуется выбрать хотя бы один порт с большим номером. Многие плохо настроенные межсетевые экраны по умолчанию запрещают только привилегированные порты, то есть порты с номерами ниже 1024. Обычно я наугад выбираю большой номер, например 40 000 или 10 042, чтобы обнаружить машины за такими межсетевыми экранами.

Выбирая порты для проб, не забывайте о разнообразии платформ. Если ping-сканирование ограничивается двумя портами, HTTP (80) и SSH (22), вероятно, лучше, чем HTTP (80) и HTTPS (443), поскольку последние два относятся к взаимосвязанным веб-службам, и многие машины с HTTPS часто также предоставляют HTTP. Обнаружение двух доступных портов на одной машине для ping-сканирования ничем не лучше обнаружения одного. Цель состоит в выборе портов, чтобы хотя бы одному из них соответствовал широкий набор узлов.

Обратите внимание, что в таблице ценных портов нет многих ориентированных на клиентов портов, например повсеместного порта Windows SMB 135. Главная причина в том, что при составлении таблицы рассматривались только узлы за межсетевыми экранами с запретом по умолчанию, где подавляющее большинство портов фильтруется. В таких условиях порты Windows 135-139 и 445 обычно блокируются. Когда эти машины находятся не за межсетевым экраном, открытые порты для ping-сканирования несущественны, поскольку тысячи закрытых портов работают не хуже.

Выбор портов UDP

При выборе портов UDP помните, что открытый порт вряд ли ответит на пробы. Нужны нефильтруемые порты. Чтобы избежать открытых портов, можно исключить распространённые службы UDP, такие как DNS (порт 53) и SNMP (161). С другой стороны, правила межсетевых экранов часто настолько широки, что эти пробы, особенно направленные порту 53, могут пройти и попасть в закрытый порт. Поэтому я рекомендую выбрать по меньшей мере порт 53 и произвольно выбранный порт с большим номером, например 37 452.

Выбор проб ICMP

Для ICMP обычно стоит попробовать стандартный ping, то есть эхо-запрос. Многие администраторы специально разрешают его, поскольку он полезен для отладки или потому, что этого требует RFC 1122. Я также применил бы хотя бы один запрос маски адреса или временной метки. Они ценны в сетях, где администраторы намеренно блокируют пакеты эхо-запроса, но забывают о других запросах ICMP.

Проектирование идеальных сочетаний проб

Способ объединения всех этих типов ping в стратегию ping-сканирования зависит от особенностей целевой сети и целей сканирования. Для внутренних сетей стандартный тип ping обычно работает хорошо. Он подходит и для большинства нерегулярных сканирований, когда пропустить изредка встречающийся узел не страшно. Добавление проб помогает обнаружить такие редкие скрытые машины, но немного увеличивает продолжительность ping-сканирования.

Затраченное время приблизительно пропорционально числу проб, отправляемых каждой машине. При проверке безопасности целевых сетей через Интернет обычно рекомендуется добавлять больше проб. Постарайтесь включить разнообразный набор рассмотренных ранее методов. Вот набор параметров ping, который должен обнаружить подавляющее большинство узлов: `-PE -PP -PS21,22,23,25,80,113,31339 -PA80,113,443,10042`. Возможно, стоит также добавить `--source-port 53`. Насколько улучшатся результаты и сколько дополнительного времени это займёт? Разумеется, всё зависит от целевой сети, но параметр случайного выбора целей Nmap (`-iR`) позволяет легко провести быструю проверку. В примере 3.13 Nmap создаёт 50 000 случайных IP-адресов, а затем выполняет стандартное ping-сканирование. Не забывайте, что стандартный набор - это пакет TCP ACK на порт 80 и пакет эхо-запроса ICMP.

Пример 3.13. Создание 50 000 IP-адресов с последующим ping-сканированием со стандартными параметрами

```
# nmap -n -sL -iR 50000 -oN - | grep "not scanned" | awk '{print $2}' \
  | sort -n > 50K_IPs
# head -5 50K_IPs
3.100.147.9
3.100.148.119
3.10.160.33
3.10.201.11
3.101.154.139
# nmap -sP -T4 -iL 50K_IPs
# nmap -sP -T4 -iL 50K_IPs -S -oA 50KHosts_DefaultPing
Starting Nmap ( http://nmap.org )
Host dialup-4.177.9.75.SanDiego1.Level3.net (4.177.9.75) appears to be up.
Host dialup-4.181.100.97.SanJose1.Level3.net (4.181.100.97) appears to be up.
Host firewall2.baymountain.com (8.7.97.2) appears to be up.
[thousands of lines cut]
Host 222.91.121.22 appears to be up.
Nmap done: 50000 IP addresses (3348 hosts up) scanned in 1598.07 seconds
```

Сканирование 50 000 адресов заняло чуть менее 27 минут, и было обнаружено 3348 узлов. Большинство имён DNS уже находилось в кэше после предыдущего пробного запуска, хотя при отключении разрешения DNS параметром `-n` сканирование, вероятно, всё равно прошло бы быстрее. Чтобы определить влияние более широкого набора методов ping, те же 50 тысяч узлов просканировали повторно, применив 13 проб на порт вместо стандартных двух. Как показано в примере 3.14, Nmap смог обнаружить на 1125 узлов (34 %) больше. Сканирование заняло около 71 минуты, то есть более чем в 2,5 раза дольше. Учитывая все обнаруженные новые узлы, дополнительное время было потрачено не зря. Обратите внимание, что не все новые узлы обязательно реальны. Увеличение числа ping-проб повышает вероятность того, что Nmap столкнётся с сетевыми артефактами, из-за которых несуществующий узел покажется активным. Один из примеров - межсетевые экраны, возвращающие RST в ответ на пакеты SYN или ACK, направленные порту 113.

Пример 3.14. Повторное ping-сканирование с дополнительными пробами

```
# nmap -sP -PE -PP -PS21,22,23,25,80,113,31339 -PA80,113,443,10042 \
  -T4 --source-port 53 -iL 50K_IPs -oA 50KHosts_ExtendedPing
Starting Nmap ( http://nmap.org )
Host sim7124.agni.lindenlab.com (8.10.144.126) appears to be up.
Host firewall2.baymountain.com (8.7.97.2) appears to be up.
Host 12.1.6.201 appears to be up.
```

```
Host psor.inshealth.com (12.130.143.43) appears to be up.
[thousands of hosts cut]
Host ZM088019.ppp.dion.ne.jp (222.8.88.19) appears to be up.
Host 222.92.136.102 appears to be up.
Nmap done: 50000 IP addresses (4473 hosts up) scanned in 4259.28 seconds
```

При аудите безопасности клиентов я обычно начинаю анализ TCP со сканирования 1000 самых распространённых портов, то есть стандартного набора, применяя полный набор параметров ping-сканирования, подобных показанным в примере 3.14 «Повторное ping-сканирование с дополнительными пробами». Такое сканирование не занимает особенно много времени и позволяет быстро приступить к работе. Одновременно в фоновом режиме я запускаю сканирование всех 65 тысяч портов TCP с `-PN`, то есть с отключённым ping. Когда оно завершается, иногда через несколько дней, я сравниваю результаты с первоначальным быстрым сканированием и исследую все найденные дополнительно порты и машины.

3.8. Алгоритмы кода обнаружения узлов

Одно из величайших преимуществ программного обеспечения с открытым исходным кодом, такого как Nmap, состоит в том, что любознательные пользователи всегда могут изучить исходный код, когда им нужны ответы о работе программы. Функция ping-сканирования самого верхнего уровня - `nexthost` в `targets.cc`; она вызывает `massping` для инициализации списка целей. В свою очередь, `massping` передаёт список функции `ultra_scan` в `scan_engine.cc`. `Ultra_scan` - универсальная функция сканирования Nmap, выполняющая всю трудную работу по отправке, получению и интерпретации пакетов. Подробнее об `ultra_scan` рассказано в разделе 5.13 «Код и алгоритмы сканирования».

Хотя только анализ исходного кода способен дать действительно полную картину работы Nmap вплоть до каждой мелочи, это не всегда самый простой способ понять программу. Во многих случаях наиболее эффективный способ изучить поведение Nmap при заданном наборе параметров командной строки - добавить `--packet-trace`, который выводит все отправленные и полученные Nmap пакеты.

Поскольку исходный код и параметр `--packet-trace` служат прекрасными источниками для изучения мельчайших подробностей работы Nmap, здесь я рассмотрю обнаружение узлов только на высоком уровне. При запуске Nmap ему могут быть переданы сети, содержащие сотни тысяч или даже миллионы узлов. Поэтому Nmap делит их на блоки достаточно малого размера для одновременной обработки - от нескольких десятков до нескольких тысяч узлов. Затем `ultra_scan` проходит по блоку, отправляя пакеты настолько быстро, насколько позволяют механизмы управления перегрузкой. Вместо одновременной отправки каждой цели всех запрошенных пользователем проб Nmap сначала отправляет всем целям первую пробу, затем вторую и так далее. При получении однозначного ответа на пробу узел соответствующим образом помечается работающим или неработающим, и дальнейшие пробы ему не отправляются. Целевой узел, не ответивший ни на одну пробу даже после повторных передач, помечается неработающим. Nmap ожидает, пока для каждого узла не будет получен однозначный ответ или не истечёт время ожидания. В конце концов новые узлы в блоке заканчиваются, а число ожидающих ответа проб по мере завершения повторных передач сокращается до нуля. Подсистема ping-сканирования возвращает результаты, чтобы Nmap мог начать сканирование портов или другие запрошенные проверки целевых машин. Полностью завершив работу с блоком узлов, Nmap выводит результаты и передаёт ping-сканеру следующий блок.

Несколько узлов, обычно с несколькими пробами на каждый, обрабатываются параллельно. Число проб, ожидающих ответа, и периоды ожидания изменяются в реальном времени с учётом задержки

и надёжности сети. Алгоритмы производительности `ultra_scan` подробнее описаны в разделе 5.13 «Код и алгоритмы сканирования».

Сноски

[1] [назад] Вместо этого можно было использовать сценарий NSE `zone-transfer` Nmap (см. главу 9 «Механизм сценариев Nmap»).
[2] [назад] Пакет RST отправляет ядро машины с Nmap в ответ на неожиданный SYN/ACK, а не сам Nmap.

Глава 4. Обзор сканирования портов

4.1. Введение в сканирование портов

Хотя с годами функциональность Nmap расширялась, программа начиналась как эффективный сканер портов, и это остаётся её основной функцией. Простая команда `nmap <target>` сканирует 1000 наиболее часто используемых портов TCP узла `<target>`, относя каждый порт к одному из состояний: `open`, `closed`, `filtered`, `unfiltered`, `open|filtered` или `closed|filtered`.

4.1.1. Что именно представляет собой порт?

Порты - всего лишь программная абстракция, служащая для различения каналов связи. Подобно тому как IP-адреса применяются для идентификации машин в сетях, порты определяют конкретные приложения, используемые на одной машине. Например, веб-браузер по умолчанию подключается к порту TCP 80 машин в URL-адресах HTTP. Если вместо этого указать защищённый протокол HTTPS, браузер по умолчанию попытается обратиться к порту 443.

Nmap работает с двумя протоколами, использующими порты: TCP и UDP. Соединение каждого протокола однозначно определяется четырьмя элементами: исходным и целевым IP-адресами и соответствующими исходным и целевым портами. Все эти элементы - просто числа, помещаемые в заголовки каждого пакета, передаваемого между узлами. Протокол задаётся восьмибитным полем, которое указывает тип пакета в разделе данных IP, то есть полезной нагрузке. Например, TCP имеет номер протокола 6, а UDP - 17. Адреса IPv4 имеют длину 32 бита, а порты - 16 бит. Длина адресов IPv6 составляет 128 бит. Дополнительные сведения о структуре заголовков IP, TCP и UDP приведены в разделе 7 «Справочник по TCP/IP» [xxvi].

Поскольку большинство популярных служб зарегистрировано за хорошо известными номерами портов, нередко можно предположить, какие службы представлены открытыми портами. Nmap включает файл `nmap-services`, содержащий хорошо известные службы для зарегистрированных номеров портов и протоколов, а также распространённые порты троянских бэкдоров и других приложений, которые не утруждают себя регистрацией в Internet Assigned Numbers Authority (IANA). Для справки Nmap выводит это имя службы рядом с номером порта.

Поскольку поле номера порта имеет ширину 16 бит, значения могут достигать 65 535. Наименьшее возможное значение, ноль, недопустимо. API сокетов Berkeley, определяющий обычный способ написания программ для сетевого взаимодействия, не позволяет использовать нулевой порт как таковой. Вместо этого запрос нулевого порта трактуется как подстановочное значение: программисту не важно, какой именно порт будет применён. Система выбирает доступный номер порта. Например, программистам редко важно, какой номер исходного порта используется для исходящего соединения. Поэтому они задают ноль и позволяют операционной системе выбрать номер.

Хотя нулевой порт недопустим, ничто не мешает кому-либо указать его в поле заголовка. Некоторые вредоносные троянские бэкдоры слушают нулевой порт скомпрометированных систем, скрытно предоставляя незаконный доступ и не появляясь в результатах большинства сканирований портов. Для противодействия этому Nmap позволяет сканировать нулевой порт, если он указан явно, например `-p0-65535`.

Первый класс допустимых портов, номера от 1 до 1023, известен как зарезервированные порты. В системах Unix, в отличие от Windows, приложениям требуются специальные привилегии `root`, чтобы привязываться к этим портам и слушать их. Идея состоит в том, чтобы удалённые пользователи могли быть уверены, что подключаются к действительной службе, запущенной администратором, а не каким-либо злонамеренным непривилегированным пользователем. Если бы зарегистрированным портом SSH был 2222 вместо 22, злоумышленник мог бы запустить на этом порту поддельный демон SSH, собирая пароли всех подключающихся. Поскольку большинство распространённых серверных приложений слушает зарезервированные порты, их сканирование часто приносит наиболее ценные результаты.

Диапазон эфемерных портов представляет собой ещё один класс портов. Этот пул предоставляется системой для выделения по мере необходимости. Когда приложение указывает нулевой порт, то есть «любой порт», система выбирает порт из данного диапазона. Диапазон различается между операционными системами и обычно допускает настройку. Он должен содержать не менее нескольких тысяч портов, чтобы они не закончились при большом числе одновременно открытых соединений. При сканировании методом `connect` Nmap может использовать сотни портов одновременно, сканируя каждый заданный порт каждой целевой машины. В Linux просмотреть или задать диапазон можно через файл `/proc/sys/net/ipv4/ip_local_port_range`. Пример 4.1 показывает, что в моей системе Linux диапазон простирается от 32 768 до 61 000. Столь большого диапазона должно хватить почти во всех случаях, но я расширяю его, просто чтобы показать, как это делается.

Пример 4.1. Просмотр и расширение диапазона эфемерных портов в Linux

```
felix/# cat /proc/sys/net/ipv4/ip_local_port_range
32768 61000
felix/# echo "10000 65000" > /proc/sys/net/ipv4/ip_local_port_range
felix/# cat /proc/sys/net/ipv4/ip_local_port_range
10000 65000
felix/#
```

Порты SunRPC часто находятся в эфемерном диапазоне. Другие приложения временно открывают эфемерные порты для передачи файла или другого события. Клиенты FTP часто делают это при запросе передачи в активном режиме. Так поступают и некоторые клиенты P2P и мгновенного обмена сообщениями.

IANA использует собственную схему классификации портов, немного отличающуюся от терминологии этой книги. Её официальный список портов по адресу http://www.iana.org/assignments/port-numbers делит пространство на следующие три класса:

• **Хорошо известные порты** — Это зарезервированные порты в диапазоне от 1 до 1023, как рассмотрено выше, зарегистрированные в IANA за определённой службой. Знакомые примеры - порты 22, 25 и 80 для служб SSH, SMTP и HTTP соответственно.

• **Зарегистрированные порты** — Эти порты находятся в диапазоне от 1024 до 49 151 и зарегистрированы в IANA так же, как хорошо известные порты. Большинство из них используется не так часто, как хорошо известные. Главное отличие состоит в том, что непривилегированные пользователи могут привязываться к этим портам и тем самым запускать службы на зарегистрированном порту. На большинстве платформ пользователи не могут делать этого с хорошо известными портами, поскольку те находятся в зарезервированном диапазоне.

• **Динамические и/или частные порты** — IANA резервирует номера портов от 49 152 до 65 535 для динамического применения, подобного рассмотренному в разделе об эфемерных портах. Эти порты могут также использоваться проприетарными службами, работающими только внутри компании.

Когда в этой книге без ссылки на IANA упоминаются зарегистрированные или хорошо известные порты, обычно имеются в виду порты, зарегистрированные в Nmap в файле `nmap-services`,

независимо от того, находятся ли они в зарезервированном диапазоне.

Файл регистрации портов Nmap (`nmap-services`) содержит эмпирические данные о том, насколько часто каждый порт TCP или UDP обнаруживается открытым. По умолчанию Nmap сканирует 1000 самых популярных портов каждого протокола, сканирование которого запрошено. Существует множество способов указать другой набор портов - по частоте или явным перечислением; они описаны в разделе 4.3.2 «Выбор сканируемых портов».

4.1.2. Какие порты наиболее популярны?

Летом 2008 года я сканировал десятки миллионов узлов Интернета и собирал данные предприятий, чтобы определить, насколько часто каждый номер порта обнаруживается открытым. Важно знать наиболее распространённые порты служб, а также интересно увидеть, какие из них попали в список. В следующих двух списках приведены ведущие порты TCP и UDP по нашим эмпирическим данным сканирования. Указанная служба взята из нашего файла `nmap-services`. Мы стараемся приводить в нём самую распространённую службу для каждого порта, хотя, разумеется, порт может использоваться для разных целей.

20 наиболее часто открытых портов TCP

1. Порт 80 (HTTP) - если вы даже не знаете эту службу, то читаете не ту книгу. На неё пришлось более 14 % обнаруженных нами открытых портов.

2. Порт 23 (Telnet) - Telnet продолжает жить, особенно в качестве порта администрирования таких устройств, как маршрутизаторы и интеллектуальные коммутаторы, хотя он небезопасен: данные не шифруются.

3. Порт 443 (HTTPS) - веб-серверы с шифрованием SSL по умолчанию используют этот порт.

4. Порт 21 (FTP) - FTP, как и Telnet, представляет собой ещё один небезопасный протокол, которому следовало бы умереть. Даже при анонимном FTP, когда нет опасности перехвата данных аутентификации, передаваемые данные всё равно можно подменить.

5. Порт 22 (SSH) - Secure Shell, шифрованная замена Telnet, а в некоторых случаях и FTP.

6. Порт 25 (SMTP) - Standard Mail Transfer Protocol, также небезопасный.

7. Порт 3389 (`ms-term-server`) - порт администрирования Microsoft Terminal Services.

8. Порт 110 (POP3) - Post Office Protocol версии 3 для получения электронной почты, небезопасный.

9. Порт 445 (`Microsoft-DS`) - для взаимодействия SMB поверх IP со службами MS Windows, например для общего доступа к файлам и принтерам.

10. Порт 139 (`NetBIOS-SSN`) - NetBIOS Session Service для взаимодействия со службами MS Windows, например для общего доступа к файлам и принтерам. Поддерживается на машинах Windows дольше, чем порт 445.

11. Порт 143 (IMAP) - Internet Message Access Protocol версии 2, небезопасный протокол получения электронной почты.

12. Порт 53 (Domain) - Domain Name System (DNS), небезопасная система преобразования между именами узлов или доменов и IP-адресами.

13. Порт 135 (MSRPC) - ещё один распространённый порт служб MS Windows.

14. Порт 3306 (MySQL) - для взаимодействия с базами данных MySQL.

15. Порт 8080 (HTTP-Proxy) - обычно используется прокси-серверами HTTP или как альтернативный порт обычных веб-серверов, например когда другой сервер уже слушает порт 80 либо сервер запущен непривилегированным пользователем Unix, способным привязываться только к портам с большими номерами.

16. Порт 1723 (PPTP) - Point-to-Point Tunneling Protocol, метод реализации VPN, который часто требуется для широкополосных подключений к поставщикам услуг Интернета.

17. Порт 111 (RPCBind) - сопоставляет номера программ SunRPC с их текущими номерами портов TCP или UDP.

18. Порт 995 (POP3S) - POP3 с добавленным SSL для защиты.

19. Порт 993 (IMAPS) - IMAPv2 с добавленным SSL для защиты.

20. Порт 5900 (VNC) - небезопасная система совместного доступа к графическому рабочему столу.

20 наиболее часто открытых портов UDP

1. Порт 631 (IPP) - Internet Printing Protocol.

2. Порт 161 (SNMP) - Simple Network Management Protocol.

3. Порт 137 (`NETBIOS-NS`) - один из многих портов UDP для служб Windows, например общего доступа к файлам и принтерам.

4. Порт 123 (NTP) - Network Time Protocol.

5. Порт 138 (`NETBIOS-DGM`) - ещё одна служба Windows.

6. Порт 1434 (`MS-SQL-DS`) - Microsoft SQL Server.

7. Порт 445 (`Microsoft-DS`) - ещё один порт служб Windows.

8. Порт 135 (MSRPC) - ещё один порт служб Windows.

9. Порт 67 (DHCPS) - сервер Dynamic Host Configuration Protocol, выдающий IP-адреса клиентам при их подключении к сети.

10. Порт 53 (Domain) - сервер Domain Name System (DNS).

11. Порт 139 (`NETBIOS-SSN`) - ещё один порт служб Windows.

12. Порт 500 (ISAKMP) - Internet Security Association and Key Management Protocol применяется для настройки VPN IPsec.

13. Порт 68 (DHCPC) - порт клиента DHCP.

14. Порт 520 (Route) - Routing Information Protocol (RIP).

15. Порт 1900 (UPNP) - Microsoft Simple Service Discovery Protocol, обеспечивающий обнаружение устройств Universal Plug and Play.

16. Порт 4500 (`nat-t-ike`) - для согласования прохождения Network Address Translation при установлении соединений IPsec во время Internet Key Exchange.

17. Порт 514 (Syslog) - стандартный демон журналирования Unix.

18. Порт 49 152 (различные службы) - первый из динамических/частных портов, определённых IANA. Начиная отсюда и до конца диапазона портов (65 536) нельзя регистрировать официальные порты. Некоторые системы используют этот диапазон для эфемерных портов, поэтому службе, которая привязывается к порту, не запрашивая конкретный номер, часто выделяется 49 152, если она стала первой такой программой.

19. Порт 162 (SNMPTrap) - порт ловушек Simple Network Management Protocol. Агент SNMP обычно использует 161, а диспетчер SNMP - 162.

20. Порт 69 (TFTP) - Trivial File Transfer Protocol.

4.1.3. Что такое сканирование портов?

Сканирование портов - это удалённая проверка множества портов для определения их состояния. Наиболее интересным обычно является состояние open, означающее, что приложение слушает порт и принимает на нём соединения. Существует множество методов такого сканирования. В главе 5 «Методы и алгоритмы сканирования портов» объясняется, в каких обстоятельствах каждый из них подходит лучше всего.

Хотя многие сканеры портов традиционно объединяли все порты в состояния open и closed, Nmap использует гораздо более подробную классификацию. Он делит порты на шесть состояний. Эти состояния не являются внутренними свойствами самого порта, а описывают, каким его видит Nmap. Например, сканирование Nmap из той же сети, где находится цель, может показать порт 135/tcp как open, тогда как одновременное сканирование с теми же параметрами через Интернет способно показать этот порт как filtered.

Шесть состояний портов, распознаваемых Nmap

• open — Приложение активно принимает на этом порту соединения TCP или пакеты UDP. Их обнаружение часто является основной целью сканирования портов. Люди, заботящиеся о безопасности, знают, что каждый открытый порт представляет собой путь для атаки. Злоумышленники и специалисты по тестированию на проникновение хотят воспользоваться открытыми портами, а администраторы пытаются закрыть или защитить их межсетевыми экранами, не мешая законным пользователям. Открытые порты интересны и при сканировании не ради безопасности, поскольку они показывают доступные для использования в сети службы. Прежде чем слишком радоваться открытому порту, обратите внимание: приложение может быть защищено оболочкой TCP (tcpd) либо настроено на обслуживание только разрешённых IP-адресов клиентов. Такие случаи всё равно оставляют большую поверхность атаки, чем закрытый порт.

• closed — Закрытый порт доступен: он получает пакеты проб Nmap и отвечает на них, но никакое приложение его не слушает. Такие порты помогают показать, что узел работает и использует IP-адрес, то есть служат для обнаружения узлов или ping-сканирования, а также участвуют в определении ОС. Поскольку закрытые порты достижимы, их может быть полезно просканировать позднее на случай, если некоторые откроются. Администраторам стоит подумать о блокировании таких портов межсетевым экраном, чтобы они отображались в состоянии filtered, рассматриваемом далее.

• filtered — Nmap не может определить, открыт ли порт, поскольку фильтрация пакетов не позволяет пробам достичь его. Фильтрацию может выполнять отдельное устройство межсетевого экрана, правила маршрутизатора или программный межсетевой экран узла. Такие порты разочаровывают злоумышленников, поскольку предоставляют очень мало сведений. Иногда они отвечают сообщениями об ошибках ICMP, например типа 3 с кодом 13 («цель недоступна: связь административно запрещена»), но гораздо чаще фильтры просто отбрасывают пробы, не отвечая. Это заставляет Nmap повторять попытку несколько раз на случай, если проба была отброшена из-за перегрузки сети, а не фильтрации. Такая фильтрация резко замедляет сканирование.

• `unfiltered` — Состояние `unfiltered` означает, что порт доступен, но Nmap не способен определить, открыт он или закрыт. Только ACK-сканирование, применяемое для отображения наборов правил межсетевого экрана, относит порты к этому состоянию. Сканирование нефильтруемых портов другими методами, такими как Window, SYN или FIN, может помочь установить, открыт ли порт.

• `open|filtered` — Nmap помещает порт в это состояние, когда не может определить, открыт он или фильтруется. Это происходит при тех видах сканирования, где открытые порты не отвечают. Отсутствие ответа может также означать, что фильтр пакетов отбросил пробу или вызванный ею ответ. Поэтому Nmap не знает наверняка, открыт порт или фильтруется. К этому состоянию порты относят сканирования UDP, IP protocol, FIN, NULL и Xmas.

• `closed|filtered` — Это состояние применяется, когда Nmap не может определить, закрыт порт или фильтруется. Оно используется только при сканировании IP ID Idle, рассматриваемом в разделе 5.10 «Сканирование TCP Idle (`-sI`)».

Хотя Nmap стремится выдавать точные результаты, помните, что все его выводы основаны на пакетах, возвращаемых целевыми машинами или стоящими перед ними межсетевыми экранами. Такие узлы могут быть ненадёжными и отправлять ответы, призванные запутать или ввести Nmap в заблуждение. Гораздо чаще встречаются узлы, не соответствующие RFC и отвечающие на пробы Nmap не так, как должны. Сканирования FIN, NULL и Xmas особенно подвержены этой проблеме. Подобные вопросы зависят от конкретного метода сканирования и потому рассматриваются в соответствующих разделах главы 5 «Методы и алгоритмы сканирования портов».

4.1.4. Зачем сканировать порты?

Сканирование портов выполняется не только ради развлечения. Регулярное сканирование своих сетей приносит множество практических преимуществ. Главное из них - безопасность. Один из центральных принципов сетевой безопасности состоит в том, что сокращение числа и сложности предоставляемых служб уменьшает возможности злоумышленников для проникновения. Большинство удалённых компрометаций сетей происходит из-за эксплуатации серверного приложения, слушающего порт TCP или UDP. Во многих случаях эксплуатируемое приложение даже не используется целевой организацией, а было включено по умолчанию при настройке машины. Если бы эту службу отключили или защитили межсетевым экраном, атаку удалось бы предотвратить.

Понимая, что каждый открытый порт представляет возможность для компрометации, злоумышленники регулярно сканируют цели, составляя перечень всех открытых портов. Они сравнивают этот список слушающих служб со своим набором любимых эксплойтов для уязвимого программного обеспечения. Чтобы скомпрометировать машину и создать плацдарм, который затем часто используется для заражения всей сети, достаточно одного совпадения. Менее разборчивые в выборе целей злоумышленники нередко сканируют только стандартный порт эксплуатируемого приложения. Это намного быстрее сканирования всех портов, хотя служба останется незамеченной, если работает на нестандартном порту. Таких злоумышленников часто презрительно называют «скрипт-кидди», поскольку о безопасности они зачастую знают лишь немногим больше, чем требуется для запуска написанного более умелым человеком сценария эксплуатации. Среди множества организаций они непременно находят уязвимые узлы. Они могут сильно досаждать, однако их огромное число и непрерывные удары по доступным из Интернета машинам часто заставляют людей быстро устанавливать исправления. Это снижает вероятность успеха более серьёзных целевых атак.

Важная защита от таких взломщиков - регулярное сканирование системными администраторами собственных сетей с помощью таких средств, как Nmap. Получите список открытых портов и

отключите все неиспользуемые службы. Убедитесь, что службы, которые должны оставаться доступными, полностью исправлены и что вы подписаны на уведомления поставщика о безопасности. Где возможно, следует добавить правила межсетевого экрана, ограничивающие доступ только законными пользователями. В Интернете доступны инструкции по усилению защиты большинства популярных приложений, которые ещё больше сокращают возможности взломщика. Nmap не может сделать за вас большую часть этой работы, но создаёт исходный список доступных служб. Некоторые администраторы пытаются вместо этого использовать `netstat`, но такой подход плохо масштабируется. Он требует доступа к каждой машине, а некоторые мобильные машины легко пропустить. Кроме того, нельзя запустить `netstat` на обычной беспроводной точке доступа, телефоне VoIP или принтере. Всегда существует и риск, что на скомпрометированной машине установлена троянизированная версия `netstat`, выдающая ложные сведения. Большинство современных руткитов, устанавливаемых злоумышленниками, включает такую возможность. Полагаться исключительно на Nmap тоже ошибочно. Рекомендуется сочетать тщательное проектирование, аудит конфигурации и регулярное сканирование.

Хотя безопасность является самой распространённой причиной сканирования портов, администраторы часто находят ему и другие применения. Перечень машин и предоставляемых ими служб может быть полезен для учёта активов, проектирования сети, проверки соблюдения политик, учёта лицензий программного обеспечения, проверки доступности, отладки сети и многого другого.

4.2. Краткое руководство по сканированию портов

Одна из моих целей при разработке Nmap - сохранить простоту наиболее распространённых сценариев, одновременно обеспечив гибкость для специализированных и расширенных сканирований. В интерфейсе командной строки это достигается десятками параметров и выбором разумных стандартных значений, когда они не указаны. Новичок может начать с такой простой команды, как `nmap <target>`. Тем временем опытные пользователи иногда задают столько параметров, что строка терминала переносится.

Подобное равновесие необходимо соблюдать и в выводе команды. Важнейшие результаты должны бросаться в глаза даже случайному пользователю, который не читал страницу руководства. В то же время вывод должен быть достаточно полным и кратким для профессиональных специалистов по тестированию на проникновение, ежедневно запускающих Nmap против тысяч машин. Пользователи, достаточно сообразительные, чтобы прочитать эту книгу или исходный код Nmap, получают более полный контроль над сканером и лучше понимают истинный смысл вывода Nmap.

Это руководство демонстрирует несколько распространённых сценариев сканирования портов Nmap и объясняет вывод. Его цель не во всеохватности, а лишь в том, чтобы познакомить новых пользователей с программой достаточно хорошо для понимания остальной части главы.

Самая простая команда Nmap - просто `nmap` без аргументов. Она выводит краткую памятку по распространённым параметрам и синтаксису Nmap. Более интересна команда `nmap <target>`, выполняющая следующие действия:

1. Преобразует `<target>` из имени узла в адрес IPv4 с помощью DNS. Если вместо имени узла задан IP-адрес, этот поиск пропускается.

2. Проверяет узел, по умолчанию отправляя пакет эхо-запроса ICMP и пакет TCP ACK на порт 80, чтобы определить, включён ли он и работает ли. Если нет, Nmap сообщает об этом и завершает работу. Чтобы пропустить эту проверку, можно указать `-PN`. См. главу 3 «Обнаружение узлов (ping-сканирование)».

3. Преобразует целевой IP-адрес обратно в имя с помощью обратного запроса DNS. Из-за особенностей работы DNS обратное имя может отличаться от `<target>`, указанного в командной строке. Этот запрос можно пропустить параметром `-n`, повысив скорость и скрытность.

4. Запускает сканирование TCP 1000 самых популярных портов, перечисленных в `nmap-services`. Обычно применяется скрытое SYN-сканирование, но для непривилегированных пользователей Unix, которым не хватает прав на отправку необработанных пакетов, оно заменяется сканированием `connect`.

5. Выводит результаты в стандартный поток вывода в обычном удобочитаемом формате и завершает работу. Можно задать другие форматы и места вывода, то есть файлы, как описано в главе 13 «Форматы вывода Nmap». В примере 4.2 показаны результаты использования `scanme.nmap.org` в качестве `<target>`.

Пример 4.2. Простое сканирование: nmap scanme.nmap.org

```
# nmap scanme.nmap.org
Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE  SERVICE
22/tcp  open   ssh
25/tcp  closed smtp
53/tcp  open   domain
70/tcp  closed gopher
80/tcp  open   http
113/tcp closed auth
Nmap done: 1 IP address (1 host up) scanned in 4.99 seconds
```

Первая строка вывода в примере 4.2 просто содержит URL-адрес для загрузки Nmap. Обычно также указываются время запуска и номер версии Nmap, хотя в этой книге их, как правило, удаляли для единообразия и предотвращения переноса строк.

Следующая строка содержит целевой IP-адрес, в данном случае IPv4, и имя обратного DNS, также известное как запись PTR, если оно доступно. Nmap обещает показать «интересные порты», хотя учитываются все просканированные порты. Порты, считающиеся наиболее интересными, поскольку они открыты или находятся в редко встречающемся для данного узла состоянии, перечисляются по отдельности. Когда много портов находятся в одном неоткрытом состоянии, оно считается стандартным, и порты объединяются в одной строке, чтобы не разбавлять результаты тысячами неинтересных записей. В данном случае Nmap отмечает, что фильтруются 994 порта.

Далее следует таблица интересных портов, содержащая основные результаты сканирования. Столбцы зависят от применённых параметров, но в данном случае для каждого порта приводятся его номер и протокол, состояние и протокол службы. Служба здесь - лишь предположение, сделанное по результату поиска порта в `nmap-services`. Если бы имя какого-либо порта не было зарегистрировано в этом файле, служба была бы указана как `unknown`. Три из этих портов открыты, а три закрыты.

Наконец, перед завершением Nmap сообщает основные статистические данные о времени. Это число заданных целей, число целей, которые ping-сканирование признало работающими, и общее затраченное время.

Хотя этой простой команды часто достаточно, опытные пользователи обычно идут гораздо дальше. В примере 4.3 сканирование изменено четырьмя параметрами. `-p0-` предписывает Nmap сканировать все возможные порты TCP, `-v` включает подробный вывод, `-A` - агрессивные проверки, такие как удалённое определение ОС, определение служб и версий и Nmap Scripting Engine (NSE). Наконец, `-T4` включает более агрессивную политику времени, ускоряющую сканирование.

Пример 4.3. Более сложное сканирование: nmap -p0- -v -A -T4 scanme.nmap.org

```
# nmap -p0- -v -A -T4 scanme.nmap.org
```

```
Starting Nmap ( http://nmap.org )
Completed Ping Scan at 00:03, 0.01s elapsed (1 total hosts)
Scanning scanme.nmap.org (64.13.134.52) [65536 ports]
Discovered open port 22/tcp on 64.13.134.52
Discovered open port 53/tcp on 64.13.134.52
Discovered open port 80/tcp on 64.13.134.52
SYN Stealth Scan Timing: About 6.20% done; ETC: 00:11 (0:07:33 remaining)
Completed SYN Stealth Scan at 00:10, 463.55s elapsed (65536 total ports)
Completed Service scan at 00:10, 6.03s elapsed (3 services on 1 host)
Initiating OS detection (try #1) against scanme.nmap.org (64.13.134.52)
Initiating Traceroute at 00:10
64.13.134.52: guessing hop distance at 9
Completed SCRIPT ENGINE at 00:10, 4.04s elapsed
Host scanme.nmap.org (64.13.134.52) appears to be up ... good.
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 65530 filtered ports
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain  ISC BIND 9.3.4
70/tcp  closed gopher
80/tcp  open   http    Apache httpd 2.2.2 ((Fedora))
|_ HTML title: Go ahead and ScanMe!
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.20-1 (Fedora Core 5)
Uptime guess: 2.457 days (since Thu Sep 18 13:13:24 2008)
TCP Sequence Prediction: Difficulty=204 (Good luck!)
IP ID Sequence Generation: All zeros
TRACEROUTE (using port 80/tcp)
HOP RTT   ADDRESS
[First eight hops cut for brevity]
9   10.36 metro0.sv.svcolo.com (208.185.168.173)
10  10.29 scanme.nmap.org (64.13.134.52)
Nmap done: 1 IP address (1 host up) scanned in 477.23 seconds
       Raw packets sent: 131432 (5.783MB) | Rcvd: 359 (14.964KB)
```

В примере 4.3 Nmap определённо предоставил запрошенную подробность! К счастью, дополнительный вывод легко понять. Первые 13 новых строк содержат сведения о ходе работы и сообщают пользователю, что происходит, пока он с надеждой смотрит на терминал в ожидании хороших новостей. Что считать хорошей новостью, зависит от того, является ли пользователь системным администратором, которому предстоит исправлять проблемы, специалистом по тестированию на проникновение, которому нужны вопросы для отчёта, или взломщиком в чёрной шляпе, пытающимся ими воспользоваться. Для краткости удалено ещё около десятка подобных строк. Строки `discovered open port` сообщают об открытых портах по мере их обнаружения, чтобы пользователь мог начать работу с ними ещё до завершения сканирования. Строка `scan timing` содержит оценку времени завершения, позволяя решить, продолжать ли смотреть на экран или пойти пообедать. Поскольку условия сети - задержка, перегрузка, пропускная способность и прочие - и правила фильтрации пакетов сильно различаются, сканирование с одними и теми же параметрами может занять 30 секунд для одного узла и 45 минут для другого. Чтобы во время сканирования узнать текущую оценку времени, просто нажмите Enter.

Таблица портов не показывает новых портов. Все дополнительные просканированные порты находятся в состоянии `filtered`, из-за чего общее число фильтруемых портов выросло с 994 до 65 530. Хотя новых отдельно перечисленных портов нет, записи изменились. Новый столбец `VERSION` содержит имя приложения и сведения о версии слушающей службы. Эти данные получены посредством определения служб - одной из функций, включаемых параметром `-A`. Ещё одна особенность определения служб состоит в том, что теперь все протоколы служб в столбце `SERVICE` действительно проверены. При предыдущем сканировании они основывались на сравнительно ненадёжной эвристике поиска номера порта в `nmap-services`. На этот раз результат поиска в таблице оказался правильным, но так бывает не всегда.

Другая функция, добавленная параметром `-A`, - Nmap Scripting Engine, подробно рассматриваемая в главе 9 «Движок сценариев Nmap». Единственный показанный здесь сценарий - `HTML title`. Существуют десятки других сценариев, но ни один из них не нашёл полезного вывода для этой машины. Параметр `-A` добавил и результаты трассировки. Этот вариант эффективнее и мощнее большинства программ `traceroute`, поскольку пробы выполняются параллельно, а Nmap использует результаты сканирования для выбора подходящего типа пробы, в данном случае пакетов TCP на порт 80.

Большинство оставшихся новых строк получено в результате определения ОС, также включённого параметром `-A` и подробно рассматриваемого в главе 8 «Удалённое определение ОС». Последняя строка показывает цену всех этих дополнительных сведений: сканирование заняло почти в 100 раз больше времени, чем пример 4.2 «Простое сканирование: nmap scanme.nmap.org» - 477 секунд вместо 5.

4.3. Параметры командной строки

Хотя руководство показало, насколько просто запустить сканирование портов Nmap, для повышения мощности и гибкости системы доступны десятки флагов командной строки. В этом разделе рассматриваются только параметры, связанные со сканированием портов, и нередко описывается лишь относящаяся к нему часть их функциональности. Полный список флагов и всех выполняемых ими действий приведён в главе 15 «Справочное руководство Nmap».

4.3.1. Выбор методов сканирования

Один из первых вопросов при планировании сканирования портов - какие методы применять. Nmap предлагает около дюжины таких методов, и в этом разделе приведено их краткое описание. Полное рассмотрение следует в следующей главе. Одновременно можно применять только один метод сканирования, за исключением того, что сканирование UDP (`-sU`) допускается сочетать с любым одним типом сканирования TCP. Для облегчения запоминания параметры типов сканирования портов имеют вид `-s<C>`, где `<C>` - заметная буква в названии метода, обычно первая. Единственное исключение - устаревшее сканирование с отражением от FTP (`-b`). По умолчанию Nmap выполняет SYN-сканирование, но заменяет его сканированием `connect`, если пользователю не хватает прав для отправки необработанных пакетов, что в Unix требует доступа `root`, либо если указаны цели IPv6.

Методы сканирования портов, поддерживаемые Nmap

• **TCP SYN Stealth (`-sS`)** — Это безусловно самый популярный тип сканирования, поскольку он представляет собой самый быстрый способ сканирования портов наиболее популярного протокола TCP. Он скрытнее сканирования `connect` и работает со всеми исправными стеками TCP, в отличие от некоторых специализированных методов, например FIN-сканирования.

• **TCP Connect (`-sT`)** — Сканирование `connect` использует для сканирования машин одноимённый системный вызов, а не необработанные пакеты, на которые опирается большинство других методов. Обычно его применяют непривилегированные пользователи Unix и при работе с целями IPv6, поскольку SYN-сканирование в таких случаях не работает.

• **UDP (`-sU`)** — Не забывайте о портах UDP - они тоже предоставляют множество дыр в безопасности.

• **TCP FIN, Xmas и Null (**`-sF`**,** `-sX`**,** `-sN`**)** — Эти специализированные типы сканирования хорошо умеют незаметно проходить межсетевые экраны для исследования находящихся за ними систем. К сожалению, они зависят от поведения цели, которого некоторые системы, особенно разновидности Windows, не демонстрируют.

• **TCP ACK (**`-sA`**)** — ACK-сканирование обычно применяется для отображения наборов правил межсетевых экранов. В частности, оно помогает понять, учитывают ли правила состояние соединения. Недостаток состоит в невозможности отличить открытые порты от закрытых.

• **TCP Window (**`-sW`**)** — Window-сканирование подобно ACK-сканированию, но способно отличать открытые порты от закрытых на некоторых машинах.

• **TCP Maimon (**`-sM`**)** — Этот малоизвестный тип сканирования для обхода межсетевых экранов похож на FIN-сканирование, но дополнительно включает флаг ACK. Благодаря этому он проходит больше межсетевых экранов с фильтрацией пакетов, однако работает с ещё меньшим числом систем, чем FIN-сканирование.

• **TCP Idle (**`-sI <zombie host>`**)** — Idle-сканирование - самый скрытный тип из всех, и иногда он позволяет использовать отношения доверия между IP-адресами. К сожалению, он также медленный и сложный.

• **IP Protocol (**`-sO`**)** — Сканирование протоколов определяет, какие протоколы IP - TCP, ICMP, IGMP и другие - поддерживает целевая машина. Технически это не сканирование портов, поскольку перебираются номера протоколов IP, а не номера портов TCP или UDP. Тем не менее для выбора сканируемых номеров протоколов по-прежнему применяется `-p`, результаты выводятся в обычном формате таблицы портов и используется тот же базовый механизм сканирования, что и у настоящих методов сканирования портов. Поэтому этот метод достаточно близок к сканированию портов, чтобы рассматриваться здесь.

• **TCP FTP Bounce (**`-b <FTP bounce proxy>`**)** — Этот устаревший тип сканирования обманом заставляет серверы FTP выполнять сканирование портов через посредника. Теперь большинство серверов FTP исправлено и предотвращает такую возможность, но когда метод срабатывает, он хорошо подходит для незаметного прохождения строгих межсетевых экранов.

4.3.2. Выбор сканируемых портов

Файл регистрации портов Nmap (`nmap-services`) содержит эмпирические данные о том, насколько часто каждый порт TCP или UDP обнаруживается открытым. Эти данные были собраны путём сканирования десятков миллионов адресов Интернета и объединения результатов с данными внутренних сканирований, предоставленными крупными предприятиями. По умолчанию Nmap сканирует 1000 самых популярных портов каждого запрошенного протокола. Вместо этого можно указать `-F` (быстро), чтобы сканировать только 100 самых распространённых портов каждого протокола, либо `--top-ports`, чтобы задать произвольное число сканируемых портов.

Если ни один из этих готовых наборов портов не соответствует вашим потребностям, в командной строке можно задать произвольный список номеров с помощью `-p`. Синтаксис `-p` бывает сложным, и лучше всего описать его примерами.

Примеры выбора портов с параметром -p

• `-p 22` — Сканировать один порт, в данном случае 22, просто указав его номер как аргумент `-p`.

• `-p ssh` — Вместо номеров можно задавать имена портов. Обратите внимание, что одно имя может соответствовать нескольким портам.

• `-p 22,25,80` — Несколько портов можно разделять запятыми. Поскольку протокол не задан, одни и те же номера будут использоваться для любых методов сканирования, указанных в командной строке. Если задано сканирование TCP, например SYN (`-sS`), сканируются порты TCP 22, 25 и 80, соответствующие службам SSH, SMTP и HTTP. Если выбрано сканирование UDP (`-sU`), сканируются эти три порта UDP. Если заданы оба варианта, три порта сканируются для каждого протокола, всего шесть портов. При сканировании протоколов IP (`-sO`) проверяются три протокола IP, соответствующие XNS IDP, Leaf-1 и ISO-IP.

• `-p80-85,443,8000-8005,8080-8085` — Диапазоны портов задаются дефисом между начальным и конечным портами. Несколько диапазонов или отдельных портов можно разделять запятыми. Этот параметр сканирует порты 80, 81, 82, 83, 84, 85, 443, 8000 и так далее. Судя по номерам, пользователь, вероятно, сканирует TCP в поисках веб-серверов.

• `-p-100,60000-` — Начало диапазона можно опустить, подразумевая порт 1, а конец - подразумевая последний возможный порт: 65 535 для TCP и UDP либо 255 при сканировании протоколов. В этом примере сканируются порты от 1 до 100 и все порты с номером не менее 60 000.

• `-p-` — Опустите начальный и конечный номера, чтобы просканировать весь диапазон, кроме нуля.

• `-pT:21,23,110,U:53,111,137,161` — Отдельные списки портов TCP и UDP можно задать, поставив перед ними `T:` для TCP или `U:`. В этом примере сканируются три порта TCP - FTP, Telnet и POP3 - и четыре службы UDP - DNS, `rpcbind`, NetBIOS и SNMP. Отдельное указание портов TCP и UDP имеет смысл, только если Nmap также предписано выполнить сканирование UDP (`-sU`) и один из методов TCP, например `-sS`, `-sA` или `-sF`.

• `-p http*` — Для сопоставления портов с похожими именами можно применять подстановочные знаки. Это выражение соответствует восьми номерам портов, включая `http` (80), `http-mgmt` (280), `https` (443) и `http-proxy` (8080). В зависимости от командной оболочки звёздочку может потребоваться экранировать, чтобы она не трактовалась как шаблон имени файла.

• `-p 1-1023,[1024-]` — Если заключить диапазон в квадратные скобки, номера портов будут сканироваться только при их регистрации в `nmap-services`. В этом примере сканируются все зарезервированные порты от 1 до 1023, а также все более высокие порты, зарегистрированные в `nmap-services`. Таково было стандартное поведение Nmap до того, как `nmap-services` дополнили данными о частоте открытых портов для более точного выбора.

4.3.3. Параметры, связанные со временем

Сканирование портов часто является самой продолжительной частью сканирования Nmap, которое может также включать определение ОС, определение версий и сценарии NSE. Хотя по умолчанию Nmap старается работать быстро и эффективно, ручная оптимизация часто помогает. Nmap предлагает десятки параметров для настройки интенсивности и скорости сканирования в точном соответствии с вашими потребностями. В этом разделе перечислены важнейшие параметры оптимизации времени сканирования портов. Значения параметров времени задаются в миллисекундах, если к числу не добавлено `s` (секунды), `m` (минуты) или `h` (часы). Дополнительные сведения о каждом параметре приведены в разделе 15.11 «Время и производительность». Гораздо более подробное рассмотрение с примерами и передовыми методами повышения производительности Nmap доступно в главе 6 «Оптимизация производительности Nmap».

Основные параметры производительности сканирования портов

• `-T0` - `-T5` — Эти шаблоны времени влияют на множество переменных, предоставляя простой способ настроить общую скорость Nmap от очень медленной (`-T0`) до чрезвычайно агрессивной (`-T5`). Шаблон времени можно сочетать с более точными параметрами, описанными ниже, причём наиболее точный параметр имеет приоритет.

• `--min-rtt-timeout`, `--max-rtt-timeout`, `--initial-rtt-timeout` — Минимальный, максимальный и начальный период, в течение которого Nmap ожидает ответ на пробу сканирования портов.

• `--host-timeout` — Предписывает Nmap отказаться от сканирования узлов, на которые затрачено больше заданного времени.

• `--min-rate`, `--max-rate` — Задают соответственно нижнюю и верхнюю границы числа пакетов проб, отправляемых Nmap в секунду.

• `--max-retries` — Задаёт максимальное число повторных передач проб сканирования портов одному порту.

• `--min-hostgroup`, `--max-hostgroup` — Задают минимальное и максимальное число узлов, порты которых Nmap сканирует параллельно.

• `--min-parallelism`, `--max-parallelism` — Ограничивают минимальное или максимальное число проб сканирования портов для всех одновременно сканируемых узлов, которые могут ожидать ответа.

• `--scan-delay`, `--max-scan-delay` — Предписывают Nmap ожидать не менее заданного времени между отправкой проб каждому отдельному узлу. Задержка сканирования может расти, когда Nmap обнаруживает потерю пакетов, поэтому с помощью `--max-scan-delay` можно задать её максимум.

4.3.4. Параметры формата и подробности вывода

Nmap позволяет записывать отчёты в стандартном формате, простом построчном формате, пригодном для `grep`, или в XML. Эти отчёты включаются параметрами `-oN` (обычный), `-oG` (пригодный для `grep`) и `-oX` (XML). Каждый параметр принимает имя файла; их можно сочетать, одновременно выводя данные в нескольких форматах. Для повышения подробности вывода доступны разные параметры. В этом разделе перечислены важнейшие параметры вывода и описано их применение к сканированию портов. Дополнительные сведения о каждом из них приведены в разделе 15.13 «Вывод».

Гораздо более подробное рассмотрение параметров и форматов вывода со множеством примеров приведено в главе 13 «Форматы вывода Nmap».

Основные параметры вывода Nmap, применимые к сканированию портов

• `-v` — Повышает уровень подробности, заставляя Nmap выводить больше сведений о выполняющемся сканировании. Открытые порты показываются по мере обнаружения, а если Nmap считает, что сканирование займёт больше нескольких минут, приводится оценка времени завершения. Укажите параметр дважды или больше для ещё большей подробности.

• `-d` — Повышает уровень отладки, заставляя Nmap выводить подробности работы, полезные для поиска ошибок или просто понимания принципов программы. Более высокие уровни дают огромные объёмы данных. Однократное применение параметра устанавливает уровень отладки 1, и каждое

дополнительное -d увеличивает его. Можно также указать после -d требуемый уровень, например -d5. Если сведений недостаточно, попробуйте более высокий уровень. Максимальный эффективный уровень - 9. Если экран затоплен избыточными данными отладки, уменьшите уровень. Снижение интенсивности сканирования, например числа сканируемых портов или целей и используемых функций, также помогает выделить только нужные сообщения отладки.

• --packet-trace — Заставляет Nmap выводить сводку каждого отправленного или полученного пакета. Часто это применяется для отладки, но также служит ценным способом для новых пользователей точно понять, что Nmap делает внутри. Чтобы не выводить тысячи строк, можно задать небольшое число сканируемых портов, например -p20-30.

• -oN <filename> **(обычный вывод)** — Записывает вывод в обычном формате Nmap в файл <filename>. Этот формат примерно совпадает со стандартным интерактивным выводом, печатаемым Nmap во время работы.

• -oX <filename> **(вывод XML)** — Записывает вывод в формате XML Nmap в файл <filename>. Обычный удобочитаемый вывод по-прежнему печатается в stdout, если только вы не попросите направить туда XML, указав - в качестве <filename>. Это предпочтительный формат для сценариев и программ, обрабатывающих результаты Nmap.

• -oG <filename> **(вывод в формате, пригодном для** grep**)** — Записывает вывод в так называемом формате Nmap, пригодном для grep, в файл <filename>. Этот табличный формат помещает вывод каждого узла в одну строку, облегчая поиск с помощью grep открытых портов, определённых операционных систем, имён приложений и других данных. Обычный вывод по-прежнему печатается в stdout, если только вы не попросите направить туда пригодный для grep вывод, указав - в качестве <filename>. Хотя этот формат хорошо подходит для разбора простыми командными строками grep и awk, значительным сценариям и программам следует использовать XML. Формат XML содержит важные сведения, для которых в пригодном для grep формате нет места, а расширяемость позволяет легче добавлять в XML новые данные, не ломая зависящие от него средства.

• -oA <basename> **(вывод во всех форматах)** — Для удобства можно задать -oA <basename>, чтобы одновременно сохранить результаты сканирования в обычном, XML и пригодном для grep форматах. Они записываются соответственно в <basename>.nmap, <basename>.xml и <basename>.gnmap. Как и в большинстве программ, перед именами файлов можно указать путь к каталогу, например ~/nmaplogs/foocorp/ в Unix или C:\hacking\sco в Windows.

• --resume <filename> — Возобновляет прерванное сканирование по обычному (-oN) или пригодному для grep (-oG) файлу вывода, созданному во время неудачного запуска. Не используйте никакие параметры, кроме --resume, поскольку Nmap возьмёт их из файла вывода. Затем программа разбирает файл и возобновляет сканирование и запись в него с того узла, над которым предыдущий запуск Nmap работал в момент остановки.

• --append-output — Предписывает Nmap добавлять результаты сканирования в заданные файлы вывода с такими аргументами, как -oN или -oX, а не перезаписывать их.

• --open — Показывает в таблицах интересных портов Nmap только открытые порты.

4.3.5. Параметры обхода межсетевых экранов и IDS

Nmap предлагает множество параметров для незаметного прохождения IDS и обхода правил межсетевых экранов. Обзор приведён в разделе 15.12 «Обход межсетевых экранов/IDS и подмена». Всестороннее рассмотрение методов обхода межсетевых экранов и IDS с практическими примерами см. в главе 10 «Обнаружение и обход межсетевых экранов и систем

обнаружения вторжений».

4.3.6. Указание целей

Чтобы просканировать один или несколько узлов, просто добавьте их имена или IP-адреса в конец командной строки Nmap. В Nmap также есть структурированный синтаксис, упрощающий сканирование крупных сетей. Можно передать Nmap файл со списком целей или даже попросить программу создать их случайным образом. Всё это описано в разделе 3.2 «Указание целевых узлов и сетей».

4.3.7. Прочие параметры

Ниже приведены параметры, которые могут оказаться весьма полезными, хотя не относятся к определённым категориям. Описания сосредоточены на связи каждого параметра со сканированием портов. Более полное рассмотрение каждого из них приведено в главе 15 «Справочное руководство Nmap».

• `-6` — Предписывает Nmap сканировать цель по протоколу IPv6. Этот процесс описан в разделе 4.4 «Сканирование IPv6 (`-6`)».

• `-r` — По умолчанию Nmap перемешивает порядок сканирования портов, немного затрудняя обнаружение. Параметр `-r` заставляет сканировать их по порядку номеров.

• `-PN` — Предписывает Nmap пропустить ping-проверку и просто сканировать каждый заданный целевой узел. Другие параметры управления обнаружением узлов описаны в главе 3 «Обнаружение узлов (ping-сканирование)».

• `--reason` — Добавляет к таблице интересных портов столбец, объясняющий, почему Nmap классифицировал порт именно так.

4.4. Сканирование IPv6 (-6)

С 2002 года Nmap поддерживает IPv6 для своих самых популярных функций. В частности, IPv6 поддерживают ping-сканирование только по TCP, сканирование `connect` и определение версий. Синтаксис команд остаётся обычным, но дополнительно указывается `-6`. Разумеется, если вместо имени узла задаётся адрес, нужно применять синтаксис IPv6. Адрес может выглядеть как `3ffe:7501:4819:2000:210:f3ff:fe03:14d0`, поэтому рекомендуется использовать имена узлов. В примере 4.4 показан типичный сеанс сканирования портов. Вывод выглядит как обычно; единственным признаком IPv6 является адрес IPv6 в строке «интересных портов».

Пример 4.4. Простое сканирование IPv6

```
# nmap -6 -sV www.eurov6.org
Starting Nmap ( http://nmap.org )
Interesting ports on ns1.euro6ix.com (2001:800:40:2a03::3):
Not shown: 996 closed ports
PORT   STATE SERVICE VERSION
21/tcp open  ftp     Pure-FTPd
22/tcp open  ssh     OpenSSH 3.5p1 (protocol 2.0)
53/tcp open  domain  ISC BIND 9.2.1
80/tcp open  http    Apache httpd
Nmap done: 1 IP address (1 host up) scanned in 56.78 seconds
```

Хотя IPv6 не то чтобы покорил мир, в некоторых странах он широко применяется, и большинство современных операционных систем его поддерживает. Для применения Nmap с IPv6 и исходная, и целевая стороны сканирования должны быть настроены для IPv6. Если поставщик услуг Интернета, как большинство из них, не выделяет вам адреса IPv6, широко доступны бесплатные туннельные брокеры, нормально работающие с Nmap. Я пользуюсь бесплатной службой туннельного брокера IPv6 по адресу http://www.tunnelbroker.net. Другие туннельные брокеры перечислены в Википедии.[1] Туннели 6to4 - ещё один популярный бесплатный подход.

В системах с поддержкой IPv6 правила межсетевого экрана IPv4 и IPv6 не всегда синхронизированы. В разделе 10.4.3 «Атаки IPv6» приведён реальный пример доступа через IPv6 к портам, фильтруемым в IPv4.

4.5. РЕШЕНИЕ: сканирование крупной сети в поисках определённого открытого порта TCP

4.5.1. Задача

Требуется быстро найти все машины сети, на которых открыт определённый порт TCP. Например, после обнаружения новой уязвимости Microsoft IIS может понадобиться найти все машины с открытым портом TCP 80 и удостовериться, что на них не работает уязвимая версия этого программного обеспечения. Или при исследовании скомпрометированной машины вы обнаружили, что злоумышленник оставил бэкдор на порту 31337; сканирование всей сети на наличие этого порта поможет быстро выявить другие скомпрометированные системы. Полное сканирование всех портов будет выполнено позднее.

4.5.2. Решение

Прямолинейный способ - выполнить:

```
nmap -PN -p<portnumber> -oG <logfilename.gnmap> <target networks>
```

Вот конкретный пример поиска веб-серверов, то есть открытого порта 80, среди 4096 IP-адресов:

```
nmap -PN -p80 -oG logs/pb-port80scan-%D.gnmap 216.163.128.0/20
```

Последовательность `%D` в имени файла заменяется числовой датой выполнения сканирования, например `090107` для 1 сентября 2007 года. Эта команда сканирования работает, но небольшие усилия по выбору подходящих значений времени для сканируемой сети существенно сокращают продолжительность. Приведённое выше сканирование заняло 1236 секунд, а оптимизированная версия ниже дала те же результаты за 869 секунд:

```
nmap -T4 -PN -p80 --max-rtt-timeout 200 --initial-rtt-timeout 150 --min-hostgroup 512 -oG
    \
logs/pb-port80scan2-%D.gnmap 216.163.128.0/20
```

Значительная часть этого времени тратится на обратное разрешение DNS. Его исключение путём добавления `-n` к приведённой выше командной строке сокращает время сканирования 4096 узлов до 193 секунд. Проявить терпение в течение трёх минут гораздо легче, чем ждать прежнюю 21 минуту.

Приведённые выше команды сохраняют результаты в указанном файле в формате, пригодном для grep. Затем простая команда egrep находит машины с открытым портом 80:

```
egrep '[^0-9]80/open' logs/pb-port80scan2-*.gnmap
```

Перед шаблоном egrep стоит [^0-9], чтобы не получить ложное совпадение с такими портами, как 3180. Разумеется, здесь это невозможно, поскольку сканируется только порт 80, но такой приём полезно помнить для сканирований множества портов. Если нужны только IP-адреса и ничего больше, передайте вывод egrep команде awk '{print $2}'.

4.5.3. Обсуждение

Иногда история лучше всего помогает понять принятые решения, например мой выбор командных строк в разделе с решением. Мне было скучно дома, и я начал исследовать сеть популярного журнала Playboy. Основной сайт содержит огромное собрание изображений, но большинство из них скрыто за платной системой аутентификации подписчиков. Мне стало интересно, удастся ли найти в их сети другие системы, бесплатно предоставляющие изображения. Я предположил, что у них могут быть промежуточные серверы или серверы разработки, полагающиеся на неизвестность, а не на парольную аутентификацию. Теоретически такие серверы способны слушать любой номер порта, но наиболее вероятен порт TCP 80. Поэтому я решил как можно быстрее просканировать всю их сеть на наличие этого открытого порта.

Первый шаг - определить сканируемые IP-адреса. Я выполняю поиск whois в American Registry for Internet Numbers (ARIN) организаций с именем Playboy. Результаты показаны в примере 4.5.

Пример 4.5. Обнаружение адресного пространства IP Playboy

```
core~> whois -h whois.arin.net n playboy
[Querying whois.arin.net]
[whois.arin.net]

OrgName:    Playboy
OrgID:      PLAYBO
Address:    680 N. Lake Shore Drive
City:       Chicago
StateProv:  IL
PostalCode: 60611
Country:    US

NetRange:   216.163.128.0 - 216.163.143.255
CIDR:       216.163.128.0/20
NetName:    PLAYBOY-BLK-1
NetHandle:  NET-216-163-128-0-1
Parent:     NET-216-0-0-0-0
NetType:    Direct Assignment
NameServer: NS1-CHI.PLAYBOY.COM
NameServer: NS2-CHI.PLAYBOY.COM
[...]
```

Это показывает 4096 IP-адресов - сетевой диапазон 216.163.128.0/20, зарегистрированный за Playboy. С помощью методов из раздела 3.3 «Поиск IP-адресов организации» я мог бы найти гораздо больше контролируемых ею сетевых блоков, но для этого примера достаточно 4096 адресов.

Далее я хочу оценить задержку до этих машин, чтобы Nmap знал, чего ожидать. Это необязательно, но передача Nmap подходящих значений времени может ускорить программу. Особенно это справедливо для однопортовых сканирований -PN, подобных этому. Nmap получает недостаточно ответов от каждого узла для точной оценки задержки и доли потерянных пакетов, поэтому я помогу ему в командной строке. Сначала мне приходит в голову проверить с помощью ping их основной веб-сервер, как показано в примере 4.6.

Пример 4.6. Ping-проверка веб-сервера Playboy для оценки задержки

```
# ping -c5 www.playboy.com
PING www.phat.playboy.com (209.247.228.201) from 205.217.153.56
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=1 time=57.5 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=2 time=56.7 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=3 time=56.9 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=4 time=57.0 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=5 time=56.6 ms

--- www.phat.playboy.com ping statistics ---
5 packets transmitted, 5 received, 0% loss, time 4047ms
rtt min/avg/max/mdev = 56.652/57.004/57.522/0.333 ms
```

Максимальное время прохождения туда и обратно составляет 58 миллисекунд. К сожалению, этот IP-адрес (`209.247.228.201`) не входит в сетевой блок `216.163.128.0/20`, который я хочу просканировать. Обычно я добавил бы новый блок в список целей, но уже решил ограничить сканирование исходными 4096 IP-адресами. Эти значения времени, вероятно, вполне можно использовать, однако получить действительные значения для адресов целевой сети было бы ещё лучше. С помощью `dig` я получаю общедоступные записи DNS Playboy от сервера имён, показанного в предыдущем запросе `whois`. Вывод приведён в примере 4.7.

Пример 4.7. Исследование записей DNS Playboy с помощью Dig

```
e~> dig @ns1-chi.playboy.com playboy.com. any
<<>> DiG 8.3 <<>> @ns1-chi.playboy.com playboy.com. any

playboy.com.  1D IN A    209.247.228.201
playboy.com.  1D IN MX   10 mx.la.playboy.com.
playboy.com.  1D IN MX   5 mx.chi.playboy.com.
playboy.com.  1D IN NS   ns15.customer.level3.net.
playboy.com.  1D IN NS   ns21.customer.level3.net.
playboy.com.  1D IN NS   ns29.customer.level3.net.
playboy.com.  1D IN NS   ns1-chi.playboy.com.
playboy.com.  1D IN NS   ns2-chi.playboy.com.
playboy.com.  1D IN SOA  ns1-chi.playboy.com. dns.playboy.com.
                         2004092010     serial
                         12H            refresh
                         2h30m          retry
                         2w1d           expiry
                         1D             minimum

ADDITIONAL SECTION:
mx.chi.playboy.com.  1D IN A  216.163.143.4
mx.la.playboy.com.   1D IN A  216.163.128.15
ns1-chi.playboy.com. 1D IN A  209.247.228.135
ns2-chi.playboy.com. 1D IN A  64.202.105.36

Total query time: 107 msec
```

Запрос DNS обнаруживает два сервера MX (почты) в целевом сетевом блоке `216.163.128.0/20`. Поскольку имена `mx.chi` и `mx.la` указывают, что они находятся в разных регионах - Чикаго и Лос-Анджелесе, - я решаю проверить задержку до обоих. Результаты `ping` показаны в примере 4.8.

Пример 4.8. Ping-проверка серверов MX

```
core~> ping -c5 mx.chi.playboy.com
PING mx.chi.playboy.com (216.163.143.4) 56(84) bytes of data.

--- mx.chi.playboy.com ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4000ms

core~> ping -c5 mx.la.playboy.com
PING mx.la.playboy.com (216.163.128.15) 56(84) bytes of data.

--- mx.la.playboy.com ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4011ms
```

Что ж, эта попытка завершилась полным провалом! Похоже, узлы блокируют пакеты ICMP ping. Поскольку это почтовые серверы, у них должен быть открыт порт TCP 25, поэтому я повторяю попытку с помощью `hping2`[2], выполняя TCP ping порта 25, как показано в примере 4.9.

Пример 4.9. TCP ping-проверка серверов MX

```
core# hping2 --syn -p 25 -c 5 mx.chi.playboy.com
eth0 default routing interface selected (according to /proc)
HPING mx.chi.playboy.com (eth0 216.163.143.4): S set, 40 headers + 0 data bytes
46 bytes from 216.163.143.4: flags=SA seq=0 ttl=51 id=14221 rtt=56.8 ms
46 bytes from 216.163.143.4: flags=SA seq=1 ttl=51 id=14244 rtt=56.9 ms
46 bytes from 216.163.143.4: flags=SA seq=2 ttl=51 id=14274 rtt=56.9 ms
46 bytes from 216.163.143.4: flags=SA seq=3 ttl=51 id=14383 rtt=61.8 ms
46 bytes from 216.163.143.4: flags=SA seq=4 ttl=51 id=14387 rtt=57.5 ms

--- mx.chi.playboy.com hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 56.8/58.0/61.8 ms

core# hping2 --syn -p 25 -c 5 mx.la.playboy.com
eth0 default routing interface selected (according to /proc)
HPING mx.la.playboy.com (eth0 216.163.128.15): S set, 40 headers + 0 data bytes
46 bytes from 216.163.128.15: flags=SA seq=0 ttl=52 id=58728 rtt=16.0 ms
46 bytes from 216.163.128.15: flags=SA seq=1 ttl=52 id=58753 rtt=15.4 ms
46 bytes from 216.163.128.15: flags=SA seq=2 ttl=52 id=58790 rtt=15.5 ms
46 bytes from 216.163.128.15: flags=SA seq=3 ttl=52 id=58870 rtt=16.4 ms
46 bytes from 216.163.128.15: flags=SA seq=4 ttl=52 id=58907 rtt=15.5 ms

--- mx.la.playboy.com hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 15.4/15.8/16.4 ms
```

Это именно те результаты, которые я искал. Узел в Лос-Анджелесе всегда отвечает не более чем за 16 миллисекунд, а чикагскому требуется до 62 миллисекунд. Это неудивительно, поскольку я отправляю пробы с машины в Калифорнии. Осторожность не повредит, а при интенсивном сканировании задержка может увеличиваться, поэтому я решаю позволить Nmap ожидать ответы до 200 миллисекунд. Начальный период ожидания составит 150 мс. Поэтому я передаю параметры `--max-rtt-timeout 200 --initial-rtt-timeout 150`. Чтобы задать в целом агрессивный режим времени, в начале строки я указываю `-T4`.

Поскольку сокращение общего времени завершения сканирования для меня важнее сокращения ожидания первой группы результатов по узлам, я задаю большую группу сканирования. Параметр `--min-hostgroup 512` требует по возможности параллельно сканировать не менее 512 IP-адресов. Точный делитель размера целевой сети (4096) предотвращает появление в конце маленького и менее эффективного блока из 96 узлов, которое произошло бы при `--min-hostgroup 500`. Все эти вопросы времени гораздо подробнее объясняются в главе 6 «Оптимизация производительности Nmap».

Нет смысла тратить время на предварительный этап ping, поскольку он занял бы столько же, сколько само сканирование одного порта. Поэтому для его отключения задаётся `-PN`. Значительное время экономится за счёт пропуска обратного разрешения DNS параметром `-n`. Иначе при отключённом ping-сканировании Nmap попытался бы найти имена всех 4096 IP-адресов. Я ищу веб-серверы, поэтому запрашиваю порт 80 с помощью `-p80`. Разумеется, будут пропущены все серверы HTTP на нестандартных портах, таких как 81 или 8080. Не будут найдены и серверы SSL на порту 443. Эти порты можно добавить к параметру `-p`, но даже один дополнительный порт удвоит время сканирования, которое приблизительно пропорционально числу сканируемых портов.

Последний параметр - `-oG`, после которого следует имя файла для сохранения результатов в формате, пригодном для `grep`. Я добавляю к команде целевую сеть и нажимаю Enter, чтобы запустить Nmap. Вывод показан в примере 4.10.

Пример 4.10. Запуск сканирования

```
# nmap -T4 -p80 -PN --max-rtt-timeout 200 --initial-rtt-timeout 150 \
  --min-hostgroup 512 -n -oG pb-port80scan-%D.gnmap 216.163.128.0/20
Warning: You specified a highly aggressive --min-hostgroup.
Starting Nmap ( http://nmap.org )
Interesting ports on 216.163.128.0:
PORT   STATE SERVICE

Interesting ports on 216.163.128.1:
PORT   STATE    SERVICE
80/tcp filtered http

Interesting ports on 216.163.128.2:
PORT   STATE SERVICE

Interesting ports on 216.163.128.3:
PORT   STATE SERVICE

Interesting ports on 216.163.143.255:
PORT   STATE    SERVICE
80/tcp filtered http
Nmap done: 4096 IP addresses (4096 hosts up) scanned in 192.97 seconds
```

Nmap сканирует все 4096 IP-адресов примерно за три минуты. Обычный вывод показывает множество портов в состоянии `filtered`. Большинство этих IP-адресов, вероятно, не относятся к активным узлам: порт просто выглядит фильтруемым, поскольку Nmap не получает ответа на свои пробы SYN. Список веб-серверов я получаю простой командой `egrep` для файла вывода, как показано в примере 4.11.

Пример 4.11. Поиск открытых портов с помощью Egrep

```
# egrep '[^0-9]80/open' pb-port80scan-*.gnmap
Host: 216.163.140.20 () Ports: 80/open/tcp//http///
Host: 216.163.142.135 () Ports: 80/open/tcp//http///
```

После всех усилий среди 4096 IP-адресов обнаружено лишь два доступных веб-сервера! Иногда так бывает. Первый, `216.163.140.20`, без имени обратного DNS, приводит меня к серверу Microsoft Outlook Web Access, то есть веб-почты. Это могло бы обрадовать меня, если бы я пытался скомпрометировать их сеть, но сейчас результат не радует. Следующий сервер с обратным именем `mirrors.playboy.com` гораздо лучше. Он предоставляет те гигабайты бесплатных образов, на которые я надеялся! В частности, там есть образы ISO Linux, а также обширные архивы FreeBSD, CPAN и Apache. Я загружаю последние ISO Fedora Core с приличной скоростью 6 Мбит/с. Обилие пропускной способности у Playboy неудивительно. Позднее я сканирую другие сетевые блоки Playboy и нахожу ещё десятки веб-серверов, хотя часть их содержимого не подходит для этой книги.

Хотя это необычная причина сканирования портов, проходы по одному порту часто применяются для многих других целей, упомянутых ранее. Рассмотренные здесь методы легко применить к любому проходу по одному порту TCP.

4.5.4. См. также

Определение версий можно применять для поиска в сети конкретных слушающих приложений. Например, можно искать определённую уязвимую версию OpenSSH вместо всех узлов с открытым портом 22. Это полезно и для однопортового сканирования UDP, поскольку методы данного решения хорошо работают только с TCP. Инструкции приведены в разделе 7.8 «РЕШЕНИЕ: поиск всех серверов с небезопасной или нестандартной версией приложения».

В главе 6 «Оптимизация производительности Nmap» оптимизация скорости сканирования рассматривается гораздо подробнее.

Сноски

[1] [назад] http://en.wikipedia.org/wiki/List_of_IPv6_tunnel_brokers
[2] [назад] http://www.hping.org

Глава 5. Методы и алгоритмы сканирования портов

5.1. Введение

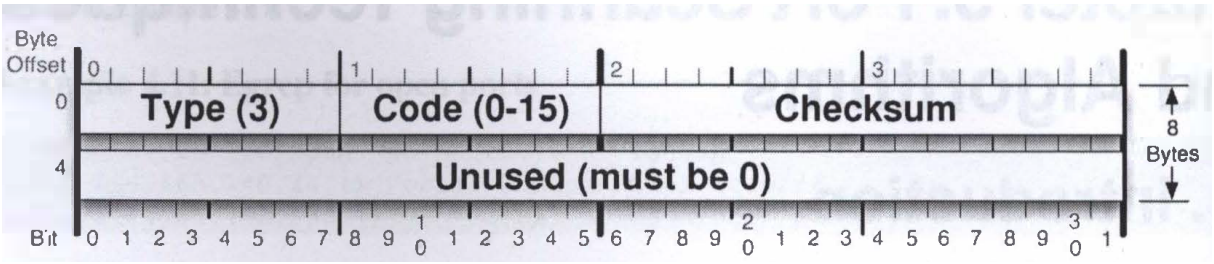
Будучи новичком в ремонте автомобилей, я могу часами пытаться приспособить свои примитивные инструменты - молоток, клейкую ленту, гаечный ключ и прочие - к текущей задаче. Когда я терплю полный провал и буксирую свою развалюху к настоящему механику, тот неизменно роется в огромном ящике с инструментами, пока не извлечёт идеальное приспособление, с которым работа кажется совсем лёгкой. Искусство сканирования портов устроено подобным образом. Специалисты понимают десятки методов сканирования и выбирают подходящий метод или сочетание для конкретной задачи. Неопытные пользователи и скрипт-кидди, напротив, пытаются решить любую проблему стандартным SYN-сканированием. Поскольку Nmap бесплатен, единственное препятствие на пути к мастерству сканирования портов - знания. Это определённо лучше автомобильного мира, где можно обладать огромным умением и установить, что вам нужен стяжной компрессор пружин стойки, но затем всё равно придётся заплатить за него тысячи долларов.

В предыдущей главе сканирование портов с помощью Nmap описывалось в общих чертах, включая краткую сводку поддерживаемых типов сканирования в разделе 4.3.1 «Выбор методов сканирования». В этой главе каждый из этих типов рассматривается подробно. Для каждого приведены типичные сценарии применения и инструкции, а также трассировки передаваемых по сети пакетов, показывающие принципы работы. Затем обсуждается алгоритм `ultra_scan`, применяемый большинством методов сканирования, с акцентом на аспектах, которые можно настраивать для повышения производительности.

Большинство типов сканирования доступно только привилегированным пользователям. Причина в том, что они отправляют и получают необработанные пакеты IP или даже кадры Ethernet, для чего в системах Unix требуется доступ `root`. В Windows рекомендуется использовать учётную запись администратора, хотя иногда Nmap работает на этой платформе и для непривилегированных пользователей, если WinPcap уже загружен в операционную систему. Требование привилегий `root` было серьёзным ограничением при выпуске Nmap в 1997 году, поскольку многие пользователи имели доступ только к общим учётным записям командной оболочки. Теперь мир изменился. Компьютеры подешевели, гораздо больше людей имеют постоянный прямой доступ к Интернету, а настольные системы Unix, включая Linux и Mac OS X, широко распространены. Теперь доступна и версия Nmap для Windows, позволяющая запускать программу на ещё большем числе настольных компьютеров. По всем этим причинам пользователям редко приходится запускать Nmap из ограниченных общих учётных записей оболочки. Это удачно, поскольку привилегированные параметры делают Nmap гораздо мощнее и гибче.

При обсуждении обработки Nmap ответов на пробы во многих разделах сообщения об ошибках ICMP обозначаются номерами типа и кода. Тип и код представляют собой восьмибитные поля заголовка ICMP, описывающие назначение сообщения. Методы сканирования портов Nmap имеют дело только с ICMP типа 3 - сообщениями о недоступности цели. На рисунке 5.1 показана структура заголовка ICMP такого пакета; он инкапсулируется в разделе данных пакета IP, как показано на рисунке 1 «Заголовок IPv4» [xxvii].

Рисунок 5.1. Структура заголовка сообщения ICMPv4 о недоступности цели



Существует шестнадцать кодов, представляющих разные сообщения о недоступности цели. Все они приведены в таблице 5.1, хотя Nmap интересуют только коды 0-3, 9, 10 и 13, отмеченные звёздочкой.

Таблица 5.1. Значения кодов сообщения ICMP «цель недоступна» (тип 3)

Код — описание

- **0*** — Сеть недоступна
- **1*** — Узел недоступен
- **2*** — Протокол недоступен
- **3*** — Порт недоступен
- **4** — Требуется фрагментация, но установлен бит запрета фрагментации
- **5** — Ошибка исходной маршрутизации
- **6** — Целевая сеть неизвестна
- **7** — Целевой узел неизвестен
- **8** — Исходный узел изолирован (устарело)
- **9*** — Доступ к целевой сети административно запрещён
- **10*** — Доступ к целевому узлу административно запрещён
- **11** — Сеть недоступна для данного типа обслуживания (TOS)
- **12** — Узел недоступен для TOS
- **13*** — Связь административно запрещена фильтрацией
- **14** — Нарушение приоритета узла
- **15** — Действует отсечение по приоритету

5.2. Сканирование TCP SYN (скрытое) (-sS)

SYN-сканирование неслучайно является стандартным и самым популярным вариантом. Оно выполняется быстро, сканируя тысячи портов в секунду в быстрой сети, не затруднённой навязчивыми межсетевыми экранами. SYN-сканирование сравнительно ненавязчиво и скрытно, поскольку никогда не завершает соединения TCP. Оно работает с любым соответствующим стандартам стеком TCP, а не зависит от особенностей конкретных платформ, как сканирования Nmap FIN/NULL/Xmas, Maimon и Idle. Кроме того, оно позволяет чётко и надёжно различать состояния `open`, `closed` и `filtered`.

SYN-сканирование запрашивается передачей Nmap параметра `-sS`. Для него требуются права на работу с необработанными пакетами, и при их наличии оно является стандартным сканированием TCP. Поэтому при запуске Nmap от имени `root` или администратора `-sS` обычно опускают. Стандартное поведение SYN-сканирования показано в примере 5.1, где найдено по одному порту в каждом из трёх основных состояний.

Пример 5.1. SYN-сканирование, показывающее три состояния портов

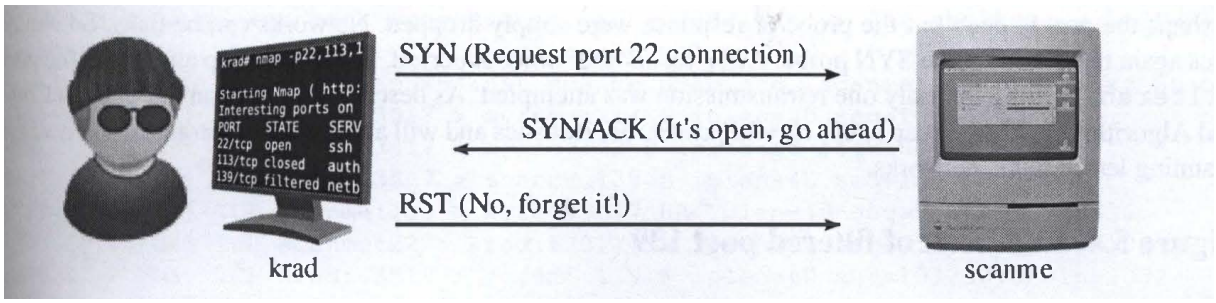
```
krad# nmap -p22,113,139 scanme.nmap.org

Starting Nmap ( http://nmap.org )
```

```
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT    STATE    SERVICE
22/tcp  open     ssh
113/tcp closed   auth
139/tcp filtered netbios-ssn
Nmap done: 1 IP address (1 host up) scanned in 1.35 seconds
```

Хотя SYN-сканирование довольно легко применять без низкоуровневых знаний TCP, понимание метода помогает при толковании необычных результатов. К счастью для нас, грозный взломщик в чёрной шляпе Эрит Хагивара сделал перерыв в терроризировании японских пользователей Windows[1], чтобы на уровне пакетов проиллюстрировать SYN-сканирование из примера 5.1. Сначала на рисунке 5.2 показано поведение для открытого порта 22.

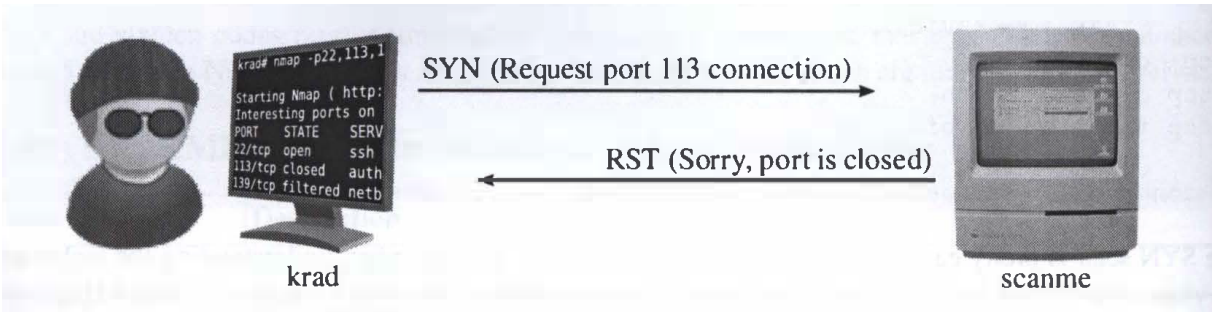
Рисунок 5.2. SYN-сканирование открытого порта 22



Как показывает этот пример, Nmap начинает с отправки порту 22 пакета TCP с установленным флагом SYN (если вы забыли, как выглядят заголовки пакетов, см. рисунок 2 «Заголовок TCP» [xxviii]). Это первый шаг трёхэтапного рукопожатия TCP, выполняемого при любой законной попытке соединения. Поскольку целевой порт открыт, Scanme делает второй шаг, отправляя в ответ пакет с флагами SYN и ACK. При обычном соединении машина Эрита с именем `krad` завершила бы трёхэтапное рукопожатие, отправив пакет ACK, подтверждающий SYN/ACK. Nmap не требуется этого делать, поскольку ответ SYN/ACK уже сообщил, что порт открыт. Если бы Nmap завершил соединение, затем пришлось бы заботиться о его закрытии. Обычно для этого требуется ещё одно рукопожатие, использующее пакеты FIN вместо SYN. Поэтому ACK - плохая идея, однако что-то сделать всё же необходимо. Если полностью проигнорировать SYN/ACK, Scanme решит, что пакет потерян, и продолжит его повторную отправку. Правильный ответ, поскольку полноценное соединение нам не нужно, - пакет RST, показанный на схеме. Он предписывает Scanme забыть о попытке соединения, то есть сбросить её. Nmap без труда мог бы отправить этот пакет RST, но в действительности в этом нет необходимости. Работающая на `krad` операционная система тоже получает SYN/ACK, которого не ожидает, поскольку пробу SYN сформировал сам Nmap. Поэтому ОС отвечает на неожиданный SYN/ACK пакетом RST. У всех пакетов RST, описанных в этой главе, также установлен бит ACK, поскольку они всегда отправляются в ответ на полученный пакет и подтверждают его. Поэтому для пакетов RST этот бит явно не показывается. Так как трёхэтапное рукопожатие никогда не завершается, SYN-сканирование иногда называют полуоткрытым.

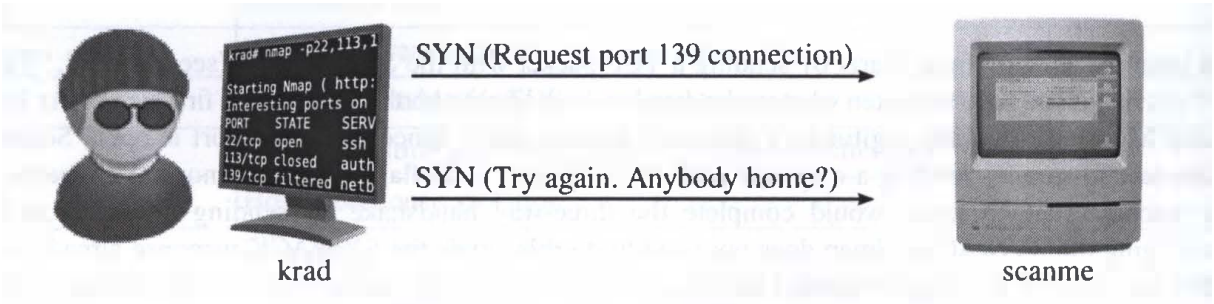
На рисунке 5.3 показано, как Nmap определяет, что порт 113 закрыт. Это ещё проще, чем случай с открытым портом. Первый шаг всегда одинаков: Nmap отправляет пробу SYN машине Scanme. Но вместо ответного SYN/ACK приходит RST. Вопрос решён - порт закрыт. Дальнейшее взаимодействие по этому порту не требуется.

Рисунок 5.3. SYN-сканирование закрытого порта 113



Наконец, на рисунке 5.4 Эрит показывает, как фильтруемый порт выглядит для Nmap. Сначала, как обычно, отправляется исходный SYN, но Nmap не получает ответа. Возможно, ответ просто задерживается. По предыдущим ответам или стандартным значениям времени Nmap знает, как долго ожидать, и в конце концов прекращает ожидание. Не отвечающий порт обычно фильтруется, то есть блокируется устройством межсетевого экрана, либо сам узел не работает, но одной проверки недостаточно для окончательного вывода. Возможно, порт открыт, а проба или ответ просто потерялись. Сети бывают ненадёжными. Поэтому Nmap повторяет попытку, снова отправляя пробу SYN. После ещё одного периода ожидания Nmap сдаётся и помечает порт как `filtered`. В данном случае предпринята только одна повторная передача. Как описано в разделе 5.13 «Код и алгоритмы сканирования», Nmap тщательно ведёт статистику потери пакетов и выполняет больше повторных передач при сканировании менее надёжных сетей.

Рисунок 5.4. SYN-сканирование фильтруемого порта 139



Nmap также считает порт фильтруемым, если получает определённые сообщения об ошибках ICMP. В таблице 5.2 показано, как Nmap назначает состояния портов на основании ответов на пробу SYN.

Таблица 5.2. Интерпретация Nmap ответов на пробу SYN

Ответ на пробу — назначенное состояние

• Ответ TCP SYN/ACK — `open`
• Ответ TCP RST — `closed`

• Ответ не получен даже после повторных передач — `filtered`
• Ошибка ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13) — `filtered`

Хотя красивые иллюстрации из этого раздела полезны, когда они под рукой, при указании `--packet-trace` в дополнение к другим нужным флагам командной строки Nmap сообщает, что именно делает на уровне пакетов. Для новичков это прекрасный способ понять поведение Nmap, когда Эрита нет рядом и он не может помочь. Даже опытные пользователи считают этот параметр полезным, когда Nmap выдаёт неожиданные результаты. Возможно, стоит также повысить уровень отладки с помощью `-d` или даже `-d5`. Затем сканируйте минимально необходимое для вашей цели число портов и узлов, иначе можно буквально получить миллионы строк вывода. В примере 5.2 повторяется трёхпортовое SYN-сканирование Эрита с включённой трассировкой пакетов; вывод сокращён. Прочитайте командную строку, а затем проверьте себя, определив до дальнейшего

чтения, какие пакеты будут отправлены. После чтения трассировки до строки `The SYN Stealth Scan took 1.25s` по строкам `RCVD` уже должно быть понятно, как будет выглядеть таблица состояний портов.

Пример 5.2. Применение `--packet-trace` для понимания SYN-сканирования

```
krad# nmap -d --packet-trace -p22,113,139 scanme.nmap.org
Starting Nmap ( http://nmap.org )
SENT (0.0130s) ICMP krad > scanme echo request (type=8/code=0) ttl=52 id=1829
SENT (0.0160s) TCP krad:63541 > scanme:80 A iplen=40 seq=91911070 ack=99850910
RCVD (0.0280s) ICMP scanme > krad echo reply (type=0/code=0) iplen=28
We got a ping packet back from scanme: id = 48821 seq = 714 checksum = 16000
massping done: num_hosts: 1 num_responses: 1
Initiating SYN Stealth Scan against scanme.nmap.org (scanme) [3 ports] at 00:53
SENT (0.1340s) TCP krad:63517 > scanme:113 S iplen=40 seq=10438635
SENT (0.1370s) TCP krad:63517 > scanme:22 S iplen=40 seq=10438635
SENT (0.1400s) TCP krad:63517 > scanme:139 S iplen=40 seq=10438635
RCVD (0.1460s) TCP scanme:113 > krad:63517 RA iplen=40 seq=0 ack=10438636
RCVD (0.1510s) TCP scanme:22 > krad:63517 SA iplen=44 seq=75897108 ack=10438636
SENT (1.2550s) TCP krad:63518 > scanme:139 S iplen=40 seq=10373098 win=3072
The SYN Stealth Scan took 1.25s to scan 3 total ports.
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT    STATE    SERVICE
22/tcp  open     ssh
113/tcp closed   auth
139/tcp filtered netbios-ssn
Nmap done: 1 IP address (1 host up) scanned in 1.40 seconds
```

SYN-сканирование давно называют скрытым, поскольку оно менее заметно, чем рассматриваемое далее TCP-сканирование `connect`, бывшее самым распространённым типом до выпуска Nmap. Несмотря на такое название, не рассчитывайте, что стандартное SYN-сканирование незамеченным пройдёт через чувствительные сети. Широко распространённые системы обнаружения вторжений и даже персональные межсетевые экраны вполне способны выявлять стандартные SYN-сканирования. Более эффективные методы скрытого сканирования показаны в главе 10 «Обнаружение и обход межсетевых экранов и систем обнаружения вторжений».

5.3. Сканирование TCP Connect (-sT)

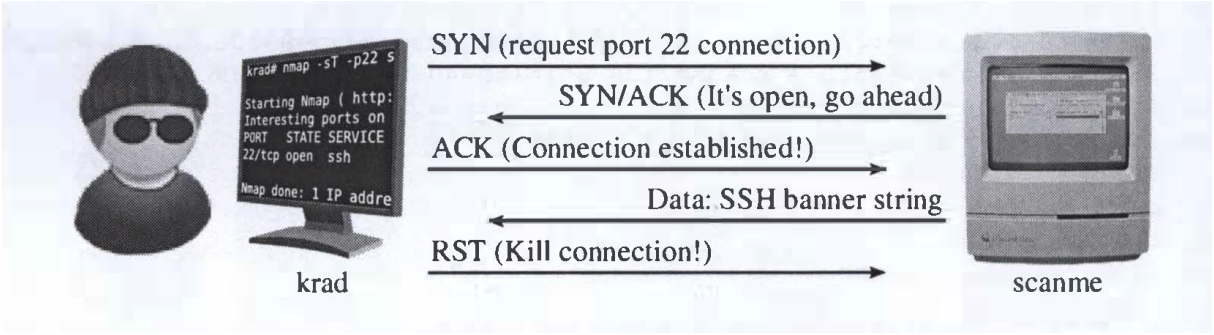
TCP-сканирование `connect` является стандартным типом сканирования TCP, когда SYN-сканирование невозможно. Так бывает, когда у пользователя нет прав на необработанные пакеты или сканируются сети IPv6. Вместо самостоятельного создания необработанных пакетов, как у большинства других типов, Nmap просит базовую операционную систему установить соединение с целевой машиной и портом посредством системного вызова `connect`. Это тот же высокоуровневый системный вызов, который веб-браузеры, клиенты P2P и большинство других сетевых приложений применяют для установления соединения. Он входит в программный интерфейс, известный как API сокетов Berkeley. Вместо чтения необработанных ответных пакетов непосредственно из сети Nmap получает через этот API сведения о состоянии каждой попытки соединения. Этот метод и сканирование с отражением от FTP (раздел 5.12 «Сканирование TCP FTP Bounce (`-b`)») - единственные типы сканирования, доступные непривилегированным пользователям.

Когда SYN-сканирование доступно, обычно лучше выбрать его. Nmap меньше управляет высокоуровневым вызовом `connect`, чем необработанными пакетами, поэтому он менее эффективен. Системный вызов полностью устанавливает соединения с открытыми целевыми портами, а не выполняет полуоткрытый сброс, как SYN-сканирование. Это не только занимает больше времени и требует большего числа пакетов для получения тех же сведений, но и повышает вероятность записи соединения в журналы целевой машины. Приличная IDS обнаружит оба

метода, однако на большинстве машин такой сигнализации нет. Когда Nmap подключается и затем закрывает соединение без отправки данных, многие службы обычной системы Unix добавляют запись в `syslog`, а иногда и загадочное сообщение об ошибке. Совсем плохие службы при этом аварийно завершаются, хотя такое случается редко. Администратор, увидевший в журналах множество попыток соединения от одной системы, должен понять, что её просканировали методом `connect`.

На рисунке 5.5 показано сканирование `connect` открытого порта 22 узла `scanme.nmap.org`. Напомним, что на рисунке 5.2 «SYN-сканирование открытого порта 22» для этого потребовалось лишь три пакета. Точное поведение для открытого порта зависит от платформы, где работает Nmap, и слушающей на другом конце службы, но этот пример из шести пакетов типичен.

Рисунок 5.5. Сканирование Connect открытого порта 22 (`nmap -sT -p22 scanme.nmap.org`)



Первые два шага, SYN и SYN/ACK, полностью совпадают с SYN-сканированием. Затем вместо прерывания полуоткрытого соединения пакетом RST машина `krad` подтверждает SYN/ACK собственным пакетом ACK, завершая установление соединения. В этом случае Scanme даже успела отправить строку баннера SSH (`SSH-1.99-OpenSSH_3.1p1\n`) через уже открытое соединение. Как только Nmap узнаёт от операционной системы узла об успешном соединении, он завершает его. Обычно соединения TCP заканчиваются ещё одним рукопожатием с флагом FIN, но Nmap просит ОС узла немедленно разорвать соединение пакетом RST.

Хотя в этом примере сканирования `connect` потребовалось вдвое больше пакетов, чем при SYN-сканировании, различие в расходе пропускной способности редко бывает столь существенным. Подавляющее большинство портов при крупном сканировании будет находиться в состоянии `closed` или `filtered`. Трассировки пакетов для них совпадают с описанными для SYN-сканирования на рисунке 5.3 «SYN-сканирование закрытого порта 113» и рисунке 5.4 «SYN-сканирование фильтруемого порта 139». Дополнительный сетевой трафик создают только открытые порты.

Вывод сканирования `connect` почти не отличается от SYN-сканирования. В примере 5.3 показано сканирование Scanme методом `connect`. Параметр `-sT` можно было не указывать, поскольку Nmap запущен из непривилегированной учётной записи и `connect` является стандартным типом.

Пример 5.3. Пример сканирования Connect

```
krad~> nmap -sT scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE  SERVICE
22/tcp  open   ssh
25/tcp  closed smtp
53/tcp  open   domain
70/tcp  closed gopher
80/tcp  open   http
113/tcp closed auth
Nmap done: 1 IP address (1 host up) scanned in 4.74 seconds
```

5.4. Сканирование UDP (-sU)

Хотя большинство популярных служб Интернета работает по протоколу TCP, службы UDP также широко распространены. DNS, SNMP и DHCP с зарегистрированными портами 53, 161/162 и 67/68 относятся к самым распространённым. Поскольку сканирование UDP обычно медленнее и сложнее TCP, некоторые аудиторы безопасности игнорируют эти порты. Это ошибка: эксплуатируемые службы UDP встречаются довольно часто, и злоумышленники определённо не игнорируют протокол целиком. К счастью, Nmap помогает составить перечень портов UDP.

Сканирование UDP включается параметром `-sU`. Его можно сочетать с типом сканирования TCP, например SYN (`-sS`), чтобы проверить оба протокола за один запуск.

UDP-сканирование работает, отправляя пустой, не содержащий данных, заголовок UDP каждому целевому порту. По ответу или его отсутствию порту назначается одно из четырёх состояний, показанных в таблице 5.3.

Таблица 5.3. Интерпретация Nmap ответов на пробу UDP

Ответ на пробу — назначенное состояние

• Любой ответ UDP от целевого порта (редкость) — `open`
• Ответ не получен даже после повторных передач — `open|filtered`
• Ошибка ICMP «порт недоступен» (тип 3, код 3) — `closed`
• Другие ошибки ICMP «недоступно» (тип 3, код 1, 2, 9, 10 или 13) — `filtered`

Самым любопытным элементом таблицы может быть состояние `open|filtered`. Это симптом главной трудности UDP-сканирования: открытые порты редко отвечают на такие пробы. Целевой стек TCP/IP просто передаёт пустой пакет слушающему приложению, которое обычно сразу отбрасывает его как недопустимый. Если бы порты во всех остальных состояниях отвечали, открытые порты можно было бы установить методом исключения. К сожалению, межсетевые экраны и фильтрующие устройства тоже известны тем, что отбрасывают пакеты без ответа. Поэтому, не получив ответа после нескольких попыток, Nmap не способен определить, открыт порт или фильтруется. Во время выпуска Nmap фильтрующие устройства встречались достаточно редко, чтобы программа могла - и действительно так делала - просто считать порт открытым. Теперь Интернет защищён лучше, поэтому в 2004 году, начиная с версии 3.70, Nmap стал сообщать о не отвечающих портах UDP как `open|filtered`. Это видно в примере 5.4, где Эрит сканирует машину Linux по имени Felix.

Пример 5.4. Пример UDP-сканирования

```
krad# nmap -sU -v felix
Starting Nmap ( http://nmap.org )
Interesting ports on felix.nmap.org (192.168.0.42):
(The 997 ports scanned but not shown below are in state: closed)
PORT    STATE         SERVICE
53/udp  open|filtered domain
67/udp  open|filtered dhcpserver
111/udp open|filtered rpcbind
MAC Address: 00:02:E3:14:11:02 (Lite-on Communications)
Nmap done: 1 IP address (1 host up) scanned in 999.25 seconds
```

Это сканирование Felix демонстрирует неоднозначность `open|filtered`, а также другую проблему: UDP-сканирование бывает медленным. Сканирование тысячи портов заняло в данном случае почти 17 минут из-за ограничения частоты ответов ICMP, применяемого Felix и большинством других систем Linux. Nmap предлагает способы обойти обе проблемы, как описано в следующих двух разделах.

5.4.1. Различение открытых и фильтруемых портов UDP

При сканировании Felix все порты, кроме трёх `open|filtered`, были закрыты. Таким образом, сканирование всё же успешно сузило потенциально открытые порты до небольшой группы. Так бывает не всегда. В примере 5.5 показано UDP-сканирование строго фильтруемого сайта Scanme.

Пример 5.5. Пример UDP-сканирования

```
krad# nmap -sU -T4 scanme.nmap.org
Starting Nmap ( http://nmap.org )
All 1000 scanned ports on scanme.nmap.org (64.13.134.52) are open|filtered

Nmap done: 1 IP address (1 host up) scanned in 5.50 seconds
```

В этом случае сканирование совсем не сузило множество открытых портов. Все 1000 находятся в состоянии `open|filtered`. Требуется новая стратегия.

Таблица 5.3 «Интерпретация Nmap ответов на пробу UDP» показывает, что состояние `open|filtered` возникает, когда Nmap не получает никаких ответов на пробы UDP определённого порта. Однако она также показывает, что в редких случаях слушающая порт служба UDP отвечает пакетом того же рода, доказывая открытость порта.

Причина, по которой эти службы часто не отвечают, состоит в том, что отправляемые Nmap пустые пакеты считаются недопустимыми. К сожалению, службы UDP обычно определяют собственную структуру пакетов, а не придерживаются какого-либо общего формата, который Nmap мог бы отправлять всегда. Пакет SNMP выглядит совершенно иначе, чем пакет запроса SunRPC, DHCP или DNS.

Чтобы отправлять правильный пакет для каждой популярной службы UDP, Nmap потребовалась бы большая база данных, определяющая форматы их проб. К счастью, у Nmap такая база есть в виде файла `nmap-service-probes`, который является частью подсистемы определения служб и версий, описанной в главе 7 «Определение версий служб и приложений».

Когда определение версий включено параметром `-sV` (или `-A`), Nmap отправляет пробы UDP каждому порту `open|filtered` (а также заведомо открытым портам). Если какая-либо из проб вызывает ответ от порта `open|filtered`, состояние изменяется на `open`. Результаты добавления `-sV` к сканированию Felix показаны в примере 5.6.

Пример 5.6. Улучшение результатов UDP-сканирования Felix с помощью определения версий

```
krad# nmap -sUV -F felix.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on felix.nmap.org (192.168.0.42):
Not shown: 997 closed ports
PORT    STATE         SERVICE    VERSION
53/udp  open          domain     ISC BIND 9.2.1
67/udp  open|filtered dhcpserver
111/udp open          rpcbind    2 (rpc #100000)
MAC Address: 00:02:E3:14:11:02 (Lite-on Communications)

Nmap done: 1 IP address (1 host up) scanned in 1037.57 seconds
```

Новое сканирование показывает, что порты 111 и 53 определённо открыты. Однако система несовершенна - порт 67 всё ещё имеет состояние `open|filtered`. В данном конкретном случае порт открыт, но у Nmap нет работающей пробы определения версии для DHCP. Другая сложная служба - SNMP, которая обычно отвечает лишь при указании правильной строки сообщества. На многих устройствах настроена строка сообщества `public`, но не на всех. Хотя эти результаты

несовершенны, узнать истинное состояние двух из трёх проверенных портов всё же полезно.

После успеха в устранении неоднозначности результатов Felix Эрит снова обращает внимание на Scanme, где в прошлый раз все порты были указаны как `open|filtered`. Он повторяет попытку с определением версий, как показано в примере 5.7.

Пример 5.7. Улучшение результатов UDP-сканирования Scanme с помощью определения версий

```
krad# nmap -sUV -T4 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 999 open|filtered ports
PORT   STATE SERVICE VERSION
53/udp open  domain  ISC BIND 9.3.4

Nmap done: 1 IP address (1 host up) scanned in 3691.89 seconds
```

Этот результат потребовал часа вместо пяти секунд для предыдущего сканирования Scanme, но теперь результаты действительно полезны. Улыбка Эрита становится шире, а глаза сверкают при виде свидетельства открытого сервера имён ISC BIND на машине, которую он хочет взломать. У этого программного обеспечения долгая история уязвимостей, поэтому, возможно, ему удастся найти изъян в этой свежей версии.

Хотя Эрит сосредоточит атаки UDP на порте 53, поскольку подтверждено, что он открыт, он не забывает и о других портах. Эти 1007 портов указаны как `open|filtered`. Как мы видели на примере порта `dhcpserver` машины Felix, некоторые открытые службы UDP способны скрываться даже от механизма определения версий Nmap. Кроме того, пока он сканировал лишь стандартные порты; существуют ещё 64 529 других, которые могут быть открыты. Для полноты отметим, что 53 - единственный открытый порт UDP на Scanme.

Хотя этот метод определения версий является единственным способом, которым Nmap может автоматически устранить неоднозначность портов `open|filtered`, вручную можно испробовать ещё пару приёмов. Иногда помогает специализированная трассировка маршрута. С помощью такого инструмента, как `hping2`[2], можно выполнить трассировку до заведомо открытого порта TCP или UDP. Затем следует попробовать то же самое для сомнительного порта UDP. Различия в числе переходов могут отличить открытые порты от фильтруемых. Эрит проверяет это на Scanme в примере 5.8. Первая команда `hping2` выполняет трассировку UDP до заведомо открытого порта 53. Параметр `-t 8` предписывает `hping2` начинать с восьмого перехода и используется здесь лишь для экономии места. Вторая команда делает то же самое для предположительно закрытого порта 54.

Пример 5.8. Попытка устранить неоднозначность портов UDP по различиям TTL

```
krad# hping2 --udp --traceroute -t 8 -p 53 scanme.nmap.org
HPING scanme.nmap.org (ppp0): udp mode set, 28 headers + 0 data bytes
hop=8 TTL 0 during transit from 206.24.211.77 (dcr2.SanFranciscosfo.savvis.net)
hop=9 TTL 0 during transit from 208.172.147.94 (bpr2.PaloAltoPaix.savvis.net)
hop=10 TTL 0 during transit from 206.24.240.194 (meer.PaloAltoPaix.savvis.net)
hop=11 TTL 0 during transit from 205.217.152.21 (vlan21.sv.meer.net)

--- scanme.nmap.org hping statistic ---
12 packets transmitted, 4 packets received, 67% packet loss
round-trip min/avg/max = 13.4/13.8/14.1 ms

krad# hping2 --udp --traceroute -t 8 -p 54 scanme.nmap.org
HPING scanme.nmap.org (ppp0): udp mode set, 28 headers + 0 data bytes
hop=8 TTL 0 during transit from 206.24.211.77 (dcr2.SanFranciscosfo.savvis.net)
hop=9 TTL 0 during transit from 208.172.147.94 (bpr2.PaloAltoPaix.savvis.net)
hop=10 TTL 0 during transit from 206.24.240.194 (meer.PaloAltoPaix.savvis.net)
hop=11 TTL 0 during transit from 205.217.152.21 (vlan21.sv.meer.net)
--- scanme.nmap.org hping statistic ---
```

```
12 packets transmitted, 4 packets received, 67% packet loss
round-trip min/avg/max = 12.5/13.6/14.7 ms
```

В этом примере Эрит смог достичь лишь одиннадцатого перехода как для открытого, так и для закрытого порта. Поэтому по этим результатам нельзя различить состояния портов данного узла. Попробовать стоило, и этот способ действительно работает в значительном числе случаев. Вероятность успеха выше, когда экранирующий межсетевой экран находится хотя бы за один-два перехода до целевого узла. На Scanme, напротив, работает собственный межсетевой экран узла на основе Linux `iptables`. Поэтому число переходов для фильтруемых и открытых портов не различается.

Другой метод - применить к распространённым портам инструменты, предназначенные для конкретных приложений. Например, против порта 161 можно испытать программу перебора строк сообщества SNMP. По мере роста базы проб определения версий Nmap необходимость дополнять её результаты внешними специализированными инструментами уменьшается. Но они по-прежнему полезны в особых случаях, например для устройств SNMP с нестандартной строкой сообщества.

5.4.2. Ускорение UDP-сканирования

Другая серьёзная проблема UDP-сканирования - выполнить его быстро. Открытые и фильтруемые порты редко отправляют какие-либо ответы, поэтому Nmap приходится дожидаться истечения времени ожидания, а затем выполнять повторные передачи на случай, если проба или ответ были потеряны. Закрытые порты часто создают ещё более серьёзную проблему. Обычно они отправляют ошибку ICMP «порт недоступен». Но, в отличие от пакетов RST, отправляемых закрытыми портами TCP в ответ на SYN-сканирование или сканирование `connect`, многие узлы по умолчанию ограничивают частоту сообщений ICMP «порт недоступен». Особенно строго это делают Linux и Solaris. Например, ядро Linux 2.4.20 на Felix ограничивает сообщения о недоступности назначения одним в секунду (в `net/ipv4/icmp.c`). Этим объясняется столь низкая скорость сканирования в примере 5.4 «Пример UDP-сканирования».

Nmap обнаруживает ограничение частоты и соответственно замедляет работу, чтобы не наводнять сеть бесполезными пакетами, которые целевая машина отбросит. К сожалению, ограничение в стиле Linux до одного пакета в секунду растягивает сканирование 65 536 портов более чем на 18 часов. Ниже приведены рекомендации по повышению производительности UDP-сканирования. Более подробное обсуждение и общие советы см. также в главе 6 «Оптимизация производительности Nmap».

Увеличивайте параллелизм узлов

Если Nmap получает от одного целевого узла лишь одну ошибку «порт недоступен» в секунду, то, одновременно сканируя 100 таких узлов, он может получать 100 ошибок в секунду. Для этого передайте параметру `--min-hostgroup` большое значение, например 100.

Сначала сканируйте популярные порты

Лишь очень немногие номера портов UDP широко используются. Сканирование 100 самых распространённых портов UDP (с параметром `-F`) завершится быстро. Затем можно исследовать эти результаты, одновременно запустив в фоновом режиме многодневное сканирование сети по всем 65 тысячам портов.

Добавляйте `--version-intensity 0` при сканировании с определением версий

Как отмечалось в предыдущем разделе, определение версий (`-sV`) часто требуется, чтобы отличать открытые порты UDP от фильтруемых. Определение версий сравнительно медленно, поскольку требует отправки большого числа проб, специфичных для протоколов приложений,

каждому порту `open` или `open|filtered`, найденному на целевых машинах. Указание `--version-intensity 0` предписывает Nmap пробовать лишь те пробы, которые с наибольшей вероятностью окажутся эффективными для данного номера порта. Для этого используются данные из файла `nmap-service-probes`. Этот параметр существенно влияет на производительность, что будет показано далее в этом разделе.

Сканируйте из-за межсетевого экрана

Как и в случае TCP, пакетные фильтры способны резко замедлять сканирование. Многие современные межсетевые экраны позволяют легко задавать ограничения частоты пакетов. Если эту проблему можно обойти, запустив сканирование не через межсетевой экран, а с его внутренней стороны, сделайте это.

Используйте `--host-timeout`, чтобы пропускать медленные узлы

Сканирование узлов с ограниченной частотой ICMP может занимать на порядки больше времени, чем узлов, которые отвечают на каждую пробу быстрым пакетом «назначение недоступно». Если указать максимальное время сканирования, например 900000 для 15 минут, Nmap прекратит работу с отдельными узлами, если не успеет завершить их сканирование за это время. Так можно быстро просканировать все отзывчивые узлы, а затем работать с медленными узлами в фоновом режиме.

Используйте `-v` и расслабьтесь

При включённом подробном выводе (`-v`) Nmap показывает предполагаемое время завершения сканирования каждого узла. Нет нужды непрерывно наблюдать за процессом. Поспите, отправляйтесь в любимый паб, почитайте книгу, закончите другую работу или развлекайтесь иным способом, пока Nmap неутомимо сканирует от вашего имени.

Прекрасным примером необходимости оптимизировать UDP-сканирование служит пример 5.7 «Улучшение результатов UDP-сканирования Scanme с помощью определения версий». Сканирование дало желаемые данные, но проверка всего одного узла заняла больше часа! В примере 5.9 это сканирование выполняется снова. На этот раз добавлены параметры `-F --version-intensity 0`, и продолжавшееся час сканирование сокращается до 13 секунд! При этом обнаруживаются те же ключевые сведения: демон ISC BIND, работающий на порте 53.

Пример 5.9. Оптимизация времени UDP-сканирования

```
krad# nmap -sUV -T4 -F --version-intensity 0 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 99 open|filtered ports
PORT   STATE SERVICE VERSION
53/udp open  domain  ISC BIND 9.3.4

Nmap done: 1 IP address (1 host up) scanned in 12.92 seconds
```

5.5. Сканирования TCP FIN, NULL и Xmas (-sF, -sN, -sX)

Эти три типа сканирования (ещё больше вариантов возможно с параметром `--scanflags`, описанным в следующем разделе) используют малозаметную лазейку в RFC по TCP, чтобы различать открытые и закрытые порты. На странице 65 RFC 793 сказано, что «если состояние [целевого] порта CLOSED... входящий сегмент, не содержащий RST, вызывает отправку RST в ответ». Затем на следующей странице обсуждаются пакеты, отправленные открытым портам без установленных битов SYN, RST или ACK: «вы вряд ли сюда попадёте, но если это произошло, отбросьте сегмент и возвратитесь».

При сканировании систем, соответствующих этому тексту RFC, любой пакет без битов SYN, RST или ACK вызовет ответ RST, если порт закрыт, и не вызовет никакого ответа, если порт открыт. Если ни один из этих трёх битов не установлен, допустима любая комбинация остальных трёх: FIN, PSH и URG. Nmap использует это в трёх типах сканирования:

NULL-сканирование (`-sN`)
Не устанавливает ни одного бита (заголовок флагов TCP равен 0).

FIN-сканирование (`-sF`)
Устанавливает только бит TCP FIN.

Xmas-сканирование (`-sX`)
Устанавливает флаги FIN, PSH и URG, отчего пакет сияет, словно рождественская ёлка.

Все три типа сканирования ведут себя совершенно одинаково, за исключением флагов TCP, установленных в пакетах-пробах. Ответы интерпретируются, как показано в таблице 5.4.

Таблица 5.4. Интерпретация Nmap ответов на пробу сканирования NULL, FIN или Xmas

Ответ на пробу — назначенное состояние

• Ответ не получен даже после повторных передач — `open|filtered`
• Ответ RST — `closed`
• Ошибка ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13) — `filtered`

Главное преимущество этих типов сканирования состоит в том, что они способны незаметно проходить через некоторые межсетевые экраны без контроля состояния и маршрутизаторы с пакетной фильтрацией. Такие межсетевые экраны пытаются предотвращать входящие соединения TCP, разрешая при этом исходящие, блокируя все пакеты TCP с установленным битом SYN и сброшенным ACK. Такая конфигурация встречается достаточно часто, чтобы команда межсетевого экрана Linux `iptables` предлагала специальный параметр `--syn` для её реализации. При сканированиях NULL, FIN и Xmas бит SYN сброшен, поэтому пакеты беспрепятственно пролетают сквозь такие правила.

Ещё одно преимущество состоит в том, что эти типы сканирования немного скрытнее даже SYN-сканирования. Однако рассчитывать на это не стоит: большинство современных продуктов IDS можно настроить на их обнаружение.

Главный недостаток - не все системы буквально следуют RFC 793. Некоторые системы отвечают RST на пробы независимо от того, открыт порт или нет. В результате все порты помечаются как `closed`. К основным операционным системам с таким поведением относятся Microsoft Windows, многие устройства Cisco и IBM OS/400. Однако против большинства систем на базе Unix это сканирование работает. Поскольку механизм определения ОС Nmap проверяет эту особенность, узнать, сработает ли сканирование против конкретного типа системы, можно, изучив файл `nmap-os-db`. Тест T2 отправляет пакет NULL открытому порту. Поэтому строка вида `T2(R=N)` означает, что система, по-видимому, поддерживает RFC и одно из этих сканирований должно против неё сработать. Если строка T2 длиннее, система нарушила RFC, отправив ответ, и эти сканирования не сработают. В главе 8 «Удалённое определение ОС» снятие отпечатков ОС объясняется подробнее.

Другой недостаток этих сканирований в том, что они не способны отличать открытые порты от некоторых фильтруемых. Если пакетный фильтр отправляет ошибку ICMP о запрете назначения, Nmap понимает, что порт фильтруется. Но большинство фильтров просто отбрасывают запрещённые пробы без ответа, из-за чего порты кажутся открытыми. Поскольку Nmap не может установить, какой именно это случай, не отвечающие порты помечаются как `open|filtered`. Добавление определения версий (`-sV`) способно устранить неоднозначность, как при UDP-сканировании, но это уничтожает значительную часть скрытности данного сканирования. Если

вы всё равно готовы и способны соединяться с портами, можно с тем же успехом применить SYN-сканирование.

Использовать эти методы сканирования просто. Для выбора типа достаточно добавить параметр `-sN`, `-sF` или `-sX`. В примере 5.10 показаны два случая. Первый, FIN-сканирование Para, определяет все пять открытых портов как `open|filtered`. Следующий запуск, Xmas-сканирование `scanme.nmap.org`, работает не столь хорошо. Он обнаруживает закрытый порт, но не может отличить 995 фильтруемых портов от четырёх открытых: все 999 указаны как `open|filtered`. Это показывает, почему Nmap предлагает так много методов сканирования. Ни один из них не является наилучшим во всех случаях. Эриту просто придётся попробовать другой метод, чтобы узнать о Scanme больше.

Пример 5.10. Примеры сканирования FIN и Xmas

```
krad# nmap -sF -T4 para
Starting Nmap ( http://nmap.org )
Interesting ports on para (192.168.10.191):
Not shown: 995 closed ports
PORT     STATE         SERVICE
22/tcp   open|filtered ssh
53/tcp   open|filtered domain
111/tcp  open|filtered rpcbind
515/tcp  open|filtered printer
6000/tcp open|filtered X11
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)
Nmap done: 1 IP address (1 host up) scanned in 4.64 seconds

krad# nmap -sX -T4 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 999 open|filtered ports
PORT    STATE  SERVICE
113/tcp closed auth
Nmap done: 1 IP address (1 host up) scanned in 23.11 seconds
```

Чтобы продемонстрировать всю способность этих сканирований обходить межсетевые экраны, у целевого экрана должна быть довольно бестолковая конфигурация. К сожалению, найти такие нетрудно. В примере 5.11 показано SYN-сканирование машины SCO/Caldera по имени Docsrv.

Пример 5.11. SYN-сканирование Docsrv

```
# nmap -sS -T4 docsrv.caldera.com

Starting Nmap ( http://nmap.org )
Interesting ports on docsrv.caldera.com (216.250.128.247):
(The 997 ports scanned but not shown below are in state: filtered)
PORT    STATE  SERVICE
80/tcp  open   http
113/tcp closed auth
507/tcp open   crs
Nmap done: 1 IP address (1 host up) scanned in 28.62 seconds
```

Этот пример выглядит нормально. Открыты лишь два порта, а остальные, кроме 113, фильтруются. При современном межсетевом экране с контролем состояния FIN-сканирование не должно дать дополнительной информации. Но Эрит всё равно пробует его и получает вывод из примера 5.12.

Пример 5.12. FIN-сканирование Docsrv

```
# nmap -sF -T4 docsrv.caldera.com

Starting Nmap ( http://nmap.org )
Interesting ports on docsrv.caldera.com (216.250.128.247):
Not shown: 961 closed ports
PORT      STATE         SERVICE
7/tcp     open|filtered echo
9/tcp     open|filtered discard
```

```
11/tcp    open|filtered systat
13/tcp    open|filtered daytime
15/tcp    open|filtered netstat
19/tcp    open|filtered chargen
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
23/tcp    open|filtered telnet
25/tcp    open|filtered smtp
37/tcp    open|filtered time
79/tcp    open|filtered finger
80/tcp    open|filtered http
110/tcp   open|filtered pop3
111/tcp   open|filtered rpcbind
135/tcp   open|filtered msrpc
143/tcp   open|filtered imap
360/tcp   open|filtered scoi2odialog
389/tcp   open|filtered ldap
465/tcp   open|filtered smtps
507/tcp   open|filtered crs
512/tcp   open|filtered exec
513/tcp   open|filtered login
514/tcp   open|filtered shell
515/tcp   open|filtered printer
636/tcp   open|filtered ldapssl
712/tcp   open|filtered unknown
955/tcp   open|filtered unknown
993/tcp   open|filtered imaps
995/tcp   open|filtered pop3s
1434/tcp  open|filtered ms-sql-m
2000/tcp  open|filtered callbook
2766/tcp  open|filtered listen
3000/tcp  open|filtered ppp
3306/tcp  open|filtered mysql
6112/tcp  open|filtered dtspc
32770/tcp open|filtered sometimes-rpc3
32771/tcp open|filtered sometimes-rpc5
32772/tcp open|filtered sometimes-rpc7
Nmap done: 1 IP address (1 host up) scanned in 7.64 seconds
```

Ого! Как много предположительно открытых портов. Вероятно, большинство из них действительно открыты, поскольку было бы необычно фильтровать лишь эти 39 портов, оставляя остальные 961 закрытыми и отправляющими пакет RST. Но всё же возможно, что некоторые или все они фильтруются, а не открыты. FIN-сканирование не способно установить это наверняка. Мы ещё вернёмся к этому случаю и позже в этой главе узнаем о Docsrv больше.

5.6. Пользовательские типы сканирования с --scanflags

По-настоящему опытным пользователям Nmap незачем ограничиваться готовыми типами сканирования. Параметр `--scanflags` позволяет спроектировать собственное сканирование, задав произвольные флаги TCP. Дайте волю творческой мысли, одновременно уклоняясь от систем обнаружения вторжений, поставщики которых лишь пролистали справочную страницу Nmap и добавили отдельные правила!

Аргументом `--scanflags` может быть числовое значение флагов, например 9 (PSH и FIN), но символьные имена использовать проще. Просто объедините любую комбинацию `URG`, `ACK`, `PSH`, `RST`, `SYN` и `FIN`. Например, `--scanflags URGACKPSHRSTSYNFIN` устанавливает все флаги, хотя для сканирования это не слишком полезно. Порядок их указания не имеет значения.

Помимо желаемых флагов, можно указать тип TCP-сканирования, например `-sA` или `-sF`. Этот базовый тип сообщает Nmap, как интерпретировать ответы. Например, при SYN-сканировании отсутствие ответа считается признаком порта `filtered`, тогда как FIN-сканирование трактует его

как open|filtered. Nmap будет вести себя так же, как для базового типа сканирования, за исключением того, что станет использовать указанные вами флаги TCP. Если базовый тип не задан, используется SYN-сканирование.

5.6.1. Пользовательское SYN/FIN-сканирование

Один из интересных пользовательских типов сканирования - SYN/FIN. Иногда администратор межсетевого экрана или изготовитель устройства пытается блокировать входящие соединения правилом вроде «отбрасывать все входящие пакеты, в которых установлен только флаг SYN». Ограничение касается именно единственного флага SYN, поскольку они не хотят блокировать пакеты SYN/ACK, возвращаемые на втором шаге исходящего соединения.

Проблема этого подхода в том, что большинство конечных систем принимают начальные пакеты SYN, в которых также установлены другие флаги, кроме ACK. Например, система снятия отпечатков ОС Nmap отправляет открытому порту пакет SYN/FIN/URG/PSH. Более половины отпечатков в базе отвечают SYN/ACK. Следовательно, они допускают сканирование портов таким пакетом и обычно позволяют также установить полное соединение TCP. Известны даже системы, отвечающие SYN/ACK на пакет SYN/RST! RFC по TCP неоднозначен в вопросе о том, какие флаги допустимы в начальном пакете SYN, хотя SYN/RST определённо кажется бессмыслицей.

В примере 5.13 Эрит успешно выполняет SYN/FIN-сканирование Google. По-видимому, scanme.nmap.org ему уже наскучил.

Пример 5.13. SYN/FIN-сканирование Google

```
krad# nmap -sS --scanflags SYNFIN -T4 www.google.com

Starting Nmap ( http://nmap.org )
Warning: Hostname www.google.com resolves to 4 IPs. Using 74.125.19.99.
Interesting ports on cf-in-f99.google.com (74.125.19.99):
Not shown: 996 filtered ports
PORT    STATE  SERVICE
80/tcp  open   http
113/tcp closed auth
179/tcp closed bgp
443/tcp open   https
Nmap done: 1 IP address (1 host up) scanned in 7.58 seconds
```

Схожие типы сканирования, например SYN/URG или SYN/PSH/URG/FIN, обычно тоже работают. Если пробиться не удаётся, не забывайте об уже упомянутом варианте SYN/RST.

5.6.2. PSH-сканирование

В разделе 5.5 «Сканирования TCP FIN, NULL и Xmas (-sF, -sN, -sX)» отмечалось, что соответствующие RFC системы позволяют сканировать порты любой комбинацией флагов FIN, PSH и URG. Хотя существует восемь возможных перестановок, Nmap предлагает лишь три готовых режима: NULL, FIN и Xmas. Проявите индивидуальность, попробовав вместо них сканирование PSH/URG или FIN/PSH. Результаты редко отличаются от трёх готовых режимов, но остаётся небольшой шанс ускользнуть от систем обнаружения сканирования.

Чтобы выполнить такое сканирование, просто задайте желаемые флаги с помощью --scanflags и укажите FIN-сканирование (-sF) как базовый тип; выбор NULL или Xmas ничего бы не изменил. В примере 5.14 показано PSH-сканирование машины Linux в моей локальной сети.

Пример 5.14. Пользовательское PSH-сканирование

```
krad# nmap -sF --scanflags PSH para
```

```
Starting Nmap ( http://nmap.org )
Interesting ports on para (192.168.10.191):
(The 995 ports scanned but not shown below are in state: closed)
PORT     STATE         SERVICE
22/tcp   open|filtered ssh
53/tcp   open|filtered domain
111/tcp  open|filtered rpcbind
515/tcp  open|filtered printer
6000/tcp open|filtered X11
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)
Nmap done: 1 IP address (1 host up) scanned in 5.95 seconds
```

Поскольку все эти сканирования работают одинаково, я мог бы оставить только один из параметров `-sF`, `-sN` и `-sX`, позволив пользователям имитировать остальные с помощью `--scanflags`. Делать это не планируется, поскольку сокращённые параметры легче запомнить и применять. Тем не менее вы можете попробовать имитацию, чтобы похвастаться своим мастерством Nmap. Выполните `nmap -sF --scanflags FINPSHURG target` вместо более обыденной команды `nmap -sX target`.

> **Предупреждение**
>
> По моему опыту, излишне сложные командные строки Nmap не производят впечатления на девушек. Обычно они отвечают снисходительной усмешкой, вероятно понимая, что команда избыточна.

5.7. TCP ACK-сканирование (-sA)

Это сканирование отличается от уже рассмотренных тем, что никогда не определяет порты `open` или даже `open|filtered`. Оно применяется для отображения наборов правил межсетевого экрана: выясняет, контролирует ли он состояние и какие порты фильтруются.

ACK-сканирование включается параметром `-sA`. В его пакете-пробе установлен только флаг ACK, если не используется `--scanflags`. При сканировании нефильтруемых систем и открытые, и закрытые порты возвращают пакет RST. Затем Nmap помечает их как `unfiltered`, то есть доступные для пакета ACK, но не устанавливает, открыты они или закрыты. Порты, которые не отвечают либо возвращают определённые сообщения об ошибках ICMP, помечаются как `filtered`. Полные сведения приведены в таблице 5.5.

Таблица 5.5. Интерпретация Nmap ответов на пробу ACK-сканирования

Ответ на пробу	— назначенное состояние
• Ответ RST	— `unfiltered`
• Ответ не получен даже после повторных передач	— `filtered`
• Ошибка ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13)	— `filtered`

ACK-сканирование применяется так же, как большинство других типов: достаточно добавить один параметр, в данном случае `-sA`. В примере 5.15 показано ACK-сканирование Scanme.

Пример 5.15. Типичное ACK-сканирование

```
krad# nmap -sA -T4 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE      SERVICE
22/tcp  unfiltered ssh
25/tcp  unfiltered smtp
53/tcp  unfiltered domain
```

```
70/tcp  unfiltered gopher
80/tcp  unfiltered http
113/tcp unfiltered auth
Nmap done: 1 IP address (1 host up) scanned in 4.01 seconds
```

Одно из самых интересных применений ACK-сканирования - различение межсетевых экранов с контролем состояния и без него. О том, как и зачем это делать, см. в разделе 10.3.2 «ACK-сканирование».

Иногда сочетание типов сканирования позволяет получить о системе дополнительные сведения. Для примера сначала вернёмся к FIN-сканированию Docsrv из примера 5.12 «FIN-сканирование Docsrv». В том случае Nmap находит закрытые порты, но 39 из них перечислены как `open|filtered`, поскольку FIN-сканирование не способно различить эти два состояния. Теперь рассмотрим ACK-сканирование того же узла в примере 5.16 «ACK-сканирование Docsrv». Два из этих 39 ранее неопределённых портов оказываются `filtered`. Остальные 37, судя по строке стандартных портов над таблицей, находятся в состоянии `unfiltered`. Это значит - открыты или закрыты. Если один тип сканирования определяет порт как открытый или фильтруемый, а другой - как открытый или закрытый, логика требует признать его открытым. Сочетая оба типа сканирования, мы выяснили, что на Docsrv открыты 37 портов, два фильтруются, а 961 закрыт. Хотя логическая дедукция хорошо помогла здесь определить состояния портов, полагаться на этот метод можно не всегда. Он предполагает, что разные типы сканирования всегда возвращают для одного порта согласованные состояния, а это неверно. Из-за межсетевых экранов и свойств стека TCP результаты разных сканирований одной машины могут заметно различаться. На Docsrv мы увидели, что SYN-сканирование считает порт SSH (`tcp/22`) фильтруемым, а ACK-сканирование - нефильтруемым. При исследовании граничных случаев и странно настроенных сетей интерпретация результатов Nmap превращается в искусство, которому помогают опыт и интуиция.

Пример 5.16. ACK-сканирование Docsrv

```
# nmap -sA -T4 docsrv.caldera.com

Starting Nmap ( http://nmap.org )
Interesting ports on docsrv.caldera.com (216.250.128.247):
Not shown: 998 unfiltered ports
PORT     STATE    SERVICE
135/tcp  filtered msrpc
1434/tcp filtered ms-sql-m
Nmap done: 1 IP address (1 host up) scanned in 7.20 seconds
```

5.8. TCP Window-сканирование (-sW)

Window-сканирование полностью совпадает с ACK-сканированием, за исключением того, что использует особенность реализации некоторых систем, чтобы отличать открытые порты от закрытых, а не всегда выводить `unfiltered` при получении RST. Для этого оно проверяет значение TCP Window в возвращённых пакетах RST. В некоторых системах открытые порты используют положительный размер окна даже в пакетах RST, тогда как у закрытых портов окно нулевое. Window-сканирование отправляет ту же простую пробу ACK, что и ACK-сканирование, интерпретируя результаты согласно таблице 5.6.

Таблица 5.6. Интерпретация Nmap ответов на ACK-пробу Window-сканирования

Ответ на пробу — назначенное состояние

• Ответ TCP RST с ненулевым полем окна — `open`
• Ответ TCP RST с нулевым полем окна — `closed`
• Ответ не получен даже после повторных передач — `filtered`

• Ошибка ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13) — `filtered`

Это сканирование опирается на особенность реализации, встречающуюся у меньшинства систем в Интернете, поэтому доверять ему можно не всегда. Системы, которые её не поддерживают, обычно возвращают все порты как `closed`. Разумеется, возможно, что на машине действительно нет открытых портов. Если большинство просканированных портов закрыто, но несколько распространённых номеров, например 22, 25 и 53, открыты, система, скорее всего, подвержена этому методу. Иногда системы демонстрируют даже совершенно противоположное поведение. Если сканирование показывает 997 открытых портов и три закрытых или фильтруемых, то именно эти три вполне могут быть по-настоящему открытыми.

Хотя это сканирование подходит не для всякой ситуации, иногда оно бывает весьма полезно. Вспомним пример 5.12 «FIN-сканирование Docsrv», где показано множество портов `open|filtered`, не найденных обычным SYN-сканированием. Проблема в том, что с помощью того FIN-сканирования нельзя отличить открытые порты от фильтруемых. В предыдущем разделе было показано, что их можно различить, объединив результаты FIN- и ACK-сканирования. В данном случае Window-сканирование ещё больше упрощает задачу, не требуя результатов FIN-сканирования, как показано в примере 5.17.

Пример 5.17. Window-сканирование `docsrv.caldera.com`

```
# nmap -sW -T4 docsrv.caldera.com

Starting Nmap ( http://nmap.org )
Interesting ports on docsrv.caldera.com (216.250.128.247):
Not shown: 961 closed ports
PORT      STATE    SERVICE
7/tcp     open     echo
9/tcp     open     discard
11/tcp    open     systat
13/tcp    open     daytime
15/tcp    open     netstat
19/tcp    open     chargen
21/tcp    open     ftp
22/tcp    open     ssh
23/tcp    open     telnet
25/tcp    open     smtp
37/tcp    open     time
79/tcp    open     finger
80/tcp    open     http
110/tcp   open     pop3
111/tcp   open     rpcbind
135/tcp   filtered msrpc
[14 open ports omitted for brevity]
1434/tcp  filtered ms-sql-m
2000/tcp  open     callbook
2766/tcp  open     listen
3000/tcp  open     ppp
3306/tcp  open     mysql
6112/tcp  open     dtspc
32770/tcp open     sometimes-rpc3
32771/tcp open     sometimes-rpc5
32772/tcp open     sometimes-rpc7
Nmap done: 1 IP address (1 host up) scanned in 7.30 seconds
```

Эрит получил именно те результаты, которые хотел! Показаны те же 39 интересующих портов, что и при FIN-сканировании, но на этот раз различаются два фильтруемых порта (MS-SQL и MSRPC) и 37 действительно открытых. Это те же результаты, которые Эрит получил в предыдущем разделе, объединив результаты FIN- и ACK-сканирования. Проверка согласованности результатов - ещё одна веская причина пробовать против целевой сети несколько типов сканирования.

5.9. TCP Maimon-сканирование (-sM)

Maimon-сканирование названо в честь его первооткрывателя Уриэля Маймона. Он описал этот метод в выпуске 49 журнала *Phrack* за ноябрь 1996 года. Nmap, уже включавший этот метод, был выпущен двумя номерами позже. Этот метод полностью совпадает со сканированиями NULL, FIN и Xmas, за исключением того, что проба имеет флаги FIN/ACK. Согласно RFC 793 по TCP, в ответ на такую пробу должен создаваться пакет RST независимо от того, открыт порт или закрыт. Однако Уриэль заметил, что многие системы, происходящие от BSD, просто отбрасывают пакет, если порт открыт. Nmap использует это для определения открытых портов, как показано в таблице 5.7.

Таблица 5.7. Интерпретация Nmap ответов на пробу Maimon-сканирования

Ответ на пробу	— назначенное состояние
• Ответ не получен даже после повторных передач	— `open|filtered`
• Пакет TCP RST	— `closed`
• Ошибка ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13)	— `filtered`

Параметр Nmap для Maimon-сканирования - `-sM`. Хотя в 1996 году этот вариант был весьма полезен, современные системы редко проявляют данную ошибку. Они возвращают RST для всех портов, из-за чего каждый порт кажется закрытым. Такой результат показан в примере 5.18.

Пример 5.18. Неудачное Maimon-сканирование

```
# nmap -sM -T4 para

Starting Nmap ( http://nmap.org )
All 1000 scanned ports on para (192.168.10.191) are: closed
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)
Nmap done: 1 IP address (1 host up) scanned in 4.19 seconds
```

5.10. TCP Idle-сканирование (-sI)

В 1998 году исследователь безопасности Antirez, написавший также часто используемый в этой книге инструмент `hping2`, опубликовал в списке рассылки Bugtraq новый остроумный метод сканирования портов. Idle-сканирование, как его стали называть, позволяет выполнять полностью слепое сканирование портов. Злоумышленники могут сканировать цель, не отправляя ей со своего IP-адреса ни единого пакета! Вместо этого хитроумная атака по стороннему каналу позволяет отразить сканирование от простодушного «узла-зомби». Отчёты системы обнаружения вторжений (IDS) укажут на невиновного зомби как на нападающего. Помимо исключительной скрытности, этот тип сканирования позволяет обнаруживать основанные на IP доверительные отношения между машинами.

Хотя Idle-сканирование сложнее всех рассмотренных до сих пор методов, для его понимания не требуется быть специалистом по TCP/IP. Его можно составить из следующих основных фактов:

• Один из способов определить, открыт ли порт TCP, - отправить ему пакет SYN, устанавливающий сеанс. Если порт открыт, целевая машина ответит пакетом SYN/ACK, подтверждающим запрос сеанса, а если закрыт - пакетом RST. На этом основано рассмотренное ранее SYN-сканирование.

• Машина, получившая незапрошенный пакет SYN/ACK, ответит RST. Незапрошенный RST будет проигнорирован.

• Каждый пакет IP в Интернете имеет идентификационный номер фрагмента (IP ID). Поскольку многие операционные системы просто увеличивают этот номер с каждым отправленным пакетом, опрос IP ID может сообщить злоумышленнику, сколько пакетов было отправлено после

предыдущей пробы.

Объединив эти свойства, можно сканировать целевую сеть с подделкой своей личности так, чтобы казалось, будто сканирование выполняла невиновная машина-зомби.

5.10.1. Idle-сканирование шаг за шагом

В основе Idle-сканирования лежат три шага, повторяемые для каждого порта:

1. Запросить IP ID зомби и записать его.

2. Подделать пакет SYN от имени зомби и отправить его желаемому порту цели. В зависимости от состояния порта реакция цели может увеличить IP ID зомби, а может и не увеличить.

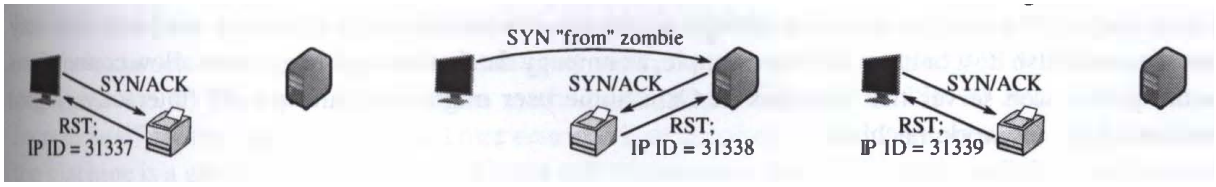
3. Снова запросить IP ID зомби. Затем состояние целевого порта определяется сравнением нового IP ID со значением, записанным на шаге 1.

После этого процесса IP ID зомби должен увеличиться на единицу или на два. Увеличение на единицу означает, что зомби не отправлял никаких пакетов, кроме ответа на пробу злоумышленника. Отсутствие отправленных пакетов означает, что порт не открыт: цель должна была либо отправить зомби пакет RST, который был проигнорирован, либо вообще ничего не отправить. Увеличение на два означает, что между двумя пробами зомби отправил пакет. Этот дополнительный пакет обычно означает, что порт открыт: предположительно, цель отправила зомби пакет SYN/ACK в ответ на поддельный SYN, что вызвало пакет RST от зомби. Увеличение больше чем на два обычно указывает на неподходящий узел-зомби. Возможно, его номера IP ID непредсказуемы или он ведёт обмен данными, не связанный с Idle-сканированием.

Хотя происходящее с закрытым портом немного отличается от происходящего с фильтруемым, в обоих случаях злоумышленник измеряет один и тот же результат - увеличение IP ID на 1. Поэтому Idle-сканирование не может различать закрытые и фильтруемые порты. Когда Nmap регистрирует увеличение IP ID на 1, он помечает порт как `closed|filtered`.

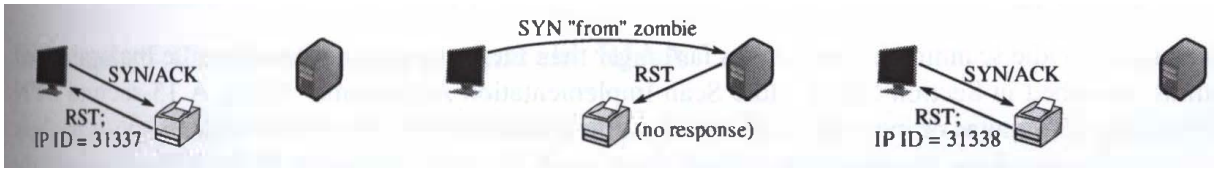
Для желающих узнать подробности следующие три схемы точно показывают, что происходит в трёх случаях: с открытым, закрытым и фильтруемым портом. Действующие лица на каждой из них - злоумышленник, зомби и цель.

Рисунок 5.6. Idle-сканирование открытого порта



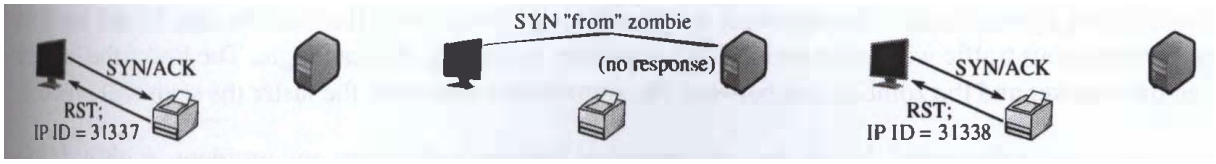
Шаг 1: запросить IP ID зомби. Злоумышленник отправляет зомби пакет SYN/ACK. Зомби не ожидает SYN/ACK и возвращает RST, раскрывая свой IP ID. Шаг 2: подделать пакет SYN от имени зомби. Цель отвечает пакетом SYN/ACK на SYN, который выглядит пришедшим от зомби. Зомби не ожидает его и возвращает RST, увеличивая при этом свой IP ID. Шаг 3: снова запросить IP ID зомби. После шага 1 IP ID зомби увеличился на 2, следовательно, порт открыт.

Рисунок 5.7. Idle-сканирование закрытого порта



Шаг 1: запросить IP ID зомби. Злоумышленник отправляет зомби пакет SYN/ACK. Зомби не ожидает SYN/ACK и возвращает RST, раскрывая свой IP ID. Этот шаг всегда одинаков. Шаг 2: подделать пакет SYN от имени зомби. Цель отвечает на SYN, который выглядит пришедшим от зомби, пакетом RST, поскольку порт закрыт. Зомби игнорирует незапрошенный RST, поэтому его IP ID не меняется. Шаг 3: снова запросить IP ID зомби. После шага 1 IP ID зомби увеличился лишь на 1, следовательно, порт не открыт.

Рисунок 5.8. Idle-сканирование фильтруемого порта



Шаг 1: запросить IP ID зомби. Как и в двух других случаях, злоумышленник отправляет зомби SYN/ACK, и зомби раскрывает свой IP ID. Шаг 2: подделать пакет SYN от имени зомби. Цель, упорно фильтрующая свой порт, игнорирует SYN, который выглядит пришедшим от зомби. Зомби не знает о случившемся и не увеличивает свой IP ID. Шаг 3: снова запросить IP ID зомби. После шага 1 IP ID зомби увеличился лишь на 1, следовательно, порт не открыт. С точки зрения злоумышленника этот фильтруемый порт неотличим от закрытого.

Idle-сканирование - вершина скрытного сканирования. Nmap предлагает сканирование с ложными адресами (`-D`), помогающее пользователям скрывать свою личность, но при нём, в отличие от Idle-сканирования, злоумышленнику всё равно приходится отправлять цели некоторые пакеты со своего настоящего IP-адреса, чтобы получить результаты. Одно из следствий Idle-сканирования состоит в том, что системы обнаружения вторжений обычно выдают предупреждения о том, что сканирование против них запустила машина-зомби. Поэтому этот метод можно использовать, чтобы подставить за сканирование другую сторону. Помните о такой возможности, читая предупреждения своей IDS.

Уникальное преимущество Idle-сканирования состоит в том, что с его помощью можно преодолевать некоторые межсетевые экраны и маршрутизаторы с пакетной фильтрацией. Фильтрация по исходному IP-адресу - распространённый, хотя и слабый, механизм безопасности, ограничивающий список машин, которым разрешено соединяться с защищённым узлом или сетью. Например, сервер базы данных компании может разрешать соединения только с обращающегося к нему общедоступного веб-сервера. Или домашний пользователь может разрешать соединения SSH для интерактивного входа только со своих рабочих машин.

Более тревожный сценарий возникает, когда какая-нибудь важная персона в компании требует, чтобы сетевые администраторы проделали дыру в межсетевом экране и позволили ей обращаться к внутренним сетевым ресурсам с домашнего IP-адреса. Такое случается, когда руководители не хотят или не умеют пользоваться безопасными альтернативами VPN.

Иногда Idle-сканирование позволяет отобразить такие доверительные отношения. Ключевой фактор в том, что результаты Idle-сканирования перечисляют открытые порты с точки зрения узла-зомби. Обычное сканирование упомянутого сервера базы данных может не показать открытых портов, но Idle-сканирование с IP-адресом веб-сервера в качестве зомби способно выявить доверительное отношение, показав открытыми порты служб базы данных.

Отображение таких доверительных отношений бывает очень полезно злоумышленникам для определения приоритетных целей. Обсуждавшийся выше веб-сервер может казаться злоумышленнице заурядным, пока она не заметит его особого доступа к базе данных.

Недостаток Idle-сканирования в том, что оно занимает намного больше времени, чем большинство других типов. Несмотря на оптимизированные алгоритмы, описанные в разделе 5.10.4 «Алгоритмы реализации Idle-сканирования», SYN-сканирование длительностью 15 секунд в режиме Idle может

занять 15 минут и больше. Другая проблема: вы должны иметь возможность подделывать пакеты так, будто они исходят от зомби, и доставлять их целевой машине. Многие интернет-провайдеры, особенно коммутируемого доступа и домашнего широкополосного подключения, теперь применяют фильтрацию исходящего трафика, чтобы предотвращать такую подделку пакетов. Провайдеры более высокого класса, например услуг размещения и линий T1, делают это гораздо реже. Если такая фильтрация действует, Nmap быстро выведет сообщение об ошибке для каждого испробованного зомби. Если сменить провайдера нельзя, можно попробовать другой IP-адрес в сети того же провайдера. Иногда фильтрация блокирует лишь подделку IP-адресов вне диапазона, используемого клиентами. Ещё одна трудность Idle-сканирования - необходимость найти работающий узел-зомби, как описано в следующем разделе.

5.10.2. Поиск работающего узла-зомби для Idle-сканирования

Первый шаг выполнения Idle-сканирования по IP ID - найти подходящего зомби. Он должен присваивать пакетам IP ID последовательно и глобально, а не отдельно для каждого узла, с которым взаимодействует. Он должен бездействовать, откуда и произошло название сканирования, поскольку посторонний трафик будет увеличивать последовательность IP ID и путать логику сканирования. Чем меньше задержка между злоумышленником и зомби, а также между зомби и целью, тем быстрее пойдёт сканирование.

При попытке Idle-сканирования Nmap проверяет предложенного зомби и сообщает о любых проблемах с ним. Если один не работает, попробуйте другого. В Интернете достаточно уязвимых узлов, поэтому кандидатов в зомби найти несложно. Поскольку узлы должны бездействовать, выбор известного узла вроде `www.yahoo.com` или `google.com` почти никогда не срабатывает.

Обычный подход - просто выполнить ping-сканирование Nmap какой-либо сети. Можно использовать режим случайного выбора IP-адресов Nmap (`-iR`), но тогда, скорее всего, будут найдены далёкие зомби с большой задержкой. Лучшие результаты даёт выбор сети рядом с вашим исходным адресом или рядом с целью. Можно пробовать Idle-сканирование с каждым доступным узлом из результатов ping-сканирования, пока не найдётся работающий. Как обычно, перед использованием чужих машин для неожиданных целей вроде Idle-сканирования лучше спросить разрешения.

Значок принтера для обозначения зомби на иллюстрациях был выбран не только ради шутки: простые сетевые устройства часто оказываются отличными зомби, поскольку обычно мало используются, то есть бездействуют, и построены на простых сетевых стеках, уязвимых для обнаружения трафика по IP ID.

Выбрать хорошего зомби помогает сканирование портов и определение ОС (`-O`) в сети кандидатов, а не одно лишь ping-сканирование. Если включён подробный режим (`-v`), механизм определения ОС обычно устанавливает способ формирования последовательности IP ID и выводит строку вроде `IP ID Sequence Generation: Incremental`. Если тип указан как `Incremental` или `Broken little-endian incremental`, машина является хорошим кандидатом в зомби. Это ещё не гарантирует успеха, поскольку Solaris и некоторые другие системы создают новую последовательность IP ID для каждого узла, с которым взаимодействуют. Кроме того, узел может быть слишком занят. Определение ОС и список открытых портов также помогают выявлять системы, которые, вероятно, бездействуют.

Хотя поиск подходящего зомби требует некоторой первоначальной работы, удачные варианты можно использовать повторно.

5.10.3. Выполнение Idle-сканирования

Когда подходящий зомби найден, выполнить сканирование легко. Просто передайте имя узла-зомби параметру `-sI`, а всё остальное сделает Nmap. В примере 5.19 Эрит сканирует Американскую ассоциацию звукозаписывающих компаний, отражая Idle-сканирование от машины Adobe по имени Kiosk.

Пример 5.19. Idle-сканирование против RIAA

```
# nmap -PN -p- -sI kiosk.adobe.com www.riaa.com

Starting Nmap ( http://nmap.org )
Idlescan using zombie kiosk.adobe.com (192.150.13.111:80); Class: Incremental
Interesting ports on 208.225.90.120:
(The 65522 ports scanned but not shown below are in state: closed)
Port      State Service
21/tcp    open  ftp
25/tcp    open  smtp
80/tcp    open  http
111/tcp   open  sunrpc
135/tcp   open  loc-srv
443/tcp   open  https
1027/tcp  open  IIS
1030/tcp  open  iad1
2306/tcp  open  unknown
5631/tcp  open  pcanywheredata
7937/tcp  open  unknown
7938/tcp  open  unknown
36890/tcp open  unknown
Nmap done: 1 IP address (1 host up) scanned in 2594.47 seconds
```

Из приведённого сканирования мы узнаём, что RIAA не слишком заботится о безопасности: обратите внимание на открытые порты PC Anywhere, `portmapper` и Legato `nsrexec`. Поскольку межсетевого экрана у них, по-видимому, нет, маловероятно и наличие IDS. Но если IDS всё же есть, виновником сканирования она покажет `kiosk.adobe.com`. Параметр `-PN` не позволяет Nmap отправить машине RIAA начальный ping-пакет, который раскрыл бы настоящий адрес Эрита. Сканирование заняло много времени, потому что для проверки всех 65 тысяч портов был указан `-p-`. Не пытайтесь использовать Kiosk для своих сканирований: эта машина уже удалена.

По умолчанию Nmap подделывает пробы цели, используя исходный порт 80 зомби. Другой порт можно выбрать, добавив к имени зомби двоеточие и номер порта, например `-sI kiosk.adobe.com:113`. Выбранный порт не должен фильтроваться ни от злоумышленника, ни от цели. SYN-сканирование зомби должно показывать порт в состоянии `open` или `closed`.

5.10.4. Алгоритмы реализации Idle-сканирования

Хотя в разделе 5.10.1 «Idle-сканирование шаг за шагом» описан основополагающий уровень Idle-сканирования, реализация Nmap намного сложнее. Ключевые отличия - параллелизм для быстрого выполнения и избыточность для уменьшения ложноположительных результатов.

Распараллелить Idle-сканирование сложнее, чем другие методы, из-за косвенного способа определения состояний портов. Если Nmap отправляет пробы множеству портов цели, а затем проверяет новое значение IP ID зомби, число увеличений IP ID показывает, сколько целевых портов открыто, но не какие именно. На самом деле это не такая серьёзная проблема, поскольку подавляющее большинство портов в крупном сканировании будут `closed|filtered`. Так как лишь открытые порты вызывают увеличение IP ID, Nmap не увидит промежуточных увеличений и сможет пометить всю группу портов как `closed|filtered`. Nmap может параллельно сканировать группы до 100 портов. Если после проверки группы Nmap обнаруживает, что IP ID зомби увеличился <N>

раз, в группе должно быть <N> открытых портов. Затем Nmap находит их двоичным поиском: делит группу надвое и отдельно отправляет пробы каждой половине. Если подгруппа не показывает открытых портов, все её порты помечаются как `closed|filtered`. Если подгруппа показывает один или несколько открытых портов, она снова делится, и процесс продолжается до их выявления. Этот метод усложняет работу, но способен сократить время сканирования на порядок по сравнению с последовательной проверкой по одному порту.

Надёжность - ещё одна важная проблема Idle-сканирования. Если во время сканирования узел-зомби отправляет пакеты каким-либо посторонним машинам, его IP ID увеличивается. Из-за этого Nmap решает, что нашёл открытый порт. К счастью, здесь тоже помогает параллельное сканирование. Если Nmap проверяет 100 портов в группе, а увеличение IP ID указывает на два открытых порта, он делит группу на две подгруппы по 50 портов. Когда Nmap выполняет сканирование IP ID обеих подгрупп, суммарное увеличение IP ID зомби снова обязано равняться двум. В противном случае Nmap обнаруживает несогласованность и повторно сканирует группы. Он также изменяет размер группы и временные параметры сканирования в зависимости от обнаруженного уровня надёжности зомби. Если Nmap выявляет слишком много несогласованных результатов, он прекращает работу и просит пользователя указать лучшего зомби.

Иногда трассировка пакетов - лучший способ понять такие сложные алгоритмы и методы. Параметр Nmap `--packet-trace` снова позволяет без труда получить её при необходимости. В оставшейся части раздела приводится снабжённая пояснениями трассировка пакетов настоящего Idle-сканирования семи портов. IP-адреса заменены словами `Attacker`, `Zombie` и `Target`, а некоторые несущественные части строк трассировки, например размер окна TCP, удалены для ясности.

```
Attacker# nmap -sI Zombie -PN -p20-25,110 -r --packet-trace -v Target
Starting Nmap ( http://nmap.org )
```

Параметр `-PN` необходим для скрытности, иначе цели отправлялись бы ping-пакеты с настоящего адреса злоумышленника. Определение версий также раскрыло бы истинный адрес, поэтому `-sV` не указан. Параметр `-r`, отключающий перемешивание портов, используется лишь для того, чтобы за примером было легче следить.

Сначала Nmap проверяет способ формирования последовательности IP ID у Zombie, отправляя ему шесть пакетов SYN/ACK и анализируя ответы. Это помогает Nmap сразу отсеять неподходящих зомби. Проверка необходима ещё и потому, что некоторые системы, обычно машины Microsoft Windows, хотя так ведут себя не все машины Windows, увеличивают IP ID с каждым отправленным пакетом не на единицу, а на 256. Это происходит на машинах с порядком байтов little-endian, когда они не преобразуют IP ID в сетевой порядок байтов big-endian. Nmap использует эти начальные пробы, чтобы обнаружить и обойти данную проблему.

```
SENT (0.0060s) TCP Attacker:51824 > Zombie:80 SA id=35996
SENT (0.0900s) TCP Attacker:51825 > Zombie:80 SA id=25914
SENT (0.1800s) TCP Attacker:51826 > Zombie:80 SA id=39591
RCVD (0.1550s) TCP Zombie:80 > Attacker:51824 R id=15669
SENT (0.2700s) TCP Attacker:51827 > Zombie:80 SA id=43604
RCVD (0.2380s) TCP Zombie:80 > Attacker:51825 R id=15670
SENT (0.3600s) TCP Attacker:51828 > Zombie:80 SA id=34186
RCVD (0.3280s) TCP Zombie:80 > Attacker:51826 R id=15671
SENT (0.4510s) TCP Attacker:51829 > Zombie:80 SA id=27949
RCVD (0.4190s) TCP Zombie:80 > Attacker:51827 R id=15672
RCVD (0.5090s) TCP Zombie:80 > Attacker:51828 R id=15673
RCVD (0.5990s) TCP Zombie:80 > Attacker:51829 R id=15674
Idlescan using zombie Zombie (Zombie:80); Class: Incremental
```

Эта проверка показывает, что зомби работает нормально. Каждый IP ID увеличивается на единицу относительно предыдущего. Следовательно, система, по-видимому, бездействует и уязвима для обнаружения трафика по IP ID. Эти многообещающие результаты ещё должны пройти следующую проверку, при которой Nmap подделывает четыре пакета к Zombie так, будто они исходят от Target.

Затем он опрашивает зомби, чтобы удостовериться в увеличении IP ID. Если увеличения нет, вероятно, либо провайдер злоумышленника блокирует поддельные пакеты, либо зомби использует отдельный счётчик последовательности IP ID для каждого узла, с которым взаимодействует. Оба явления распространены, поэтому Nmap всегда выполняет эту проверку. Последний известный IP ID Zombie, как показано выше, равен 15674.

```
SENT (0.5990s) TCP Target:51823 > Zombie:80 SA id=1390
SENT (0.6510s) TCP Target:51823 > Zombie:80 SA id=24025
SENT (0.7110s) TCP Target:51823 > Zombie:80 SA id=15046
SENT (0.7710s) TCP Target:51823 > Zombie:80 SA id=48658
SENT (1.0800s) TCP Attacker:51987 > Zombie:80 SA id=27659
RCVD (1.2290s) TCP Zombie:80 > Attacker:51987 R id=15679
```

Четыре поддельных пакета вместе с пробой от Attacker заставили Zombie увеличить IP ID с 15674 до 15679. Прекрасно! Теперь начинается настоящее сканирование. Помните, что 15679 - последний IP ID Zombie.

```
Initiating Idlescan against Target
SENT (1.2290s) TCP Zombie:80 > Target:20 S id=13200
SENT (1.2290s) TCP Zombie:80 > Target:21 S id=3737
SENT (1.2290s) TCP Zombie:80 > Target:22 S id=65290
SENT (1.2290s) TCP Zombie:80 > Target:23 S id=10516
SENT (1.4610s) TCP Attacker:52050 > Zombie:80 SA id=33202
RCVD (1.6090s) TCP Zombie:80 > Attacker:52050 R id=15680
```

Nmap проверяет порты 20-23. Затем он опрашивает Zombie и обнаруживает новый IP ID 15680, всего на единицу больше предыдущего значения 15679. Между этими двумя известными пакетами увеличений IP ID не было, поэтому порты 20-23, вероятно, имеют состояние `closed|filtered`. Возможно также, что SYN/ACK от Target просто ещё не прибыл. В таком случае Zombie пока не ответил RST и его IP ID ещё не увеличился. Для точности Nmap позже повторно проверит эти порты.

```
SENT (1.8510s) TCP Attacker:51986 > Zombie:80 SA id=49278
RCVD (1.9990s) TCP Zombie:80 > Attacker:51986 R id=15681
```

Nmap выполняет ещё один опрос, поскольку после последней отправленной пробы прошло четыре десятых секунды. За это время Zombie, если он на самом деле не бездействует, мог взаимодействовать с другими узлами, что позднее привело бы к неточностям, если не обнаружить это здесь. К счастью, такого не произошло: следующий IP ID, как и ожидалось, равен 15681.

```
SENT (2.0000s) TCP Zombie:80 > Target:24 S id=23928
SENT (2.0000s) TCP Zombie:80 > Target:25 S id=50425
SENT (2.0000s) TCP Zombie:80 > Target:110 S id=14207
SENT (2.2300s) TCP Attacker:52026 > Zombie:80 SA id=26941
RCVD (2.3800s) TCP Zombie:80 > Attacker:52026 R id=15684
```

Nmap проверяет порты 24, 25 и 110, а затем запрашивает IP ID Zombie. Он перескочил с 15681 на 15684. Значения 15682 и 15683 были пропущены, поэтому два из трёх портов, вероятно, открыты. Nmap не знает, какие именно два, а результат может быть ложноположительным. Поэтому Nmap углубляет проверку, разделяя сканирование на подгруппы.

```
SENT (2.6210s) TCP Attacker:51867 > Zombie:80 SA id=18869
RCVD (2.7690s) TCP Zombie:80 > Attacker:51867 R id=15685
SENT (2.7690s) TCP Zombie:80 > Target:24 S id=30023
SENT (2.7690s) TCP Zombie:80 > Target:25 S id=47253
SENT (3.0000s) TCP Attacker:51979 > Zombie:80 SA id=12077
RCVD (3.1480s) TCP Zombie:80 > Attacker:51979 R id=15687
```

Первая подгруппа состоит из портов 24 и 25. IP ID перескакивает с 15685 на 15687, а значит, один из этих двух портов, скорее всего, открыт. Nmap снова применяет подход «разделяй и властвуй», проверяя каждый порт отдельно.

```
SENT (3.3910s) TCP Attacker:51826 > Zombie:80 SA id=32515
RCVD (3.5390s) TCP Zombie:80 > Attacker:51826 R id=15688
```

```
SENT (3.5390s) TCP Zombie:80 > Target:24 S id=47868
SENT (3.7710s) TCP Attacker:52012 > Zombie:80 SA id=14042
RCVD (3.9190s) TCP Zombie:80 > Attacker:52012 R id=15689
```

Проверка порта 24 не показывает скачка IP ID. Следовательно, этот порт не открыт. По полученным результатам Nmap предварительно установил:

• порты 20-23 имеют состояние `closed|filtered`;

• два из портов 24, 25 и 110 открыты;

• один из портов 24 и 25 открыт;

• порт 24 имеет состояние `closed|filtered`.

Достаточно долго рассматривая эту головоломку, вы найдёте единственное решение: порты 25 и 110 открыты, а остальные пять имеют состояние `closed|filtered`. Руководствуясь этой логикой, Nmap уже мог бы прекратить сканирование и вывести результаты. Раньше он так и делал, но это приводило к слишком большому числу ложноположительных открытых портов, когда Zombie не был по-настоящему бездействующим. Поэтому Nmap продолжает сканирование, чтобы проверить результаты:

```
SENT (4.1600s) TCP Attacker:51858 > Zombie:80 SA id=6225
RCVD (4.3080s) TCP Zombie:80 > Attacker:51858 R id=15690
SENT (4.3080s) TCP Zombie:80 > Target:25 S id=35713

SENT (4.5410s) TCP Attacker:51856 > Zombie:80 SA id=28118
RCVD (4.6890s) TCP Zombie:80 > Attacker:51856 R id=15692
Discovered open port 25/tcp on Target
SENT (4.6900s) TCP Zombie:80 > Target:110 S id=9943
SENT (4.9210s) TCP Attacker:51836 > Zombie:80 SA id=62254
RCVD (5.0690s) TCP Zombie:80 > Attacker:51836 R id=15694
```

Пробы портов 25 и 110 показывают, что они открыты, как мы и заключили ранее.

```
SENT (5.0690s) TCP Zombie:80 > Target:20 S id=8168
SENT (5.0690s) TCP Zombie:80 > Target:21 S id=36717
SENT (5.0690s) TCP Zombie:80 > Target:22 S id=4063
SENT (5.0690s) TCP Zombie:80 > Target:23 S id=54771
SENT (5.3200s) TCP Attacker:51962 > Zombie:80 SA id=38763
RCVD (5.4690s) TCP Zombie:80 > Attacker:51962 R id=15695
SENT (5.7910s) TCP Attacker:51887 > Zombie:80 SA id=61034
RCVD (5.9390s) TCP Zombie:80 > Attacker:51887 R id=15696
```

Для полной уверенности Nmap снова пробует порты 20-23. Запрос IP ID Zombie не показывает скачка последовательности. На случай, если SYN/ACK от Target к Zombie пришёл с опозданием, Nmap выполняет ещё один запрос IP ID. Он снова не показывает открытых портов. Теперь Nmap достаточно уверен в результатах, чтобы вывести их.

```
The Idlescan took 5 seconds to scan 7 ports.
Interesting ports on Target:
PORT    STATE           SERVICE
20/tcp  closed|filtered ftp-data
21/tcp  closed|filtered ftp
22/tcp  closed|filtered ssh
23/tcp  closed|filtered telnet
24/tcp  closed|filtered priv-mail
25/tcp  open            smtp
110/tcp open            pop3

Nmap finished: 1 IP address (1 host up) scanned in 5.949 seconds
```

Полные сведения о реализации Idle-сканирования Nmap можно найти в файле `idle_scan.cc` из дистрибутива исходного кода Nmap.

Хотя сканирование портов является хитроумным злоупотреблением предсказуемыми последовательностями IP ID, их можно использовать и для многих других целей. Примеры

разбросаны по всей книге, особенно по главе 10 «Обнаружение и преодоление межсетевых экранов и систем обнаружения вторжений».

5.11. Сканирование протоколов IP (-sO)

Сканирование протоколов IP позволяет определить, какие протоколы IP, например TCP, ICMP и IGMP, поддерживаются целевыми машинами. Технически это не сканирование портов, поскольку перебираются номера протоколов IP, а не номера портов TCP или UDP. Но оно всё равно использует параметр `-p` для выбора сканируемых номеров протоколов, сообщает результаты в обычном формате таблицы портов и даже применяет тот же базовый механизм сканирования, что и настоящие методы сканирования портов. Поэтому оно достаточно близко к сканированию портов, чтобы рассматривать его здесь.

Помимо собственной пользы, сканирование протоколов демонстрирует силу программного обеспечения с открытым исходным кодом. Хотя основная идея довольно проста, мне не приходило в голову добавить её, и запросов на такую возможность я не получал. Затем летом 2000 года Герхард Ригер придумал этот метод, написал превосходную реализующую его заплату и отправил её в список рассылки `nmap-hackers`. Я включил заплату в дерево Nmap и на следующий день выпустил новую версию. Немногие коммерческие программы могут похвастаться пользователями, настолько увлечёнными, что они проектируют и присылают собственные улучшения!

Сканирование протоколов работает сходно с UDP-сканированием. Вместо перебора поля номера порта в пакете UDP оно отправляет заголовки пакетов IP и перебирает восьмибитное поле протокола IP. Заголовки обычно пусты: не содержат данных и даже правильного заголовка заявленного протокола. Исключение сделано для некоторых популярных протоколов, включая TCP, UDP и ICMP. Для них добавляются правильные заголовки, поскольку иначе некоторые системы их не отправят и поскольку у Nmap уже есть функции для их создания. Вместо ожидания сообщений ICMP «порт недоступен» сканирование протоколов отслеживает сообщения ICMP «протокол недоступен». В таблице 5.8 показано, как ответы на пробы IP отображаются в состояния портов.

Таблица 5.8. Интерпретация Nmap ответов на пробу протокола IP

Ответ на пробу — назначенное состояние

• Любой ответ по любому протоколу от целевого узла — `open` для протокола, использованного в ответе, не обязательно для протокола пробы.
• Ошибка ICMP «протокол недоступен» (тип 3, код 2) — `closed`
• Другие ошибки ICMP «недоступно» (тип 3, код 1, 3, 9, 10 или 13) — `filtered`, хотя при отправке целевой машиной они доказывают, что ICMP открыт.
• Ответ не получен даже после повторных передач — `open|filtered`

Как и открытые порты протоколов TCP или UDP, каждый открытый протокол представляет потенциальное направление эксплуатации. Кроме того, результаты сканирования протоколов помогают определить назначение машины и действующую пакетную фильтрацию. У конечных узлов обычно открыты лишь TCP, UDP, ICMP и иногда IGMP, тогда как маршрутизаторы часто предлагают намного больше, включая связанные с маршрутизацией протоколы GRE и EGP. Межсетевые экраны и шлюзы VPN могут показывать связанные с шифрованием протоколы вроде IPsec и SWIPE.

Подобно сообщениям ICMP «порт недоступен», получаемым при UDP-сканировании, сообщения ICMP «протокол недоступен» часто имеют ограниченную частоту. Например, стандартная машина Linux 2.4.20 отправляет не более одного ответа ICMP «назначение недоступно» в секунду. Поскольку возможны лишь 256 номеров протоколов, эта проблема менее серьёзна, чем при

UDP-сканировании 65 536 портов. Советы из раздела 5.4.2 «Ускорение UDP-сканирования» применимы и к ускорению сканирования протоколов IP.

В командной строке сканирование протоколов применяется так же, как большинство других методов. Просто укажите `-sO` вместе с любыми общими параметрами Nmap. Обычный параметр порта (`-p`) используется для выбора номеров протоколов. Или можно применить `-F`, чтобы просканировать все протоколы из базы `nmap-protocols`. По умолчанию Nmap проверяет все 256 возможных значений. В примере 5.20 Эрит сканирует маршрутизатор в Польше, а затем типичную машину Linux в моей локальной сети.

Пример 5.20. Сканирование протоколов IP маршрутизатора и типичной машины Linux 2.4

```
# nmap -sO 62.233.173.90 para

Starting Nmap ( http://nmap.org )
Interesting protocols on ntwklan-62-233-173-90.devs.futuro.pl (62.233.173.90):
Not shown: 240 closed ports
PROTOCOL STATE         SERVICE
1        open          icmp
0        open|filtered ip
6        open          tcp
8        open|filtered egp
9        open|filtered igp
17       filtered      udp
47       open|filtered gre
53       filtered      swipe
54       open|filtered narp
55       filtered      mobile
77       filtered      sun-nd
80       open|filtered iso-ip
88       open|filtered eigrp
89       open|filtered ospfigp
94       open|filtered ipip
103      filtered      pim

Interesting protocols on para (192.168.10.191):
Not shown: 252 closed ports
PROTOCOL STATE         SERVICE
1        open          icmp
2        open|filtered igmp
6        open          tcp
17       filtered      udp
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)
```

5.12. TCP FTP Bounce-сканирование (-b)

Интересная особенность протокола FTP (RFC 959) - поддержка так называемых прокси-соединений FTP. Она позволяет пользователю подключиться к одному серверу FTP, а затем попросить отправить файлы стороннему серверу. Такая возможность допускает злоупотребления на многих уровнях, поэтому большинство серверов прекратило её поддерживать. Одно из злоупотреблений состоит в том, чтобы заставить сервер FTP сканировать порты других узлов. Достаточно по очереди просить сервер FTP отправить файл каждому интересующему порту целевого узла. Сообщение об ошибке покажет, открыт порт или нет. Это хороший способ обходить межсетевые экраны, потому что серверы FTP организаций часто располагаются там, где имеют более широкий доступ к другим внутренним узлам, чем любой посторонний узел Интернета. Nmap поддерживает FTP Bounce-сканирование параметром `-b`. Он принимает аргумент вида `<username>:<password>@<server>:<port>`. `<Server>` - имя или IP-адрес уязвимого сервера FTP. Как и в обычном URL, часть `<username>:<password>` можно опустить; тогда используются учётные данные анонимного входа: пользователь `anonymous`, пароль `-wwwuser@`. Номер порта и

предшествующее двоеточие также можно опустить; в этом случае используется стандартный порт FTP 21 на `<server>`.

В примере 5.21 я пытаюсь отразить сканирование Google от основного сервера FTP Microsoft.

Пример 5.21. Попытка FTP Bounce-сканирования

```
# nmap -PN -b ftp.microsoft.com google.com

Starting Nmap ( http://nmap.org )
Your FTP bounce server doesn't allow privileged ports, skipping them.
Your FTP bounce server sucks, it won't let us feed bogus ports!
```

Постоянным пользователям FTP Bounce-сканирования лучше привыкнуть к этому сообщению об ошибке. В 1997 году, когда вышел Nmap, эта уязвимость была широко распространена, но с тех пор её в основном исправили. Уязвимые серверы по-прежнему встречаются, поэтому, если всё остальное не помогло, попытка оправданна. Если цель - обойти межсетевой экран, просканируйте целевую сеть на открытый порт 21 или даже на любые службы FTP, если сканируете все порты с определением версий, а затем попробуйте Bounce-сканирование через каждую из них. Nmap сообщит, уязвим узел или нет. Если вы лишь пытаетесь замести следы, не нужно и, более того, не следует ограничиваться узлами целевой сети. Прежде чем искать уязвимые серверы FTP на случайных адресах Интернета, подумайте: системные администраторы могут не оценить такое злоупотребление их серверами.

В примере 5.22 показано успешное Bounce-сканирование нескольких интересующих портов Scanme. Для дополнительных подробностей указан параметр подробного вывода (`-v`). Приведённый тип сервера `JD FTP Server` означает, что это сервер печати HP JetDirect.

Пример 5.22. Успешное FTP Bounce-сканирование

```
krad~> nmap -p 22,25,135 -PN -v -b XXX.YY.111.2 scanme.nmap.org
Starting Nmap ( http://nmap.org )
Attempting connection to ftp://anonymous:-wwwuser@@XXX.YY.111.2:21
Connected:220 JD FTP Server Ready
Login credentials accepted by ftp server!
Initiating TCP ftp bounce scan against scanme.nmap.org (64.13.134.52)
Adding open port 22/tcp
Adding open port 25/tcp
Scanned 3 ports in 12 seconds via the Bounce scan.
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT    STATE    SERVICE
22/tcp  open     ssh
25/tcp  open     smtp
135/tcp filtered msrpc
Nmap done: 1 IP address (1 host up) scanned in 21.79 seconds
```

5.13. Код и алгоритмы сканирования

В 2004 году основной механизм сканирования портов Nmap был переписан ради повышения производительности и точности. Новый механизм, названный `ultra_scan` по имени своей функции, обслуживает сканирования SYN, `connect`, UDP, NULL, FIN, Xmas, ACK, Window, Maimon и протоколов IP, а также различные сканирования обнаружения узлов. Только Idle-сканирование и FTP Bounce-сканирование по-прежнему используют собственные механизмы.

Хотя схемы во всей этой главе показывают, как работает каждый тип сканирования, реализация Nmap намного сложнее, поскольку должна учитывать распараллеливание портов и узлов, оценку задержки, обнаружение потери пакетов, профили времени, аномальные условия сети, пакетные фильтры, ограничения частоты ответов и многое другое.

В этом разделе приведены не все низкоуровневые подробности механизма `ultra_scan`. Если ваше любопытство требует их, лучше обратиться к исходному коду. Функцию `ultra_scan` и её высокоуровневые вспомогательные функции можно найти в файле `scan_engine.cc` из архива Nmap. Здесь рассматриваются наиболее важные алгоритмические особенности. Их понимание помогает оптимизировать сканирования ради лучшей производительности, как описано в главе 6 «Оптимизация производительности Nmap».

5.13.1. Наблюдение за состоянием сети

Некоторые авторы хвастаются, что их сканеры быстрее Nmap благодаря работе без сохранения состояния. Они просто выпускают поток пакетов, затем слушают ответы и надеются на лучшее. Хотя это может быть полезно для быстрых обзоров и в других случаях, когда скорость важнее полноты и точности, я не считаю такой подход подходящим для сканирования безопасности. Сканер без сохранения состояния не способен обнаруживать отброшенные пакеты, чтобы повторно их передать и снизить частоту отправки. Если загруженный маршрутизатор на полпути сетевого маршрута отбросит 80 процентов потока пакетов сканера, тот всё равно сочтёт запуск успешным и выведет вопиюще неточные результаты. Nmap, напротив, во время работы хранит обширное состояние в оперативной памяти. Обычно памяти вполне достаточно даже на КПК. Nmap помечает каждую пробу номерами последовательности, исходными или целевыми портами, полями ID либо другими признаками в зависимости от типа пробы, позволяющими распознавать ответы и, следовательно, потери. Затем он соответственно регулирует скорость, чтобы работать настолько быстро, насколько позволяют сеть и заданные параметры командной строки, но не переходить границу, после которой страдает точность или несправедливо захватывается общая сеть. Некоторые администраторы без установленной IDS могут не заметить SYN-сканирование Nmap всей своей сети. Но можете не сомневаться: администратор начнёт расследование, если вы примените грубый сканер с потоком пакетов, влияющим на задержку его Quake!

Хотя алгоритмы управления перегрузкой Nmap рекомендуются для большинства сканирований, их можно переопределить. Параметр `--min-rate` отправляет пакеты с заданной вами или большей частотой, даже если она превышает обычные ограничения управления перегрузкой Nmap. Аналогично, `--max-retries` управляет числом повторных передач пакета. Параметры вроде `--min-rate 100 --max-retries 0` имитируют поведение простых сканеров без сохранения состояния. Скорость можно удвоить, задав 200 пакетов в секунду вместо 100, но не жадничайте: предельно быстрое сканирование малоценно, если результаты неверны или неполны. Любое применение `--min-rate` выполняется на ваш страх и риск.

5.13.2. Распараллеливание узлов и портов

Большинство схем этой главы показывает применение метода для определения состояния одного порта. Отправка пробы и получение ответа требуют времени кругового прохождения (RTT) между исходной и целевой машинами. Если RTT равен 200 мс и сканируются 65 536 портов одной машины, последовательная обработка заняла бы не менее 3,6 часа. Сканирование таким способом сети из 20 000 машин растянулось бы более чем на восемь лет. Это явно неприемлемо, поэтому Nmap распараллеливает сканирования и способен одновременно проверять сотни портов на каждой из десятков машин. Это повышает скорость на несколько порядков. Число одновременно сканируемых узлов и портов зависит от аргументов, описанных в главе 6 «Оптимизация производительности Nmap», включая `--min-hostgroup`, `--min-parallelism`, `-T4`, `--max-rtt-timeout` и многие другие. Оно также зависит от обнаруженных Nmap условий сети.

При сканировании нескольких машин Nmap старается эффективно распределять между ними нагрузку. Если машина кажется перегруженной, то есть отбрасывает пакеты или её задержка растёт, Nmap замедляет работу с этим узлом, продолжая сканировать остальные на полной скорости.

5.13.3. Оценка времени кругового прохождения

Каждый раз при получении ответа на пробу Nmap вычисляет число микросекунд, прошедших после её отправки. Назовём это значение `instanceRTT`; Nmap использует его, чтобы вести текущий учёт трёх критически важных временных величин: `srtt`, `rttvar` и `timeout`. Nmap хранит отдельные значения для каждого узла, а также объединённые значения для всей группы узлов, сканируемых параллельно. Они вычисляются следующим образом.

`srtt`

Сглаженное среднее время кругового прохождения. Nmap использует его как наиболее точную оценку RTT. Вместо настоящего арифметического среднего формула отдаёт предпочтение более свежим результатам, поскольку условия сети часто меняются:

```
newsrtt = oldsrtt + (instanceRTT - oldsrtt) / 8
```

`rttvar`

Наблюдаемая дисперсия или отклонение времени кругового прохождения. Идея в том, что при довольно стабильных значениях RTT Nmap может сдаться вскоре после ожидания `srtt`. Если разброс велик, Nmap должен ждать намного дольше `srtt`, прежде чем отказаться от пробы, поскольку сравнительно медленные ответы встречаются часто. Формула ниже, где `ABS` обозначает операцию абсолютного значения:

```
newrttvar = oldrttvar + (ABS(instanceRTT - oldsrtt) - oldrttvar) / 4
```

`timeout`

Время, которое Nmap готов ждать, прежде чем отказаться от пробы. Оно вычисляется так:

```
timeout = newsrtt + newrttvar * 4
```

Когда время ожидания пробы истекает, Nmap может передать её повторно или назначить порту состояние вроде `filtered` в зависимости от типа сканирования. Nmap сохраняет часть сведений о состоянии даже после истечения времени ожидания на случай, если запоздалый ответ придёт, пока общее сканирование ещё продолжается.

Эти простые формулы оценки времени, по-видимому, работают очень хорошо. Они в общих чертах основаны на сходных методах TCP, обсуждаемых в RFC 2988 «Вычисление таймера повторной передачи TCP». С годами мы оптимизировали эти алгоритмы, чтобы они лучше подходили для сканирования портов.

5.13.4. Управление перегрузкой

Таймеры повторной передачи - далеко не единственный приём, заимствованный Nmap из TCP. Поскольку Nmap чаще всего применяется с TCP, справедливо следовать многим тем же правилам, особенно потому, что эти правила являются результатом обширных исследований максимизации пропускной способности без скатывания к трагедии общин, где каждый эгоистично захватывает сеть. Со стандартными параметрами Nmap достаточно вежлив. Для управления агрессивностью сканирования Nmap применяет три алгоритма, созданных по образцу TCP: окно перегрузки,

экспоненциальную задержку и медленный старт. Окно перегрузки управляет числом проб Nmap, которые могут одновременно ожидать ответа. Если окно заполнено, Nmap не отправляет новые, пока не будет получен ответ или у пробы не истечёт время ожидания. Экспоненциальная задержка заставляет Nmap резко замедляться при обнаружении потерянных пакетов. Окно перегрузки обычно уменьшается до единицы при каждом обнаружении потерь. Несмотря на слово «медленный» в названии, медленный старт - довольно быстрый алгоритм постепенного повышения скорости сканирования для определения пределов производительности сети.

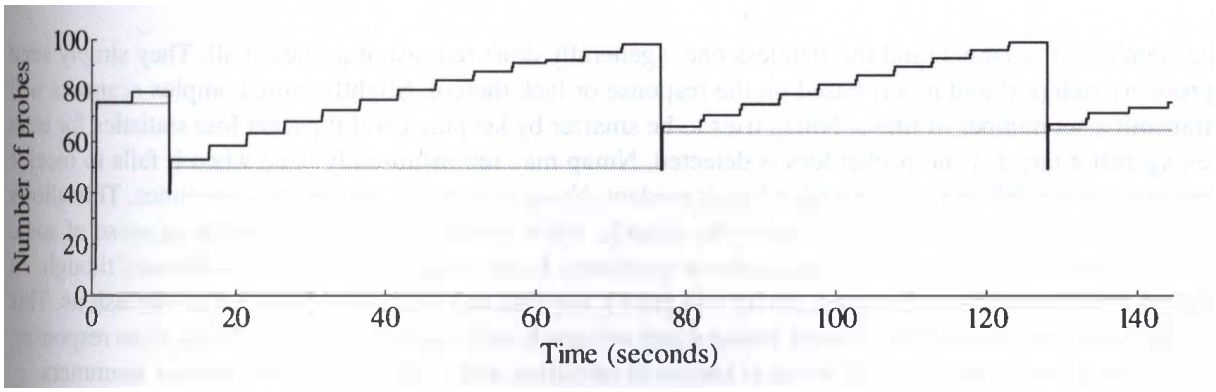
Все эти методы описаны в RFC 2581 «Управление перегрузкой TCP». Документ написали специалисты по сетям Ричард Стивенс, Верн Паксон и Марк Оллман. В нём всего 10 страниц, и любой, кого интересует реализация эффективных стеков TCP, других сетевых протоколов или сканеров портов, должен найти его увлекательным.

Когда Nmap сканирует группу целей, он хранит в памяти окно и порог перегрузки для каждой цели, а также окно и порог для всей группы. Окно перегрузки - число проб, которые разрешено отправить одновременно. Порог перегрузки задаёт границу между режимами медленного старта и предотвращения перегрузки. При медленном старте окно перегрузки быстро растёт в ответ на получаемые ответы. Как только оно превышает порог перегрузки, начинается режим предотвращения перегрузки, при котором окно растёт медленнее. После потери и окно перегрузки, и порог уменьшаются до некоторой доли прежнего значения.

Однако между потоками TCP и сканированием портов Nmap есть важное различие. В потоках TCP обычно ожидаются ACK в ответ на каждый отправленный пакет или хотя бы на значительную их долю. Правильный рост окна перегрузки фактически зависит от этого предположения. Nmap часто оказывается в иной ситуации: при работе с целью, защищённой межсетевым экраном с запретом по умолчанию, ответы придут лишь на очень немногие отправленные пакеты. То же происходит при ping-сканировании блока сетевых адресов, содержащего всего несколько активных узлов. Для компенсации Nmap отслеживает отношение числа отправленных пакетов к числу полученных ответов. При каждом изменении группового окна перегрузки величина изменения умножается на это отношение. Иными словами, когда ответы приходят лишь на немногие пакеты, каждый ответ имеет больший вес.

Графическое представление изменения группового окна и порога перегрузки во время типичного сканирования портов показано на рисунке 5.9. Окно перегрузки изображено чёрным, а порог - серым.

Рисунок 5.9. Окно и порог перегрузки



В начале окно перегрузки мало, а порог велик. Запускается режим медленного старта, и размер окна быстро увеличивается. Крупные ступенчатые скачки являются результатом временных ping-проб. Примерно на 10-й секунде окно перегрузки вырастает до 80 проб, после чего обнаруживается потеря. И окно, и порог уменьшаются. Окно продолжает расти примерно до 80-й секунды, когда обнаруживается ещё одна потеря. Затем цикл повторяется, что типично для

стабильных условий сети.

Потерь во время сканирования бояться не следует. Назначение алгоритмов управления перегрузкой - динамически проверять сеть, чтобы определить её пропускную способность. С этой точки зрения потери являются ценной обратной связью, помогающей Nmap выбрать правильный размер окна перегрузки.

5.13.5. Временные пробы

Каждый метод, рассмотренный в этом разделе алгоритмов, на некотором уровне включает наблюдение за сетью для обнаружения и оценки потери пакетов и задержки. Это действительно критически важно для достижения малого времени сканирования. К сожалению, при сканировании систем с жёсткой фильтрацией хорошие данные часто получить трудно. Такие фильтры нередко отбрасывают подавляющее большинство пакетов без ответа. Nmap может потребоваться отправить 20 000 проб или больше, чтобы найти один отвечающий порт, что затрудняет наблюдение за условиями сети.

Для борьбы с этой проблемой Nmap применяет временные пробы, также называемые ping-пробами сканирования портов. Если Nmap нашёл хотя бы один отвечающий порт на сильно фильтруемом узле, он отправляет ему пробу каждые 1,25 секунды, в течение которых не получает ответов от других портов. Это обеспечивает достаточный уровень наблюдения, чтобы ускорять или замедлять сканирование в соответствии с условиями сети.

5.13.6. Выведенные временные параметры соседей

Иногда даже ping-пробы сканирования портов не помогают, поскольку не найдено вообще ни одного отвечающего порта. Машина может не работать и сканироваться с `-PN`, либо фильтруется каждый её порт. Возможно также, что у цели есть пара отвечающих портов, но Nmap пока не посчастливилось их найти. В таких случаях Nmap использует временные значения, поддерживаемые для всей группы машин, сканируемых одновременно. Пока получен хотя бы один ответ от любой машины группы, у Nmap есть данные для работы. Разумеется, Nmap не может предполагать, что узлы одной группы всегда имеют сходные временные характеристики. Поэтому он отслеживает разброс временных параметров между отвечающими узлами группы. Если они сильно различаются, ради безопасности Nmap выводит длительные значения времени ожидания для соседних узлов.

5.13.7. Адаптивная повторная передача

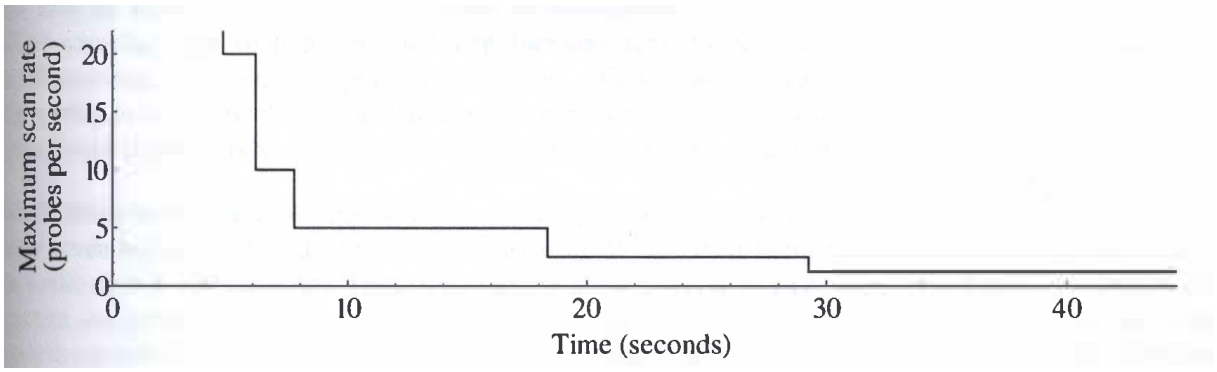
Самые простые сканеры, в том числе не сохраняющие состояние, обычно вообще не передают пробы повторно. Они лишь отправляют пробу каждому порту и сообщают результат на основании ответа или его отсутствия. Чуть более сложные сканеры выполняют заданное число повторных передач. Nmap старается действовать умнее, тщательно ведя статистику потери пакетов для каждого сканирования цели. Если потерь не обнаружено, не получив ответа на пробу, Nmap может повторить её лишь один раз. При очевидной массовой потере пакетов Nmap может выполнить десять и более повторных передач. Это позволяет быстро сканировать узлы в быстрых и надёжных сетях, сохраняя точность ценой некоторого снижения скорости при сканировании проблемных сетей или машин. Но даже терпение Nmap не безгранично. В определённый момент, после десяти повторных передач, Nmap выводит предупреждение и отказывается от дальнейших попыток. Это не позволяет вредоносным узлам слишком сильно замедлять Nmap намеренным отбрасыванием

пакетов, медленными ответами и подобными проделками. Такая атака называется tarpitting и часто применяется против спамеров.

5.13.8. Задержка сканирования

Ограничение частоты ответов на пакеты, возможно, самая пагубная проблема, с которой сталкиваются сканеры портов вроде Nmap. Например, ядра Linux 2.4 ограничивают сообщения об ошибках ICMP, возвращаемые при UDP-сканировании (`-sU`) или сканировании протоколов IP (`-s0`), одним в секунду. Если бы Nmap считал их обычными потерями, он постоянно замедлялся бы, вспомните экспоненциальную задержку, но всё равно сталкивался бы с отбрасыванием подавляющего большинства проб. Вместо этого Nmap пытается обнаружить такую ситуацию. Когда отбрасывается значительная доля пакетов, он вводит между пробами, отправляемыми одной цели, короткую задержку, иногда всего 5 миллисекунд. Если потери остаются серьёзной проблемой, Nmap продолжает удваивать задержку, пока они не прекратятся или не будет достигнута максимально разрешённая задержка сканирования. Влияние задержки при UDP-сканировании портов 1-50 узла Linux с ограниченной частотой ответов показано на рисунке 5.10. В начале частота сканирования не ограничена задержкой, хотя собственные ограничения, разумеется, накладывают другие механизмы вроде управления перегрузкой. При обнаружении потерь задержка удваивается, то есть максимальная частота сканирования фактически уменьшается вдвое. Например, на графике максимальной частоте пять пакетов в секунду соответствует задержка сканирования 200 миллисекунд.

Рисунок 5.10. Влияние задержки сканирования на его частоту



Стандартная максимальная задержка сканирования составляет одну секунду между пробами. Иногда задержка включается, когда медленный узел не справляется, даже если на нём нет явных правил ограничения частоты. Это может значительно уменьшить общий сетевой трафик за счёт сокращения числа бесполезных, отброшенных пакетов-проб. К сожалению, даже небольшие задержки способны увеличить продолжительность сканирования в несколько раз. По умолчанию Nmap действует консервативно, допуская секундные задержки для проб TCP и UDP. Если ваши приоритеты иные, максимальную задержку можно настроить параметром `--max-scan-delay`, как обсуждается в главе 5 «Методы и алгоритмы сканирования портов».

Сноски

[1] [назад] http://www.microsoft.com/japan/security/bulletins/MS04-003e.mspx
[2] [назад] http://www.hping.org

Глава 6. Оптимизация производительности Nmap

6.1. Введение

Одним из моих главных приоритетов при разработке Nmap всегда была производительность. Стандартное сканирование (`nmap <hostname>`) узла в моей локальной сети занимает одну пятую секунды. Этого едва хватает, чтобы моргнуть, но при сканировании сотен или тысяч узлов время складывается. Кроме того, некоторые параметры, например UDP-сканирование и определение версий, способны значительно увеличить длительность. То же относится к некоторым конфигурациям межсетевых экранов, особенно с ограничением частоты ответов. Хотя Nmap применяет параллелизм и множество передовых алгоритмов для ускорения сканирования, окончательный контроль над его работой принадлежит пользователю. Опытные пользователи тщательно составляют команды Nmap, чтобы получать лишь нужные сведения и укладываться в ограничения времени.

Хотя производительность Nmap имеет высокий приоритет, точность ещё важнее. Авторы конкурирующих сканеров выступали на известных конференциях с докладами о том, что их сканер проверяет всё адресное пространство класса B всего за четыре секунды. На самом деле такие сканеры элементарно написать: они исключают все алгоритмы управления перегрузкой и обнаружения потери пакетов, оставляя лишь тесный цикл, выбрасывающий пакеты-пробы настолько быстро, насколько их способна создавать система или выдерживает канал. Такие сканеры часто рекламируют как не сохраняющие состояние, то есть из них также удалён код отслеживания и повторной передачи проб. Подобного поведения можно добиться в Nmap, добавив флаги вроде `--min-rate 1000`, чтобы Nmap отправлял не менее 1000 пакетов в секунду, и `--max-retries 0`, чтобы отключить повторную передачу проб с истёкшим временем ожидания. Но я редко это рекомендую. Девяносто девять процентов пакетов может отбросить уже следующий вышестоящий маршрутизатор, а сканер никогда не заметит разницы.

Сканеры с неограниченным потоком пакетов вроде Scanrand[1] полезны в некоторых ситуациях, но Nmap выбирает гораздо более консервативный и точный путь. Сначала Nmap предполагает наихудшие условия целевых сетей: большую задержку и потерю пакетов, а затем ускоряется по мере накопления статистики, показывающей, что это безопасно. Хотя всё происходит автоматически, администратор может ускорить обучение, передав Nmap подсказки о сети. Пример такой подсказки - `--max-rtt-timeout 200`, позволяющий Nmap предположить, что любые ответы на пробу целевого узла придут в течение 200 миллисекунд.

В этой главе сначала обсуждаются высокоуровневые методы сокращения времени сканирования. Затем рассматривается применение шаблонов времени и низкоуровневых средств управления для ускорения Nmap без ущерба точности. В заключение приводится учебный пример Джека Могрена из клиники Мэйо, подробно описывающий, как он сократил время сканирования своей сети из 676 352 IP-адресов почти с недели до 46 часов. Учитывая огромное значение производительности сканера, глава может показаться короткой. Причина в том, что здесь основное внимание уделяется общим высокоуровневым советам по производительности, а рекомендации по оптимизации отдельных методов сканирования распределены по всей книге рядом с описанием соответствующих методов.

6.2. Методы сокращения времени сканирования

Идеальное решение проблемы длительных сканирований - сократить их. В этом разделе предлагается множество высокоуровневых советов. В отличие от многих жизненных обстоятельств, настройка командной строки Nmap способна дать огромный эффект.

Установка на Honda Accord выхлопной насадки из банки из-под кофе, спойлера высотой в три фута и большой красной наклейки «type R» почти не уменьшит время разгона до 60 миль в час. Но в разделе 6.7 «Сканирование 676 352 IP-адресов за 46 часов» описано, как Джек Могрен сократил время работы Nmap на несколько дней, просто добавив несколько наклеек, то есть параметров, в командную строку Nmap.

6.2.1. Исключайте некритичные проверки

Электронный эквивалент покупки Hummer, когда вы никогда не съезжаете с асфальта и не перевозите ничего тяжелее продуктов, - запуск интенсивного и всеобъемлющего сканирования Nmap ради сравнительно незначительного объёма сведений. Потеря нескольких секунд на узел редко имеет значение в домашней сети, но способна сделать ежедневные сканирования WAN невыполнимыми для крупных предприятий. Ниже перечислены самые распространённые ошибки избыточного сканирования: сначала наиболее вопиющие промахи новичков, затем более тонкие проблемы, с которыми сталкиваются даже опытные пользователи.

Указывайте ping-сканирование (`-sP`), когда требуется лишь определить работающие узлы.

Некоторые определяют, работает ли узел, командой `nmap <hostname>`. Она срабатывает, но это излишество. Nmap отправит два пакета, чтобы установить, что узел работает, а затем не менее 1000 пакетов для сканирования его портов. Проблема усиливается, когда таким способом сканируют целую сеть, чтобы найти все работающие узлы или один конкретный узел.

Вместо напрасного сканирования портов укажите `-sP`, чтобы выполнить ping-сканирование, если требуется узнать лишь работающие узлы или их MAC-адреса.

Ограничивайте число сканируемых портов.

По умолчанию Nmap сканирует 1000 наиболее распространённых портов. В быстрой сети с отзывчивыми машинами это может занимать долю секунды на узел. Но при ограничении частоты или встрече с межсетевыми экранами, которые отбрасывают пакеты-пробы без ответа, Nmap приходится резко замедляться. По этим причинам UDP-сканирование может быть мучительно медленным. Однако подавляющее большинство открытых портов приходится лишь на несколько сотен номеров. Сканирование портов будет примерно в десять раз быстрее, если проверять 100 портов вместо стандартной тысячи. Сто самых популярных портов можно просканировать параметром `-F` для быстрого сканирования, произвольное число ведущих портов - параметром `--top-ports`, а собственный список передать параметру `-p`.

Пропускайте расширенные типы сканирования (`-sC`, `-sV`, `-O`, `--traceroute` и `-A`).

Некоторые регулярно указывают параметр Nmap `-A`, который даёт им полный набор. Он заставляет Nmap выполнять определение ОС, определение версий, сканирование сценариями NSE и трассировку маршрута наряду со стандартным сканированием портов. Определение версий бывает исключительно полезным, но способно серьёзно затормозить крупное сканирование. То же относится к NSE. При нехватке времени всегда можно пропустить `-sC` и `-sV` в крупномасштабном сканировании, а позднее при необходимости выполнить их для отдельных портов.

Определение ОС не настолько медленно, как определение версий, но всё же легко занимает 5-10 секунд на каждый работающий узел. Даже без него ОС часто можно угадать по имени, открытым портам и MAC-адресу в LAN. Во многих случаях ОС вообще может не интересовать. Поэтому `-O` -

ещё один кандидат на применение только при необходимости. В качестве компромисса можно указать `--osscan-limit --max-os-tries 1`: Nmap не станет повторять неудачные попытки определения ОС и пропустит эту проверку для всех работающих узлов, у которых нет хотя бы одного открытого и одного закрытого порта TCP. В любом случае определение ОС для таких узлов менее точно.

Не забывайте отключать разрешение DNS, когда оно не требуется.

По умолчанию Nmap выполняет обратное разрешение DNS для каждого узла, который признан работающим. Оно выполняется для всех узлов, если пропустить этап ping параметром `-PN` или указать `-R`. Это было крупным узким местом, когда системные библиотеки DNS использовались для последовательного поиска по одному IP-адресу.

Теперь у Nmap есть быстрая параллельная система обратного DNS, ускоряющая запросы, но и они могут занимать значительное время. Отключайте их параметром `-n`, когда эти данные не нужны. При простых сканированиях, например ping-сканированиях большого числа узлов, отказ от DNS иногда сокращает время на 20 процентов и больше. В более сложных сканированиях, которые проверяют тысячи портов или применяют интенсивные функции вроде определения версий, время DNS не является главным фактором. Если требуется, чтобы разрешением имён занималась машина, на которой работает Nmap, с помощью функции `gethostbyaddr`, укажите `--system-dns`. Это способно резко замедлить сканирование.

6.2.2. Оптимизируйте временные параметры

Nmap предлагает десятки параметров для передачи подсказок и правил, управляющих сканированием. Они простираются от высокоуровневых степеней агрессивности времени, задаваемых параметром `-T` и описанных в разделе 6.6 «Шаблоны времени (`-T`)», до более детальных средств управления из раздела 6.5 «Низкоуровневое управление временем». Их можно даже сочетать. Эти параметры особенно полезны при сканировании сильно фильтруемых сетей, где Nmap получает мало ответов для самостоятельного вычисления временных оценок. Часто время сканирования можно безопасно сократить вдвое. В локальной LAN с отзывчивыми узлами большинство этих параметров даст небольшой эффект, поскольку там Nmap способен сам определить оптимальные значения.

6.2.3. Разделяйте и оптимизируйте UDP-сканирования

Сканировать порты UDP важно, поскольку этот протокол используют многие уязвимые службы, но временные характеристики и требования к производительности UDP-сканирований сильно отличаются от TCP. Особую проблему представляет ограничение частоты ошибок ICMP, которое встречается чрезвычайно часто и влияет на UDP-сканирования намного сильнее, чем на TCP.

По этим причинам при критичной производительности я не рекомендую объединять TCP- и UDP-сканирования, хотя Nmap поддерживает это, например параметром `-sSU`. Для каждого протокола часто требуются разные временные флаги, а значит, отдельные командные строки. В разделе 5.4.2 «Ускорение UDP-сканирования» приведены полезные приёмы и практические примеры повышения производительности UDP-сканирования.

6.2.4. Обновляйте Nmap

Мне много раз приходилось расследовать сообщения о низкой производительности Nmap лишь затем, чтобы обнаружить: автор использовал древнюю версию, устаревшую на много лет. Новейшие версии Nmap содержат важные алгоритмические улучшения, исправления ошибок, повышающие производительность возможности вроде ARP-сканирования локальной сети и многое другое. Первой реакцией на проблемы производительности должно быть сравнение своей версии Nmap, выводимой командой `nmap -V`, с последней версией на nmap.org. При необходимости обновитесь. Если и этого недостаточно, попробуйте остальные методы из этой главы.

6.2.5. Запускайте экземпляры Nmap параллельно

Некоторые пытаются ускорить Nmap, параллельно запуская множество копий, каждая против одной цели. Например, раньше сканер Nessus делал это по умолчанию. Обычно это намного менее эффективно и медленнее, чем позволить Nmap работать со всей сетью. У Nmap есть собственная система распараллеливания, приспособленная к его потребностям, и при сканировании большой группы Nmap способен ускоряться по мере получения сведений о надёжности сети. Кроме того, просьба к ОС породить 65 536 отдельных экземпляров Nmap лишь для сканирования класса B создаёт значительные накладные расходы. Десятки параллельно работающих копий Nmap также расходуют память, поскольку каждый экземпляр загружает собственные копии файлов данных вроде `nmap-services` и `nmap-os-db`.

Хотя параллельный запуск одноузловых сканирований Nmap - плохая идея, общую скорость обычно можно повысить, разделив сканирование на несколько крупных групп и выполняя их одновременно. Но не переусердствуйте. Пять или десять процессов Nmap допустимы, а запуск сразу 100 не рекомендуется. Слишком много параллельных процессов Nmap приводит к конкуренции за ресурсы. Другой вид параллелизма - одновременный запуск Nmap с разных узлов. Можно поручить `cron`, или At в Windows, запланировать на локальных узлах каждой сети одновременное сканирование местных машин, а затем отправить результаты по почте на центральный сервер данных. Сканирование австралийской сети из США будет медленнее, чем с локальной машины этой сети. Различие окажется ещё больше, если американской машине приходится пересекать дополнительные межсетевые экраны, чтобы добраться до удалённой сети.

6.2.6. Сканируйте из выгодного сетевого расположения

Ограничительные межсетевые экраны способны превратить пятисекундное сканирование в многочасовую работу. Не помогают и задержка с потерей пакетов на некоторых маршрутах Интернета. Если Nmap можно запускать с узла или узлов внутри целевой сети, делайте это. Разумеется, если цель состоит в том, чтобы увидеть сеть глазами внешнего злоумышленника или проверить межсетевой экран, требуется внешнее сканирование. С другой стороны, сканирование и защита внутренней сети обеспечивают глубоко эшелонированную оборону, критически важную против внутренних угроз и хитрых злоумышленников, обходящих межсетевой экран; см. главу 10 «Обнаружение и преодоление межсетевых экранов и систем обнаружения вторжений».

При обратном разрешении DNS, особенно если локальный сервер имён сильно загружен, может помочь менее занятый сервер или прямые запросы к авторитетным серверам имён. Выигрыш обычно невелик и оправдан лишь для повторяющихся либо огромных сканирований. Разумеется, иногда серверы имён выбирают по причинам, не связанным с производительностью.

6.2.7. Увеличивайте доступную пропускную способность и время ЦП

Иногда время сканирования Nmap можно сократить, увеличив доступную пропускную способность или вычислительную мощность ЦП. Для этого можно установить новую линию передачи данных или процессор либо остановить параллельно работающие приложения, конкурирующие за эти ресурсы. Например, Nmap будет работать медленнее, если одновременно полностью загрузить линию DSL скачиванием пиратского торрента *The Matrix Reloaded*.

Намного чаще Nmap ограничен собственными алгоритмами управления перегрузкой, а не ЦП или доступной локальной пропускной способностью. Эти средства предотвращают затопление сети и повышают точность. Увеличение мощности ЦП и локальной пропускной способности не поможет против такого самоограничения Nmap: вместо этого нужно настраивать временные параметры. Проверить, ограничен ли Nmap процессором, можно, наблюдая загрузку ЦП программой вроде `top` в Unix или Диспетчером задач в Windows. Если ЦП большую часть времени простаивает, обновление почти не поможет. Чтобы проверить использование Nmap пропускной способности, запустите его в подробном режиме (`-v`). Тогда Nmap сообщит число отправленных и полученных байтов и время выполнения, как показано в примере 6.1.

Пример 6.1. Использование пропускной способности в локальной сети Ethernet 100 Мбит/с

```
# nmap -v -n -p- sec.titan.net

Starting Nmap ( http://nmap.org )
[10 lines cut]
Interesting ports on 192.168.0.8:
Not shown: 65534 closed ports
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 00:1A:6B:C1:33:37 (USI)
Nmap done: 1 IP address (1 host up) scanned in 2.20 seconds
           Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)
```

Умножьте значения байтов на восемь и разделите на время выполнения, чтобы получить среднее использование пропускной способности в битах в секунду. В примере 6.1 Nmap получил 2 621 000 байтов, поскольку Nmap считает мегабайтом 1 000 000 байтов, за 2,20 секунды. Следовательно, входящий трафик составлял около 9,5 Мбит/с, а исходящий - 10,5 Мбит/с. Поэтому линия Ethernet 100 Мбит/с вряд ли ограничивает Nmap, и переход на гигабитный Ethernet почти не поможет.

Некоторым потребительским широкополосным устройствам и другому оборудованию трудно справляться с частотой пакетов Nmap, хотя малый размер пакетов, обычно Nmap отправляет пустые заголовки, сохраняет низкое потребление полосы. В примере 6.1 «Использование пропускной способности в локальной сети Ethernet 100 Мбит/с» Nmap отправлял около 30 000 пакетов в секунду и получал примерно столько же. Такая высокая частота создаёт проблемы для низкокачественных устройств. В данном случае число отправленных и полученных пакетов равно 65 536: это число сканируемых портов, 65 535, плюс одна начальная ARP ping-проба. Следовательно, Nmap не столкнулся с потерей пакетов, требующей повторной передачи. Это снова указывает на то, что сетевое оборудование не было ограничивающим фактором: вероятно, Nmap был ограничен процессором.

6.3. Стратегии работы с длительными сканированиями

Оптимизация параметров способна значительно ускорить сканирование, но существует предел скорости, с которой Nmap может работать, сохраняя точность и справедливо относясь к конкурирующим сетевым потокам. Крупные сканирования тысяч узлов, всех 65 тысяч портов, UDP или с определением версий, скорее всего, займут много времени даже после оптимизации. В этом разделе предлагаются действенные стратегии работы с такими длительными сканированиями.

6.3.1. Применяйте многоэтапный подход

Всеобъемлющий аудит безопасности должен включать UDP- и TCP-сканирование всех 65 536 портов каждого протокола, обычно с `-PN` на случай, если машина работает, но жёстко фильтруется. Однако широко используются менее 100 таких номеров портов, а большинство узлов отвечают при умеренных параметрах обнаружения. Поэтому сначала укажите `-F`, чтобы быстро просканировать популярные порты заведомо работающих узлов. Это позволит анализировать работающие узлы и большинство открытых портов, пока в фоне запускается огромное сканирование всех портов TCP и UDP с `-PN`, определением версий и ОС. Сокращённые параметры для ускорения быстрой проверки обсуждаются в разделе 6.2.1 «Исключайте некритичные проверки». После завершения медленного сканирования сравните его с ранними результатами, чтобы найти вновь обнаруженные узлы или порты.

6.3.2. Оценивайте и планируйте время сканирования

Во многих случаях самая неприятная черта длительного сканирования - полное неведение о времени его завершения. Теперь Nmap полезнее прежнего: при включённом подробном режиме (`-v`) он регулярно выводит оценки времени сканирования.

Пример 6.2. Оценка времени сканирования

```
# nmap -T4 -sS -p0- -iR 500 -n --min-hostgroup 100 -v

Starting Nmap ( http://nmap.org )
Initiating SYN Stealth Scan against 29 hosts [65536 ports/host] at 23:27
[...]
SYN Stealth Scan Timing: About 0.30% done; ETC: 09:45 (10:15:45 remaining)
```

Пример 6.2 показывает, что SYN-сканирование 29 узлов, вероятно, займёт десять часов и восемнадцать минут, с 23:27 до 9:45. Поэтому общее время, которое Nmap потратит на сканирование сети, можно приблизительно оценить, умножив 21 минуту на узел на число работающих узлов. Если также выполняется определение версий или UDP-сканирование, придётся следить и за их оценками времени.

Другой вариант - дождаться, пока Nmap полностью закончит сканировать первую группу узлов. Затем экстраполировать затраченное на эту группу время на размер всей целевой сети. Это проще, поскольку не нужно беспокоиться об отдельных компонентах сканирования. Оценка по числу завершённых целевых IP-адресов относительно размера целевого адресного пространства может вводить в заблуждение, поскольку работающие узлы редко распределены в нём равномерно. Обычно они встречаются скоплениями, часто возле начала адресного пространства. Поэтому, если само сканирование включает обнаружение узлов, то есть параметр `-PN` отсутствует, точнее сначала выполнить ping-сканирование всей сети, а затем основывать оценку на числе уже просканированных Nmap работающих узлов относительно общего числа, найденного ping-сканированием.

Хотя в подробном режиме оценки периодически печатаются автоматически, текущую оценку всегда можно запросить нажатием `<enter>`; см. раздел 15.15 «Взаимодействие во время выполнения». Если оценка укладывается в ваши сроки, можно запланировать другие занятия на время работы. Это лучше, чем каждые 20 минут проверять, закончил ли Nmap. Оценка, показывающая, что Nmap не успеет вовремя, ещё ценнее. Можно сразу заняться оптимизацией сканирования или продлением задания. Ваши возможности намного ограниченнее, если слишком низкая скорость обнаружится лишь после истечения срока, когда Nmap всё ещё работает.

6.4. Данные и стратегии выбора портов

Сканирование портов может быть самой длительной частью запуска Nmap, даже если он включает определение версий или сценарии NSE. Время примерно пропорционально числу проверяемых портов, поэтому их сокращение значительно повышает производительность. Недостаток в том, что уменьшенные сканирования менее полны и могут пропустить открытые порты.

В действительности в каждом протоколе имеется 65 536 портов, и большинство из них почти никогда не открыты. Я провёл лето за крупномасштабными сканированиями, определяя распространённость каждого порта TCP и UDP. Результаты включают данные сканирования десятков миллионов IP-адресов Интернета, а также корпоративных сетей изнутри. В этом разделе приводятся эмпирические результаты, на которые можно опереться при выборе правильного баланса между скоростью и эффективностью сканирования.

Хотя всего существует более ста тысяч портов TCP и UDP, подавляющее большинство открытых портов входит в гораздо меньший набор. Согласно нашему исследованию, 10 ведущих портов TCP и 1075 ведущих портов UDP составляют половину открытых портов соответствующего протокола. Чтобы обнаружить 90 процентов открытых портов, нужно просканировать 576 портов TCP и 11 307 портов UDP. По умолчанию Nmap проверяет 1000 ведущих портов для каждого запрошенного протокола сканирования. Это обнаруживает примерно 93 процента портов TCP и 49 процентов портов UDP. С параметром `-F` для быстрого сканирования проверяются лишь 100 ведущих портов, что даёт эффективность 78 процентов для TCP и 39 процентов для UDP. Чтобы указать другое число портов, передайте его параметру `--top-ports`. В таблице 6.1 приведена приблизительная оценка числа портов TCP или UDP, которые нужно просканировать для достижения заданной эффективности по этому протоколу.

Таблица 6.1. Требуемые значения `--top-ports` для различных уровней эффективности

Эффективность · требуется портов TCP · требуется портов UDP

- **10%** — TCP: 1; UDP: 5.
- **20%** — TCP: 2; UDP: 12.
- **30%** — TCP: 4; UDP: 27.
- **40%** — TCP: 6; UDP: 135.
- **50%** — TCP: 10; UDP: 1 075.
- **60%** — TCP: 18; UDP: 2 618.
- **70%** — TCP: 44; UDP: 5 157.
- **80%** — TCP: 122; UDP: 7 981.
- **85%** — TCP: 236; UDP: 9 623.
- **90%** — TCP: 576; UDP: 11 307.
- **95%** — TCP: 1 558; UDP: 13 035.
- **99%** — TCP: 3 328; UDP: 15 094.
- **100%** — TCP: 65 536; UDP: 65 536.

Хотя Nmap способен автоматически выбирать порты, если полагаться на стандартные значения или параметры вроде `-F` и `--top-ports`, часто полезно явно задавать их через `-p`. В любом случае важно знать наиболее часто встречающиеся открытые порты. Ведущие порты согласно нашим данным описаны в разделе 4.1.2 «Какие порты наиболее популярны?».

6.5. Низкоуровневое управление временем

Nmap предлагает множество детальных параметров управления скоростью сканирования. Большинство применяет их для ускорения Nmap, но они также полезны для замедления. Это делают, чтобы уклониться от IDS, уменьшить нагрузку на сеть или даже повысить точность, если условия сети настолько плохи, что даже консервативные стандартные настройки Nmap слишком агрессивны.

В таблице 6.2 каждый низкоуровневый параметр времени приведён рядом со своей функцией. Подробные сведения об использовании каждого параметра см. в разделе 15.11 «Время и производительность». Предполагается, что читатель уже знаком с алгоритмами сканирования Nmap, описанными в разделе 5.13 «Код и алгоритмы сканирования».

Таблица 6.2. Низкоуровневые средства управления временем по функциям

Функция — параметры

• **Размер группы узлов, сканируемых одновременно** — `--min-hostgroup`, `--max-hostgroup`.
• **Число параллельно запускаемых проб** — `--min-parallelism`, `--max-parallelism`.
• **Значения времени ожидания проб** — `--min-rtt-timeout`, `--max-rtt-timeout`, `--initial-rtt-timeout`.
• **Максимально допустимое число повторных передач проб** — `--max-retries`.
• **Максимальное время до отказа от целого узла** — `--host-timeout`.
• **Задержка между пробами одного узла** — `--scan-delay`, `--max-scan-delay`.
• **Частота отправки пакетов-проб в секунду** — `--min-rate`, `--max-rate`.
• **Преодоление ограничения целевыми узлами частоты ответов RST** — `--defeat-rst-ratelimit`.

6.6. Шаблоны времени (-T)

Хотя детальные средства управления временем из предыдущего раздела мощны и эффективны, некоторым они кажутся запутанными. Более того, выбор подходящих значений иногда занимает больше времени, чем само оптимизируемое сканирование. Поэтому Nmap предлагает более простой подход: шесть шаблонов времени. Их можно указать параметром `-T` и номером от 0 до 5 либо именем. Шаблоны называются `paranoid` (0), `sneaky` (1), `polite` (2), `normal` (3), `aggressive` (4) и `insane` (5). Первые два предназначены для уклонения от IDS. Режим `polite` замедляет сканирование, уменьшая потребление пропускной способности и ресурсов целевой машины. `Normal` является стандартным, поэтому `-T3` ничего не меняет. `Aggressive` ускоряет сканирование, предполагая, что сеть сравнительно быстра и надёжна. Наконец, `insane` предполагает исключительно быструю сеть либо готовность пожертвовать некоторой точностью ради скорости.

Эти шаблоны позволяют пользователю указать желаемую агрессивность, оставляя Nmap выбор точных временных значений. Они также вносят небольшие корректировки скорости, для которых пока нет детальных параметров. Например, `-T4` запрещает динамической задержке сканирования портов TCP превышать 10 мс, а `-T5` ограничивает её 5 мс. Шаблоны можно сочетать с детальными средствами, причём последние переопределяют общие шаблоны для соответствующих значений. Я рекомендую `-T4` при сканировании достаточно современных и надёжных сетей. Оставляйте этот параметр в начале командной строки даже при добавлении детальных средств, чтобы получить дополнительные мелкие оптимизации, которые он включает.

В таблице 6.3 показано, как временные переменные меняются для каждого значения `-T`. Все значения времени указаны в миллисекундах.

Таблица 6.3. Шаблоны времени и их действие

Переменная · T0 · T1 · T2 · T3 · T4 · T5

• **Имя** — T0: Paranoid; T1: Sneaky; T2: Polite; T3: Normal; T4: Aggressive; T5: Insane.
• `min-rtt-timeout` — T0: 100; T1: 100; T2: 100; T3: 100; T4: 100; T5: 50.
• `max-rtt-timeout` — T0: 300 000; T1: 15 000; T2: 10 000; T3: 10 000; T4: 1 250; T5: 300.
• `initial-rtt-timeout` — T0: 300 000; T1: 15 000; T2: 1 000; T3: 1 000; T4: 500; T5: 250.
• `max-retries` — T0: 10; T1: 10; T2: 10; T3: 10; T4: 6; T5: 2.
• **Начальная и минимальная задержка сканирования (**`--scan-delay`**)** — T0: 300 000; T1: 15 000; T2: 400; T3: 0; T4: 0; T5: 0.
• **Максимальная задержка TCP-сканирования** — T0: 300 000; T1: 15 000; T2: 1 000; T3: 1 000; T4: 10; T5: 5.
• **Максимальная задержка UDP-сканирования** — T0: 300 000; T1: 15 000; T2: 1 000; T3: 1 000; T4: 1 000; T5: 1 000.
• `host-timeout` — T0: 0; T1: 0; T2: 0; T3: 0; T4: 0; T5: 900 000.
• `min-parallelism` — T0: динамически, шаблоны времени не влияют; T1-T5: динамически.
• `max-parallelism` — T0: 1; T1: 1; T2: 1; T3-T5: динамически.
• `min-hostgroup` — T0: динамически, шаблоны времени не влияют; T1-T5: динамически.
• `max-hostgroup` — T0: динамически, шаблоны времени не влияют; T1-T5: динамически.
• `min-rate` — T0: нет минимального ограничения частоты; T1-T5: не задано.
• `max-rate` — T0: нет максимального ограничения частоты; T1-T5: не задано.
• `defeat-rst-ratelimit` — T0: по умолчанию не включено; T1-T5: не задано.

При достаточно хорошем широкополосном соединении или Ethernet я рекомендую всегда применять `-T4`. Некоторым нравится `-T5`, хотя для меня он слишком агрессивен. Иногда люди указывают `-T2`, считая, что он с меньшей вероятностью приведёт к сбою узлов, или просто полагая себя вежливыми. Они часто не понимают, насколько медленным на самом деле является `-T polite`: сканирование может занять в десять раз больше времени, чем стандартное. Сбои машин и проблемы с пропускной способностью редко возникают при стандартных временных параметрах (`-T3`), поэтому осторожным пользователям я обычно рекомендую их. Отказ от определения версий намного эффективнее снижает такие проблемы, чем изменение временных значений.

Хотя `-T0` и `-T1` могут быть полезны для уклонения от предупреждений IDS, сканирование тысяч машин или портов займёт чрезвычайно много времени. Для столь длительной работы лучше задать точные необходимые временные значения, а не полагаться на готовые `-T0` и `-T1`.

6.7. Сканирование 676 352 IP-адресов за 46 часов

Эту историю прислал Джек Л. Могрен из клиники Мэйо. Она служит учебным примером, показывающим предпринятые им шаги для внедрения регулярного сканирования Nmap и сокращения времени проверки этой огромной сети с недели до 46 часов.

Клиника Мэйо построила сравнительно большую частную сеть, причём таблицы ARP показывали более 70 000 используемых IP-адресов. Раньше управление нашей сетью было сосредоточено на создании и сопровождении физической архитектуры в трёх основных кампусах и нескольких десятках удалённых подразделений по всей стране. Наш девиз звучал: «Вам это нужно? Мы это построим». Тому, что в действительности подключалось к сети, уделяли мало внимания. Управление сетью удобно заканчивалось у информационной розетки и страдало синдромом шоколадного батончика: снаружи всё было твёрдым, хрустящим и защищённым, а внутри - мягким и тягучим. У нас были хорошо защищённые границы, но мало внутренних средств контроля.

Такое отношение резко изменилось в январе 2003 года, когда червь Slammer (W32.SQLExp) и его варианты проникли в нашу среду. Внезапно стало чрезвычайно важно знать, что подключено к сети. В случае Slammer требовалось установить расположение всех устройств с MS SQL Server

2000 или MSDE 2000 и узнать их администраторов. Из-за отсутствия этих сведений искоренение Slammer заняло несколько месяцев.

Так родилась инициатива «Знать, что находится в сети». Это звучит упрощённо, но с учётом размера, сложности и истории сети стало крупным шагом вперёд и новым направлением наших служб управления сетью.

Nmap оказался ценным инструментом для этой работы. С его ценой трудно соперничать, и я высоко ценю преимущества, которые сообщество открытого исходного кода приносит его развитию, особенно снятие отпечатков ОС и многочисленные вклады конечных пользователей.

Я начал экспериментировать с Nmap. Моей целью было создать содержательную инвентаризацию сети, используя параметр Nmap `-O` для быстрого удалённого определения узлов по отпечаткам TCP/IP.

Сначала несколько слов о нашей IP-среде и платформе сканирования. Сейчас нам принадлежит один диапазон класса B и 44 диапазона класса C, а также используется большая часть частного адресного пространства. В сумме это 676 352 возможных IP-адреса. Я выполнял сканирования с Compaq DL380 под управлением Red Hat Linux 8.0. Первой попыткой было обычное TCP SYN-сканирование с определением ОС (`-O`) и только эхо-запросами ICMP для обнаружения узлов (`-PE`):

```
# nmap -O -PE -v -oX mayo.xml -iL ip_networks.txt
```

К сожалению, оно шло так медленно, что проверка всей сети заняла бы неделю. Поскольку все значимые части сети были соединены линиями не медленнее T1 (1,54 Мбит/с), я добавил готовую временную политику `insane` (`-T5`). Также был добавлен режим быстрого сканирования (`-F`), сокративший число проверяемых портов примерно с 1600 до 1200[2]. Кроме того, я добавил `--osscan-limit`, чтобы Nmap не тратил время на определение ОС узлов без открытых портов. Получилась следующая команда:

```
# nmap -O -T5 -PE -F --osscan-limit -v -oX mayo.xml -iL ip_networks.txt
```

К сожалению, и она, судя по всему, заняла бы несколько дней. Поэтому я отредактировал файл `nmap-services`, сократив число портов до 270. После этого сканирование завершилось немногим более чем за 49 часов и обнаружило 66 558 устройств. Настройка временных переменных, удаление параметра подробного вывода и перенаправление вывода в `/dev/null` сократили время до 46 часов. В результате получилась такая окончательная команда:

```
# nmap -O -T5 -PE -F --osscan-limit --max-rtt-timeout 100 \
       --max-parallelism 100 --min-hostgroup 100 -oX mayo.xml \
       -iL ip_networks.txt
```

Я планирую выполнять это сканирование еженедельно и передавать вывод в формате XML базе данных MS SQL. Результаты других наших методов сканирования уже поступают в эту базу, и мы можем создавать отчёты, помогающие достигать первоначальной цели: знать, что находится в сети. Возможно, я решу распределить нагрузку, запуская части сканирования на нескольких системах.

Сноски

[1] [назад] http://sectools.org/tools4.html#scanrand
[2] [назад] В Nmap версии 4.75 и выше `-F` ещё эффективнее: он сокращает число сканируемых портов до 100.

Глава 7. Определение версий служб и приложений

7.1. Введение

Хотя Nmap умеет многое, его самая основополагающая функция - сканирование портов. Направьте Nmap на удалённую машину, и он может сообщить, что порты 25/tcp, 80/tcp и 53/udp открыты. Используя свою базу `nmap-services`, содержащую более 2200 общеизвестных служб, Nmap сообщит, что эти порты, вероятно, соответствуют почтовому серверу (SMTP), веб-серверу (HTTP) и серверу имён (DNS). Такой поиск обычно точен: подавляющее большинство демонов, прослушивающих TCP-порт 25, действительно являются почтовыми серверами. Однако не следует ставить на это безопасность! Люди могут запускать и запускают службы на необычных портах. Возможно, их основной веб-сервер уже работает на порту 80, поэтому для промежуточного или тестового сервера они выбрали другой порт. Может быть, они думают, что, спрятав уязвимую службу на каком-нибудь малоизвестном порту, не дадут «злым хакерам» её найти. В последнее время ещё чаще люди выбирают порты не в зависимости от службы, которую хотят запустить, а в зависимости от того, что проходит через межсетевой экран. Когда интернет-провайдеры заблокировали порт 80 после крупных червей CodeRed и Nimda для Microsoft IIS, толпы пользователей в ответ перенесли свои личные веб-серверы на другой порт. Когда компании блокируют доступ по Telnet из-за его ужасных рисков для безопасности, я видел, как пользователи вместо этого просто запускали `telnetd` на порту Secure Shell (SSH).

Даже если Nmap прав и на приведённом выше гипотетическом сервере работают серверы SMTP, HTTP и DNS, это не так уж много сведений. При оценке уязвимостей (или даже простой инвентаризации сети) своих компаний или клиентов вам действительно нужно знать, какие именно почтовые и DNS-серверы и каких версий работают. Точный номер версии чрезвычайно помогает определить, каким эксплойтам уязвим сервер. Не забывайте, что исправления безопасности часто переносят в более ранние версии программ, поэтому полагаться исключительно на номер версии как на доказательство уязвимости службы нельзя. Ложноотрицательные результаты встречаются реже, но возможны, когда глупые администраторы подменяют номер версии уязвимой службы, чтобы она выглядела исправленной.

Ещё одна веская причина определять типы служб и номера версий состоит в том, что многие службы используют один и тот же номер порта. Например, порт 258/tcp используется и интерфейсом управления Checkpoint Firewall-1, и клиентом чата yak для Windows. Из-за этого предположение на основе таблицы `nmap-services` становится ещё менее точным. Всякий, кто много занимался сканированием, знает, что нередко встречаются службы, прослушивающие незарегистрированные порты: без определения версий это полная загадка. Последняя проблема заключается в том, что для простого сканера портов фильтруемые UDP-порты часто выглядят так же, как открытые (см. раздел 5.4 «UDP-сканирование (`-sU`)»). Но если они отвечают на предназначенные для конкретных служб пробы, которые отправляет подсистема определения версий Nmap, вы наверняка знаете, что они открыты (а часто и точно знаете, что на них работает).

Сканирование служб иногда раскрывает сведения о цели помимо типа службы и номера версии. Разнородные сведения, обнаруженные о службе, собираются в поле `info`. Оно отображается в столбце VERSION в круглых скобках после названия продукта и номера версии. Это поле может содержать номера протоколов SSH, модули Apache и многое другое.

Некоторые службы также сообщают настроенные имена своих узлов, которые на удивление часто отличаются от имён машин, полученных обратным разрешением DNS. Поле имени узла выводится в строке `Service Info` после таблицы портов. Это кажется незначительной утечкой информации,

но может иметь последствия. Однажды на конференции по безопасности CanSecWest я сидел с ноутбуком в своём номере. Внезапно окно `tcpdump` в углу моего экрана пришло в неистовство, и я понял, что мою машину атакуют. Я просканировал в ответ и обнаружил открытый необычно высокий порт. После подключения порт извергнул множество двоичных символов, но одно поле ASCII в выводе выдало настроенное доменное имя. Домен принадлежал достаточно небольшой компании по безопасности, чтобы я точно знал, кто за это отвечает. Я попросил администратора отеля позвонить в его номер, и, надо же, как он удивился, когда я попросил его прекратить прощупывать мою машину.

Ещё два поля, которые способно обнаружить определение версий, - операционная система и тип устройства. Они также выводятся в строке `Service Info`. Здесь применяются два метода. Первый - исключительная принадлежность приложения. Если мы определили службу как Microsoft Exchange, то знаем, что операционная система - Windows, поскольку Exchange ни на чём другом не работает. Второй метод - убедить более переносимые приложения раскрыть сведения о платформе. Многие серверы (особенно веб-серверы) требуют совсем немного уговоров. Этот вид определения ОС призван дополнять систему определения ОС Nmap (`-O`) и иногда может сообщать отличающиеся результаты. Представьте сервер Microsoft Exchange, скрытый за перенаправляющим порты межсетевым экраном Unix.

Подсистема сканирования версий Nmap получает все эти данные, подключаясь к открытым портам и опрашивая их ради дополнительных сведений с помощью проб, понятных конкретным службам. Благодаря этому Nmap может дать подробную оценку того, что работает на самом деле, а не только сообщить номера открытых портов. В примере 7.1 показан фактический вывод.

Пример 7.1. Простое применение определения версий

```
# nmap -A -T4 -F insecure.org

Starting Nmap ( http://nmap.org
Interesting ports on insecure.org (205.217.153.53):
(The 1206 ports scanned but not shown below are in state: filtered)
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 3.1p1 (protocol 1.99)
25/tcp  open   smtp    Qmail smtpd
53/tcp  open   domain  ISC BIND 9.2.1
80/tcp  open   http    Apache httpd 2.0.39 ((Unix) mod_perl/1.99_07-dev)
113/tcp closed auth
Device type: general purpose
Running: Linux 2.4.X|2.5.X
OS details: Linux Kernel 2.4.0 - 2.5.20

Nmap finished: 1 IP address (1 host up) scanned in 34.962 seconds
```

Определение версий Nmap предлагает следующие расширенные возможности (полностью описанные далее):

• Высокоскоростная параллельная работа посредством неблокирующих сокетов и грамматики определения проб и совпадений, предназначенной для эффективной и в то же время мощной реализации.

• Определение названия приложения и номера версии, если они доступны, а не только протокола службы.

• Поддержка протоколов TCP и UDP, а также как текстовых служб ASCII, так и служб с упакованными двоичными данными.

• Поддержка множества платформ, включая Linux, Windows, Mac OS X, FreeBSD/NetBSD/OpenBSD, Solaris и все остальные платформы, на которых, как известно, работает Nmap.

• Если обнаружен SSL, Nmap подключается с помощью OpenSSL (если он доступен) и пытается определить, какая служба прослушивает за этим слоем шифрования. Благодаря этому он может

обнаруживать такие службы, как HTTPS, POP3S, IMAPS и другие, а также сообщать сведения о версиях.

• Если обнаружена служба SunRPC, Nmap запускает свой RPC-переборщик методом грубой силы, чтобы найти номер программы, название и номер версии.

• Поддерживается IPv6, включая TCP, UDP и SSL поверх TCP.

• Вклад сообщества: если Nmap получает данные от службы, которую не распознаёт, он печатает отпечаток службы вместе с URL для его отправки. Эта система создана по образцу чрезвычайно успешного процесса отправки отпечатков определения ОС Nmap. Можно также отправлять новые пробы и исправления.

• Всеобъемлющая база данных: Nmap распознаёт более тысячи сигнатур служб, охватывающих свыше 180 уникальных протоколов служб - от ACAP, AFP и AIM до XML-RPC, Zebedee и Zebra.

7.2. Применение и примеры

Прежде чем углубляться в технические подробности реализации определения версий, рассмотрим несколько примеров, демонстрирующих его применение и возможности. Чтобы включить определение версий, просто добавьте `-sV` к тем флагам Nmap, которыми обычно пользуетесь. Либо примените параметр `-A`, включающий определение версий и другие расширенные и агрессивные возможности, описанные далее. Всё действительно настолько просто, как показано в примере 7.2.

Пример 7.2. Определение версий для www.microsoft.com

```
# nmap -A -T4 -F www.microsoft.com

Starting Nmap ( http://nmap.org )
Interesting ports on 80.67.68.30:
(The 1208 ports scanned but not shown below are in state: closed)
PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      Akamai-I SSH (protocol 1.5)
80/tcp  open  http     AkamaiGHost (Akamai's HTTP Acceleration service)
443/tcp open  ssl/http AkamaiGHost (Akamai's HTTP Acceleration service)
Device type: general purpose
Running: Linux 2.1.X|2.2.X
OS details: Linux 2.1.19 - 2.2.25

Nmap finished: 1 IP address (1 host up) scanned in 19.223 seconds
```

Предыдущее сканирование демонстрирует две вещи. Во-первых, приятно видеть, что `www.Microsoft.Com` обслуживается одной из Linux-машин Akamai. Для этой главы важнее то, что для порта 443 указана служба `ssl/http`. Это означает, что определение служб сначала обнаружило на порту SSL, затем загрузило OpenSSL и снова выполнило определение служб через SSL-соединения, чтобы обнаружить работающий за шифрованием веб-сервер AkamiGHost. Напомним, что `-T4` заставляет Nmap работать быстрее (с более агрессивными временными параметрами), а `-F` велит Nmap сканировать только порты, зарегистрированные в `nmap-services`.

Пример 7.3 длиннее и разнообразнее.

Пример 7.3. Сложное определение версий

```
# nmap -A -T4 localhost

Starting Nmap ( http://nmap.org
Interesting ports on felix (127.0.0.1):
(The 1640 ports scanned but not shown below are in state: closed)
PORT     STATE SERVICE    VERSION
21/tcp   open  ftp        WU-FTPD wu-2.6.1-20
```

```
22/tcp   open  ssh        OpenSSH 3.1p1 (protocol 1.99)
53/tcp   open  domain     ISC BIND 9.2.1
79/tcp   open  finger     Linux fingerd
111/tcp  open  rpcbind    2 (rpc #100000)
443/tcp  open  ssl/http   Apache httpd 2.0.39 ((Unix) mod_perl/1.99_04-dev)
515/tcp  open  printer
631/tcp  open  ipp        CUPS 1.1
953/tcp  open  rndc?
5000/tcp open  ssl/ftp    WU-FTPD wu-2.6.1-20
5001/tcp open  ssl/ssh    OpenSSH 3.1p1 (protocol 1.99)
5002/tcp open  ssl/domain ISC BIND 9.2.1
5003/tcp open  ssl/finger Linux fingerd
6000/tcp open  X11        (access denied)
8000/tcp open  http-proxy Junkbuster webproxy
8080/tcp open  http       Apache httpd 2.0.39 ((Unix) mod_perl/1.99_04-dev)
8081/tcp open  http       Apache httpd 2.0.39 ((Unix) mod_perl/1.99_04-dev)
Device type: general purpose
Running: Linux 2.4.X|2.5.X
OS details: Linux Kernel 2.4.0 - 2.5.20

Nmap finished: 1 IP address (1 host up) scanned in 42.494 seconds
```

Здесь видно, как обрабатываются службы RPC: RPC-сканер методом грубой силы определил, что на порту 111 работает `rpcbind` версии 2. Можно также заметить, что для порта 515 указана служба `printer`, но столбец версии пуст. Nmap определил название службы посредством проб, но не смог выяснить ничего больше. С другой стороны, для порта 953 указана служба «`rndc?`». Знак вопроса сообщает, что Nmap даже не смог определить название службы посредством проб. В качестве запасного варианта упомянута `rndc`, поскольку за ней в `nmap-services` зарегистрирован порт 953. К сожалению, ни одна из проб Nmap не вызвала от `rndc` никакого ответа. Если бы ответ был, Nmap напечатал бы отпечаток службы и URL для отправки, чтобы её можно было распознать в следующей версии. В нынешнем виде Nmap требуется особая проба. Возможно, к тому времени, когда вы будете это читать, она уже появится. В разделе 7.7 «Вклад сообщества» подробно рассказано о написании собственных проб.

Стоит также отметить, что некоторые службы предоставляют гораздо больше сведений, чем один лишь номер версии. В приведённых выше примерах это разрешение X11 на подключения, номер протокола SSH и список версий модулей Apache. Некоторые модули Apache даже пришлось убрать из вывода, чтобы он поместился на этой странице.

Несколько первых рецензентов усомнились в разумности запуска таких служб, как SSH и finger, поверх SSL. На самом деле это была всего лишь забава со `stunnel`[1], отчасти предназначенная для проверки того, что параллельное SSL-сканирование действительно работает.

7.3. Описание метода

В действительности сканирование версий Nmap довольно незамысловато. Оно проектировалось настолько простым, насколько возможно, оставаясь при этом масштабируемым, быстрым и точным. Совсем уж мелкие подробности лучше всего изучать, загрузив и просмотрев исходный код, но ниже приведён краткий обзор применяемых методов.

Сначала Nmap выполняет сканирование портов в соответствии с вашими указаниями, а затем передаёт все TCP- и/или UDP-порты в состоянии `open` или `open|filtered` модулю сканирования служб. После этого порты опрашиваются параллельно, хотя для простоты здесь описывается один порт.

1. Nmap проверяет, не относится ли порт к исключаемым портам, заданным директивой `Exclude` в `nmap-service-probes`. Если относится, Nmap не будет сканировать этот порт по причинам, упомянутым в разделе 7.6 «Формат файла nmap-service-probes».

2. Если это TCP-порт, Nmap начинает с подключения к нему. Если подключение удалось, а порт находился в состоянии `open|filtered`, оно изменяется на `open`. Для TCP такое встречается редко, поскольку люди, стремящиеся к такой скрытности, что применяют разновидность TCP-сканирования, дающую порты `open|filtered` (например, FIN-сканирование), обычно понимают, что не стоит уничтожать всю свою скрытность определением версий.

3. Установив TCP-соединение, Nmap примерно пять секунд прослушивает его. Многие распространённые службы, включая большинство серверов FTP, SSH, SMTP, Telnet, POP3 и IMAP, представляются в начальном приветственном баннере. Nmap называет это «NULL-пробой», поскольку лишь слушает ответы, не отправляя никаких данных пробы. Если получены какие-либо данные, Nmap сравнивает их с сотнями регулярных выражений сигнатур в своём файле `nmap-service-probes` (описанном в разделе 7.6 «Формат файла nmap-service-probes»). Если служба полностью определена, работа с этим портом закончена! Регулярное выражение включает подстроки, с помощью которых из ответа можно извлечь номера версий. В некоторых случаях Nmap получает «мягкое совпадение» типа службы, но не сведения о версии. Тогда Nmap продолжает работу, однако отправляет только пробы, способные распознать службу с полученным мягким совпадением.

4. На этом этапе запускаются UDP-пробы Nmap; сюда же попадают TCP-соединения, если описанная выше NULL-проба завершилась неудачей или дала мягкое совпадение. Поскольку в действительности большинство портов используется службами, за которыми они зарегистрированы в `nmap-services`, у каждой пробы есть список номеров портов, на которых она считается наиболее эффективной. Например, проба GetRequest, распознающая веб-серверы (помимо других служб), указывает порты 80-85, 8000-8010 и 8080-8085 как вероятные. Nmap последовательно выполняет пробу или пробы, соответствующие номеру сканируемого порта.

Каждая проба содержит строку пробы (это может быть произвольный текст ASCII или двоичные данные с экранированием `\xHH`), отправляемую на порт. Возвращающиеся ответы сравниваются со списком регулярных выражений того же типа, что обсуждался выше в описании NULL-пробы. Как и при NULL-пробе, эти проверки могут привести либо к полному совпадению (обработка удалённой службы прекращается), либо к мягкому совпадению (дальнейшие пробы ограничиваются соответствующими определённой службе), либо к полному отсутствию совпадения. Точный список регулярных выражений, по которым Nmap проверяет совпадение, зависит от настройки резервных вариантов пробы. Например, данные, возвращённые пробой X11Probe, вряд ли совпадут с регулярными выражениями, созданными для пробы GetRequest. С другой стороны, результаты такой пробы, как RTSPRequest, вполне могут совпасть с регулярным выражением для GetRequest, поскольку проверяемые ими протоколы тесно связаны. Поэтому для пробы RTSPRequest совпадения GetRequest служат резервным вариантом. Более полное объяснение см. в разделе 7.3.1 «Уловки и резервные варианты».

Если при определении версий от UDP-порта в состоянии `open|filtered` когда-либо приходит какой-нибудь ответ, это состояние изменяется на `open`. Благодаря этому определение версий превосходно дополняет UDP-сканирование, которое при действии некоторых распространённых правил межсетевых экранов вынуждено помечать все просканированные UDP-порты как `open|filtered`. Хотя объединение UDP-сканирования с определением версий может занять во много раз больше времени, чем простое UDP-сканирование, это эффективный и полезный метод. Он описан в разделе 5.4.1 «Как отличить открытые UDP-порты от фильтруемых».

5. В большинстве случаев служба совпадает с NULL-пробой или описанной выше пробой либо пробами вероятного порта (обычно такая проба только одна). Поскольку NULL-проба использует одно соединение совместно с пробой вероятного порта, в большинстве случаев определение службы удаётся выполнить всего за одно короткое соединение. Для UDP обычно требуется лишь один пакет. Но если NULL-проба и пробы вероятного порта не срабатывают, Nmap

последовательно перебирает все имеющиеся пробы. В случае TCP Nmap должен устанавливать для каждой пробы новое соединение, чтобы предыдущие пробы не исказили результаты. Этот наихудший сценарий может занять некоторое время, особенно потому, что из-за медленных сетевых соединений и просто медленно отвечающих служб Nmap приходится около пяти секунд ждать результатов каждой пробы. К счастью, Nmap применяет несколько автоматических способов ускорить сканирование:

• Nmap делает большинство проб достаточно универсальными, чтобы они совпадали со множеством служб. Например, проба GenericLines отправляет службе две пустые строки (`"\r\n\r\n"`). Это соответствует демонам множества различных типов служб, включая FTP, ident, POP3, UUCP, Postgres и whois. Проба GetRequest совпадает с ещё большим числом типов служб. Среди других примеров - `"help\r\n"`, а также универсальные пробы RPC и MS SMB.

• Если служба совпадает с директивой `softmatch`, Nmap достаточно испытать лишь те пробы, которые потенциально способны совпасть с этой службой.

• Не все пробы созданы равными! Некоторые совпадают с гораздо большим числом служб, чем другие. Поэтому Nmap использует показатель редкости, чтобы не испытывать пробы, совпадение которых крайне маловероятно. Опытные пользователи Nmap могут принудительно испытать все пробы без исключения или ограничить число попыток ещё сильнее, чем по умолчанию, с помощью параметров `--version-intensity`, `--version-all` и `--version-light`, обсуждаемых в разделе 7.3.2 «Выбор проб и редкость».

6. Одна из проб проверяет, работает ли на целевом порту SSL. Если да (и если доступен OpenSSL), Nmap повторно подключается через SSL и заново запускает сканирование служб, чтобы определить, что прослушивает за шифрованием. Специальная директива позволяет задавать разные вероятные порты для обычных и туннелированных через SSL соединений. Например, для порта 443 (HTTPS) Nmap следует начинать с SSL-пробы. Но после обнаружения и включения SSL Nmap следует испытать на порту 443 пробу GetRequest, поскольку за SSL-шифрованием на этом порту обычно прослушивает веб-сервер.

7. Другая универсальная проба распознаёт службы на основе RPC. При их обнаружении запускается RPC-переборщик Nmap (обсуждаемый далее), который методом грубой силы находит номер и название программы RPC, а также номера поддерживаемых версий. Аналогичным образом со временем может быть добавлен постпроцессор SMB для снятия отпечатков служб Windows.

8. Если хотя бы одна из проб вызывает какой-нибудь ответ, но Nmap не может распознать службу, содержимое ответа выводится пользователю в виде отпечатка. Если пользователи знают, какие службы на самом деле прослушивают порт, им предлагается отправить отпечаток разработчикам Nmap для включения в Nmap, как описано в разделе 7.7.1 «Отправка отпечатков служб».

7.3.1. Уловки и резервные варианты

Хотя Nmap отводит службам щедрое время для ответа, иногда приложение медленно отвечает на NULL-пробу. Это может происходить по ряду причин, включая медленные запросы обратного DNS, выполняемые некоторыми службами. Поэтому иногда Nmap может сопоставить результаты последующей пробы со строкой совпадения, предназначенной для NULL-пробы.

Предположим, например, что мы сканируем порт 25 (SMTP) сервера, чтобы определить, что его прослушивает. Сразу после нашего подключения эта служба может выполнить множество запросов к чёрным спискам DNS, чтобы решить, следует ли считать нас спамерами и отказать в обслуживании. Прежде чем она закончит, Nmap перестаёт ждать ответа на NULL-пробу и отправляет следующую пробу, для которой зарегистрирован порт 25, - `"HELP\r\n"`. Когда служба наконец завершает антиспам-проверки, она печатает приветственный баннер, читает пробу Help и

отвечает, как показано в примере 7.4.

Пример 7.4. Вывод примера с уловкой NULL-пробы

```
220 hcsw.org ESMTP Sendmail 8.12.3/8.12.3/Debian-7.1; Tue, [cut]
214-2.0.0 This is sendmail version 8.12.3
214-2.0.0 Topics:
214-2.0.0       HELO    EHLO    MAIL    RCPT    DATA
214-2.0.0       RSET    NOOP    QUIT    HELP    VRFY
                EXPN    VERB    ETRN    DSN     AUTH
                STARTTLS
        For more info use "HELP <topic>".
        To report bugs in the implementation send email to
214-2.0.0       sendmail-bugs@sendmail.org.
214-2.0.0 For local information send email to Postmaster at your site.
214 2.0.0 End of HELP info
```

Nmap читает эти данные из сокета и обнаруживает, что ни одно регулярное выражение пробы Help не совпадает с возвращёнными данными. Причина в том, что обычно Nmap ожидает получить баннер ESMTP во время NULL-пробы и сопоставить его именно тогда.

Поскольку такой сценарий встречается сравнительно часто, Nmap «жульничает»: если ни одна строка, относящаяся к конкретной пробе, не совпала, он пытается сопоставить ответ с любой строкой совпадения NULL-пробы. В данном случае существует строка NULL-совпадения, сообщающая, что программа - Sendmail, версия - 8.12.3/8.12.3/Debian-7.1, а имя узла - hcsw.org.

Уловка NULL-пробы на самом деле лишь частный пример более общей возможности Nmap - резервных вариантов. Директива `fallback` подробно описана в разделе 7.6 «Формат файла nmap-service-probes». По существу, всякая проба, которая с большой вероятностью может получить результаты, совпадающие с регулярными выражениями других проб, имеет директиву `fallback`, задающую эти другие пробы.

Например, в некоторых конфигурациях популярного веб-сервера Apache он не отвечает на пробу GetRequest (`"GET / HTTP/1.0\r\n\r\n"`), поскольку не указано имя виртуального узла. Тем не менее Nmap правильно распознаёт такие серверы, потому что они обычно отвечают на пробу HTTPOptions. У неё резервным вариантом служат регулярные выражения GetRequest, достаточно общие, чтобы распознать ответы Apache на пробы HTTPOptions.

7.3.2. Выбор проб и редкость

Определяя, какие пробы использовать, Nmap учитывает их `rarity` (редкость). Это показатель вероятности того, что проба вернёт полезные данные. Проба с высокой редкостью считается менее распространённой, и вероятность её испытания ниже. Пользователи Nmap могут задавать, какие пробы испытываются, изменяя описанный ниже уровень интенсивности сканирования версий. Точный алгоритм, по которому Nmap определяет используемые пробы, таков:

1. Для TCP первой всегда испытывается NULL-проба.

2. Все пробы, для которых сканируемый порт указан как вероятный (см. раздел 7.6 «Формат файла nmap-service-probes»), испытываются в порядке их следования в `nmap-service-probes`.

3. Все остальные пробы, значение редкости которых меньше текущего значения интенсивности сканирования или равно ему, также испытываются в порядке их следования в `nmap-service-probes`.

Как только обнаруживается совпавшая проба, алгоритм завершается и сообщаются результаты.

Поскольку со всеми пробами Nmap (кроме NULL-пробы) связано значение редкости, управлять количеством испытываемых при сканировании версий проб сравнительно легко. Просто выберите

подходящий для сканирования уровень интенсивности. Чем выше уровень интенсивности, тем больше проб будет испытано. Поэтому для очень всеобъемлющего сканирования подходит высокий уровень интенсивности, хотя оно может занять больше времени, чем сканирование с низким уровнем. Уровень интенсивности Nmap по умолчанию равен 7, однако для разных потребностей сканирования Nmap предоставляет следующие ключи:

• `--version-intensity <уровень интенсивности от 0 до 9>` — Задаёт указанный уровень интенсивности сканирования версий. Если указано 0, испытываются только NULL-проба (для TCP) и пробы, указывающие порт как вероятный. Пример: `nmap -sV --version-intensity 3 scanme.nmap.org`

• `--version-intensity` — Задаёт уровень интенсивности 2. Пример: `nmap -sV --version-light scanme.nmap.org`

• `--version-all` — Задаёт уровень интенсивности 9. Поскольку уровень редкости всех проб лежит между 1 и 9, испытываются все пробы. Пример: `nmap -sV --version-all scanme.nmap.org`

7.4. Демонстрация метода

Если приведённое выше описание на английском языке недостаточно понятно, вы можете сами увидеть, как всё работает, добавив к командной строке Nmap параметры `--version-trace` (и обычно `-d` (отладка)). Они показывают всю активность подключений и чтения/записи данных при сканировании служб. Далее приведён снабжённый пояснениями пример из реальной практики.

```
# nmap -sSV -T4 -F -d --version-trace insecure.org

Starting Nmap ( http://nmap.org )
Host insecure.org (205.217.153.53) appears to be up ... good.

Initiating SYN Stealth Scan against insecure.org (205.217.153.53) at 19:53
Initiating service scan against 4 services on 1 host at 19:53
```

SYN-сканирование обнаружило четыре открытых порта; теперь мы начинаем параллельное сканирование служб на каждом из них. Начинаем с TCP-соединения для NULL-пробы:

```
Starting probes against new service: 205.217.153.53:22 (tcp)
NSOCK (2.0750s) TCP connection requested to 205.217.153.53:22 (IOD #1) EID 8
Starting probes against new service: 205.217.153.53:25 (tcp)
NSOCK (2.0770s) TCP connection requested to 205.217.153.53:25 (IOD #2) EID 16
Starting probes against new service: 205.217.153.53:53 (tcp)
NSOCK (2.0830s) TCP connection requested to 205.217.153.53:53 (IOD #3) EID 24
Starting probes against new service: 205.217.153.53:80 (tcp)
NSOCK (2.0860s) TCP connection requested to 205.217.153.53:80 (IOD #4) EID 32
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 32 [205.217.153.53:80]
NSOCK (2.0870s) Read request from IOD #4 [205.217.153.53:80]
                (timeout: 5000ms) EID 42
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 24 [205.217.153.53:53]
NSOCK (2.0870s) Read request from IOD #3 [205.217.153.53:53]
                (timeout: 5000ms) EID 50
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 16 [205.217.153.53:25]
NSOCK (2.0870s) Read request from IOD #2 [205.217.153.53:25]
                (timeout: 5000ms) EID 58
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 8 [205.217.153.53:22]
NSOCK (2.0870s) Read request from IOD #1 [205.217.153.53:22]
                (timeout: 5000ms) EID 66
```

К этому моменту соединения NULL-пробы успешно установлены со всеми четырьмя службами. Отсчёт начинается с двух секунд, поскольку именно столько заняли ping- и SYN-сканирование.

```
NSOCK (2.0880s) Callback: READ SUCCESS for EID 66 [205.217.153.53:22]
                        (23 bytes): SSH-1.99-OpenSSH_3.1p1.
Service scan match: 205.217.153.53:22 is ssh.
```

```
                    Version: |OpenSSH|3.1p1|protocol 1.99|
```

SSH любезно полностью представился сразу после подключения как OpenSSH 3.1p1. Один готов, осталось три.

```
NSOCK (2.0880s) Callback: READ SUCCESS for EID 58 [205.217.153.53:25]
                        (27 bytes): 220 core.lnxnet.net ESMTP..
Service scan soft match: 205.217.153.53:25 is smtp
```

Почтовый сервер на порту 25 также выдал полезный баннер. Мы не знаем, что это за тип почтового сервера, но начало с `220` и наличие слова `ESMTP` сообщают нам, что это почтовый сервер (SMTP). Поэтому Nmap получает мягкое совпадение с `smtp`, а значит, отныне испытываются только пробы, способные совпасть с SMTP-серверами. Обратите внимание: непечатаемые символы представлены точками, поэтому «`..`» после ESMTP на самом деле является последовательностью завершения строки «`\r\n`».

```
NSOCK (2.0880s) Read request from IOD #2 [205.217.153.53:25]
                (timeout: 4996ms) EID 74
NSOCK (7.0880s) Callback: READ TIMEOUT for EID 74 [205.217.153.53:25]
NSOCK (7.0880s) Write request for 6 bytes to IOD #2 EID 83
                [205.217.153.53:25]: HELP..

NSOCK (7.0880s) Read request from IOD #2 [205.217.153.53:25]
                (timeout: 5000ms) EID 90
```

Nmap ещё немного прослушивает SMTP-соединение на случай, если серверу есть что добавить. Через пять секунд время ожидания запроса чтения истекает. Затем Nmap находит следующую пробу, зарегистрированную для порта 25 и имеющую сигнатуры SMTP. Эта проба состоит всего лишь из `HELP\r\n`, которое Nmap записывает в соединение.

```
NSOCK (7.0880s) Callback: READ TIMEOUT for EID 50 [205.217.153.53:53]
NSOCK (7.0880s) Write request for 32 bytes to IOD #3 EID 99
                [205.217.153.53:53]: ..............version.bind.....
NSOCK (7.0880s) Read request from IOD #3 [205.217.153.53:53]
                (timeout: 5000ms) EID 106
```

DNS-сервер на порту 53 вообще ничего не возвращает. Первая проба, зарегистрированная для порта 53 в `nmap-service-probes`, - DNSVersionBindReq, запрашивающая у DNS-сервера номер его версии. Она отправляется в сеть.

```
NSOCK (7.0880s) Callback: READ TIMEOUT for EID 42 [205.217.153.53:80]
NSOCK (7.0880s) Write request for 18 bytes to IOD #4 EID 115
                [205.217.153.53:80]: GET / HTTP/1.0....
NSOCK (7.0880s) Read request from IOD #4 [205.217.153.53:80]
                (timeout: 5000ms) EID 122
```

NULL-проба порта 80 также не вернула никаких данных. Поскольку для порта 80 зарегистрирована соответствующая проба, отправляется HTTP-запрос GET.

```
NSOCK (7.0920s) Callback: READ SUCCESS for EID 122
                [205.217.153.53:80] [EOF] (15858 bytes)
Service scan match: insecure.org (205.217.153.53):80 is http.
                   Version: |Apache httpd|2.0.39|(Unix) mod_perl/1.99_07-dev..
```

Apache вернул огромный ответ (15 КБ), поэтому он не напечатан. В ответе содержались подробные сведения о конфигурации, которые Nmap из него извлекает. Других проб, зарегистрированных для порта 80, нет. Поэтому, если бы эта завершилась неудачей, Nmap испытал бы первую TCP-пробу из `nmap-service-probes`. Она всего лишь отправляет пустые строки (`"\r\n\r\n"`). На случай, если проба GET привела службу в замешательство, было бы установлено новое соединение.

```
NSOCK (7.0920s) Callback: READ SUCCESS for EID 106 [205.217.153.53:53]
                         (50 bytes): .0.........version.bind......9.2.1
Service scan match: insecure.org (205.217.153.53):53 is domain.
                   Version: |ISC BIND|9.2.1||
```

Порт 53 ответил на наш запрос версии DNS. Большая часть ответа (как и пробы) двоичная, но версию 9.2.1 в нём видно совершенно ясно. Если бы эта проба завершилась неудачей, следующей зарегистрированной для порта 53 пробой был бы запрос состояния DNS-сервера (14 байт: `\0\x0C\0\0\x10\0\0\0\0\0\0\0\0\0`). Такая резервная проба помогает, поскольку на запрос состояния отвечает гораздо больше серверов, чем на запрос номера версии.

```
NSOCK (7.0920s) Callback: READ SUCCESS for EID 90 [205.217.153.53:25]
                          (55 bytes): 214 qmail home page: http...

Service scan match: insecure.org (205.217.153.53):25 is smtp.
                  Version: |qmail smtpd|||
```

Порт 25 дал очень полезный ответ на пробу Help. С помощью этой пробы нередко можно также распознать другие SMTP-серверы, такие как Postfix, Courier и Exim. Если бы ответ не совпал, Nmap прекратил бы работу с этой службой, поскольку уже получил мягкое совпадение с `smtp`, а других SMTP-проб в `nmap-service-probes` нет.

```
The service scan took 5 seconds to scan 4 services on 1 host.
```

Этот запуск сканирования служб прошёл довольно хорошо. Ни одной службе не потребовалось больше одного соединения. Работа заняла пять секунд, поскольку для Qmail и Apache истекло пятисекундное время ожидания NULL-пробы, прежде чем Nmap отправил первые настоящие пробы. Вот награда за эти усилия:

```
Interesting ports on insecure.org (205.217.153.53):
(The 1212 ports scanned but not shown below are in state: closed)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 3.1p1 (protocol 1.99)
25/tcp open  smtp    qmail smtpd
53/tcp open  domain  ISC BIND 9.2.1
80/tcp open  http    Apache httpd 2.0.39 ((Unix) mod_perl/1.99_07-dev)

Nmap finished: 1 IP address (1 host up) scanned in 7.104 seconds
```

7.5. Постпроцессоры

Обычно Nmap заканчивает работу с портом, определив службу и сведения о версии, как показано выше. Однако для некоторых служб Nmap выполняет дополнительную работу. В настоящее время доступны постпроцессоры интеграции с Nmap Scripting Engine, RPC-перебора и SSL-туннелирования. Рассматривается возможность опроса Windows SMB.

7.5.1. Интеграция с Nmap Scripting Engine

Основанный на регулярных выражениях подход к определению версий обладает большими возможностями, но распознаёт не всё. Некоторые службы нельзя распознать, просто отправив стандартную пробу и сопоставив ответ с шаблоном. Для некоторых служб нужны нестандартные строки проб или сложный многоэтапный процесс установления связи. Другим для распознавания ответа требуется более сложная обработка, чем регулярное выражение. Например, службу Skype v2 спроектировали так, чтобы её было трудно обнаружить, поскольку существующие операторы (например, телефонные компании, предоставляющие линии DSL) могли счесть её конкурентом и ухудшить либо заблокировать эту службу для своих абонентов. Единственный найденный нами способ обнаружить эту службу предполагал анализ ответов на две разные пробы. Аналогичным образом мы могли бы распознавать больше служб SNMP, если бы перебирали методом грубой силы несколько сотен разных имён сообществ. Ни одна из этих задач не слишком хорошо подходит для традиционного определения версий Nmap, но обе легко решаются с помощью языка сценариев

Nmap. По этим причинам определение версий теперь по умолчанию вызывает NSE для обработки некоторых труднораспознаваемых служб, как описано в разделе 9.10 «Определение версий с помощью NSE».

7.5.2. RPC-перебор

SunRPC (Sun Remote Procedure Call) - распространённый протокол Unix, применяемый для реализации многих служб, включая NFS. Nmap поставляется с базой данных `nmap-rpc`, содержащей почти 600 программ RPC. Многие службы RPC используют порты с высокими номерами и/или транспортный протокол UDP, благодаря чему оказываются доступными через множество плохо настроенных межсетевых экранов. У программ RPC (как и у самих инфраструктурных библиотек) также долгая история серьёзных дыр в безопасности, пригодных для удалённой эксплуатации. Поэтому сетевые администраторы и аудиторы безопасности часто хотят узнать больше обо всех программах RPC в своих сетях.

Если доступна служба portmapper (`rpcbind`) (UDP- или TCP-порт 111), службы RPC можно перечислить Unix-командой `rpcinfo`. В примере 7.5 это продемонстрировано на сервере Solaris 9 с настройками по умолчанию.

Пример 7.5. Перечисление служб RPC с помощью rpcinfo

```
> rpcinfo -p ultra
   program vers proto   port
    100000    4   tcp    111  rpcbind
    100000    4   udp    111  rpcbind
    100232   10   udp  32777  sadmind
    100083    1   tcp  32775  ttdbserverd
    100221    1   tcp  32777  kcms_server
    100068    5   udp  32778  cmsd
    100229    1   tcp  32779  metad
    100230    1   tcp  32781  metamhd
    100242    1   tcp  32783  rpc.metamedd
    100001    4   udp  32780  rstatd
    100002    3   udp  32782  rusersd
    100002    3   tcp  32785  rusersd
    100008    1   udp  32784  walld
    100012    1   udp  32786  sprayd
    100011    1   udp  32788  rquotad
    100024    1   udp  32790  status
    100024    1   tcp  32787  status
    100133    1   udp  32790  nsm_addrand
    100133    1   tcp  32787  nsm_addrand
    [ Dozens of lines cut for brevity ]
```

Этот пример показывает, что узлы часто предоставляют множество служб RPC, повышая вероятность того, что одна из них пригодна для эксплуатации. Следует также заметить, что большинство служб работает на необычных портах с высокими номерами (которые могут изменяться по самым разным причинам) и распределено между транспортными протоколами UDP и TCP.

Поскольку сведения RPC настолько чувствительны, многие администраторы пытаются скрыть их, блокируя порт portmapper (111). К сожалению, дыру это не закрывает. Nmap может определить все те же сведения, напрямую обмениваясь данными с открытыми RPC-портами в ходе следующего трёхэтапного процесса.

1. Сканирование TCP- и/или UDP-портов находит все открытые порты.

2. Определение версий устанавливает, какие из открытых портов используют протокол SunRPC.

3. Механизм RPC-перебора устанавливает, какая программа работает на каждом RPC-порту, испытывая пустую команду для каждого из 600 номеров программ в `nmap-rpc`. В большинстве случаев Nmap угадывает неверно и получает сообщение об ошибке, указывающее, что запрошенный номер программы не прослушивает порт. Nmap продолжает испытывать каждый номер из своего списка, пока один из них не вернёт успех. Nmap сдаётся в маловероятном случае исчерпания всех известных ему номеров программ либо если порт отправляет неправильно сформированные ответы, позволяющие предположить, что в действительности это не RPC.

Пробы для определения программ RPC выполняются параллельно, а для UDP-портов обрабатываются повторные передачи. Эта возможность автоматически активируется всякий раз, когда определение версий находит какие-либо RPC-порты. Её можно также применить без определения версий, указав параметр `-sR`. В примере 7.6 показано прямое RPC-сканирование, выполняемое как часть определения версий.

Пример 7.6. Прямое RPC-сканирование Nmap

```
# nmap -F -A -sSU ultra

Starting Nmap ( http://nmap.org
Interesting ports on ultra.nmap.org (192.168.0.50):
(The 2171 ports scanned but not shown below are in state: closed)
PORT      STATE SERVICE     VERSION
[A whole bunch of ports cut for brevity]
32776/tcp open  kcms_server 1 (rpc #100221)
32776/udp open  sadmind     10 (rpc #100232)
32777/tcp open  kcms_server 1 (rpc #100221)
32777/udp open  sadmind     10 (rpc #100232)
32778/tcp open  metad       1 (rpc #100229)
32778/udp open  cmsd        2-5 (rpc #100068)
32779/tcp open  metad       1 (rpc #100229)
32779/udp open  rstatd      2-4 (rpc #100001)
32780/tcp open  metamhd     1 (rpc #100230)
32780/udp open  rstatd      2-4 (rpc #100001)
32786/tcp open  status      1 (rpc #100024)
32786/udp open  sprayd      1 (rpc #100012)
32787/tcp open  status      1 (rpc #100024)
32787/udp open  rquotad     1 (rpc #100011)
Device type: general purpose
Running: Sun Solaris 9
OS details: Sun Solaris 9

Nmap finished: 1 IP address (1 host up) scanned in 252.701 seconds
```

7.5.3. Примечания о постпроцессоре SSL

Как обсуждалось в разделе с описанием метода, Nmap способен обнаружить протокол шифрования SSL, а затем открыть зашифрованный сеанс, через который выполняется обычное определение версий. Как и в случае с рассмотренным выше RPC-переборщиком, постпроцессор SSL запускается автоматически при каждом обнаружении подходящего (SSL) порта. Это показано в примере 7.7.

Пример 7.7. Сканирование версий через SSL

```
nmap -PN -sSV -T4 -F www.amazon.com

Starting Nmap ( http://nmap.org )
Interesting ports on 207-171-184-16.amazon.com (207.171.184.16):
(The 1214 ports scanned but not shown below are in state: filtered)
PORT    STATE SERVICE  VERSION
80/tcp  open  http     Apache Stronghold httpd 2.4.2 (based on Apache 1.3.6)
443/tcp open  ssl/http Apache Stronghold httpd 2.4.2 (based on Apache 1.3.6)

Nmap finished: 1 IP address (1 host up) scanned in 35.038 seconds
```

Обратите внимание: сведения о версии одинаковы для обоих открытых портов, но служба на порту 80 - `http`, а на порту 443 - `ssl/http`. Распространённый случай HTTPS на порту 443 не зашит в программе жёстко: Nmap должен уметь обнаруживать SSL на любом порту и определять лежащий в его основе протокол для любой службы, которую Nmap способен обнаружить в открытом виде. Если бы Nmap не обнаружил сервер, прослушивающий за SSL, указанной службой была бы `ssl/unknown`. Если бы Nmap был собран без поддержки SSL, указанной службой была бы просто `ssl`. В обоих случаях столбец версии оказался бы пустым.

Поддержка SSL в Nmap зависит от свободной библиотеки OpenSSL[2]. Она не включается в двоичные RPM-пакеты Linux, чтобы не нарушать работу систем, в которых этих библиотек нет. Дистрибутив исходного кода Nmap пытается обнаружить OpenSSL в системе и при наличии скомпоноваться с ним. Подробности настройки процесса сборки для включения или исключения OpenSSL см. в главе 2 «Получение, компиляция, установка и удаление Nmap».

7.6. Формат файла nmap-service-probes

Как и при удалённом определении ОС (`-O`), Nmap использует плоский файл для хранения проб определения версий и строк совпадений. Хотя поставляемой с Nmap версии `nmap-services` достаточно большинству пользователей, понимание формата файла позволяет продвинутым хакерам Nmap добавлять в механизм обнаружения собственные службы. Как и многие файлы Unix, `nmap-service-probes` ориентирован на строки. Строки, начинающиеся с решётки (#), считаются комментариями и игнорируются синтаксическим анализатором. Пустые строки также игнорируются. Остальные строки должны содержать одну из описанных ниже директив. Некоторые читатели предпочитают взглянуть на примеры из раздела 7.6.9 «Собираем всё вместе», прежде чем браться за следующий подробный разбор.

7.6.1. Директива Exclude

Синтаксис: `Exclude <спецификация портов>`

Примеры:

```
Exclude 53,T:9100,U:30000-40000
```

Эта директива исключает указанные порты из сканирования версий. Её можно применить только один раз, и располагаться она должна возле начала файла, выше всех директив Probe. Директива Exclude использует тот же формат, что и ключ Nmap `-p`, поэтому поддерживаются диапазоны и разделённые запятыми списки портов. В файле `nmap-service-probes`, входящем в состав Nmap, исключены только TCP-порты с 9100 по 9107. Это распространённые порты для принтеров: они прослушивают эти порты и нередко печатают все отправленные им данные. Поэтому сканирование версий может заставить их напечатать множество страниц с пробами, которые отправляет Nmap, например запросами SunRPC, командами помощи и пробами X11.

Такое поведение часто нежелательно, особенно когда сканирование задумано как скрытное. Однако стандартное поведение Nmap, избегающего сканирования этих портов, способно облегчить хитрому пользователю сокрытие службы: достаточно запустить её на исключённом порту, таком как 9100, и вероятность распознавания по имени снизится. Сканирование портов всё равно покажет его открытым. Пользователи могут переопределить директиву Exclude параметром `--allports`. Тогда определение версий опрашивает все открытые порты.

7.6.2. Директива Probe

Синтаксис: `Probe <протокол> <имя_пробы> <отправляемая_строка_пробы>`

Примеры:

```
Probe TCP GetRequest q|GET / HTTP/1.0\r\n\r\n|
Probe UDP DNSStatusRequest q|\0\0\x10\0\0\0\0\0\0\0\0\0|
Probe TCP NULL q||
```

Директива Probe сообщает Nmap, какую строку отправить для распознавания различных служб. Все рассматриваемые далее директивы относятся к последней инструкции Probe. Аргументы таковы:

• `<протокол>` — Это должен быть TCP или UDP. Nmap использует только пробы, соответствующие протоколу службы, которую пытается сканировать.

• `<имя_пробы>` — Это простое английское название пробы. Оно используется в отпечатках служб, чтобы описать, какие пробы вызвали ответы.

• `<строка_пробы>` — Сообщает Nmap, что отправлять. Она должна начинаться с `q`, после которого идёт символ-разделитель, начинающий и заканчивающий строку. Между символами-разделителями находится фактически отправляемая строка. Она форматируется подобно строке C или Perl и допускает следующие стандартные экранированные символы: `\`, `\0`, `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, `\xHH`. Одна строка Probe в `nmap-service-probes` имеет пустую строку пробы, как показано в третьем примере выше. Это TCP NULL-проба, которая лишь слушает начальные баннеры, отправляемые многими службами. Если символ-разделитель (`|` в этих примерах) нужен в строке пробы, следует выбрать другой разделитель.

7.6.3. Директива match

Синтаксис: `match <служба> <шаблон> [<сведения_о_версии>]`

Примеры:

```
match ftp m/^220.*Welcome to PureFTPd (\d\S+)/ p/PureFTPd/ v/$1/
match ssh m/^SSH-([.\d]+)-OpenSSH_(\S+)/ p/OpenSSH/ v/$2/ i/protocol $1/
match mysql m/^.\0\0\0\n(4\.[-.\w]+)\0...\0/s p/MySQL/ i/$1/

match chargen m|@ABCDEFGHIJKLMNOPQRSTUVWXYZ|
match uucp m|^login: Password: Login incorrect\.$| p/SunOS uucpd/ o/SunOS/
match printer m|^([\w-_.]+): lpd: Illegal service request\n$| p/lpd/ h/$1/
match afs m|^[\d\D]{28}\s*(OpenAFS) ([\d\.]{3}[^\s\0]*)\0| p/$1/ v/$2/
```

Директива match сообщает Nmap, как распознавать службы по ответам на строку, отправленную предшествующей директивой Probe. За одной строкой Probe могут следовать десятки или сотни инструкций match. Если заданный шаблон совпадает, необязательный спецификатор версии формирует название приложения, номер версии и дополнительные сведения, которые сообщает Nmap. Аргументы этой директивы таковы:

• `<служба>` — Это просто название службы, с которой совпадает шаблон. Примерами могут служить `ssh`, `smtp`, `http` или `snmp`. В особом случае перед названием службы можно поставить `ssl/`, как в `ssl/vmware-auth`. Тогда служба будет сохранена как `vmware-auth`, туннелированная через SSL. Это полезно для служб, которые можно полностью распознать без накладных расходов на установление SSL-соединения.

• `<шаблон>` — Этот шаблон используется, чтобы определить, соответствует ли полученный ответ службе, заданной предыдущим параметром. Формат подобен Perl, а синтаксис имеет вид `m/[regex]/[opts]`. Буква `m` сообщает Nmap о начале строки совпадения. Косая черта (`/`) - разделитель, который можно заменить почти любым печатаемым символом при условии, что

вторая косая черта заменена тем же символом. Регулярное выражение оформляется в стиле Perl[3]. Это стало возможным благодаря превосходной библиотеке Perl Compatible Regular Expressions (PCRE) (pcre.org). Сейчас поддерживаются только параметры `i`, делающий совпадение независимым от регистра, и `s`, при котором спецификатор `.` охватывает символы новой строки. Как и можно ожидать, семантика этих двух параметров такая же, как в Perl. Захватываемые подвыражения (например, номера версий) заключаются в круглые скобки, как в большинстве приведённых выше примеров.

• `<сведения_о_версии>` — Раздел `<сведения_о_версии>` в действительности содержит шесть необязательных полей. Каждое поле начинается с идентифицирующей буквы (например, `h` для «hostname»). Затем идёт выбранный автором сигнатуры символ-разделитель. Предпочтительный разделитель - косая черта (`/`), если только она не используется в самом поле. Далее идёт значение поля, за которым следует символ-разделитель. В следующей таблице описаны шесть полей.

Таблица 7.1. Форматы и значения полей versioninfo

Формат поля — описание значения

• `p/vendorproductname/` — Включает поставщика и часто название службы; имеет вид «Sun Solaris rexecd», «ISC BIND named» или «Apache httpd».
• `v/version/` — «Номер» версии приложения, который может содержать нечисловые символы и даже несколько слов.
• `i/info/` — Разнородные дополнительные сведения, которые были доступны сразу и могут оказаться полезными. Например, открыт ли X-сервер для соединений без аутентификации или каков номер протокола серверов SSH.
• `h/hostname/` — Имя узла (если оно есть), предлагаемое службой. Это распространено в таких протоколах, как SMTP и POP3, и полезно, поскольку такие имена узлов могут относиться к внутренним сетям или по иной причине отличаться от прямолинейных ответов обратного DNS.
• `o/operatingsystem/` — Операционная система, в которой работает служба. Она вполне обоснованно может отличаться от ОС, о которой сообщает основанное на стеке IP определение ОС Nmap. Например, целевой IP-адрес может принадлежать Linux-машине, использующей преобразование сетевых адресов для пересылки запросов серверу Microsoft IIS в DMZ. В этом случае определение ОС по стеку должно сообщить Linux, а определение службы - Windows на порту 80.
• `d/devicetype/` — Тип устройства, на котором работает служба. Некоторые службы раскрывают эти сведения, а во многих других случаях их можно вывести косвенно. Например, веб-сервер HP-ChaiServer работает только на принтерах.

Любое из шести полей можно опустить. В действительности, если никаких дополнительных сведений о службе нет, можно опустить все поля. Любое поле версии может содержать нумерованные строки, такие как `$1` или `$2`, которые заменяются (подобно Perl) соответствующей заключённой в круглые скобки подстрокой из `<шаблона>`.

В редких случаях перед вставкой к тексту замены можно применить вспомогательную функцию. Вспомогательная функция `$P()` отфильтровывает непечатаемые символы. Это полезно для преобразования строк Unicode в кодировке UTF-16, таких как `W\0O\0R\0K\0G\0R\0O\0U\0P\0`, в приближение ASCII `WORKGROUP`. Её можно использовать в любом поле `versioninfo`, передав номер совпадения, которое требуется сделать печатаемым, например: `i/$P(3)/`.

Другая вспомогательная функция - `$SUBST()`. Она используется для выполнения замен в совпадениях до их печати. Функция принимает три аргумента. Первый - номер подстановки в шаблоне, такой же, как в обычной переменной замены вроде `$1` или `$3`. Второй и третий аргументы задают соответственно подстроку, которую нужно найти, и строку, которой её нужно заменить. Заменяются все найденные в подстроке экземпляры искомой строки, а не только первый. Например, `sshd` VanDyke VShell сообщает номер версии в формате `2_2_3_578`. Мы используем

поле `versioninfo v/$SUBST(1,"_",".")/`, чтобы преобразовать его в более привычный вид `2.2.3.578`.

7.6.4. Директива softmatch

Синтаксис: `softmatch <служба> <шаблон>`

Примеры:

```
softmatch ftp m/^220 [-.\w ]+ftp.*\r\n$/i
softmatch smtp m|^220 [-.\w ]+SMTP.*\r\n|
softmatch pop3 m|^\+OK [-\[\]\(\)!,/+:<>@.\w ]+\r\n$|
```

Директива softmatch по формату подобна рассмотренной выше директиве match. Главное отличие состоит в том, что после мягкого совпадения сканирование продолжается, но ограничивается пробами, которые, как известно, способны совпасть с данной службой. Благодаря этому позднее можно найти обычное («жёсткое») совпадение, которое способно предоставить полезные сведения о версии. Подробнее о том, как это работает, см. в разделе 7.3 «Описание метода». Аргументы здесь не определяются, поскольку они совпадают с аргументами match выше, за исключением того, что аргумента `<versioninfo>` не бывает. Как и в случае с match, в одном разделе Probe может находиться множество инструкций softmatch.

7.6.5. Директивы ports и sslports

Синтаксис: `ports <список_портов>`

Примеры:

```
ports 21,43,110,113,199,505,540,1248,5432,30444
ports 111,4045,32750-32810,38978
```

Эта строка сообщает Nmap, на каких портах обычно встречаются службы, распознаваемые данной пробой. В каждом разделе Probe её следует применять только один раз. Синтаксис представляет собой немного упрощённую версию синтаксиса параметра Nmap `-p`. См. примеры выше. Подробнее о том, как это работает, рассказано в разделе 7.3 «Описание метода».

Синтаксис: `sslports <список_портов>`

Пример:

```
sslports 443
```

Это то же самое, что описанная выше директива `ports`, за исключением того, что эти порты часто используются для обёртывания службы в SSL. Например, проба HTTP объявляет `sslports 443`, а распознающие SMTP пробы содержат строку `sslports 465`, поскольку это стандартные порты соответственно HTTPS и SMTPS. Формат `<список_портов>` такой же, как у ports. Эта необязательная директива не может встречаться в одном разделе Probe больше одного раза.

7.6.6. Директива totalwaitms

Синтаксис: `totalwaitms <миллисекунды>`

Пример:

```
totalwaitms 5000
```

Эта редко необходимая директива задаёт время, в течение которого Nmap должен ждать, прежде чем отказаться от последней определённой пробы Probe для конкретной службы. Обычно стандартное значение Nmap подходит.

7.6.7. Директива rarity

Синтаксис: `rarity <значение от 1 до 9>`

Пример:

```
rarity 6
```

Директива rarity приблизительно соответствует тому, насколько часто от этой пробы можно ожидать полезных результатов. Чем больше число, тем более редкой считается проба и тем меньше вероятность, что её испытают на службе. Подробнее см. в разделе 7.3.2 «Выбор проб и редкость».

7.6.8. Директива fallback

Синтаксис: `fallback <разделённый запятыми список проб>`

Пример:

```
fallback GetRequest,GenericLines
```

Эта необязательная директива задаёт пробы, которые следует использовать как резервные, если в текущем разделе Probe нет совпадений. Подробнее о резервных вариантах см. в разделе 7.3.1 «Уловки и резервные варианты». Для TCP-проб без директивы fallback Nmap сначала испытывает строки match из самой пробы, а затем неявно переходит к резервной NULL-пробе. Если директива fallback присутствует, Nmap сначала испытывает строки match из самой пробы, затем строки из проб, перечисленных в директиве fallback (слева направо). Наконец, Nmap испытывает NULL-пробу. Для UDP поведение такое же, за исключением того, что NULL-проба не испытывается никогда.

7.6.9. Собираем всё вместе

Ниже приведено несколько примеров из `nmap-service-probes`, объединяющих всё сказанное (для экономии места многие строки пропущены). После столь подробного чтения раздела следующий фрагмент должен быть понятен.

```
# The Exclude directive takes a comma separated list of ports.
# The format is exactly the same as the -p switch.
Exclude T:9100-9107

# This is the NULL probe that just compares any banners given to us
##############################NEXT PROBE##############################
Probe TCP NULL q||
# Wait for at least 5 seconds for data.  Otherwise an Nmap default is used.
totalwaitms 5000
# Windows 2003
match ftp m/^220[ -]Microsoft FTP Service\r\n/ p/Microsoft ftpd/
match ftp m/^220 ProFTPD (\d\S+) Server/ p/ProFTPD/ v/$1/
softmatch ftp m/^220 [-.\w ]+ftp.*\r\n$/i
match ident m|^flock\(\) on closed filehandle .*midentd| p/midentd/ i/broken/
match imap m|^\* OK Welcome to Binc IMAP v(\d[-.\w]+)| p/Binc IMAPd/ v/$1/
softmatch imap m/^\* OK [-.\w ]+imap[-.\w ]+\r\n$/i
match lucent-fwadm m|^0001;2$| p/Lucent Secure Management Server/
match meetingmaker m/^\xc1,$/ p/Meeting Maker calendaring/
```

```
# Lopster 1.2.0.1 on Linux 1.1
match napster m|^1$| p/Lopster Napster P2P client/

Probe TCP Help q|help\r\n\r\n|

match chargen m|@ABCDEFGHIJKLMNOPQRSTUVWXYZ|
match echo m|^help\r\n\r\n$|
```

7.7. Вклад сообщества

Какой бы технически совершенной ни была среда определения служб, без всеобъемлющей базы данных служб для сопоставления она была бы почти бесполезна. Именно здесь природа Nmap как проекта с открытым исходным кодом раскрывается во всей красе. Лаборатория Insecure.Org довольно внушительна по меркам компьютерных энтузиастов, но она никогда не сможет запускать больше ничтожной доли всех существующих типов машин и служб. К счастью, опыт с отпечатками определения ОС показал, что все пользователи Nmap вместе запускают всё распространённое оборудование, а вдобавок ошеломляющее разнообразие диковинных устройств. База отпечатков ОС Nmap содержит более тысячи записей, включая всевозможные коммутаторы, точки беспроводного доступа, VoIP-телефоны, игровые приставки, Unix-машины, узлы Windows, принтеры, маршрутизаторы, КПК, межсетевые экраны и так далее. Определение версий также поддерживает материалы пользователей. Пользователи Nmap внесли сведения о тысячах служб. Сообщество Nmap помогает делать эту базу исключительной тремя основными способами: отправляя отпечатки служб, исправления базы данных и новые пробы.

7.7.1. Отправка отпечатков служб

Если служба отвечает на одну или несколько проб Nmap, но Nmap не может её распознать, он печатает отпечаток службы, подобный следующему:

```
SF-Port21-TCP:V=3.40PVT16%D=9/6%Time=3F5A961C%r(NULL,3F,"220\x20stage\x20F
SF:TP\x20server\x20\(Version\x202\.1WU\(1\)\+SCO-2\.6\.1\+-sec\)\x20ready\
SF:.\r\n")%r(GenericLines,81,"220\x20stage\x20FTP\x20server\x20\(Version\x
SF:202\.1WU\(1\)\+SCO-2\.6\.1\+-sec\)\x20ready\.\r\n500\x20'':\x20command\
SF:x20not\x20understood\.\r\n500\x20'':\x20command\x20not\x20understood\.\
SF:r\n");
```

Если вы получили такой отпечаток и уверены, что знаете версию демона, работающего на целевом узле, отправьте отпечаток по URL, который выдаёт Nmap. Весь процесс отправки анонимен (если только вы сами не решите сообщить идентифицирующие сведения) и должен занять не больше пары минут. Если вы хотите оказать особенно большую помощь, снова просканируйте систему с параметром `-d` (так Nmap иногда выдаёт более длинные отпечатки) и вставьте оба отпечатка в поле отпечатка на форме отправки. Иногда люди читают раздел о формате файла и отправляют собственные работающие строки match. Это допустимо, но, пожалуйста, присылайте и отпечатки служб, поскольку имеющиеся сценарии позволяют сравнительно легко интегрировать и проверять их.

Для тех, кому интересно: в приведённом выше отпечатке содержатся номер порта (21), протокол (TCP), версия Nmap (3.40PVT16), дата (6 сентября), время Unix в шестнадцатеричном виде и последовательность ответов на пробы в форме `r({<имя_пробы>},{<длина_ответа>},"{<строка_ответа>}")`.

7.7.2. Отправка исправлений базы данных

Это ещё один простой способ помочь улучшить базу данных. При интеграции отпечатка службы, присланного для «chargen в Windows XP» или «FooBar FTP server 3.9.213», трудно определить, насколько общим является совпадение. Совпадёт ли оно также с chargen в Solaris или FooBar FTP 2.7? Поскольку надёжного способа узнать это нет, используется очень конкретное название в надежде, что люди сообщат, когда совпадение потребуется обобщить. База данных Nmap настолько всеобъемлюща лишь потому, что тысячи пользователей потратили по несколько минут, чтобы отправить новые сведения. Если вы сканируете узел, а отпечаток службы сообщает неверную ОС, номер версии, название приложения или даже тип службы, сообщите нам об этом описанным ниже способом.

Обновитесь до новейшей версии Nmap (необязательно). Многие дистрибутивы Linux и другие операционные системы поставляются с древними версиями Nmap. База определения версий Nmap улучшается почти с каждым выпуском, поэтому проверьте номер своей версии, выполнив `nmap -V`, а затем сравните её с новейшей, доступной на nmap.org. Возможно, наблюдаемая вами проблема уже исправлена. Установка новейшей версии на большинстве платформ занимает лишь несколько минут и полезна независимо от того, сохранился ли сообщаемый вами недостаток определения версий. Но даже если сейчас у вас нет времени на обновление, материалы из старых выпусков всё равно ценны.

Будьте совершенно уверены, что знаете, что работает. Недействительные «исправления» способны испортить базу определения версий. Если вы не уверены, что именно работает на удалённой машине, выясните это перед отправкой.

Создайте отпечаток. Выполните команду `nmap -O -PN -sSV -T4 -d --version-trace -p<порт> <цель>`, где `<порт>` - порт, на котором работает неверно распознанная служба узла `<цель>`. Если служба использует UDP, а не TCP, замените `-sSV` на `-sUV`.

Отправьте нам исправление. Теперь просто отправьте исправление по адресу insecure.org. Спасибо за вклад в сообщество Nmap и помощь в дальнейшем улучшении определения версий!

7.7.3. Отправка новых проб

Предположим, Nmap не смог обнаружить службу. Если он получил хоть какой-нибудь ответ на любую пробу, то должен предоставить отпечаток, который можно отправить описанным выше способом. Но что, если ответа нет и потому отпечаток недоступен? Создайте и отправьте собственную пробу! Такие материалы весьма приветствуются. Процесс описан следующими шагами.

Шаги создания новой пробы определения версий

1. Загрузите новейшую версию Nmap с nmap.org и попробуйте снова. Было бы немного неловко потратить время на разработку новой пробы, а затем узнать, что её уже добавили. Убедитесь, что отпечаток недоступен, поскольку по возможности лучше распознавать службы с помощью существующих проб, чем создавать слишком много новых. Если служба не отвечает ни на одну существующую пробу, иного выбора нет.

2. Выберите хорошую строку пробы для распознавания службы. Идеальная проба должна вызывать ответ у как можно большего числа экземпляров службы, а ответы в идеале должны быть достаточно уникальными, чтобы их можно было различать. Этот шаг проще всего выполнить, если вы очень хорошо понимаете протокол, поэтому стоит прочитать соответствующие RFC и документацию продукта. Один из простых подходов - запустить клиент данной службы и

посмотреть на начальный обмен при установлении связи, перехватывая сетевой трафик с помощью Wireshark или `tcpdump` либо подключившись к прослушивающему Netcat.

3. Выбрав подходящую строку, добавьте в Nmap соответствующую новую строку Probe (см. раздел 7.3 «Описание метода» и раздел 7.6 «Формат файла nmap-service-probes»). Сначала не добавляйте никаких строк match, хотя директива ports, заставляющая испытывать новый тест первым на зарегистрированном порту, вполне допустима. Затем несколько раз просканируйте службу с помощью Nmap. Вы должны получить отпечаток, показывающий ответ службы на новую пробу. Отправьте новую строку пробы и отпечатки (по возможности с разных машин, но даже несколько отпечатков одного демона помогают заметить различия) Фёдору по адресу `<fyodor@insecure.org>`. Скорее всего, после этого проба будет включена в будущие версии Nmap. Полезно также любое описание природы строки вашей пробы. Нестандартные службы, встречающиеся только в вашей сети, лучше просто добавить в собственный `nmap-service-probes`, а не в глобальную версию Nmap.

7.8. РЕШЕНИЕ: поиск всех серверов с небезопасной или нестандартной версией приложения

7.8.1. Проблема

Распространённая задача - просканировать диапазон IP-адресов, чтобы найти все серверы конкретной версии или даже удовлетворяющие определённому свойству. Именно в этом определение версий Nmap особенно сильно.

Одно из самых популярных приложений баз данных - сервер MySQL с открытым исходным кодом. MySQL можно настроить так, чтобы запретить все удалённые входы с недоверенных IP-адресов. Если удалённые входы не требуются, это хорошая практика безопасности. Показательный случай: в 2005 году была обнаружена и опубликована уязвимость MySQL, позволявшая удалённо выполнять код[4]. К счастью, злоумышленник сначала должен был получить возможность войти в систему, что, несомненно, спасло Интернет от ещё одного разрушительного червя. Учитывая подобные проблемы, а также то, что логины и пароли SQL часто можно угадать или обнаружить с помощью атак SQL-инъекций, интуиции и внутренних знаний о сети, удалённые входы следует по возможности запрещать.

Задача сетевого администратора - обнаружить серверы MySQL, без необходимости допускающие входы с недоверенных IP-адресов, и принять соответствующие защитные меры.

> **Примечание.** Это решение предоставил разработчик Nmap Даг Хойт.

7.8.2. Решение

Определение версий Nmap полезно в этой ситуации, поскольку добавляет слово `unauthorized` в строку сведений об обнаружении службы, когда сервер запрещает нашему узлу любой доступ. Если требуется просканировать сеть `10.0.0.0/24`, простая, но эффективная стратегия - выполнить следующую команду из недоверенного источника:

```
nmap -sV -p 3306 -oG 10.0.0-mysqls-032506.gnmap 10.0.0.0/24
```

Затем можно воспользоваться Unix-утилитой grep, чтобы найти IP-адреса, принимающие соединения с нашего IP и по умолчанию не запрещающие входы (ключ -v программы grep задаёт обратный результат и печатает только строки, которые не совпадают с данным шаблоном):

```
grep 'Ports: 3306/open/tcp//mysql' 10.0.0-mysqls-032506.gnmap | grep -v unauthorized
```

Полученный вывод показывает серверы MySQL, разрешающие удалённые входы:

```
Host: 10.0.0.33 (foo.com) Ports: 3306/open/tcp//mysql//MySQL 4.1.11/
Host: 10.0.0.72 (bar.com) Ports: 3306/open/tcp//mysql//MySQL 4.0.24-standard/
Host: 10.0.0.99 () Ports: 3306/open/tcp//mysql//MySQL 4.1.11-Debian_4sarge2/
Host: 10.0.0.154 () Ports: 3306/open/tcp//mysql//MySQL 4.0.25-standard/
Host: 10.0.0.155 () Ports: 3306/open/tcp//mysql//MySQL 4.0.25-standard/
```

7.8.3. Обсуждение

Весь секрет состоит в понимании основ протокола MySQL и умении читать файл nmap-service-probes. Поиск в файле строк Probe и строк совпадения mysql с помощью grep даёт следующий вывод (строки перенесены):

```
$ cat /usr/local/share/nmap/nmap-service-probes | egrep '^(Probe|match mysql)'
Probe TCP NULL q||
match mysql m/^.\0\0\0\xffj\x04.*Host .* is not allowed to connect to this
    MySQL server$/ p/MySQL/ i/unauthorized/
match mysql m|^.\0\0\0\xffj\x04Host hat keine Berechtigung, eine Verbindung
    zu diesem MySQL Server herzustellen\.| p/MySQL/
    i/unauthorized; German/
match mysql m/^.\0\0\0..Al sistema '[-.\w]+' non e consentita la
    connessione a questo server MySQL$/ p/MySQL/
    i/unauthorized; Italian/
match mysql m|^.\0\0\0\xffi?\x04?Host .* is blocked because of many connection
    errors\.| p/MySQL/ i/blocked - too many connection errors/
match mysql m/^.\0\0\0.(3\.[-.\w]+)\0.*\x08\x02\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0$/s
    p/MySQL/ v/$1/
match mysql m/^.\0\0\0\n(3\.[-.\w]+)\0....\0/s p/MySQL/ v/$1/
match mysql m/^.\0\0\0\n(4\.[-.\w]+)\0.../s p/MySQL/ v/$1/
match mysql m|^.\0\0\0\n(5\.[-.\w]+)\0....\0|s p/MySQL/ v/$1/
match mysql m|^.\0\0\0\xffj\x04'[\d.]+' .* MySQL|s p/MySQL/
Probe TCP GenericLines q|\r\n\r\n|
Probe TCP GetRequest q|GET / HTTP/1.0\r\n\r\n|
Probe TCP HTTPOptions q|OPTIONS / HTTP/1.0\r\n\r\n|
```

Мы видим, что строки совпадения mysql предназначены для срабатывания от NULL-пробы, поэтому для определения серверов, разрешающих удалённые входы, нестандартные пробы не нужны (о них см. раздел 7.9 «РЕШЕНИЕ: адаптация определения версий к нестандартным задачам, таким как обнаружение открытых прокси»). Из этих строк совпадения mysql мы узнаём, что для служб MySQL, запрещающих удалённые входы, поле сведений будет содержать слово unauthorized.

Помимо типов служб и номеров версий, во многих случаях определение версий способно собирать полезные сведения о целях сканирования. Файл проб полон подобных жемчужин, способных превратить трудоёмкую работу по изучению протокола, написанию сценария, поиску тестовых серверов и отладке в простую команду Nmap. Вот несколько интересных фактов, которые иногда может раскрыть определение версий:

• Правильно ли настроен pserver CVS.

• Имена пользователей, применяемые популярными клиентами однорангового обмена файлами.

• Принимает ли X-сервер соединения.

• Язык и другие параметры локализации многих служб.

• Разрядность процессора цели.

• Настроенные имена популярных IRC-ботов, таких как eggdrop.

• Разрешена ли публикация на серверах новостей Интернета (NNTP).

База определения версий постоянно растёт и совершенствуется благодаря удивительному сообществу пользователей Nmap и присылаемым ими отпечаткам служб. Это решение - хороший пример того, как исследование возможностей определения служб Nmap способно дать изящные, порой неочевидные решения множества разнообразных задач.

7.9. РЕШЕНИЕ: адаптация определения версий к нестандартным задачам, таким как обнаружение открытых прокси

7.9.1. Проблема

Важная часть защиты любой сети - выявление опасных узлов. Система определения служб Nmap предоставляет гибкий и надёжный способ это сделать. Она помогает выявлять уязвимые версии программ, находить неверно настроенные серверы и многое другое. Но иногда лучший способ определить, действительно ли службы уязвимы, - на самом деле попытаться злоупотребить ими так, как не осмеливается стандартное сканирование версий.

Открытые прокси - серверы, слепо пересылающие запросы от недоверенных узлов к выбранным теми серверам. Их работа внутри сети может быть чрезвычайно опасна по многим причинам, поскольку злоумышленники способны:

• Проводить атаки, которые выглядят исходящими из вашей сети.

• Похищать у вас пропускную способность или другие сетевые службы.

• Выдавать себя за внутреннего клиента, чтобы дальше повышать свои привилегии внутри вашей организации.

Это даёт вескую причину модифицировать определение версий специально для попытки эксплуатации открытых прокси. Вероятно, можно было бы составить карту прокси-портов с помощью обычных строк совпадения прокси Nmap, но лучший и единственный настоящий способ доказать уязвимость приложения - самостоятельно его эксплуатировать.

> **Примечание.** Это решение предоставил разработчик Nmap Даг Хойт.

7.9.2. Решение

Первым делом скопируем файл `nmap-service-probes`, чтобы работать с временной копией:

```
mkdir ~/proxydetect
cp /usr/local/share/nmap/nmap-service-probes ~/proxydetect
```

Теперь нужно временно заставить Nmap использовать наш временный файл:

```
export NMAPDIR=$HOME/proxydetect
```

Теперь в файл требуется добавить пробу и строку совпадения, поэтому откройте любимый редактор и поместите следующий текст в свою копию `nmap-service-probes`. Хорошее место для

него - после всех строк match в NULL-пробе, но непосредственно перед следующей строкой Probe (GenericLines).

```
Probe TCP ProxyProbe q|GET http://insecure.org/ HTTP/1.1\r\nHost: insecure.org\r\n\r\n|
rarity 1
ports 1-65535
totalwaitms 20000
match proxy m|^HTTP/1.[01] 200 OK\r?\n.*TITLE>Insecure.O|s p/Open HTTP Proxy!!/
```

Теперь Nmap действительно попытается запросить загрузку по HTTP с `insecure.org`, считая любой сканируемый порт прокси. В результатах сканирования сетей с открытыми прокси начнёт появляться следующее:

```
PORT   STATE SERVICE VERSION
80/tcp open  proxy   Open HTTP Proxy!!
```

7.9.3. Обсуждение

Расположение нашей пробы, низкое значение rarity и обширный диапазон ports помогают гарантировать, что нестандартная проба будет испытана в самом начале сканирования служб, чтобы другие пробы вроде GetRequest не распознали приложение просто как прокси до того, как у нас появится возможность применить активную пробу.

Мы также использовали директиву totalwaitms, чтобы Nmap дольше ждал истечения времени ожидания этой пробы. Это может быть необходимо, поскольку приходится иметь дело не только с задержкой и ненадёжностью соединения между нами и прокси, но и с задержкой и ненадёжностью соединения между прокси и сервером, содержащим запрошенную страницу (`insecure.org`).

Помните, что кроме HTTP через прокси можно передавать множество других протоколов. Определение версий распознаёт прокси для многих из них, включая FTP, POP3, IMAP и SMTP. Для прокси SOCKS существуют специальные строки match, определяющие сведения о настроенных на прокси вариантах аутентификации. Как и в этом решении, с помощью нестандартных файлов проб часто можно применить определение версий, чтобы установить, открыты ли такие прокси. Однако более сложные проверки, вероятно, лучше выполнять сценариями NSE.

Сноски

[1] [назад] http://www.stunnel.org/
[2] [назад] http://www.openssl.org
[3] [назад] http://www.perl.com/doc/manual/html/pod/perlre.html
[4] [назад] http://www.securityfocus.com/bid/12781

Глава 8. Удалённое определение ОС

8.1. Введение

Исследуя сеть для аудита безопасности или инвентаризации и администрирования, обычно хочется знать больше, чем просто IP-адреса обнаруженных машин. Реакция на обнаружение принтера может сильно отличаться от реакции на маршрутизатор, точку беспроводного доступа, телефонную АТС, игровую приставку, настольную систему Windows или сервер Unix. Более детальное распознавание (например, различение Mac OS X 10.4 и 10.3) полезно для определения подверженности конкретным изъянам и адаптации эффективных эксплойтов к этим уязвимостям.

Отчасти из-за ценности таких сведений для злоумышленников многие системы неохотно раскрывают свою точную природу и конфигурацию операционной системы. К счастью, Nmap содержит огромную базу эвристических правил для распознавания тысяч различных систем по их ответам на набор проб TCP/IP. Другая система (часть определения версий) опрашивает открытые TCP- или UDP-порты, чтобы определить тип устройства и сведения об ОС. Результаты этих двух систем сообщаются независимо, благодаря чему можно распознавать такие сочетания, как межсетевой экран Checkpoint, перенаправляющий порт 80 серверу Windows IIS.

Хотя Nmap поддерживает определение ОС с 1998 года, в этой главе описывается система второго поколения, выпущенная в 2006 году.

8.1.1. Причины определять ОС

Одни преимущества обнаружения лежащих в основе ОС и типов устройств сети очевидны, другие менее заметны. В этом разделе перечислены главные причины, которые мне доводилось слышать в пользу обнаружения этих дополнительных сведений.

Определение уязвимости целевых узлов

Иногда очень трудно удалённо определить, подвержена ли доступная служба определённой уязвимости или уже исправлена. Даже получение номера версии приложения помогает не всегда, поскольку поставщики ОС часто переносят исправления безопасности в старые версии, не меняя номера версии. Самый надёжный способ подтвердить реальность уязвимости - эксплуатировать её, но при этом есть риск аварийно завершить службу и потратить часы или даже дни на мучительные попытки эксплуатации, если служба окажется исправленной.

Определение ОС помогает сократить число таких ложноположительных результатов. Например, демон Rwho в неисправленных версиях Sun Solaris с 7 по 9 может допускать удалённую эксплуатацию (предупреждение Sun № 57659). Удалённо определить уязвимость трудно, но её можно исключить, установив, что целевая система работает под управлением Solaris 10.

Если взглянуть на это с позиции системного администратора, а не специалиста по тестированию на проникновение, представьте, что вы управляете большим хозяйством Sun в момент выхода предупреждения № 57659. Просканируйте всю сеть с определением ОС, чтобы найти машины, которые нужно исправить раньше, чем до них доберутся злоумышленники.

Адаптация эксплойтов

Даже после обнаружения уязвимости в целевой системе определение ОС может помочь при её эксплуатации. Переполнения буфера, эксплойты строк форматирования и многие другие уязвимости часто требуют специально адаптированного шелл-кода со смещениями и полезной нагрузкой на языке ассемблера, сформированными в соответствии с целевой ОС и аппаратной архитектурой. В некоторых случаях у вас есть лишь одна попытка, поскольку при неверном шелл-коде служба аварийно завершает работу. Сначала примените определение ОС, иначе можете в итоге отправить шелл-код Linux серверу FreeBSD.

Инвентаризация сети и поддержка

Хотя это и не так увлекательно, как получение прав root с помощью специально созданного эксплойта строки форматирования, отслеживать работающие в сети системы необходимо по многим административным причинам. Прежде чем продлевать договор поддержки IRIX ещё на год, просканируйте сеть и проверьте, пользуется ли кто-нибудь такими машинами. Инвентаризация также может пригодиться при составлении ИТ-бюджета и для учёта всего оборудования компании.

Обнаружение несанкционированных и опасных устройств

Из-за повсеместного распространения мобильных устройств и дешёвого массового сетевого оборудования компании всё чаще обнаруживают, что сотрудники расширяют сети нежелательными способами. Сотрудник может установить в своей кабинке беспроводную точку доступа (WAP) за 20 долларов, не понимая (или не заботясь о том), что только что открыл защищённую корпоративную сеть потенциальным злоумышленникам на парковке или в соседних зданиях. WAP могут быть настолько опасными, что для их обнаружения в Nmap есть специальная категория, как показано в разделе 8.8 «РЕШЕНИЕ: обнаружение посторонних беспроводных точек доступа в корпоративной сети». Пользователи также способны причинить системным администраторам немало хлопот, подключая к корпоративной сети небезопасные и/или заражённые червями ноутбуки. Регулярное сканирование позволяет обнаруживать несанкционированные устройства для расследования и изоляции.

Социальная инженерия

Ещё одно возможное применение - социальная инженерия. Допустим, вы сканируете целевую компанию, и Nmap сообщает «Datavoice TxPORT PRISM 3000 T1 CSU/DSU 6.22/2.06». Можно позвонить цели, выдать себя за службу поддержки Datavoice и обсудить какие-нибудь проблемы с их PRISM 3000. Скажите, что вскоре собираетесь объявить о серьёзной дыре в безопасности, но сначала предоставляете исправление ценным клиентам. Некоторые наивные администраторы могут предположить, что так много об их CSU/DSU способен знать лишь уполномоченный инженер Datavoice. Разумеется, отправленное им исправление - троянский конь, предоставляющий вам удалённый доступ для перехвата трафика и блуждания по их сети. Прежде чем пытаться это проделать, обязательно прочитайте оставшуюся часть главы с советами по точности обнаружения и проверке. Если вы неверно угадаете целевую систему и они вызовут полицию, у вас появится неловкая история для сокамерников.

8.2. Применение и примеры

Внутреннее устройство определения ОС довольно сложно, но пользоваться этой возможностью проще простого. Просто добавьте `-O` к параметрам сканирования. Можно также повысить подробность с помощью `-v`, чтобы получить ещё больше сведений об ОС. Это показано в примере 8.1.

Пример 8.1. Определение ОС с повышенной подробностью (`-O -v`)

```
# nmap -O -v scanme.nmap.org

Starting Nmap ( http://nmap.org
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE  SERVICE
22/tcp  open   ssh
25/tcp  closed smtp
53/tcp  open   domain
70/tcp  closed gopher
80/tcp  open   http
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.20-1 (Fedora Core 5)
Uptime guess: 11.433 days (since Thu Sep 18 13:13:01 2008)
TCP Sequence Prediction: Difficulty=204 (Good luck!)
IP ID Sequence Generation: All zeros

Nmap done: 1 IP address (1 host up) scanned in 6.21 seconds
Raw packets sent: 2021 (90.526KB) | Rcvd: 23 (1326B)
```

Параметры `-v` заставили Nmap сформировать следующие шесть дополнительных строк:

Device type

Все отпечатки классифицируются по одному или нескольким высокоуровневым типам устройств, таким как `router`, `printer`, `firewall` или, как в этом случае, `general purpose`. Они подробнее описаны в разделе «Классификация устройств и ОС (строки Class)». Может быть показано несколько типов устройств; тогда они разделяются символом вертикальной черты, как в `Device Type: router|firewall`.

Running

Это поле также связано со схемой классификации ОС, описанной в разделе «Классификация устройств и ОС (строки Class)». Оно показывает семейство ОС (`Linux` в данном случае) и поколение ОС (`2.6.X`), если оно доступно. Несколько семейств ОС разделяются запятыми. Когда Nmap не может сузить поколения ОС до одного конкретного варианта, они разделяются символом вертикальной черты (`|`). Примеры: `OpenBSD 3.X`, `NetBSD 3.X|4.X` и `Linux 2.4.X|2.5.X|2.6.X`.

Если Nmap находит слишком много семейств ОС, чтобы напечатать их кратко, он опускает эту строку. При отсутствии идеальных совпадений Nmap изменяет поле на `Running (JUST GUESSING)` и добавляет в круглых скобках после названия каждого семейства-кандидата процент точности (100% означает идеальное совпадение). Если ни один отпечаток не даёт близкого совпадения, строка опускается.

OS details

Эта строка даёт подробное описание каждого совпавшего отпечатка. Хотя строки Device type и Running берутся из заранее заданных перечислимых списков, которые легко анализировать компьютеру, строка OS details содержит данные свободной формы, полезные человеку, читающему отчёт. Они могут включать более точные номера версий, модели устройств и архитектуры, относящиеся к конкретному отпечатку. В этом примере единственным совпавшим отпечатком был `Linux 2.6.20-1 (Fedora Core 5)`. Несколько точных совпадений разделяются запятыми. Если идеальных совпадений нет, но есть несколько близких предположений, поле переименовывается в Aggressive OS guesses, а после отпечатков в круглых скобках указывается процент, показывающий близость каждого совпадения.

Uptime guess

В ходе определения ОС Nmap получает подряд несколько TCP-пакетов SYN/ACK и проверяет их заголовки на наличие параметра временной метки. Многие операционные системы используют для него простой счётчик, начинающийся с нуля при загрузке и затем увеличивающийся с постоянной частотой, например дважды в секунду. Рассматривая несколько ответов, Nmap может определить текущие значения и скорость увеличения. Простая линейная экстраполяция определяет время загрузки. Алгоритм временных меток также используется для определения ОС (см. раздел «Алгоритм параметра временной метки TCP (TS)»), поскольку частота увеличения в разных системах меняется от 2 до 1000 Гц.

Оценка времени работы называется «оценкой», поскольку различные факторы способны сделать её совершенно неточной. Некоторые операционные системы не запускают счётчик временных меток с нуля, а инициализируют случайным значением, из-за чего экстраполяция к нулю теряет смысл. Даже в системах с простым счётчиком, начинающимся с нуля, он в конце концов переполняется и возвращается к началу. При частоте увеличения 1000 Гц счётчик сбрасывается в ноль примерно каждые 50 дней. Поэтому узел, работающий 102 дня, будет выглядеть работающим лишь два дня. Несмотря на эти оговорки, для большинства операционных систем оценка времени работы часто точна, поэтому при наличии она печатается, но только в подробном режиме. Оценка опускается, если цель выдаёт нули или не содержит параметров временной метки в пакетах SYN/ACK либо вообще не отвечает. Строка также опускается, если Nmap не может определить частоту увеличения временной метки или она выглядит подозрительно (например, время работы 30 лет).

Network Distance

Побочный эффект одной из проверок определения ОС позволяет Nmap вычислить, сколько маршрутизаторов находится между ним и целевым узлом. При сканировании localhost расстояние равно нулю, а для машины в том же сегменте сети - единице. Каждый дополнительный маршрутизатор на пути добавляет к числу переходов единицу. Строка Network Distance в этом примере не печатается, поскольку Nmap опускает её, когда расстояние нельзя вычислить (нет ответа на соответствующую пробу).

TCP Sequence Prediction

Системы с плохой генерацией начальных порядковых номеров TCP уязвимы для слепых атак с подменой TCP. Иными словами, можно установить с такой системой полное соединение и отправлять (но не получать) данные, подменив IP-адрес. Журналы цели покажут подменённый IP, а вы сможете воспользоваться любыми доверительными отношениями между ними. Эта атака была чрезвычайно популярна в середине девяностых, когда люди часто применяли `rlogin`, разрешая вход в учётную запись без пароля с доверенных IP-адресов. Утверждается, что Кевин Митник использовал эту атаку для взлома компьютеров Цутому Симомуры в декабре 1994 года.

Хорошая новость в том, что теперь почти никто не использует `rlogin`, а многие операционные системы исправлены и применяют непредсказуемые начальные порядковые номера, как предложено в RFC 1948. По этим причинам эта строка печатается только в подробном режиме. К сожалению, многие поставщики по-прежнему выпускают уязвимые операционные системы и устройства[1]. Даже исправленные системы часто различаются реализацией, что сохраняет их ценность для определения ОС. Класс описывает применяемый целью алгоритм генерации ISN, а сложность представляет собой приблизительную оценку того, насколько система затрудняет слепую подмену IP (0 - проще всего). Комментарий в круглых скобках зависит от индекса сложности и изменяется от `Trivial joke` через `Easy`, `Medium`, `Formidable`, `Worthy challenge` до, наконец, `Good luck!`. Дополнительные сведения о проверках последовательностей приведены в разделе «Наибольший общий делитель ISN TCP (GCD)».

Хотя семейство `rlogin` по большей части стало пережитком прошлого, изобретательные злоумышленники всё ещё могут найти эффективные применения слепой подмене TCP. Например, она позволяет подделывать HTTP-запросы. Результатов вы не видите, но один лишь URL (запрос POST или GET) способен вызвать серьёзные побочные эффекты. Подмена позволяет злоумышленникам скрывать свою личность, подставлять других или обходить ограничения по IP-адресам.

IP ID sequence generation

Многие системы невольно раскрывают чувствительные сведения об объёме своего трафика тем, как они формируют скромное 16-битное поле ID в IP-пакетах. Этим можно злоупотребить для поддельного сканирования портов других систем и ради иных озорных целей, обсуждаемых в разделе 5.10 «Сканирование TCP вхолостую (`-sI`)». Это поле описывает алгоритм генерации ID, который удалось определить Nmap. Дополнительные сведения о классификации алгоритмов приведены в разделе «Алгоритм генерации последовательности ID IP (TI)».

Обратите внимание: многие системы используют отдельное пространство IP ID для каждого узла, с которым обмениваются данными. В таком случае они могут выглядеть уязвимыми (например, показывать класс `Incremental`), оставаясь защищёнными от таких атак, как сканирование вхолостую. По этой причине, а также потому, что проблема редко бывает критической, строка генерации последовательности IP ID печатается только в подробном режиме. Если при определении ОС Nmap не получает достаточно ответов, он опускает всю строку. Лучший способ проверить, уязвим ли узел и может ли служить зомби для сканирования вхолостую, - испытать его с `-sI`.

Хотя снятие отпечатков TCP - мощный метод определения ОС, опрос открытых портов в поисках подсказок представляет собой другой эффективный подход. Некоторые приложения, такие как Microsoft IIS, работают только на одной платформе (тем самым выдавая её), а многие другие приложения раскрывают платформу в чрезмерно подробных баннерах. Параметр `-sV` включает определение версий Nmap, обученное искать эти (и другие) подсказки. В примере 8.2 Nmap извлекает сведения о платформе из FTP-сервера.

Пример 8.2. Применение сканирования версий для определения ОС

```
# nmap -sV -O -v 129.128.X.XX
Starting Nmap ( http://nmap.org )
Interesting ports on [hostname] (129.128.X.XX):
Not shown: 994 closed ports
PORT      STATE    SERVICE      VERSION
21/tcp    open     ftp          HP-UX 10.x ftpd 4.1
22/tcp    open     ssh          OpenSSH 3.7.1p1 (protocol 1.99)
111/tcp   open     rpc
445/tcp   filtered microsoft-ds
1526/tcp  open     oracle-tns   Oracle TNS Listener
32775/tcp open     rpc
No exact OS matches for host
TCP Sequence Prediction: Class=truly random
                         Difficulty=9999999 (Good luck!)
IP ID Sequence Generation: Incremental
Service Info: OS: HP-UX
```

В этом примере строка `No exact OS matches for host` означает, что снятие отпечатков TCP/IP не нашло точного совпадения. К счастью, расположенное несколькими строками ниже поле Service Info раскрывает, что ОС - HP-UX. Если обнаружено несколько операционных систем (что возможно у шлюзов NAT, перенаправляющих порты нескольким разным машинам), поле называется OSs, а значения разделяются запятыми. Строка Service Info также может содержать имена узлов и типы устройств, найденные при сканировании версий. Однако основное внимание в этой главе уделяется снятию отпечатков TCP/IP, поскольку определение версий рассмотрено в главе 7 «Определение версий служб и приложений».

Какой из двух доступных эффективных методов определения ОС следует использовать? Обычно лучший ответ - оба. В некоторых случаях, например когда прокси-межсетевой экран перенаправляет данные приложению на другом узле, ответы могут обоснованно различаться. Снятие отпечатков TCP/IP распознает прокси, тогда как сканирование версий обычно обнаружит сервер, на котором работает проксируемое приложение. Даже без проксирования и перенаправления портов полезно применять оба метода. Совпадение результатов повышает их достоверность. Если результаты резко различаются, прежде чем полагаться на любой из них, проведите дополнительное расследование и выясните происходящее. Поскольку определение ОС и версий прекрасно сочетаются, параметр `-A` включает обе возможности.

Определение ОС намного эффективнее, если найден хотя бы один открытый и один закрытый TCP-порт. Задайте параметр `--osscan-limit`, и Nmap даже не станет пытаться определить ОС узлов, не отвечающих этому критерию. Это способно существенно сэкономить время, особенно при сканировании множества узлов с `-PN`. Чтобы параметр возымел действие, всё равно нужно включить определение ОС с помощью `-O` (или `-A`).

Другой параметр определения ОС - `--osscan-guess`. Когда Nmap не может найти идеальное совпадение ОС, иногда он предлагает как возможные варианты близкие совпадения. По умолчанию для этого совпадение должно быть очень близким. Если указать этот параметр (или эквивалентный `--fuzzy`), Nmap будет выдвигать более смелые предположения. Nmap по-прежнему сообщает о печати несовершенного совпадения и показывает уровень уверенности (в процентах) для каждого предположения.

Когда Nmap выполняет определение ОС цели и не находит идеального совпадения, он обычно повторяет попытку. По умолчанию Nmap делает пять попыток, если условия благоприятны для отправки отпечатка ОС, и две, если условия не так хороши. Параметр `--max-os-tries` позволяет изменить это максимальное число попыток определения ОС. Уменьшение значения (обычно до 1) ускоряет Nmap, хотя вы лишаетесь повторных попыток, которые потенциально могли бы распознать ОС. И наоборот, можно задать большое значение, чтобы разрешить ещё больше попыток при благоприятных условиях. Так поступают редко, кроме случаев создания более качественных отпечатков для отправки и включения в базу ОС Nmap.

Как и почти во всех остальных частях Nmap, результаты в конечном счёте поступают от самой целевой машины. Хотя это случается редко, системы иногда настраивают так, чтобы запутывать Nmap или вводить его в заблуждение. Несколько программ даже были разработаны специально для обмана определения ОС Nmap (см. раздел 11.5.4 «Подмена ОС»). Лучше всего исследовать сеть множеством методов разведки и не доверять ни одному из них безоговорочно.

Для снятия отпечатков TCP/IP требуется собирать подробные сведения о стеке IP цели. Наиболее полезные результаты, такие как сведения TTL, печатаются в выводе Nmap при каждом их получении. Менее существенные сведения, такие как генерация последовательности IP ID и сложность предсказания последовательности TCP, печатаются только в подробном режиме. Но если нужны все собранные Nmap подробности стека IP, их можно найти в компактной форме, называемой отпечатком субъекта. Иногда Nmap печатает его (для отправки пользователем), когда не распознаёт узел. Можно также принудительно заставить Nmap печатать его (в обычном, интерактивном и XML-форматах), включив отладку с помощью `-d`. Затем прочитайте раздел 8.5 «Понимание отпечатка Nmap», чтобы его интерпретировать.

8.3. Поддерживаемые Nmap методы снятия отпечатков TCP/IP

Для снятия отпечатка ОС Nmap отправляет до 15 проб TCP, UDP и ICMP на известные открытые и закрытые порты целевой машины. Эти пробы специально разработаны для эксплуатации различных неоднозначностей в стандартных RFC протоколов. Затем Nmap слушает ответы. Десятки атрибутов этих ответов анализируются и объединяются для создания отпечатка. Каждый пакет-проба отслеживается и при отсутствии ответа повторно отправляется как минимум один раз. Все пакеты используют IPv4 со случайным значением IP ID. Пробы открытого TCP-порта пропускаются, если такой порт не найден. Для закрытых портов TCP или UDP Nmap сначала проверяет, был ли найден такой порт. Если нет, Nmap просто выбирает случайный порт и надеется на лучшее.

Следующие разделы очень технические и раскрывают скрытую работу определения ОС Nmap. Nmap можно эффективно использовать и без их понимания, хотя материал поможет лучше разбираться в удалённых сетях, а также обнаруживать и объяснять некоторые аномалии. Кроме того, отдельные методы довольно интересны. Спешащие читатели могут перейти к разделу 8.7 «Работа с неверно распознанными и нераспознанными узлами». Но тех, кто готов отправиться в путешествие по явному уведомлению о перегрузке TCP, зарезервированным битам заголовка UDP, начальным порядковым номерам, фиктивным флагам и пакетам «рождественская ёлка», приглашаю читать дальше!

Даже лучшие из нас иногда забывают смещения байтов для полей и флагов заголовков пакетов. Для быстрой справки схемы заголовков IPv4, TCP, UDP и ICMP приведены в разделе 7 «Справочник TCP/IP» [xxvi]. Схемы пакетов ICMP echo request и destination unreachable показаны на рисунках 8.1 и 8.2.

Рисунок 8.1. Схема заголовка запроса или ответа ICMP echo

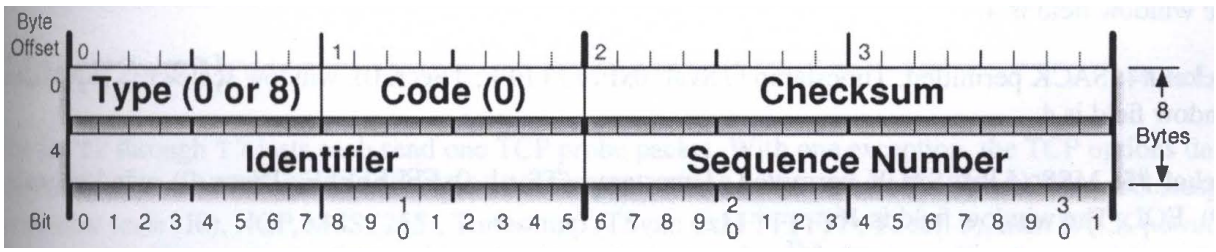
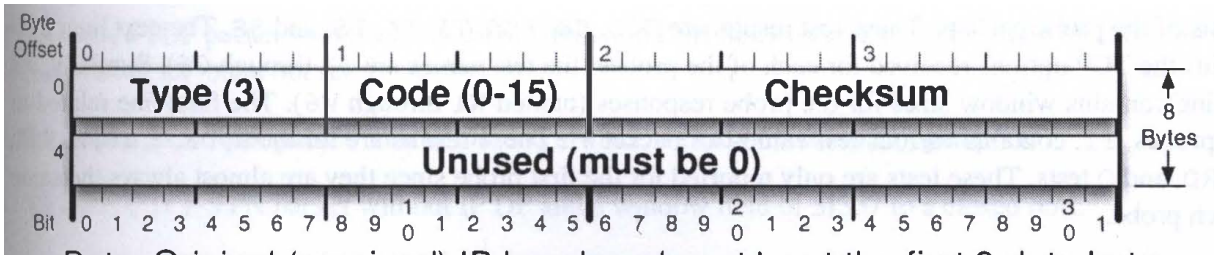


Рисунок 8.2. Схема заголовка ICMP destination unreachable



8.3.1. Отправляемые пробы

В этом разделе описана каждая IP-проба, которую Nmap отправляет при снятии отпечатков TCP/IP. Здесь упоминаются проверки ответов Nmap и параметры TCP, объясняемые в следующем разделе.

Генерация последовательностей (SEQ, OPS, WIN и T1)

Для создания этих четырёх строк результатов проверки отправляется серия из шести TCP-проб. Пробы посылаются с интервалом ровно 110 миллисекунд, поэтому общее время составляет 550 мс. Точное время важно, поскольку некоторые распознаваемые нами алгоритмы последовательностей (начальные порядковые номера, IP ID и временные метки TCP) зависят от времени. Эта величина интервала выбрана так, чтобы работа занимала больше 500 мс и мы могли надёжно распознавать распространённые последовательности временных меток TCP с частотой 2 Гц.

Каждая проба представляет собой TCP-пакет SYN, отправляемый на обнаруженный открытый порт удалённой машины. Порядковые номера и номера подтверждения случайны (но сохраняются, чтобы Nmap мог различать ответы). Для точности распознавания пробы должны быть единообразны, поэтому полезная нагрузка данных отсутствует, даже если пользователь запросил её параметром `--data-length`.

Эти пакеты различаются используемыми параметрами TCP и значением поля окна TCP. В следующем списке приведены параметры и значения всех шести пакетов. Указанные значения поля окна не учитывают масштабирование окна. EOL - параметр конца списка параметров, который многие инструменты перехвата трафика по умолчанию не показывают.

• **Пакет № 1:** масштаб окна (10), NOP, MSS (1460), временная метка (TSval: `0xFFFFFFFF`; TSecr: 0), SACK разрешён. Поле окна равно 1.

• **Пакет № 2:** MSS (1400), масштаб окна (0), SACK разрешён, временная метка (TSval: `0xFFFFFFFF`; TSecr: 0), EOL. Поле окна равно 63.

• **Пакет № 3:** временная метка (TSval: `0xFFFFFFFF`; TSecr: 0), NOP, NOP, масштаб окна (5), NOP, MSS (640). Поле окна равно 4.

• **Пакет № 4:** SACK разрешён, временная метка (TSval: `0xFFFFFFFF`; TSecr: 0), масштаб окна (10), EOL. Поле окна равно 4.

• **Пакет № 5:** MSS (536), SACK разрешён, временная метка (TSval: `0xFFFFFFFF`; TSecr: 0), масштаб окна (10), EOL. Поле окна равно 16.

• **Пакет № 6:** MSS (265), SACK разрешён, временная метка (TSval: `0xFFFFFFFF`; TSecr: 0). Поле окна равно 512.

Результаты этих проверок включают четыре строки категорий результатов. Первая, SEQ, содержит результаты анализа последовательностей пакетов-проб. Эти результаты проверок называются GCD, SP, ISR, TI, II, TS и SS. Следующая строка, OPS, содержит параметры TCP, полученные для каждой пробы (проверки называются от O1 до O6). Аналогично, строка WIN содержит размеры окна в ответах на пробы (от W1 до W6). Последняя относящаяся к этим пробам строка T1 содержит различные значения проверок для пакета № 1. Это результаты проверок R, DF, T, TG, W, S, A, F, O, RD и Q. Они сообщаются только для первой пробы, поскольку почти всегда одинаковы для каждой пробы.

ICMP echo (IE)

Проверка IE предполагает отправку цели двух пакетов ICMP echo request. В первом установлен бит IP DF, байт типа обслуживания (TOS) равен нулю, код равен девяти (хотя должен быть нулевым), порядковый номер - 295, IP ID и идентификатор запроса ICMP случайны, а полезная нагрузка данных состоит из случайного символа, повторённого 120 раз.

Второй ping-запрос похож, за исключением того, что используется TOS 4 (`IP_TOS_RELIABILITY`), код равен нулю, отправляется 150 байт данных, а IP ID, идентификатор запроса и порядковый номер увеличены на единицу относительно значений предыдущего запроса.

Результаты обеих проб объединяются в строку IE, содержащую проверки R, DFI, T, TG, TOSI, CD, SI и DLI. Значение R истинно (Y), только если обе пробы вызвали ответы. Значения T и CD относятся к ответу лишь на первую пробу, поскольку они почти наверняка не будут различаться. DFI, TOSI, SI и DLI - специальные проверки для этого особого случая с двумя пробами ICMP.

Эти ICMP-пробы следуют непосредственно за TCP-пробами последовательностей, чтобы обеспечить достоверные результаты общей проверки последовательности номеров IP ID (см. раздел «Логическое значение общей последовательности IP ID (SS)»).

Явное уведомление TCP о перегрузке (ECN)

Эта проба проверяет поддержку явного уведомления о перегрузке (ECN) в целевом стеке TCP. ECN - метод повышения производительности Интернета, позволяющий маршрутизаторам сигнализировать о проблемах перегрузки до того, как им придётся начать отбрасывать пакеты. Он описан в RFC 3168. Nmap проверяет это, отправляя пакет SYN, в котором также установлены флаги управления перегрузкой ECN CWR и ECE. Для не связанной с ECN проверки используется значение поля срочности `0xF7F5`, хотя флаг срочности не установлен. Номер подтверждения равен нулю, порядковый номер случаен, поле размера окна равно трём, а зарезервированный бит, непосредственно предшествующий биту CWR, установлен. Параметры TCP: масштаб окна (10), NOP, MSS (1460), SACK разрешён, NOP, NOP. Проба отправляется на открытый порт.

Если получен ответ, выполняются и записываются проверки R, DF, T, TG, W, O, CC и Q.

TCP (T2-T7)

Каждая из шести проверок с T2 по T7 отправляет один пакет-пробу TCP. За одним исключением данные параметров TCP во всех случаях (в шестнадцатеричном виде) равны `03030A0102040109080AFFFFFFFF000000000402`. Эти 20 байт соответствуют масштабу окна (10), NOP, MSS (265), временной метке (TSval: `0xFFFFFFFF`; TSecr: 0), а затем разрешению SACK. Исключение состоит в том, что T7 использует значение масштаба окна 15 вместо 10.

Изменяющиеся характеристики каждой пробы описаны ниже:

• T2 отправляет на открытый порт пустой TCP-пакет (ни один флаг не установлен) с установленным битом IP DF и полем окна 128.

• T3 отправляет на открытый порт TCP-пакет с установленными флагами SYN, FIN, URG и PSH и полем окна 256. Бит IP DF не установлен.

• T4 отправляет на открытый порт TCP-пакет ACK с IP DF и полем окна 1024.

• T5 отправляет на закрытый порт TCP-пакет SYN без IP DF и с полем окна 31337.

• T6 отправляет на закрытый порт TCP-пакет ACK с IP DF и полем окна 32768.

• T7 отправляет на закрытый порт TCP-пакет с установленными флагами FIN, PSH и URG и полем окна 65535. Бит IP DF не установлен.

В каждом из этих случаев к отпечатку добавляется строка с результатами проверок R, DF, T, TG, W, S, A, F, O, RD и Q.

Эта проба представляет собой UDP-пакет, отправляемый на закрытый порт. В поле данных символ `C` (`0x43`) повторяется 300 раз. Для операционных систем, позволяющих нам задавать это значение, IP ID устанавливается в `0x1042`. Если порт действительно закрыт и межсетевой экран отсутствует, Nmap ожидает получить сообщение ICMP о недоступности порта в ответ. Затем этот ответ подвергается проверкам R, DF, T, TG, TOS, IPL, UN, RIPL, RID, RIPCK, RUCK, RUL и RUD.

8.3.2. Проверки ответов

В предыдущем разделе описаны пробы, отправляемые Nmap, а этот завершает картину описанием шквала проверок, выполняемых над ответами. Короткие названия (такие как DF, R и RIPCK) используются в базе отпечатков `nmap-os-db` для экономии места. Если не указано иное, все числовые значения проверок приводятся в шестнадцатеричной записи без ведущих нулей. Проверки документированы примерно в том порядке, в котором они появляются в отпечатках.

Наибольший общий делитель TCP ISN (GCD)

Проверка SEQ отправляет шесть TCP-пакетов SYN на открытый порт целевой машины и получает в ответ пакеты SYN/ACK. Каждый из этих пакетов SYN/ACK содержит 32-битный начальный порядковый номер (ISN). Эта проверка пытается определить наименьшее число, на которое целевой узел увеличивает эти значения. Например, многие узлы (особенно старые) всегда увеличивают ISN на числа, кратные 64 000.

Первый шаг вычисления - создание массива разностей между ответами на пробы. Первый элемент - разность между ISN ответов на первую и вторую пробы. Второй элемент - разность между вторым и третьим ответами. Если Nmap получает ответы на все шесть проб, элементов будет пять. Поскольку в следующих двух разделах этот массив упоминается неоднократно, назовём его `diff1`. Если ISN меньше предыдущего, Nmap рассматривает как число значений, которое пришлось бы вычесть из первого значения, чтобы получить второе, так и число значений, на которое пришлось бы отсчитать вперёд (включая переход 32-битного счётчика через ноль). В `diff1` сохраняется меньшее из этих двух значений. Поэтому разность между `0x20000`, за которым следует `0x15000`, равна `0xB000`. Разность между `0xFFFFFF00` и `0xC000` равна `0xC0FF`. Затем в качестве значения этой проверки записывается наибольший общий делитель всех этих элементов. Этот GCD также используется для вычисления результата SP.

Скорость счётчика TCP ISN (ISR)

Это значение сообщает среднюю скорость увеличения возвращаемого начального порядкового номера TCP. Напомним, что разность вычисляется между каждой парой последовательных ответов на пробы и сохраняется в рассмотренном ранее массиве `diff1`. Каждая из этих разностей делится на время (в секундах, обычно около 0,1), прошедшее между отправкой двух породивших их проб. В результате получается массив, который мы назовём `seq_rates`, содержащий скорости увеличения счётчика ISN в секунду. Для каждого значения `diff1` в массиве есть один элемент. Затем вычисляется среднее значений массива. Если это среднее меньше единицы (например, используется постоянный ISN), ISR равен нулю. В противном случае ISR равен восьмикратному двоичному логарифму (логарифму по основанию 2) этого среднего значения, округлённому до ближайшего целого.

Индекс предсказуемости последовательности TCP ISN (SP)

В то время как проверка ISR измеряет среднюю скорость увеличения начальных порядковых номеров, это значение измеряет изменчивость ISN. Оно приблизительно оценивает, насколько трудно было бы предсказать следующий ISN по известной последовательности из шести ответов на пробы. Для вычисления используются рассмотренные в предыдущем разделе массив разностей (`seq_rates`) и значения GCD.

Эта проверка выполняется, только если получено как минимум четыре ответа. Если вычисленное ранее значение GCD больше девяти, элементы вычисленного ранее массива `seq_rates` делятся на это значение.

Для меньших значений GCD деление не выполняется, поскольку обычно они возникают случайно. Затем вычисляется стандартное отклонение массива полученных значений. Если результат равен единице или меньше, SP равен нулю. В противном случае вычисляется двоичный логарифм результата, затем он умножается на восемь, округляется до ближайшего целого и сохраняется как SP.

Помните, что эта проверка выполняется только в целях определения ОС и не является полномасштабным аудитом генератора ISN цели. Существует множество слабостей алгоритмов, которые делают предсказание лёгким даже при высоком значении SP.

Алгоритм генерации последовательности TCP IP ID (TI)

Эта проверка исследует поле ID заголовка IP каждого ответа на TCP-пробы SEQ. Проверка включается, только если получены ответы как минимум на три пробы. Затем она классифицирует генератор IP ID цели по приведённому ниже алгоритму. Обратите внимание: значения разностей предполагают возможность перехода счётчика через границу диапазона. Поэтому разность между IP ID 65 100 и следующим значением 700 равна 1136. Разность между 2000 и следующим значением 1100 равна 64 636. Вот подробности вычисления:

1. Если все номера ID равны нулю, TI получает значение Z.

2. Если последовательность IP ID когда-либо увеличивается как минимум на 20 000, TI получает значение RD (случайная).

3. Если все IP ID одинаковы, TI получает это значение в шестнадцатеричном виде.

4. Если какая-либо разность между двумя последовательными ID превышает 1000 и не делится без остатка на 256, TI получает значение RI (случайные положительные приращения). Если разность

делится без остатка на 256, для получения результата RI она должна составлять как минимум 256 000.

5. Если все разности делятся на 256 и не превышают 5120, TI получает значение BI (нарушенное приращение). Так происходит в системах наподобие Microsoft Windows, где IP ID отправляется в порядке байтов узла, а не в сетевом порядке байтов. Это прекрасно работает и никоим образом не нарушает RFC, хотя и раскрывает сведения об архитектуре узла, которые могут пригодиться атакующим.

6. Если все разности меньше десяти, TI получает значение I (последовательное приращение). Здесь мы допускаем разность до десяти (вместо требования строго последовательного порядка), поскольку трафик от других узлов может создавать разрывы последовательности.

7. Если ни один из предыдущих шагов не определяет алгоритм генерации, проверка исключается из отпечатка.

Алгоритм генерации последовательности ICMP IP ID (II)

Эта проверка похожа на описанную выше TI, за исключением того, что она оценивает IP ID из ответов ICMP на две наши ping-пробы. Она включается, только если получены оба ответа. Разности IP ID абсолютны (предполагается переход через границу диапазона) и вычисляются так, как описано для TI. Результат вычисляется проще, чем TI. Результата RD нет, поскольку для его обоснования недостаточно образцов. II вычисляется следующим образом:

1. Если оба номера ID равны нулю, II получает значение Z.

2. Если оба IP ID одинаковы, II получает это значение в шестнадцатеричном виде.

3. Если абсолютная разность ID превышает 1000 и не делится без остатка на 256, II получает значение RI (случайные положительные приращения). Если разность делится без остатка на 256, для получения результата RI она должна составлять как минимум 256 000.

4. Если разность IP ID делится на 256 и не превышает 5120, II получает значение BI (нарушенное приращение). Так происходит в системах наподобие Microsoft, где IP ID отправляется в порядке байтов узла, а не в сетевом порядке байтов. Это прекрасно работает и никоим образом не нарушает RFC, хотя и раскрывает сведения об архитектуре узла, которые могут пригодиться атакующим.

5. Если разность меньше десяти, II получает значение I (последовательное приращение). Здесь мы допускаем разность до десяти (вместо требования строго последовательного порядка), поскольку трафик от других узлов может создавать разрывы последовательности.

6. Если ни один из предыдущих шагов не определяет алгоритм генерации, проверка исключается из отпечатка.

Логическое значение общей последовательности IP ID (SS)

Это логическое значение указывает, использует ли цель общую последовательность IP ID для протоколов TCP и ICMP. Если наши шесть значений TCP IP ID равны 117, 118, 119, 120, 121 и 122, а результаты ICMP - 123 и 124, ясно, что обе последовательности не только возрастают, но и являются частями одной последовательности. Если же значения TCP IP ID равны 117-122, а значения ICMP - 32 917 и 32 918, используется другая последовательность.

Эта проверка включается, только если II имеет значение RI, BI или I, а TI имеет такое же значение. Если SS включена, результат равен S, когда последовательность общая, и O (иная), когда она не

общая. Это определяется следующим алгоритмом:

Пусть avg - IP ID последнего ответа на TCP-пробу последовательности минус IP ID первого ответа на TCP-пробу последовательности, делённый на разность номеров проб. Если проба № 1 возвращает IP ID 10 000, а проба № 6 возвращает 20 000, avg будет равно (20 000 - 10 000) / (6 - 1), то есть 2000.

Если IP ID первого ответа ICMP echo меньше IP ID последнего ответа на TCP-пробу последовательности плюс утроенное avg, результат SS равен S. В противном случае он равен O.

Алгоритм параметра временной метки TCP (TS)

TS - ещё одна проверка, пытающаяся определить характеристики целевой ОС по тому, как она генерирует ряд чисел. Здесь рассматривается параметр временной метки TCP (если он есть) в ответах на пробы SEQ. Исследуется значение TSval (первые четыре байта параметра), а не возвращённое в ответе значение TSecr (последние четыре байта). Разность между каждой парой последовательных значений TSval делится на время, прошедшее между отправкой Nmap двух проб, породивших эти ответы. Полученное значение даёт скорость увеличения временной метки в секунду. Nmap вычисляет среднее число приращений в секунду по всем последовательным пробам, а затем вычисляет TS следующим образом:

1. Если хотя бы в одном ответе нет параметра временной метки, TS получает значение U (не поддерживается).

2. Если хотя бы одно значение временной метки равно нулю, TS получает значение 0.

3. Если среднее число приращений в секунду попадает в диапазоны 0-5,66, 70-150 или 150-350, TS получает значение 1, 7 или 8 соответственно. Эти три диапазона обрабатываются особо, поскольку соответствуют частотам 2 Гц, 100 Гц и 200 Гц, используемым многими узлами.

4. Во всех остальных случаях Nmap записывает двоичный логарифм среднего числа приращений в секунду, округлённый до ближайшего целого. Поскольку большинство узлов используют частоту 1000 Гц, результат A встречается часто.

Параметры TCP (O, O1-O6)

Эта проверка записывает параметры заголовка TCP в пакете. Она сохраняет исходный порядок, а также предоставляет некоторые сведения о значениях параметров. Поскольку RFC 793 не требует какого-либо определённого порядка, реализации часто создают уникальные варианты порядка. Некоторые платформы реализуют не все параметры (разумеется, они необязательны). Сочетание всех этих перестановок с количеством различных значений параметров, используемых реализациями, превращает эту проверку в настоящую сокровищницу информации. Значение проверки - строка символов, представляющих используемые параметры. Несколько параметров принимают аргументы, которые следуют непосредственно за символом. Все поддерживаемые параметры и аргументы показаны в таблице 8.1.

Таблица 8.1. Значения проверки O

Название параметра · символ · аргумент (если есть)

• **Конец списка параметров (EOL)** — символ: L; аргумент отсутствует.
• **Нет операции (NOP)** — символ: N; аргумент отсутствует.
• **Максимальный размер сегмента (MSS)** — символ: M; значение добавляется следом. Многие системы возвращают значение, использованное в соответствующей пробе.

• **Масштаб окна (WS)** — символ: `W`; фактическое значение добавляется следом.
• **Временная метка (TS)** — символ: `T`; после него следуют два двоичных символа, представляющих соответственно значения TSval и TSecr. Символ равен 0, если поле равно нулю, и 1 в противном случае.
• **Избирательное подтверждение разрешено (SACK)** — символ: `S`; аргумент отсутствует.

Например, строка `M5B4NW3NNT11` означает, что пакет содержит параметр MSS (значение `0x5B4`), за которым следует NOP. Далее идёт параметр масштаба окна со значением три, затем ещё два NOP. Последний параметр - временная метка, и ни одно из двух её полей не было равно нулю. Если в ответе нет параметров TCP, проверка будет присутствовать, но строка значения окажется пустой. Если ответа на пробу не было, проверка исключается.

Хотя обычно эта проверка называется O, шесть проб, отправляемых для генерации последовательности, являются особым случаем. Они помещаются в специальную строку проверки OPS и получают названия от O1 до O6, чтобы различать, к какому пакету-пробе они относятся. «O» означает «options» («параметры»). Несмотря на разные названия, каждая проверка от O1 до O6 обрабатывается точно так же, как остальные проверки O.

Начальный размер окна TCP (W, W1-W6)

Эта проверка просто записывает 16-битный размер окна TCP полученного пакета. Она весьма эффективна, поскольку известно более 80 значений, каждое из которых отправляет по крайней мере одна ОС. Недостаток в том, что некоторые операционные системы сами по себе имеют более дюжины возможных значений. Это приводит к ложноотрицательным результатам, пока мы не соберём все возможные размеры окна, используемые операционной системой.

Хотя обычно эта проверка называется W, шесть проб, отправляемых для генерации последовательности, являются особым случаем. Они помещаются в специальную строку проверки WIN и получают названия от W1 до W6. Размер окна записывается для всех проб порядковых номеров, поскольку они различаются значениями параметра TCP MSS, что заставляет некоторые операционные системы объявлять другой размер окна. Несмотря на разные названия, каждая проверка обрабатывается совершенно одинаково.

Наличие ответа (R)

Эта проверка просто записывает, ответила ли цель на данную пробу. Возможные значения - Y и N. Если ответа нет, остальные поля этой проверки исключаются.

Риск этой проверки связан с пробами, отбрасываемыми межсетевым экраном. Это приводит к R=N в отпечатке субъекта. Однако в эталонном отпечатке из `nmap-os-db` может быть R=Y, если целевая ОС обычно отвечает. Таким образом, межсетевой экран может помешать правильному определению ОС. Чтобы уменьшить эту проблему, эталонные отпечатки обычно исключают проверку R=Y из проб IE и U1, которые отбрасываются с наибольшей вероятностью. Кроме того, если Nmap не нашёл у цели закрытого TCP-порта, он не устанавливает R=N для проверок T5, T6 или T7, даже если выбранный им порт не отвечает. В конце концов, закрытого порта может не быть потому, что все они фильтруются.

Бит запрета фрагментации IP (DF)

Заголовок IP содержит один бит, запрещающий маршрутизаторам фрагментировать пакет. Если пакет слишком велик для обработки маршрутизаторами, им придётся просто отбросить его (и в идеале вернуть ответ «цель недоступна, требуется фрагментация»). Эта проверка записывает Y, если бит установлен, и N, если он не установлен.

Запрет фрагментации (ICMP) (DFI)

Это всего лишь изменённая версия проверки DF, используемая для особых проб IE. Она сравнивает результаты бита запрета фрагментации для двух отправленных проб ICMP echo request. У неё четыре возможных значения, перечисленных в таблице 8.2.

Таблица 8.2. Значения проверки DFI

Значение — описание

• **N** — Ни в одном из ответов ping бит DF не установлен.
• **S** — Оба ответа повторяют значение DF пробы.
• **Y** — В обоих ответах биты DF установлены.
• **O** — Единственная оставшаяся комбинация: в обоих ответах значение бита DF переключено.

Начальное время жизни IP (T)

IP-пакеты содержат поле времени жизни (TTL), которое уменьшается при каждом прохождении маршрутизатора. Если поле достигает нуля, пакет должен быть отброшен. Это предотвращает бесконечное зацикливание пакетов. Поскольку операционные системы различаются исходным значением TTL, его можно использовать для определения ОС. Nmap определяет расстояние до цели в переходах, исследуя ответ ICMP о недоступности порта на пробу U1. Этот ответ включает исходный IP-пакет, в том числе уже уменьшенное поле TTL, полученное целью. Вычитая это значение из TTL, с которым мы отправили пакет, мы узнаём расстояние до машины в переходах. Затем Nmap прибавляет это расстояние к TTL ответа на пробу, чтобы определить начальное значение TTL в момент отправки пакета ответа ICMP на пробу. Это начальное значение TTL сохраняется в отпечатке как результат T.

Хотя восьмибитное поле наподобие TTL никогда не может содержать значения больше `0xFF`, эта проверка иногда даёт значения `0x100` или выше. Так происходит, когда какая-либо система (это может быть источник, цель или промежуточная система) повреждает TTL либо иным образом неправильно его уменьшает. Причиной также могут быть асимметричные маршруты.

Nmap также может узнать из системного интерфейса и таблиц маршрутизации, что расстояние равно нулю переходов (сканирование localhost) или одному (в том же сегменте сети). Это значение используется, когда Nmap печатает расстояние в переходах для пользователя, но не используется при вычислении результата T.

Предположение о начальном времени жизни IP (TG)

Nmap нередко не получает ответа на пробу U1 и поэтому не может узнать расстояние до цели в переходах. Межсетевые экраны и устройства NAT обожают блокировать незапрошенные UDP-пакеты. Но поскольку распространённые значения TTL далеко разнесены, а цели редко находятся дальше чем в 20 переходах, Nmap всё равно может сделать довольно хорошее предположение. Большинство систем отправляют пакеты с начальным TTL 32, 60, 64, 128 или 255. Поэтому полученное в ответе значение TTL округляется вверх до следующего значения из 32, 64,

128 или 255. Числа 60 в этом списке нет, поскольку его невозможно надёжно отличить от 64. В любом случае оно встречается редко. Полученное предположение сохраняется в поле TG. Это поле предполагаемого TTL не печатается в отпечатке субъекта, если было обнаружено фактическое значение TTL (T).

Явное уведомление о перегрузке (CC)

Эта проверка используется только для пробы ECN. Такая проба представляет собой пакет SYN, включающий флаги управления перегрузкой CWR и ECE. После получения ответа SYN/ACK эти флаги исследуются, чтобы установить значение проверки CC (управление перегрузкой), как описано в таблице 8.3.

Таблица 8.3. Значения проверки CC

Значение — описание

• **Y** — Установлен только бит ECE (не CWR). Этот узел поддерживает ECN.
• **N** — Ни один из этих двух битов не установлен. Цель не поддерживает ECN.
• **S** — Оба бита установлены. Цель не поддерживает ECN, но возвращает то, что считает зарезервированным битом.
• **O** — Единственная оставшаяся комбинация этих двух битов (иная).

Разные особенности TCP (Q)

Эта проверка ищет две особенности, имеющиеся в стеке TCP некоторых реализаций. Первая состоит в том, что зарезервированное поле заголовка TCP (сразу после длины заголовка) не равно нулю. Особенно вероятно это в ответе на проверку ECN, поскольку она устанавливает зарезервированный бит в пробе. Если такое наблюдается в пакете, в строку Q записывается «R».

Другая проверяемая Nmap особенность - ненулевое значение поля указателя срочности при неустановленном флаге URG. Это также особенно вероятно в ответе на пробу ECN, устанавливающую ненулевое поле срочности. Когда такое наблюдается, к строке Q добавляется «U».

Строка Q всегда должна формироваться в алфавитном порядке. Если особенностей нет, проверка Q пуста, но всё равно показывается.

Порядковый номер TCP (S)

Эта проверка исследует 32-битное поле порядкового номера в заголовке TCP. Вместо записи значения поля, как делают некоторые другие проверки, она исследует его соотношение с номером подтверждения TCP из пробы, вызвавшей ответ. Затем записывается соответствующее значение, показанное в таблице 8.4.

Таблица 8.4. Значения проверки S

Значение — описание

• **Z** — Порядковый номер равен нулю.
• **A** — Порядковый номер совпадает с номером подтверждения в пробе.
• **A+** — Порядковый номер равен номеру подтверждения в пробе плюс один.
• **O** — Порядковый номер имеет иное значение.

Порядковый номер ICMP (SI)

Эта проверка рассматривает порядковый номер в пакетах ответа ICMP echo. Она используется только для двух проб IE echo request. Четыре возможных значения показаны в таблице 8.5.

Таблица 8.5. Значения проверки SI

Значение — описание

• **Z** — Оба порядковых номера установлены в 0.
• **S** — Оба порядковых номера повторяют номера из проб.
• <NNNN> — Если оба используют одно и то же ненулевое число, оно записывается здесь.
• **O** — Любая иная комбинация.

Номер подтверждения TCP (A)

Эта проверка совпадает с S, за исключением того, что она исследует соотношение номера подтверждения в ответе с порядковым номером в соответствующей пробе. Четыре возможных значения приведены в таблице 8.6.

Таблица 8.6. Значения проверки A

Значение — описание

• **Z** — Номер подтверждения равен нулю.
• **S** — Номер подтверждения совпадает с порядковым номером в пробе.
• **S+** — Номер подтверждения равен порядковому номеру в пробе плюс один.
• **O** — Номер подтверждения имеет иное значение.

Флаги TCP (F)

Это поле записывает флаги TCP в ответе. Каждая буква представляет один флаг, и они следуют в том же порядке, что и в TCP-пакете (от старшего бита слева к младшим). Поэтому значение SA представляет установленные биты SYN и ACK, тогда как значение AS недопустимо (неверный порядок). Возможные флаги показаны в таблице 8.7.

Таблица 8.7. Значения проверки F

Символ · название флага · значение байта флагов

• E — Эхо ECN (ECE); значение байта флагов: 64.
• U — Срочные данные (URG); значение байта флагов: 32.
• A — Подтверждение (ACK); значение байта флагов: 16.
• P — Проталкивание (PSH); значение байта флагов: 8.
• R — Сброс (RST); значение байта флагов: 4.
• S — Синхронизация (SYN); значение байта флагов: 2.
• F — Завершение (FIN); значение байта флагов: 1.

Контрольная сумма данных TCP RST (RD)

Некоторые операционные системы возвращают в пакетах сброса данные ASCII, например сообщения об ошибках. Это явно разрешено разделом 4.2.2.12 RFC 1122. Когда Nmap встречает такие данные, он вычисляет контрольную сумму CRC16 и сообщает результат. Если данных нет, RD устанавливается в ноль. Среди немногих операционных систем, способных возвращать данные в пакетах сброса, - HP-UX и версии Mac OS до Mac OS X.

Тип обслуживания IP (TOS)

Эта проверка просто записывает байт типа обслуживания из IP-заголовка пакетов ICMP о недоступности порта. Этот байт описан в RFC 791. Значение не записывается для других ответов (например, пакетов TCP или ответов echo), поскольку изменения в них обычно вызваны сетевыми устройствами или службами узла, а не отражают саму целевую ОС.

Тип обслуживания IP для ответов ICMP (TOSI)

Эта проверка сравнивает байты типа обслуживания IP (TOS) в ответах на обе пробы ICMP echo request проверки IE. Возможные значения показаны в таблице 8.8.

Таблица 8.8. Значения проверки TOSI

Значение — описание

- **Z** — Оба значения TOS равны нулю.
- **S** — Оба значения TOS совпадают со значениями в соответствующих пробах.
- <NN> — Если оба используют одно и то же ненулевое число, оно записывается здесь.
- **O** — Любая иная комбинация.

Общая длина IP (IPL)

Эта проверка записывает общую длину IP-пакета (в октетах). Она используется только для ответа о недоступности порта, вызванного проверкой U1. Эта длина зависит от реализации, поскольку реализации могут выбирать, сколько данных из исходной пробы включать, если соблюдается минимальное требование RFC 792. Требуется включить исходный IP-заголовок и как минимум восемь байт данных.

Ненулевое неиспользуемое поле сообщения о недоступности порта (UN)

Заголовок сообщения ICMP о недоступности порта имеет длину восемь байт, но используются только первые четыре. RFC 792 устанавливает, что последние четыре байта должны быть нулевыми. Некоторые реализации (в основном коммутаторы Ethernet и отдельные специализированные встроенные устройства) всё же задают их. Значение этих последних четырёх байт записывается в данное поле.

Возвращённое значение общей длины IP пробы (RIPL)

Сообщения ICMP о недоступности порта (такие сообщения отправляются в ответ на пробу U1) должны включать породивший их заголовок. Этот заголовок следует возвращать точно в том виде,

в каком он был получен, но некоторые реализации возвращают повреждённую версию из-за изменений, внесённых ими при обработке IP. Эта проверка просто записывает возвращённое значение общей длины IP. Если возвращено правильное значение `0x148` (328), вместо фактического значения сохраняется G (good, правильное).

Возвращённое значение IP ID пробы (RID)

Проба U1 имеет постоянное значение IP ID `0x1042`. Если это значение возвращается в сообщении о недоступности порта, для этой проверки сохраняется значение G. В противном случае сохраняется точное возвращённое значение. Некоторые системы, например Solaris, изменяют значения IP ID для отправляемых Nmap сырых IP-пакетов. В таких случаях эта проверка пропускается. Мы обнаружили, что некоторые системы, особенно принтеры HP и Xerox, меняют байты местами и вместо него возвращают `0x4210`.

Целостность возвращённого значения контрольной суммы IP пробы (RIPCK)

Контрольная сумма IP - одно из значений, которое мы не ожидаем получить неизменным в сообщении о недоступности порта. В конце концов, каждый сетевой переход во время передачи изменяет контрольную сумму при уменьшении TTL. Однако полученная нами контрольная сумма должна соответствовать объемлющему IP-пакету. Если это так, для этой проверки сохраняется значение G (good, правильное). Если возвращённое значение равно нулю, сохраняется Z. В противном случае результат равен I (invalid, недействительное).

Целостность возвращённых длины и контрольной суммы UDP пробы (RUL и RUCK)

Значения длины заголовка UDP и контрольной суммы должны возвращаться точно в том виде, в каком они были отправлены. Если это так, для этих проверок записывается G. В противном случае записывается фактически возвращённое значение. Правильная длина равна `0x134` (308).

Целостность возвращённых данных UDP (RUD)

Если возвращённая полезная нагрузка UDP, как и ожидается, состоит из 300 символов `C` (`0x43`), для этой проверки записывается G. В противном случае записывается I (invalid, недействительное).

Код ответа ICMP (CD)

Значение кода пакета ответа ICMP echo (тип ноль) должно быть нулевым. Но некоторые реализации ошибочно отправляют другие значения, особенно если запрос echo имеет ненулевой код (как в одной из проверок IE). Значения кода ответа для двух проб объединяются в значение CD, как описано в таблице 8.9.

Таблица 8.9. Значения проверки CD

Значение — описание

• **Z** — Оба значения кода равны нулю.

• **S** — Оба значения кода совпадают со значениями в соответствующих пробах.
• <NN> — Если оба используют одно и то же ненулевое число, оно показывается здесь.
• **O** — Любая иная комбинация.

Длина данных в ответах ICMP (DLI)

Когда данные включены в пакет ICMP echo request, они должны в неизменном виде возвращаться в соответствующем ответе echo. Но некоторые реализации всё же усекают данные. Эта проверка рассматривает оба ответа ICMP на пробы IE и присваивает значение, описанное в таблице 8.10.

Таблица 8.10. Значения проверки DLI

Значение — описание

• **Z** — Ни один ответ не содержит данных.
• **S** — Оба ответа возвращают все данные, отправленные в соответствующем запросе.
• <NN> — Если хотя бы один ответ усекает данные, здесь сохраняется наибольший объём возвращённых данных (в любом из пакетов). Когда оба усекают длину данных до одного и того же ненулевого числа, оно показывается здесь. Это значение учитывает только сами данные, а не заголовки IP или ICMP.

8.4. Методы снятия отпечатков, не используемые Nmap

Nmap поддерживает гораздо больше методов определения ОС, чем любая другая программа, и нам всегда интересны новые идеи. Присылайте их для обсуждения в список разработки Nmap (nmap-dev). Однако некоторые методы просто плохо подходят. В этом разделе подробно рассматриваются наиболее интересные из них. Хотя Nmap их не поддерживает, некоторые полезно применять вместе с Nmap, чтобы подтвердить результаты и узнать дополнительные подробности.

8.4.1. Пассивное снятие отпечатков

Пассивное снятие отпечатков использует большинство тех же методов, что и выполняемое Nmap активное снятие отпечатков. Разница в том, что пассивная система просто прослушивает сеть, при удобном случае классифицируя узлы по наблюдаемому трафику. Это сложнее активного снятия отпечатков, поскольку приходится принимать любые происходящие сеансы связи, а не создавать собственные специальные пробы. Это ценный метод, но ему не место в принципиально активном инструменте наподобие Nmap. К счастью, Михал Залевский (Michal Zalewski) написал превосходный инструмент пассивного определения ОС p0f[2]. Он также разработал несколько нынешних проверок Nmap для снятия отпечатков ОС. Другой вариант - SinFP[3] Гомора (GomoR), поддерживающий как активное, так и пассивное снятие отпечатков.

8.4.2. Хронология эксплойтов

Снятие отпечатков TCP/IP хорошо различает разные операционные системы, но обнаружение разных версий одной и той же операционной системы может быть затруднительным. Компания должна как-нибудь изменить свой стек, чтобы мы могли заметить различие. К счастью, многие поставщики ОС регулярно обновляют системы для соответствия новейшим стандартам. Но что насчёт тех, кто этого не делает? Большинство из них в конце концов хотя бы исправляют пригодные

для эксплуатации ошибки стека. А такие исправления легко обнаружить удалённо. Сначала отправьте полезную нагрузку эксплойта: атаку land, teardrop, ping of death, SYN flood или WinNuke. Отправляйте по одной атаке, после чего немедленно снова пытайтесь связаться с системой. Если она внезапно перестала отвечать, круг вариантов ОС сузился до версий, поставлявшихся без исправления.

> **Предупреждение.** Если вы используете эксплойты отказа в обслуживании (DoS) как часть набора средств определения ОС, не забудьте выполнять эти проверки последними.

8.4.3. Время повторной передачи

Реализации TCP обладают значительной свободой в выборе точного времени ожидания перед повторной передачей пакетов. Для эксплуатации этого свойства существуют экспериментальные инструменты Ring и Cron-OS. Они отправляют пакет SYN на открытый порт, а затем игнорируют полученный SYN/ACK, вместо того чтобы подтвердить его пакетом ACK (завершив соединение) или пакетом RST (прервав его). Целевой узел ещё несколько раз повторно отправляет SYN/ACK, а эти инструменты отслеживают каждую долю секунды ожидания. Хотя этим методом действительно можно получить некоторые сведения, есть несколько причин, по которым я не включил соответствующую заплату в Nmap:

• Обычно требуется изменить правила межсетевого экрана исходного узла, чтобы система не отвечала пакетом RST на полученный SYN/ACK. Сделать это переносимым способом трудно. И даже будь это просто, многие пользователи не одобряют вмешательство приложений в правила межсетевого экрана.

• Это может быть очень медленно. Повторные передачи способны продолжаться несколько минут. Долго ждать проверки, которая к тому же даёт не так уж много сведений.

• Метод может быть неточным, поскольку потеря пакетов и задержки (которых следует ожидать в реальных условиях) способны приводить к ложным результатам.

Я перечислил эти причины здесь, поскольку они применимы и к некоторым другим предложенным методам определения ОС. Я с удовольствием добавил бы новые проверки, но они должны быть быстрыми и требовать мало пакетов. Вмешательство в межсетевой экран узла неприемлемо. Я стараюсь избегать установления полных TCP-соединений для снятия отпечатков стека, хотя это делается для определения ОС в рамках системы сканирования версий.

8.4.4. Фрагментация IP

Фрагментация IP - сложная система, а её реализации кишат ошибками и несоответствиями. Возможные проверки могли бы исследовать сборку перекрывающихся фрагментов или измерять время ожидания дефрагментации. Nmap избегает этих проверок, поскольку многие межсетевые экраны и другие промежуточные устройства дефрагментируют трафик на шлюзах. Поэтому Nmap может снять отпечаток межсетевого экрана вместо настоящего узла назначения. Кроме того, в некоторых операционных системах фрагменты трудно отправлять. Ядра Linux 2.6 склонны помещать отправляемые вами фрагменты в очередь и самостоятельно собирать их перед передачей.

8.4.5. Шаблоны открытых портов

О целевой ОС узла часто можно догадаться, просто посмотрев, какие порты открыты. У машин Microsoft Windows часто открыты TCP-порты 135 и 139. Windows 2000 и более новые версии также прослушивают порт 445. Между тем машина со службами на портах 22 (ssh) и 631 (Internet Printing Protocol) скорее всего работает под Unix.

Хотя эта эвристика часто полезна, для Nmap она просто недостаточно надёжна. Сочетания портов могут скрываться правилами межсетевого экрана, а большинство распространённых протоколов доступны на нескольких платформах. Серверы OpenSSH могут работать в Windows[4], а «порты Windows SMB» может обслуживать Samba[5], работающая на машине Unix. Перенаправление портов ещё больше запутывает дело. Машина, на которой, судя по всему, работает Microsoft IIS, может быть межсетевым экраном Unix, просто перенаправляющим порт 80 машине Windows.

По этим причинам Nmap не учитывает номера открытых портов при снятии отпечатков стека TCP/IP. Однако Nmap может использовать сведения определения версий (см. главу 7 «Определение версий служб и приложений»), чтобы отдельно обнаруживать сведения об операционной системе и типе устройства. Сохраняя отдельно результаты определения ОС, полученные определением ОС и определением версий, Nmap способен корректно обрабатывать межсетевой экран Checkpoint, использующий перенаправление TCP-порта на веб-сервер Windows. Результаты снятия отпечатков стека должны быть «Checkpoint Firewall-1», тогда как определение версий должно предположить, что ОС - Windows. Помните, что сведения об ОС и типе устройства включены лишь в малую долю сигнатур определения версий: мы можем заполнять эти поля, только когда приложение раскрывает сведения либо работает лишь на одной ОС или одном типе устройств.

8.5. Понимание отпечатка Nmap

Когда Nmap хранит отпечаток в памяти, он использует дерево атрибутов и значений в структурах данных, о которых пользователям даже не нужно знать. Но существует также специальная версия в кодировке ASCII, которую Nmap может напечатать для пользователей, когда машина не распознана. Тысячи таких сериализованных отпечатков также считываются из базы данных `nmap-os-db` при каждом запуске Nmap (с включённым определением ОС). Формат отпечатка - компромисс между понятностью для человека и краткостью. Формат настолько сжат, что многим неопытным пользователям кажется линейным шумом, но прочитавшие этот документ должны легко расшифровывать отпечатки. На самом деле существует два типа отпечатков, хотя их общая структура одинакова. Считываемые Nmap отпечатки известных операционных систем называются эталонными отпечатками, а отпечаток, который Nmap выводит после сканирования системы, - отпечатком субъекта. Эталонные отпечатки немного сложнее, поскольку их можно настраивать для совпадения с целым классом операционных систем, добавляя допуски к ненадёжным проверкам (или исключая их) и разрешая лишь одно возможное значение для других проверок.

Эталонные отпечатки также содержат подробности и классификации ОС. Поскольку проверки субъекта проще, сначала опишем их.

8.5.1. Расшифровка формата отпечатка субъекта

Если Nmap снимает отпечаток ОС узла и, несмотря на благоприятные условия (например, доступны как открытые, так и закрытые порты цели), не получает идеального совпадения ОС, он печатает отпечаток субъекта со всеми результатами проверок, которые считает значимыми, а затем просит пользователя отправить данные на Nmap.Org. Проверки не показываются, когда у Nmap нет полезных результатов, например если не были получены соответствующие ответы на

пробы. Специальная строка SCAN содержит дополнительные подробности сканирования (такие как номер версии Nmap), предоставляющие полезный контекст для включения отправленных отпечатков в `nmap-os-db`. Типичный отпечаток субъекта показан в примере 8.3.

Пример 8.3. Типичный отпечаток субъекта

```
OS:SCAN(V=4.62%D=5/21%OT=80%CT=1%CU=36069%PV=Y%DS=1%G=Y%M=001839%TM=483466E
OS:0%P=i686-pc-linux-gnu)SEQ(SP=C9%GCD=1%ISR=CE%TI=Z%II=I%TS=8)OPS(O1=M5B4S
OS:T11NW0%O2=M5B4ST11NW0%O3=M5B4NNT11NW0%O4=M5B4ST11NW0%O5=M5B4ST11NW0%O6=M
OS:5B4ST11)WIN(W1=16A0%W2=16A0%W3=16A0%W4=16A0%W5=16A0%W6=16A0)ECN(R=Y%DF=Y
OS:%T=40%W=16D0%O=M5B4NNSNW0%CC=N%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=
OS:)T2(R=N)T3(R=Y%DF=Y%T=40%W=16A0%S=O%A=S+%F=AS%O=M5B4ST11NW0%RD=0%Q=)T4(R
OS:=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=
OS:AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=
OS:40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%TOS=C0%IPL=164%UN=0%RIP
OS:L=G%RID=G%RIPCK=G%RUCK=G%RUL=G%RUD=G)IE(R=Y%DFI=N%T=40%TOSI=S%CD=S%SI=S%
OS:DLI=S)
```

Теперь вы, возможно, смотрите на этот отпечаток и немедленно понимаете, что всё означает. Если так, можете просто пропустить этот раздел. Но я никогда не видел такой реакции. Многие, вероятно, думают, что Nmap извергает на них мусорные данные из-за какого-то переполнения буфера или ошибки незавершённой строки. Этот раздел поможет расшифровать сведения, чтобы вы сразу могли определить: атаки слепого предсказания последовательности TCP против этой машины имеют умеренную сложность, но она может стать хорошим зомби для сканирования бездействия (`-sI`). Первый шаг к пониманию этого отпечатка - исправить перенос строк. Все проверки слеплены вместе, а каждая строка перенесена на 71-м символе. Затем к каждой строке добавляется префикс `OS:`, увеличивая длину до 74 символов. Благодаря этому отпечатки легко копировать и вставлять в форму отправки отпечатков Nmap (см. раздел 8.7.2 «Когда Nmap не находит совпадения и печатает отпечаток»). Удаление префикса и исправление переноса (каждая строка должна оканчиваться правой круглой скобкой) приводит к очищенной версии в примере 8.4.

Пример 8.4. Очищенный отпечаток субъекта

```
SCAN(V=4.62%D=5/21%OT=80%CT=1%CU=36069%PV=Y%DS=1%G=Y%M=001839%
     TM=483466E0%P=i686-pc-linux-gnu)
SEQ(SP=C9%GCD=1%ISR=CE%TI=Z%II=I%TS=8)
OPS(O1=M5B4ST11NW0%O2=M5B4ST11NW0%O3=M5B4NNT11NW0%O4=M5B4ST11NW0%
    O5=M5B4ST11NW0%O6=M5B4ST11)
WIN(W1=16A0%W2=16A0%W3=16A0%W4=16A0%W5=16A0%W6=16A0)
ECN(R=Y%DF=Y%T=40%W=16D0%O=M5B4NNSNW0%CC=N%Q=)
T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=N)
T3(R=Y%DF=Y%T=40%W=16A0%S=O%A=S+%F=AS%O=M5B4ST11NW0%RD=0%Q=)
T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
U1(R=Y%DF=N%T=40%TOS=C0%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUL=G%RUD=G)
IE(R=Y%DFI=N%T=40%TOSI=S%CD=S%SI=S%DLI=S)
```

Хотя это всё ещё не самый интуитивный формат в мире (нам пришлось сделать его коротким), теперь он гораздо яснее. Каждая строка является категорией: например, SEQ - проверки генерации последовательностей, T3 - результаты конкретной TCP-пробы, а IE - проверки, относящиеся к двум пробам ICMP echo.

После названия каждой проверки следует пара круглых скобок, заключающая результаты отдельных проверок. Проверки имеют формат `<имя_проверки>=<значение>`. Все возможные категории, проверки и значения описаны в разделе 8.3 «Поддерживаемые Nmap методы снятия отпечатков TCP/IP». Каждая пара проверок разделяется знаком процента (%). Значения проверок могут быть пустыми, и тогда сразу после знака равенства следует знак процента или завершающая категорию правая круглая скобка. Строка `O=%RD=0%Q=)` в T4 нашего примера показывает две такие пустые проверки. Пустое значение проверки должно совпадать с другим пустым значением,

поэтому эта пустая проверка особенностей TCP Q не совпадёт с отпечатком, где Q имеет значение RU.

В некоторых случаях отсутствует целая проверка, а не только её значение. Например, в T2 нашего образца отпечатка нет проверок W (окно TCP), S (порядковый номер), A (номер подтверждения), T (TTL) и TG (предположение TTL). Причина в том, что единственная включённая проверка и её значение R=N означают отсутствие ответа на пробу T2. Поэтому включать размер окна или порядковый номер не имело бы особого смысла. Аналогично пропускаются проверки, плохо поддерживаемые системой, на которой работает Nmap. Пример - проверка RID (поле IP ID, возвращённое в пакете ICMP), которая плохо работает в Solaris, поскольку эта система склонна повреждать отправляемое Nmap поле ID. Неубедительные проверки (например, когда не удалось определить последовательность IP ID для проверок TI и II) также исключаются.

Расшифровка строки SCAN отпечатка субъекта

Строка SCAN - особый случай в отпечатке субъекта. Вместо описания целевой системы эти проверки описывают различные условия сканирования. Они помогают нам включать отправленные на Nmap.Org отпечатки. В этой строке находятся следующие проверки:

• Номер версии Nmap (V).

• Дата сканирования (D) в формате месяц/день.

• Открытый и закрытый TCP-порты цели, использованные для сканирования (OT и CT). В отличие от большинства проверок, они печатаются в десятичном формате. Если Nmap не смог найти открытый или закрытый порт, проверка включается с пустым значением (даже когда Nmap предполагает, что некий порт, возможно, закрыт, и отправляет туда пробу).

• Закрытый UDP-порт (CU). Это то же самое, что CT, но для UDP. Поскольку большинство сканирований не включает UDP, значение этой проверки обычно пусто.

• Частное пространство IP (PV) равно Y, если цель находится в частных сетях `10.0.0.0/8`, `172.16.0.0/12` или `192.168.0.0/16` (RFC 1918). В противном случае оно равно N.

• Сетевое расстояние (DS) - расстояние до цели в сетевых переходах. Оно равно 0, если цель - localhost, 1 при прямом подключении по сети Ethernet или точному расстоянию, если его обнаружил Nmap. Если расстояние неизвестно, проверка исключается.

• Хорошие результаты (G) равны Y, если условия и результаты кажутся достаточно хорошими для отправки этого отпечатка на Nmap.Org. В противном случае значение равно N. Если не заставить Nmap печатать их, включив отладку (`-d`) или предельно подробный вывод (`-vv`), отпечатки G=N не выводятся.

• Префикс MAC цели (M) - первые шесть шестнадцатеричных цифр MAC-адреса цели, соответствующие названию поставщика. Ведущие нули не включаются. Это поле исключается, если цель не находится в той же сети Ethernet (DS=1).

• Время сканирования ОС (TM) приводится в формате Unix `time_t` (в шестнадцатеричном виде).

• Платформа, для которой был скомпилирован Nmap, указывается в поле P.

8.5.2. Расшифровка формата эталонного отпечатка

Когда Nmap сканирует цель для создания отпечатка субъекта, затем он пытается сопоставить эти данные с тысячами эталонных отпечатков в базе `nmap-os-db`. Эталонные отпечатки изначально

формируются из одного или нескольких отпечатков субъекта и потому имеют с ними много общего. Но они содержат немного дополнительных сведений, облегчающих сопоставление и, разумеется, описывающих представляемые ими операционные системы. Например, только что рассмотренный отпечаток субъекта мог бы стать основой эталонного отпечатка из примера 8.5.

Пример 8.5. Типичный эталонный отпечаток

```
Fingerprint Sony PlayStation 3 game console
Class Sony | embedded || game console
SEQ(SP=F7-101%GCD=<7%ISR=FC-106%TI=RD%TS=21)
OPS(O1=M5B4NNSNW1NNT11%O2=M5B4NNSNW1NNT11%O3=M5B4NW1NNT11%
    O4=M5B4NNSNW1NNT11%O5=M5B4NNSNW1NNT11%O6=M5B4NNSNNT11)
WIN(W1=FFFF%W2=FFFF%W3=FFFF%W4=FFFF%W5=FFFF%W6=FFFF)
ECN(R=Y%DF=N%T=41%TG=41%W=FFFF%O=M5B4NNSNW1%CC=N%Q=)
T1(R=Y%DF=N%T=41%TG=41%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=Y%DF=N%T=41%TG=41%W=0%S=Z%A=O|S%F=AR%O=%RD=0%Q=)
T3(R=Y%DF=N%T=41%TG=41%W=FFFF%S=O%A=S+%F=AS%O=M5B4NNSNW1NNT11%RD=0%Q=)
T4(R=Y%DF=N%T=41%TG=41%W=0%S=A|O%A=Z%F=R%O=%RD=0%Q=)
T5(R=Y%DF=N%T=40%TG=40%W=0%S=Z%A=O|S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=N%T=40%TG=40%W=0%S=A|O%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=N%T=40%TG=40%W=0%S=Z%A=O|S%F=AR%O=%RD=0%Q=)
U1(DF=N%T=FF%TG=FF%TOS=0%IPL=38%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUL=G%RUD=G)
IE(DFI=N%T=FF%TG=FF%TOSI=S%CD=S%SI=S%DLI=S)
```

Некоторые различия очевидны сразу. Перенос строк не выполняется, поскольку он важен только для процесса отправки. Строка SCAN также удалена, поскольку эти сведения описывают конкретный экземпляр сканирования, а не общие характеристики целевой ОС.

Вероятно, вы также заметили две новые строки, `Fingerprint` и `Class`, которых не было в отпечатке субъекта. Более тонкое изменение состоит в том, что некоторые результаты отдельных проверок удалены, тогда как другие дополнены логическими выражениями.

Свободное описание ОС (строка Fingerprint)

Строка `Fingerprint` прежде всего служит маркером, по которому Nmap узнаёт о начале загрузки нового отпечатка. В каждом отпечатке есть только одна такая строка. Непосредственно после маркера `Fingerprint` (и пробела) следует текстовое описание операционной системы (или систем), представленной этим отпечатком. Это текст на английском языке в свободной форме, предназначенный для интерпретации человеком, а не машинным анализатором. Тем не менее Nmap старается придерживаться единообразного формата, включающего поставщика, название продукта, а затем номер версии. В этом поле могут встречаться рассмотренные ранее диапазоны номеров версий и альтернативы, разделённые запятыми. Вот несколько примеров:

```
Fingerprint HP LaserJet printer (4050, 4100, 4200, or 8150)
Fingerprint Sun Solaris 9 or 10 (SPARC)
Fingerprint Linux 2.6.22 - 2.6.24
Fingerprint Microsoft Windows Server 2003 SP1
Fingerprint Microsoft Windows XP Professional SP1
Fingerprint Minolta Di550 laser printer
```

В идеальном мире каждой отличающейся ОС соответствовал бы ровно один уникальный отпечаток. К сожалению, поставщики ОС не делают нашу жизнь настолько простой. Один и тот же выпуск ОС может давать разные отпечатки в зависимости от используемых сетевых драйверов, настраиваемых пользователем параметров, уровня исправлений, архитектуры процессора, объёма доступной оперативной памяти, настроек межсетевого экрана и многого другого. Иногда отпечатки различаются без какой-либо различимой причины. Хотя формат эталонного отпечатка имеет синтаксис выражений для работы с небольшими вариациями, при обнаружении значительных различий часто предпочтительно создавать несколько отпечатков одной ОС.

Так же как одной ОС нередко требуется несколько отпечатков, иногда один отпечаток описывает несколько систем. Если две системы дают совершенно одинаковые результаты во всех без исключения проверках, Nmap почти ничего не остаётся, кроме как предложить обе в качестве возможных вариантов. Обычно это происходит по нескольким причинам. Одна из них: поставщики могут выпустить новую версию своей ОС без каких-либо существенных изменений стека IP. Возможно, они внесли важные изменения в других частях системы, а возможно, почти ничего не сделали, но хотят заработать кучу денег на продаже «обновлений». В таких случаях Nmap часто печатает диапазон наподобие `Apple Mac OS X 10.4.8 - 10.4.11` или `Sun Solaris 9 or 10`.

Другая причина дублирующихся отпечатков - встроенные устройства, использующие общую ОС. Например, принтер одного поставщика и коммутатор Ethernet другого в действительности могут использовать встроенную ОС третьего поставщика. Во многих случаях тонкие различия между устройствами всё же позволяют их отличить. Но иногда Nmap приходится просто перечислить группу возможных вариантов, например `Cisco 1200-series WAP, HP ProCurve 2650 switch, or Xerox Phaser 7400N or 8550DT printer`.

Бывают и случаи, когда множество поставщиков продаёт одно и то же OEM-устройство под собственными торговыми марками и номерами моделей. Здесь Nmap снова приходится просто перечислять возможные варианты. Но различать их менее важно, поскольку в основе своей все они являются одним и тем же устройством.

> **Совет.** Если напечатанное Nmap описание (получаемое из строки `Fingerprint`) для вас недостаточно информативно, более подробные сведения могут находиться в комментариях над самим отпечатком в `nmap-os-db`. Его установленное в вашей системе расположение описано в главе 14 «Понимание и настройка файлов данных Nmap»; последнюю версию также можно посмотреть по адресу nmap.org. Ищите точное описание ОС, выданное Nmap. Помните, что строк `Fingerprint` с совершенно одинаковым описанием может быть несколько, поэтому, возможно, придётся исследовать их все. Либо используйте XML-вывод Nmap, где показан номер строки каждого совпадения.

Классификация устройства и ОС (строки Class)

Хотя описание `Fingerprint` прекрасно подходит аналитикам, читающим вывод Nmap напрямую, многие запускают Nmap из других сценариев и приложений. Эти приложения могут использовать сведения об ОС для проверки характерных для неё уязвимостей либо просто для создания красивого графа или отчёта.

Для этих целей существует более структурированная система классификации ОС. Она также полезна при нескольких совпадениях. Если получен лишь частичный отпечаток (например, на цели не найдено открытых портов, поэтому многие проверки пришлось пропустить), он может совпасть с десятками разных отпечатков из базы `nmap-os-db`. Печать подробностей всех этих отпечатков создала бы беспорядок. Но благодаря классификации ОС Nmap может найти общее. Если все совпадения классифицированы как Linux, Nmap просто напечатает, что цель - машина Linux.

Каждый отпечаток имеет одну или несколько строк `Class`. Каждая содержит четыре чётко определённых поля: поставщик, название ОС, семейство ОС и тип устройства. Поля разделяются символом вертикальной черты (`|`).

Тип устройства - широкая классификация, например `router`, `printer` или `game console`, уже рассмотренная ранее в этой главе. Операционные системы общего назначения наподобие Linux и Windows, которые можно использовать почти для чего угодно, классифицируются как `general purpose`.

Поставщик - компания, выпускающая ОС или устройство. Примеры: Apple, Cisco, Microsoft и Linksys. Для общественных проектов наподобие OpenBSD и Linux, не имеющих контролирующего поставщика, название семейства ОС повторяется в столбце поставщика.

Семейство ОС включает такие продукты, как Windows, Linux, IOS (для маршрутизаторов Cisco), Solaris и OpenBSD. Существуют также сотни устройств, например коммутаторы, широкополосные маршрутизаторы и принтеры, использующие нераскрытые операционные системы. Когда лежащая в основе ОС неясна, используется `embedded`.

Поколение ОС - более подробное описание ОС. К поколениям Linux относятся `2.4.X` и `2.6.X`, а к поколениям Windows - 95, 98, Me, 2000, XP и Vista. FreeBSD использует поколения наподобие `4.X` и `5.X`. Для малоизвестных операционных систем, которые мы не подразделяли на поколения (или когда ОС указана просто как `embedded`), это поле оставляется пустым.

Каждое поле может содержать лишь одно значение. Когда отпечаток представляет более одной возможной комбинации этих четырёх полей, используется несколько строк `Class`. В примере 8.6 приведены несколько строк `Fingerprint`, за которыми следуют соответствующие классификации.

Пример 8.6. Несколько типичных описаний отпечатков и соответствующих классификаций

```
Fingerprint D-Link DSL-500G ADSL router
Class D-Link | embedded || broadband router

Fingerprint Linksys WRT54GC or TRENDnet TEW-431BRP WAP
Class Linksys | embedded || WAP
Class TRENDnet | embedded || WAP

Fingerprint Apple Mac OS X 10.3.9 (Panther) - 10.4.7 (Tiger)
Class Apple | Mac OS X | 10.3.X | general purpose
Class Apple | Mac OS X | 10.4.X | general purpose

Fingerprint Sony PlayStation 3 game console
Class Sony | embedded || game console
```

Если этих примеров недостаточно, список классификаций, распознаваемых последней версией Nmap, поддерживается по адресу nmap.org.

Выражения проверок

Выражения проверок не обязаны меняться между отпечатком субъекта и эталонным отпечатком, но почти всегда меняются. Эталонный отпечаток часто требуется немного обобщить, чтобы он совпадал со всеми экземплярами конкретной ОС, а не только со сканируемой машиной. Например, некоторые машины Windows XP возвращают размер окна F424 для пробы T1, тогда как другие возвращают FAF0. Причиной может быть конкретный используемый драйвер Ethernet или, возможно, объём доступной памяти. В любом случае мы хотим определять Windows XP независимо от используемого размера окна.

Один из способов обобщить отпечаток - просто удалить проверки, дающие непостоянные результаты. Удалите из эталонного отпечатка все проверки размера окна, и системы будут совпадать с этим отпечатком независимо от используемого размера. Недостаток состоит в возможной потере большого количества важных сведений. Если единственные размеры окна, когда-либо отправляемые конкретной системой, - F424 и FAF0, желательно разрешить только эти два значения, а не все 65 536 вариантов.

Хотя удаление проверок в некоторых ситуациях избыточно, в других оно полезно. Значение проверки R=Y, означающее наличие ответа, обычно удаляется из проверок U1 и IE перед их добавлением в `nmap-os-db`. Эти пробы часто блокируются межсетевым экраном, поэтому отсутствие ответа не должно снижать степень совпадения ОС.

Когда удалять проверки нежелательно, Nmap предлагает синтаксис выражений, позволяющий проверке совпадать с несколькими значениями. Например, `W=F424|FAF0` разрешит эти два значения окна Windows XP, не разрешая никаких других. В таблице 8.11 показаны допустимые операторы в значениях проверок.

Таблица 8.11. Операторы выражений проверок эталонного отпечатка

Название операции · символ · пример · описание

• **Или** — символ: `\|`; пример: `O=\|ME\|MNNTNW`. Совпадает, если соответствующая проверка отпечатка субъекта принимает значение любого из вариантов. В этом примере начальная вертикальная черта означает, что пустой список параметров тоже совпадёт.
• **Диапазон** — символ: `-`; пример: `SP=7-A`. Совпадает, если соответствующая проверка отпечатка субъекта выдаёт числовое значение в указанном диапазоне.
• **Больше** — символ: `>`; пример: `SP=>8`. Совпадает, если соответствующая проверка отпечатка субъекта выдаёт числовое значение больше указанного.
• **Меньше** — символ: `<`; пример: `GCD=<5`. Совпадает, если соответствующая проверка отпечатка субъекта выдаёт числовое значение меньше указанного.

В выражениях можно сочетать операторы, как в `GCD=<7|64|256|>1024`, которое совпадает, если GCD меньше семи, в точности равен 64, в точности равен 256 или больше 1024.

8.6. Алгоритмы сопоставления ОС

Алгоритм Nmap для обнаружения совпадений относительно прост. Он берёт отпечаток субъекта и проверяет его по очереди против каждого эталонного отпечатка в `nmap-os-db`.

При проверке по эталонному отпечатку Nmap по очереди рассматривает каждую строку категории пробы из отпечатка субъекта (например, SEQ или T1). Все строки проб, которых нет в эталонном отпечатке, пропускаются. Если в эталонном отпечатке есть совпадающая строка, они сравниваются.

Для сравнения строки пробы Nmap по очереди исследует каждую отдельную проверку (R, DF, W и так далее) в строке категории субъекта. Все проверки, которых нет в эталонной строке, пропускаются. Когда обнаружена совпадающая проверка, Nmap увеличивает аккумулятор `PossiblePoints` на число баллов, назначенных этой проверке. Затем сравниваются значения проверки. Если эталонная проверка имеет пустое значение, проверка субъекта совпадает, только если её значение тоже пусто. Если эталонная проверка - простая строка или число (без операторов), проверка субъекта должна совпасть с ней в точности. Если эталонная строка содержит операторы (`|`, `-`, `>` или `<`), субъект должен совпасть так, как описано в разделе «Выражения проверок». Если проверка совпадает, аккумулятор `NumMatchPoints` увеличивается на вес этой проверки в баллах.

После проверки всех строк проб для одного отпечатка Nmap делит `NumMatchPoints` на `PossiblePoints`. Результат - коэффициент уверенности, описывающий вероятность совпадения отпечатка субъекта с конкретным эталонным отпечатком. Он рассматривается как доля единицы, поэтому 1,00 - идеальное совпадение, а 0,95 - очень близкое.

Вес проверок в баллах назначается специальной записью `MatchPoints` (которая может встречаться лишь один раз) в `nmap-os-db`. Эта запись очень похожа на обычный отпечаток, но вместо результатов каждой проверки содержит их веса в баллах (неотрицательные целые числа). Проверки, перечисленные в структуре `MatchPoints`, применяются только тогда, когда находятся в той же проверке, в которой перечислены. Поэтому значение, заданное для проверки W (размер

окна) в T1, не влияет на проверку W в T3. Пример структуры `MatchPoints` приведён в примере 8.7.

Пример 8.7. Структура MatchPoints

```
MatchPoints
SEQ(SP=25%GCD=75%ISR=25%TI=100%II=100%SS=80%TS=100)
OPS(O1=20%O2=20%O3=20%O4=20%O5=20%O6=20)
WIN(W1=15%W2=15%W3=15%W4=15%W5=15%W6=15)
ECN(R=100%DF=20%T=15%TG=15%W=15%O=15%CC=100%Q=20)
T1(R=100%DF=20%T=15%TG=15%S=20%A=20%F=30%RD=20%Q=20)
T2(R=80%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T3(R=80%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T4(R=100%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T5(R=100%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T6(R=100%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T7(R=80%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
U1(R=50%DF=20%T=15%TG=15%TOS=50%IPL=100%UN=100%RIPL=100%RID=100%RIPCK=100%
   RUCK=100%RUL=100%RUD=100)
IE(R=50%DFI=40%T=15%TG=15%TOSI=25%CD=100%SI=100%DLI=100)
```

После оценки всех эталонных отпечатков Nmap упорядочивает их и печатает идеальные совпадения (если их не слишком много). Если идеальных совпадений нет, но некоторые очень близки, Nmap может напечатать их. Предположения печатаются с большей вероятностью, если задан параметр `--osscan-guess`.

8.7. Работа с неверно распознанными и нераспознанными узлами

Хотя база данных Nmap огромна, он не может определить всё. У Nmap нет шансов определить большинство тостеров, холодильников, стульев или автомобилей, поскольку у них нет стека IP. И всё же, учитывая постоянно растущий список подключённых устройств, я бы не исключал ни одно из них. База отпечатков Nmap включает множество игровых приставок, телефонов, термометров, камер, интерактивных игрушек и медиапроигрывателей.

Наличие IP-адреса необходимо, но недостаточно для гарантии правильного отпечатка. Nmap всё ещё может предположить неверно либо вообще не сделать никакого предположения. Вот несколько советов по улучшению результатов:

Обновитесь до последней версии Nmap

Многие дистрибутивы Linux и другие операционные системы поставляются с древними версиями Nmap. База ОС Nmap улучшается почти с каждым выпуском, поэтому проверьте номер своей версии, выполнив `nmap -V`, а затем сравните его с последней доступной по адресу nmap.org. Установка новейшей версии на большинстве платформ занимает лишь несколько минут.

Сканируйте все порты

Когда Nmap обнаруживает проблемы определения ОС определённого узла, он выдаёт предупреждения. Одно из самых распространённых: `Warning: OS detection will be MUCH less reliable because we did not find at least 1 open and 1 closed TCP port`. Возможно, такие порты действительно отсутствуют на машине, но повторное сканирование с `-p-` для проверки всех портов может найти отвечающие порты, пригодные для определения ОС. Дополнительное сканирование UDP (`-sU`) способно помочь ещё больше, хотя существенно замедлит сканирование.

Попробуйте более смелое предположение

Если Nmap сообщает об отсутствии достаточно близких совпадений для печати, вероятно, что-то не так. Возможно, находящийся на пути межсетевой экран или устройство NAT изменяет пакеты проб или ответов. Это может вызвать гибридную ситуацию, когда одна группа проверок выглядит пришедшей от одной ОС, а другой набор выглядит совершенно иначе. Добавление `--osscan-guess`

может дать больше подсказок о том, что работает на машине.

Сканируйте из другого места

Чем больше сетевых переходов должен пройти пакет, чтобы достичь цели, тем выше вероятность, что сетевое устройство изменит (или отбросит) пробу либо ответ. Шлюзы NAT, межсетевые экраны и особенно перенаправление портов могут запутать определение ОС. Если вы сканируете IP-адрес устройства балансировки нагрузки, которое просто перенаправляет пакеты разнородной сети серверов, даже неясно, каким должен быть «правильный» результат определения ОС.

Многие интернет-провайдеры фильтруют трафик к «плохим» портам, а другие используют прозрачные прокси для перенаправления определённых портов на собственные серверы. Порт 25 или 80, который вы считаете открытым на цели, на самом деле может быть подменён вашим интернет-провайдером для подключения к его прокси-серверам. Ещё одно поведение, способное запутать определение ОС, - подмена межсетевыми экранами пакетов сброса TCP так, будто они пришли от узла назначения. Особенно часто это происходит с портом 113 (identd). И подмену сбросов, и прозрачные прокси часто можно обнаружить, заметив, что такое поведение демонстрирует каждая машина целевой сети, даже те, которые во всех прочих отношениях выглядят неработающими. Обнаружив подобную бессмыслицу, обязательно исключите эти порты из сканирования, чтобы они не исказили результаты. Возможно, стоит также попробовать сканирование из совершенно другого участка сети. Чем ближе вы к цели, тем точнее будут результаты. В идеальном случае цель всегда следовало бы сканировать из того же сегмента сети, в котором она находится.

8.7.1. Когда Nmap предполагает неверно

Иногда Nmap сообщает предположение об ОС, которое, как вам известно, неверно. Ошибки обычно незначительны (например, машина с Linux 2.4.16 определяется как «Linux kernel 2.4.8 - 2.4.15»), но сообщалось и о совершенно ошибочных результатах Nmap (например, ваш веб-сервер определён как принтер AppleWriter). Столкнувшись с такими проблемами (незначительными или серьёзными), сообщите о них, чтобы пользу получили все. База Nmap настолько полна лишь потому, что тысячи пользователей потратили по несколько минут на отправку новых сведений. Следуйте этим инструкциям:

Используйте недавнюю версию Nmap

Выполните `nmap -V`, чтобы определить имеющуюся версию Nmap. Необязательно использовать самую последнюю версию Nmap (хотя это было бы идеально), но убедитесь, что версия не ниже 4.20, поскольку нам нужны только отпечатки ОС второго поколения, а не старого формата из предыдущих версий. Последнюю доступную версию Nmap можно узнать, посетив nmap.org. После обновления может оказаться, что ошибка распознавания уже исправлена.

Будьте абсолютно уверены, что знаете, что работает на машине

Недействительные «исправления» могут повредить базу ОС. Если вы точно не уверены, что работает на удалённой машине, выясните это перед отправкой.

Создайте отпечаток

Выполните команду `nmap -O -sSU -F -T4 -d <target>`, где `<target>` - неверно распознанная система. Просмотрите результаты определения ОС и убедитесь, что неверное распознавание всё ещё присутствует.

Если в выводе Nmap с результатами ОС узла написано (`JUST GUESSING`), небольшая неточность результатов ожидаема. В этом случае не отправляйте исправление.

В противном случае команда Nmap должна была выдать результаты, включающие строку `OS fingerprint:`. Под ней расположен отпечаток (ряд строк, каждая из которых начинается с `OS:`).

Проверьте работу определения ОС на других узлах

Попробуйте просканировать ещё пару узлов целевой сети, которые, как вам известно, имеют другую ОС. Если они определяются неправильно, возможно, между системами есть сетевое препятствие, повреждающее пакеты.

Если вы дошли до этого места и по-прежнему можете отправить данные - отлично! Отправьте сведения по адресу insecure.org.

8.7.2. Когда Nmap не находит совпадения и печатает отпечаток

Когда Nmap обнаруживает, что условия определения ОС кажутся идеальными, но не находит точных совпадений, он печатает примерно такое сообщение:

```
No OS matches for host (If you know what OS is running on it, see
http://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=4.62%D=5/20%OT=21%CT=1%CU=42293%PV=Y%DS=1%G=Y%M=008077%TM=48336D6
OS:D%P=i686-pc-linux-gnu)SEQ(SP=11%GCD=1E848%ISR=A4%TI=I%II=I%SS=S%TS=A)OPS
OS:(O1=M5B4NW0NNSNNT11%O2=M578NW0NNSNNT11%O3=M280NW0NNT11%O4=M5B4NW0NNSNNT1
OS:1%O5=M218NW0NNSNNT11%O6=M109NNSNNT11)WIN(W1=21F0%W2=2088%W3=2258%W4=21F0
OS:%W5=20C0%W6=209D)ECN(R=Y%DF=N%T=40%W=2238%O=M5B4NW0NNS%CC=N%Q=)T1(R=Y%DF
OS:=N%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=Y%DF=N%T=40%W=209D%S=O%A=S+%F=
OS:AS%O=M109NW0NNSNNT11%RD=0%Q=)T4(R=Y%DF=N%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=
OS:)T5(R=Y%DF=N%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=N%T=40%W=0%S=A%
OS:A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=N%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%
OS:DF=N%T=FF%TOS=0%IPL=38%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUL=G%RUD=G)IE(R
OS:=Y%DFI=N%T=FF%TOSI=Z%CD=S%SI=S%DLI=S)
```

Рассмотрите возможность отправить отпечаток, чтобы пользу получили все пользователи Nmap. Это займёт лишь минуту или две и может означать, что при сканировании узла следующей версией Nmap вам больше не придётся видеть это уродливое сообщение! Просто посетите предоставленный Nmap URL и следуйте инструкциям.

Если Nmap не находит совпадений, но не печатает отпечаток, условия не были идеальными. Даже если вы получите отпечаток через режим отладки или XML-вывод, не отправляйте его, если Nmap не просит об этом (как в предыдущем примере).

8.7.3. Самостоятельное изменение базы данных nmap-os-db

Люди часто спрашивают о самостоятельном включении отпечатка вместо его отправки на Nmap.Org (или вдобавок к ней). Хотя мы не предлагаем подробных инструкций или сценариев для этого, такое, безусловно, возможно после глубокого знакомства с разделом 8.5 «Понимание отпечатка Nmap». Надеюсь, это пригодится для ваших целей, но отправлять нам созданные вами эталонные отпечатки не нужно. Мы можем включать лишь необработанные отпечатки субъекта, отправленные через веб-форму.

8.8. РЕШЕНИЕ: обнаружение несанкционированных беспроводных точек доступа в корпоративной сети

8.8.1. Задача

Из-за повсеместного распространения мобильных устройств и дешёвого массового сетевого оборудования компании всё чаще обнаруживают, что сотрудники нежелательным образом расширяют их сети. Среди самых опасных устройств - беспроводные точки доступа 802.11 (WAP). Пользователь может установить в своей кабинке WAP за 20 долларов, чтобы работать из комнаты отдыха, не осознавая (или не заботясь о том), что только что открыл защищённую корпоративную сеть потенциальным злоумышленникам на парковке или в соседних зданиях.

Некоторые установки WAP ещё хуже тех, которые выполняют наивные пользователи. Нарушить физическую защиту здания для атакующего гораздо рискованнее, чем получить удалённый доступ к корпоративным данным по сети: существует риск быть арестованным на месте. Поэтому известны случаи, когда атакующие устанавливали компактные WAP, чтобы затем по своему желанию проникать в сеть из относительной безопасности автомобиля, припаркованного дальше по улице. WAP, приклеенная под столом или спрятанная иным образом, скорее всего, некоторое время останется незамеченной.

Хотя это решение сосредоточено на поиске WAP, ту же стратегию можно использовать для поиска почти чего угодно. Возможно, вам потребуется найти все маршрутизаторы Cisco, чтобы применить новое исправление, или все машины Solaris, чтобы определить, достаточно ли у вас систем для оправдания оплаты поддержки.

Один из способов найти несанкционированные беспроводные устройства - обследовать территорию беспроводным анализатором наподобие Kismet[6] или NetStumbler[7]. Другой подход - просканировать проводную сторону с помощью Nmap. Неудивительно, что это решение сосредоточено исключительно на втором подходе. Каждый метод может пропустить некоторые WAP, поэтому лучше всего применить оба и объединить результаты.

8.8.2. Решение

Просканируйте всё адресное пространство с параметром `-A`. Можно ускорить процесс, ограничив сканируемые порты диапазонами 1-85, 113, 443 и 8080-8100. На большинстве WAP этого должно быть достаточно, чтобы найти как открытый, так и закрытый порт, что повышает точность определения ОС. Если сеть охватывает несколько сегментов Ethernet, сканируйте каждый сегмент с выделенной машины в том же сегменте. Это ускоряет сканирование (особенно потому, что их можно выполнять параллельно), а также даёт MAC-адрес каждого устройства. Сканирование из того же сегмента позволяет обнаружить и скрытные устройства. Даже WAP со всеми фильтруемыми портами обычно ответит на запрос ARP. Результаты следует сохранять по крайней мере в обычном формате и XML, поэтому можно сразу использовать `-oA`. Рассмотрите все параметры повышения производительности, описанные в главе 6 «Оптимизация производительности Nmap». Хорошее и относительно безопасное начало для параметров производительности: `-T4 --min-hostgroup 50 --max-rtt-timeout 1000 --initial-rtt-timeout 300 --max-retries 3 --host-timeout 20m --max-scan-delay 1000`. Всё вместе даёт примерно такую команду:

```
nmap -A -oA ~/nmap-logs/wapscan -p 1-85,113,443,8080-8100 -T4 --min-hostgroup 50 --max-
    rtt-timeout 1000 --initial-rtt-timeout 300 --max-retries 3 --host-timeout 20m --max-
    scan-delay 1000 <target_network>
```

После завершения сканирования ищите характеристики WAP. В сети с менее чем парой сотен работающих узлов лучше всего просмотреть каждый по отдельности. Для более крупных сетей задачу, вероятно, придётся автоматизировать. Искать отдельные характеристики можно с помощью grep, хотя предпочтителен сценарий Perl, анализирующий XML-вывод. Благодаря

существующим модулям разбора XML-вывода Nmap, например Nmap::Scanner и Nmap::Parser, это довольно просто. Примеры приведены в разделе 13.7 «Обработка XML-вывода с помощью Perl».

Определив список кандидатов, вероятно, лучше всего открыть файл обычного вывода Nmap и исследовать каждого, чтобы исключить ложноположительные результаты. Например, устройство Linksys может быть отмечено как возможная WAP, хотя на самом деле это может быть один из обычных коммутаторов компании без беспроводных возможностей.

После обнаружения WAP пора найти их физически. Обычно это можно сделать, запросив у коммутатора, к которому они подключены, номер их физического порта Ethernet.

8.8.3. Характеристики WAP

Теперь пора обсудить характеристики WAP, которые следует искать. Их понимание полезно для ручной проверки или изменения сценария поиска WAP с целью найти что-либо другое. Вероятно, многие из них вы сразу увидите в результатах сканирования типичной WAP из примера 8.8.

Пример 8.8. Результаты сканирования потребительской WAP

```
nmap -A -v wap.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on wap.nmap.org (192.168.0.6):
Not shown: 999 closed ports
PORT   STATE SERVICE VERSION
80/tcp open  http    Netgear MR-series WAP (MR814; Embedded HTTPD 1.00)
MAC Address: 00:09:5B:3F:7D:5E (Netgear)
Device type: WAP
Running: Compaq embedded, Netgear embedded
OS details: WAP: Compaq iPAQ Connection Point or Netgear MR814
Service Info: Device: WAP

Nmap done: 1 IP address (1 host up) scanned in 10.90 seconds
           Raw packets sent: 1703 (75.706KB) | Rcvd: 1686 (77.552KB)
```

Это устройство демонстрирует много очевидных признаков WAP (`Device type: WAP` особенно недвусмысленен) и несколько более тонких. Но WAP не всегда так легко обнаружить. В этом разделе приведён список характеристик WAP, начиная с самых сильных и заканчивая эвристиками, которые имеют мало шансов на успех или с большей вероятностью дают ложноположительные результаты. Каждая перечисленная характеристика сопровождается выражением XPath[8], показывающим, где найти её в XML-выводе Nmap. Поскольку это связано с безопасностью, я рекомендую попробовать их все, а ложноположительные результаты удалять вручную.

Тип устройства по отпечатку TCP/IP

Как описано в разделе «Классификация устройства и ОС (строки Class)», с каждым эталонным отпечатком связана как минимум одна классификация (включающая тип устройства). Поскольку WAP вызывают так много споров, когда подходят несколько типов, мы стараемся использовать именно WAP (или давать две классификации). Поэтому такие устройства, как беспроводной широкополосный маршрутизатор D-Link DI-624, классифицируются как WAP, а не как `switch` или `router`. Тип устройства можно найти в XML-выводе с помощью выражения XPath `/nmaprun/host/os/osclass/@type`. (То есть это атрибут `type` элемента `osclass` элемента `os` любого из элементов `host` внутри корневого элемента `nmaprun`.)

Подробности снятия отпечатка TCP/IP

Хотя устройства с беспроводными возможностями следует классифицировать с типом устройства WAP, на всякий случай стоит поискать в подробном описании ОС такие слова, как `wireless` или `wap`. Описание находится в XML-выводе по адресу `/nmaprun/host/os/osmatch/@name`.

Тип устройства по определению версий

Определение версий тоже пытается определить типы устройств, но делает это по отпечаткам работающих служб цели, а не по её стеку IP. Проверьте, равен ли WAP XML-атрибут `devicetype`, расположенный по адресу `/nmaprun/host/ports/port/service/@devicetype`. Для полной уверенности стоит также проверить поле `/nmaprun/host/ports/port/service/@extrainfo` на подстроки `wap` или `wireless`.

Поставщик (по MAC-адресу, отпечатку TCP/IP и определению версий)

Некоторые поставщики специализируются на производстве дешёвых потребительских сетевых устройств, которые с наибольшей вероятностью тайком попадут в офисные сети. Примеры: Linksys, Netgear, Belkin, SMC, D-Link, Motorola, Trendnet, Zyxel и Gateway. Этих поставщиков можно проверить по результатам поиска MAC-адреса (в XML-выводе это `/nmaprun/host/address/@vendor`), определения ОС (`/nmaprun/host/os/osclass/@vendor`) или определения версий (`/nmaprun/host/ports/port/service/@product`). Обязательно ищите поставщика как подстроку полей, поскольку поле может содержать тип юридического лица (например, Inc.) или другие сведения.

Эта проверка может дать много ложноположительных результатов. Если вы широко используете поставщика для разрешённых устройств, например устанавливаете сетевые платы Netgear в настольные машины, возможно, придётся удалить этого поставщика и повторно запустить сценарий.

Имя узла

Не помешает проверить имена узлов (обратное разрешение DNS) на такие слова, как `wap`, `wireless` или `airport`. Их можно найти в XML-выводе по адресу `/nmaprun/host/hostnames/hostname/@name`. Сотрудники без административных полномочий редко меняют имена DNS, но это может быть полезно специалистам по тестированию на проникновение, новым администраторам и другим людям, которые сканируют новую сеть в поисках разрешённых точек доступа.

Сноски

[1] [назад] Увлекательное визуальное представление этого явления доступно по адресу http://lcamtuf.coredump.cx/newtcp/
[2] [назад] lcamtuf.coredump.cx
[3] [назад] gomor.org
[4] [назад] sshwindows.sourceforge.net
[5] [назад] samba.org
[6] [назад] kismetwireless.net
[7] [назад] netstumbler.com
[8] [назад] w3.org

Глава 9. Движок сценариев Nmap

9.1. Введение

Движок сценариев Nmap (NSE) - одна из самых мощных и гибких возможностей Nmap. Он позволяет пользователям писать простые сценарии (и делиться ими) для автоматизации самых разных сетевых задач. Затем эти сценарии выполняются параллельно со скоростью и эффективностью, ожидаемыми от Nmap. Пользователи могут полагаться на растущий и разнообразный набор сценариев, распространяемых с Nmap, либо писать собственные для особых потребностей.

Мы проектировали NSE как универсальный инструмент, имея в виду следующие задачи:

Обнаружение сети

Это хлеб с маслом для Nmap. Среди примеров - поиск данных whois по домену цели; запрос к ARIN, RIPE или APNIC по IP-адресу цели для определения владельца; выполнение запросов identd к открытым портам; запросы SNMP; перечисление доступных общих ресурсов и служб NFS/SMB/RPC.

Более сложное определение версий

Система определения версий Nmap (глава 7 «Определение версий служб и приложений») способна распознавать тысячи различных служб с помощью системы сопоставления проб и сигнатур на основе регулярных выражений, но распознать всё она не может. Например, для определения службы Skype v2 требуются две независимые пробы, а определение версий недостаточно гибко для их обработки. Nmap также мог бы распознавать больше служб SNMP, если бы перебирал несколько сотен разных имён сообщества. Ни одна из этих задач не подходит традиционному определению версий Nmap, но обе легко решаются с помощью NSE. По этим причинам определение версий теперь по умолчанию вызывает NSE для обработки некоторых сложных служб. Это описано в разделе 9.10 «Определение версий с помощью NSE».

Обнаружение уязвимостей

Когда обнаруживается новая уязвимость, часто хочется быстро просканировать сети и найти уязвимые системы раньше злоумышленников. Хотя Nmap не является комплексным сканером уязвимостей, NSE достаточно мощен даже для требовательных проверок уязвимостей. Уже доступно много сценариев обнаружения уязвимостей, и по мере написания новых мы планируем распространять их.

Обнаружение бэкдоров

Многие атакующие и некоторые автоматические черви оставляют бэкдоры для последующего повторного проникновения. Часть из них можно обнаружить системой определения версий Nmap на

основе регулярных выражений. Например, через несколько часов после появления червя MyDoom в Интернете Джей Моран (Jay Moran) опубликовал пробу и сигнатуру определения версий Nmap, чтобы другие могли быстро сканировать свои сети на заражение MyDoom. Для надёжного обнаружения более сложных червей и бэкдоров необходим NSE.

Эксплуатация уязвимостей

Как универсальный язык сценариев NSE можно использовать даже для эксплуатации уязвимостей, а не только для их поиска. Возможность добавлять собственные сценарии эксплуатации может быть ценна для некоторых людей (особенно специалистов по тестированию на проникновение), хотя мы не планируем превращать Nmap в среду эксплуатации наподобие Metasploit[1].

Перечисленные пункты были нашими первоначальными целями, и мы ожидаем, что пользователи Nmap придумают ещё более изобретательные способы применения NSE.

Сценарии пишутся на встроенном языке программирования Lua[2]. Сам язык хорошо документирован в книгах *Programming in Lua, Second Edition* и *Lua 5.1 Reference Manual*. Справочное руководство также свободно доступно в Интернете[3], как и первое издание *Programming in Lua*[4]. Учитывая доступность этих превосходных общих источников по программированию на Lua, этот документ рассматривает только аспекты и расширения, характерные для движка сценариев Nmap.

NSE активируется параметром `-sC` (или `--script`, если вы хотите указать собственный набор сценариев), а результаты включаются в обычный и XML-вывод Nmap. Поддерживаются два типа сценариев: сценарии служб и сценарии узлов. Сценарии служб относятся к определённому открытому порту (службе) целевого узла, а все полученные ими результаты включаются рядом с этим портом в таблицу портов вывода Nmap. Сценарии узлов, напротив, выполняются не более одного раза для каждого целевого IP-адреса и выводят результаты под таблицей портов. В примере 9.1 показано типичное сканирование сценариями. Вывод в этом примере создают сценарии служб `ssh-hostkey`, предоставляющий ключи SSH RSA и DSA системы, и `rpcinfo`, опрашивающий portmapper для перечисления доступных служб. Единственный сценарий узла, создающий вывод в этом примере, - `smb-os-discovery`, собирающий разнообразные сведения с серверов SMB. Nmap обнаружил все эти сведения за треть секунды.

Пример 9.1. Типичный вывод NSE

```
# nmap -sC -p22,111,139 -T4 localhost

Starting Nmap ( http://nmap.org )
Interesting ports on flog (127.0.0.1):
PORT    STATE SERVICE
22/tcp  open  ssh
| ssh-hostkey: 1024 b1:36:0d:3f:50:dc:13:96:b2:6e:34:39:0d:9b:1a:38 (DSA)
|_ 2048 77:d0:20:1c:44:1f:87:a0:30:aa:85:cf:e8:ca:4c:11 (RSA)
111/tcp open  rpcbind
| rpcinfo:
| 100000  2,3,4    111/udp  rpcbind
| 100024  1      56454/udp  status
|_ 100000 2,3,4    111/tcp  rpcbind
139/tcp open  netbios-ssn

Host script results:
| smb-os-discovery: Unix
| LAN Manager: Samba 3.0.31-0.fc8
| Name: WORKGROUP
|_

Nmap done: 1 IP address (1 host up) scanned in 0.33 seconds
```

9.2. Использование и примеры

Хотя для эффективности NSE имеет сложную реализацию, пользоваться им поразительно легко. Просто укажите `-sC`, чтобы включить наиболее распространённые сценарии. Или задайте параметр `--script`, чтобы выбрать собственные сценарии для выполнения, указав категории, имена файлов сценариев или имена каталогов, полных сценариев, которые требуется выполнить. Некоторые сценарии можно настраивать, передавая им аргументы через параметр `--script-args`. Два остальных параметра, `--script-trace` и `--script-updatedb`, обычно используются только для отладки и разработки сценариев. Сканирование сценариями также входит в параметр `-A` (агрессивное сканирование).

9.2.1. Категории сценариев

Сценарии NSE определяют список категорий, к которым они принадлежат. В настоящее время определены категории `auth`, `default`, `discovery`, `external`, `intrusive`, `malware`, `safe`, `version` и `vuln`. Регистр символов в названиях категорий не учитывается. В следующем списке описана каждая категория.

auth

Эти сценарии пытаются определить учётные данные аутентификации в целевой системе, часто с помощью перебора. Примеры: `snmp-brute`, `http-auth` и `ftp-anon`.

default

Эти сценарии образуют набор по умолчанию и выполняются при использовании параметров `-sC` или `-A` вместо перечисления сценариев с помощью `--script`. Эту категорию, как и любую другую, можно также указать явно с помощью `--script=default`. При решении, следует ли запускать сценарий по умолчанию, учитывается множество факторов:

Скорость

Сканирование по умолчанию должно завершаться быстро, что исключает средства подбора аутентификационных данных, веб-пауков и любые другие сценарии, которым могут потребоваться минуты или часы для сканирования одной службы.

Полезность

Сканирования по умолчанию должны выдавать ценные сведения, на основании которых можно действовать. Если даже автор сценария затрудняется объяснить, почему рядовой специалист по сетям или безопасности сочтёт вывод ценным, сценарий не должен выполняться по умолчанию. Возможно, сценарий всё же стоит включить в Nmap, чтобы администраторы могли запускать его в тех случаях, когда дополнительные сведения им действительно нужны.

Подробность

Вывод Nmap используется для самых разных целей и должен оставаться удобочитаемым и лаконичным. Сценарий, часто выдающий целые страницы вывода, не следует добавлять в категорию `default`. Когда сообщить нечего важного, сценарии NSE (особенно выполняемые по умолчанию) не должны ничего возвращать. Проверять по умолчанию малоизвестную уязвимость допустимо, если вывод создаётся только при её обнаружении.

Надёжность

Многие сценарии используют эвристики и нечёткое сопоставление сигнатур, чтобы делать выводы о целевом узле или службе. Примеры: `sniffer-detect` и `sql-injection`. Если сценарий часто ошибается, ему не место в категории `default`, где он может запутать или ввести в заблуждение обычных пользователей. Пользователи, явно указывающие сценарий или категорию, обычно более опытны и, вероятно, знают, как работает сценарий, или хотя бы где найти его документацию.

Агрессивность

Некоторые сценарии очень агрессивны, поскольку используют значительные ресурсы удалённой системы, могут вызвать сбой системы или службы либо, вероятно, будут восприняты удалёнными администраторами как атака. Чем агрессивнее сценарий, тем меньше он подходит для категории `default`.

Конфиденциальность

Некоторые сценарии, особенно относящиеся к описанной далее категории `external`, по самой своей природе раскрывают сведения третьим сторонам. Например, сценарий `whois` должен раскрывать IP-адрес цели региональным реестрам whois. Мы также рассматривали (и решили отказаться от) добавление сценариев, которые сверяют отпечатки ключей SSH и SSL цели с интернет-базами слабых ключей. Чем сильнее сценарий вторгается в конфиденциальность, тем меньше он подходит для включения в категорию `default`.

Для каждого из этих критериев у нас нет точных порогов, и многие из них субъективны. Все факторы рассматриваются вместе при принятии решения, повышать ли сценарий до категории `default`. Несколько сценариев по умолчанию: `identd-owners` (определяет с помощью identd имя пользователя, от которого работают удалённые службы), `http-auth` (получает схему аутентификации и область веб-сайтов, требующих аутентификации) и `ftp-anon` (проверяет, разрешает ли FTP-сервер анонимный доступ).

discovery

Эти сценарии пытаются активно узнать больше о сети, опрашивая публичные реестры, устройства с поддержкой SNMP, службы каталогов и тому подобное. Примеры: `html-title` (получает заголовок корневого пути веб-сайтов), `smb-enum-shares` (перечисляет общие ресурсы Windows) и `snmp-sysdescr` (извлекает подробности системы через SNMP).

external

Сценарии этой категории могут отправлять данные сторонней базе данных или другому сетевому ресурсу. Пример - `whois`, который подключается к серверам whois, чтобы получить сведения об адресе цели. Всегда существует вероятность, что операторы сторонней базы запишут всё отправленное вами, что во многих случаях включает ваш IP-адрес и адрес цели. Большинство сценариев создают трафик исключительно между сканирующим компьютером и клиентом; все остальные помещаются в эту категорию.

intrusive

Это сценарии, которые нельзя отнести к категории `safe`, поскольку слишком велик риск, что они вызовут сбой целевой системы, израсходуют значительные ресурсы целевого узла (например, пропускную способность или процессорное время) либо иным образом будут восприняты системными администраторами цели как вредоносные. Примеры: `http-open-proxy` (пытается использовать целевой сервер как HTTP-прокси) и `snmp-brute` (пытается угадать строку сообщества SNMP устройства, отправляя распространённые значения, такие как `public`, `private` и `cisco`).

malware

Эти сценарии проверяют, заражена ли целевая платформа вредоносной программой или бэкдором. Примеры: `smtp-strangeport`, ищущий SMTP-серверы на необычных номерах портов, и `auth-spoof`, обнаруживающий демоны подмены identd, которые выдают фальшивый ответ ещё до получения запроса. Оба вида поведения обычно связаны с заражением вредоносными программами.

safe

Сценарии, которые не были предназначены для аварийного завершения служб, использования больших объёмов сетевой пропускной способности или других ресурсов либо эксплуатации уязвимостей безопасности, относятся к категории `safe`. Они с меньшей вероятностью возмутят удалённых администраторов, хотя (как и для всех остальных возможностей Nmap) мы не можем гарантировать, что они никогда не вызовут неблагоприятной реакции. Большинство из них выполняет общее обнаружение сети. Примеры: `ssh-hostkey` (получает ключ SSH узла) и `html-title` (извлекает заголовок веб-страницы).

version

Сценарии этой особой категории являются расширением возможности определения версий и не могут быть выбраны явно. Они выбираются для выполнения, только если запрошено определение версий (`-sV`). Их вывод неотличим от вывода определения версий, и они не создают результатов сценариев служб или узлов. Примеры: `skypev2-version`, `pptp-version` и `iax2-version`.

vuln

Эти сценарии проверяют конкретные известные уязвимости и обычно сообщают результаты, только если уязвимости найдены. Примеры: `realvnc-auth-bypass` и `xampp-default-auth`.

9.2.2. Аргументы командной строки

Ниже приведены пять аргументов командной строки, относящихся к сканированию сценариями:

-sC

Выполняет сканирование сценариями с набором сценариев по умолчанию. Эквивалентен `--script=default`. Некоторые сценарии категории `default` считаются агрессивными, и их не следует запускать против целевой сети без разрешения.

--script <script-categories>|<directory>|<filename>|all

Выполняет сканирование сценариями (как `-sC`), используя вместо набора по умолчанию разделённый запятыми список категорий сценариев, отдельных сценариев или каталогов со сценариями. Сначала Nmap пытается интерпретировать аргументы как категории, а затем (если это не удалось) как файлы или каталоги. Сценарий или каталог сценариев можно указать абсолютным или относительным путём. Абсолютные пути используются как есть. Относительные пути ищутся до первого совпадения в следующих местах: `--datadir/`; `$NMAPDIR/`; `~/.nmap/` (не ищется в Windows); `NMAPDATADIR/` или `./`. В каждом из них также проверяется подкаталог `scripts/`.

Если каталог указан и найден, Nmap загружает из него все сценарии NSE (все имена файлов, оканчивающиеся на `.nse`). Имена файлов без расширения `nse` игнорируются. Nmap не ищет сценарии рекурсивно во вложенных каталогах. Если указаны отдельные имена файлов, расширение файла необязательно должно быть `nse`.

По умолчанию сценарии Nmap хранятся в подкаталоге `scripts` каталога данных Nmap (см. главу 14 «Понимание и настройка файлов данных Nmap»). Для эффективности сценарии индексируются в базе `scripts/script.db`, где перечислены категории, к которым относится каждый сценарий. Чтобы выполнить все сценарии из базы сценариев Nmap, передайте аргумент `all`.

Сценарии не выполняются в песочнице и потому могут случайно или намеренно повредить систему либо нарушить вашу конфиденциальность. Никогда не запускайте сторонние сценарии, если не доверяете их авторам или не провели тщательный аудит сценариев самостоятельно.

--script-args

Передаёт аргументы сценариям. Подробное объяснение приведено в разделе 9.2.3 «Аргументы сценариев».

--script-trace

Этот параметр похож на `--packet-trace`, но работает на уровне приложений, а не отдельных пакетов. Если он указан, печатаются все входящие и исходящие сеансы связи, выполняемые сценариями. Показанные сведения включают протокол связи, адреса источника и цели, а также

переданные данные. Если более 5% переданных данных непечатаемы, вместо них приводятся шестнадцатеричные дампы. Указание `--packet-trace` также включает трассировку сценариев.

--script-updatedb

Этот параметр обновляет базу сценариев `scripts/script.db`, которую Nmap использует для определения доступных сценариев по умолчанию и категорий. Обновлять базу требуется, только если вы добавили или удалили сценарии NSE из каталога сценариев по умолчанию либо изменили категории какого-либо сценария. Параметр используется отдельно, без аргументов: `nmap --script-updatedb`.

Некоторые другие параметры Nmap влияют на сканирование сценариями. Наиболее заметный из них - `-sV`. Сканирование версий автоматически выполняет сценарии категории `version`. Сценарии этой категории немного отличаются от остальных: их вывод сливается с результатами сканирования версий, и они не создают вывода сканирования сценариями.

Другой параметр, влияющий на движок сценариев, - `-A`. Агрессивный режим Nmap подразумевает параметр `-sC`.

9.2.3. Аргументы сценариев

Аргументы можно передавать сценариям NSE с помощью параметра `--script-args`. Аргументы сценариев обычно представляют собой пары «имя-значение». Сценариям они предоставляются как таблица Lua с именем `args` внутри `nmap.registry`. Имена аргументов являются ключами соответствующих значений. Значения могут быть строками или таблицами. Вложенные таблицы можно использовать для более точной передачи аргументов сценариям, например для передачи разным сценариям разных имён пользователей. Вот типичный вызов Nmap с аргументами сценариев:

```
$ nmap -sC --script-args user=foo,pass=bar,whois={whodb=nofollow+ripe}
```

Приведённая команда создаёт следующую таблицу Lua:

```
{user="foo",pass="bar",whois={whodb="nofollow+ripe"}}
```

Поэтому получить имя пользователя (`foo`) внутри сценария можно таким оператором:

```
local username = nmap.registry.args.user
```

Вложенные таблицы, используемые для переопределения параметров сценариев, обычно называют по имени сценария, чтобы облегчить их получение.

9.2.4. Примеры использования

Простое сканирование сценариями с набором сценариев по умолчанию:

```
$ nmap -sC example.com
```

Выполнение определённого сценария с включённой трассировкой:

```
$ nmap --script=./showSSHVersion.nse --script-trace example.com
```

Выполнение всех сценариев из каталога `mycustomscripts`, а также всех сценариев по умолчанию из категории `safe`:

```
$ nmap --script=mycustomscripts,safe example.com
```

9.3. Формат сценария

Сценарии NSE состоят из двух-пяти описательных полей, правила порта или узла, определяющего, когда следует выполнять сценарий, и блока действия, содержащего собственно инструкции сценария. Описательным полям можно присваивать значения так же, как любым другим переменным Lua. Их имена должны быть записаны строчными буквами, как показано в этом разделе.

9.3.1. Поле description

Поле `description` описывает, что проверяет сценарий, и все важные замечания, о которых должен знать пользователь. В зависимости от сложности сценария описание может занимать от нескольких предложений до нескольких абзацев. Первый абзац должен быть кратким изложением функции сценария, пригодным для самостоятельного представления пользователю. Последующие абзацы могут содержать гораздо более подробное описание сценария.

9.3.2. Поле categories

Поле `categories` определяет одну или несколько категорий, к которым относится сценарий (см. раздел 9.2.1 «Категории сценариев»). Регистр символов в категориях не учитывается, и их можно указывать в любом порядке. Они перечисляются в таблице Lua в стиле массива, как в следующем примере:

```
categories = {"default", "discovery", "safe"}
```

9.3.3. Поле author

Поле `author` содержит имена авторов сценария и контактные сведения. Если вы опасаетесь спама, можете опустить или замаскировать адрес электронной почты либо вместо него дать URL домашней страницы. Это необязательное поле не используется NSE, но отдаёт авторам сценария должное признание или возлагает на них вину.

9.3.4. Поле license

Nmap - общественный проект, и мы приветствуем любые вклады в код, включая сценарии NSE. Поэтому, написав ценный сценарий, не держите его при себе! Необязательное поле `license` помогает удостовериться, что у нас есть законное разрешение распространять все сценарии, поставляемые с Nmap. Сейчас все эти сценарии используют стандартную лицензию Nmap (описанную в разделе 15.19.1 «Авторские права и лицензирование Nmap»). Они содержат следующую строку:

```
license = "Same as Nmap--See http://nmap.org/book/man-legal.html"
```

Лицензия Nmap похожа на GNU GPL. При желании авторы сценариев могут вместо неё использовать лицензию в стиле BSD (без рекламного положения).

9.3.5. Поле runlevel

Это необязательное поле определяет порядок выполнения сценариев. При отсутствии этого раздела уровень выполнения по умолчанию равен 1,0. Для одной целевой машины сценарии с заданным `runlevel` выполняются после всех сценариев с меньшим `runlevel` и до всех сценариев с большим уровнем выполнения. Порядок сценариев с одинаковым `runlevel` не определён, и часто они выполняются параллельно. Одно из применений уровней выполнения - возможность зависимости сценариев друг от друга. Если сценарий A полагается на сведения, собранные сценарием B, назначьте B более низкий уровень выполнения, чем A. Сценарий B может сохранить сведения в реестре NSE, чтобы A получил их позже. Сведения о реестре NSE приведены в разделе 9.7.5 «Реестр».

9.3.6. Правила портов и узлов

Nmap использует правила сценариев, чтобы определить, следует ли выполнять сценарий для цели. Сценарий содержит либо правило порта, управляющее тем, для каких портов цели можно выполнять сценарий, либо правило узла, указывающее, что сценарий следует выполнить для целевого IP-адреса лишь один раз и только при выполнении заданных условий. Правило - функция Lua, возвращающая `true` или `false`. Действие сценария выполняется, только если его правило вычисляется как `true`. Правила узлов принимают таблицу узла как аргумент и могут проверять, например, IP-адрес или имя цели. Правило порта принимает таблицы узла и порта как аргументы для любого TCP- или UDP-порта в состоянии `open`, `open|filtered` или `unfiltered`. При решении, запускаться ли для порта, правила портов обычно проверяют такие факторы, как номер порта, состояние порта или имя прослушивающей службы. Примеры правил показаны в разделе 9.8.2 «Правило».

9.3.7. Действие

Действие - сердце сценария NSE. Оно содержит все инструкции, которые должны выполняться при срабатывании правила порта или узла сценария. Это функция Lua, принимающая те же аргументы, что и правило, и способная вернуть `nil` или строку. Если строку возвращает сценарий службы, строка и имя файла сценария печатаются в таблице портов вывода Nmap. Строка, возвращённая сценарием узла, печатается под таблицей портов. Если сценарий возвращает `nil`, вывод не создаётся. Пример действия NSE см. в разделе 9.8.3 «Механизм».

9.4. Язык сценариев

Ядро движка сценариев Nmap - встраиваемый интерпретатор Lua. Lua - лёгкий язык, разработанный для расширяемости. Он предлагает мощный и хорошо документированный API для взаимодействия с другим программным обеспечением, таким как Nmap.

Вторая часть движка сценариев Nmap - библиотека NSE, связывающая Lua и Nmap. Этот слой решает такие вопросы, как инициализация интерпретатора Lua, планирование параллельного выполнения сценариев, получение сценариев и многое другое. Он также является ядром среды сетевого ввода-вывода NSE и механизма обработки исключений. Кроме того, он включает вспомогательные библиотеки, делающие сценарии мощнее и удобнее. Модули и расширения

вспомогательных библиотек описаны в разделе 9.6 «Библиотеки NSE».

9.4.1. Базовый язык Lua

Язык сценариев Nmap представляет собой встроенный интерпретатор Lua[5], расширенный библиотеками для взаимодействия с Nmap. API Nmap находится в пространстве имён Lua `nmap`. Это означает, что все вызовы предоставляемых Nmap ресурсов имеют префикс `nmap`. Например, `nmap.new_socket()` возвращает новый объект-обёртку сокета. Библиотечный слой Nmap также заботится об инициализации контекста Lua, планировании параллельных сценариев и сборе вывода завершённых сценариев.

На этапе планирования мы рассматривали несколько языков программирования в качестве основы сценариев Nmap. Другой вариант состоял в реализации совершенно нового языка программирования. Наши критерии были строгими: NSE должен был быть простым в использовании, небольшим, совместимым с лицензией Nmap, масштабируемым, быстрым и допускающим распараллеливание. Несколько прежних попыток (в других проектах) спроектировать с нуля собственный язык аудита безопасности привели к неудобным решениям, поэтому мы сразу решили не идти этим путём. Сначала рассматривался интерпретатор Guile Scheme, но предпочтение сместилось к интерпретатору Elk из-за более благоприятной лицензии. Однако распараллелить сценарии Elk было бы трудно. Кроме того, мы ожидаем, что большинство пользователей Nmap предпочитают процедурное программирование функциональным языкам наподобие Scheme. Более крупные интерпретаторы, такие как Perl, Python и Ruby, хорошо известны и любимы, но их трудно эффективно встраивать. В итоге Lua превзошёл все наши критерии. Он мал, распространяется под либеральной открытой лицензией MIT, имеет сопрограммы для эффективного параллельного выполнения сценариев, спроектирован с расчётом на встраивание, обладает превосходной документацией и активно развивается большим и преданным сообществом. Теперь Lua встроен даже в другие популярные открытые инструменты безопасности, включая анализатор трафика Wireshark и IDS Snort.

9.5. Сценарии NSE

В этом разделе в алфавитном порядке перечислены все сценарии NSE, поставлявшиеся с Nmap на момент написания книги. Список получен непосредственно из исходного кода сценариев благодаря системе документации NSEDoc, описанной в разделе 9.9 «Написание документации сценариев (NSEDoc)». Разумеется, бумажная документация не может оставаться актуальной для программного обеспечения, развивающегося так же активно, как NSE. Самую полную и актуальную документацию смотрите на интернет-портале документации NSE по адресу nmap.org.

asn-query.nse

Категории: discovery, external

Сопоставляет IP-адреса номерам автономных систем (AS).

Сценарий работает, отправляя запросы DNS TXT DNS-серверу, который, в свою очередь, опрашивает стороннюю службу Team Cymru (`team-cymru.org`) с помощью зоны в стиле `in-addr.arpa`, созданной специально для использования Nmap.

Ответы на эти запросы содержат ASN происхождения и ASN соседа с их описаниями, которые выводятся вместе с префиксом BGP и кодом страны.

Сценарий кэширует результаты, чтобы уменьшить число запросов, и должен выполнять один запрос для всех сканируемых целей в префиксе BGP, присутствующем в базе Team Cymru.

Помните, что все цели, для которых выполняется этот сценарий, будут отправлены одному или нескольким DNS-серверам и Team Cymru и потенциально записаны ими. Кроме того, ваш IP-адрес будет отправлен DNS-серверу (серверу DNS по умолчанию или указанному вами аргументом сценария `dns`) вместе с ASN.

Аргументы сценария

dns

Адрес используемого рекурсивного сервера имён (необязательно).

Использование

```
nmap --script asn-query.nse [--script-args dns=<DNS server>] <target>
```

Пример вывода

```
Host script results:
| asn-query:
| BGP: 64.13.128.0/21 | Country: US
|   Origin AS: 10565 SVCOLO-AS - Silicon Valley Colocation, Inc.
|   Peer AS: 3561 6461
| BGP: 64.13.128.0/18 | Country: US
|   Origin AS: 10565 SVCOLO-AS - Silicon Valley Colocation, Inc.
|_  Peer AS: 174 2914 6461
```

auth-owners.nse

Категории: default, safe

Пытается определить владельца открытого TCP-порта, опрашивая демон auth (identd - порт 113), который также должен быть открыт.

Пример вывода

```
21/tcp  open  ftp        ProFTPD 1.3.1
|_ auth-owners: nobody
22/tcp  open  ssh        OpenSSH 4.3p2 Debian 9etch2 (protocol 2.0)
|_ auth-owners: root
25/tcp  open  smtp       Postfix smtpd
|_ auth-owners: postfix
80/tcp  open  http       Apache httpd 2.0.61 ((Unix) PHP/4.4.7 ...)
|_ auth-owners: dhapache
113/tcp open  auth?
|_ auth-owners: nobody
```

```
587/tcp open  submission Postfix smtpd
|_ auth-owners: postfix
5666/tcp open  unknown
|_ auth-owners: root
```

auth-spoof.nse

Категории: malware

Проверяет наличие сервера identd (auth), подделывающего свои ответы.

Проверяет, отвечает ли сервер identd (auth) ещё до того, как мы отправим запрос. Такая подделка ответов identd может быть признаком заражения вредоносной программой, хотя она также может использоваться из законных соображений конфиденциальности.

daytime.nse

Категории: discovery

Получает дату и время из службы Daytime по UDP.

dns-random-srcport.nse

Категории: external, intrusive

Проверяет DNS-сервер на уязвимость рекурсии с предсказуемым портом. Предсказуемые исходные порты могут сделать DNS-сервер уязвимым для атак с отравлением кэша (см. CVE-2008-1447).

Сценарий работает, опрашивая `porttest.dns-oarc.net`. Имейте в виду, что любые цели, против которых выполняется этот сценарий, будут отправлены одному или нескольким DNS-серверам и серверу porttest и потенциально записаны ими. Кроме того, ваш IP-адрес будет отправлен вместе с запросом porttest DNS-серверу, работающему на цели.

dns-random-txid.nse

Категории: external, intrusive

Проверяет DNS-сервер на уязвимость рекурсии DNS с предсказуемым TXID. Предсказуемые значения TXID могут сделать DNS-сервер уязвимым для атак с отравлением кэша (см. CVE-2008-1447).

Сценарий работает, опрашивая `txidtest.dns-oarc.net`. Имейте в виду, что любые цели, против которых выполняется этот сценарий, будут отправлены одному или нескольким DNS-серверам и серверу txidtest и потенциально записаны ими. Кроме того, ваш IP-адрес будет отправлен вместе с запросом txidtest DNS-серверу, работающему на цели.

dns-recursion.nse

Категории: default, intrusive

Проверяет, разрешает ли DNS-сервер запросы имён сторонних узлов.

Предполагается, что рекурсия будет включена на ваших собственных внутренних серверах имён.

dns-zone-transfer.nse

Категории: default, intrusive, discovery

Запрашивает у DNS-сервера передачу зоны AXFR.

Сценарий отправляет DNS-серверу запрос AXFR. Запрашиваемый домен определяется путём проверки имени, заданного в командной строке, имени узла DNS-сервера либо может быть указан с помощью аргумента сценария `dnszonetransfer.domain`. Если запрос выполнен успешно, возвращаются все домены и типы доменов вместе с обычными данными, характерными для типа (SOA/MX/NS/PTR/A).

Если у нас нет «истинного» имени узла DNS-сервера, мы не можем определить вероятную зону, передачу которой следует выполнить.

Полезные ресурсы

• DNS для учёных-ракетчиков: zytrax.com

• Как работает протокол AXFR: cr.yp.to

Аргументы сценария

dnszonetransfer.domain

Домен, передачу зоны которого следует запросить.

Использование

```
nmap --script dns-zone-transfer.nse \
  --script-args 'dnszonetransfer={domain=<domain>}'
```

Пример вывода

```
53/tcp open  domain
| dns-zone-transfer:
| foo.com.          SOA    ns2.foo.com. piou.foo.com.
| foo.com.          TXT
| foo.com.          NS     ns1.foo.com.
| foo.com.          NS     ns2.foo.com.
| foo.com.          NS     ns3.foo.com.
| foo.com.          A      127.0.0.1
| foo.com.          MX     mail.foo.com.
| anansie.foo.com.  A      127.0.0.2
| dhalgren.foo.com. A      127.0.0.3
| drupal.foo.com.   CNAME
| goodman.foo.com.  A      127.0.0.4 i
```

```
| goodman.foo.com.  MX     mail.foo.com.
| isaac.foo.com.    A      127.0.0.5
| julie.foo.com.    A      127.0.0.6
| mail.foo.com.     A      127.0.0.7
| ns1.foo.com.      A      127.0.0.7
| ns2.foo.com.      A      127.0.0.8
| ns3.foo.com.      A      127.0.0.9
| stubing.foo.com.  A      127.0.0.10
| vicki.foo.com.    A      127.0.0.11
| votetrust.foo.com. CNAME
| www.foo.com.      CNAME
|_ foo.com.         SOA    ns2.foo.com. piou.foo.com.
```

finger.nse

Категории: default, discovery

Получает список пользователей, вошедших в систему, запрашивая службу Finger.

ftp-anon.nse

Категории: default, auth, safe

Проверяет, разрешает ли FTP-сервер анонимный вход.

Пример вывода

```
|_ ftp-anon: Anonymous FTP login allowed
```

ftp-bounce.nse

Категории: default, intrusive

Проверяет, разрешает ли FTP-сервер сканирование портов методом FTP bounce.

html-title.nse

Категории: default, discovery, safe

Показывает заголовок страницы веб-сервера по умолчанию.

Сценарий следует не более чем по одному перенаправлению HTTP и только в том случае, если перенаправление ведёт на тот же узел. Сценарий может отправить DNS-запрос, чтобы определить, имеет ли узел, на который ведёт перенаправление, тот же IP-адрес, что и первоначальная цель.

Пример вывода

```
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT   STATE SERVICE
80/tcp open  http
|_ html-title.nse: Go ahead and ScanMe!
```

http-auth.nse

Категории: default, auth, intrusive

Получает схему и область аутентификации веб-службы, требующей аутентификации.

Пример вывода

```
80/tcp open  http
|  http-auth: HTTP Service requires authentication
|    Auth type: Basic, realm = Password Required
|_   HTTP server may accept admin:admin combination for Basic authentication
```

http-open-proxy.nse

Категории: default, discovery, external, intrusive

Проверяет, открыт ли HTTP-прокси.

Сценарий пытается подключиться к `www.google.com` через (возможный) прокси-сервер и проверяет наличие в ответе поля заголовка `Server: gws`.

Если цель является открытым прокси-сервером, этот сценарий заставляет её получить веб-страницу с `www.google.com`.

http-passwd.nse

Категории: intrusive, vuln

Проверяет, уязвим ли веб-сервер для обхода каталогов, пытаясь получить `/etc/passwd` различными методами обхода, например запросив `../../../../etc/passwd`.

http-trace.nse

Категории: discovery

Отправляет запрос HTTP TRACE и показывает поля заголовка, которые были изменены в ответе.

Пример вывода

```
80/tcp open  http
|  http-trace: Response differs from request.  First 5 additional lines:
|  Cookie: UID=d4287aa38d02f409841b4e0c0050c131...
|  Country: us
|  Ip_is_advertise_combined: yes
|  Ip_conntype-Confidence: -1
|_ Ip_line_speed: medium
```

iax2-version.nse

Категории: version

Обнаруживает службу IAX2 по UDP.

Сценарий отправляет управляющий кадр POKE протокола Inter-Asterisk eXchange (IAX) Revision 2 и проверяет наличие правильного ответа. Этот протокол используется для обеспечения VoIP-соединений между серверами, а также связи между клиентом и сервером.

irc-info.nse

Категории: default, discovery

Собирает сведения с IRC-сервера.

Для получения этих сведений он использует STATS, LUSERS и другие запросы.

Пример вывода

```
6665/tcp open  irc
|  irc-info: Server: target.example.org
|  Version: hyperion-1.0.2b(381). target.example.org
|  Lservers/Lusers: 0/4204
|  Uptime: 106 days, 2:46:30
|  Source host: source.example.org
|_ Source ident: OK n=nmap
```

ms-sql-info.nse

Категории: default, discovery, intrusive

Пытается извлечь сведения из экземпляров Microsoft SQL Server.

mysql-info.nse

Категории: default, discovery, safe

Подключается к серверу MySQL и выводит такие сведения, как номера протокола и версии, идентификатор потока, состояние, возможности и соль пароля.

Если выполняется определение службы и сервер, по-видимому, блокирует наш узел либо заблокирован из-за слишком большого числа соединений, этот сценарий не запускается (см. правило порта).

Пример вывода

```
3306/tcp open  mysql
|  mysql-info: Protocol: 10
|  Version: 5.0.51a-3ubuntu5.1
|  Thread ID: 7
|  Some Capabilities: Connect with DB, Transactions, Secure Connection
|  Status: Autocommit
|_ Salt: bYyt\NQ/4V6IN+*3'imj
```

nbstat.nse

Категории: default, discovery, safe

Пытается получить имена NetBIOS и MAC-адрес цели.

По умолчанию сценарий показывает имя компьютера и вошедшего в систему пользователя; если повысить уровень подробности, он показывает все имена, которые система считает принадлежащими ей.

Использование

```
sudo nmap -sU --script nbstat.nse -p137 <host>
```

Пример вывода

```
(no verbose)
|_ nbstat: NetBIOS name: TST, NetBIOS user: RON, NetBIOS MAC: 00:0c:29:f9:d9:28

(verbose)
|  nbstat: NetBIOS name: TST, NetBIOS user: RON, NetBIOS MAC: 00:0c:29:f9:d9:28
|  Name: TST<00>          Flags: <unique><active>
|  Name: TST<20>          Flags: <unique><active>
|  Name: WORKGROUP<00>    Flags: <group><active>
|  Name: TST<03>          Flags: <unique><active>
|  Name: WORKGROUP<1e>    Flags: <group><active>
|  Name: RON<03>          Flags: <unique><active>
|  Name: WORKGROUP<1d>    Flags: <unique><active>
|_ Name: \x01\x02__MSBROWSE__\x02<01> Flags: <group><active>
```

pop3-brute.nse

Категории: intrusive, auth

Пытается войти в учётную запись POP3, угадывая имена пользователей и пароли.

pop3-capabilities.nse

Категории: default

Получает возможности почтового сервера POP3.

Пример вывода

```
110/tcp open  pop3
|_ pop3-capabilities: USER CAPA RESP-CODES UIDL PIPELINING STLS TOP SASL(PLAIN)
```

pptp-version.nse

Категории: version

Пытается извлечь сведения о системе из службы протокола туннелирования «точка-точка» (PPTP).

realvnc-auth-bypass.nse

Категории: default, vuln

Проверяет, уязвим ли VNC-сервер для обхода аутентификации RealVNC (CVE-2006-2369).

robots.txt.nse

Категории: default, discovery, safe

Проверяет наличие запрещённых записей в `robots.txt`.

. Чем выше уровень подробности или отладки, тем больше запрещённых записей показывается.

Пример вывода

```
80/tcp open  http  syn-ack
|  robots.txt: has 156 disallowed entries (40 shown)
|  /news?output=xhtml& /search /groups /images /catalogs
|  /catalogues /news /nwshp /news?btcid=*& /news?btaid=*&
|  /setnewsprefs? /index.html? /? /addurl/image? /pagead/ /relpage/
|  /relcontent /sorry/ /imgres /keyword/ /u/ /univ/ /cobrand /custom
|  /advanced_group_search /googlesite /preferences /setprefs /swr /url /default
|  /m? /m/? /m/lcb /m/news? /m/setnewsprefs? /m/search? /wml?
|_ /wml/? /wml/search?
```

rpcinfo.nse

Категории: default, safe, discovery

Подключается к portmapper и получает список всех зарегистрированных программ.

Пример вывода

```
111/tcp open  rpcbind
|  rpcinfo:
|  100000  2        111/udp  rpcbind
|  100005  1,2,3    705/udp  mountd
|  100003  2,3,4   2049/udp  nfs
|  100024  1      32769/udp  status
|  100021  1,3,4  32769/udp  nlockmgr
|  100000  2        111/tcp  rpcbind
|  100005  1,2,3    706/tcp  mountd
|  100003  2,3,4   2049/tcp  nfs
|  100024  1      50468/tcp  status
|_ 100021  1,3,4  50468/tcp  nlockmgr
```

skypev2-version.nse

Категории: version

Обнаруживает службу Skype версии 2.

smb-check-vulns.nse

Категории: intrusive

Проверяет, уязвим ли узел для MS08-067 - уязвимости Windows RPC, способной допустить удалённое выполнение кода. В будущем этот сценарий предполагается использовать для проверки большего числа уязвимостей.

Проверка на MS08-067 очень опасна, поскольку она, скорее всего, приведёт к сбою систем. В ходе довольно широкого сканирования, проведённого Брэндоном Энрайтом, мы установили, что в среднем уязвимая система скорее даст сбой, чем выдержит проверку. Из 82 уязвимых систем 52 дали сбой. Поэтому при использовании этой проверки следует соблюдать большую осторожность.

У вас есть возможность указать имя пользователя и пароль, но для конфигурации по умолчанию это не должно быть необходимо.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

Использование

```
nmap --script smb-check-vulns.nse -p445 <host>
sudo nmap -sU -sS --script smb-check-vulns.nse -p U:137,T:139 <host>
```

Пример вывода

```
Host script results:
|_ smb-check-vulns: This host is vulnerable to MS08-067
```

smb-enum-domains.nse

Категории: discovery, intrusive

Пытается перечислить домены системы вместе с их политиками. Без учётных данных это, скорее всего, будет работать только против Windows 2000.

После первоначальной привязки к SAMR последовательность вызовов такова:

• `Connect4`: получить `connect_handle`.

• `EnumDomains`: получить список доменов (остановитесь здесь, если вам нужны только имена).

• `QueryDomain`: получить SID домена.

• `OpenDomain`: получить дескриптор каждого домена.

• `QueryDomainInfo2`: получить сведения о домене.

• `QueryDomainUsers`: получить список пользователей домена.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

Использование

```
nmap --script smb-enum-domains.nse -p445 <host>
sudo nmap -sU -sS --script smb-enum-domains.nse -p U:137,T:139 <host>
```

Пример вывода

```
Host script results:
|  smb-enum-domains:
|  Domain: LOCALSYSTEM
|  |_ SID: S-1-5-21-2956463495-2656032972-1271678565
|  |_ Users: Administrator, Guest, SUPPORT_388945a0
|  |_ Creation time: 2007-11-26 15:24:04
|  |_ Passwords: min length: 11 characters; min age: 5 days; max age: 63 days
|  |_ Password lockout: 3 attempts in under 15 minutes will lock the account u
ntil manually reset
|  |_ Password history : 5 passwords
|  |_ Password properties:
|    |_ Password complexity requirements exist
|    |_ Administrator account cannot be locked out
|  Domain: Builtin

|  |_ SID: S-1-5-32
|  |_ Users:
|  |_ Creation time: 2007-11-26 15:24:04
|  |_ Passwords: min length: n/a; min age: n/a; max age: 42 days
|  |_ Account lockout disabled
|  |_ Password properties:
|    |_ Password complexity requirements do not exist
|_   |_ Administrator account cannot be locked out
```

smb-enum-sessions.nse

Категории: discovery, intrusive

Перечисляет пользователей, вошедших в систему либо локально, либо через клиент удалённого рабочего стола (службы терминалов), либо через общий ресурс SMB.

Перечисление пользователей локальной системы и служб терминалов выполняется путём чтения удалённого реестра. Ключи в `HKEY_USERS` - это SID, представляющие пользователей, вошедших в систему в данный момент, и эти SID можно преобразовать в правильные имена с помощью

функции LsaLookupSids. Для этого требуется любой уровень доступа выше анонимного. На операционных системах, которые я (Рон Боуз) проверял, этот запрос могут выполнить гости, пользователи или администраторы.

Перечисление SMB-соединений выполняется с помощью функции srvsvc.netsessenum, возвращающей сведения о том, кто вошёл в систему, когда он вошёл и как долго бездействовал. К сожалению, мне не удалось найти способ получить с помощью этой функции домен пользователя, поэтому домен не выводится. Требуемый уровень доступа различается между версиями Windows, но в Windows 2000 доступ к этому имеет любой (включая анонимную учётную запись), а в Windows 2003 требуется учётная запись пользователя или администратора.

Поскольку оба этих действия связаны с пользователями, вошедшими на сервер, казалось логичным объединить их в одном сценарии.

Идею и методику для этого я узнал из инструмента Sysinternals PsLoggedOn.exe. Я использую вызовы функций, подобные тем, которые использует он, поэтому выражаю им благодарность. Спасибо также Мэтту за идею написать этот сценарий.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: smbusername, smbpassword, smbhash, smbguest и smbtype.

Использование

```
smb-enum-sessions.nse -p445 <host>
-sS --script smb-enum-sessions.nse -p U:137,T:139 <host>

|  smb-enum-sessions:
|  Users logged in:
|  |_ TESTBOX\Administrator since 2008-10-21 08:17:14
|_ |_ DOMAIN\rbowes since 2008-10-20 09:03:23

|  Active SMB Sessions:
|_ |_ ADMINISTRATOR is connected from 10.100.254.138 for [just logged in, i
probably you], idle for [not idle]
```

smb-enum-shares.nse

Категории: discovery, intrusive

Пытается перечислить общие ресурсы с помощью функции MSRPC srvsvc.NetShareEnumAll, а затем получить сведения о каждом общем ресурсе с помощью srvsvc.NetShareGetInfo.

Вызов NetShareEnumAll будет работать анонимно в Windows 2000, а в любой другой версии Windows требует доступа уровня пользователя. Вызов NetShareGetInfo требует учётной записи администратора в одной версии Windows из проверенных мной (Роном Боузом).

Хотя NetShareEnumAll ограничен в некоторых системах, фактическое подключение к общему ресурсу для проверки его существования всегда будет работать. Поэтому, если NetShareEnumAll завершится неудачно, будет испробован список распространённых общих ресурсов.

После нахождения списка общих ресурсов, полного или нет, мы пытаемся анонимно подключиться к каждому из них, что позволяет разделить их на классы «анонимные» и «ограниченные».

Когда это возможно, после определения списка общих ресурсов вызывается `NetShareGetInfo`, чтобы получить дополнительные сведения об общем ресурсе. Скорее всего, это завершится неудачно, если только мы не проводим проверку с аутентификацией.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

Использование

```
nmap --script smb-enum-shares.nse -p445 <host>
sudo nmap -sU -sS --script smb-enum-shares.nse -p U:137,T:139 <host>
```

Пример вывода

```
Standard:
|  smb-enum-shares:
|  Anonymous shares: IPC$
|_ Restricted shares: F$, ADMIN$, C$

Verbose:
Host script results:
|  smb-enum-shares:
|  Anonymous shares:
|    IPC$
|    |_ Type: STYPE_IPC_HIDDEN
|    |_ Comment: Remote IPC
|    |_ Users: 1, Max: <unlimited>
|    |_ Path:
|    test
|    |_ Type: STYPE_DISKTREE

|    |_ Comment: This is a test share, with a maximum of 7 users
|    |_ Users: 0, Max: 7
|    |_ Path: C:\Documents and Settings\Ron\Desktop\test
|  Restricted shares:
|    ADMIN$
|    |_ Type: STYPE_DISKTREE_HIDDEN
|    |_ Comment: Remote Admin
|    |_ Users: 0, Max: <unlimited>
|    |_ Path: C:\WINNT
|    C$
|    |_ Type: STYPE_DISKTREE_HIDDEN
|    |_ Comment: Default share
|    |_ Users: 0, Max: <unlimited>
|_   |_ Path: C:\
```

smb-enum-users.nse

Категории: discovery, intrusive

Пытается перечислить пользователей удалённой системы Windows с максимально возможным объёмом сведений, применяя различные методы (через SMB и MSRPC, использующий порт 445 или 139). Для перечисления пользователей используются некоторые функции SAMR, а также предпринимается перебор с угадыванием с помощью функций LSA.

Один из применяемых методов - вызов функции `QueryDisplayInfo` в библиотеке SAMR. В случае успеха она возвращает подробный список пользователей. Это можно сделать анонимно против Windows 2000 и с учётной записью уровня пользователя в других версиях Windows (но не с учётной записью уровня гостя).

Для выполнения этой проверки используются следующие функции:

• `Bind`: выполнить привязку к службе SAMR.

• `Connect4`: получить `connect_handle`.

• `EnumDomains`: получить список доменов.

• `QueryDomain`: получить SID домена.

• `OpenDomain`: получить дескриптор каждого домена.

• `QueryDisplayInfo`: получить список пользователей домена.

• `Close`: закрыть дескриптор домена.

• `Close`: закрыть дескриптор соединения.

Преимущество этого метода заключается в возврате множества подробностей, включая полное имя и описание; недостаток - в том, что во всех системах, кроме Windows 2000, для него требуется учётная запись уровня пользователя. Кроме того, он получает только настоящие учётные записи пользователей, но не группы или псевдонимы.

Независимо от того, завершится ли это успешно, для получения учётных записей пользователей применяется второй метод, называемый перебором LSA. Перебор LSA можно выполнять анонимно против Windows 2000, а в других системах для него требуется учётная запись гостя или выше. Его преимущество состоит в работе с меньшими разрешениями и в обнаружении большего числа типов учётных записей (то есть групп, псевдонимов и т. д.). Недостатки заключаются в том, что он возвращает меньше сведений и, поскольку это перебор с угадыванием, некоторые учётные записи можно пропустить.

Однако это не метод полного перебора в обычном смысле: это перебор RID пользователей. RID пользователя - значение (обычно 500, 501 или 1000+), однозначно идентифицирующее пользователя в домене или системе. LSA предоставляет функцию, позволяющую преобразовать RID (например, 1000) в имя пользователя (например, «Ron»). Таким образом, этот метод фактически пытается преобразовать 1000 в имя, затем 1001, 1002 и так далее, пока мы не решим, что закончили.

Пользователи разбиваются на группы по пять RID, а затем проверяются по отдельности (одновременная проверка слишком большого числа вызывает проблемы). Мы продолжаем проверку, пока не достигнем 1100 и не получим пустую группу. Вероятно, это не самый эффективный способ, но, похоже, он работает. Возможно, в будущем стоило бы усовершенствовать его, добавив некоторую интеллектуальность. Я (Рон Боуз) провёл проверку на старом сервере с большим числом учётных записей и получил следующие результаты: 500, 501, 1000, 1030, 1031, 1053, 1054, 1055, 1056, 1057, 1058, 1059, 1060, 1061, 1062, 1063, 1064, 1065, 1066, 1067, 1070, 1075, 1081, 1088, 1090. Скачок от 1000 до 1030 довольно велик и при автоматизированной проверке легко может привести к пропуску учётных записей.

Перед попыткой этого преобразования необходимо определить SID сервера. SID определяется выполнением обратной операции, то есть преобразованием имени в RID. Имя определяется поиском любого имени, присутствующего в системе. Мы пробуем:

• имя компьютера и имя домена, возвращённые в SMB_COM_NEGOTIATE;

• запрос nbstat для получения имени сервера и пользователя, вошедшего в систему в данный момент; и

• некоторые распространённые имена: «administrator», «guest» и «test».

Теоретически имени компьютера должно быть достаточно, чтобы это всегда работало, и пока в моих проверках так и было, но для надёжности я включил остальные имена.

Имена и подробности, полученные обоими этими методами, объединяются и отображаются. Если вывод подробный, показываются дополнительные сведения. Вывод упорядочен по алфавиту.

Следует отдать должное программам enum.exe, sid2user.exe и user2sid.exe: написанный мной для этого код в значительной степени основан на применяемых ими методах.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: smbusername, smbpassword, smbhash, smbguest и smbtype.

Использование

```
nmap --script smb-enum-users.nse -p445 <host>
sudo nmap -sU -sS --script smb-enum-users.nse -p U:137,T:139 <host>
```

Пример вывода

```
Host script results:
|  smb-enum-users:
|_ TESTBOX\Administrator, EXTERNAL\DnsAdmins, TESTBOX\Guest, EXTERNAL\HelpServi
cesGroup, EXTERNAL\PARTNERS$, TESTBOX\SUPPORT_388945a0

Host script results:
|  smb-enum-users:
|  Administrator
|    |_ Type: User
|    |_ Domain: LOCALSYSTEM
|    |_ Full name: Built-in account for administering the computer/domain

|    |_ Flags: Normal account, Password doesn't expire
|  DnsAdmins
|    |_ Type: Alias
|    |_ Domain: EXTRANET
|  EventViewer
|    |_ Type: User
|    |_ Domain: SHARED
|  ProxyUsers
|    |_ Type: Group
|    |_ Domain: EXTRANET
|  ComputerAccounts
```

```
|    |_ Type: Group
|    |_ Domain: EXTRANET
|  Helpdesk
|    |_ Type: Group
|    |_ Domain: EXTRANET
|  Guest
|    |_ Type: User
|    |_ Domain: LOCALSYSTEM
|    |_ Full name: Built-in account for guest access to the computer/domain
|    |_ Flags: Normal account, Disabled, Password not required, Password doesn't expire
|  Staff
|    |_ Type: Alias
|    |_ Domain: LOCALSYSTEM
|  Students
|    |_ Type: Alias
|_   |_ Domain: LOCALSYSTEM
```

smb-os-discovery.nse

Категории: default, discovery, safe

Пытается определить операционную систему по протоколу SMB (порты 445 и 139).

Хотя стандартные аргументы сценария `smb*` можно использовать, они, скорее всего, никак существенно не изменят результат.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

Использование

```
nmap --script smb-os-discovery.nse -p445 127.0.0.1
sudo nmap -sU -sS --script smb-os-discovery.nse -p U:137,T:139 127.0.0.1
```

Пример вывода

```
|  smb-os-discovery: Windows 2000
|  LAN Manager: Windows 2000 LAN Manager

|  Name: WORKGROUP\TEST1
|_ System time: 2008-09-09 20:55:55 UTC-5
```

smb-security-mode.nse

Категории: discovery, safe

Возвращает сведения об уровне безопасности SMB, определённом посредством SMB.

Вот как следует интерпретировать вывод:

Аутентификация на уровне пользователя: у каждого пользователя есть отдельные имя пользователя и пароль, которые используются для входа в систему. В наши дни это конфигурация по умолчанию практически для всего.

Аутентификация на уровне общего ресурса: для входа следует использовать анонимную учётную запись, а затем при обращении к общему ресурсу передаётся пароль (открытым текстом). Этот пароль используют все пользователи, имеющие доступ к общему ресурсу. Так всё делалось первоначально, но сейчас это встречается нечасто. Если сервер использует безопасность на уровне общего ресурса, он уязвим для перехвата трафика.

Поддерживаются пароли с запросом и ответом: если этот параметр включён, сервер может принимать пароль любого типа:

• Открытый текст

• LM и NTLM

• LMv2 и NTLMv2

Если этот параметр не задан, сервер может принимать только пароли открытым текстом. В наши дни большинство серверов настроены на использование запроса и ответа. Если сервер настроен принимать пароли открытым текстом, он уязвим для перехвата трафика. LM и NTLM довольно безопасны, хотя против них существуют некоторые атаки полного перебора.

Подпись сообщений: если она обязательна, все сообщения между клиентом и сервером должны быть подписаны общим ключом, полученным из пароля и запроса сервера. Если она поддерживается, но не обязательна, подпись сообщений согласовывается между клиентами и серверами и используется, если обе стороны её поддерживают и запрашивают. По умолчанию клиенты Windows не подписывают сообщения, поэтому, если сервер не требует подписи сообщений, сообщения, вероятно, подписываться не будут; кроме того, при выполнении атаки посредника злоумышленник может согласовать отсутствие подписи сообщений. Если подпись сообщений не обязательна, сервер уязвим для атак посредника.

Этот сценарий позволит вам использовать аргументы сценария `smb*` (для задания имени пользователя, пароля и т. д.), но, вероятно, никогда не потребует их.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

Использование

```
nmap --script smb-security-mode.nse -p445 127.0.0.1
sudo nmap -sU -sS --script smb-security-mode.nse -p U:137,T:139 127.0.0.1
```

Пример вывода

```
|  smb-security-mode: User-level authentication
|  smb-security-mode: Challenge/response passwords supported
|_ smb-security-mode: Message signing supported
```

smb-server-stats.nse

Категории: discovery, intrusive

Пытается получить статистику сервера через SMB и MSRPC, использующий TCP-порты 445 или 139.

Для получения этой статистики в большинстве версий Windows требуется учётная запись администратора, а Vista, по-видимому, не позволяет получить её даже учётной записи администратора.

Некоторые возвращаемые здесь числа кажутся мне неверными, но это определённо те числа, которые возвращает Windows. Относитесь к приведённым здесь значениям с долей скептицизма.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

```
--script smb-server-stats.nse -p445 <host>
nmap -sU -sS --script smb-server-stats.nse -p U:137,T:139 <host>
```

```
|  smb-server-stats:
|  Server statistics collected since 2008-10-17 09:32:41 (4d0h24m29s):
|_ Traffic 133467 bytes (0.38b/s) sent, 167696 bytes (0.48b/s) received
|_ Failed logins: 5
|_ Permission errors: 1, System errors: 0
|_ Print jobs spooled: 0
|_ Files opened (including pipes): 18
```

Категории: discovery, intrusive

Получает из реестра сведения об удалённой системе. Для получения всех сведений требуется учётная запись администратора, хотя учётная запись пользователя всё равно получит значительную их часть. Гость, вероятно, не получит ничего, как и анонимный пользователь. Это относится ко всем операционным системам, включая Windows 2000.

Windows Vista, по-видимому, не имеет привязки WINREG (либо она отличается, а я об этом не знаю), поэтому этот сценарий вообще не поддерживает Vista.

Аргументы сценария

smb*

Этот сценарий поддерживает аргументы сценария модуля smb: `smbusername`, `smbpassword`, `smbhash`, `smbguest` и `smbtype`.

Использование

```
nmap --script smb-system-info.nse -p445 <host>
sudo nmap -sU -sS --script smb-system-info.nse -p U:137,T:139 <host>
```

Пример вывода

```
Host script results:
|  smb-system-info:
|  OS Details
|  |_ Microsoft Windows Server 2003 Service Pack 2 (ServerNT 5.2 build 3790)
|  |_ Installed on 2007-11-26 23:40:40
|  |_ Registered to IPC (organization: MYCOMPANY)
|  |_ Path: %SystemRoot%\system32;%SystemRoot%;%SystemRoot%\System32\Wbem;C:\Pr
ogram Files\Microsoft SQL Server\90\Tools\binn\;C:\Program Files\IBM\Rational
ppScan\
|  |_ Systemroot: C:\WINDOWS
|  |_ Page files: C:\pagefile.sys 2046 4092 (cleared at shutdown => 0)
|  Hardware
|  |_ CPU 0: Intel(R) Xeon(TM) CPU 2.80GHz [2780mhz GenuineIntel]
|  |_ Identifier 0: x86 Family 15 Model 2 Stepping 9
|  |_ CPU 1: Intel(R) Xeon(TM) CPU 2.80GHz [2780mhz GenuineIntel]
|  |_ Identifier 1: x86 Family 15 Model 2 Stepping 9
|  |_ CPU 2: Intel(R) Xeon(TM) CPU 2.80GHz [2780mhz GenuineIntel]
|  |_ Identifier 2: x86 Family 15 Model 2 Stepping 9
|  |_ CPU 3: Intel(R) Xeon(TM) CPU 2.80GHz [2780mhz GenuineIntel]
|  |_ Identifier 3: x86 Family 15 Model 2 Stepping 9
|  |_ Video driver: RAGE XL PCI Family (Microsoft Corporation)
|  Browsers
|  |_ Internet Explorer 7.0000
|_ |_ Firefox 3.0.3 (en-US)
```

smtp-commands.nse

Категории: default, discovery, safe

Пытается с помощью EHLO и HELP собрать расширенные команды, поддерживаемые SMTP-сервером.

Пример вывода

```
25/tcp open  smtp
|  smtp-commands: EHLO uninvited.example.net Hello root at localhost [127.0.0.1]
|, SIZE 52428800, PIPELINING, HELP
|_ HELP Commands supported: AUTH HELO EHLO MAIL RCPT DATA NOOP QUIT RSET HELP
```

smtp-open-relay.nse

Категории: demo

Проверяет, является ли SMTP-сервер открытым ретранслятором.

smtp-strangeport.nse

Категории: malware

Проверяет, работает ли SMTP на нестандартном порту.

Это может указывать на то, что взломщики или скрипт-кидди установили в системе бэкдор для рассылки спама или управления машиной.

Пример вывода

```
/tcp open  smtp
|_ smtp-strangeport: Mail server on unusual port: possible malware
```

sniffer-detect.nse

Категории: discovery

Проверяет, переведена ли сетевая плата цели в локальной сети Ethernet в неразборчивый режим.

Используемые методы описаны по адресу securityfriday.com.

Пример вывода

```
Host script results:
|_ sniffer-detect: Likely in promiscuous mode (tests: "11111111")
```

snmp-brute.nse

Категории: intrusive, auth

Пытается найти строку сообщества SNMP путём угадывания полным перебором.

snmp-sysdescr.nse

Категории: default, discovery, safe

Пытается извлечь сведения о системе из службы SNMP версии 1.

Пример вывода

```
|  snmp-sysdescr: HP ETHERNET MULTI-ENVIRONMENT,ROM A.25.80,JETDIRECT,JD117,EEP
ROM V.28.22,CIDATE 08/09/2006
|_ System uptime: 28 days, 17:18:59 (248153900 timeticks)
```

sql-injection.nse

Категории: intrusive, vuln

Обходит HTTP-сервер в поисках URL, содержащих запросы, уязвимые для атаки с внедрением SQL-кода.

Сценарий обходит HTTP-сервер в поисках URL, содержащих запросы. Затем он объединяет команды SQL с уязвимыми URL, чтобы получить ошибки. Ошибки анализируются для определения уязвимости сайта к атаке. Здесь используется самая простая форма внедрения SQL-кода, а для всего более сложного лучше подходит отдельный инструмент. Поддерживаются как перенаправления в стиле meta, так и перенаправления HTTP.

У нас может не быть доступа к истинному имени узла целевого веб-сервера, что может препятствовать доступу к сайтам на виртуальных узлах. Этот сценарий следует только по абсолютным ссылкам, в которых компонент имени узла совпадает с именем целевого сервера из обратной DNS-записи.

ssh-hostkey.nse

Категории: safe, default, intrusive

Показывает ключи узла SSH.

Показывает fingerprint ключа целевого SSH-сервера и (при достаточно высоком уровне подробности) сам открытый ключ. Он записывает обнаруженные ключи узла в `nmap.registry` для использования другими сценариями. Выводом можно управлять с помощью аргумента сценария `ssh_hostkey`.

Аргументы сценария

ssh_hostkey

Управляет форматом вывода ключей. Можно задать несколько значений, разделённых пробелами. Возможные значения:

• "`full`": весь ключ, а не только fingerprint.

• "`bubble`": вывод Bubble Babble.

• "`visual`": визуальное представление в виде ASCII-графики.

• "`all`": всё перечисленное выше.

Использование

```
nmap host --script SSH-hostkey --script-args ssh_hostkey=full
nmap host --script SSH-hostkey --script-args ssh_hostkey=all
nmap host --script SSH-hostkey --script-args ssh_hostkey='visual bubble'
```

Пример вывода

```
22/tcp open  ssh
|  ssh-hostkey: 2048 f0:58:ce:f4:aa:a4:59:1c:8e:dd:4d:07:44:c8:25:11 (RSA)
22/tcp open  ssh
```

```
|  ssh-hostkey: 2048 f0:58:ce:f4:aa:a4:59:1c:8e:dd:4d:07:44:c8:25:11 (RSA)
|  +--[ RSA 2048]----+
|      .E*+      |
|       oo       |
|      . o .     |
|       o . .    |

|      o S o .
|     = o + .
|    . * o
|       o .
|_ +-----------------+
22/tcp open  ssh
|  ssh-hostkey: 2048 xuvah-degyp-nabus-zegah-hebur-nopig-bubig-difeg-hisym-rume
f-cuxex (RSA)
|_ ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAwVuv2gcr0maaKQ69VVIEv2ob40xnuI64fkeOnCXD
1lUx5tTA+vefXUWEMxgMuA7iX4irJHy2zerONQ3Z3yJvr5scPgTYIaEOp5Uo/eGFG9Agpk5wE8CoFOe
47iCAPHqz1mP2V7aNURLMODb3jVZuI07A2ZRrMGrD8d888E2ORVORv1rYeTYCqcMMoVFmX913gWEdk4
yx3w5sD8v501Iuyd1v19mPfyhrI5E1E1n1/Xjp5NO/xP2GUBrdkDMxKaxqTPMie/f0dXBUPQQN697a5
q+51BRPhKYOtn6yQKCd9s1Q22nxn72Jmi1RzbMyYJ52FosDT755Qmb46GLrDMaZMQ==
```

sshv1.nse

Категории: default, safe

Проверяет, поддерживает ли SSH-сервер устаревшую и менее безопасную версию 1 протокола SSH.

sslv2.nse

Категории: default, safe

Определяет, поддерживает ли сервер устаревший и менее безопасный SSL-v2, и выясняет, какие шифры он поддерживает.

Пример вывода

```
43/tcp open  https  syn-ack
|  sslv2: server still supports SSLv2
|    SSL2_RC4_128_WITH_MD5
|    SSL2_DES_192_EDE3_CBC_WITH_MD5
|    SSL2_RC2_CBC_128_CBC_WITH_MD5
|    SSL2_DES_64_CBC_WITH_MD5
|    SSL2_RC4_128_EXPORT40_WITH_MD5
|    SSL2_RC2_CBC_128_CBC_WITH_MD5
```

telnet-brute.nse

Категории: auth, intrusive

Пытается получить учётные данные для входа по Telnet, угадывая имена пользователей и пароли.

upnp-info.nse

Пытается извлечь сведения о системе из службы UPnP.

Пример вывода

```
|  upnp-info: System/1.0 UPnP/1.0 IGD/1.0
|_ Location: http://192.168.1.1:80/UPnP/IGD.xml
```

whois.nse

Категории: discovery, external, safe

Опрашивает службы WHOIS региональных интернет-регистратур (RIR) и пытается получить сведения о назначении IP-адресов, содержащем целевой IP-адрес.

Отображаемые поля содержат сведения о назначении и организации, отвечающей за управление адресным пространством. Если в командной строке Nmap запрошен подробный вывод (`-v`), будут показаны дополнительные сведения о назначении.

Чтобы определить, какую RIR опрашивать для заданного целевого IP-адреса, этот сценарий использует назначения, размещённые IANA. Данные кэшируются локально, а затем разбираются для использования в качестве таблицы поиска. Локально кэшированные файлы периодически обновляются, чтобы данные оставались актуальными. Если по какой-либо причине эти файлы недоступны сценарию, по очереди опрашивается последовательность служб Whois по умолчанию, пока не будет найдена нужная запись; либо перенаправление к другой (заданной) службе Whois; либо пока последовательность не исчерпается без нахождения перенаправления или нужной записи.

Сценарий распознает перенаправление к другой службе Whois, если эта служба определена в сценарии, и продолжит работу, отправив запрос указанной службе. Запись считается нужной, если она не содержит перенаправления.

Чтобы сократить число ненужных запросов, отправляемых службам Whois, используется кэш записей, а записи в кэше можно применять к любым целям в пределах диапазона адресов, представленного в записи.

В определённых обстоятельствах возможность кэшировать ответы препятствует обнаружению других, меньших назначений IP-адресов, применимых к цели, поскольку кэшированный ответ принимается вместо отправки запроса Whois. Когда важно обеспечить получение наиболее точных сведений о назначении IP-адреса, следует использовать аргумент сценария `whodb` со значением `"nocache"` (см. аргументы сценария). Это уменьшает диапазон адресов, которые могут использовать кэшированную запись, до размера, помогающего гарантировать обнаружение меньших назначений. Этот параметр следует использовать с осторожностью из-за возможности отправить большое число запросов Whois и, возможно, получить запрет на использование служб.

При использовании этого сценария ваш IP-адрес будет отправлен на `iana.org`. Кроме того, ваш адрес и адрес цели сканирования будут отправлены одной из RIR.

Аргументы сценария

whodb

Принимает любые из следующих значений, которые можно комбинировать:

• whodb=nofile - запретить использование данных о назначениях IANA и вместо этого опрашивать службы по умолчанию.

• whodb=nofollow - игнорировать перенаправления и вместо этого отображать первую полученную запись.

• whodb=nocache - запретить принимать записи из кэша, когда они применимы к большим диапазонам адресов.

• whodb=[service-ids] - переопределить опрашиваемые службы по умолчанию. Подразумевает nofile.

Использование

```
# Basic usage:
nmap target --script whois

# To prevent the use of IANA assignments data supply the nofile value
# to the whodb argument:
nmap target --script whois --script-args whodb=nofile
nmap target --script whois --script-args whois={whodb=nofile}

# Supplying a sequence of whois services will also prevent the use of
# IANA assignments data and override the default sequence:
nmap target --script whois --script-args whodb=arin+ripe+afrinic
nmap target --script whois --script-args whois={whodb=apnic*lacnic}
# The order in which the services are supplied is the order in which
# they will be queried.  (N.B. commas or semi-colons should not be
# used to delimit argument values.)

# To return the first record obtained even if it contains a referral
# to another service, supply the nofollow value to whodb:
nmap target --script whois --script-args whodb=nofollow
nmap target --script whois --script-args whois={whodb=nofollow+ripe}
# Note that only one service (the first one supplied) will be used in
# conjunction with nofollow.

# To ensure discovery of smaller assignments even if larger ones
# exist in the cache, supply the nocache value to whodb:
nmap target --script whois --script-args whodb=nocache
nmap target --script whois --script-args whois={whodb=nocache}
```

Пример вывода

```
Host script results:
|  whois: Record found at whois.arin.net
|  netrange: 64.13.134.0 - 64.13.134.63
|  netname: NET-64-13-143-0-26
|  orgname: Titan Networks
|  orgid: INSEC
|_ country: US stateprov: CA
```

xampp-default-auth.nse

Категории: auth, vuln

Проверяет, использует ли FTP-сервер XAMP или XAMPP имя пользователя и пароль по умолчанию.

XAMP - дистрибутив Apache, предназначенный для простой установки и администрирования.

Пример вывода

```
1/tcp open  ftp
|_ xampp-default-auth: Login success with u/p: nobody/xampp
```

9.6. Библиотеки NSE

В дополнение к значительным встроенным возможностям Lua мы написали или интегрировали множество библиотек расширений, которые делают написание сценариев мощнее и удобнее. Эти библиотеки (иногда называемые модулями) при необходимости компилируются и устанавливаются вместе с Nmap. У них есть собственный каталог `nselib`, который устанавливается в настроенном каталоге данных. Чтобы использовать библиотеки по умолчанию, сценариям достаточно вызвать `require`[6].

9.6.1. Список всех библиотек

Этот список представляет собой лишь обзор, дающий представление о доступных библиотеках. Разработчикам следует обратиться к полной документации по адресу nmap.org.

base64

Кодирование и декодирование Base64. Соответствует RFC 4648.

bin

Упаковка и распаковка двоичных данных.

bit

Побитовые операции над целыми числами.

comm

Общие функции связи для задач сетевого обнаружения, таких как получение баннеров и обмен данными.

datafiles

Чтение и разбор некоторых файлов данных Nmap: `nmap-protocols`, `nmap-rpc` и `nmap-services`.

dns

Простая библиотека DNS, поддерживающая создание, кодирование, декодирование и опрос пакетов.

http

Клиентская библиотека HTTP.

ipOps

Вспомогательные функции для обработки и сравнения IP-адресов.

listop

Операции со списками в функциональном стиле.

match

Вспомогательные функции буферизованного сетевого ввода-вывода.

msrpc

Вызов различных функций MSRPC.

netbios

Создание и разбор трафика NetBIOS. Основное назначение - отправка запросов имён NetBIOS.

nmap

Интерфейс с внутренними механизмами Nmap.

openssl

Привязки OpenSSL.

packet

Средства для работы с необработанными пакетами.

pcre

Регулярные выражения, совместимые с Perl.

pop3

Функции POP3.

shortport

Функции для построения коротких правил портов.

smb

Трафик Server Message Block (SMB, также известный как CIFS).

snmp

Функции SNMP.

ssh1

Функции для протокола SSH-1.

ssh2

Функции для протокола SSH-2.

stdnse

Стандартные функции движка сценариев Nmap.

strbuf

Средства строкового буфера.

tab

Размещение вывода в таблицах.

unpwdb

Библиотека базы данных имён пользователей и паролей.

url

Разбор и составление URI, а также разрешение относительных URL.

9.6.2. Добавление модулей C в Nselib

Несколько модулей, включённых в `nselib`, написаны на C или C++, а не на Lua. Два примера - `bit` и `pcre`. Мы рекомендуем по возможности писать модули на Lua, но C и C++ могут быть более подходящими при критической важности производительности или если (как в случае модулей `pcre` и `openssl`) вы связываетесь с существующей библиотекой. В этом разделе описано, как писать собственные компилируемые расширения для `nselib`.

API Lua для C подробно описан в книге *Programming in Lua, Second Edition*, поэтому здесь приводится лишь краткое резюме[7]. Модули C состоят из функций, следующих протоколу типа `lua_CFunction`. Функции регистрируются в Lua и собираются в библиотеку вызовом функции `luaL_register`. Инициализирующая функция предоставляет интерфейс между модулем и остальным кодом NSE. По соглашению имя инициализирующей функции имеет вид `luaopen_<module>`.

Самый маленький компилируемый модуль, поставляемый с NSE, - `bit`, а один из наиболее понятных - `openssl`. Эти модули служат хорошими примерами для начинающего автора модулей. Исходный код `bit` находится в `nse_bit.cc` и `nse_bit.h`, а исходный код `openssl` - в `nse_openssl.cc` и `nse_openssl.h`. Большинство остальных компилируемых модулей следуют этому соглашению `nse_<module>`.

Изучение модуля `openssl` показывает, что одна из функций в `nse_openssl.cc` - `l_md5`, вычисляющая дайджест MD5. Её прототип:

```
static int l_md5(lua_State *L);
```

Прототип показывает, что `l_md5` соответствует типу `lua_CFunction`. Функция является статической, потому что ей не нужно быть видимой другому компилируемому коду. Для регистрации в Lua требуется только адрес. Далее в файле `l_md5` вводится в массив типа `luaL_reg` и связывается с именем `md5`:

```
static const struct luaL_reg openssllib[] = {
  { "md5", l_md5 },
  { NULL, NULL }
};
```

Теперь эта функция будет известна NSE как `md5`. Затем библиотека регистрируется вызовом `luaL_register` внутри инициализирующей функции `luaopen_openssl`, как показано далее. Некоторые строки, относящиеся к регистрации типов OpenSSL BIGNUM, опущены:

```
LUALIB_API int luaopen_openssl(lua_State *L) {
  luaL_register(L, OPENSSLLIBNAME, openssllib);
  return 1;
}
```

Функция `luaopen_openssl` - единственная функция в файле, которая выставлена в `nse_openssl.h`. `OPENSSLLIBNAME` - просто строка `"openssl"`.

После написания компилируемого модуля его необходимо добавить в NSE, включив в список стандартных библиотек в `nse_init.cc`. Затем имена файлов исходного кода модуля необходимо добавить в соответствующие места `Makefile.in`. В обеих задачах можно просто следовать примеру других модулей C. Для сборки Windows новые исходные файлы необходимо добавить в файл проекта `mswin32/nmap.vcproj` с помощью MS Visual Studio (см. раздел 2.4.4 «Компиляция из исходного кода»).

9.7. API Nmap

Сценарии NSE имеют доступ к нескольким средствам Nmap для написания гибких и изящных сценариев. API предоставляет сведения о целевом узле, такие как состояния портов и результаты определения версий. Он также предлагает интерфейс к библиотеке Nsock для эффективного сетевого ввода-вывода.

9.7.1. Сведения, передаваемые сценарию

Для эффективного движка сценариев Nmap требуется нечто большее, чем просто интерпретатор Lua. Пользователям нужен простой доступ к сведениям, которые Nmap узнал о целевых узлах. Эти данные передаются как аргументы методу `action` сценария NSE. Аргументы `host` и `port` - таблицы Lua, содержащие сведения о цели, против которой выполняется сценарий. Если сценарий соответствует `hostrule`, он получает только таблицу `host`, а если соответствует `portrule`, то получает и `host`, и `port`. В следующем списке описана каждая переменная в этих двух таблицах.

host

Эта таблица передаётся как параметр функциям правила и действия. Она содержит сведения об операционной системе, работающей на узле (если был указан переключатель `-O`), IP-адрес и имя узла сканируемой цели.

host.os

Запись `os` в таблице `host` представляет собой массив строк. Строки (до восьми) являются названиями операционных систем, которые, возможно, работают на цели. Строки добавляются в этот массив только в том случае, если целевая машина идеально соответствует одной или нескольким записям базы данных ОС. Если Nmap был запущен без параметра `-O`, то `host.os` равен `nil`.

host.ip

Содержит строковое представление IP-адреса целевого узла. Если сканирование было запущено против имени узла и обратный DNS-запрос вернул более одного IP-адреса, используется тот же IP-адрес, который был выбран для сканирования.

host.name

Содержит обратную DNS-запись сканируемого целевого узла, представленную строкой. Если у узла нет обратной DNS-записи, значение поля представляет собой пустую строку.

host.targetname

Содержит имя узла в том виде, в котором оно указано в командной строке. Если заданная в командной строке цель содержит маску сети или является IP-адресом, значение поля равно `nil`.

host.directly_connected

Логическое значение, указывающее, подключён ли целевой узел непосредственно (то есть находится ли он в том же сегменте сети) к узлу, на котором работает Nmap.

host.mac_addr

MAC-адрес узла назначения (двоичная строка длиной шесть байт) или `nil`, если узел не подключён непосредственно.

host.mac_addr_src

Наш собственный MAC-адрес, использованный для подключения к узлу (либо адрес нашей сетевой платы, либо, с `--spoof-mac`, поддельный адрес).

host.interface

Строка, содержащая имя интерфейса (в стиле dnet), через который отправляются пакеты узлу.

host.bin_ip

IPv4-адрес целевого узла в виде 32-битного двоичного значения.

host.bin_ip_src

Исходный IPv4-адрес нашего узла (на котором работает Nmap) в виде 32-битного двоичного значения.

port

Таблица портов передаётся сценарию службы NSE (то есть только сценариям с `portrule`, а не с `hostrule`) так же, как таблица узла. Она содержит сведения о порте, против которого выполняется

сценарий. Хотя эта таблица не передаётся сценариям узла, состояния портов цели всё равно можно запросить у Nmap с помощью вызова `nmap.get_port_state()`.

port.number

Содержит номер целевого порта.

port.protocol

Определяет протокол целевого порта. Допустимые значения - `"tcp"` и `"udp"`.

port.service

Содержит строковое представление службы, работающей на `port.number`, как определено системой обнаружения служб Nmap. Если поле `port.version` равно `nil`, Nmap угадал службу по номеру порта. В противном случае определение версии смогло установить прослушивающую службу, и это поле равно `port.version.name`.

port.version

Эта запись представляет собой таблицу, содержащую сведения, полученные движком сканирования версий Nmap. Некоторые значения (например, имя службы, достоверность типа службы и значения, связанные с RPC) могут быть получены Nmap, даже если сканирование версий не выполнялось. Неопределённые значения по умолчанию равны `nil`. Значение каждого параметра приведено в следующей таблице:

Таблица 9.1. Значения `port.version`

Имя — описание

• `name` — Содержит имя службы, выбранное Nmap для порта.
• `name_confidence` — Оценивает уверенность Nmap в точности `name` от 1 (наименьшая уверенность) до 10.
• `product`, `version`, `extrainfo`, `hostname`, `ostype`, `devicetype` — Эти пять переменных описаны в `<versioninfo>`.
• `service_tunnel` — Содержит строку `"none"` или `"ssl"` в зависимости от того, использовал ли Nmap туннелирование SSL для определения службы.

• `service_fp` — Fingerprint службы, если он есть, предоставляется в этом значении. Это описано в разделе 7.7 «Вклад сообщества».
• `rpc_status` — Содержит строковое значение `good_prog`, если нам удалось определить номер программы службы RPC, прослушивающей порт; `unknown`, если порт, по-видимому, относится к RPC, но определить номер программы не удалось; `not_rpc`, если порт не похож на RPC; или `untested`, если мы не проверяли состояние RPC.
• `rpc_program`, `rpc_lowver`, `rpc_highver` — Обнаруженный номер программы RPC и диапазон номеров версий, поддерживаемых этой программой. Они будут равны `nil`, если `rpc_status` имеет любое значение, кроме `good_prog`.

port.state

Содержит сведения о состоянии порта. Сценарии служб запускаются только против портов в состояниях `open` или `open|filtered`, поэтому `port.state` обычно содержит одно из этих значений. Другие значения могут появиться, если таблица портов является результатом функции `get_port_state`. Состояние порта можно изменить с помощью вызова `nmap.set_port_state()`. Обычно это делается, когда установлено, что порт `open|filtered` открыт.

9.7.2. API сетевого ввода-вывода

Чтобы обеспечить эффективный и распараллеливаемый сетевой ввод-вывод, NSE предоставляет интерфейс к Nsock, библиотеке сокетов Nmap. Интеллектуальный механизм обратных вызовов, используемый Nsock, полностью прозрачен для сценариев NSE. Главное преимущество сокетов NSE заключается в том, что они никогда не блокируются при операциях ввода-вывода, позволяя выполнять множество сценариев параллельно. Параллелизм ввода-вывода полностью прозрачен для авторов сценариев NSE. В NSE можно программировать либо так, словно используется один неблокирующий сокет, либо так, словно соединение блокирующее. Даже блокирующие вызовы ввода-вывода возвращаются после превышения заданного времени ожидания. Поддерживаются два вида сетевого ввода-вывода: в стиле соединений и с необработанными пакетами.

Сетевой ввод-вывод в стиле соединений

Эта часть сетевого API должна подходить для большинства классических сетевых применений: пользователи создают сокет, подключают его к удалённому адресу, отправляют и получают данные и, наконец, закрывают сокет. Всё вплоть до транспортного уровня (TCP, UDP или SSL) обрабатывается библиотекой.

Сокет NSE создаётся вызовом `nmap.new_socket`, возвращающим объект сокета. Объект сокета поддерживает обычные методы `connect`, `send`, `receive` и `close`. Кроме того, функции `receive_bytes`, `receive_lines` и `receive_buf` позволяют лучше управлять приёмом данных. В примере 9.2 показано использование сетевых операций в стиле соединений. Функция `try` используется для обработки ошибок, как описано в разделе 9.7.4 «Обработка исключений».

Пример 9.2. Ввод-вывод в стиле соединений

```
require("nmap")

local socket = nmap.new_socket()
socket:set_timeout(1000)
try = nmap.new_try(function() socket:close() end)
try(socket:connect(host.ip, port.number))
try(socket:send("login"))
response = try(socket:receive())
socket:close()
```

Сетевой ввод-вывод необработанных пакетов

Для случаев, когда ориентированный на соединение подход слишком высокоуровневый, NSE предоставляет разработчикам сценариев возможность сетевого ввода-вывода необработанных пакетов.

Приём необработанных пакетов обрабатывается через обёртку Libpcap внутри библиотеки Nsock. Необходимо открыть устройство захвата, зарегистрировать на нём слушателей, а затем обрабатывать пакеты по мере их получения.

Метод `pcap_open` создаёт дескриптор для чтения необработанных данных сокета из обычного объекта сокета. Этот метод принимает функцию обратного вызова, вычисляющую хэш пакета по пакету (включая его заголовки). Этот хэш может возвращать любую двоичную строку, которая впоследствии сравнивается со строками, зарегистрированными функцией `pcap_register`. Обратный вызов хэширования пакета обычно извлекает какую-либо часть пакета, например его исходный адрес.

Средству чтения pcap указывается прослушивать определённые пакеты с помощью функции `pcap_register`. Функция принимает двоичную строку, которая сравнивается со значением хэша каждого полученного пакета. Пакеты, хэши которых совпадают с любой зарегистрированной строкой, будут возвращены методом `pcap_receive`. Зарегистрируйте пустую строку, чтобы получать все пакеты.

Сценарий получает все пакеты, для которых зарегистрирован слушатель, вызывая метод `pcap_receive`. Метод блокируется до получения пакета или истечения времени ожидания.

Чем более общей остаётся функция вычисления хэша пакета, тем больше сценариев могут получить пакет и продолжить выполнение. Чтобы обрабатывать захват пакетов внутри сценария, сначала необходимо создать сокет с помощью `nmap.new_socket`, а позднее закрыть сокет с помощью `socket_object:close` - как и при сетевом вводе-выводе на основе соединений.

Приём необработанных пакетов важен, но их отправка также является ключевой возможностью. Для этого NSE может обращаться к обёртке библиотеки libdnet. Запись необработанных пакетов, в отличие от чтения, не использует стандартный объект сокета. Вместо этого вызовите функцию `nmap.new_dnet`, чтобы создать объект dnet с методами отправки Ethernet. Затем откройте интерфейс методом `ethernet_open`. После этого необработанные кадры Ethernet можно отправлять с помощью `ethernet_send`. Закончив, закройте дескриптор Ethernet с помощью `ethernet_close`.

Иногда сложные API проще всего понять на примере. Сценарий `sniffer-detect.nse`, включённый в Nmap, использует захват и отправку необработанных пакетов в попытке обнаружить машины в сети, работающие в неразборчивом режиме (машины, на которых работают анализаторы трафика).

9.7.3. Мьютексы потоков

Поток выполнения сценария (например, `ftp-anon`, работающий против FTP-сервера на целевом узле) уступает управление другим сценариям всякий раз, когда вызывает сетевые объекты (отправляя или получая данные). Некоторым сценариям требуется управление конкурентным выполнением потоков. Примером служит сценарий `whois`, опрашивающий серверы whois для каждого целевого IP-адреса. Поскольку множество одновременных запросов часто приводит к запрету IP-адреса за злоупотребление, а один запрос может вернуть дополнительные сведения для целей, против которых работают другие потоки, полезно заставить другие потоки приостановиться, пока один поток выполняет запрос.

Для решения этой проблемы NSE включает функцию `mutex`, предоставляющую мьютекс[8] (объект взаимного исключения), используемый сценариями. Мьютекс позволяет только одному потоку работать с объектом. Конкурирующие потоки, пытающиеся работать с этим объектом, помещаются в очередь ожидания, пока не смогут получить «блокировку» мьютекса. Решение описанной выше проблемы `whois` состоит в том, чтобы каждый поток блокировался на мьютексе с использованием общей строки, гарантируя тем самым, что серверы whois одновременно опрашивает только один

поток. Этот поток может сохранить результаты в реестре NSE перед освобождением мьютекса. Затем может выполняться следующий сценарий в очереди ожидания. Сначала он проверит реестр и будет опрашивать серверы whois только в том случае, если предыдущих результатов оказалось недостаточно.

Первый шаг - создать объект мьютекса с помощью такого оператора:

Возвращаемая функция `mutexfn` работает как мьютекс для переданного объекта. Этим объектом может быть значение любого типа данных Lua, кроме `nil`, логических значений и чисел. Возвращаемая функция позволяет блокировать мьютекс, пытаться заблокировать его и освобождать. Её первый и единственный параметр должен быть одним из следующих:

"lock"

Выполняет блокирующую блокировку мьютекса. Если мьютекс занят (другой поток удерживает его блокировку), поток уступит управление и будет ждать. Функция возвращается с заблокированным мьютексом.

"trylock"

Выполняет неблокирующую блокировку мьютекса. Если мьютекс занят, функция немедленно возвращает значение `false`. В противном случае она блокирует мьютекс и возвращает `true`.

"done"

Освобождает мьютекс и позволяет другому потоку заблокировать его. Если поток не удерживает блокировку мьютекса, будет возбуждена ошибка.

"running"

Возвращает поток, удерживающий блокировку мьютекса, или `nil`, если мьютекс не заблокирован. Это следует использовать только для отладки, поскольку действие мешает сборке мусора завершённых потоков.

Простой пример использования API приведён в примере 9.3. Примеры из реальной жизни можно найти в сценариях `asn-query.nse` и `whois.nse` в дистрибутиве Nmap.

Пример 9.3. Работа с мьютексом

```
local mutex = nmap.mutex("My Script's Unique ID");
function action(host, port)
  mutex "lock";
  -- Do critical section work - only one thread at a time executes this.
  mutex "done";
  return script_output;
end
```

9.7.4. Обработка исключений

NSE предоставляет механизм обработки исключений, отсутствующий в базовом языке Lua. Он специально приспособлен для сетевых операций ввода-вывода и следует парадигме функционального, а не объектно-ориентированного программирования. Метод API `nmap.new_try` используется для создания обработчика исключений. Этот метод возвращает функцию, принимающую переменное число аргументов, которые считаются возвращаемыми значениями другой функции. Если в возвращаемых значениях обнаружено исключение (первое возвращаемое значение равно `false`), выполнение сценария прерывается и вывод не создаётся. При желании в `new_try` можно передать функцию, которая будет вызвана при перехвате исключения. Обычно такая функция выполняет все необходимые операции очистки.

В примере 9.4 показана обработка исключений с очисткой в действии. Новая функция с именем `catch` определяется так, чтобы в случае ошибки просто закрыть вновь созданный сокет. Затем она используется для защиты попыток соединения и обмена данными через этот сокет. Если функция `catch` не указана, выполнение сценария прерывается без дальнейших церемоний - открытые сокеты останутся открытыми до следующего запуска сборщика мусора Lua. Если уровень подробности не меньше единицы или сканирование выполняется в режиме отладки, описание неперехваченного ошибочного состояния печатается в стандартный вывод. Обратите внимание, что в настоящее время непросто сгруппировать несколько операторов в одном блоке `try`.

Пример 9.4. Пример обработки исключений

```
local result, socket, try, catch

result = ""
socket = nmap.new_socket()
catch = function()
  socket:close()
end
try = nmap.new_try(catch)

try(socket:connect(host.ip, port.number))
result = try(socket:receive_lines(1))
try(socket:send(result))
```

Написать функцию, которая правильно обрабатывается механизмом `try`/`catch`, несложно. Функция должна возвращать несколько значений. Первым должно быть логическое значение: `true` при успешном завершении функции и `false` в противном случае. Если функция завершилась успешно, конструкция `try` поглощает значение-индикатор и возвращает остальные значения. Если функция завершилась неудачно, второе возвращаемое значение должно быть строкой, описывающей ошибочное состояние. Обратите внимание: если значение не равно `nil` или `false`, оно считается `true`, поэтому в обычном случае можно вернуть своё значение, а при ошибке вернуть `nil, <описание ошибки>`.

9.7.5. Реестр

Реестр - таблица Lua (доступная как `nmap.registry`) с особым свойством: она видима всем сценариям и сохраняет состояние между выполнениями сценариев. Реестр является временным - он не сохраняется между запусками Nmap. Каждый сценарий может читать реестр и записывать в него. Сценарии обычно используют его для сохранения сведений для других экземпляров того же сценария. Например, сценарии `whois` и `asn-query` могут запросить один IP-адрес, но получить сведения, применимые к десяткам тысяч IP-адресов в этой сети. Сохранение сведений в реестре может избавить другие потоки сценариев от необходимости повторять запрос.

Реестр также можно использовать для передачи сведений совершенно другим сценариям. Например, сценарий `snmp-brute` сохраняет обнаруженное имя сообщества в реестре, где его могут использовать другие сценарии SNMP. Сценарии, оставляющие сведения для второго сценария,

должны иметь меньший runlevel, чем этот второй сценарий, иначе нет гарантии, что они выполнятся первыми.

Поскольку каждый сценарий может записывать в таблицу реестра, важно избегать конфликтов, разумно (уникально) выбирая ключи.

9.8. Руководство по написанию сценариев

Предположим, вы убеждены в мощи NSE. Как же написать собственный сценарий? Допустим, вы хотите извлечь сведения с сервера идентификации, чтобы определить владельца процесса, прослушивающего TCP-порт. На самом деле это не назначение identd (он предназначен для запроса владельца исходящих соединений, а не прослушивающих демонов), но многие серверы identd всё равно это позволяют. Раньше в Nmap была такая функция (называвшаяся ident-сканированием), но её удалили при переходе к новой архитектуре движка сканирования. Протокол, используемый identd, довольно прост, но всё же слишком сложен для языка определения версий Nmap. Сначала вы подключаетесь к серверу идентификации и отправляете запрос вида <порт-на-сервере>, <порт-на-клиенте>, завершая его символом новой строки. Затем сервер должен ответить строкой, содержащей порт сервера, порт клиента, тип ответа и адресные сведения. При ошибке адресные сведения опускаются. Более подробные сведения доступны в RFC 1413, но для наших целей этого описания достаточно. Протокол нельзя смоделировать на языке определения версий Nmap по двум причинам. Во-первых, необходимо знать и локальный, и удалённый порт соединения. Определение версий этих данных не предоставляет. Второе, более серьёзное препятствие: требуются два открытых соединения с целью - одно с сервером идентификации, а другое с прослушивающим портом, который требуется запросить. Оба препятствия легко преодолеваются с помощью NSE.

Анатомия сценария описана в разделе 9.3 «Формат сценария». В этом разделе мы покажем, как используется описанная структура.

9.8.1. Заголовок

Заголовок сценария по существу представляет собой его метаданные. Сюда входят поля description, categories, runlevel, author и license, а также исходные сведения NSEDoc, такие как теги usage, args и output (см. раздел 9.9 «Написание документации сценариев (NSEDoc)»).

Поле description должно содержать один или несколько абзацев, описывающих, что делает сценарий. Если что-либо в результатах сценария может запутать или ввести пользователей в заблуждение и устранить проблему улучшением сценария или текста результатов невозможно, это следует задокументировать в description. Если абзацев несколько, первый при необходимости используется как краткое резюме. Убедитесь, что первый абзац может служить самостоятельной аннотацией. Это описание короткое, потому что сценарий очень прост:

```
description = [[
Attempts to find the owner of an open TCP port by querying an auth
(identd - port 113) daemon which must also be open on the target system.
]]
```

Далее следуют сведения NSEDoc. В этом сценарии отсутствуют распространённые теги @usage и @args, поскольку он очень прост, но у него есть тег NSEDoc @output:

```
--@output
  21/tcp open  ftp        ProFTPD 1.3.1
  |_ auth-owners: nobody
```

```
22/tcp open  ssh        OpenSSH 4.3p2 Debian 9etch2 (protocol 2.0)
|_ auth-owners: root
25/tcp open  smtp       Postfix smtpd
|_ auth-owners: postfix
80/tcp open  http       Apache httpd 2.0.61 ((Unix) PHP/4.4.7 ...)
|_ auth-owners: dhapache
113/tcp open auth?
|_ auth-owners: nobody
587/tcp open submission Postfix smtpd
|_ auth-owners: postfix
5666/tcp open unknown
|_ auth-owners: root
```

Затем следуют теги `author`, `license` и `categories`. Этот сценарий относится к категории `safe`, потому что мы не используем службу ни для чего, для чего она не предназначена. Поскольку этот сценарий должен запускаться по умолчанию, он также относится к категории `default`. Вот переменные в контексте:

```
.author = "Diman Todorov <diman.todorov@gmail.com>"

license = "Same as Nmap--See http://nmap.org/book/man-legal.html"

categories = {"default", "safe"}
```

9.8.2. Правило

Раздел правила - метод Lua, решающий, пропустить или выполнить метод действия сценария для определённой службы или узла. Обычно это решение основывается на сведениях об узле и порте, переданных функции правила. В случае сценария идентификации всё немного сложнее. Чтобы решить, запускать ли сценарий идентификации против данного порта, необходимо знать, работает ли на целевой машине сервер auth. Иными словами, сценарий следует запускать, только если открыты и сканируемый в данный момент TCP-порт, и TCP-порт 113. Пока мы будем полагаться на то, что серверы идентификации прослушивают TCP-порт 113. К сожалению, NSE предоставляет нам сведения только о сканируемом в данный момент порте.

Чтобы выяснить, открыт ли порт 113, мы используем функцию `nmap.get_port_state`. Если порт auth не сканировался, функция `get_port_state` возвращает `nil`. Поэтому мы проверяем, что таблица не равна `nil`. Мы также проверяем, что оба порта находятся в состоянии `open`. В этом случае действие выполняется, иначе мы пропускаем действие.

```
portrule = function(host, port)
  local auth_port = { number=113, protocol="tcp" }
  local identd = nmap.get_port_state(host, auth_port)

  if
    identd ~= nil
    and identd.state == "open"
    and port.protocol == "tcp"
    and port.state == "open"
  then
    return true
  else
    return false
  end
end
```

9.8.3. Механизм

Наконец-то мы реализуем собственно функциональность! Сценарий сначала подключается к порту, на котором мы ожидаем найти сервер идентификации, затем подключается к порту, о котором нам нужны сведения. Для этого сначала создаются два объекта сокета вызовом `nmap.new_socket`. Затем мы определяем функцию обработки ошибок `catch`, которая закрывает эти сокеты при обнаружении сбоя. После этого можно безопасно использовать такие методы объектов, как `open`, `close`, `send` и `receive`, для работы с сетевым сокетом. В данном случае для установления соединений мы вызываем `connect`. Чтобы избежать избыточного кода обработки ошибок, используется механизм обработки исключений NSE. Мы просто оборачиваем сетевые вызовы в вызов `try`, который, в свою очередь, вызовет нашу функцию `catch`, если что-либо пойдёт не так.

Если оба соединения установлены успешно, мы формируем строку запроса и разбираем ответ. Если получен удовлетворительный ответ, мы возвращаем извлечённые сведения.

```
action = function(host, port)
  local owner = ""

  local client_ident = nmap.new_socket()
  local client_service = nmap.new_socket()

  local catch = function()
    client_ident:close()
    client_service:close()
  end

  local try = nmap.new_try(catch)

  try(client_ident:connect(host.ip, 113))
  try(client_service:connect(host.ip, port.number))

  local localip, localport, remoteip, remoteport =
    try(client_service:get_info())

  local request = port.number .. ", " .. localport .. "\n"

  try(client_ident:send(request))

  owner = try(client_ident:receive_lines(1))

  if string.match(owner, "ERROR") then
    owner = nil
  else
    owner = string.match(owner, "USERID : .+ : (.+)\n", 1)
  end

  try(client_ident:close())
  try(client_service:close())

  return owner
end
```

Обратите внимание: поскольку мы знаем, что удалённый порт хранится в `port.number`, можно было бы проигнорировать два последних возвращаемых значения `client_service:get_info()` вот так:

```
local localip, localport = try(client_service:get_info())
```

В этом примере мы молча завершаем работу, если служба отвечает ошибкой. Для этого переменной `owner`, которая будет возвращена, присваивается `nil`. Сценарии NSE обычно возвращают сообщения только при успешном выполнении, чтобы не затоплять пользователя бессмысленными предупреждениями.

9.9. Написание документации сценариев (NSEDoc)

Сценариями пользуются не только их авторы, поэтому им нужна хорошая документация. Модулям NSE нужна документация, чтобы разработчики могли использовать их в своих сценариях. Система документации NSE, описанная в этом разделе, призвана удовлетворить обе эти потребности. Читая этот раздел, вы можете просматривать интернет-документацию NSE, создаваемую с помощью этой системы. Она находится по адресу nmap.org.

NSE использует адаптированную версию системы документации LuaDoc[9], называемую NSEDoc. Документация сценариев и модулей содержится в их исходном коде в виде комментариев особой формы. В примере 9.5 показан комментарий NSEDoc, взятый из функции `stdnse.print_debug()`.

Пример 9.5. Комментарий NSEDoc для функции

```
--- Prints a formatted debug message if the current verbosity level is greater
-- or equal to a given level.
--
-- This is a convenience wrapper around
-- <code>nmap.print_debug_unformatted()</code>. The first optional numeric
-- argument, <code>verbosity</code>, is used as the verbosity level necessary
-- to print the message (it defaults to 1 if omitted). All remaining arguments
-- are processed with Lua's <code>string.format()</code> function.
-- @param level Optional verbosity level.
-- @param fmt Format string.
-- @param ... Arguments to format.
```

Комментарии документации начинаются с трёх дефисов: `---`. Тело комментария представляет собой описание следующего за ним кода. Первый абзац описания должен быть кратким резюме, а последующие абзацы - содержать больше подробностей. Специальные теги, начинающиеся с `@`, отделяют другие части документации. В приведённом выше примере вы видите `@param`, используемый для описания каждого параметра функции. Полный список тегов документации приведён в разделе 9.9.1 «Теги документации NSE».

Текст, заключённый в HTML-подобные теги `<code>` и `</code>`, будет отображаться моноширинным шрифтом. Его следует использовать для имён переменных и функций, а также для многострочных примеров кода. Если последовательность строк начинается с символов *, она будет отображаться как маркированный список.

Хорошей практикой является документирование каждой открытой функции и таблицы в сценарии или модуле. Кроме того, каждый сценарий и модуль должен иметь собственную документацию уровня файла. Комментарий документации в начале файла (за которым не следует определение функции или таблицы) относится ко всему файлу. Документация уровня файла может и должна занимать несколько абзацев и содержать все высокоуровневые сведения, полезные разработчику, использующему модуль, или пользователю, запускающему сценарий. В примере 9.6 показана документация модуля `comm` (несколько абзацев удалены для экономии места).

Пример 9.6. Комментарий NSEDoc для модуля

```
--- Common communication functions for network discovery tasks like
-- banner grabbing and data exchange.
--
-- These functions may be passed a table of options, but it's not required. The
-- keys for the options table are <code>"bytes"</code>, <code>"lines"</code>,
-- <code>"proto"</code>, and <code>"timeout"</code>. <code>"bytes"</code> sets
-- a minimum number of bytes to read. <code>"lines"</code> does the same for
-- lines. <code>"proto"</code> sets the protocol to communicate with,
-- defaulting to <code>"tcp"</code> if not provided. <code>"timeout"</code>
-- sets the socket timeout (see the socket function <code>set_timeout()</code>
-- for details).
-- @author Kris Katterjohn 04/2008
-- @copyright Same as Nmap--See http://nmap.org/book/man-legal.html
```

При документировании сценариев, а не функций и модулей, существуют некоторые особые соображения. В частности, сценарии имеют специальные переменные для некоторых сведений,

которые в противном случае находились бы в комментариях с тегами @ (переменные сценария описаны в разделе 9.3 «Формат сценария»). В частности, описание сценария находится в переменной `description`, а не в комментарии документации, а сведения, которые должны были бы находиться в `@author` и `@copyright`, вместо этого находятся в переменных `author` и `license`. NSEDoc знает об этих переменных и будет использовать их вместо полей в комментариях. Сценарии также должны иметь тег `@output`, показывающий пример вывода, а при необходимости - `@args` и `@usage`. В примере 9.7 показана правильная форма документации уровня сценария, использующая сочетание комментариев документации и переменных NSE.

Пример 9.7. Комментарий NSEDoc для сценария

```
description = [[
Maps IP addresses to autonomous system (AS) numbers.

The script works by sending DNS TXT queries to a DNS server which in
turn queries a third-party service provided by Team Cymru
(team-cymru.org) using an in-addr.arpa style zone set up especially for
use by Nmap.
]]

--- @usage
-- nmap --script asn-query.nse [--script-args dns=<DNS server>] <target>
-- @args dns The address of a recursive nameserver to use (optional).
-- @output
-- Host script results:
-- |  AS Numbers:
-- |  BGP: 64.13.128.0/21 | Country: US
-- |    Origin AS: 10565 SVCOLO-AS - Silicon Valley Colocation, Inc.
-- |    Peer AS: 3561 6461
-- |  BGP: 64.13.128.0/18 | Country: US
-- |    Origin AS: 10565 SVCOLO-AS - Silicon Valley Colocation, Inc.
-- |_   Peer AS: 174 2914 6461

author = "jah, Michael"
license = "Same as Nmap--See http://nmap.org/book/man-legal.html"
categories = {"discovery", "external"}
```

Компилируемые модули NSE также документируются с помощью NSEDoc, хотя у них нет исходного кода Lua. У каждого компилируемого модуля есть файл `<имя-модуля>.luadoc`, который хранится в каталоге `nselib` рядом с модулями Lua. В этом файле перечисляются и документируются функции и таблицы компилируемого модуля так, словно они написаны на Lua. Требуется только имя каждой функции, а не её определение (даже не `end`). При документировании таблицы необходимо использовать теги `@name` и `@class`, чтобы помочь анализатору документации идентифицировать её. В дистрибутиве исходного кода Nmap есть несколько примеров этого метода документирования (включая `nmap.luadoc`, `bit.luadoc` и `pcre.luadoc`).

9.9.1. Теги документации NSE

NSEDoc понимает следующие теги:

@param

Описывает параметр функции. Первое слово после `@param` - имя описываемого параметра. Тег должен появляться по одному разу для каждого параметра функции.

@see

Добавляет перекрёстную ссылку на другую функцию или таблицу.

@return

Описывает возвращаемое значение функции. Для нескольких возвращаемых значений `@return` можно использовать несколько раз.

@usage

Предоставляет пример использования функции или сценария. Для функции пример представляет собой код Lua; для сценария - командную строку Nmap. `@usage` можно задавать более одного раза.

@name

Определяет имя документируемой функции или таблицы. Обычно этот тег не нужен, поскольку NSEDoc выводит имена путём анализа кода.

@class

Определяет «класс» изменяемого объекта: `function`, `table` или `module`. Как и `@name`, обычно он выводится автоматически.

@field

В документации таблицы `@field` описывает значение именованного поля.

@args

Описывает аргумент сценария, используемый с параметром `--script-args` (см. раздел 9.2.3 «Аргументы сценариев»). Первое слово после `@args` - имя аргумента, а всё последующее - описание. Этот тег предназначен специально для комментариев уровня сценария.

@output

Этот тег, предназначенный исключительно для комментариев уровня сценария, показывает пример вывода сценария.

@author

Этот тег, который можно задавать несколько раз, перечисляет авторов модуля NSE. Для сценариев вместо него используется переменная `author`.

@copyright

Этот тег описывает состояние авторских прав модуля. Для сценариев вместо него используется переменная `license`.

9.10. Определение версий с помощью NSE

Встроенная в Nmap система определения версий была разработана для эффективного распознавания подавляющего большинства протоколов с помощью простого синтаксиса проб и сопоставления с шаблонами. Некоторым протоколам требуется более сложный обмен данными, чем может обработать определение версий. Обобщённый язык сценариев, предоставляемый NSE, идеально подходит для таких трудных случаев.

Категория NSE `version` содержит сценарии, расширяющие стандартное определение версий. Сценарии этой категории запускаются всякий раз, когда вы запрашиваете определение версий с помощью `-sV`; для их запуска не требуется использовать `-sC`. Верно и обратное: если использовать `-sC`, сценарии `version` не будут запущены, если также не использовать `-sV`.

Один из протоколов, который нам не удалось обнаружить обычным определением версий, - Skype версии 2. Протокол, вероятно, был разработан так, чтобы затруднить обнаружение из опасения, что связанные с телекоммуникационными компаниями интернет-провайдеры могут счесть Skype конкурентом и вмешиваться в трафик. И всё же мы нашли один способ его обнаружения. Если Skype получает запрос HTTP GET, он притворяется веб-сервером и возвращает ошибку 404. Но на другие запросы он отправляет блок данных, выглядящих случайными. Для правильной идентификации необходимо отправить две пробы и сравнить два ответа - идеальная задача для NSE. Простой сценарий NSE, выполняющий это, показан в примере 9.8.

Пример 9.8. Типичный сценарий определения версии (обнаружение Skype версии 2)

```
description = [[
Detects the Skype version 2 service.
]]
author = "Brandon Enright <bmenrigh@ucsd.edu>"
license = "Same as Nmap--See http://nmap.org/book/man-legal.html"
categories = {"version"}

require "comm"

portrule = function(host, port)
  return (port.number == 80 or port.number == 443 or
    port.service == nil or port.service == "" or
    port.service == "unknown")
    and port.protocol == "tcp" and port.state == "open"
    and port.service ~= "http" and port.service ~= "ssl/http"
end

action = function(host, port)
  local status, result = comm.exchange(host, port,
    "GET / HTTP/1.0\r\n\r\n", {bytes=26, proto=port.protocol})
  if (not status) then
    return
  end
  if (result ~= "HTTP/1.0 404 Not Found\r\n\r\n") then
    return
  end
  -- So far so good, now see if we get random data for another request
  status, result = comm.exchange(host, port,
    "random data\r\n\r\n", {bytes=15, proto=port.protocol})
  if (not status) then
    return
```

```
  end
  if string.match(result, "[^%s!~-].*[^%s!~-].*[^%s!~-]") then
    -- Detected
    port.version.name = "skype2"
    port.version.product = "Skype"
    nmap.set_port_version(host, port, "hardmatched")
    return
  end
  return
end
```

Если сценарий обнаруживает Skype, он дополняет свою таблицу `port` ставшими известными полями `name` и `product`. Затем он отправляет эти новые сведения Nmap, вызывая `nmap.set_port_version`. Несколько других полей версии можно задать, если они известны, но в данном случае у нас есть только имя и продукт. Полный список полей версии см. в документации `nmap.set_port_version`.

Обратите внимание, что этот сценарий ничего не делает, если не обнаруживает протокол. Сценарий не должен создавать вывод (кроме отладочного) только для того, чтобы сообщить, что он ничего не узнал.

9.11. Пример сценария: finger.nse

Сценарий finger (`finger.nse`) - идеальный пример короткого и простого сценария NSE.

Сначала присваиваются информационные поля. Подробное описание того, что на самом деле делает сценарий, помещается в поле `description`.

```
description = [[
Attempts to get a list of usernames via the finger service.
]]
author = "Eddie Bell <ejlbell@gmail.com>"
license = "Same as Nmap--See http://nmap.org/book/man-legal.html"
```

Поле `categories` представляет собой таблицу, содержащую все категории, к которым относится сценарий. Они используются для выбора сценариев с помощью параметра `--script`:

```
categories = {"default", "discovery"}
```

Предоставляемые `nselib` средства (раздел 9.6 «Библиотеки NSE») можно использовать с помощью `require`. Здесь мы хотим использовать общие функции связи и более короткие правила портов:

```
require "comm"
require "shortport"
```

Мы хотим запускать сценарий против службы finger. Поэтому проверяем, использует ли она общеизвестный порт finger (79/tcp), либо носит ли служба имя `"finger"` согласно результатам определения версии или записи номера порта в `nmap-services`:

```
portrule = shortport.port_or_service(79, "finger")
```

Сначала сценарий использует `nmap.new_try` для создания обработчика исключений, который завершит сценарий в случае ошибки. Затем он передаёт управление `comm.exchange`, обрабатывающей сетевую транзакцию. Здесь мы попросили ждать в процессе обмена данными, пока не будет получено не менее 100 строк, не пройдёт не менее 5 секунд или удалённая сторона не закроет соединение. Любые ошибки обрабатываются обработчиком исключений `try`. Сценарий возвращает строку, если вызов `comm.exchange()` завершился успешно.

```
action = function(host, port)
  local try = nmap.new_try()
```

```
  return try(comm.exchange(host, port, "\r\n",
    {lines=100, proto=port.protocol, timeout=5000}))
end
```

9.12. Подробности реализации

Теперь пришло время подробно изучить реализацию NSE. Понимание того, как работает NSE, полезно для проектирования эффективных сценариев и библиотек. Каноническим справочником по реализации NSE является исходный код, но в этом разделе дан обзор ключевых подробностей. Он должен быть полезен тем, кто пытается понять и расширить исходный код NSE, а также авторам сценариев, желающим лучше понять, как выполняются их сценарии.

9.12.1. Фаза инициализации

На этапе инициализации Nmap загружает интерпретатор Lua и предоставляемые им библиотеки. Эти библиотеки полностью документированы в *Справочном руководстве Lua*[10]. Ниже приведено краткое описание библиотек в алфавитном порядке имён их пространств имён:

debug

Библиотека debug предоставляет низкоуровневый API к интерпретатору Lua, позволяя обращаться к функциям вдоль стека выполнения, получать замыкания функций и метатаблицы объектов и многое другое.

io

Библиотека ввода-вывода предлагает такие функции, как чтение из файлов или из вывода выполняемых вами программ.

math

Числа в Lua обычно соответствуют типу C `double`, поэтому библиотека math предоставляет доступ к функциям округления, тригонометрическим функциям, генерации случайных чисел и многому другому.

os

Библиотека операционной системы предоставляет системные средства, такие как операции с файловой системой (включая переименование или удаление файлов и создание временных файлов) и доступ к системному окружению.

package

Среди функций, предоставляемых библиотекой Lua package, есть `require`, используемая для загрузки модулей `nselib`.

string

Библиотека string предоставляет функции для работы со строками Lua, включая форматирование строк в стиле printf, сопоставление с шаблонами Lua, извлечение подстрок и многое другое.

table

Библиотека работы с таблицами необходима для операций над центральной структурой данных Lua (таблицами).

Помимо загрузки библиотек, предоставляемых Lua, загружаются функции пространства имён `nmap`. Пути поиска совпадают с каталогами, в которых Nmap ищет свои файлы данных, за исключением того, что к каждому добавляется каталог `nselib`. На этом этапе все предоставленные аргументы сценариев сохраняются в реестре.

Следующая фаза инициализации NSE - загрузка выбранных сценариев на основе значений по умолчанию или аргументов, переданных параметру `--script`. Если включено определение версий, загружаются также сценарии категории `version`. Сначала NSE пытается интерпретировать каждый аргумент `--script` как категорию. Это выполняется функцией Lua в `nse_init.cc` с именем `entry` на основе данных из базы категоризации сценариев `script.db`. Если категория найдена, загружаются соответствующие сценарии. В противном случае Nmap пытается интерпретировать аргументы `--script` как файлы или каталоги. Если в пути поиска Nmap не найдены файлы или каталоги с заданным именем, возникает ошибка и движок сценариев прекращает работу.

Если указан каталог, загружаются все находящиеся в нём файлы `.nse`. Каждый загруженный файл выполняется Lua. Если присутствует `portrule`, оно сохраняется в таблице `porttests` с ключом `portrule` и замыканием файла в качестве значения. В противном случае, если сценарий имеет `hostrule`, оно таким же образом сохраняется в таблице `hosttests`.

9.12.2. Сопоставление сценариев с целями

После завершения инициализации `hostrules` и `portrules` вычисляются для каждого узла в текущей группе целей. Правило каждого выбранного сценария проверяется против каждого узла и (в случае сценариев служб) каждого порта узла в состоянии `open` или `open|filtered`. Число сочетаний может стать довольно большим, поэтому `portrules` следует делать как можно проще. Сохраняйте любые тяжёлые вычисления для `action` сценария.

Затем для каждого соответствующего сочетания сценария и цели создаётся поток Lua[11]. Каждый поток хранится с относящимися к нему сведениями, такими как `runlevel`, цель, целевой порт (если применимо), таблицы узла и порта (передаваемые `action`) и тип сценария (сценарий службы или узла). Затем функция главного цикла по порядку обрабатывает каждую группу потоков одного `runlevel`.

9.12.3. Выполнение сценария

Nmap выполняет сканирование сценариями NSE параллельно, используя библиотеку параллельного ввода-вывода Nsock из Nmap и языковую возможность сопрограмм Lua[12]. Сопрограммы обеспечивают кооперативную многопоточность, позволяя сценариям приостанавливать себя в определённых точках и разрешать выполнение других сопрограмм. Сетевой ввод-вывод, особенно ожидание ответов от удалённых узлов, часто сопряжён с длительным ожиданием, поэтому именно тогда сценарии уступают управление другим. Ключевые функции обёртки Nsock заставляют сценарии уступать управление (приостанавливаться). Когда Nsock завершает обработку такого запроса, он выполняет обратный вызов, из-за которого сценарий перемещается из очереди ожидания обратно в очередь выполнения, чтобы возобновить работу, когда снова наступит его очередь.

Функция `mainloop` по мере необходимости перемещает потоки между очередями ожидания и выполнения. Поток, уступающий управление, перемещается из очереди выполнения в список ожидания. Выполняющиеся потоки работают, пока не уступят управление, не завершатся или не закончатся ошибкой. Потоки подготавливаются к выполнению (помещаются в очередь выполнения) вызовом `process_waiting2running`. Этот процесс планирования выполняющихся потоков и перемещения потоков между очередями продолжается, пока в обеих очередях не останется потоков.

Сноски

[1] [назад] metasploit.com
[2] [назад] lua.org
[3] [назад] lua.org
[4] [назад] lua.org
[5] [назад] lua.org
[6] [назад] lua.org
[7] [назад] lua.org
[8] [назад] en.wikipedia.org
[9] [назад] luadoc.luaforge.net
[10] [назад] lua.org
[11] [назад] lua.org
[12] [назад] lua.org

Глава 10. Обнаружение и обход брандмауэров и систем обнаружения вторжений

10.1. Введение

Многие пионеры Интернета представляли себе глобальную открытую сеть с универсальным пространством IP-адресов, допускающим виртуальные соединения между любыми двумя узлами. Это позволяет узлам действовать как настоящие равноправные участники, предоставляя и получая сведения друг от друга. Люди могли бы получать с работы доступ ко всем своим домашним системам, изменяя настройки климат-контроля или отпирая двери для гостей, пришедших пораньше. Это представление о всеобщей связности было подавлено нехваткой адресного пространства и соображениями безопасности. В начале 1990-х годов организации начали развёртывать брандмауэры с явной целью сокращения связности. Огромные сети были отгорожены от нефильтрованного Интернета прокси-серверами приложений, устройствами преобразования сетевых адресов и фильтрами пакетов. Неограниченный поток сведений уступил место строгому регулированию разрешённых каналов связи и проходящего по ним содержимого.

Сетевые препятствия, такие как брандмауэры, могут чрезвычайно усложнить составление карты сети. Проще не станет, поскольку подавление поверхностной разведки часто является одной из главных целей внедрения этих устройств. Тем не менее Nmap предлагает множество возможностей, помогающих разобраться в этих сложных сетях и проверить, что фильтры работают так, как задумано. Он даже поддерживает механизмы обхода плохо реализованных средств защиты. Один из лучших способов понять состояние безопасности своей сети - попытаться победить её. Поставьте себя на место атакующего и примените методы из этой главы против своих сетей. Запустите сканирование FTP bounce, idle-сканирование, атаку с фрагментацией или попробуйте проложить туннель через один из собственных прокси-серверов.

Помимо ограничения сетевой активности, компании всё чаще отслеживают трафик с помощью систем обнаружения вторжений (IDS). Все основные IDS поставляются с правилами, предназначенными для обнаружения сканирований Nmap, поскольку сканирования иногда предшествуют атакам. Многие из этих продуктов превратились в системы предотвращения вторжений (IPS), активно блокирующие трафик, который считается вредоносным. К сожалению для сетевых администраторов и поставщиков IDS, надёжное обнаружение злых намерений путём анализа данных пакетов - трудная задача. Атакующие, обладающие терпением, навыками и поддержкой некоторых параметров Nmap, обычно могут пройти мимо IDS незамеченными. Тем временем администраторам приходится справляться с большим числом ложноположительных результатов, когда невинная активность ошибочно диагностируется, вызывает предупреждение или блокируется.

10.2. Почему этичные специалисты («белые шляпы») вообще это делают?

У некоторых из вас, читателей в белых шляпах, может возникнуть соблазн пропустить эту главу. Зачем при разрешённом использовании против собственных сетей вообще уклоняться от собственных систем безопасности? Потому что это помогает понять опасность, исходящую от настоящих атакующих. Если вы можете обойти заблокированный порт portmapper с помощью

прямого сканирования RPC в Nmap, то и плохие парни могут. При настройке сложных брандмауэров и других устройств легко допустить ошибку. Многие из них даже поставляются с вопиющими дырами в безопасности, которые добросовестные пользователи должны найти и закрыть.

Регулярное сканирование сети может помочь обнаружить опасные неявные правила (например, в вашем брандмауэре Checkpoint или фильтрах IPsec Windows) раньше атакующих.

Для уклонения от IDS тоже есть веские причины. Одна из самых распространённых - оценка продукта. Если атакующие могут проскользнуть под радаром, просто добавив один-два флага Nmap, система обеспечивает не слишком большую защиту. Возможно, она всё ещё поймает скрипт-кидди и червей, но они и без того обычно вопиюще очевидны.

Иногда люди предлагают не включать в Nmap возможности уклонения от правил брандмауэров или незаметного прохождения мимо IDS. Они утверждают, что атакующие с такой же вероятностью будут злоупотреблять этими возможностями, с какой администраторы будут использовать их для повышения безопасности. Проблема этой логики в том, что атакующие всё равно применяли бы эти методы: они просто нашли бы другие инструменты или добавили бы эту функциональность в Nmap. Тем временем администраторам стало бы гораздо труднее выполнять свою работу. Развёртывание только современных FTP-серверов со всеми исправлениями - гораздо более мощная защита, чем попытка предотвратить распространение инструментов, реализующих атаку FTP bounce.

10.3. Определение правил брандмауэра

Первый шаг к обходу правил брандмауэра - понять их. Там, где это возможно, Nmap различает порты, которые доступны, но закрыты, и порты, которые активно фильтруются. Эффективный метод - начать с обычного SYN-сканирования портов, а затем перейти к более экзотическим методам, таким как ACK-сканирование и анализ последовательности IP ID, чтобы лучше понять сеть.

10.3.1. Стандартное SYN-сканирование

Одна из полезных особенностей протокола TCP заключается в том, что RFC 793 требует от систем отправлять отрицательный ответ на неожиданные запросы соединения в виде пакета TCP RST (reset). Благодаря пакету RST Nmap легко распознаёт закрытые порты. Устройства фильтрации, такие как брандмауэры, напротив, обычно отбрасывают пакеты, предназначенные для запрещённых портов. В некоторых случаях вместо этого они отправляют сообщения об ошибках ICMP (обычно «порт недоступен»). Поскольку отброшенные пакеты и ошибки ICMP легко отличить от пакетов RST, Nmap может надёжно отличать фильтруемые TCP-порты от открытых или закрытых и делает это автоматически. Это показано в примере 10.1.

Пример 10.1. Обнаружение закрытых и фильтруемых TCP-портов

```
# nmap -sS -T4 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE  SERVICE
22/tcp  open   ssh
25/tcp  closed smtp
53/tcp  open   domain
70/tcp  closed gopher
80/tcp  open   http
```

```
113/tcp closed auth

Nmap done: 1 IP address (1 host up) scanned in 5.40 seconds
```

Одна из самых важных строк в примере 10.1 - примечание в скобках «Not shown: 994 filtered ports». Иными словами, на этом узле действует правильная политика брандмауэра «запрещено по умолчанию». Доступны только те порты, которые администратор явно разрешил, тогда как действие по умолчанию - запретить (фильтровать) их. Три из перечисленных портов находятся в состоянии `open` (22, 53 и 80), а ещё три - в состоянии `closed` (25, 70 и 113). Остальные 994 проверенных порта недоступны для этого стандартного сканирования (`filtered`).

Коварные брандмауэры, возвращающие RST

Хотя различие Nmap между закрытыми портами (которые возвращают пакет RST) и фильтруемыми портами (которые ничего не возвращают или возвращают ошибку ICMP) обычно точно, многие устройства брандмауэров теперь способны подделывать пакеты RST так, будто они поступают от целевого узла и утверждают, что порт закрыт. Один из примеров такой возможности - система Linux iptables, предлагающая множество способов отклонения нежелательных пакетов. Страница руководства iptables описывает эту возможность следующим образом:

> `--reject-with type`
>
> Заданный тип может быть `icmp-net-unreachable`, `icmp-host-unreachable`, `icmp-port-unreachable`, `icmp-proto-unreachable`, `icmp-net-prohibited` или `icmp-host-prohibited`, что возвращает соответствующее сообщение об ошибке ICMP (`port-unreachable` используется по умолчанию). Параметр `tcp-reset` можно использовать в правилах, соответствующих только протоколу TCP: это приводит к отправке в ответ пакета TCP RST. Главным образом это полезно для блокирования проб ident (113/tcp), которые часто возникают при отправке почты неисправным почтовым узлам (которые иначе не примут вашу почту).

Подделка пакетов RST брандмауэрами и IDS/IPS не особенно распространена за пределами порта 113, поскольку она может запутать законных операторов сети, а также позволяет сканерам немедленно переходить к следующему порту, не ожидая истечения времени ожидания, вызываемого отброшенными пакетами. Тем не менее это происходит. Такую подделку обычно можно обнаружить тщательным анализом пакета RST в сравнении с другими пакетами, отправленными машиной. В разделе 10.6 «Обнаружение подделки пакетов брандмауэрами и системами обнаружения вторжений» описаны эффективные методы для этого.

10.3.2. ACK-сканирование

Как подробно описано в разделе 5.7 «TCP ACK-сканирование (`-sA`)», ACK-сканирование отправляет TCP-пакеты, в которых установлен только бит ACK. Независимо от того, открыты порты или закрыты, RFC 793 требует от цели ответить пакетом RST. Брандмауэры, блокирующие пробу, напротив, обычно не отвечают или возвращают ошибку ICMP «узел назначения недоступен». Это различие позволяет Nmap сообщать, фильтруются ли пакеты ACK. Набор фильтруемых портов, сообщаемый ACK-сканированием Nmap, часто меньше, чем при SYN-сканировании той же машины, поскольку ACK-сканирования труднее фильтровать. Многие сети разрешают почти неограниченные исходящие соединения, но хотят запретить интернет-узлам инициировать обратные соединения с ними. Блокирование входящих пакетов SYN (без установленного бита ACK) - простой способ сделать это, но при этом любые пакеты ACK всё равно пропускаются.

Блокировать такие пакеты ACK сложнее, поскольку они не указывают, какая сторона начала соединение. Чтобы блокировать незапрошенные пакеты ACK (такие, какие отправляет ACK-сканирование Nmap), пропуская при этом пакеты ACK, принадлежащие законным соединениям, брандмауэры должны отслеживать состояние каждого установленного соединения, чтобы определить, уместен ли данный ACK. Такие брандмауэры с отслеживанием состояния обычно безопаснее, поскольку могут вводить более строгие ограничения. Блокирование ACK-сканирований - ещё одно доступное ограничение. Недостатки заключаются в том, что для работы им требуется больше ресурсов, а перезагрузка брандмауэра с отслеживанием состояния может привести к потере устройством состояния и разрыву всех установленных соединений, проходящих через него.

Хотя брандмауэры с отслеживанием состояния широко распространены и набирают популярность, подход без отслеживания состояния всё ещё довольно обычен. Например, система Linux Netfilter/iptables поддерживает удобный параметр `--syn`, упрощающий реализацию описанного выше подхода без отслеживания состояния.

В предыдущем разделе SYN-сканирование показало, что все, кроме шести, из 1000 распространённых портов `scanme.nmap.org` находились в состоянии `filtered`. В примере 10.2 показано ACK-сканирование того же узла, позволяющее определить, использует ли он брандмауэр с отслеживанием состояния.

Пример 10.2. ACK-сканирование Scanme

```
# nmap -sA -T4 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE      SERVICE
22/tcp  unfiltered ssh
25/tcp  unfiltered smtp
53/tcp  unfiltered domain
70/tcp  unfiltered gopher
80/tcp  unfiltered http
113/tcp unfiltered auth

Nmap done: 1 IP address (1 host up) scanned in 5.96 seconds
```

Здесь показаны те же шесть портов, что и при SYN-сканировании. Остальные 994 всё ещё фильтруются. Причина в том, что Scanme защищён следующей директивой iptables с отслеживанием состояния: `iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT`. Она принимает только пакеты, являющиеся частью установленного соединения или относящиеся к нему. Незапрошенные пакеты ACK, отправленные Nmap, отбрасываются, кроме пакетов к шести показанным особым портам. Специальные правила разрешают все пакеты к портам 22, 25, 53, 70 и 80, а также отправляют пакет RST в ответ на пробы порта 113. Обратите внимание, что шесть показанных портов находятся в состоянии `unfiltered`, поскольку ACK-сканирование не может дополнительно разделить их на `open` (22, 53 и 80) и `closed` (25, 70 и 113).

Теперь рассмотрим другой пример. Узел Linux с именем Para в моей локальной сети использует следующий (упрощённый для экономии места) сценарий брандмауэра:

```
#!/bin/sh
#
# A simple, stateless, host-based firewall script.

# First of all, flush & delete any existing tables
iptables -F
iptables -X

# Deny by default (input/forward)
```

```
iptables --policy INPUT DROP
iptables --policy OUTPUT ACCEPT
iptables --policy FORWARD DROP

# I want to make ssh and www accessible from outside
iptables -A INPUT -m multiport -p tcp --destination-port 22,80 -j ACCEPT

# Allow responses to outgoing TCP requests
iptables -A INPUT --proto tcp ! --syn -j ACCEPT
```

Этот брандмауэр не отслеживает состояние, поскольку в нём нет ни параметра `--state`, ни запроса модуля `-m state`. В примере 10.3 показаны SYN- и ACK-сканирования этого узла.

Пример 10.3. Сопоставление SYN- и ACK-сканирований Para

```
# nmap -sS -p1-100 -T4 para

Starting Nmap ( http://nmap.org )
Interesting ports on para (192.168.10.191):
Not shown: 98 filtered ports
PORT   STATE  SERVICE
22/tcp open   ssh
80/tcp closed http
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 1 IP address (1 host up) scanned in 3.81 seconds

# nmap -sA -p1-100 -T4 para

Starting Nmap ( http://nmap.org )
All 100 scanned ports on para (192.168.10.191) are: unfiltered
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds
```

При SYN-сканировании 98 из 100 портов фильтруются. Однако ACK-сканирование показывает, что каждый сканируемый порт находится в состоянии `unfiltered`. Иными словами, все пакеты ACK беспрепятственно проскальзывают и вызывают ответы RST. Эти ответы также делают сканирование более чем в пять раз быстрее, поскольку ему не приходится ждать истечения времени ожидания.

Теперь мы знаем, как отличать брандмауэры с отслеживанием состояния от брандмауэров без него, но какая от этого польза? ACK-сканирование Para показывает, что некоторые IP-пакеты, вероятно, достигают узла назначения. Я говорю «вероятно», потому что подделка брандмауэром возможна всегда. Хотя установить TCP-соединения с этими портами, возможно, не удастся, они могут быть полезны для определения используемых адресов, проверок определения ОС, некоторых манипуляций с IP ID и как канал для туннелирования команд к руткитам, установленным на этих машинах. Другие типы сканирования, такие как FIN-сканирование, могут даже определить, какие порты открыты, и тем самым позволить сделать вывод о назначении узлов. Такие узлы могут пригодиться в качестве зомби для idle-сканирования по IP ID.

Эта пара сканирований также показывает, что называемое нами состоянием порта не является свойством исключительно самого порта. Здесь один и тот же номер порта считается `filtered` при одном типе сканирования и `unfiltered` при другом. На то, как Nmap видит порты, могут влиять IP-адрес, с которого выполняется сканирование, правила всех находящихся на пути устройств фильтрации и интерфейс целевой машины, к которому вы обращаетесь. Таблица портов отражает только то, что Nmap увидел при запуске с определённой машины, с заданным набором параметров и в данное время.

10.3.3. Приёмы с IP ID

Скромное поле идентификации в заголовках IP может раскрыть удивительно много сведений. Далее в этой главе оно будет использоваться для сканирования портов (метод idle-сканирования) и обнаружения случаев, когда брандмауэры и системы обнаружения вторжений подделывают пакеты RST так, будто они поступают от защищённых узлов. Ещё один изящный приём - определить, какие исходные адреса проходят через брандмауэр. Нет смысла тратить часы на слепую атаку с подделкой адреса «от» 192.168.0.1, если какой-либо брандмауэр на пути отбрасывает все такие пакеты.

Обычно я проверяю это условие с помощью свободного инструмента сетевого зондирования hping2[1]. Это довольно сложный метод, но иногда он может быть ценен. Вот предпринимаемые мной шаги:

1. Найдите хотя бы один доступный (открытый или закрытый) порт одной машины во внутренней сети. Часто хорошо подходят маршрутизаторы, принтеры и машины Windows. В последних выпусках Linux, Solaris и OpenBSD проблема предсказуемых последовательностей IP ID в основном устранена, поэтому они не подойдут. У выбранной машины должно быть мало сетевого трафика, чтобы избежать запутывающих результатов.

2. Убедитесь, что машина имеет предсказуемые последовательности IP ID. Следующая команда проверяет машину Windows XP с именем Playground. Параметры hping2 требуют отправить пять пакетов SYN на порт 80 с интервалом в одну секунду.

```
# hping2 -c 5 -i 1 -p 80 -S playground
HPING playground (eth0 192.168.0.40): S set, 40 headers + 0 data bytes
len=46 ip=192.168.0.40 ttl=128 id=64473 sport=80 flags=RA seq=0 rtt=0.7 ms
len=46 ip=192.168.0.40 ttl=128 id=64474 sport=80 flags=RA seq=1 rtt=0.3 ms
len=46 ip=192.168.0.40 ttl=128 id=64475 sport=80 flags=RA seq=2 rtt=0.3 ms
len=46 ip=192.168.0.40 ttl=128 id=64476 sport=80 flags=RA seq=3 rtt=0.3 ms
len=46 ip=192.168.0.40 ttl=128 id=64477 sport=80 flags=RA seq=4 rtt=0.3 ms

--- playground hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 0.3/0.3/0.7 ms
```

Поскольку поля IP ID совершенно последовательны, можно переходить к следующей проверке. Если бы они были случайными или находились очень далеко друг от друга, пришлось бы искать новый доступный узел.

3. Начните поток проб к цели от узла, расположенного рядом с вашим (подойдёт почти любой узел). Пример команды: `hping2 --spoof scanme.nmap.org --fast -p 80 -c 10000 -S playground`. Замените `scanme.nmap.org` другим выбранным вами узлом, а `playground` - своим целевым узлом. Получать ответы обратно необязательно, поскольку цель состоит лишь в увеличении последовательности IP ID. Не используйте настоящий адрес машины, с которой запускается hping2. Рекомендуется использовать находящуюся поблизости машину сети, чтобы уменьшить вероятность блокирования пакетов вашим интернет-провайдером.

Пока это происходит, повторите проверку из предыдущего шага против своей целевой машины.

```
# hping2 -c 5 -i 1 -p 80 -S playground
HPING playground (eth0 192.168.0.40): S set, 40 headers + 0 data bytes
len=46 ip=192.168.0.40 ttl=128 id=64672 sport=80 flags=RA seq=0 rtt=0.6 ms
len=46 ip=192.168.0.40 ttl=128 id=64683 sport=80 flags=RA seq=1 rtt=0.2 ms
len=46 ip=192.168.0.40 ttl=128 id=64694 sport=80 flags=RA seq=2 rtt=0.2 ms
len=46 ip=192.168.0.40 ttl=128 id=64705 sport=80 flags=RA seq=3 rtt=0.2 ms
len=46 ip=192.168.0.40 ttl=128 id=64716 sport=80 flags=RA seq=4 rtt=0.2 ms

--- playground hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 0.2/0.3/0.6 ms
```

На этот раз IP ID увеличиваются примерно на 11 в секунду, а не на один. Цель получает десять наших поддельных пакетов в секунду и отвечает на каждый из них. Каждый ответ увеличивает IP

ID. Некоторые узлы используют уникальную последовательность IP ID для каждого IP-адреса, с которым обмениваются данными. Если бы это было так, мы не увидели бы подобных скачков IP ID и нам пришлось бы искать в сети другой целевой узел.

4. Повторите шаг 3, используя поддельные адреса, которые, по вашему предположению, могут пропускаться брандмауэром или считаться доверенными. Попробуйте адреса за их брандмауэром, а также частные сети RFC 1918, такие как 10.0.0.0/8, 192.168.0.0/16 и 172.16.0.0/12. Попробуйте также localhost (127.0.0.1) и, возможно, другой адрес из 127.0.0.0/8, чтобы обнаружить случаи, когда 127.0.0.1 жёстко задан в коде. С поддельными пакетами localhost было связано множество дыр в безопасности, включая печально известную атаку отказа в обслуживании Land. Неверно настроенные системы иногда доверяют этим адресам, не проверяя, пришли ли они с интерфейса обратной петли. Если исходный адрес проходит до конечного узла, IP ID совершит скачок, как на шаге 3. Если он продолжит медленно увеличиваться, как на шаге 2, пакеты, вероятно, были отброшены брандмауэром или маршрутизатором.

Конечный результат этого метода - список блоков исходных адресов, которые пропускаются через брандмауэр, и тех, которые блокируются. Эти сведения ценны по нескольким причинам. IP-адреса, которые компания решает блокировать или разрешать, могут подсказать, какие адреса используются внутри или считаются доверенными. Например, машины в производственной сети компании могут доверять IP-адресам корпоративной сети или личной машине системного администратора. Машины одной производственной сети также иногда доверяют друг другу или localhost. Распространённые отношения доверия на основе IP встречаются в экспортах NFS, правилах брандмауэров узлов, TCP wrappers, пользовательских приложениях, rlogin и т. д. Другой пример - SNMP, где поддельный запрос к маршрутизатору Cisco может заставить маршрутизатор передать (по TFTP) свои конфигурационные данные обратно атакующему. Прежде чем тратить значительное время на поиск и эксплуатацию этих проблем, воспользуйтесь описанной здесь проверкой, чтобы определить, проходят ли поддельные пакеты вообще.

Конкретный пример этой проблемы с доверенными исходными адресами: однажды я обнаружил, что пользовательская UDP-служба компании позволяла пользователям пропускать аутентификацию, если они приходили из специальных блоков сети, внесённых в конфигурационный файл. Эти блоки сети соответствовали различным местоположениям компании, а функция предназначалась для облегчения администрирования и отладки. Их обращённый к Интернету брандмауэр разумно пытался блокировать эти адреса, поскольку настоящие сотрудники могли обращаться к производственной среде по частному каналу. Но с помощью методов, описанных в этом разделе, я обнаружил, что брандмауэр не был идеально синхронизирован с конфигурационным файлом. Оставалось несколько адресов, от имени которых я мог успешно подделывать управляющие UDP-сообщения и захватывать их приложение.

Этот метод построения карты правил брандмауэра не использует Nmap, но результаты ценны для будущих запусков. Например, эта проверка может показать, следует ли использовать определённые ложные цели (`-D`). Лучшие ложные цели дойдут до самой целевой системы. Кроме того, для работы idle-сканирования по IP ID (обсуждаемого далее) поддельные пакеты должны проходить. Проверить потенциальные исходные IP-адреса этим методом обычно проще, чем находить и проверять каждую потенциальную машину idle-прокси в сети. Потенциальные idle-прокси нужно проверять только в том случае, если они проходят шаг номер два выше.

10.3.4. UDP-сканирование версий

Все предыдущие разделы были посвящены распространённому протоколу TCP. Работать с UDP часто сложнее, поскольку протокол не подтверждает открытые порты так, как TCP. Многие приложения UDP просто игнорируют неожиданные пакеты, оставляя Nmap в неведении, открыт

порт или фильтруется. Поэтому Nmap помещает такие неоднозначные порты в состояние `open|filtered`, как показано в примере 10.4.

Пример 10.4. UDP-сканирование узла за брандмауэром

```
# nmap -sU -p50-59 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT   STATE         SERVICE
50/udp open|filtered re-mail-ck
51/udp open|filtered la-maint
52/udp open|filtered xns-time
53/udp open|filtered domain
54/udp open|filtered xns-ch
55/udp open|filtered isi-gl
56/udp open|filtered xns-auth
57/udp open|filtered priv-term
58/udp open|filtered xns-mail
59/udp open|filtered priv-file

Nmap done: 1 IP address (1 host up) scanned in 1.38 seconds
```

Это сканирование десяти портов оказалось не слишком полезным. Ни один порт не ответил на пакеты-пробы, поэтому все они перечислены как открытые или фильтруемые. Один из способов лучше понять, какие порты действительно открыты, - отправить множество UDP-проб для десятков различных известных UDP-служб в надежде вызвать ответ от какого-либо открытого порта. Именно это делает определение версий Nmap (глава 7 «Определение версий служб и приложений»). В примере 10.5 показано то же сканирование с добавлением определения версий (`-sV`).

Пример 10.5. UDP-сканирование версий узла за брандмауэром

```
# nmap -sV -sU -p50-59 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT   STATE         SERVICE    VERSION
50/udp open|filtered re-mail-ck
51/udp open|filtered la-maint
52/udp open|filtered xns-time
53/udp open          domain     ISC BIND 9.3.4
54/udp open|filtered xns-ch
55/udp open|filtered isi-gl
56/udp open|filtered xns-auth
57/udp open|filtered priv-term
58/udp open|filtered xns-mail
59/udp open|filtered priv-file

Nmap done: 1 IP address (1 host up) scanned in 56.59 seconds
```

Определение версии без всяких сомнений показывает, что порт 53 (domain) открыт, и даже сообщает, что на нём работает. Остальные порты всё ещё находятся в состоянии `open|filtered`, поскольку не ответили ни на одну пробу. Вероятно, они фильтруются, хотя это не гарантировано. На них может работать служба, такая как SNMP, которая отвечает только на пакеты с правильной строкой сообщества. Или там может работать малоизвестная или пользовательская UDP-служба, для которой не существует пробы определения версий Nmap. Обратите также внимание, что это сканирование заняло более чем в 40 раз больше времени, чем предыдущее. Отправка всех этих проб каждому порту - относительно медленный процесс. Добавление параметра `--version-intensity 0` значительно сократило бы время сканирования, поскольку отправлялись бы только пробы, которые с наибольшей вероятностью вызовут ответ от служб с данным номером порта.

10.4. Обход правил брандмауэра

Хотя построение карты правил брандмауэра может быть ценным, главной целью часто является обход правил. Nmap реализует множество методов для этого, хотя большинство из них эффективно только против плохо настроенных сетей. К сожалению, такие сети встречаются часто. Вероятность успеха каждого отдельного метода невелика, поэтому пробуйте как можно больше разных способов. Атакующему достаточно найти одну ошибку конфигурации, чтобы добиться успеха, тогда как защитники сети должны закрыть каждую дыру.

10.4.1. Экзотические флаги сканирования

В предыдущем разделе обсуждалось использование ACK-сканирования для построения карты фильтруемых портов целевой сети. Однако оно не могло определить, какие из доступных портов открыты, а какие закрыты. Nmap предлагает несколько методов сканирования, хорошо подходящих для незаметного прохождения через брандмауэры и при этом предоставляющих нужные сведения о состоянии портов. FIN-сканирование - один из таких методов. В разделе 10.3.2 «ACK-сканирование» против машины с именем Para выполнялись SYN- и ACK-сканирования. SYN-сканирование показало только два открытых порта, возможно, из-за ограничений брандмауэра. Между тем ACK-сканирование не способно отличать открытые порты от закрытых. В примере 10.6 показана ещё одна попытка сканирования Para, на этот раз с помощью FIN-сканирования. Поскольку отправляется голый пакет FIN, он пролетает мимо правил, блокирующих пакеты SYN. В то время как SYN-сканирование нашло ниже порта 100 только один открытый порт, FIN-сканирование находит оба.

Пример 10.6. FIN-сканирование брандмауэра без отслеживания состояния

```
# nmap -sF -p1-100 -T4 para

Starting Nmap ( http://nmap.org )
Interesting ports on para (192.168.10.191):
Not shown: 98 filtered ports
PORT   STATE         SERVICE
22/tcp open|filtered ssh
53/tcp open|filtered domain
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 1 IP address (1 host up) scanned in 1.61 seconds
```

Стоит попробовать многие другие типы сканирования, поскольку правила целевого брандмауэра и тип целевого узла определяют, какие методы сработают. Особенно ценными могут быть сканирования FIN, Maimon, Window, SYN/FIN и NULL. Все они описаны в главе 5 «Методы и алгоритмы сканирования портов».

10.4.2. Манипулирование исходным портом

Одна удивительно распространённая ошибка конфигурации - доверять трафику только на основании номера исходного порта. Легко понять, как это возникает. Администратор устанавливает новенький блестящий брандмауэр, но тут же оказывается завален жалобами неблагодарных пользователей, чьи приложения перестали работать. В частности, может нарушиться DNS, поскольку UDP-ответы DNS от внешних серверов больше не могут войти в сеть. Другой распространённый пример - FTP. При активной передаче FTP удалённый сервер пытается установить обратное соединение с клиентом для передачи запрошенного файла.

Безопасные решения этих проблем существуют, часто в виде прокси-серверов уровня приложений или модулей брандмауэров, разбирающих протокол. К сожалению, существуют и более простые небезопасные решения. Заметив, что ответы DNS приходят с порта 53, а активный FTP - с порта 20, многие администраторы попали в ловушку, просто разрешив входящий трафик с этих портов. Они часто предполагают, что ни один атакующий не заметит и не использует такие дыры в брандмауэре. В других случаях администраторы считают это краткосрочной временной мерой, пока не смогут внедрить более безопасное решение. Затем они забывают об обновлении безопасности.

В эту ловушку попадают не только перегруженные сетевые администраторы. С такими небезопасными правилами поставлялось множество продуктов. Виновата была даже Microsoft. Фильтры IPsec, поставлявшиеся с Windows 2000 и Windows XP, содержат неявное правило, разрешающее весь трафик TCP или UDP с порта 88 (Kerberos). Поклонникам Apple тоже не стоит слишком злорадствовать, потому что брандмауэр, поставлявшийся с Mac OS X Tiger, ничем не лучше. Джей Бил обнаружил, что даже если включить флажок «Block UDP Traffic» в графическом интерфейсе брандмауэра, пакеты с порта 67 (DHCP) и 5353 (Zeroconf) проходят насквозь. Ещё один жалкий пример такой конфигурации - персональный брандмауэр Zone Alarm (версии до 2.1.25), разрешавший любые входящие UDP-пакеты с исходным портом 53 (DNS) или 67 (DHCP).

Для эксплуатации этих слабостей Nmap предлагает эквивалентные параметры `-g` и `--source-port`. Просто укажите номер порта, и Nmap будет по возможности отправлять пакеты с этого порта. Для правильной работы некоторых проверок определения ОС Nmap должен использовать другие номера портов. Большинство TCP-сканирований, включая SYN-сканирование, полностью поддерживают этот параметр, как и UDP-сканирование. В мае 2004 года Джей-Джей Грей опубликовал в Bugtraq примеры сканирований Nmap, демонстрирующие эксплуатацию ошибки исходного порта 88 в Windows IPsec против одного из его клиентов. Обычное сканирование, а затем сканирование с `-g 88` показаны в примере 10.7. Часть вывода удалена для краткости и ясности.

Пример 10.7. Обход фильтра Windows IPsec с помощью исходного порта 88

```
# nmap -sS -v -v -PN 172.25.0.14

Starting Nmap ( http://nmap.org )
Interesting ports on 172.25.0.14:
Not shown: 1658 filtered ports
PORT   STATE  SERVICE
88/tcp closed kerberos-sec

Nmap done: 1 IP address (1 host up) scanned in 7.02 seconds

# nmap -sS -v -v -PN -g 88 172.25.0.14

Starting Nmap ( http://nmap.org )
Interesting ports on 172.25.0.14:
Not shown: 1653 filtered ports
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
1025/tcp open  NFS-or-IIS
1027/tcp open  IIS
1433/tcp open  ms-sql-s

Nmap done: 1 IP address (1 host up) scanned in 0.37 seconds
```

Обратите внимание, что закрытый порт 88 стал подсказкой, заставившей Джей-Джея попробовать использовать его как исходный порт. Дополнительные сведения об этой уязвимости см. в статье 811832 базы знаний Microsoft.

10.4.3. Атаки IPv6

Хотя IPv6 не захватил мир штурмом, он довольно популярен в Японии и некоторых других регионах. Внедряя этот протокол, организации часто забывают закрыть его так же, как они инстинктивно научились делать с IPv4. Или они могут попытаться, но обнаружить, что их оборудование не поддерживает правила фильтрации IPv6. Иногда фильтрация IPv6 может быть важнее фильтрации IPv4, поскольку расширенное адресное пространство часто позволяет выделять глобально адресуемые IPv6-адреса узлам, которым обычно пришлось бы использовать частные IPv4-адреса, заданные RFC 1918.

Выполнить сканирование IPv6 вместо используемого по умолчанию IPv4 часто так же просто, как добавить в командную строку `-6`. Некоторые возможности, такие как определение ОС и UDP-сканирование, для этого протокола пока не поддерживаются, но самые популярные возможности работают. В примере 10.8 показаны сканирования IPv4 и IPv6, давно выполненные для известной организации, занимающейся разработкой и продвижением IPv6.

Пример 10.8. Сопоставление сканирований IPv4 и IPv6

```
> nmap www.kame.net

Starting Nmap ( http://nmap.org )
Interesting ports on kame220.kame.net (203.178.141.220):
Not shown: 984 closed ports
Port      State    Service
19/tcp    filtered chargen
21/tcp    open     ftp
22/tcp    open     ssh
53/tcp    open     domain
80/tcp    open     http
111/tcp   filtered sunrpc
137/tcp   filtered netbios-ns
138/tcp   filtered netbios-dgm
139/tcp   filtered netbios-ssn
513/tcp   filtered login
514/tcp   filtered shell
2049/tcp  filtered nfs
2401/tcp  open     cvspserver
5999/tcp  open     ncd-conf
7597/tcp  filtered qaz
31337/tcp filtered Elite

Nmap done: 1 IP address (1 host up) scanned in 34.47 seconds

> nmap -6 www.kame.net

Starting Nmap ( http://nmap.org )
Interesting ports on 3ffe:501:4819:2000:210:f3ff:fe03:4d0:
Not shown: 994 closed ports
Port     State Service
21/tcp   open  ftp
22/tcp   open  ssh
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  sunrpc
2401/tcp open  cvspserver

Nmap done: 1 IP address (1 host up) scanned in 19.01 seconds
```

Первое сканирование показывает множество фильтруемых портов, включая часто эксплуатируемые службы, такие как SunRPC, Windows NetBIOS и NFS. Однако сканирование того же узла по IPv6 не показывает ни одного фильтруемого порта! Внезапно SunRPC (порт 111) оказывается доступен и ждёт запроса от поддерживающего IPv6 `rpcinfo` или от системы определения версий Nmap, которая поддерживает IPv6. Они исправили проблему вскоре после того, как я уведомил их о ней.

Чтобы выполнить сканирование IPv6, система должна быть настроена для IPv6. У неё должны быть IPv6-адрес и сведения о маршрутизации. Поскольку мои интернет-провайдеры не предоставляют IPv6-адреса, я использую бесплатную службу туннельного брокера IPv6 по адресу tunnelbroker.net. Другие туннельные брокеры перечислены в Википедии[2]. Другим популярным бесплатным подходом являются туннели 6to4. Разумеется, этот метод также требует, чтобы цель использовала IPv6.

10.4.4. Холостое сканирование с IP ID

Холостое сканирование с IP ID имеет репутацию одного из самых скрытных типов сканирования, поскольку с вашего настоящего адреса цели не отправляется ни одного пакета. Открытые порты определяются по последовательностям IP ID выбранной машины-зомби. Менее известная особенность холостого сканирования состоит в том, что полученные результаты фактически совпадают с теми, которые вы получили бы, если бы машина-зомби непосредственно сканировала целевой узел. Подобно тому как параметр `-g` позволяет эксплуатировать доверенные исходные порты, холостое сканирование иногда может эксплуатировать доверенные исходные IP-адреса. Этот изобретательный тип сканирования, первоначально задуманный исследователем безопасности Antirez, полностью описан в разделе 5.10 «Холостое TCP-сканирование (`-sI`)».

10.4.5. Несколько ping-проб

Распространенная проблема при попытке сканирования через защищенные брандмауэрами сети заключается в том, что отбрасываемые ping-пробы могут привести к пропуску узлов. Чтобы уменьшить эту проблему, Nmap позволяет параллельно отправлять очень широкий спектр проб. Будем надеяться, что хотя бы одна из них пройдет. В главе 3 «Обнаружение узлов (ping-сканирование)» эти методы рассматриваются подробно, включая эмпирические данные о лучших приемах преодоления брандмауэров.

10.4.6. Фрагментация

Некоторые пакетные фильтры плохо справляются с фрагментами IP-пакетов. Они могли бы самостоятельно собирать пакеты заново, но для этого требуются дополнительные ресурсы. Кроме того, существует вероятность, что фрагменты пойдут разными маршрутами, что помешает обратной сборке. Из-за этой сложности некоторые фильтры игнорируют все фрагменты, тогда как другие автоматически пропускают все фрагменты, кроме первого. Интересные вещи могут происходить, если первый фрагмент недостаточно длинен, чтобы вместить весь заголовок TCP, или если второй пакет частично его перезаписывает. Число фильтрующих устройств, уязвимых для этих проблем, сокращается, хотя попытаться никогда не помешает.

Сканирование Nmap будет использовать крошечные IP-фрагменты, если указан параметр `-f`. По умолчанию Nmap включает в каждый фрагмент до восьми байтов данных, поэтому типичный TCP-пакет размером 20 или 24 байта (в зависимости от параметров) отправляется тремя крошечными фрагментами. Каждый экземпляр `-f` добавляет восемь к максимальному размеру данных фрагмента. Поэтому `-f -f` позволяет поместить в каждый фрагмент до 16 байтов данных. В качестве альтернативы можно указать параметр `--mtu` и передать максимальное число байтов данных в качестве аргумента. Аргумент `--mtu` должен быть кратен восьми и не может сочетаться с параметром `-f`.

Некоторые исходные системы дефрагментируют исходящие пакеты в ядре. Одним из таких примеров является Linux с модулем отслеживания соединений iptables. Выполните сканирование при работающем анализаторе пакетов, таком как Wireshark, чтобы убедиться, что отправляемые пакеты фрагментированы. Если ОС вашего узла создает проблемы, попробуйте параметр `--send-eth`, чтобы обойти уровень IP и отправлять необработанные кадры Ethernet.

Фрагментация поддерживается только для функций Nmap, работающих с необработанными пакетами, к которым относятся сканирования портов TCP и UDP (за исключением сканирования с установлением соединения и сканирования через FTP bounce) и определение ОС. Такие функции, как определение версий и движок сценариев Nmap, как правило, не поддерживают фрагментацию, поскольку для связи с целевыми службами они полагаются на стек TCP вашего узла.

IP-фрагменты, идущие не по порядку и частично перекрывающиеся, могут быть полезны для исследования и эксплуатации сетей, но для этого требуется сетевой инструмент еще более низкого уровня, чем Nmap. Nmap отправляет фрагменты по порядку и без перекрытий.

Если фрагментированное сканирование портов проходит, такой инструмент, как Fragroute[3], можно использовать для фрагментации других инструментов и эксплойтов, применяемых для атаки на узел.

10.4.7. Прокси-серверы

Прокси-серверы уровня приложений, особенно веб-прокси, стали популярными благодаря предполагаемым преимуществам в безопасности и эффективности сети (за счет кеширования). Как и брандмауэры и IDS, неправильно настроенные прокси-серверы могут создать гораздо больше проблем с безопасностью, чем решить. Самая частая проблема - неспособность установить надлежащие средства управления доступом. В Интернете существуют сотни тысяч полностью открытых прокси-серверов, позволяющих любому использовать их как анонимные промежуточные пункты для перехода на другие интернет-сайты. Десятки организаций используют автоматизированные сканеры, чтобы находить эти открытые прокси-серверы и распространять их IP-адреса. Иногда прокси-серверы используются для целей, которые можно считать положительными, например для обхода суровой цензуры, навязанной китайским правительством своим жителям. Известно, что этот «великий китайский брандмауэр» блокировал веб-сайт New York Times, а также другие новостные, политические и духовные сайты, с которыми правительство не согласно. К сожалению, гораздо чаще открытыми прокси-серверами злоупотребляют более зловещие люди, желающие анонимно взламывать сайты, совершать мошенничество с кредитными картами или наводнять Интернет спамом.

Хотя размещение полностью открытого прокси-сервера для интернет-ресурсов может вызвать многочисленные проблемы, более серьезная ситуация возникает, когда открытые прокси-серверы разрешают соединения обратно в защищенную сеть. Администраторы, решившие, что внутренние узлы должны использовать прокси-сервер для доступа к интернет-ресурсам, часто непреднамеренно разрешают трафик и в противоположном направлении. Хакер Эдриан Ламо известен взломами Microsoft, Excite, Yahoo, WorldCom, New York Times и других крупных сетей, обычно посредством эксплуатации этого приема обратного прокси-сервера.

В настоящее время Nmap не предлагает возможности сканирования через прокси-сервер, хотя она занимает высокое место в списке приоритетов. В разделе 7.9 «РЕШЕНИЕ: изменение определения версий для особых нужд, например для обнаружения открытых прокси-серверов» рассматривается способ поиска открытых прокси-серверов с помощью определения версий Nmap. Кроме того, на таких интернет-сайтах, как Packet Storm[4], доступно множество специализированных бесплатных сканеров прокси-серверов. Также широко распространены списки из тысяч открытых прокси-серверов.

10.4.8. Подмена MAC-адреса

Устройства Ethernet (включая Wi-Fi) идентифицируются уникальным шестибайтовым адресом управления доступом к среде (MAC). Первые три байта образуют организационно уникальный идентификатор (OUI). Этот префикс назначается поставщику организацией IEEE. Затем поставщик отвечает за уникальное назначение оставшихся трех байтов адаптерам и устройствам, которые он продает. Nmap включает базу данных, сопоставляющую OUI с названиями поставщиков, которым они назначены. Это помогает идентифицировать устройства при сканировании сети, хотя в этом разделе объясняется, почему ей нельзя полностью доверять. Файл базы данных OUI `nmap-mac-prefixes` описан в разделе 14.6 «Префиксы поставщиков MAC-адресов: `nmap-mac-prefixes`».

Хотя MAC-адреса заранее назначаются устройствам Ethernet, на большинстве современного оборудования их можно изменить с помощью драйвера. Но поскольку мало кто меняет свой MAC-адрес (или даже знает, что он у него есть), многие сети используют их для идентификации и авторизации. Например, большинство беспроводных точек доступа предоставляют параметр настройки для ограничения доступа определенным набором MAC-адресов. Аналогично, некоторые платные или частные сети после подключения заставляют вас пройти аутентификацию или заплатить через веб-форму. Затем они разрешают доступ к остальной сети на основании вашего MAC-адреса. Поскольку MAC-адреса обычно легко перехватывать (они должны передаваться в каждом отправляемом и принимаемом кадре), а затем подделывать этот MAC-адрес для получения несанкционированного доступа к сети, эта форма управления доступом довольно слаба. Кроме того, она эффективна только на границах сети, поскольку MAC-адрес конечного узла заменяется при прохождении через маршрутизатор.

Помимо управления доступом, MAC-адреса иногда используются для обеспечения ответственности. Сетевые администраторы записывают MAC-адреса, когда те получают аренду DHCP или когда новая машина обменивается данными в сети. Если позднее поступают жалобы на злоупотребление сетью или пиратство, они определяют MAC-адрес по IP-адресу и времени происшествия. Затем по MAC-адресу они находят ответственную машину и ее владельца. Простота подмены MAC-адресов в некоторой степени подрывает этот подход. Даже когда пользователи виновны, они могут сослаться на возможность подмены MAC-адреса, чтобы отвести от себя ответственность.

Nmap поддерживает подмену MAC-адреса с помощью параметра `--spoof-mac`. Передаваемый аргумент может иметь несколько форм. Если это просто число `0`, Nmap выбирает для сеанса полностью случайный MAC-адрес. Если переданная строка представляет собой четное число шестнадцатеричных цифр (пары необязательно разделены двоеточиями), Nmap использует их как MAC-адрес. Если указано меньше 12 шестнадцатеричных цифр, Nmap заполняет остаток шести байтов случайными значениями. Если аргумент не является нулем или шестнадцатеричной строкой, Nmap просматривает `nmap-mac-prefixes` в поисках названия поставщика, содержащего переданную строку (без учета регистра). Если совпадение найдено, Nmap использует OUI поставщика и случайным образом заполняет оставшиеся три байта. Примеры допустимых аргументов `--spoof-mac`: `Apple`, `0`, `01:02:03:04:05:06`, `deadbeefcafe`, `0020F2` и `Cisco`. Этот параметр подразумевает `--send-eth`, чтобы Nmap действительно отправлял пакеты уровня Ethernet. Этот параметр влияет только на сканирования необработанными пакетами, такие как SYN-сканирование или определение ОС, а не на функции, ориентированные на соединения, такие как определение версий или движок сценариев Nmap.

Даже если подмена MAC-адреса не нужна для доступа к сети, ее можно использовать для введения в заблуждение. Если на конференции я запущу сканирование со своего Thinkpad с `--spoof-mac`

Apple, подозрительные взгляды могут обратиться к пользователям MacBook в зале.

10.4.9. Маршрутизация от источника

Этот прием старой школы все еще эффективен в некоторых случаях. Если определенный маршрутизатор на пути создает вам проблемы, попробуйте найти маршрут в обход него. Эффективность этого приема ограничена, поскольку проблемы фильтрации пакетов обычно возникают в целевой сети или рядом с ней. Эти машины, скорее всего, либо отбрасывают все пакеты с маршрутизацией от источника, либо являются единственным путем в сеть. Nmap поддерживает как свободную, так и строгую маршрутизацию от источника с помощью параметра `--ip-options`. Например, указание `--ip-options "L 192.168.0.7 192.168.30.9"` требует, чтобы пакет был свободно маршрутизирован от источника через эти две заданные промежуточные IP-точки. Укажите `S` вместо `L` для строгой маршрутизации от источника. Если вы выберете строгую маршрутизацию от источника, учтите, что придется указать каждый отдельный переход на пути.

Практический пример использования маршрутизации от источника для обхода политик фильтрации в современной сети см. в разделе 10.4.12 «Практический реальный пример подрыва брандмауэра». Хотя маршрутизация от источника в IPv4 очень часто блокируется, форма маршрутизации от источника в IPv6 распространена гораздо шире. Интересная статья об этой проблеме доступна по адресу lwn.net.

Если с помощью Nmap обнаружен маршрут к целевой машине с маршрутизацией от источника, возможности эксплуатации не ограничиваются сканированием портов. Netcat Хоббита[5] - классический инструмент для обеспечения обмена данными TCP и UDP по маршрутам от источника (используйте параметр `-g`).

10.4.10. Сканирование через FTP bounce

Хотя уязвима по-прежнему лишь небольшая доля FTP-серверов, стоит проверить на эту проблему системы всех ваших клиентов. Как минимум, она позволяет внешним атакующим использовать уязвимые системы для сканирования других сторон. Еще худшие конфигурации позволяют атакующим даже обходить брандмауэры организации. Подробности и примеры этого приема приведены в разделе 5.12 «Сканирование TCP через FTP bounce (`-b`)». В примере 10.9 показано, как принтер HP используется для ретрансляции сканирования портов. Если этот принтер находится за брандмауэром организации, его можно также использовать для сканирования внутренних адресов, обычно недоступных атакующему.

Пример 10.9. Эксплуатация принтера с помощью сканирования через FTP bounce

```
felix~> nmap -p 22,25,135 -PN -v -b XXX.YY.111.2 scanme.nmap.org

Starting Nmap ( http://nmap.org )
Attempting connection to ftp://anonymous:-wwwuser@@XXX.YY.111.2:21
Connected:220 JD FTP Server Ready
Login credentials accepted by ftp server!
Initiating TCP ftp bounce scan against scanme.nmap.org (64.13.134.52)
Adding open port 22/tcp
Adding open port 25/tcp
Scanned 3 ports in 12 seconds via the Bounce scan.
Interesting ports on scanme.nmap.org (64.13.134.52):
PORT    STATE    SERVICE
22/tcp  open     ssh
25/tcp  open     smtp
135/tcp filtered msrpc

Nmap done: 1 IP address (1 host up) scanned in 21.79 seconds
```

10.4.11. Выберите альтернативный путь

Не хочется злоупотреблять клише «мыслить нестандартно», но непрерывно ломиться в парадную дверь хорошо защищенной сети - не всегда лучший подход. Ищите другие пути внутрь. Прозвоните их телефонные линии, атакуйте дочерние компании, у которых может быть особый доступ к сети, или появитесь в их офисах с оборудованием для перехвата Wi-Fi, а то и проберитесь внутрь и подключитесь к удобно расположенному разъему Ethernet. Nmap хорошо работает через все эти соединения. Только убедитесь, что ваш договор на тестирование на проникновение охватывает эти методы, прежде чем клиент поймает вас в костюме ниндзя, цепляющегося крюком за крышу его центра обработки данных.

10.4.12. Практический реальный пример подрыва брандмауэра

Теперь, когда рассмотрено множество отдельных приемов обхода правил брандмауэра, настало время объединить их в реальном сценарии тестирования на проникновение. Все началось с сообщения[6] в списке рассылки SecurityFocus pen-test от специалиста по безопасности Майкла Кейна. Он и его коллега Деметрис Папапетру проводили тестирование на проникновение во внутреннюю сеть крупной корпорации и только что обошли правила брандмауэра, предназначенные для предотвращения доступа из одной VLAN в другую. Мне было приятно прочитать, что они совершили этот подвиг с помощью Nmap, и я написал им, попросив рассказать всю историю. Она одновременно поучительна и вдохновляюща, поскольку демонстрирует ценность настойчивости и испытания каждого известного вам приема даже после неудачи самых распространенных эксплойтов. Не позволяйте этому брандмауэру победить вас!

История начинается с того, что Майкл и Деметрис выполняют сканирование Nmap, которое показывает, что они застряли в сети с жесткой фильтрацией. Они могут связаться с некоторыми корпоративными серверами, но не с какими-либо (потенциально уязвимыми) настольными клиентскими машинами, которые должны где-то существовать в сети. Возможно, они находятся в ограниченной сети конференц-зала или вестибюля, а может быть, в беспроводной точке доступа, настроенной для гостей компании. Некоторые обнаруженные узлы и сети показаны в примере 10.10. Несколько подробностей этой истории (например, IP-адреса) изменены из соображений конфиденциальности. Целевую корпорацию я буду называть Megacorp.

Пример 10.10. Некоторые интересные узлы и сети в Megacorp

```
10.10.5.1     - A router/firewall which will give us grief later
10.10.5.42    - Our protagonists are scanning from this machine
10.10.6.30      files2.megacorp.com; Nmap shows this is a Windows machine
                with port 445 open.
10.10.6.60      mail.megacorp.com; Nmap OS detection shows that it is
                Solaris 8. Port 25 is open and accessible.
10.10.10.0/24 - Nothing shows up here, but many of the IPs have
                reverse-DNS names, so Demetris suspects that a
                firewall may be blocking his probes. The goal is to
                reach any available hosts on this subnet.
```

Получив задачу определить, не скрываются ли какие-либо узлы в сети 10.10.10.0/24, Деметрис начинает с простого ping-сканирования с использованием запросов ICMP echo request (`-PE`). Результаты показаны в примере 10.11.

Пример 10.11. Ping-сканирование целевой сети

```
# nmap -n -sP -PE -T4 10.10.10.0/24
Starting Nmap ( http://nmap.org )
```

```
Nmap done: 256 IP addresses (0 hosts up) scanned in 26.167 seconds
```

Ping-сканирование не находит ни одного отвечающего узла. Деметрис по понятным причинам разочарован, но по крайней мере это делает данный раздел интереснее и поучительнее. Возможно, сеть действительно пуста, но она также может быть набита уязвимыми машинами, доступ к которым Деметрису заблокирован. Ему нужно копнуть глубже. В примере 10.12 Деметрис выбирает один IP-адрес в этой сети и выполняет ping-сканирование. Он указывает параметры трассировки пакетов (`--packet-trace`) и повышенной подробности (`-vv`), чтобы определить, что происходит на уровне пакетов. Причина выбора всего одного IP-адреса - избежать сбивающего с толку потока из сотен пакетов.

Пример 10.12. Трассировка пакетов для одного IP-адреса

```
# nmap -vv -n -sP -PE -T4 --packet-trace 10.10.10.7

Starting Nmap ( http://nmap.org )
SENT (0.3130s) ICMP 10.10.5.42 > 10.10.10.7 echo request (type=8/code=0)
               ttl=41 id=7193 iplen=28
RCVD (0.3130s) ICMP 10.10.5.1 > 10.10.5.42 host 10.10.10.7 unreachable
               (type=3/code=1) ttl=255 id=25980 iplen=56
Nmap done: 1 IP address (0 hosts up) scanned in 0.313 seconds
```

Похоже, Деметрис получает сообщения ICMP о недоступности узла, когда пытается сканировать эти IP-адреса (или по крайней мере этот адрес). Маршрутизаторы обычно делают это, когда узел недоступен и поэтому они не могут определить MAC- адрес. Иногда причиной также бывает фильтрация. Деметрис сканирует другие узлы в сети и проверяет, что они ведут себя так же. Возможно, фильтруются только пакеты ICMP, поэтому Деметрис решает попробовать TCP SYN-сканирование. Он выполняет команду `nmap -vv -n -sS -T4 -PN --reason 10.10.10.0/24`. Все порты показаны как `filtered`, а результаты `--reason` возлагают вину на несколько сообщений о недоступности узла и несколько не отвечающих портов. Порты могут не отвечать из-за ограничения частоты сообщений о недоступности узла, отправляемых маршрутизатором. Многие маршрутизаторы отправляют только одно такое сообщение раз в несколько секунд. Деметрис может проверить, является ли причиной ограничение частоты, повторно запустив сканирование и посмотрев, приходят ли сообщения о недоступности узла для точно такого же набора портов. Если порты те же, возможно, действует специальный фильтр по портам. Если Nmap каждый раз получает сообщения о недоступности узла для разных портов, вероятной причиной является ограничение частоты.

Если проблему вызывает фильтр, это может быть простой брандмауэр без отслеживания состояния, какие обычно имеются на маршрутизаторах и коммутаторах. Как обсуждалось в предыдущих разделах, такие брандмауэры иногда беспрепятственно пропускают пакеты TCP ACK. Деметрис повторяет сканирование, но вместо `-sS` для SYN-сканирования указывает `-sA` для ACK-сканирования. Любые порты `unfiltered`, найденные сканированием, означали бы, что пакеты ACK прошли и вызвали ответ TCP RST от целевого узла. К сожалению, в этом случае все результаты оказались `filtered`, как и при SYN-сканировании.

Деметрис решает попробовать что-нибудь более сложное. Из первоначального сканирования Nmap он уже знает, что на машине Windows по адресу 10.10.6.30 (`files2.megacorp.com`) открыт порт 445. Хотя Деметрис не смог напрямую связаться с сетью 10.10.10.0/24, возможно, `files2` (будучи важным файловым сервером компании) имеет доступ к этому диапазону IP-адресов. Деметрис решает попробовать ретранслировать свои сканирования через `files2`, используя холостое сканирование с IP ID. Сначала он хочет убедиться, что `files2` работает как зомби, проверив его на адресе 10.10.6.60 - заведомо отвечающей машине с открытым портом 25. Результаты этой проверки показаны в примере 10.13.

Пример 10.13. Проверка холостого сканирования

```
# nmap -vv -n -PN -sI 10.10.6.30:445 -p 25 10.10.6.60
```

```
Starting Nmap ( http://nmap.org )
Initiating idle scan against 10.10.6.60 at 13:10
Idle scan using zombie 10.10.6.30 (10.10.6.30:445); Class: Incremental
Even though your Zombie (10.10.6.30) appears to be vulnerable to IP ID
sequence prediction (class: Incremental), our attempts have failed. This
generally means that either the Zombie uses a separate IP ID base for each
host (like Solaris), or because you cannot spoof IP packets (perhaps your ISP
has enabled egress filtering to prevent IP spoofing), or maybe the target
network recognizes the packet source as bogus and drops them
QUITTING!
```

Использование 10.10.6.30 в качестве холостого зомби не принесло успеха. Если бы проблема была вызвана интенсивным трафиком, он мог бы повторить попытку посреди ночи. Параметр `--packet-trace` в сочетании с тщательным чтением раздела 5.10 «Холостое TCP-сканирование (`-sI`)» мог бы помочь определить, почему 10.10.6.30 не работает как зомби. Деметрис пробует несколько других узлов, найденных им в сети, и ни один из них не работает как зомби.

Деметрис начинает беспокоиться, удастся ли ему когда-нибудь проникнуть в сеть 10.10.10.0/24. К счастью, он в этом деле старый мастер, и у него в рукаве есть еще один трюк - маршрутизация IP от источника. В первые дни Интернета (и даже сегодня с IPv6) маршрутизация от источника была важной и широко внедренной функцией диагностики сети. Она позволяет указать переходы, по которым пакет должен пройти к своей цели, вместо того чтобы полагаться на обычные правила маршрутизации. При строгой маршрутизации от источника необходимо указать каждый переход. Свободная маршрутизация от источника позволяет задать ключевые промежуточные IP-точки, а обычная интернет-маршрутизация заполняет подробности переходов между этими точками.

Давным-давно сетевое сообщество пришло к единому мнению, что маршрутизация от источника приносит больше проблем (особенно для безопасности), чем пользы. Многие (если не большинство) маршрутизаторы настроены отбрасывать IPv4-пакеты с маршрутизацией от источника, поэтому некоторые считают эту проблему решенной с начала 90-х годов. Однако маршрутизация от источника, подобно SYN-флуду и перехвату паролей Telnet, остается редким, но серьезным риском. Деметрис проверяет эту атаку, выполняя ping-сканирование `files2` (10.10.6.30) пакетами, свободно маршрутизируемыми от источника через почтовый сервер 10.10.6.60. Результаты показаны в примере 10.14.

Пример 10.14. Проверка маршрутизации от источника

```
# nmap -n -sP -PE --ip-options "L 10.10.6.60" --reason 10.10.6.30

Starting Nmap ( http://nmap.org )
Host 10.10.6.30 appears to be up, received echo-reply.
Nmap done: 1 IP address (1 host up) scanned in .313 seconds
```

Деметрис одновременно удивлен и восхищен тем, что проверка сработала. Он немедленно обращает внимание на свою настоящую целевую сеть, повторяя первоначальное ping-сканирование с дополнительным параметром: `--ip-options "L 10.10.6.60"`. На этот раз Nmap сообщает, что машина по адресу 10.10.10.7 отвечает. Деметрис узнает, что раньше она была недоступна потому, что подсети 10.10.10.0/24 и 10.10.5.0/24 находятся в разных VLAN маршрутизатора, настроенных так, чтобы не позволять им обмениваться данными друг с другом. Прием Деметриса с маршрутизацией от источника открыл в этой политике огромную дыру! Затем Деметрис выполняет SYN-сканирование машины 10.10.10.7, как показано в примере 10.15.

Пример 10.15. Наконец-то успех

```
# nmap -vv -n -sS -PN --ip-options "L 10.10.6.60" --reason 10.10.10.7

Starting Nmap ( http://nmap.org )
Interesting ports on 10.10.10.7:
Not shown: 988 closed ports
Reason: 988 resets
PORT     STATE    SERVICE              REASON
21/tcp   filtered ftp                  no-response
```

```
23/tcp   filtered telnet               no-response
25/tcp   open     smtp                 syn-ack
80/tcp   open     http                 syn-ack
135/tcp  open     msrpc                syn-ack
139/tcp  open     netbios-ssn          syn-ack
443/tcp  open     https                syn-ack
445/tcp  open     microsoft-ds         syn-ack
515/tcp  open     printer              syn-ack
1032/tcp open     iad3                 syn-ack
1050/tcp open     java-or-OTGfileshare syn-ack
3372/tcp open     msdtc                syn-ack
Nmap done: 1 IP address (1 host up) scanned in 21.203 seconds
```

Деметрис не включал в это первоначальное сканирование определение ОС и определение версий, но по набору открытых портов это похоже на машину Windows. Теперь Деметрис может подключаться к этим портам и обращаться к ним, если использует такие инструменты, как Netcat, предлагающие параметры маршрутизации от источника. Я не знаю, что происходит дальше в этой истории, но предполагаю, что Деметрис полностью проникает в сеть, а затем помогает компании перепроектировать ее более безопасно.

10.5. Подрыв системы обнаружения вторжений

Брандмауэры - не единственное препятствие, с которым сталкиваются современные атакующие. Системы обнаружения и предотвращения вторжений также могут создавать проблемы. Сотрудники сетевого администрирования не всегда хорошо воспринимают поток уведомлений от IDS о вторжениях в 2:00 ночи. Предупредительные хакеры прилагают усилия, чтобы их действия прежде всего не вызвали ни одного из этих уведомлений. Первый шаг - определить, присутствует ли вообще IDS: многие небольшие компании их не используют. Если наличие IDS предполагается или обнаружено, существует много эффективных приемов ее подрыва. Они делятся на три категории, различающиеся степенью вмешательства: обход IDS так, словно атакующего нет; запутывание IDS вводящими в заблуждение данными; эксплуатация IDS для получения дополнительных сетевых привилегий или просто для ее отключения. В качестве альтернативы атакующие, которых не заботит скрытность, могут полностью игнорировать IDS, пока они ожесточенно атакуют целевую сеть.

10.5.1. Обнаружение системы обнаружения вторжений

На раннем этапе бесконечной битвы между сетевыми администраторами и злонамеренными хакерами администраторы защищали свою территорию, укрепляя системы и даже устанавливая брандмауэры, служившие барьером по периметру. Хакеры разработали новые инструменты для проникновения через брандмауэры или обхода их стороной и эксплуатации уязвимых узлов. Гонка вооружений обострилась, когда администраторы внедрили системы обнаружения вторжений, постоянно следящие за коварной деятельностью. Атакующие, разумеется, ответили созданием систем для обнаружения и обмана IDS. Хотя системы обнаружения вторжений задуманы как пассивные устройства, многие из них атакующие могут обнаружить по сети.

Наименее заметна IDS, которая пассивно прослушивает сетевой трафик и никогда ничего не передает. Существуют специальные аппаратные устройства сетевого отвода, гарантирующие, что IDS не сможет передавать данные, даже если она скомпрометирована атакующими. Несмотря на преимущества такой конфигурации для безопасности, по практическим соображениям она не получила широкого распространения. Современные IDS рассчитывают на возможность отправлять уведомления центральным консолям управления и подобным системам. Если бы IDS передавала

только это, риск был бы минимален. Но чтобы предоставить более полные сведения об уведомлении, они часто инициируют пробы, которые могут увидеть атакующие.

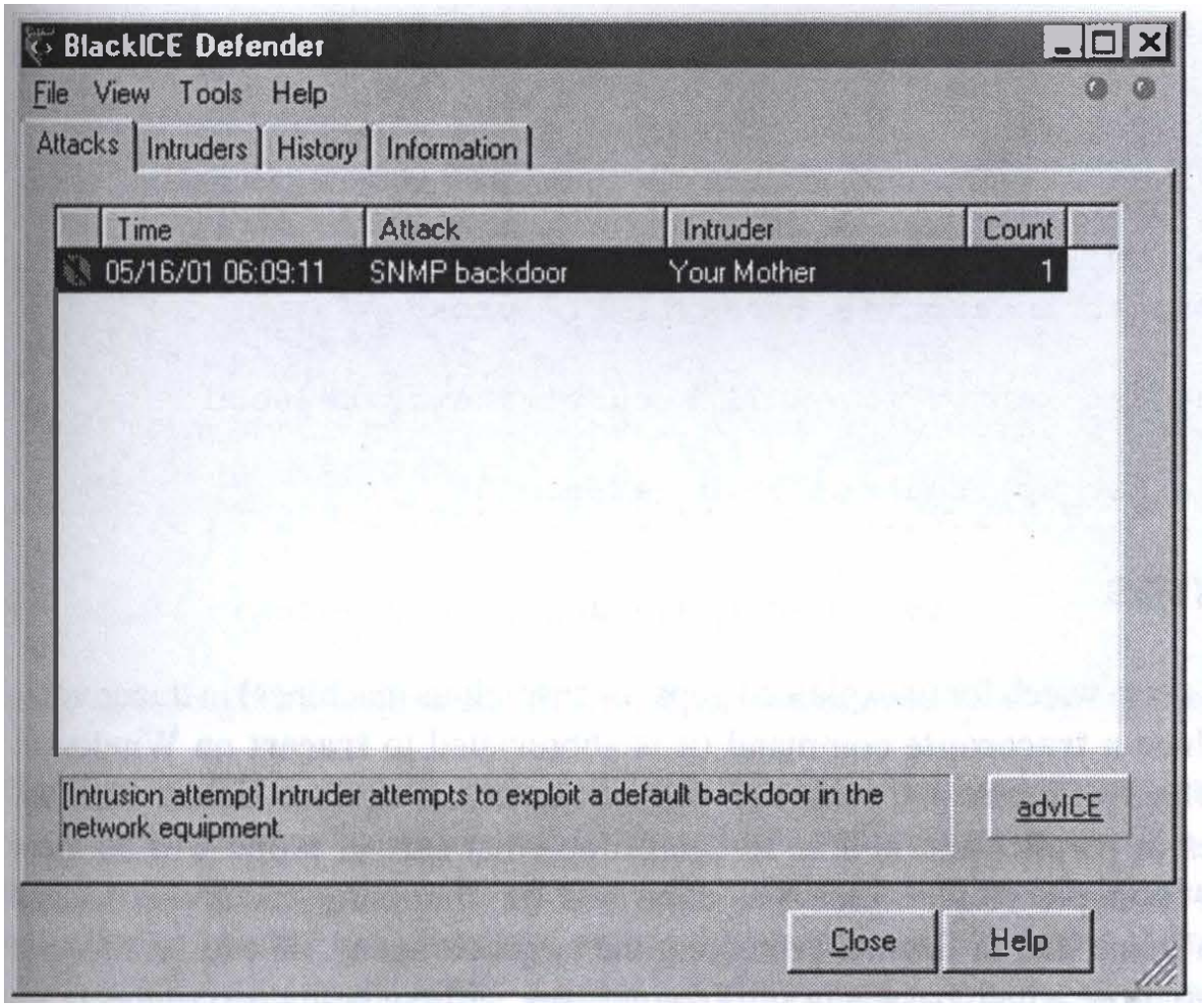
Обратные пробы

Одна из проб, часто инициируемых IDS, - обратный DNS-запрос IP-адреса атакующего. В конце концов, доменное имя в уведомлении ценнее одного лишь IP-адреса. К сожалению, атакующие, управляющие собственной обратной DNS-зоной (что встречается довольно часто), могут наблюдать за журналами в реальном времени и узнать, что их обнаружили. Это подходящий момент для того, чтобы атакующие подсунули запрашивающей IDS дезинформацию, например поддельные имена и записи кеша.

Некоторые IDS идут гораздо дальше и отправляют предполагаемым атакующим более навязчивые пробы. Когда атакующий видит, как его цель сканирует его в ответ, сомнений в срабатывании тревоги не остается. Некоторые IDS отправляют атакующему обратные запросы сведений Windows NetBIOS. ISS BlackICE Defender - один из поставщиков, чья система делает (или по крайней мере делала) это по умолчанию. Я написал небольшой инструмент под названием `icepick`, который отправляет простой пакет, вызывающий уведомление у прослушивающих экземпляров BlackICE. Затем он следит за характерными запросами NetBIOS и сообщает обо всех найденных установках BlackICE. Можно без труда просканировать крупные сети в поисках этой IDS, а затем попытаться эксплуатировать их с помощью уязвимостей, рассматриваемых далее в этой главе.

Не удовлетворившись простым обнаружением установок BlackICE или их выявлением во время тестов на проникновение, я написал простую Unix-программу `windentd`, которая отвечает на пробу дезинформацией. На рисунке 10.1 показана консоль BlackICE, где благодаря `windentd` и `icepick` нарушитель указан как «Your Mother». Эти простые инструменты доступны по адресу insecure.org, хотя они не поддерживаются.

Рисунок 10.1. BlackICE обнаруживает необычного нарушителя



Внезапные изменения брандмауэра и подозрительные пакеты

Многие системы обнаружения вторжений превратились в то, что отделы маркетинга называют системами предотвращения вторжений. Некоторые способны только перехватывать сетевой трафик, как обычная IDS, и отправлять срабатывающие ответные пакеты. Лучшие системы IPS встроены непосредственно в сеть, поэтому могут ограничивать поток пакетов при обнаружении подозрительной активности. Например, IPS может блокировать весь дальнейший трафик с IP-адреса, который, по ее мнению, выполнил сканирование портов или попытался применить эксплойт переполнения буфера. Атакующие, вероятно, заметят это, если просканируют порты системы, а затем не смогут подключиться к обнаруженным открытым портам. Атакующие могут подтвердить, что они заблокированы, попытавшись подключиться с другого IP-адреса.

Подозрительные ответные пакеты также могут выдать, что IDS отметила действия атакующего. В частности, многие IDS, не встроенные непосредственно в сеть, подделывают пакеты RST, пытаясь разорвать соединения. Способы определить, что эти пакеты поддельные, рассматриваются в разделе 10.6 «Обнаружение подделки пакетов брандмауэром и системами обнаружения вторжений».

Соглашения об именовании

Соглашения об именовании могут быть еще одним признаком присутствия IDS. Если сканирование списка Nmap возвращает имена узлов вроде `realsecure`, `ids-monitor` или `dragon-ids`, возможно, вы нашли систему обнаружения вторжений. Администраторы могли непреднамеренно выдать эту информацию или могут относиться к ней как к наклейкам о сигнализации на окнах домов и автомобилей. Возможно, они думают, что имена, связанные с IDS, отпугнут скрипт-кидди. Это также может быть дезинформацией. Никогда нельзя полностью доверять именам DNS. Например, можно предположить, что `bugzilla.securityfocus.com` - веб-сервер, на котором работает популярная веб-система отслеживания ошибок Bugzilla.

Это не так. Сканирование Nmap в примере 10.16 показывает, что вместо этого перед нами, вероятно, брандмауэр Symantec Raptor. Ни один веб-сервер недоступен, хотя он может скрываться за Raptor.

Пример 10.16. Имена узлов могут вводить в заблуждение

```
# nmap -sS -sV -T4 -p1-24 bugzilla.securityfocus.com

Starting Nmap ( http://nmap.org )
Interesting ports on 205.206.231.82:
Not shown: 21 closed ports
PORT   STATE SERVICE   VERSION
21/tcp open  ftp-proxy Symantec Enterprise Firewall FTP proxy
22/tcp open  ssh?
23/tcp open  telnet    Symantec Raptor firewall secure gateway telnetd

Nmap done: 1 IP address (1 host up) scanned in 0.94 seconds
```

Необъяснимые скачки TTL

Еще один способ обнаружить некоторые IDS - следить за необъяснимыми разрывами (или подозрительными машинами) в трассировках маршрута. Хотя большинство операционных систем включают команду `traceroute` (в Windows она сокращена до `tracert`), Nmap предлагает более быструю и эффективную альтернативу в виде параметра `--traceroute`. В отличие от стандартной `traceroute`, Nmap отправляет пробы параллельно и способен по результатам сканирования определить, какой вид пробы будет наиболее эффективен. В примере 10.17, искусственно упрощенном для ясности, `traceroute` ничего не обнаруживает на пятом переходе. Там может находиться встроенная в сеть IDS или брандмауэр, защищающий целевую компанию. Разумеется, так можно обнаружить только встроенные в сеть IDS, в отличие от тех, которые пассивно перехватывают сетевой трафик, не являясь частью маршрута. Даже некоторые встроенные устройства могут остаться незамеченными, поскольку они не уменьшают TTL или отказываются пропускать из защищенной сети обратные сообщения ICMP `ttl-exceeded`.

Пример 10.17. Обнаружение разрывов TTL с помощью traceroute

```
# nmap --traceroute www.target.com
Interesting ports on orestes.red.target.com (10.0.0.6)
Not shown: 996 filtered ports
PORT    STATE  SERVICE
22/tcp  open   ssh
53/tcp  open   domain
80/tcp  open   http
113/tcp closed auth

TRACEROUTE (using port 22/tcp)
HOP RTT   ADDRESS
1   1.10  gw (205.217.153.49)
2   10.40 metro1-ge-152.pa.meer.net (205.217.152.1)
3   12.02 208.185.168.171 (208.185.168.171)
4   14.74 p4-2-0-0.r06.us.bb.verio.net (129.250.9.129)
5   ...
```

```
6   15.07 orestes.red.target.com (10.0.0.6)

Nmap done: 1 IP address (1 host up) scanned in 4.35 seconds
```

Хотя `traceroute` - самый известный способ получить эти сведения, он не единственный. IPv4 предлагает малоизвестный параметр под названием record route для сбора этих данных. Из-за максимального размера заголовка IP можно записать не более девяти переходов. Кроме того, некоторые узлы и маршрутизаторы отбрасывают пакеты с установленным этим параметром. Тем не менее это полезный прием для случаев, когда традиционная `traceroute` не срабатывает. В Nmap этот параметр можно задать с помощью `--ip-options R`, а прочитать его из ответа - с помощью `--packet-trace`. Обычно он используется вместе с ICMP ping-сканированием (`-sP -PE`). Большинство операционных систем предлагает параметр `-R` для своей команды `ping`, и для этой цели он проще в использовании, чем Nmap. Пример этого приема приведен в примере 10.18.

Пример 10.18. Использование параметра записи маршрута IP

```
> ping -R 151.164.184.68
PING 151.164.184.68 (151.164.184.68) 56(124) bytes of data.
64 bytes from 151.164.184.68: icmp_seq=1 ttl=126 time=11.7 ms
NOP
RR:     192.168.0.100
        69.232.194.10
        192.168.0.6
        192.168.0.100

--- 151.164.184.68 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 11.765/11.765/11.765/0.000 ms
```

10.5.2. Обход систем обнаружения вторжений

Самый тонкий способ победить системы обнаружения вторжений - полностью избежать их бдительного взгляда. В действительности правила, управляющие IDS, довольно хрупки, поскольку зачастую их можно обойти, слегка изменив атаку. У атакующих есть десятки приемов - от кодирования URL до генераторов полиморфного шелл-кода - для уклонения от обнаружения их эксплойтов системой IDS. Этот раздел посвящен скрытному сканированию портов, которое даже проще, чем скрытная эксплуатация уязвимостей.

Сбавьте скорость

Когда речь идет об обходе уведомлений IDS, терпение - добродетель. Обнаружение сканирования портов обычно основано на пороговом значении. Система следит за заданным числом проб в течение определенного периода времени. Это помогает предотвратить ложные срабатывания из-за невинных пользователей. Это также необходимо для экономии ресурсов: хранение проб соединений вечно поглощало бы память и сделало бы поиск по списку в реальном времени слишком медленным. Недостаток такого порогового подхода состоит в том, что атакующие могут уклониться от него, удерживая скорость сканирования чуть ниже порога. Для этого Nmap предлагает несколько готовых режимов времени, выбираемых параметром `-T`. Например, параметр `-T paranoid` заставляет Nmap отправлять только по одной пробе за раз, ожидая между ними пять минут. Крупное сканирование может занять недели, но по крайней мере его, вероятно, не обнаружат. Параметр `-T sneaky` действует сходным образом, но ожидает между пробами всего 15 секунд.

Вместо указания готовых режимов времени, таких как `sneaky`, переменные времени можно точно настроить с помощью таких параметров, как `--max-parallelism`, `--min-rtt-timeout` и

--scan-delay. В главе 6 «Оптимизация производительности Nmap» они описаны подробно.

Практический пример: обход стандартных правил Snort 2.2.0

Изучение удобной IDS Snort с открытым исходным кодом дает урок того, как проскользнуть незамеченным. В Snort сменилось несколько поколений средств обнаружения сканирования портов. Модуль Flow-Portscan довольно серьезен. Сканирование, ускользнувшее от него, скорее всего, избежит обнаружения и многими другими IDS.

Flow-portscan состоит из двух систем обнаружения, которые могут работать совместно (или включаться по отдельности) для выявления сканеров портов. Система и десятки ее переменных конфигурации документированы в docs/README.flow-portscan из дистрибутива Snort, но я приведу краткое резюме.

Более простой метод обнаружения в Flow-portscan известен как фиксированная шкала времени. Он просто следит за scanner-fixed-threshold пакетами-пробами в течение scanner-fixed-window секунд. Обе эти переменные, задаваемые в snort.conf, по умолчанию равны 15. Обратите внимание, что счетчик включает любые пробы, отправленные с одной машины на любой узел защищенной сети. Поэтому быстрое сканирование одного порта на каждой из 15 защищенных машин вызовет уведомление с такой же неизбежностью, как сканирование 15 портов на одной машине.

Если бы это был единственный метод обнаружения, решение было бы довольно простым. Передайте параметр --scan-delay 1075, чтобы Nmap гарантированно ожидал 1,075 секунды между отправкой проб. Интуитивным выбором могло бы показаться ожидание одной секунды между пакетами, чтобы избежать 15 пакетов за 15 секунд, но этого недостаточно. Между отправкой первого и пятнадцатого пакетов есть только 14 ожиданий, поэтому ожидание должно составлять не менее 15/14, или 1,07143 секунды. Какой-нибудь бедолага, выбравший --scan-delay 1000, резко замедлит сканирование, но тревога все равно сработает. Если проверяется несколько узлов сети, их нужно сканировать по отдельности, чтобы избежать срабатывания тревоги. Параметр --max-hostgroup 1 гарантирует сканирование только одного узла за раз, но не является полностью безопасным, поскольку не обеспечит задержку --scan-delay между последней пробой, отправленной одному узлу, и первой пробой, отправленной следующему. Если на каждом узле сканируется не менее 15 портов, это можно компенсировать, установив --scan-delay как минимум в 1155 мс, либо просто запускать экземпляры Nmap для отдельных целей из сценария оболочки, ожидая между ними 1075 мс. В примере 10.19 показано такое скрытное сканирование нескольких машин в сети. Несколько экземпляров Nmap обрабатываются с помощью синтаксиса оболочки Bash. Здесь IP-адреса указаны вручную. Если требуется много целей, их можно перечислить в файле с помощью параметра -iL (сканирование списка), а затем запускать Nmap для каждой обычным циклом оболочки. Причина, по которой эти сканирования заняли больше 1,075 секунды на порт, заключается в том, что для фильтруемых портов потребовались повторные передачи, чтобы убедиться, что пакеты не были отброшены из-за перегрузки сети.

Пример 10.19. Медленное сканирование для обхода стандартного метода обнаружения по фиксированной шкале времени Flow-portscan в Snort 2.2.0

```
felix~# for target in 205.217.153.53 205.217.153.54 205.217.153.62; \
do nmap --scan-delay 1075 -p21,22,23,25,53 $target; \
usleep 1075000; \
done

Starting Nmap ( http://nmap.org )
Interesting ports on insecure.org (205.217.153.53):
PORT   STATE    SERVICE
21/tcp filtered ftp
```

```
22/tcp open     ssh
23/tcp filtered telnet
25/tcp open     smtp
53/tcp open     domain
Nmap done: 1 IP address (1 host up) scanned in 10.75 seconds

Starting Nmap ( http://nmap.org
Interesting ports on lists.insecure.org (205.217.153.54):
PORT   STATE    SERVICE
21/tcp filtered ftp
22/tcp open     ssh
23/tcp filtered telnet
25/tcp open     smtp
53/tcp open     domain
Nmap done: 1 IP address (1 host up) scanned in 10.78 seconds

Starting Nmap ( http://nmap.org
Interesting ports on scanme.nmap.org (205.217.153.62):
PORT   STATE    SERVICE
21/tcp filtered ftp
22/tcp open     ssh
23/tcp filtered telnet
25/tcp open     smtp
53/tcp open     domain
Nmap done: 1 IP address (1 host up) scanned in 10.80 seconds
```

К несчастью для любителей сканирования портов, победить Snort не так просто. У него есть еще один метод обнаружения, известный как скользящая шкала времени. Этот метод похож на только что рассмотренный метод фиксированного окна, за исключением того, что он увеличивает окно всякий раз, когда обнаруживается новая проба от узла. Тревога поднимается, если в течение окна обнаружено `scanner-sliding-threshold` проб. Окно начинается с `scanner-sliding-window` секунд и увеличивается для каждой обнаруженной пробы на величину уже прошедшего в окне времени, умноженную на `scanner-sliding-scale-factor`. В `snort.conf` эти три переменные по умолчанию равны 40 пробам, 20 секундам и коэффициенту 0,5.

Скользящая шкала довольно коварна тем, что непрерывно растет по мере поступления новых пакетов. Простым (хотя и медленным) решением было бы отправлять по одной пробе каждые 20,1 секунды. Это позволило бы обойти обе стандартные шкалы: фиксированную и скользящую. Сделать это можно так же, как в примере 10.19, но с большим значением. Можно ускорить процесс на порядок, очень быстро отправляя 14 пакетов, ожидая 20 секунд до истечения окна, а затем повторяя с очередными 14 пробами. Возможно, это удастся сделать с помощью сценария оболочки, управляющего Nmap, но для такой специальной задачи предпочтительнее написать собственную простую программу SYN-сканирования.

Рассеивайте пробы по сетям вместо последовательного сканирования узлов

Как обсуждалось в предыдущем разделе, IDS часто программируются поднимать тревогу только после достижения порога подозрительной активности. Этот порог часто является глобальным и применяется ко всей сети, защищенной IDS, а не только к одному узлу. Иногда они специально следят за трафиком с заданного исходного адреса к последовательным узлам. Если узел отправляет SYN-пакет на порт 139 узла 10.0.0.1, само по себе это не слишком подозрительно. Но если за этой пробой следуют аналогичные пакеты к 10.0.0.2, .3, .4 и .5, сканирование портов становится очевидным.

Один из способов избежать срабатывания этих тревог - рассеять пробы среди большого числа узлов, а не сканировать их последовательно. Иногда можно вообще избежать сканирования большого числа узлов в одной сети. Если вы проводите лишь исследовательский обзор, рассмотрите рассеивание проб по всему Интернету с помощью `-iR` вместо сканирования одной

большой сети. В любом случае результаты, вероятно, будут более репрезентативными.

В большинстве случаев требуется просканировать определенную сеть, и выборки по всему Интернету недостаточно. Избежать тревог о пробах последовательных узлов легко. Nmap предлагает параметр `--randomize-hosts`, который разбивает целевые сети на блоки по 16384 IP-адреса, а затем случайным образом перемешивает узлы в каждом блоке. При сканировании огромной сети, например класса B или крупнее, можно получить лучшие (более скрытные) результаты, перемешивая блоки большего размера. Для этого увеличьте `PING_GROUP_SZ` в `nmap.h`, а затем перекомпилируйте. Размер блока, используемого при сканировании с `--randomize-hosts`, в четыре раза превышает значение `PING_GROUP_SZ`. Обратите внимание, что большие значения `PING_GROUP_SZ` потребляют больше памяти узла. Альтернативное решение - сформировать список целевых IP-адресов с помощью сканирования списка (`-sL -n -oN <filename>`), перемешать его сценарием Perl, а затем передать весь список Nmap с помощью `-iL`. Вероятно, придется воспользоваться этим подходом, если вы сканируете такую огромную сеть, как 10.0.0.0/8, и хотите перемешать все 16 миллионов IP-адресов.

Фрагментируйте пакеты

IP-фрагменты могут стать серьезной проблемой для систем обнаружения вторжений, особенно потому, что обработка таких особенностей, как перекрывающиеся фрагменты и тайм-ауты сборки фрагментов, неоднозначна и существенно различается между платформами. Поэтому IDS часто приходится угадывать, как удаленная система истолкует пакет. Сборка фрагментов также может требовать значительных ресурсов. По этим причинам многие системы обнаружения вторжений по-прежнему не слишком хорошо поддерживают фрагментацию. Укажите `-f`, чтобы сканирование портов использовало крошечные IP-фрагменты (не более 8 байтов данных). Более важные подробности см. в разделе 10.4.6 «Фрагментация».

Обходите конкретные правила

Большинство поставщиков IDS хвастаются количеством поддерживаемых уведомлений, но многие (если не большинство) легко обойти. Самая популярная IDS среди пользователей Nmap - Snort[7] с открытым исходным кодом. В примере 10.20 показаны все стандартные правила Snort 2.0.0, в которых упоминается Nmap.

Пример 10.20. Стандартные правила Snort, упоминающие Nmap

```
felix~/src/snort-2.0.0/rules>grep -i nmap *
icmp.rules:alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"ICMP PING NMAP";
 dsize:0;itype:8;reference:arachnids,162;
 classtype:attempted-recon;sid:469;rev:1;)
scan.rules:alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"SCAN nmap XMAS";
 flags:FPU;reference:arachnids,30;classtype:attempted-recon; sid:1228;rev:1;)
scan.rules:alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"SCAN nmap TCP";
 flags:A;ack:0;reference:arachnids,28;classtype:attempted-recon;sid:628;rev:1;)
scan.rules:alert tcp $EXTERNAL_NET any ->
 $HOME_NET any (msg:"SCAN nmap fingerprint attempt";
 flags:SFPU;reference:arachnids,05;classtype:attempted-recon; sid:629; rev:1;)
web-attacks.rules:alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS
 (msg:"WEB-ATTACKS nmap command attempt";
 flow:to_server,established;content:"nmap%20";
 nocase;sid:1361;classtype:web-application-attack; rev:4;)
```

Теперь посмотрим на эти правила глазами атакующего. Первое правило ищет ping-пакет ICMP без полезной нагрузки (`dsize:0`). Простое указание ненулевого параметра `--data-length` победит это правило. Либо пользователь может выбрать совершенно иной тип ping-сканирования, например

TCP SYN ping.

Следующее правило ищет TCP-пакеты с установленными флагами FIN, PSH и URG (`flags:FPU`) и выдает уведомление о Xmas-сканировании Nmap. Добавление параметра `--scanflags FINPSH` к параметрам Xmas-сканирования удалит флаг URG. Сканирование по-прежнему будет работать как ожидается, но правило не сработает.

Третье правило в списке ищет TCP-пакеты с установленным битом ACK, но с нулевым номером подтверждения (`flags:A;ack:0`). Древние версии Nmap вели себя так, однако в 1999 году это было исправлено в ответ на правило Snort.

Правило номер четыре ищет TCP-пакеты с установленными флагами SYN, FIN, PSH и URG (`flags:SFPU`). Затем оно объявляет о попытке определения отпечатка ОС с помощью Nmap. Атакующий может избежать срабатывания, не указывая флаг `-O`. Если он действительно желает выполнить определение ОС, этот единственный тест можно закомментировать в `osscan2.cc`. Определение ОС по-прежнему будет довольно точным, но уведомление IDS не сработает.

Последнее правило ищет людей, отправляющих веб-серверам строку `nmap`. Оно предназначено для выявления попыток выполнять команды через веб-сервер. Атакующий может победить его, переименовав Nmap, используя символ табуляции вместо пробела или подключившись с шифрованием SSL, если оно доступно.

Разумеется, существуют другие относящиеся к делу правила, в названии которых нет Nmap, но которые все равно могут сработать при навязчивом сканировании портов. Опытные атакующие устанавливают интересующую их IDS в собственной сети, а затем заранее изменяют и проверяют сканирования, чтобы убедиться, что они не вызывают тревог.

Snort был выбран для этого примера лишь потому, что его база правил общедоступна и это родственный сетевой инструмент безопасности с открытым исходным кодом. Коммерческие IDS страдают от подобных проблем.

Избегайте легко обнаруживаемых функций Nmap

Некоторые функции Nmap заметнее других. В частности, определение версий подключается ко множеству различных служб, что часто оставляет журнальные записи на этих машинах и вызывает тревоги в системах обнаружения вторжений. Определение ОС также легко заметить системам обнаружения вторжений, поскольку в нескольких тестах используются довольно необычные пакеты и последовательности пакетов. Правила Snort, показанные в примере 10.20 «Стандартные правила Snort, упоминающие Nmap», демонстрируют типичную сигнатуру определения ОС Nmap.

Одно из решений для специалистов по тестированию на проникновение, желающих сохранить скрытность, - полностью пропустить эти заметные пробы. Определение служб и ОС ценно, но не обязательно для успешной атаки. Их также можно применять отдельно к машинам или портам, которые выглядят интересными, вместо того чтобы проверять ими всю целевую сеть.

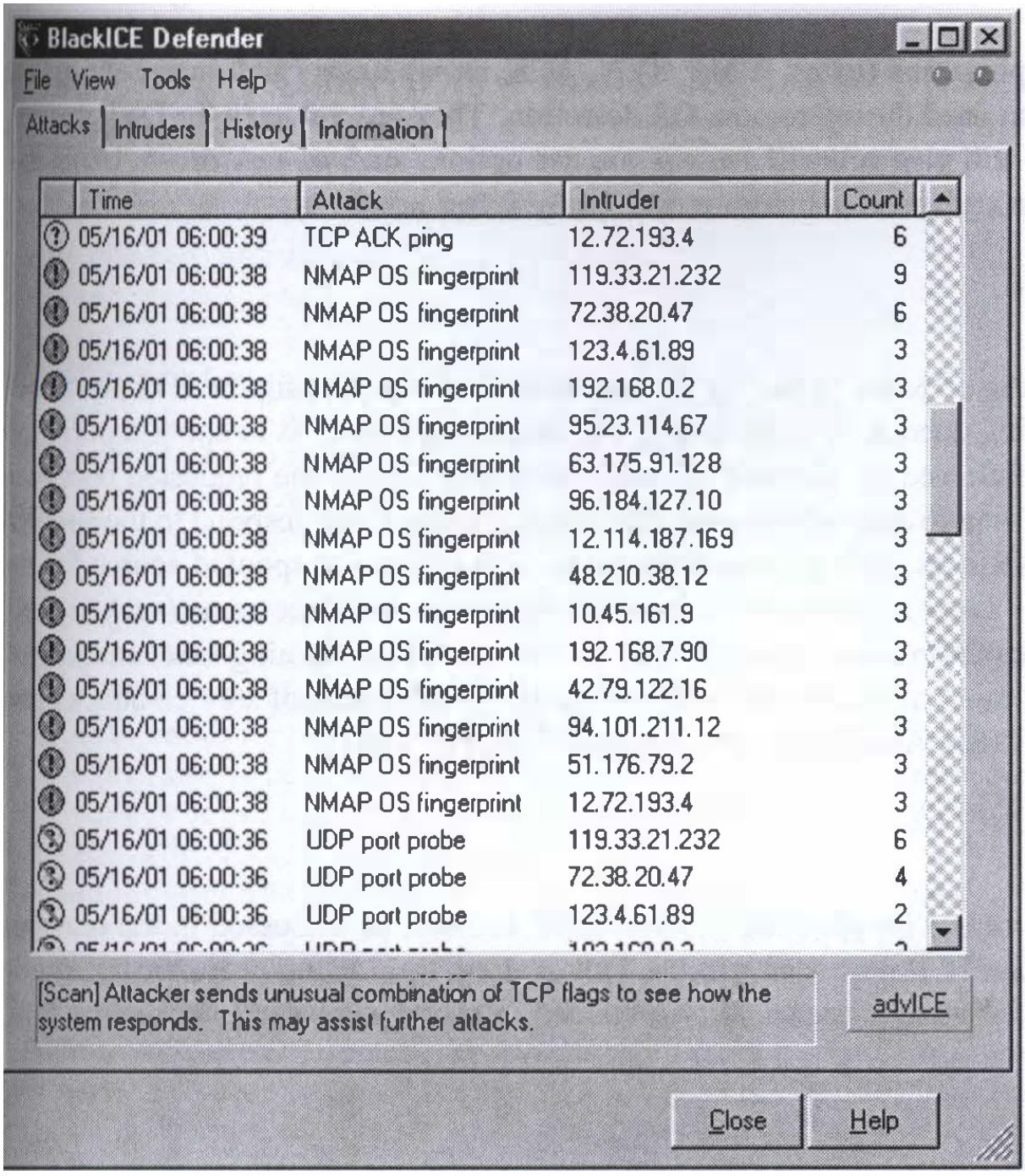
10.5.3. Введение систем обнаружения вторжений в заблуждение

В предыдущем разделе обсуждалось использование тонких приемов для уклонения от бдительного взгляда систем обнаружения вторжений. Альтернативный подход - активно вводить IDS в заблуждение или запутывать ее с помощью подделки пакетов. Nmap предлагает множество параметров для этого.

Ложные цели

Уличные преступники знают, что один из эффективных способов скрыться от властей после преступления - смешаться с любой находящейся поблизости толпой. Полиция может оказаться не способна отличить похитителя сумочки от всех невинных прохожих. В сетевой сфере Nmap может сформировать сканирование, которое выглядит исходящим от десятков узлов по всему миру. Цели будет трудно определить, какой узел представляет атакующих, а какие являются невинными ложными целями. Хотя этот прием можно победить трассировкой пути маршрутизатора, отбрасыванием ответов и другими активными механизмами, обычно он эффективно скрывает источник сканирования. На рисунке 10.2 показан экран отчета BlackICE, переполненный ложными целями. Администратор не может пожаловаться поставщикам каждого интернет-провайдера в списке. Это заняло бы много времени, а все узлы, кроме одного, невиновны.

Рисунок 10.2. Атакующий, замаскированный десятками ложных целей



Предупреждение

Многие розничные интернет-провайдеры (коммутируемый доступ, кабельный модем, DSL и так далее) отфильтровывают большинство поддельных пакетов, хотя поддельные пакеты из того же сетевого диапазона, что и ваш, могут пройти. Сначала проведите несколько проверок на какой-нибудь машине под вашим управлением через Интернет; можно даже проверить это на сторонних серверах с помощью приемов с IP ID, подобных рассмотренным в разделе 10.3.3 «Приемы с IP ID».

Ложные цели добавляются параметром `-D`. Его аргумент представляет собой список узлов, разделенных запятыми. Строку `ME` можно использовать как одну из ложных целей, чтобы обозначить, в каком месте порядка сканирования должен появиться настоящий исходный узел. В противном случае его позиция будет случайной. Размещение `ME` на шестой или более поздней позиции в списке не позволяет некоторым распространенным средствам обнаружения сканирования портов сообщить об этой активности. Например, превосходный Scanlogd Соляра Дизайнера сообщает только о первых пяти источниках сканирования, чтобы ложные цели не переполняли его журналы.

Можно также использовать `RND`, чтобы запросить случайный незарезервированный IP-адрес, либо `RND:<number>`, чтобы создать `<number>` случайных адресов.

Обратите внимание, что узлы, используемые как ложные цели, должны быть включены и работать. Было бы довольно легко определить, какой узел выполняет сканирование, если в сети действительно работает только один. Использование слишком большого числа неработающих ложных целей также может временно сделать целевые порты не отвечающими из-за состояния, известного как SYN-флуд. Рекомендуется использовать IP-адреса вместо имен, чтобы не появляться в журналах серверов имен сетей ложных целей. Сами цели в идеале также следует задавать IP-адресами.

Ложные цели используются как при первоначальном ping-сканировании (с применением ICMP, SYN, ACK или чего-либо еще), так и на этапе сканирования портов. Они также используются при удаленном определении ОС. Они не используются для запросов DNS или определения служб и версий, поэтому вы выдадите себя, если примените такие параметры, как `-sV` или `-A`. Использование большого числа ложных целей может резко замедлить сканирование, а иногда даже сделать его менее точным.

Подделка сканирования портов

Хотя огромная группа ложных целей весьма эффективно скрывает настоящий источник сканирования портов, уведомления IDS делают очевидным, что кто-то использует ложные цели. Более тонкий, но ограниченный подход - подделать сканирование портов с одного адреса. Укажите `-S`, а после него исходный IP-адрес, и Nmap запустит запрошенное сканирование портов с этого заданного источника. Полезных результатов Nmap получить не удастся, поскольку цель ответит на поддельный IP-адрес, а Nmap не увидит этих ответов. Тревоги IDS у цели возложат вину за сканирование на поддельный источник. Возможно, придется указать `-e <interfacename>`, чтобы выбрать правильное имя интерфейса (например, `eth0`, `ppp0` и так далее), через который Nmap будет отправлять поддельные пакеты. Это может быть полезно для подставы невинных сторон, посева сомнений в сознании администратора относительно точности его IDS и атак типа «отказ в обслуживании», которые будут рассмотрены в разделе 10.5.4 «DoS-атаки против реагирующих систем».

Холостое сканирование

Холостое сканирование - изобретательный прием, который позволяет подделать исходный IP-адрес, как обсуждалось в предыдущем разделе, и при этом получить точные результаты сканирования портов TCP. Это делается посредством злоупотребления свойствами поля идентификации IP в том виде, как оно реализовано во многих системах. Гораздо подробнее этот прием описан в разделе 5.10 «Холостое TCP-сканирование (`-sI`)».

Проксирование DNS

Даже самые тщательно продуманные планы может разрушить одна маленькая упущенная деталь. Если план подразумевает сверхскрытное сканирование портов, такой маленькой деталью может стать DNS. Как обсуждалось в разделе 3.4 «Разрешение DNS», по умолчанию Nmap выполняет обратное разрешение DNS для каждого отвечающего узла. Если администраторы целевой сети относятся к параноидальному типу, записывающему в журнал все подряд, или у них чрезвычайно чувствительная IDS, эти запросы поиска DNS могут быть обнаружены. Таким образом может быть обнаружено даже столь ненавязчивое действие, как сканирование списка (`-sL`). Запросы будут поступать от DNS-сервера, настроенного для машины, на которой выполняется Nmap. Обычно это отдельная машина, обслуживаемая вашим интернет-провайдером или организацией, хотя иногда это ваша собственная система.

Самый эффективный способ устранить этот риск - указать `-n`, чтобы отключить все обратное разрешение DNS. Проблема этого подхода заключается в потере ценных сведений, предоставляемых DNS. К счастью, Nmap предлагает способ получить эти сведения, скрыв источник. Значительная доля DNS-серверов в Интернете открыта для рекурсивных запросов от кого угодно. Укажите один или несколько таких серверов имен в параметре Nmap `--dns-servers`, и все обратные DNS-запросы будут проксированы через них. В примере 10.21 этот прием демонстрируется посредством сканирования списка некоторых IP-адресов SecurityFocus с использованием открытых рекурсивных DNS-серверов 4.2.2.1 и 4.2.2.2 для сокрытия следов. Имейте в виду, что прямое разрешение DNS использует настроенный DNS-сервер вашего узла, поэтому указывайте IP-адреса целей вместо доменных имен, чтобы предотвратить даже эту крошечную потенциальную утечку сведений. По этой причине в примере 10.21 сначала показано, как команда Linux `host` используется для поиска `www.securityfocus.com`, вместо указания этого имени узла в командной строке Nmap. Чтобы избежать порогов IDS, основанных на количестве запросов с одного DNS-сервера, можно указать в `--dns-servers` десятки DNS-серверов, разделенных запятыми, и Nmap будет циклически распределять запросы между ними.

Пример 10.21. Использование DNS-прокси (рекурсивного DNS) для скрытного сканирования списка SecurityFocus

```
# host www.securityfocus.com 4.2.2.1
Using domain server:
Address: 4.2.2.1#53

www.securityfocus.com has address 205.206.231.12
www.securityfocus.com has address 205.206.231.15
www.securityfocus.com has address 205.206.231.13

# nmap --dns-servers 4.2.2.1,4.2.2.2 -sL 205.206.231.12/28

Starting Nmap ( http://nmap.org )
Host 205.206.231.0 not scanned
Host mail2.securityfocus.com (205.206.231.1) not scanned
Host ns1.securityfocus.com (205.206.231.2) not scanned
Host sgs1.securityfocus.com (205.206.231.3) not scanned
Host sgs2.securityfocus.com (205.206.231.4) not scanned
```

```
Host 205.206.231.5 not scanned
Host adserver.securityfocus.com (205.206.231.6) not scanned
Host datafeeds.securityfocus.com (205.206.231.7) not scanned
Host sfcm.securityfocus.com (205.206.231.8) not scanned
Host mail.securityfocus.com (205.206.231.9) not scanned
Host www.securityfocus.com (205.206.231.10) not scanned
Host www1.securityfocus.com (205.206.231.11) not scanned
Host www2.securityfocus.com (205.206.231.12) not scanned
Host www3.securityfocus.com (205.206.231.13) not scanned
Host media.securityfocus.com (205.206.231.14) not scanned
Host www5.securityfocus.com (205.206.231.15) not scanned
Nmap done: 16 IP addresses (0 hosts up) scanned in 0.27 seconds
```

10.5.4. DoS-атаки против реагирующих систем

Многие поставщики продвигают то, что они называют системами предотвращения вторжений. По сути, это IDS, способные активно блокировать трафик и сбрасывать установленные соединения, которые сочтены вредоносными. Обычно они встроены непосредственно в сеть или работают на узлах, что дает им больше контроля над сетевой активностью. Другие системы (не встроенные в сеть) прослушивают трафик в неразборчивом режиме и пытаются бороться с подозрительными соединениями, подделывая пакеты TCP RST. Помимо традиционных поставщиков IPS, пытающихся блокировать широкий спектр подозрительной активности, многие популярные небольшие программы, такие как Port Sentry[8], предназначены специально для блокировки сканеров портов.

Хотя поначалу блокировка сканеров портов может показаться хорошей идеей, у этого подхода много проблем. Самая очевидная заключается в том, что сканирование портов обычно довольно легко подделать, как продемонстрировали предыдущие разделы. Кроме того, атакующим обычно легко определить наличие такого программного обеспечения блокировки сканирования, поскольку после сканирования портов они не смогут подключиться к якобы открытым портам. Они повторят попытку с другой системы и успешно подключатся, подтвердив, что первоначальный IP-адрес был заблокирован. Затем атакующие могут применить рассмотренные ранее приемы подделки узла (параметр `-S`), чтобы заставить целевой узел заблокировать любые желаемые атакующим системы. Среди них могут оказаться важные DNS-серверы, крупные веб-сайты, архивы обновлений программного обеспечения, почтовые серверы и тому подобное. Вероятно, не потребуется много времени, чтобы раздражение законного администратора заставило его отключить реагирующую блокировку. Хотя большинство таких продуктов предлагает белый список, предотвращающий блокировку некоторых важных узлов, перечислить их все чрезвычайно сложно. Атакующие обычно могут найти новый широко используемый узел для блокировки, раздражая пользователей до тех пор, пока администратор не определит проблему и соответствующим образом не изменит белый список.

10.5.5. Эксплуатация систем обнаружения вторжений

Самый дерзкий способ подорвать системы обнаружения вторжений - взломать их. Многие поставщики коммерческих продуктов и продуктов с открытым исходным кодом имеют жалкую историю безопасности в отношении эксплуатируемости своих продуктов. Флагманские продукты IDS RealSecure и BlackICE компании Internet Security Systems содержали уязвимость, которая позволила червю Witty скомпрометировать более десяти тысяч установок, а затем вывести IDS из строя, повредив их файловые системы. Другие поставщики IDS и брандмауэров, такие как Cisco, Checkpoint, Netgear и Symantec, также страдали от серьезных уязвимостей, допускающих удаленную эксплуатацию. Анализаторы трафика с открытым исходным кодом показали себя не

намного лучше: эксплуатируемые ошибки были найдены в Snort, Wireshark, tcpdump, FakeBO и многих других. Безопасный и эффективный разбор протоколов чрезвычайно сложен, а большинству приложений нужно разбирать сотни протоколов. Атаки типа «отказ в обслуживании», приводящие к сбою IDS (часто одним пакетом), встречаются еще чаще, чем эти уязвимости повышения привилегий. Сбойная IDS не обнаружит ни одного сканирования Nmap.

Учитывая все эти уязвимости, эксплуатация IDS может быть самым жизнеспособным путем в целевую сеть. Приятная сторона этого подхода заключается в том, что IDS даже не нужно находить. Обычно достаточно отправить вредоносный пакет любой «защищенной» машине в сети, чтобы вызвать эти ошибки IDS.

10.5.6. Игнорирование систем обнаружения вторжений

Хотя опытные атакующие часто применяют приемы подрыва IDS, описанные в этой главе, гораздо более распространенные начинающие атакующие (скрипт-кидди) редко беспокоятся об IDS. Многие компании вообще не внедряют IDS, а те, кто внедряет, часто настраивают их неправильно или почти не обращают внимания на уведомления. IDS, обращенная к Интернету, увидит столько атак скрипт-кидди и червей, что несколько сканирований Nmap для поиска уязвимой службы вряд ли вызовут тревогу.

Даже если такой атакующий скомпрометирует сеть, будет обнаружен контролируемой IDS, а затем изгнан из систем, это небольшая потеря. Взлом для них часто является игрой чисел, поэтому потеря одной скомпрометированной сети из тысяч несущественна. Такая хорошо патрулируемая сеть, скорее всего, и без того быстро заметила бы их деятельность (например, атаки типа «отказ в обслуживании», массовое сканирование или рассылку спама) и отключила бы их. Хакеры хотят компрометировать небрежно администрируемые и плохо контролируемые сети, которые предоставят долговременные узлы для преступной деятельности.

Угроза быть выслеженными и привлеченными к ответственности редко заботит тех, кто игнорирует IDS. Обычно они запускают атаки из других скомпрометированных сетей, зачастую находящихся в нескольких охватывающих весь мир переходах от их настоящего местоположения. Либо они могут использовать анонимное подключение, предоставляемое некоторыми интернет-кафе, школьными компьютерными классами, библиотеками или широко распространенными открытыми беспроводными точками доступа. Также часто используются одноразовые учетные записи коммутируемого доступа. Даже если их отключат, повторная регистрация у другого (или того же) поставщика займет всего несколько минут. Многие атакующие происходят из Румынии, Китая, Южной Кореи и других стран, где привлечение к ответственности крайне маловероятно.

Интернет-черви - еще один класс атак, который редко утруждается обходом IDS. И черви, и скрипт-кидди предпочитают бесстыдно сканировать миллионы IP-адресов, поскольку это приводит к большему числу компрометаций в час, чем осторожный целевой подход, подчеркивающий скрытность.

Хотя в большинстве атак не предпринимается никаких усилий для скрытности, тот факт, что большинство систем обнаружения вторжений так легко подорвать, вызывает серьезное беспокойство. Опытные атакующие составляют небольшое меньшинство, но часто представляют наибольшую угрозу. Не позволяйте большому числу уведомлений, извергаемых IDS, усыпить вашу бдительность. Они не могут обнаружить все и часто упускают самое важное.

Даже опытные хакеры иногда игнорируют вопросы IDS при первоначальной разведке. Они просто сканируют с какого-нибудь неотслеживаемого IP-адреса, надеясь смешаться со всеми прочими атакующими и трафиком проб в Интернете. Проанализировав результаты, они могут запустить более осторожные скрытные атаки с других систем.

10.6. Обнаружение подделки пакетов брандмауэром и системами обнаружения вторжений

В предыдущих разделах упоминалось, что некоторые брандмауэры и системы обнаружения вторжений можно настроить на подделку пакетов так, будто они пришли от одной из защищенных систем за устройством. Частый пример - пакеты TCP RST. Балансировщики нагрузки, ускорители SSL, устройства трансляции сетевых адресов и некоторые honeynet-сети также могут приводить к сбивающим с толку или противоречивым результатам. Понимание того, как Nmap интерпретирует ответы, очень помогает при воссоздании сложных топологий удаленных сетей. Когда Nmap сообщает необычные или неожиданные результаты, можно добавить параметр `--packet-trace`, чтобы увидеть необработанные пакеты, на которых Nmap основывал свои выводы. В запутанных ситуациях, возможно, придется пойти еще дальше, запуская специальные пробы и анализируя пакеты другими инструментами, такими как hping2 и Wireshark. Цель часто состоит в поиске несоответствий, помогающих понять фактическую конфигурацию сети. В следующих разделах описано несколько полезных приемов для этого. Хотя большинство этих тестов не связано с Nmap напрямую, они могут быть полезны для интерпретации неожиданных результатов Nmap.

10.6.1. Ищите согласованность TTL

Брандмауэры, балансировщики нагрузки, шлюзы NAT и подобные устройства обычно расположены на один или несколько переходов перед защищаемыми ими машинами. В таком случае можно создавать пакеты с таким TTL, чтобы они достигали сетевого устройства, но не конечного узла. Если от такой пробы получен RST, он должен быть отправлен устройством.

Во время одной неформальной оценки я через Интернет сканировал сеть крупного издателя журналов (возможно, вы помните его из раздела 4.5 «РЕШЕНИЕ: сканирование крупной сети в поисках определенного открытого TCP-порта»). Почти каждый IP-адрес показывал порт 113 как закрытый. Заподозрив подделку RST брандмауэром, я копнул немного глубже. Поскольку на этом узле были открытые, закрытые и фильтруемые порты, я решил сосредоточиться именно на нем:

```
# nmap -sS -PN -T4 mx.chi.playboy.com

Starting Nmap ( http://nmap.org )
Interesting ports on mx.chi.playboy.com (216.163.143.4):
Not shown: 998 filtered ports
PORT    STATE  SERVICE
25/tcp  open   smtp
113/tcp closed auth

Nmap done: 1 IP address (1 host up) scanned in 53.20 seconds
```

Действительно ли порт 113 закрыт, или брандмауэр подделывает пакеты RST? Я подсчитал расстояние (в сетевых переходах) до портов 25 и 113 с помощью специального режима `traceroute` бесплатной утилиты hping2, как показано в примере 10.22. Для этого можно было бы использовать более быстрый параметр Nmap `--traceroute`, но в то время такого параметра не существовало.

Пример 10.22. Обнаружение закрытых и фильтруемых портов TCP

```
# hping2 -t 5 --traceroute -p 25 -S mx.chi.playboy.com
[combined with results from hping2 -i 1 --ttl \* -p 25 -S mx.chi.playboy.com]
5->TTL 0 during transit from 64.159.2.97 (ae0-54.mp2.SanJose1.Level3.net)
6->TTL 0 during transit from 64.159.1.34 (so-3-0-0.mp2.Chicago1.Level3.net)
7->TTL 0 during transit from 200.247.10.170 (pos9-0.core1.Chicago1.level3.net)
8->TTL 0 during transit from 200.244.8.42 (gige6-0.ipcolo1.Chicago1.Level3.net)
9->TTL 0 during transit from 166.90.73.205 (ge1-0.br1.ord.playboy.net)
```

```
10->TTL 0 during transit from 216.163.228.247 (f0-0.b1.chi.playboy.com)
11->No response
12->TTL 0 during transit from 216.163.143.130 (fw.chi.playboy.com)
13->46 bytes from 216.163.143.4: flags=SA seq=0 ttl=52 id=48957 rtt=75.8 ms

# hping2 -t 5 --traceroute -p 113 -S mx.chi.playboy.com
[ results augmented again ]
5->TTL 0 during transit from 64.159.2.97 (ae0-54.mp2.SanJose1.Level3.net)
6->TTL 0 during transit from 64.159.1.34 (so-3-0-0.mp2.Chicago1.Level3.net)
7->TTL 0 during transit from 200.247.10.170 (pos9-0.core1.Chicago1.level3.net)
8->TTL 0 during transit from 200.244.8.42 (gige6-0.ipcolo1.Chicago1.Level3.net)
9->TTL 0 during transit from 166.90.73.205 (ge1-0.br1.ord.playboy.net)
10->TTL 0 during transit from 216.163.228.247 (f0-0.b1.chi.playboy.com)
11->Nothing
12->46 bytes from 216.163.143.4: flags=RA seq=0 ttl=48 id=53414 rtt=75.0 ms
```

Эта специальная трассировка показывает, что для достижения открытого порта 25 требуется 13 переходов. На расстоянии 12 переходов находится брандмауэр в Чикаго с услужливым именем `fw.chi.playboy.com`. Можно было бы ожидать, что разные порты одной машины находятся на одинаковом расстоянии в переходах. Однако порт 113 отвечает RST уже после 12 переходов. Этот RST подделывается `fw.chi.playboy.com`. Поскольку известно, что брандмауэр подделывает ответы порта 113, такие пакеты не следует считать признаком доступности узла по данному IP-адресу. Я нашел доступные узлы, снова выполнив ping-сканирование сети с распространенными типами проб, такими как запросы ICMP echo (`-PE`) и SYN-пакеты к портам 22 и 80 (`-PS22,80`), но исключив любые ping-пробы, связанные с TCP-портом 113.

10.6.2. Ищите согласованность IP ID и порядковых номеров

Каждый IP-пакет содержит 16-битное поле идентификации, используемое для дефрагментации. Его также можно эксплуатировать для получения удивительного объема сведений об удаленных узлах. Сюда входят сканирование портов с помощью приема холостого сканирования Nmap, оценка трафика, обнаружение псевдонимов узлов и многое другое. Оно также может помочь обнаружить многие сетевые устройства, например балансировщики нагрузки. Однажды я заметил странные результаты определения ОС при сканировании `beta.search.microsoft.com`. Поэтому я запустил SYN-пробы hping2 против TCP-порта 80, чтобы выяснить происходящее. Результаты показаны в примере 10.23.

Пример 10.23. Проверка согласованности последовательности номеров IP ID

```
# hping2 -c 10 -i 1 -p 80 -S beta.search.microsoft.com
HPING beta.search.microsoft.com. (eth0 207.46.197.115): S set, 40 headers
46 bytes from 207.46.197.115: flags=SA seq=0 ttl=56 id=57645 win=16616
46 bytes from 207.46.197.115: flags=SA seq=1 ttl=56 id=57650 win=16616
46 bytes from 207.46.197.115: flags=RA seq=2 ttl=56 id=18574 win=0
46 bytes from 207.46.197.115: flags=RA seq=3 ttl=56 id=18587 win=0
46 bytes from 207.46.197.115: flags=RA seq=4 ttl=56 id=18588 win=0
46 bytes from 207.46.197.115: flags=SA seq=5 ttl=56 id=57741 win=16616
46 bytes from 207.46.197.115: flags=RA seq=6 ttl=56 id=18589 win=0
46 bytes from 207.46.197.115: flags=SA seq=7 ttl=56 id=57742 win=16616
46 bytes from 207.46.197.115: flags=SA seq=8 ttl=56 id=57743 win=16616
46 bytes from 207.46.197.115: flags=SA seq=9 ttl=56 id=57744 win=16616
```

Если посмотреть на последовательность номеров IP ID (выделенных жирным в оригинале), становится ясно, что на самом деле этот IP-адрес через какой-то балансировщик нагрузки совместно используют две машины. У одной последовательности IP ID находятся в диапазоне 57 тысяч, а другая использует 18 тысяч. Учитывая это, неудивительно, что Nmap было трудно остановиться на одной догадке об операционной системе. На этих машинах могут работать совершенно разные системы.

Аналогичные тесты можно выполнять для других числовых полей, например параметра временной метки TCP или начального порядкового номера, возвращаемого открытыми портами. В данном конкретном случае видно, что размер окна TCP и флаги TCP также выдают эти узлы.

10.6.3. Прием с неверной контрольной суммой TCP

Еще один полезный прием для определения того, подделывает ли IDS или брандмауэр ответные пакеты, - отправка проб с неверной контрольной суммой TCP. Практически все конечные узлы проверяют контрольную сумму перед дальнейшей обработкой и не отвечают на такие поврежденные пакеты. Брандмауэры, напротив, часто пропускают эту проверку ради производительности. Мы можем обнаружить такое поведение с помощью параметра `--badsum`, как показано в примере 10.24.

Пример 10.24. Обнаружение брандмауэра с помощью неверных контрольных сумм TCP

```
# nmap -sS -p 113 -PN --badsum google.com

Starting Nmap ( http://nmap.org )
Warning: Hostname google.com resolves to 3 IPs. Using 64.233.187.99.
Interesting ports on jc-in-f99.google.com (64.233.187.99):
PORT    STATE  SERVICE
113/tcp closed auth

Nmap done: 1 IP address (1 host up) scanned in 0.44 seconds
```

Из примера 10.24 можно заключить, что существует какое-то сетевое устройство, возможно брандмауэр, которое обрабатывает пакеты, направленные `google.com` на порт 113, не проверяя контрольные суммы TCP. Обычно конечный узел молча отбрасывает пакеты с неверными контрольными суммами TCP, и вместо закрытого порта мы увидим фильтруемый. `--badsum` также использует неверные контрольные суммы для других протоколов поверх IP, включая UDP, ICMP и IGMP.

Этот прием, наряду с другими причинами намеренной отправки пакетов с неправильными контрольными суммами, подробнее описан в статье 12 номера 60 Phrack, написанной Ed3f[9]. Хотя иногда этот прием полезен, следует учитывать несколько оговорок:

1. Многие современные брандмауэры теперь проверяют контрольные суммы TCP (по крайней мере при принятии решения, отвечать ли на пакет), чтобы не допустить утечки этих сведений. Поэтому этот прием полезнее для доказательства того, что ответ на пробу `--badsum` был отправлен брандмауэром (или другим устройством с неполным стеком TCP), чем для доказательства того, что фильтруемая проба `--badsum` была отброшена конечным узлом.

2. Использование `--badsum` не гарантирует, что на всех платформах пакеты будут отправлены с неверными контрольными суммами. В некоторых системах ядро или сетевая плата вычисляет контрольную сумму и вставляет правильное значение, перезаписывая желаемое неверное. Один из способов убедиться, что с вами этого не происходит, - использовать удаленную машину для перехвата отправляемых пакетов. Например, при перехвате с помощью tcpdump пакеты с неверными контрольными суммами TCP будут обозначены примерно так: `[bad tcp cksum aa79 (->ab79)!]`. Другой подход - выполнить обычное SYN-сканирование одного из ваших узлов (по крайней мере с одним открытым портом). Затем выполните то же сканирование с `--badsum`. Если те же порты по-прежнему показаны как открытые, то `--badsum`, вероятно, у вас не работает. Сообщите о проблеме, как описано в разделе 15.17 «Ошибки».

10.6.4. Время прохождения туда и обратно

Когда брандмауэр подделывает ответ на пробу, этот ответ обычно возвращается немного раньше, чем вернулся бы ответ от настоящего целевого узла. В конце концов, брандмауэр обычно находится по крайней мере на один переход ближе. Он также оптимизирован для быстрого разбора и обработки пакетов и почти ничем другим не занимается. Целевой узел, напротив, может быть настолько занят выполнением приложений, что ответ на пробу занимает на несколько миллисекунд больше. Таким образом, тщательное сравнение времени прохождения туда и обратно часто может выдать махинации брандмауэра.

Сложность этого приема заключается в том, что разница во времени между ответом брандмауэра и ответом настоящей цели может составлять долю миллисекунды. Обычные колебания времени прохождения туда и обратно могут быть больше, поэтому отправки всего двух проб (одной, вызывающей ответ, о котором известно, что он исходит от целевого узла, и одного подозрительного ответа, который может исходить от брандмауэра) редко бывает достаточно. Отправка тысячи проб каждого типа сглаживает большую часть колебаний RTT, позволяя различить фундаментальные отличия. Это не обязательно займет много времени: hping2 с `-c 1000 -i u50000` отправляет тысячу проб менее чем за минуту. По этим результатам вычислите медиану, а не используйте выдаваемое среднее значение. Это не позволит огромным значениям времени (например, из-за потерянного ответа, повторно переданного через две секунды) исказить данные. Выполните тысячу проб еще один или два раза, чтобы определить согласованность результатов. Затем повторите то же с подозрительной пробой и сравните результаты. Если значения времени совершенно одинаковы до последней значащей цифры, оба ответа, вероятно, отправляет один и тот же узел. Если вы постоянно видите, что один тип проб отвечает быстрее другого, причиной может быть подделка пакетов.

Этот метод не идеален. Разница во времени может быть вызвана множеством факторов помимо брандмауэра. И все же это ценный прием, поскольку обнаружение сетевых аномалий, таких как подделка пакетов, напоминает доказывание дела в суде. Каждая крупица доказательств помогает прийти к заключению. Несоответствие может даже привести к более интересным открытиям, чем подделка брандмауэром. Возможно, некоторые порты цели перенаправляются в honeynet для более тщательного изучения атак.

10.6.5. Тщательный анализ заголовков и содержимого пакетов

Удивительно, сколько элементов могут различаться даже в небольшом заголовке TCP. В главе 8 «Удаленное определение ОС» приведены десятки тонких деталей, которые могут указывать на другую ОС. Например, разные системы отвечают с разными параметрами TCP, текстом пакетов RST, значениями типа службы и так далее. Если за балансировщиком нагрузки находится несколько систем либо пакеты отправляются брандмауэром или системами обнаружения вторжений, пакеты редко совпадают в точности.

Wireshark - превосходный инструмент для разбора заголовков пакетов, поскольку он может разложить заголовок на отдельные поля и предоставить текстовые описания двоичного содержимого пакета. Хитрость сравнения пакетов состоит в том, чтобы собрать один пакет, который, по вашему мнению, может исходить от брандмауэра, и другой пакет того же типа от целевого узла или целевой операционной системы. Два типа пакетов, которые вам, скорее всего, удастся собрать, - пакеты сброса TCP и пакеты ошибок ICMP. С помощью hping2 или параметра Nmap `--scanflags` должно быть возможно вызвать ответы с разными заголовками IP, TCP или ICMP.

10.6.6. Необычная однородность сети

Когда ответные пакеты отправляются брандмауэром, они часто более однородны, чем можно было бы ожидать от кластеров отдельных машин. При сканировании компании-издателя журналов, обсуждавшейся в предыдущем разделе о проверке TTL, я обнаружил, что сотни машин с последовательными IP-адресами отвечают RST на порт 113. В настоящем кластере машин можно ожидать, что в любой момент по крайней мере пара из них будет отключена. Кроме того, мне не удалось вызвать от большинства этих адресов ни одного ответа иного типа. Этот подозрительный результат побудил меня провести тесты TTL, показавшие, что узел `fw.chi` в действительности подделывал пакеты RST.

Брандмауэру даже не обязательно подделывать пакеты, чтобы выдать себя. Еще одна распространенная конфигурация брандмауэра - отбрасывание пакетов к определенным портам. Многие интернет-провайдеры фильтруют порты Windows 135, 139 и 445, чтобы уменьшить распространение червей. Если большое число соседних работающих узлов появляется с одинаковым набором фильтруемых портов, вероятным виновником является сетевой брандмауэр. Определив, какие порты фильтруются брандмауэром, часто можно составить карту того, сколько узлов защищено этими правилами брандмауэра, сканируя множество сетевых блоков в поисках этих фильтруемых портов. Это может привести к обнаружению случайных дыр или DMZ (демилитаризованной зоны) организации, где обычно размещаются общедоступные службы и действуют гораздо менее строгие правила брандмауэра.

Сноски

[1] [назад] hping.org
[2] [назад] en.wikipedia.org
[3] [назад] monkey.org
[4] [назад] packetstormsecurity.nl
[5] [назад] sectools.org
[6] [назад] seclists.org
[7] [назад] snort.org
[8] [назад] sourceforge.net
[9] [назад] nmap.org

Глава 11. Защита от Nmap

11.1. Введение

В главе 10 «Обнаружение и подрыв брандмауэров и систем обнаружения вторжений» рассматривалось множество способов, которыми Nmap (наряду с несколькими другими инструментами безопасности с открытым исходным кодом) можно использовать, чтобы проскользнуть через брандмауэры и перехитрить системы обнаружения вторжений. Теперь мы смотрим на ситуацию с другой стороны ограды: как такие технологии, как брандмауэры и IDS, могут защищаться от Nmap. Возможные средства защиты включают блокировку проб, ограничение возвращаемых сведений, замедление сканирования Nmap и возврат вводящих в заблуждение сведений. Также рассматриваются опасности некоторых средств защиты. Запутывание сети до такой степени, что атакующие не могут понять происходящее, не является чистым выигрышем, если ваши администраторы тоже перестают ее понимать. Аналогично, защитное программное обеспечение, предназначенное для запутывания или блокировки сканеров портов, не приносит пользы, если само открывает более серьезные уязвимости. Многие описанные здесь приемы защищают от активных проб в целом, а не только от созданных Nmap.

11.2. Сканируйте заблаговременно, затем закрывайте или блокируйте порты и устраняйте уязвимости

Часто говорят, что лучшая защита - хорошее нападение. Превосходный способ защититься от атакующих - мыслить как они. Регулярно сканируйте свои сети и тщательно анализируйте вывод в поисках уязвимостей. Используйте `crontab` в Unix или планировщик заданий в Windows вместе с такой системой, как Ndiff[1] или `nmap-report` (см. раздел 1.2.3 «Мэдхэт в Стране чудес»), чтобы получать уведомления о любых изменениях.

Заблаговременное сканирование дает возможность находить и устранять уязвимости раньше атакующих. Не менее важно закрывать и блокировать доступные без необходимости порты, чтобы предотвратить эксплуатацию уязвимостей, о которых вы еще не знаете. Заблаговременное сканирование также позволяет лучше осознавать, какие сведения могут получить атакующие. Когда вы самостоятельно проверили результаты на слабые места и уверены в своем состоянии безопасности, сканеры портов становятся гораздо менее угрожающими. Люди, наиболее подозрительно относящиеся к сканерам портов и использующие больше всего защитного программного обеспечения и средств обнаружения, часто меньше всего уверены в безопасности своей сети. Я не хочу отговаривать кого-либо от применения приемов, описанных в этой главе, а лишь предлагаю сначала найти и устранить все существующие сетевые риски и уязвимости. Устранить дыру гораздо эффективнее, чем пытаться ее скрыть. Этот подход также вызывает меньше стресса, чем постоянное беспокойство о том, что атакующие могут найти уязвимости.

После внедрения заблаговременного сканирования первым шагом является устранение всех известных уязвимостей. Затем следует проверить каждый открытый порт, доступный извне через брандмауэр или во внутренней сети. Службы, к которым публике не требуется обращаться, следует заблокировать на брандмауэре. Если сотрудникам нужен к ним доступ, возможно, вместо этого они смогут использовать VPN. Внутренние службы часто прослушивают соединения, даже когда не используются. Они могли быть установлены или включены по умолчанию либо были включены для

прошлого использования и так и не отключены. Такие ненужные службы следует отключить. Даже если вы не знаете об уязвимости в службе, атакующие могут о ней знать. Кроме того, ошибки безопасности в этой службе могут быть найдены в будущем. Закрытый порт представляет гораздо меньший риск, чем открытый. После устранения известных дыр, блокировки частных служб брандмауэром и отключения ненужных служб могут понадобиться дополнительные защитные технологии, такие как системы предотвращения вторжений, для защиты от эксплойтов нулевого дня, внутренних угроз и любых дыр, пропущенных вашей системой анализа уязвимостей.

Заблаговременное сканирование и аудит сети должны стать обычной практикой, а не разовой проверкой. В любой сети узлы и службы регулярно добавляются и изменяются. Чтобы сеть оставалась безопасной, необходимо постоянно следить за ними.

Помните, что некоторые плохо реализованные и проверенные системы могут неблагоприятно реагировать на сканирование портов, определение ОС или определение версий. При сканировании через Интернет это редко становится проблемой, поскольку машины, которые выходят из строя при сканировании, недолго живут в такой враждебной среде. Внутренние машины часто более хрупки. Перед началом программы заблаговременного сканирования убедитесь, что она одобрена и заранее доведена до сведения затрагиваемых сторон. Начните с относительно небольшой части сети и убедитесь в отсутствии проблем, а затем постепенно расширяйте охват. Можно начать с простого сканирования портов, а позже при желании перейти к определению ОС или версий.

11.3. Блокируйте и замедляйте Nmap с помощью брандмауэров

Одна из лучших защитных мер против сканирования - хорошо настроенный брандмауэр. Вместо того чтобы просто запутывать конфигурацию сети, как делают некоторые приемы, описанные далее, хорошо настроенные брандмауэры эффективно блокируют многие пути атаки.

В любой достойной книге о брандмауэрах подчеркивается это основополагающее правило: запрещать по умолчанию. Вместо попытки блокировать подозрительный вредоносный трафик сначала заблокируйте все, а затем явно переопределите это правило, чтобы разрешить необходимый трафик. Гораздо легче упустить блокировку чего-либо вредоносного, чем случайно явно разрешить то же самое. Кроме того, неспособность заблокировать плохой трафик может остаться незамеченной до тех пор, пока ее не эксплуатирует атакующий, тогда как неспособность разрешить законный трафик обычно быстро обнаруживается пострадавшими пользователями. И они будут продолжать напоминать вам, пока проблема не будет исправлена.

Двух предыдущих причин должно быть достаточно, чтобы убедить любого выбрать запрет по умолчанию, но существуют и другие преимущества. Одно из них - замедление крупномасштабной разведки такими инструментами, как Nmap. Когда TCP SYN-сканирование Nmap встречает закрытый порт, целевая машина отправляет в ответ пакет RST, и состояние порта определяется всего за одно время прохождения туда и обратно. Это меньше четверти секунды даже через весь мир - от моего веб-сервера в Калифорнии до интернет-провайдера в Москве. Если же брандмауэр фильтрует порт, отбрасывая пробу, Nmap приходится ждать тайм-аута для худшего случая, прежде чем сдаться. Затем Nmap может выполнить несколько повторных передач на случай, если пакет был отброшен каким-либо маршрутизатором из-за перегрузки, а не правилом брандмауэра. При крупномасштабных сканированиях разница может быть весьма существенной. Например, TCP SYN-сканирование 1000 портов машины в моей беспроводной сети (`nmap -sS -T4 para`) занимает всего пять секунд, когда все порты открыты или закрыты. Фильтрация примерно дюжины часто эксплуатируемых портов увеличивает время сканирования до 12 секунд. Переход к запрету по умолчанию (фильтрации всех портов, кроме пяти открытых) почти утраивает время сканирования до 33 секунд. Разница в 28 секунд может показаться незначительной, но при крупномасштабных

сканированиях она может сложиться в дополнительные дни.

Фильтруемые порты еще сильнее раздражают атакующих при использовании протокола UDP. Когда брандмауэр не задействован, практически все системы отвечают сообщением ICMP о недоступности порта, когда Nmap проверяет закрытый порт. Открытые порты обычно вообще не отвечают. Поэтому, если брандмауэр с запретом по умолчанию отбрасывает пакет-пробу, Nmap не может определить, открыт порт или фильтруется. Повторные передачи здесь не помогают, поскольку порт никогда не ответит. Тогда атакующим приходится прибегать к более медленным и гораздо более заметным приемам, таким как определение версий Nmap и перебор строк сообщества SNMP, чтобы разобраться с UDP-портами.

Чтобы действительно замедлить Nmap, убедитесь, что брандмауэр отбрасывает пакеты, а не отвечает ошибкой ICMP или TCP RST. Иначе Nmap будет работать так же быстро и точно, как если бы порты были закрыты, хотя преимущество блокировки проб сохранится. В качестве примера этого различия брандмауэр Linux iptables предлагает целевые действия `DROP` и `REJECT`. Как следует из названий, `DROP` ничего не делает, кроме блокировки пакета, тогда как `REJECT` отправляет обратно сообщение об ошибке. Первое лучше замедляет разведку и обычно рекомендуется, хотя `REJECT` может облегчить диагностику сетевых проблем, совершенно ясно показывая, что брандмауэр блокирует определенный трафик.

Еще один принцип брандмауэров - эшелонированная защита. Даже если порты заблокированы брандмауэром, все равно убедитесь, что они закрыты (ни одно приложение не прослушивает их). Исходите из того, что решительный атакующий в конце концов преодолеет брандмауэр. Даже если он пройдет с помощью приема из главы 10 «Обнаружение и подрыв брандмауэров и систем обнаружения вторжений», отдельные машины должны быть защищены, чтобы обеспечить сильную оборону. Это уменьшает масштаб и ущерб от ошибок, которые время от времени совершает каждый. Атакующим потребуется найти слабости и в брандмауэре, и в отдельных машинах. Сканер портов практически бессилен против портов, которые одновременно закрыты и фильтруются. Использование частного адресного пространства (например, с трансляцией сетевых адресов) и дополнительных брандмауэров обеспечивает еще большую защиту.

11.4. Обнаруживайте сканирования Nmap

Некоторые считают обнаружение сканирования портов пустой тратой времени. Оно настолько распространено, что любая подключенная к Интернету организация будет регулярно подвергаться сканированию. Очень немногие из этих случаев представляют целевые атаки. Многие из них - интернет-черви, бесконечно атакующие в поисках той или иной уязвимости Windows. Некоторые сканирования исходят от исследовательских интернет-проектов, другие - от любопытных или скучающих людей, исследующих Интернет. Я просканировал десятки тысяч IP-адресов в поисках хороших примеров и эмпирических данных для этой книги. Другие сканирования действительно вредоносны. Скрипт-кидди регулярно сканируют огромные диапазоны в поисках систем, уязвимых для их сегодняшнего эксплойта. Хотя у этих людей плохие намерения, они, скорее всего, уйдут сами, не найдя в вашей сети уязвимых служб. Наибольшую угрозу представляют атакующие, нацеленные именно на вашу организацию, но они составляют настолько малую долю обнаруженных сканирований, что отличить их чрезвычайно трудно. Поэтому многие администраторы даже не утруждаются записью сканирований портов.

Другие администраторы придерживаются иной точки зрения. Они утверждают, что сканирования портов часто предшествуют атакам и их следует как минимум записывать в журнал, если не реагировать на них. Они часто размещают системы обнаружения во внутренних сетях, чтобы уменьшить поток активности сканирования портов из Интернета. Журналы иногда анализируются

на предмет тенденций или передаются третьим сторонам, таким как Dshield, для глобальной корреляции и анализа. Иногда обширные журналы и пугающие графики, измеряющие атаки, представляются руководству для обоснования достаточных бюджетов.

Одних системных журналов редко достаточно для обнаружения сканирований портов. Обычно записываются только типы сканирования, устанавливающие полные TCP-соединения, тогда как стандартное SYN-сканирование Nmap проскальзывает незамеченным. Даже полные TCP-соединения записываются только тогда, когда конкретное приложение явно делает это. Такие сообщения об ошибках, если они доступны, часто загадочны. Однако множество разных служб, одновременно извергающих сообщения об ошибках, - распространенный признак сканирования. Таким способом часто можно обнаружить навязчивые сканирования, особенно использующие определение версий Nmap. Но только если администраторы действительно регулярно читают системные журналы. Подавляющее большинство сообщений журналов навсегда остается непрочитанным. Инструменты наблюдения за журналами, такие как Logwatch[2] и Swatch[3], безусловно, могут помочь, но в действительности системные журналы лишь незначительно эффективны для обнаружения активности Nmap.

Специализированные средства обнаружения сканирования портов - более эффективный подход к выявлению активности Nmap. Два распространенных примера - PortSentry[4] и Scanlogd[5]. Scanlogd существует с 1998 года и был тщательно спроектирован с учетом безопасности. За все время его существования не было зарегистрировано ни одной уязвимости. PortSentry предлагает сходные возможности, а также реагирующую функцию, блокирующую исходный IP-адрес предполагаемых сканеров. Обратите внимание, что этот реагирующий прием может быть опасен, как показано в разделе 11.5.6 «Реагирующее обнаружение сканирования портов».

Несмотря на подверженность атакам на основе пороговых значений, обсуждавшимся в разделе 10.5.2 «Обход систем обнаружения вторжений», эти средства обнаружения сканирования портов работают довольно хорошо. Однако администратор, которому достаточно важно следить за сканированиями портов, захочет также знать о более серьезных атаках, таких как попытки эксплуатации и установленные бэкдоры. По этой причине системы обнаружения вторжений, уведомляющие о широком спектре подозрительного поведения, популярнее таких специализированных инструментов.

Сейчас системы обнаружения вторжений продают многие поставщики, но пользователи Nmap тяготеют к легковесной IDS с открытым исходным кодом под названием Snort. Она заняла третье место среди самых популярных инструментов безопасности в опросе 3243 пользователей Nmap (sectools.org). Подобно Nmap, Snort совершенствуется глобальным сообществом разработчиков. Она поддерживает более двух тысяч правил для обнаружения всевозможной подозрительной активности, включая сканирования портов.

Правильно установленная и контролируемая IDS может быть огромным преимуществом для безопасности, но не забывайте о рисках, рассмотренных в разделе 10.5 «Подрыв систем обнаружения вторжений». В Snort было несколько допускающих удаленную эксплуатацию уязвимостей, как и во многих ее коммерческих конкурентах. Кроме того, опытный атакующий может победить большинство правил IDS, поэтому не теряйте бдительности. IDS слишком часто создают ложное чувство безопасности.

11.5. Хитрые уловки

Nmap, подобно другим инструментам активного зондирования, получает сведения, отправляя пакеты целевым системам, а затем пытаясь истолковать и организовать ответы в полезные отчеты. Nmap вынужден полагаться на сведения от систем и сетей, которые могут быть откровенно

враждебной средой. Некоторые администраторы оскорбляются самим фактом сканирования, а небольшой процент пытается запутать или замедлить Nmap активными мерами сверх рассмотренных ранее приемов с брандмауэрами и IDS.

Многие из этих методов активного ответа весьма изобретательны. Я бы сказал, что многие из них слишком изобретательны и создают больше проблем, чем решают. Одна из таких проблем - возможность эксплуатации. Значительная часть этого специального программного обеспечения активного ответа - всего лишь быстрая поделка, написанная без тщательного учета безопасности. Например, мой знакомый администратор по имени Пол очень гордился установкой FakeBO на своей машине. Он смеялся над перспективой обмануть скрипт-кидди, заставив их думать, что они нашли зараженную Back Orifice машину, которой можно завладеть, тогда как на самом деле Пол лишь записывал их попытки в журнал. Шутка обернулась против Пола, когда было обнаружено переполнение буфера в FakeBO, а атакующий использовал его для компрометации машины и установки настоящего бэкдора.

Другой крупный риск, общий для этих технологий, - отвлечение времени, которое лучше потратить на другое. Запутывать атакующих может быть весело и приятно, а в некоторых случаях это даже мешает атакам. Однако в конечном счете эти приемы по большей части являются безопасностью через неясность. Хотя они все же могут приносить пользу, они не так важны, как более устойчивые технологии, например брандмауэры и исправление уязвимостей. Опытные атакующие, скорее всего, все равно разглядят обман, а скрипт-кидди и черви редко утруждаются разведкой. Ежедневные попытки эксплуатации IIS против моего веб-сервера Apache служат тому подтверждением. Эти приемы следует рассматривать только тогда, когда вы уже полностью уверены в состоянии своей безопасности. Слишком многие используют их вместо настоящей защиты своих сетей.

11.5.1. Сокрытие служб на малоизвестных портах

Иногда администраторы выступают за запуск служб на необычных портах, чтобы атакующим было труднее их найти. В частности, они отмечают частоту сканирования одного порта по всему их адресному пространству атакующими, ищущими уязвимую версию какого-либо программного обеспечения. Автономные черви часто делают то же самое.

Действительно, такое запутывание может помешать некоторым червям и скрипт-кидди найти службы, но для компаний, быстро исправляющих уязвимости, они редко представляют больше чем незначительную угрозу. А компании, которые не устанавливают исправления быстро, не спасет это простое сокрытие портов. Сторонники часто утверждают, что на это попадутся даже более умелые атакующие. Некоторые даже писали в списки рассылки по безопасности, что сканирование всех 65 536 TCP-портов немыслимо. Они ошибаются. Атакующие могут и действительно сканируют все TCP-порты. Кроме того, такие приемы, как определение версий Nmap, позволяют легко определить, какая служба прослушивает необычный порт. В примере 11.1 показано такое сканирование. Примечательно, что оно занимает всего восемь минут, причем выполняется по медленной домашней линии aDSL из другого штата. С более быстрой машины такое же сканирование занимает всего три минуты. Если бы состояние по умолчанию было фильтруемым, сканирование шло бы медленнее, но не настолько, чтобы это было неразумно. Даже если сканирование занимает 10 или 20 минут, атакующему не нужно сидеть и наблюдать. Целевую атаку на компанию легко оставить на ночь, а массовые атакующие могут держать сканер запущенным неделями, периодически загружая свежие файлы данных.

Пример 11.1. Сканирование версий на всех TCP-портах

```
# nmap -sSV -T4 -O -p0-65535 apollo.sco.com
Starting Nmap ( http://nmap.org )
```

```
Interesting ports on apollo.sco.com (216.250.128.35):
Not shown: 65524 closed ports
PORT      STATE    SERVICE VERSION
0/tcp     filtered unknown
21/tcp    open     ftp     WU-FTPD 2.1WU(1)+SCO-2.6.1+-sec
22/tcp    open     ssh     SSH 1.2.22 (protocol 1.5)
199/tcp   open     smux?
457/tcp   open     http    NCSA httpd 1.3
615/tcp   open     http    NCSA httpd 1.5
1035/tcp  filtered unknown
1521/tcp  open     oracle  Oracle DB Listener 2.3.4.0.0 (for SCO System V/386)
13722/tcp open     inetd   inetd exec err /usr/openv/netbackup/bin/bpjava-msvc
13782/tcp open     inetd   inetd exec err /usr/openv/netbackup/bin/bpcd
13783/tcp open     inetd   inetd exec err /usr/openv/bin/vopied
64206/tcp open     unknown
Device type: general purpose
Running: SCO UnixWare
OS details: SCO UnixWare 7.0.0 or OpenServer 5.0.4-5.0.6

Nmap done: 1 IP address (1 host up) scanned in 501.90 seconds
```

Самый большой недостаток этого подхода - серьезные неудобства для законных пользователей. Некоторые службы, такие как SMTP и DNS, по практическим причинам почти всегда должны работать на своих общеизвестных портах. Даже для служб вроде HTTP и SSH, порты которых изменить проще, это означает, что все пользователи должны помнить необычный номер порта, например 52 147, при каждом подключении к службе. Когда «скрытых» служб несколько, особенно трудно запомнить, какая из них где находится. Использование разных портов на каждой машине запутывает еще сильнее, но стандартизация необычного сопоставления портов во всей организации уменьшает предполагаемую пользу этой схемы. Атакующие могут заметить, что SSH всегда находится на порту 52 147. Конечным результатом может стать рост числа сканирований всех портов ваших серверов с помощью Nmap, поскольку разочарованные законные пользователи пытаются найти, где скрыты необходимые службы. Менее опытные пользователи вместо этого могут завалить вас телефонными звонками.

11.5.2. Простукивание портов

Прием под названием «простукивание портов» в последнее время стал популярным способом скрывать службы от потенциальных атакующих. Этот метод хорошо описан на главной странице portknocking.org:

> Простукивание портов - метод установления соединения с сетевым компьютером, у которого нет открытых портов. Перед установлением соединения порты открываются с помощью последовательности простукивания портов, представляющей собой серию попыток подключения к закрытым портам. Удаленный узел создает и отправляет подлинную последовательность простукивания, чтобы изменить правила брандмауэра сервера и открыть один или несколько определенных портов. Эти изменения выполняются при посредничестве демона простукивания портов, работающего на сервере и отслеживающего файл журнала брандмауэра в поисках попыток соединения, которые можно преобразовать в подлинные последовательности простукивания. После открытия желаемых портов удаленный узел может установить соединение и начать сеанс. Для запуска закрытия порта может использоваться другая последовательность простукивания.

Этот метод не совершенно новый, но его популярность взлетела в 2003 году, когда Мартин Кживински ввел выражение «простукивание портов», написал реализацию, создал обширный веб-сайт и опубликовал статьи о нем в журналах Sys Admin и Linux Journal. Простукивание портов добавляет второй уровень защиты служб, хотя аутентификация обычно слабее той, которую

предоставляют основные службы, такие как SSH. Реализации обычно подвержены перехвату и атакам повторного воспроизведения, а также часто страдают от угроз перебора и отказа в обслуживании.

Преимущество заключается в сокрытии службы, которое гораздо сильнее простого и неэффективного приема с малоизвестными портами, описанного ранее. Службу, грамотно скрытую посредством простукивания портов, почти невозможно обнаружить с помощью активных проб, подобных отправляемым Nmap. С другой стороны, системы на основе перехвата трафика, такие как системы обнаружения вторжений и пассивные средства составления карты сети, без труда обнаруживают эту схему.

Решение о внедрении простукивания портов требует анализа выгод и затрат, применимых к предлагаемой реализации. Сокрытие служб полезно только для небольшого набора приложений. Цель состоит в том, чтобы помешать атакующим подключаться к уязвимым службам (и эксплуатировать их), одновременно сохраняя возможность подключения для авторизованных пользователей со всего мира. Если подключаться должны только определенные IP-адреса, ограничения брандмауэра, разрешающие соединения лишь с этих IP-адресов, обычно являются лучшим подходом. В идеальном мире приложения сами безопасно выполняли бы аутентификацию, и не было бы необходимости скрывать их для предотвращения эксплуатации. К сожалению, даже ориентированные на безопасность программы, такие как SSH, страдали от многочисленных допускающих удаленную эксплуатацию уязвимостей до аутентификации. Хотя такие ошибки в любом случае следует исправлять как можно быстрее, простукивание портов может предоставить дополнительное время до эксплуатации новой ошибки. В конце концов, некоторые эксплойты SSH распространялись в подполье задолго до появления официальных исправлений. А после объявления об ошибке даже самому добросовестному администратору может потребоваться несколько часов или дней, чтобы узнать о ней, проверить исправление, найти и исправить все уязвимые экземпляры. Время реакции владельца домашнего компьютера может быть еще больше. В конце концов, подавляющее большинство пользователей компьютеров не подписано на Bugtraq.

Хорошие парни - не единственные, кто получает выгоду от сокрытия служб. Оно по крайней мере столь же популярно (если не больше) для использования «серыми шляпами» и откровенно преступных целей. Многие интернет-провайдеры запрещают пользователям запускать какие-либо серверные демоны, например веб-службы или SSH. Клиенты могли бы скрыть личный демон SSH или веб-сервер (только для очень ограниченного использования, поскольку публика не смогла бы легко подключиться) с помощью технологии простукивания портов. Аналогично, работодатель моего друга Тома разрешал подключения из дома только с помощью VPN-клиента для Windows. Том ответил созданием системы простукивания портов (до того, как она стала так называться), которая после получения соответствующих проб устанавливала обратный туннель SSH от его рабочего сервера к домашней машине Linux. Это позволило ему работать из дома с полным доступом к рабочей сети, не испытывая унижений от использования Windows. Стоит повторить, что поставщик услуг в обоих примерах - с интернет-провайдером и работодателем - мог обнаружить эту уловку с помощью анализатора трафика или NetFlow. Переходя к еще более темным применениям, компьютерные преступники часто используют подобные приемы для сокрытия бэкдоров в скомпрометированных ими системах. Скрипт-кидди может просто оставить явный демон SSH или даже необработанную корневую оболочку, прослушивающую какой-нибудь высокий порт и уязвимую для обнаружения следующим сканированием Nmap. Более осторожные атакующие используют приемы сокрытия, включая простукивание портов, в своих бэкдорах и руткитах.

Хотя сокрытие служб, предоставляемое этой системой, может быть ценным, оно связано со множеством ограничений. Службы, предназначенные для публичного использования, не подходят, поскольку никто не станет устанавливать специальный клиент простукивания только для посещения вашего веб-сайта. Кроме того, публикация инструкций доступа уничтожила бы основную цель системы. Непубличную службу обычно следует блокировать брандмауэром, а не прикрывать

простукиванием портов. Когда доступ нужен группе людей, VPN часто является лучшим решением, поскольку предлагает шифрование и управление доступом на уровне пользователей. VPN также созданы для работы с реальными сетями, где пакеты могут теряться, дублироваться и переупорядочиваться. Относительно простая проба с использованием реализации Portknocking.Org может потребовать более 30 проб портов, каждая из которых должна прибыть к месту назначения по порядку. Для такого числа проб понадобится специальный клиент. Использование telnet или веб-браузера слишком утомительно. Кроме того, все брандмауэры на пути должны разрешать подключения к этим необычным портам. Учитывая эти ограничения и хлопоты, использование VPN может оказаться столь же удобным.

Дополнительный риск заключается в том, что реализации простукивания портов все еще незрелы. Самая известная из них, написанная Мартином Кживински, предупреждает на странице загрузки: «это прототип, включающий самый необходимый минимум для начала работы. Не используйте его в производственных средах». Также помните, что заблаговременное сканирование для инвентаризации собственной сети будет сложнее при установленных программах такого рода.

Не позволяйте этому длинному списку ограничений отговорить вас даже от рассмотрения простукивания портов. Оно может быть уместно в особых обстоятельствах, особенно связанных со скрытыми бэкдорами или удаленным администрированием личной машины.

11.5.3. Приманки и сети-приманки

Все более популярный метод запутывания атакующих - размещение в сети систем-приманок и наблюдение за атаками на них. Они известны как honeypot. Я состою в Honeynet Project[6], который устанавливает их сети в исследовательских целях. Многие корпорации внедрили такие системы для обеспечения корпоративной безопасности, хотя это рискованно. Необходимость обширного наблюдения делает их требовательными в обслуживании, и всегда существует риск, что атакующие проникнут внутрь и используют машины для совершения серьезных преступлений. Решения с меньшими требованиями к обслуживанию, такие как Honeyd, описанный в следующем разделе, или даже IDS, могут оказаться более подходящими. В любом случае honeypot предназначены для перехвата более навязчивых атак, чем простые сканирования Nmap, поэтому далее не рассматриваются.

11.5.4. Подмена ОС

Несколько программ было разработано специально для обмана определения ОС Nmap. Они изменяют операционную систему узла так, чтобы она поддерживала особые ответы на пробы Nmap. Таким образом ПК с Linux можно заставить походить на принтер Apple LaserWriter или даже веб-камеру. IP Personality[7], выпущенная в 2000 году, - одна из самых популярных систем. Она расширяет инфраструктуру Linux Netfilter для поддержки таких махинаций. К сожалению, она не обновлялась с апреля 2002 года и может не работать с версиями ядра новее 2.4.18.

Одна лишь доступность инструмента не делает подмену ОС хорошей идеей. Затраты усилий нужно как-то оправдать. FAQ IP Personality уклоняется от вопроса «Зачем вам это нужно?», отвечая: «Если вы об этом спрашиваете, значит, не нужно». Тем не менее некоторые считают это достаточно ценным, чтобы писать и использовать подобные инструменты. Одна из причин состоит в том, что конкретные сведения об ОС облегчают атакующим вывод об уязвимостях вашей сети, а также помогают решить, какой эксплойт запускать. Разумеется, настоящая проблема здесь - сама уязвимость, и ее следует исправить. Другие запускают такие инструменты, потому что стесняются используемой ОС или чрезвычайно заботятся о конфиденциальности. Если ваша операционная

система находится в юридически серой зоне из-за того, что какая-то компания заявляет о нарушении прав интеллектуальной собственности и подает иски против пользователей, подмена ОС может защитить от такого назойливого иска.

Одна серьезная проблема такого маскирования ОС узла заключается в том, что оно может вызвать проблемы безопасности и функциональности. Nmap проверяет несколько важных свойств безопасности, например предсказуемость начального порядкового номера TCP и идентификационного номера IP. Эмуляция другой системы, например принтера, может потребовать ослабления этих числовых последовательностей, чтобы они стали предсказуемыми и уязвимыми для всех вытекающих из этого атак. Неясность, полученная подменой отпечатка операционной системы, не стоит жертвы ценными механизмами безопасности. Такая подмена также может искалечить функциональность. Многие тесты определения ОС Nmap спрашивают систему, какие параметры TCP поддерживаются. Притворство, будто определенные параметры, такие как временные метки и масштабирование окна, не поддерживаются, лишит преимуществ эффективности этих параметров. Притворство, будто поддерживаются недоступные параметры, может привести к катастрофе.

В примере 11.2 IP Personality обманывает Nmap, заставляя поверить, что машина Linux на самом деле является игровой консолью Sega Dreamcast. Пример взят из статьи Дэвида Барросо Берруэты «Практический подход к победе над определением отпечатка ОС Nmap»[8]. Эта превосходная статья содержит гораздо больше примеров, а также подробные инструкции по настройке. В ней также описано множество сходных систем с полезными предупреждениями вроде: «код не очень стабилен. Я загрузил модуль, и через несколько мгновений моя машина Linux зависла».

Пример 11.2. Обман Nmap с помощью IP Personality

```
# nmap -sS -O -oN nmap2.log 192.168.0.19

Interesting ports on 192.168.0.19:
(The 1597 ports scanned but not shown below are in state: closed)
Port    State  Service
22/tcp  open   ssh
25/tcp  open   smtp
80/tcp  open   http
143/tcp open   imap
Remote operating system guess: Sega Dreamcast
Nmap finished: 1 IP address (1 host up) scanned in 5.886 seconds
```

Более новая и популярная программа для подмены операционной системы (помимо других возможностей) - Honeyd[9]. Она активно поддерживается автором Нильсом Провосом и имеет несколько значительных преимуществ перед IP Personality. Одно из них - гораздо более простая настройка. Для приведенной выше подмены Dreamcast с помощью IP Personality потребовалось почти 100 строк конфигурации. Honeyd, напротив, просто читает базу определения ОС Nmap и эмулирует любую ОС по выбору пользователя. (Учтите, что Honeyd использует базу первого поколения определения ОС Nmap, поддержка которого была прекращена в 2007 году.) Honeyd также решает проблемы безопасности и функциональности подмены ОС, создавая синтетические узлы для эмуляции. Можно попросить Honeyd занять сотни неиспользуемых IP-адресов в организации. Она отвечает на пробы, отправленные этим IP-адресам, в соответствии со своей конфигурацией. Это устраняет риски безопасности и функциональности, связанные с попыткой замаскировать собственный стек TCP узла. Вместо этого вы создаете множество синтетических узлов, поэтому скрыть ОС существующих узлов это не помогает. Синтетические узлы фактически образуют не требующую большого обслуживания honeynet, за которой можно наблюдать в поисках атак. В основном она предназначена для исследовательских целей, например для использования всемирной сети установок Honeyd с целью выявления новых червей и отслеживания активности спамеров.

Как и в случае других приемов этого раздела, я рекомендую экспериментировать с подменой ОС только тогда, когда вы полностью удовлетворены состоянием своей безопасности. Подмена одной ОС или даже добавление сотен ложных экземпляров Honeyd не заменяет исправления уязвимых систем. Многие атакующие (и особенно черви) даже не утруждаются определением ОС перед отправкой кода эксплойта.

Стоит также отметить, что опытные атакующие легко обнаруживают эти системы. Создать убедительный фасад чрезвычайно трудно, учитывая все различия приложений и стеков TCP между операционными системами. Никто не поверит, что система из примера 11.2 «Обман Nmap с помощью IP Personality», предлагающая IMAP, SMTP и SSH, действительно является Dreamcast под управлением собственной ОС. Кроме того, ошибка во всех версиях до 0.8 включительно позволяла легко идентифицировать Honeyd одним пакетом-пробой. Существует также много характеристик TCP, которые Honeyd пока не умеет обрабатывать. Их можно использовать для обнаружения Honeyd, хотя Nmap эту работу не автоматизирует. Если Honeyd получит широкое распространение, возможности обнаружения, вероятно, будут добавлены в Nmap.

Такие программы обмана, как Honeyd, - лишь одна из причин, по которой пользователям Nmap следует внимательно интерпретировать результаты Nmap и следить за несоответствиями, особенно при сканировании сетей, которыми они не управляют.

11.5.5. Ловушки

Вместо обмана атакующих некоторые стремятся просто замедлить их. Ловушки давно являются популярным средством замедления интернет-червей и спамеров. Некоторые администраторы используют такие приемы TCP, как нулевые размеры окна приема или медленную отдачу данных по одному байту. LaBrea[10] - популярная реализация этого подхода. Другие используют приемы уровня приложений, например длительные задержки перед ответом на команды SMTP. Хотя в основном ими пользуются борцы со спамом, сходные приемы можно применять для замедления сканирований Nmap. Например, ограничение частоты пакетов RST, отправляемых закрытыми портами, может резко замедлить сканеры.

11.5.6. Реагирующее обнаружение сканирования портов

Ранее мы обсуждали обнаружение сканирования с помощью таких инструментов, как Scanlogd. Другие инструменты идут гораздо дальше и действительно отвечают на сканирования. Некоторые предлагают атаковать в ответ, запуская эксплойты или атаки типа «отказ в обслуживании» против источника сканирования. Это ужасная идея по многим причинам. Во-первых, сканирования часто подделываются. Если исходный адрес точен, он может принадлежать предыдущей жертве, которую атакующий использует как козла отпущения. Либо сканирование может быть частью исследовательского обзора Интернета или исходить от законного сотрудника или клиента. Даже если исходный адрес принадлежит компьютеру настоящего атакующего, ответный удар может нарушить работу невинных систем и маршрутизаторов на пути. Кроме того, он может быть незаконным.

Хотя идея ответной атаки широко отвергается сообществом безопасности, гораздо больший интерес вызывает реагирование на обнаруженные атаки посредством изменения правил брандмауэра для блокировки IP-адреса нарушителя. Идея состоит в том, чтобы помешать ему продолжить сканирование настоящей атакой. У этого подхода несколько рисков. Один из них - вы раскрываете карты. Атакующим станет очевидно, что они заблокированы, а у большинства есть множество других IP-адресов, которые можно использовать для продолжения зондирования. Затем

они узнают о вашей реагирующей системе и могут усилить собственные атаки. Более важная проблема состоит в том, что сканирования очень легко подделать. В разделе 10.5.3 «Введение систем обнаружения вторжений в заблуждение» описано несколько способов. Заметив блокировку, атакующий может подделать сканирования от важных систем, например крупных веб-сайтов и DNS-серверов. Целевая сеть, которая затем заблокирует эти IP-адреса, совершит атаку типа «отказ в обслуживании» против самой себя. Ограничение блокировок брандмауэра сканированиями, которые инициируют полное TCP-соединение, уменьшает проблему подделки, но не останавливает даже стандартное SYN-сканирование Nmap.

11.5.7. Эскалация гонки вооружений

Хотя основное внимание в этой книге уделяется инструментам с открытым исходным кодом, ряд коммерческих поставщиков представил продукты, пытающиеся обмануть Nmap. Один из примеров - Cisco Security Agent. В руководстве по оценке заявлены следующие средства защиты от Nmap.

> Network Mapper (Nmap) определяет, какие устройства присутствуют в сети и какие операционные системы и службы на них работают, отправляя серию сетевых проб. Присутствие устройства в сети и работающие на нем порты объявляются его ответом на пробы Nmap. Последовательность возвращаемых сообщений об ошибках идентифицирует операционную систему. Nmap удивительно точен. Его часто используют на начальном этапе атаки или расследования, чтобы определить, какие системы могут ответить на эксплойты атакующего.
>
> Ожидаемый результат сканирования Nmap систем, защищенных Cisco Security Agent: Nmap не способен определить целевую операционную систему систем, работающих со стандартными политиками сервера или рабочего стола. Сканирования Nmap выглядят зависшими, пока истекают тайм-ауты его проверок безопасности. Сканирования Nmap систем, не защищенных Cisco Security Agent, сообщают результаты очень быстро.

Я исследую, как работает CSA и может ли Nmap автоматически обнаруживать ее и подстраиваться. Технология сканирования - это гонка вооружений. Компании, разрабатывающие продукты с открытым исходным кодом и коммерческие продукты, продолжат создавать средства, предназначенные для замедления, блокировки или обмана Nmap и других инструментов. Тем временем Nmap непрерывно совершенствуется, развивая устойчивость перед лицом этих трудностей.

Сноски

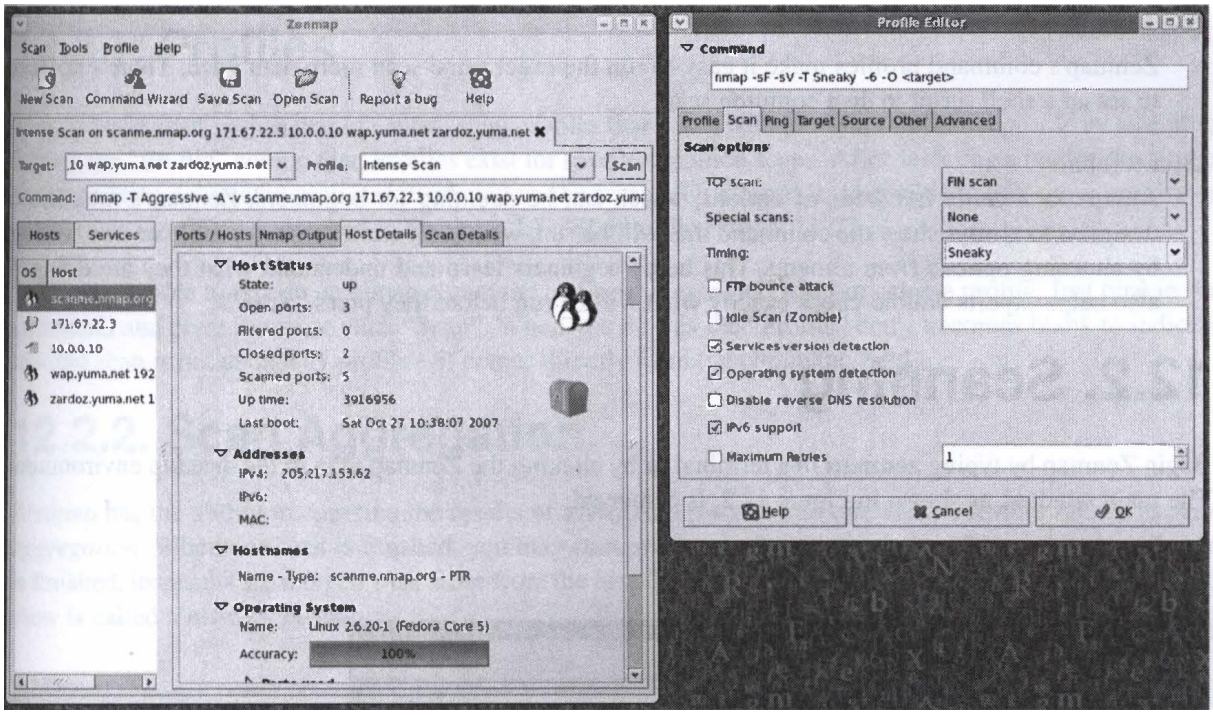
[1] [назад] nmap.org
[2] [назад] logwatch.org
[3] [назад] swatch.sourceforge.net
[4] [назад] sourceforge.net
[5] [назад] openwall.com
[6] [назад] honeynet.org
[7] [назад] ippersonality.sourceforge.net
[8] [назад] nmap.org
[9] [назад] honeyd.org
[10] [назад] labrea.sourceforge.net

Глава 12. Руководство пользователя графического интерфейса Zenmap

12.1. Введение

Zenmap - официальный графический интерфейс пользователя (GUI) для сканера безопасности Nmap. Это многоплатформенное бесплатное приложение с открытым исходным кодом, призванное облегчить начинающим работу с Nmap и в то же время предоставить опытным пользователям Nmap расширенные возможности. Часто используемые сканирования можно сохранять в виде профилей, чтобы их было легко многократно запускать. Конструктор команд позволяет интерактивно создавать командные строки Nmap. Результаты сканирования можно сохранять и просматривать позднее. Сохранённые сканирования можно сравнивать друг с другом, чтобы увидеть, чем они различаются. Результаты недавних сканирований хранятся в базе данных с возможностью поиска. Типичный снимок экрана Zenmap показан на рисунке 12.1. Другие снимки экрана см. на официальной веб-странице Zenmap[1].

Рисунок 12.1. Типичный снимок экрана Zenmap



Это руководство предназначено для того, чтобы упростить совместное использование Nmap и Zenmap, даже если раньше вы не пользовались ни тем, ни другим. О тех частях этого руководства, которые относятся непосредственно к Nmap (параметры командной строки и тому подобное), см. главу 15 «Справочное руководство по Nmap».

12.1.1. Назначение графической оболочки для Nmap

Никакая графическая оболочка не может заменить старый добрый Nmap для командной строки. Сама природа графической оболочки такова, что при выполнении своей работы она зависит от

другого инструмента. Поэтому назначение Zenmap состоит не в том, чтобы заменить Nmap, а в том, чтобы сделать Nmap *более полезным*. Вот некоторые преимущества, которые Zenmap предлагает по сравнению с обычным Nmap.

Интерактивный и графический просмотр результатов

Помимо показа обычного вывода Nmap, Zenmap может организовать отображение так, чтобы показать все порты одного узла или все узлы, на которых работает определённая служба. Он сводит сведения об одном узле или обо всём сканировании в удобное представление. Zenmap способен даже нарисовать топологическую карту обнаруженных сетей. Результаты нескольких сканирований можно объединить и просматривать одновременно.

Сравнение

Zenmap умеет графически показывать различия между двумя сканированиями. Можно увидеть, что изменилось между запусками одного и того же сканирования в разные дни, между сканированиями двух разных узлов, между сканированиями одних и тех же узлов с разными параметрами или в любой другой комбинации. Это позволяет администраторам отслеживать появление новых узлов или служб в своих сетях либо прекращение работы существующих.

Удобство

Zenmap хранит результаты ваших сканирований, пока вы не решите их удалить. Это означает, что можно запустить сканирование, посмотреть результаты, а затем решить, сохранять ли их в файл. Нет необходимости заранее придумывать имя файла.

Повторяемость

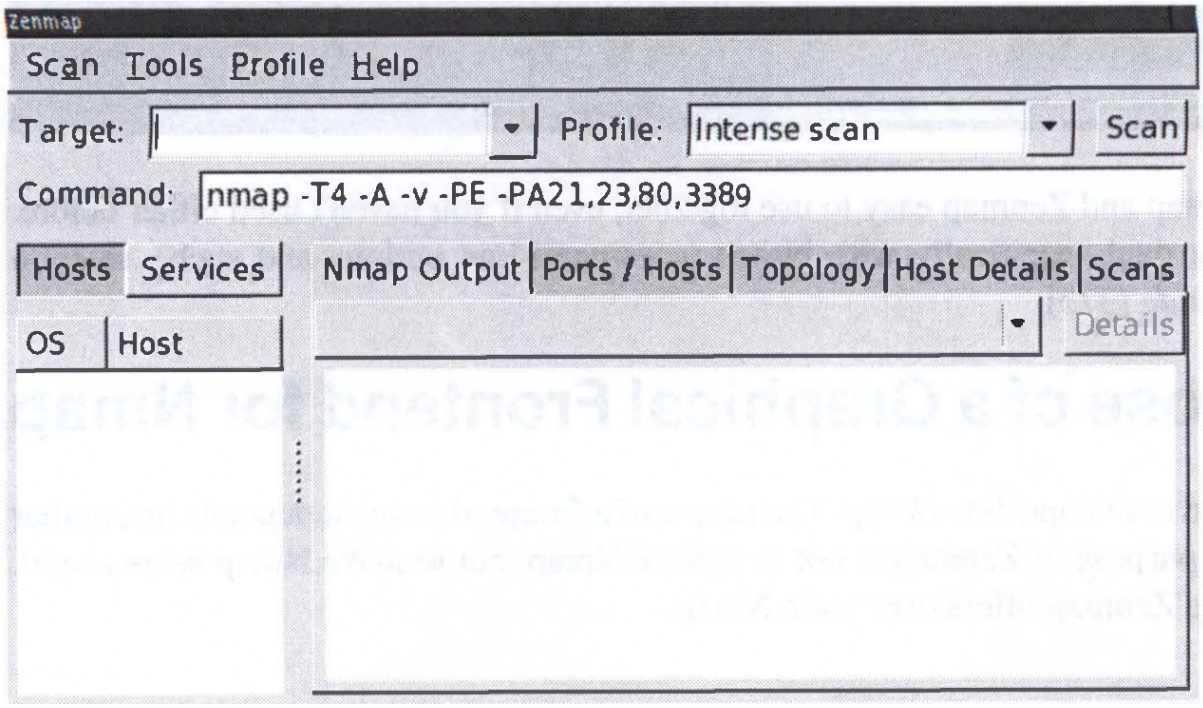
Профили команд Zenmap позволяют легко запускать в точности одно и то же сканирование больше одного раза. Для выполнения обычного сканирования не нужно создавать сценарий оболочки.

Доступность для изучения

У Nmap буквально сотни параметров, что может устрашать начинающих. Интерфейс Zenmap устроен так, чтобы всегда показывать команду, которая будет выполнена, независимо от того, получена ли она из профиля или создана выбором параметров в меню. Это помогает начинающим изучать и понимать, что они делают. Кроме того, это помогает специалистам ещё раз точно проверить, что будет запущено, прежде чем они нажмут «Scan».

12.2. Сканирование

Запустите Zenmap, введя `zenmap` в терминале или щёлкнув значок Zenmap в окружении рабочего стола. Появится главное окно, показанное на рисунке 12.2.



Одна из целей Zenmap - сделать сканирование безопасности простым и для начинающих, и для специалистов. Чтобы запустить сканирование, достаточно ввести цель в поле «Target», выбрать профиль «Intense scan» и нажать кнопку «Scan». Это показано на рисунке 12.3.

Рисунок 12.3. Выбор цели и профиля

Во время выполнения сканирования (и после его завершения) на экране показывается вывод команды Nmap.

В поле цели можно ввести любое число целей, разделённых пробелами. Zenmap поддерживает все спецификации целей, поддерживаемые Nmap, поэтому такие цели, как `192.168.0.0/24` и `10.0.0-5.*`, работают. Zenmap запоминает цели, сканировавшиеся последними. Чтобы повторно просканировать узел, выберите его из раскрывающегося списка, присоединённого к текстовому полю «Target».

12.2.1. Профили

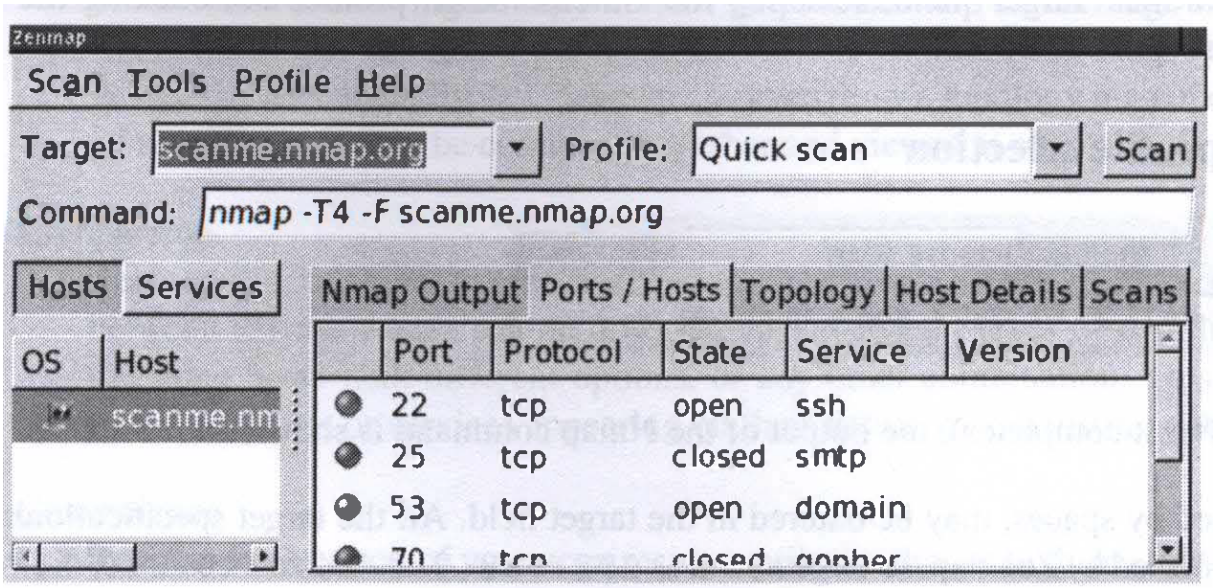
«Intense scan» - лишь один из нескольких профилей сканирования, поставляемых с Zenmap. Выберите профиль из раскрывающегося списка «Profile». Профили существуют для нескольких распространённых сканирований. После выбора профиля связанная с ним командная строка Nmap отображается на экране. Разумеется, эти профили можно редактировать или создавать новые. Это рассматривается в разделе 12.7 «Редактор профилей».

Можно также ввести команду Nmap и выполнить её без использования профиля. Просто введите её в поле команды и нажмите клавишу возврата или кнопку «Scan». При этом запись «Profile» становится пустой, показывая, что сканирование не использует никакого профиля: оно поступает непосредственно из поля команды.

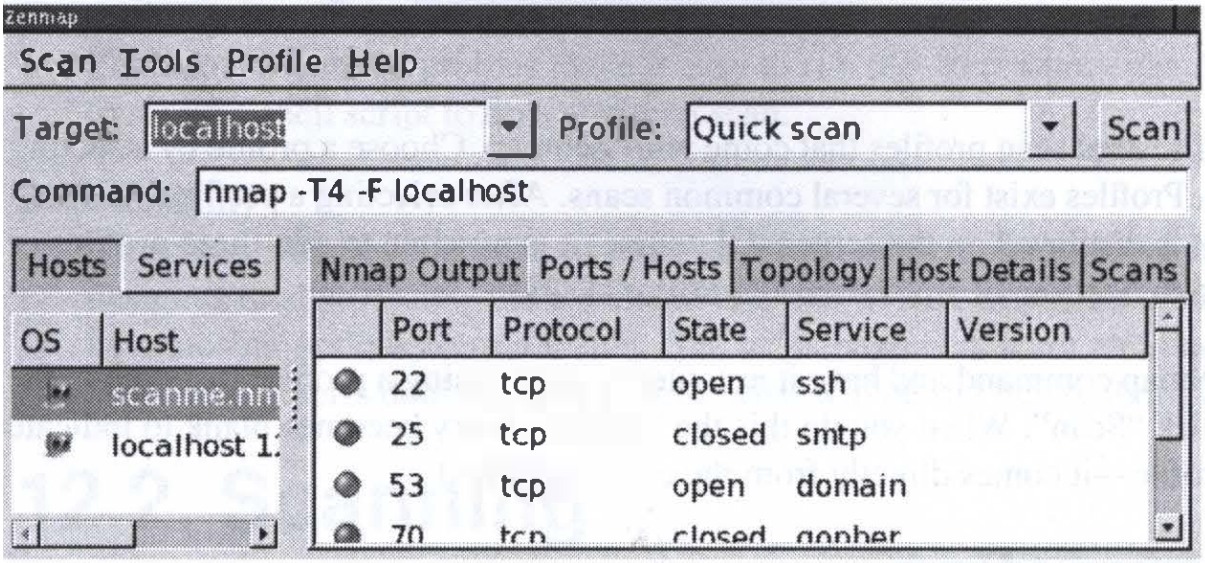
12.2.2. Объединение сканирований

Zenmap умеет объединять результаты многих сканирований Nmap в одном представлении - эта возможность называется *объединением сканирований*. Когда одно сканирование завершено, в том же окне можно начать другое. После завершения второго сканирования его результаты объединяются с результатами первого. Набор сканирований, составляющих объединённое представление, называется *инвентаризацией сети*.

Пример объединения сделает это понятие яснее. Запустим быстрое сканирование `scanme.nmap.org`.

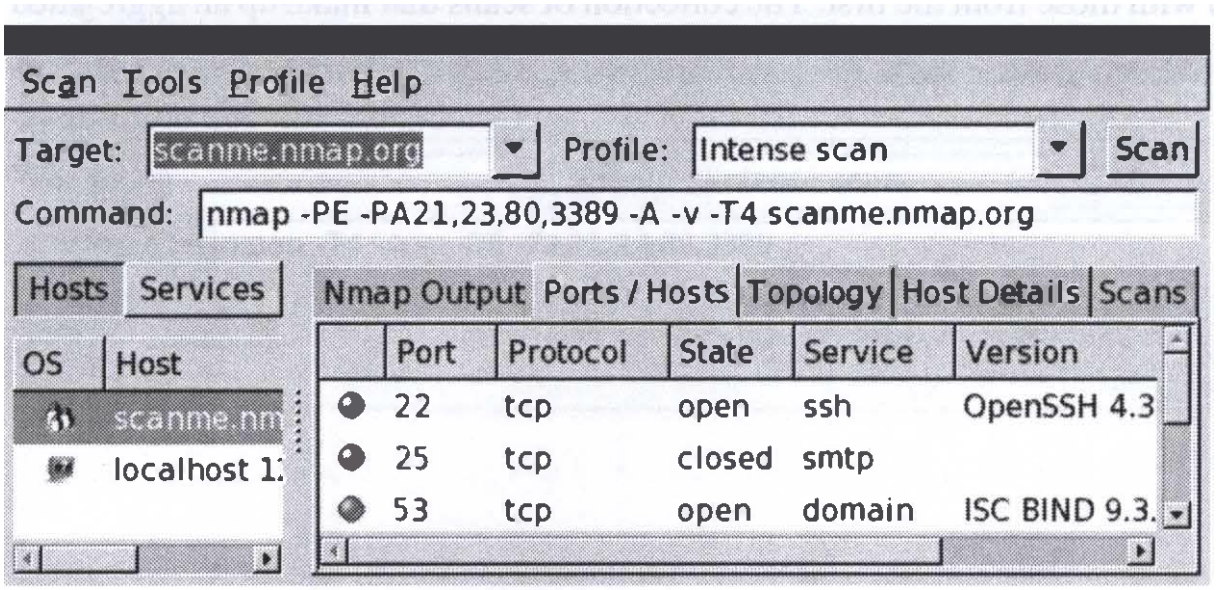


Now do the same against localhost:



Теперь сделаем то же самое для `localhost`:

Теперь показаны результаты и для `scanme`, и для `localhost`. Это можно было бы сделать одним сканированием Nmap, указав обе цели, хотя удобно не обдумывать все цели заранее. Теперь предположим, что нам нужны дополнительные сведения о `scanme`, поэтому мы запускаем для него интенсивное сканирование.



Теперь рядом с `scanme` появился маленький значок пингвина, показывающий, что его операционная система определена как Linux. Кроме того, были определены некоторые его службы. Теперь мы делаем то, чего нельзя сделать одним сканированием Nmap, поскольку в нём невозможно отдельно выбрать один узел для более интенсивного сканирования, как сделали мы. Результаты для `localhost` по-прежнему присутствуют, хотя мы не узнаем о нём больше, чем знали раньше, если только не решим провести более углублённое сканирование.

Необязательно ждать завершения одного сканирования, прежде чем начинать другое. Несколько сканирований могут выполняться одновременно. По мере завершения каждого из них его результаты добавляются в инвентаризацию. Инвентаризацию может составлять любое количество сканирований; управление набором сканирований осуществляется на вкладке результатов сканирования «Scans», полностью описанной в разделе «Вкладка Scans».

Одновременно может быть открыто несколько инвентаризаций. В Zenmap принято, что одно окно представляет одну инвентаризацию сети. Чтобы начать новую инвентаризацию, выберите «New Window» в меню «Scan» или воспользуйтесь сочетанием клавиш `Ctrl+N`. Запуск сканирования кнопкой «Scan» добавит его в инвентаризацию текущего окна. Чтобы поместить его в другую инвентаризацию, откройте отдельное окно и запустите сканирование оттуда. Загрузка результатов сканирования из файла или каталога начнёт новую инвентаризацию, если только вы не воспользуетесь пунктом меню «Open Scan in This Window». Подробнее о сохранении и загрузке сетевых инвентаризаций и отдельных сканирований см. в разделе 12.4 «Сохранение и загрузка результатов сканирования».

Чтобы закрыть окно, выберите «Close Window» в меню «Scan» или нажмите `Ctrl+W`. Когда все открытые окна будут закрыты, приложение завершит работу. Чтобы закрыть все открытые окна, выберите «Quit» или нажмите `Ctrl+Q`.

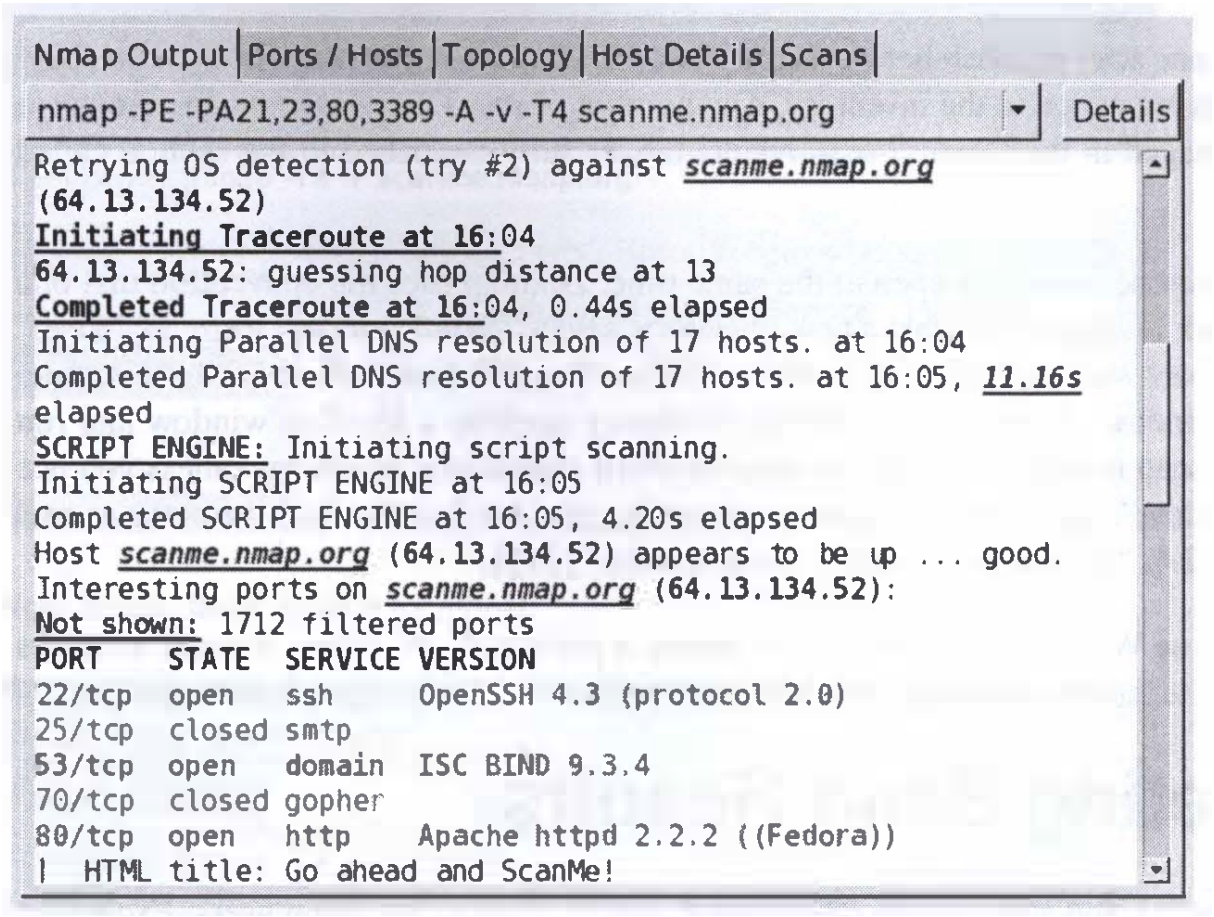
12.3. Интерпретация результатов сканирования

Вывод Nmap отображается во время и после сканирования. Пользователям Nmap этот вывод будет знаком. За исключением цветовой подсветки Zenmap, по сравнению с запуском Nmap в терминале это не даёт преимуществ в визуализации. Однако другие части интерфейса Zenmap интерпретируют и объединяют вывод терминала таким образом, чтобы результаты сканирования было легче понимать и использовать.

12.3.1. Вкладки результатов сканирования

В каждом окне сканирования имеется пять вкладок, каждая из которых отображает различные аспекты результатов сканирования. Это «Nmap Output», «Ports / Hosts», «Topology», «Host Details» и «Scans». Каждая из них рассматривается в этом разделе.

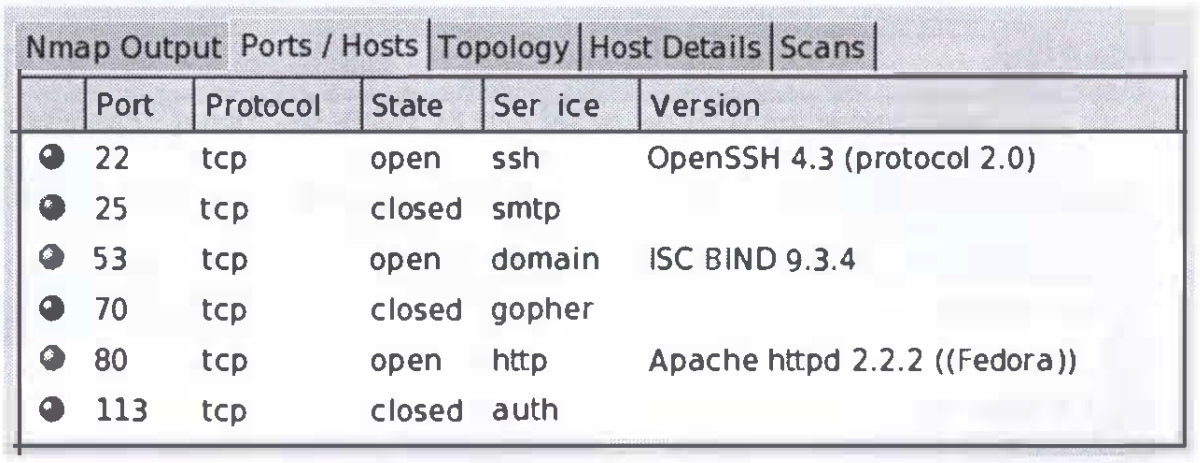
Вкладка «Nmap Output»



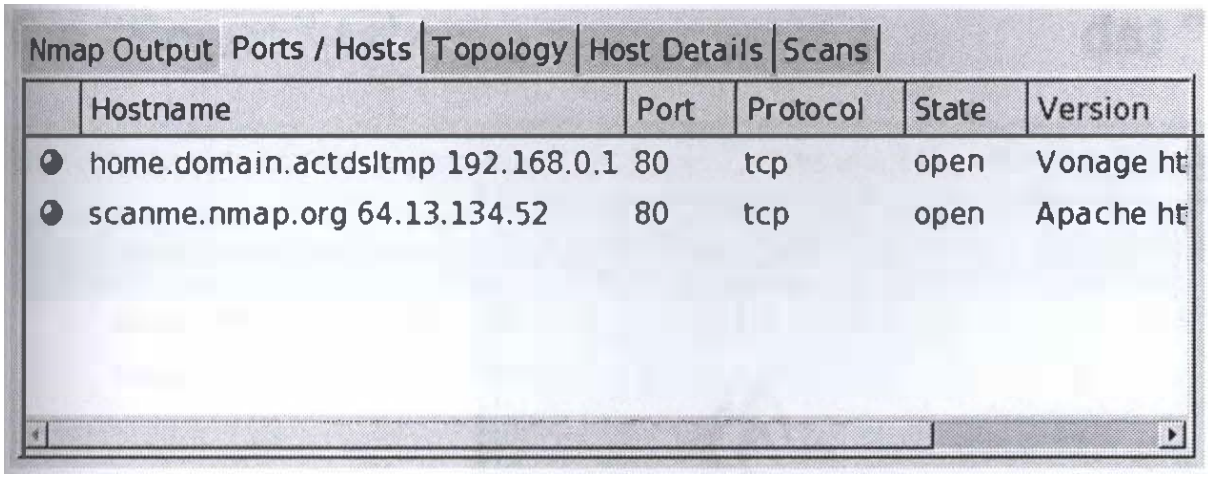
При запуске сканирования по умолчанию отображается вкладка «Nmap Output». На ней показывается знакомый терминальный вывод Nmap. Части вывода выделяются в соответствии с их смыслом; например, открытые и закрытые порты отображаются разными цветами. Пользовательскую подсветку можно настроить в `zenmap.conf` (см. раздел 12.11 «Описание zenmap.conf»).

Напомним, что в одном окне могут отображаться результаты более чем одного сканирования (см. раздел 12.2.2 «Объединение сканирований»). Раскрывающийся список в верхней части вкладки позволяет выбрать отображаемое сканирование. Кнопка «Details» открывает окно с разнообразными сведениями о сканировании, такими как временные отметки, параметры командной строки и номер использованной версии Nmap.

Вкладка «Ports / Hosts»

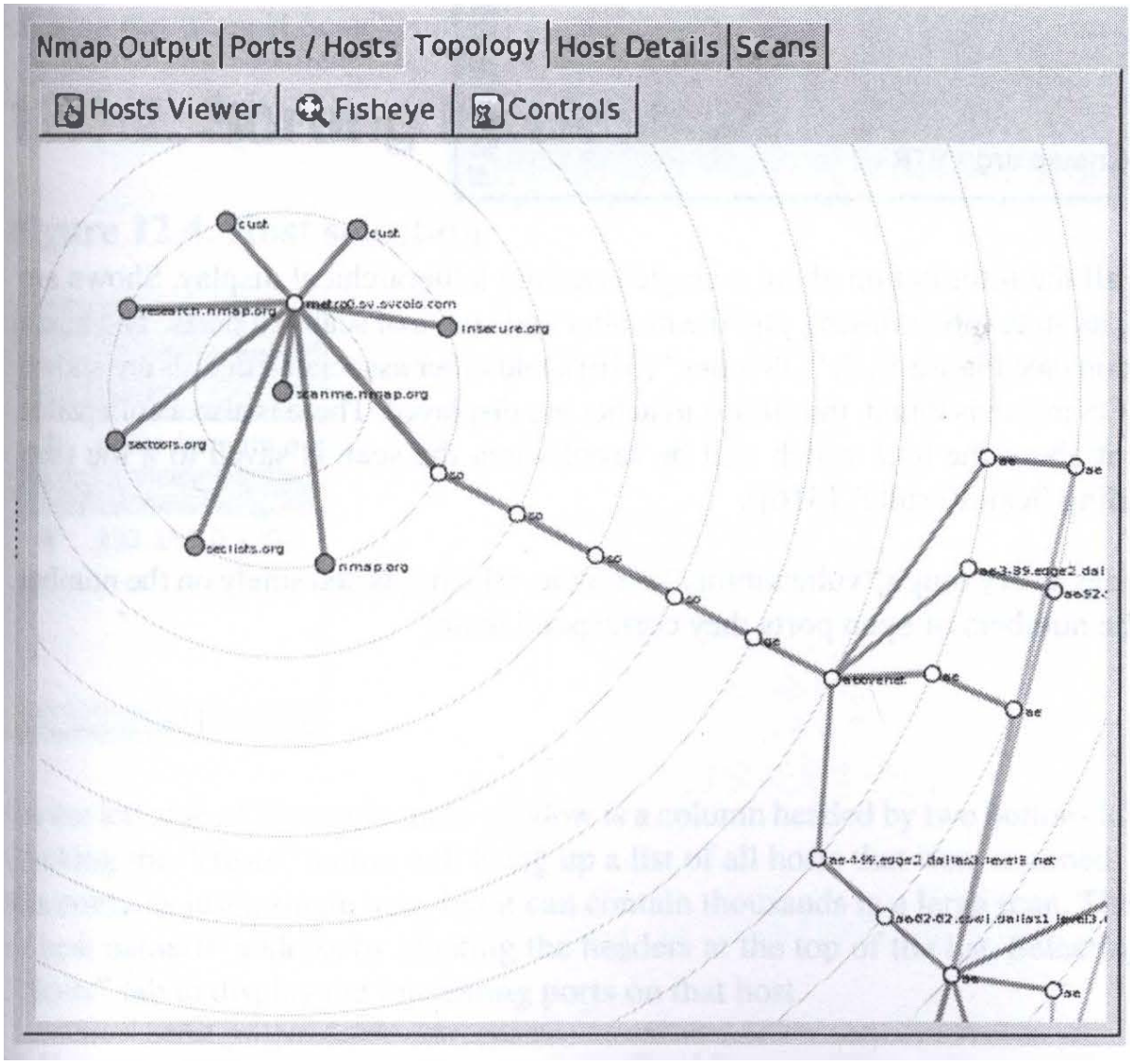


Содержимое вкладки «Ports / Hosts» различается в зависимости от того, выбран ли в данный момент узел или служба. Когда выбран узел, на ней показываются все представляющие интерес порты этого узла, а при наличии - и сведения о версиях. Выбор узла подробнее описан в разделе 12.3.2 «Сортировка по узлам».



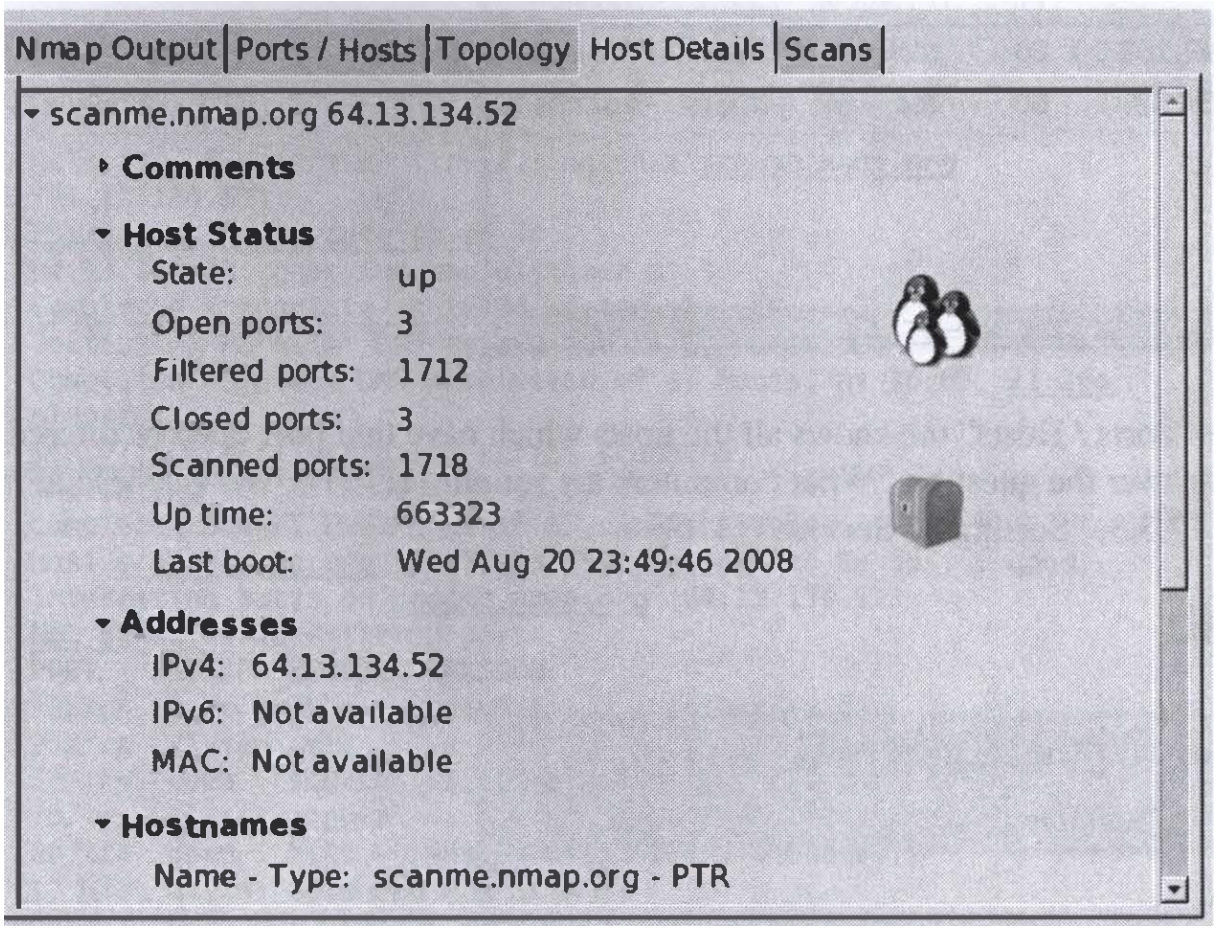
Когда выбрана служба, вкладка «Ports / Hosts» показывает все узлы, у которых этот порт открыт или фильтруется. Это хороший способ быстро ответить на вопрос: «На каких компьютерах работает HTTP?» Выбор службы подробнее описан в разделе 12.3.3 «Сортировка по службам».

Вкладка «Topology»



Вкладка «Topology» представляет собой интерактивное отображение связей между узлами сети. Узлы располагаются на концентрических окружностях. Каждая окружность представляет ещё один сетевой переход от центрального узла. Щелчок по узлу перемещает его в центр. Поскольку вкладка «Topology» отображает сетевые пути между узлами, ей полезно применение параметра `--traceroute`. Представление топологии подробнее рассматривается в разделе 12.5 «Путешествие по топологии сети».

Вкладка «Host Details»

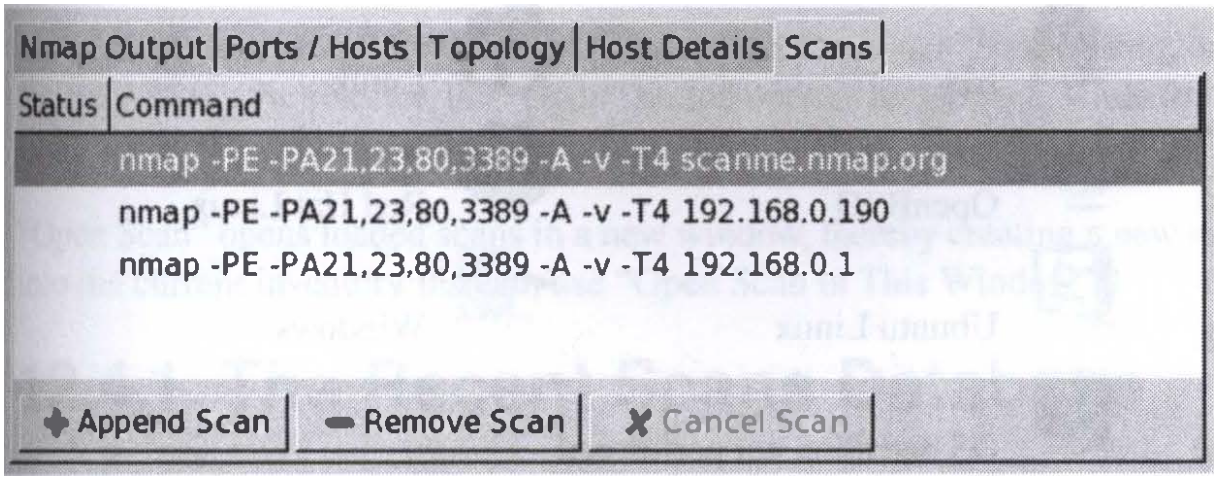


Вкладка «Host Details» раскладывает все сведения об одном узле в иерархическое представление. Показываются имена и адреса узла, его состояние (работает или не работает), а также количество и состояние просканированных портов. Когда доступны, показываются время работы узла, операционная система, значок ОС (см. рисунок 12.5 «Значки ОС») и другие связанные сведения. Если точного совпадения ОС не найдено, отображаются ближайшие совпадения. Имеется также сворачиваемое текстовое поле для хранения комментария об узле; этот комментарий будет сохранён при сохранении сканирования в файл (см. раздел 12.4 «Сохранение и загрузка результатов сканирования»).

У каждого узла есть значок, дающий очень приблизительную оценку «уязвимости», основанную исключительно на количестве открытых портов. Значки и соответствующее им количество открытых портов таковы:

• 0-2 открытых порта;
• 3-4 открытых порта;
• 5-6 открытых портов;
• 7-8 открытых портов;
• 9 или более открытых портов.

Вкладка «Scans»

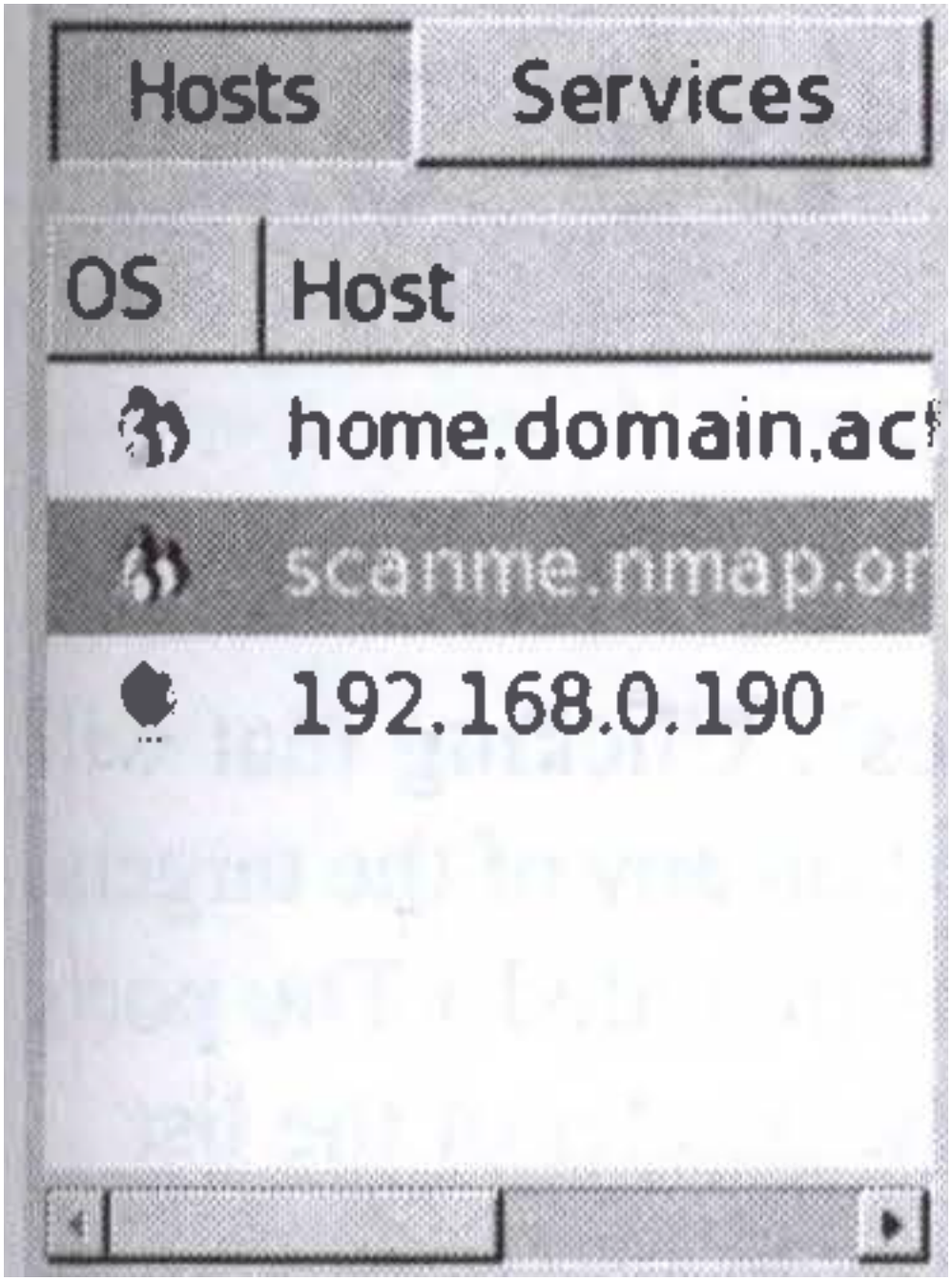


Вкладка «Scans» показывает все сканирования, объединённые в сетевую инвентаризацию. С этой вкладки можно добавлять сканирования (из файла или каталога) и удалять их.

Пока сканирование выполняется и ещё не завершено, оно имеет состояние «Running». Выполняющееся сканирование можно отменить, нажав кнопку «Cancel Scan».

12.3.2. Сортировка по узлам

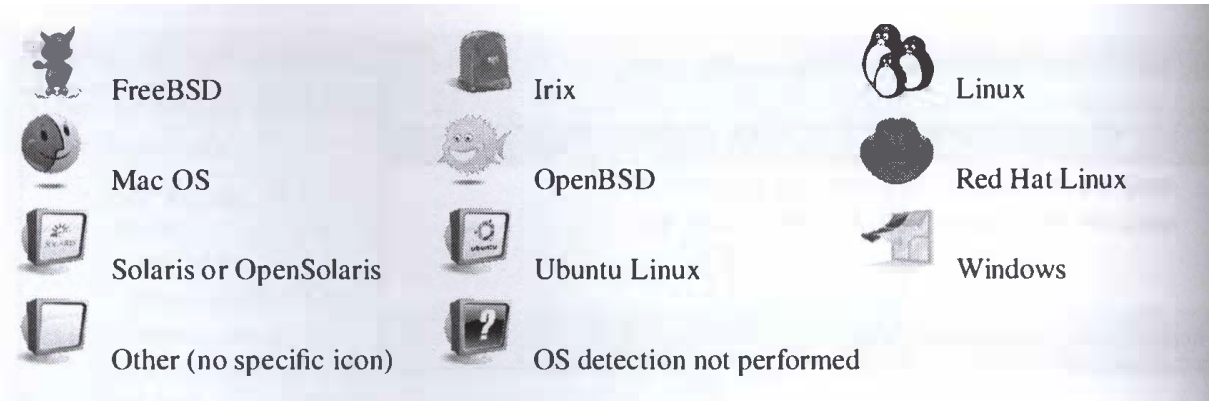
Рисунок 12.4. Выбор узла



С левой стороны главного окна Zenmap находится столбец, озаглавленный двумя кнопками: «Hosts» и «Services». Щелчок по кнопке «Hosts» открывает список всех просканированных узлов, как на рисунке 12.4. Обычно он содержит всего один узел, но при крупном сканировании может содержать тысячи. Список узлов можно сортировать по ОС или по имени узла/IP-адресу, щёлкая заголовки в верхней части списка. При выборе узла вкладка «Ports / Hosts» показывает представляющие интерес порты этого узла.

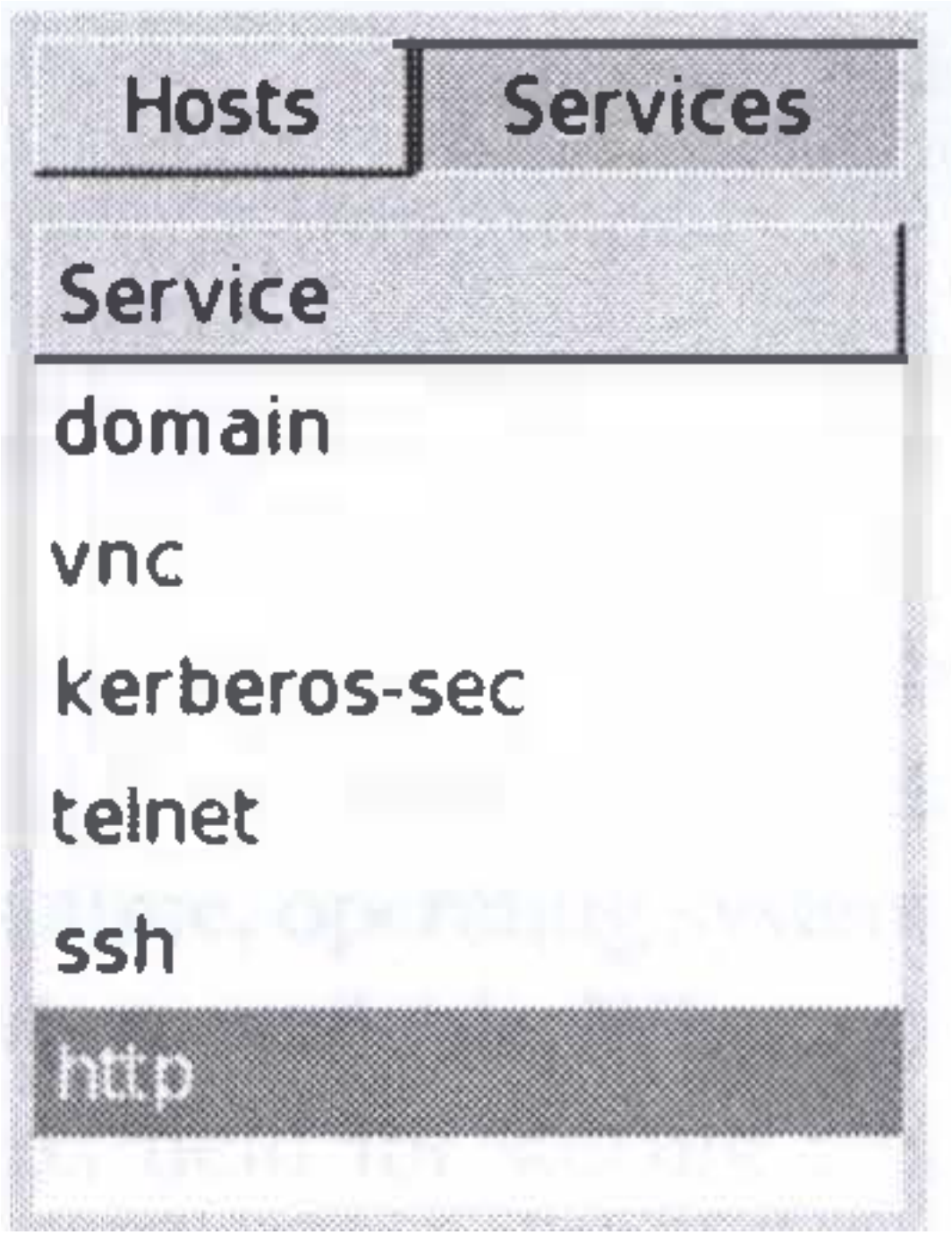
Каждый узел обозначается своим именем или IP-адресом и имеет значок, указывающий операционную систему, определённую для этого узла. Значок имеет смысл только в том случае, если выполнялось определение ОС (`-O`). В противном случае это будет стандартный значок, показывающий, что ОС неизвестна. На рисунке 12.5 показаны все возможные значки. Обратите внимание: определение ОС Nmap не всегда способно обеспечить ту степень конкретности, которую подразумевают значки; например, узел Red Hat Linux часто отображается стандартным значком Linux.

Рисунок 12.5. Значки ОС



12.3.3. Сортировка по службам

Рисунок 12.6. Выбор службы



Над тем же списком, который содержит все просканированные узлы, находится кнопка «Services». Щелчок по ней превратит этот список в список всех портов, находящихся в состоянии `open`, `filtered` или `open|filtered` на любой из целей, как показано на рисунке 12.6. (Порты, которые не были явно перечислены в выводе Nmap, не включаются.) Порты обозначаются именами служб (`http`, `ftp` и т. д.). Список можно сортировать щелчком по его заголовку.

При выборе службы вкладка «Ports / Hosts» покажет все узлы, у которых эта служба открыта или фильтруется.

12.4. Сохранение и загрузка результатов сканирования

Чтобы сохранить отдельное сканирование в файл, выберите «Save Scan» в меню «Scan» (или воспользуйтесь сочетанием клавиш `Ctrl+S`). Если в инвентаризации больше одного сканирования, вам предложат выбрать, какое из них сохранить. Результаты сохраняются в формате Nmap XML, рассматриваемом в разделе 13.6 «Вывод XML (-oX)».

Все сканирования инвентаризации можно сохранить с помощью пункта «Save to Directory» в меню «Scan» (`Ctrl+Alt+S`). При первом сохранении инвентаризации обычно создают новый каталог кнопкой «Create Folder» в диалоговом окне сохранения. При последующих сохранениях можно продолжать сохранять в тот же каталог. Чтобы снизить вероятность перезаписи посторонних файлов сканирования, функция сохранения в каталог откажется продолжать работу, если выбранный каталог содержит файл, не принадлежащий инвентаризации. Если вы уверены, что хотите сохранить данные в этот каталог, удалите все мешающие файлы, а затем повторите сохранение.

Сохранённые результаты загружаются выбором «Open Scan» в меню «Scan» или нажатием сочетания клавиш `Ctrl+0`. В окне выбора файлов кнопка «Open» открывает одно сканирование, а кнопка «Open Directory» - каждый файл в выбранном каталоге (который, возможно, был создан с помощью «Save to Directory»).

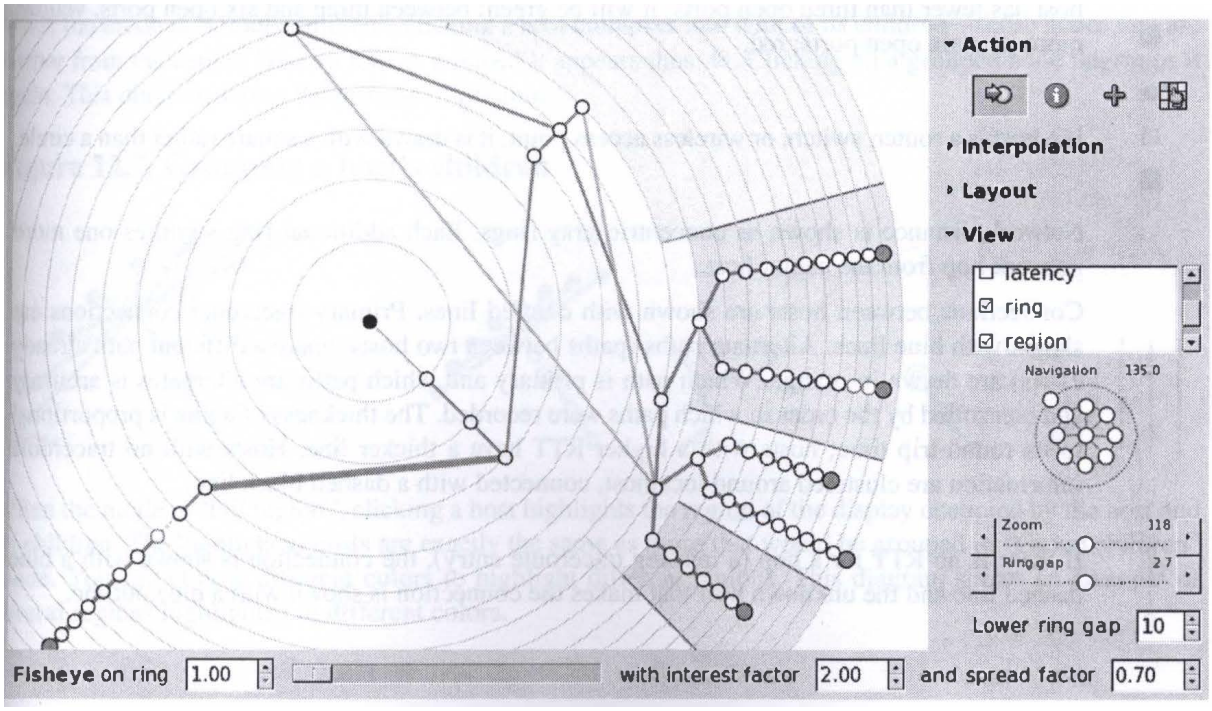
«Open Scan» открывает загруженные сканирования в новом окне, тем самым создавая новую инвентаризацию. Чтобы вместо этого объединить загруженные сканирования с текущей инвентаризацией, воспользуйтесь пунктом «Open Scan in This Window».

12.4.1. База данных недавних сканирований

Результаты сканирования, не сохранённые в файл, автоматически сохраняются в базе данных. Результаты сканирования, загруженные из файла, а затем изменённые (например, добавлением комментария об узле), но не сохранённые повторно, также сохраняются в базе данных. База данных хранится в файле `zenmap.db`, а его расположение зависит от платформы (см. раздел 12.10 «Файлы, используемые Zenmap»). По умолчанию сканирования хранятся в базе данных 60 дней, а затем удаляются. Этот интервал времени можно изменить, изменив значение переменной `save_time` в разделе `[search]` файла `zenmap.conf` (см. раздел 12.11 «Описание zenmap.conf»).

Поскольку интерфейс поиска Zenmap по умолчанию ищет в содержимом базы данных недавних сканирований, он одновременно служит средством просмотра базы данных. При открытии окна поиска показывается каждое сканирование из базы данных. Затем список сканирований можно отфильтровать строкой поиска. См. раздел 12.8 «Поиск сохранённых результатов».

12.5. Путешествие по топологии сети



12.5.1. Обзор вкладки «Topology»

Вкладка «Topology» в Zenmap обеспечивает интерактивную анимированную визуализацию связей между узлами сети. Узлы показываются вершинами графа, радиально расходящегося от центра. Для перемещения изображения щёлкните и перетащите его, а для увеличения и уменьшения масштаба используйте предусмотренные элементы управления. Щёлкните по узлу, и он станет новым центром. Граф плавно перестроится, отражая новый взгляд на сеть. Запустите новое сканирование, и каждый новый узел и сетевой путь будут автоматически добавлены в топологию.

Представление топологии наиболее полезно в сочетании с параметром трассировки Nmap `--traceroute`, потому что именно этот параметр обнаруживает сетевой путь к узлу. В топологии можно просматривать сетевую инвентаризацию без сведений трассировки, но сетевые пути видны не будут. Однако помните, что благодаря объединению сканирований Zenmap добавить сведения трассировки в сетевую инвентаризацию можно, просто запустив ещё одно сканирование.

Изначально топология показывается с точки зрения `localhost`, а вы находитесь в центре. Щёлкните по узлу, чтобы переместить его в центр и увидеть, как выглядит сеть с его точки зрения.

Представление топологии является адаптацией программы RadialNet, созданной João Paulo S. Medeiros.

12.5.2. Условные обозначения

В представлении топологии используется множество символов и цветовых соглашений. В этом разделе объясняется, что они означают.

• Каждый обычный узел сети представлен маленьким кругом. Цвет и размер круга определяются количеством открытых портов на узле. Чем больше открытых портов, тем больше круг.
• Белый круг представляет промежуточный узел сетевого пути, который не сканировался по портам. Если у узла меньше трёх открытых портов, круг будет зелёным; от трёх до шести открытых портов - жёлтым; больше шести открытых портов - красным.

• Если узел является маршрутизатором, коммутатором или беспроводной точкой доступа, он изображается квадратом, а не кругом.
• Сетевое расстояние показывается концентрическими серыми окружностями. Каждая следующая окружность означает ещё один сетевой переход от центрального узла.
• Связи между узлами показываются цветными линиями. Основные соединения трассировки показываются синими линиями. Альтернативные пути (пути между двумя узлами, для которых уже существует другой путь) рисуются оранжевыми. Какой путь является основным, а какие - альтернативными, определяется произвольно и зависит от порядка записи путей. Толщина линии пропорциональна времени прохождения в оба конца: к узлам с большим RTT ведёт более толстая линия. Узлы, о которых нет сведений трассировки, группируются вокруг `localhost` и соединяются с ним чёрной пунктирной линией.
• Если для перехода нет значения RTT (в трассировке пропущена запись), соединение показывается синей пунктирной линией, а неизвестный узел, образующий это соединение, - синим контуром.

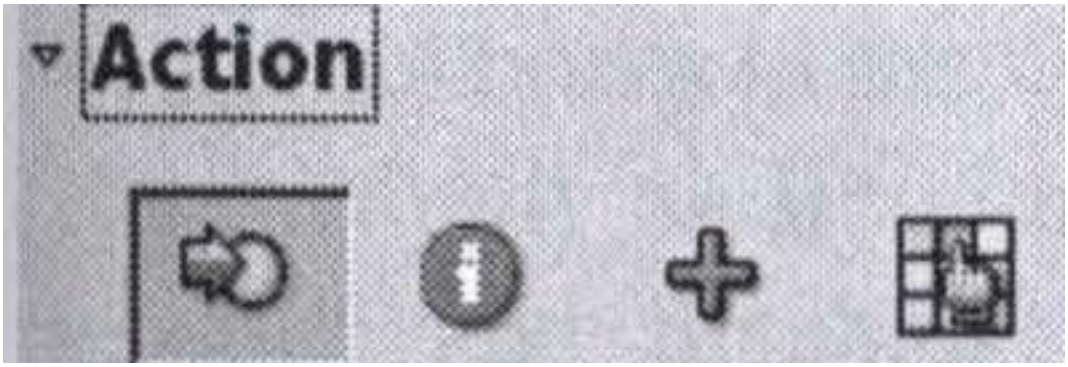
Некоторые узлы специального назначения могут иметь один или несколько значков, описывающих тип узла:

• маршрутизатор;
• коммутатор;
• беспроводная точка доступа;
• межсетевой экран;
• узел, некоторые порты которого фильтруются.

12.5.3. Элементы управления

После нажатия кнопки «Controls» элементы управления появляются в отдельном столбце. Они разделены на секции.

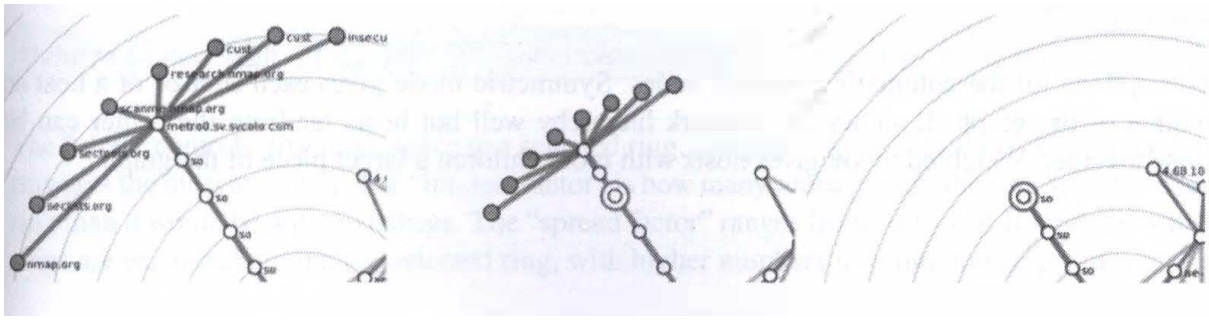
Элементы управления действиями



Элементы управления в секции «Action» определяют, что происходит при щелчке по узлу. Кнопки в этой секции слева направо: «Change focus», «Show information», «Group children» и «Fill region». В режиме «Change focus» щелчок по узлу перестраивает изображение, помещая выбранный узел в центр. В режиме «Show information» щелчок по узлу открывает окно со сведениями о нём.

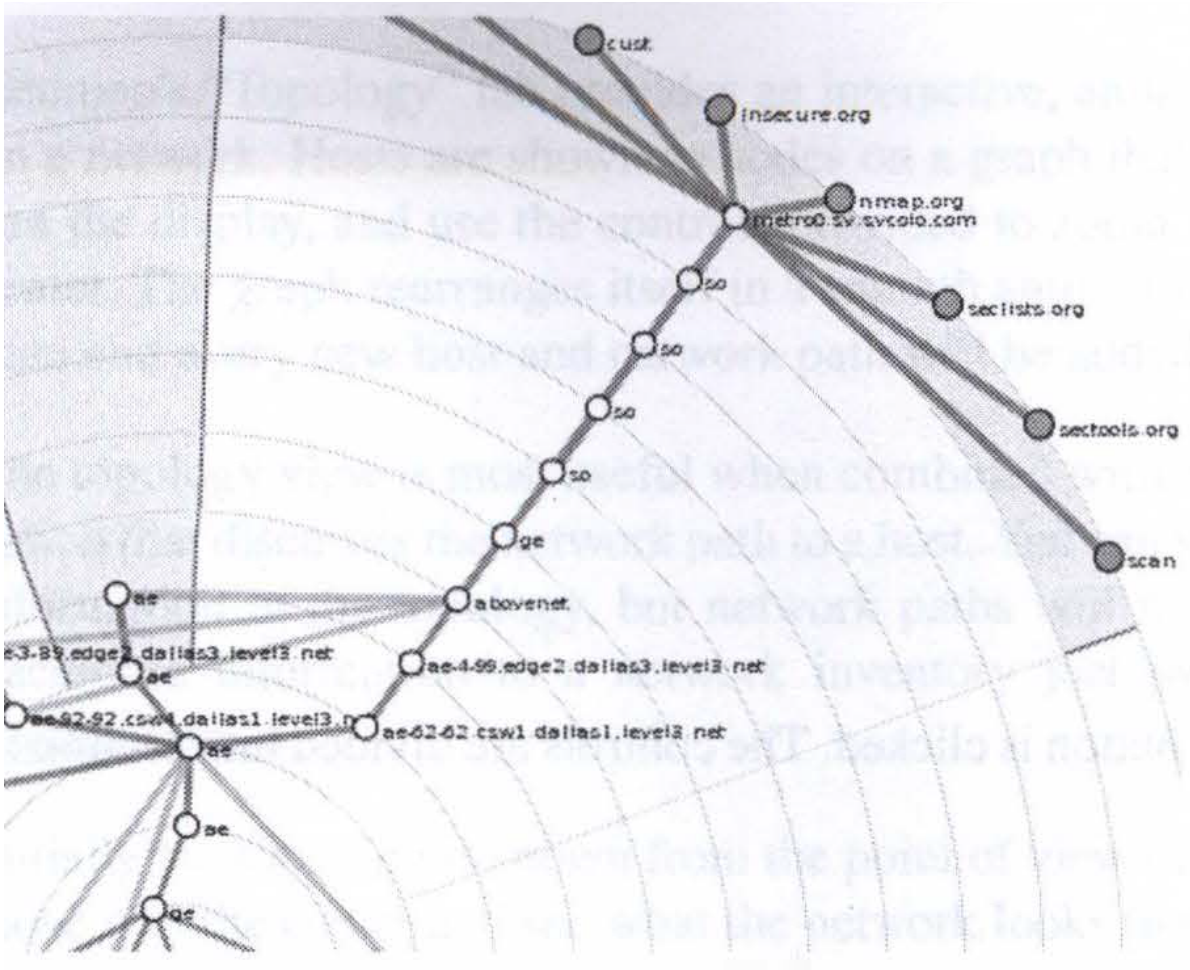
В режиме «Group children» щелчок по узлу сворачивает в него все его дочерние узлы - те вершины, которые находятся дальше от центра. Сгруппированный узел помечается специальным значком. Щелчок по сгруппированному узлу снова разгруппировывает его. На этой схеме показан процесс группирования.

Рисунок 12.7. Группирование дочерних узлов

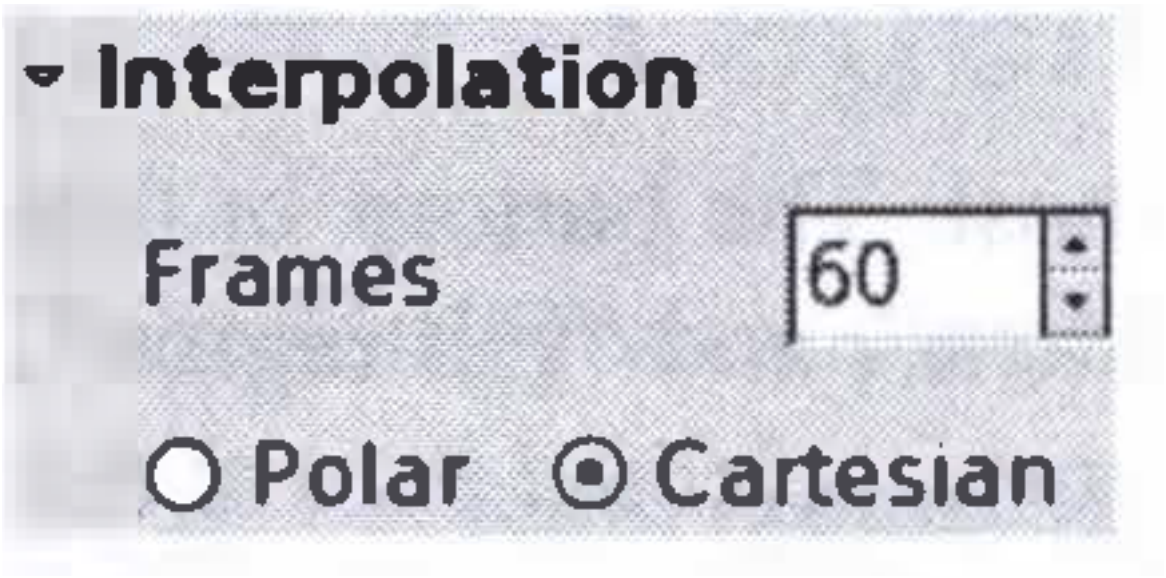


В режиме «Fill region» щелчок по узлу выделяет область изображения, занятую этим узлом и его дочерними узлами. Выделяются в точности те же узлы, которые были бы сгруппированы в режиме «Group children». Для выделения разных областей можно выбирать разные цвета. На этой схеме показан пример нескольких областей, выделенных разными цветами.

Рисунок 12.8. Выделение областей топологии

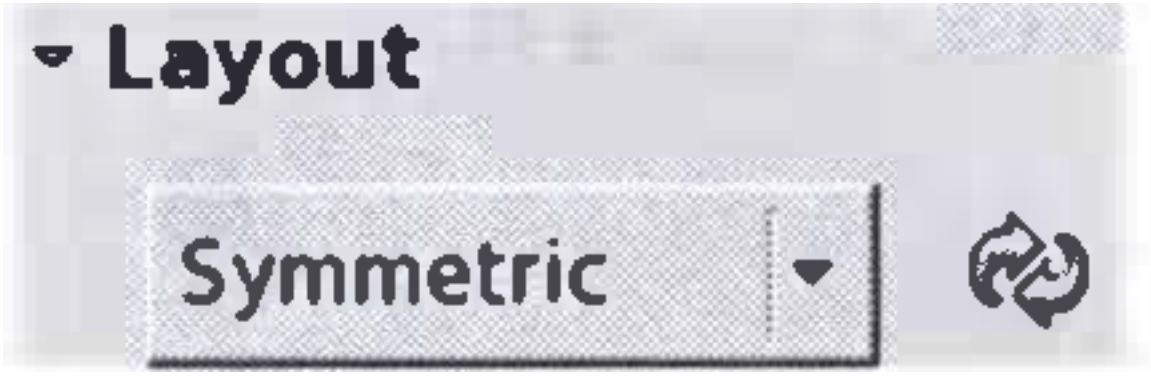


Элементы управления интерполяцией



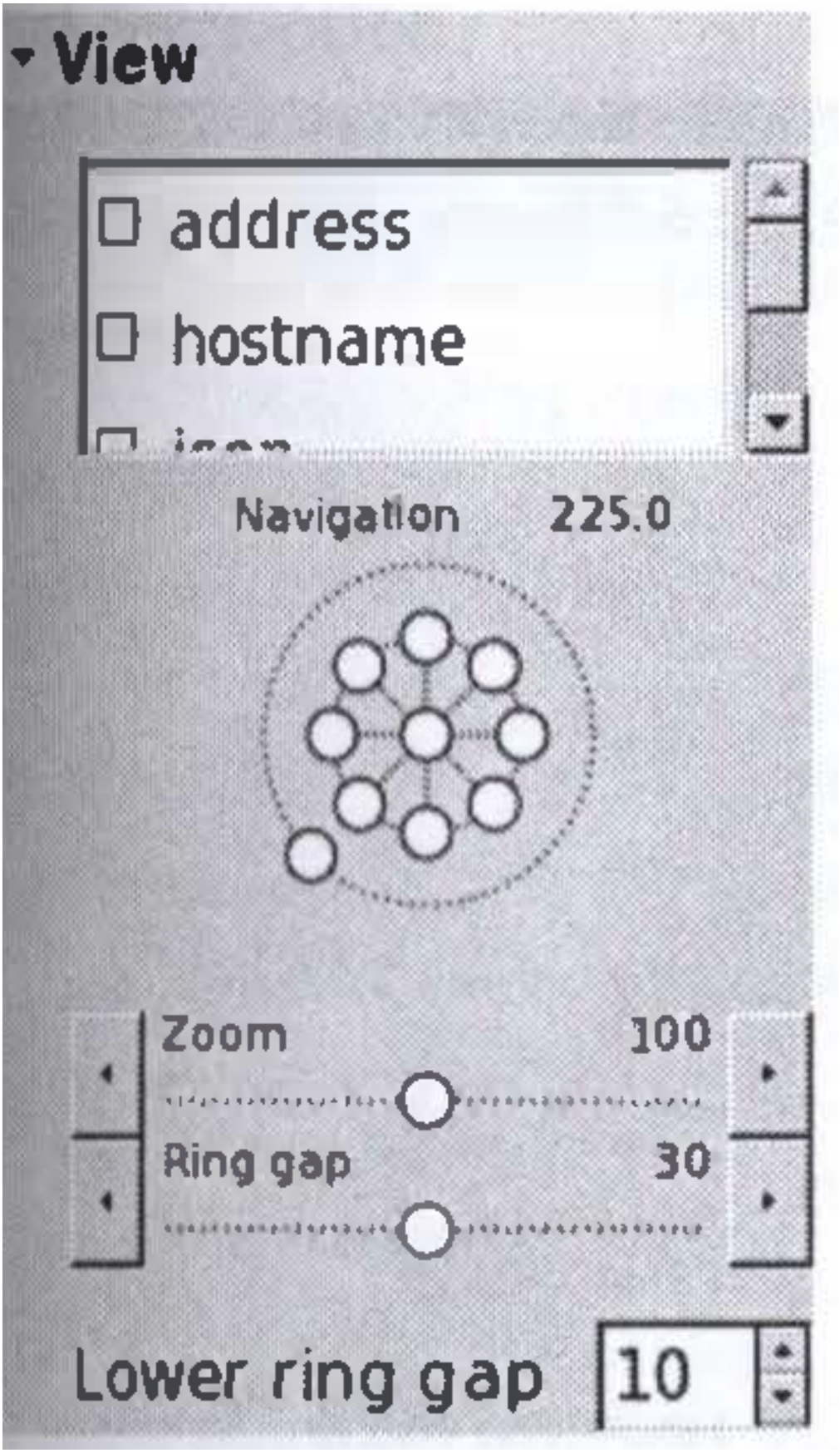
Элементы управления в секции «Interpolation» определяют скорость анимации при изменении части графа.

Элементы управления компоновкой



Для автоматической компоновки узлов предусмотрены два варианта. Симметричный режим отводит каждому поддереву узла одинаковую по размеру долю графа. Он хорошо показывает иерархию сети, но удалённые от центра узлы могут оказаться тесно прижатыми друг к другу. Взвешенный режим отводит узлам с большим количеством дочерних узлов большую часть графа.

Элементы управления отображением



Флажки в секции «View» включают и отключают части изображения. Например, отключите «hostname», чтобы для каждого узла показывался только IP-адрес, или отключите «address», чтобы вообще не использовать подписи. Параметр «latency» включает и отключает показ времени прохождения до каждого узла и обратно, определённого параметром Nmap `--traceroute`. Если

установлен флажок «slow in/out», анимация будет нелинейной: быстрее в середине и медленнее в начале и конце.

Похожий на компас элемент перемещает экран в восьми направлениях. Щёлкните по центру, чтобы вернуться к центральному узлу. Внешнее кольцо управляет вращением всего графа.

«Zoom» и «Ring gap» управляют общим размером графа. «Zoom» изменяет размер всего: узлов, подписей, соединительных линий. «Ring gap» лишь увеличивает расстояние между концентрическими окружностями, сохраняя прежний размер всего остального. «Lower ring gap» задаёт минимальный интервал между окружностями и полезен главным образом при включённом эффекте «рыбий глаз».

Элементы управления эффектом «рыбий глаз»

Элементы управления эффектом «рыбий глаз» отводят больше места выбранной окружности, сжимая все остальные. Ползунок задаёт, какой окружности уделяется больше всего внимания. «Interest factor» показывает, во сколько раз интервал для выбранной окружности больше, чем был бы без эффекта «рыбий глаз». Значение «Spread factor» находится в диапазоне от -1 до 1. Оно определяет, сколько соседних окружностей расширяется вокруг выбранной; более высокие значения означают больший охват.

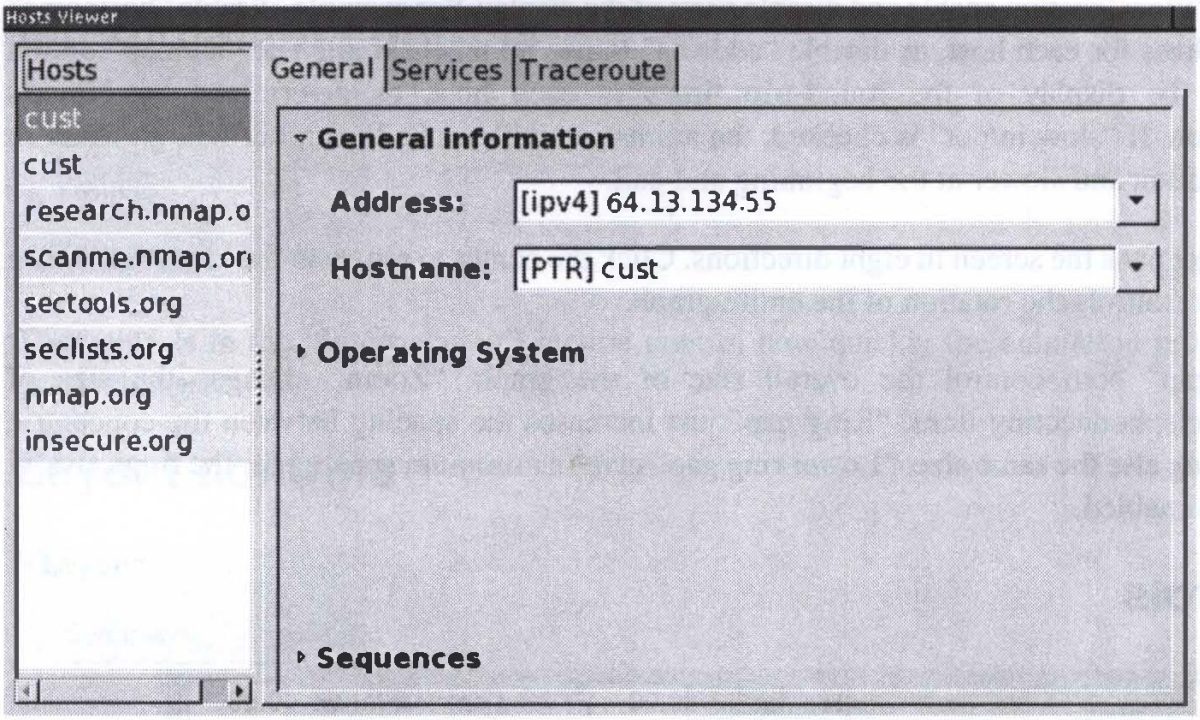
12.5.4. Сочетания клавиш

Представление топологии распознаёт следующие клавиши:

Клавиша — функция

- `c` — Вернуть изображение к центральному узлу.
- `a` — Показать или скрыть адреса узлов.
- `h` — Показать или скрыть имена узлов.
- `i` — Показать или скрыть значки узлов.
- `l` — Показать или скрыть задержку.
- `r` — Показать или скрыть окружности.

12.5.5. Средство просмотра узлов

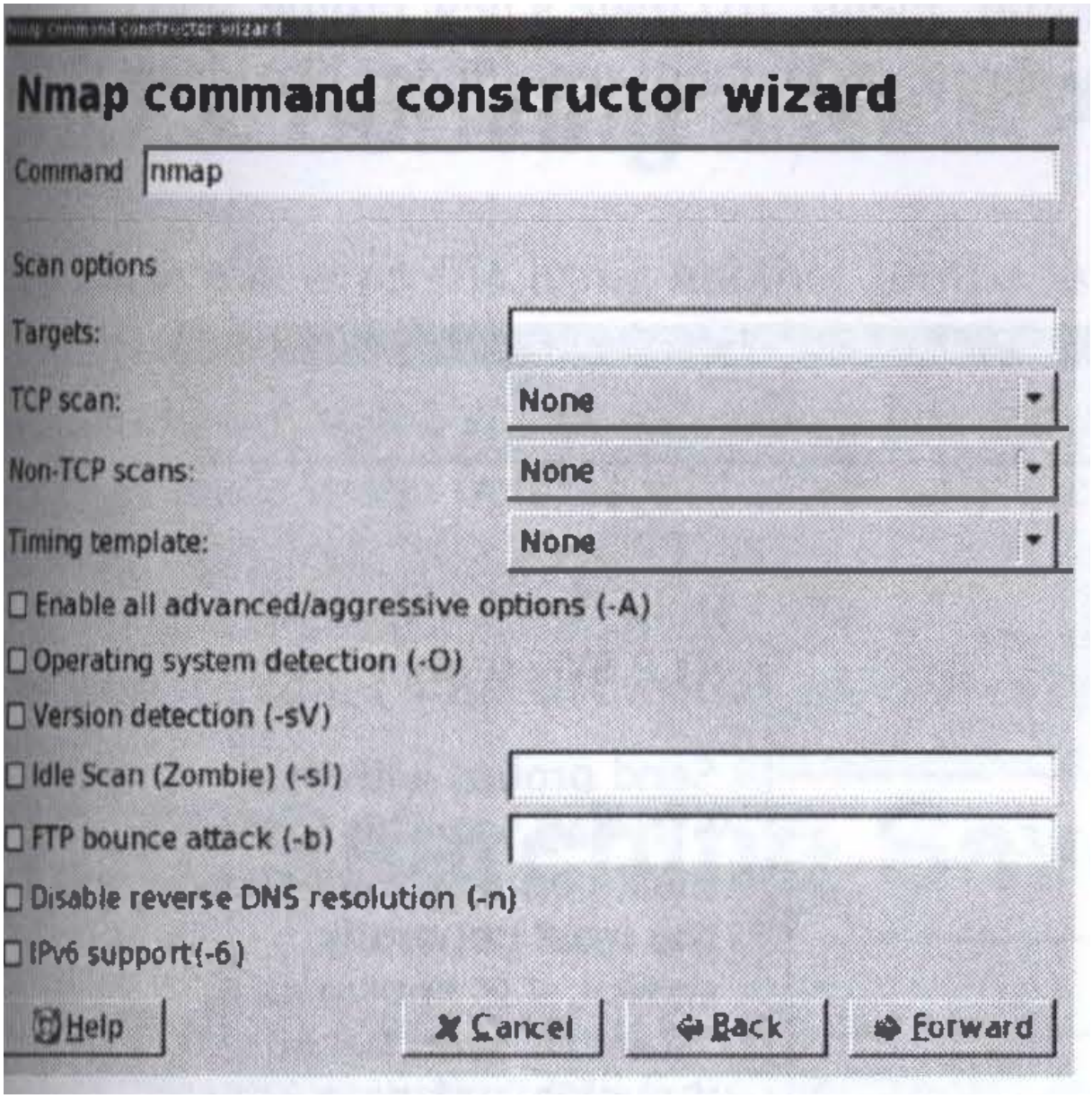


Средство просмотра узлов - альтернативный способ получать подробные сведения об узлах. Включите его, нажав кнопку «Hosts Viewer». Все узлы инвентаризации будут представлены списком. Выберите любой узел, чтобы получить подробные сведения о нём.

12.6. Мастер-конструктор команд Nmap

Мастер-конструктор команд Nmap позволяет интерактивно создавать командные строки Nmap без необходимости помнить, например, что `-sS` означает «SYN-сканирование». Запустите мастер, выбрав «Command Wizard» в меню «Tools» или нажав сочетание клавиш `Ctrl+I`. Появится начальная страница мастера.

Вы должны решить, хотите ли вы сохранить описание сканирования как профиль, чтобы запускать его снова, или просто создать команду и выполнить её один раз. Если вы решите создать профиль, вам предложат ввести имя и описание профиля.



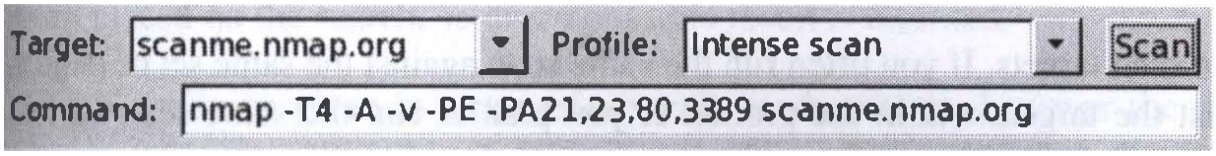
Затем вам будет предложена последовательность страниц, интерактивно запрашивающих параметры Nmap. Выбор пунктов меню или установка флажков изменяют команду в соответствии с выбором. Например, выбор типа сканирования «TCP SYN Scan» добавит в командную строку `-sS`. Установка флажка «Operating system detection» добавит в команду `-O`, а снятие флажка снова его удалит.

Когда дойдёте до конца мастера, нажмите «Apply». Если вы решили создать новый профиль, он будет создан и станет доступен в раскрывающемся списке профилей. Если вы решили создать команду и выполнить её один раз, её выполнение начнётся немедленно.

12.7. Редактор профилей

При работе с Nmap часто требуется многократно запускать одно и то же сканирование. Например, системный администратор может раз в месяц сканировать всю сеть, чтобы следить за происходящим. Механизм Zenmap, упрощающий это, называется профилями.

Рисунок 12.9. Выбор профиля

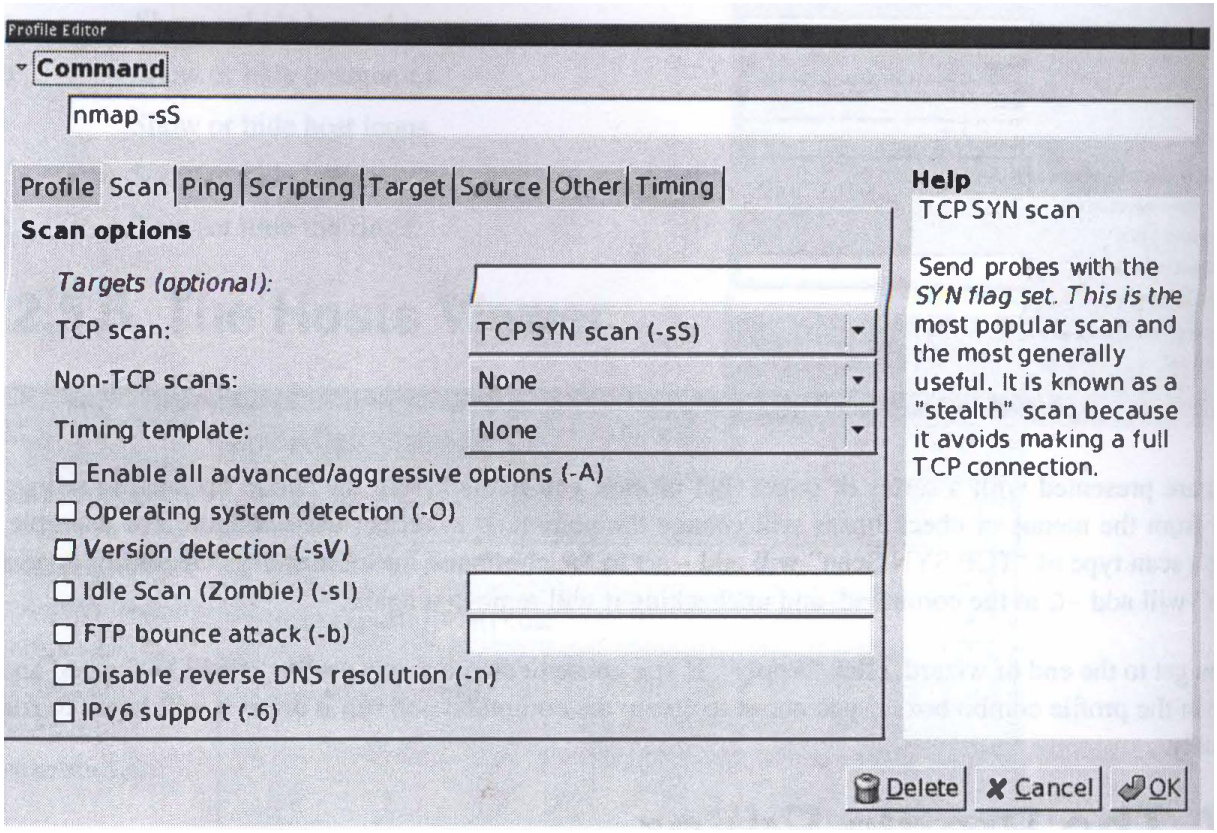


В каждом окне имеется раскрывающийся список с надписью «Profile». При его открытии показываются доступные профили. Выбор профиля приводит к тому, что в поле «Command» отображается командная строка, которая будет выполнена. Профилей, поставляемых с Zenmap, достаточно для многих задач сканирования, но рано или поздно вы захотите создать собственный.

12.7.1. Создание нового профиля

Команды для работы с профилями находятся в меню «Profile». Чтобы создать новый профиль, выберите «New Profile» в меню «Profile» или воспользуйтесь сочетанием клавиш `Ctrl+P`. Вы увидите диалоговое окно, подобное показанному на рисунке 12.10.

Рисунок 12.10. Редактор профилей



Сначала редактор профилей отображает вкладку «Profile», запрашивающую имя и описание нового профиля. Поле «Profile name» определяет, как сканирование будет обозначаться в раскрывающемся списке интерфейса сканирования. Текст в поле «Description» описывает назначение профиля.

Остальные вкладки позволяют задавать параметры Nmap либо непосредственно вводя их в поле «Command», либо устанавливая флажки. Наведите указатель мыши на параметр, чтобы получить описание его действия и ожидаемого типа входных данных.

Профиль может содержать или не содержать цели сканирования. Если вы часто запускаете одно и то же сканирование для одного и того же набора целей, удобно перечислить цели в профиле. Если вы намерены запускать одно и то же сканирование для разных целей, оставьте поле «Targets»

пустым и заполните цели позднее, при запуске сканирования.

12.7.2. Редактирование профиля

Чтобы отредактировать профиль, выберите нужный профиль, затем выберите «Edit Selected Profile» в меню «Profile» или воспользуйтесь сочетанием клавиш `Ctrl+E`. Откроется редактор профилей, на этот раз с именем и описанием выбранного профиля. Любое внесённое здесь изменение изменит профиль навсегда.

Чтобы удалить профиль, нажмите кнопку «Delete» в редакторе профилей, предварительно открыв профиль, который хотите удалить, так же, как для редактирования. Перед удалением профиля Zenmap покажет предупреждение. Чтобы выйти из редактора, не изменяя профиль, воспользуйтесь кнопкой «Cancel».

12.7.3. Создание нового профиля на основе старого

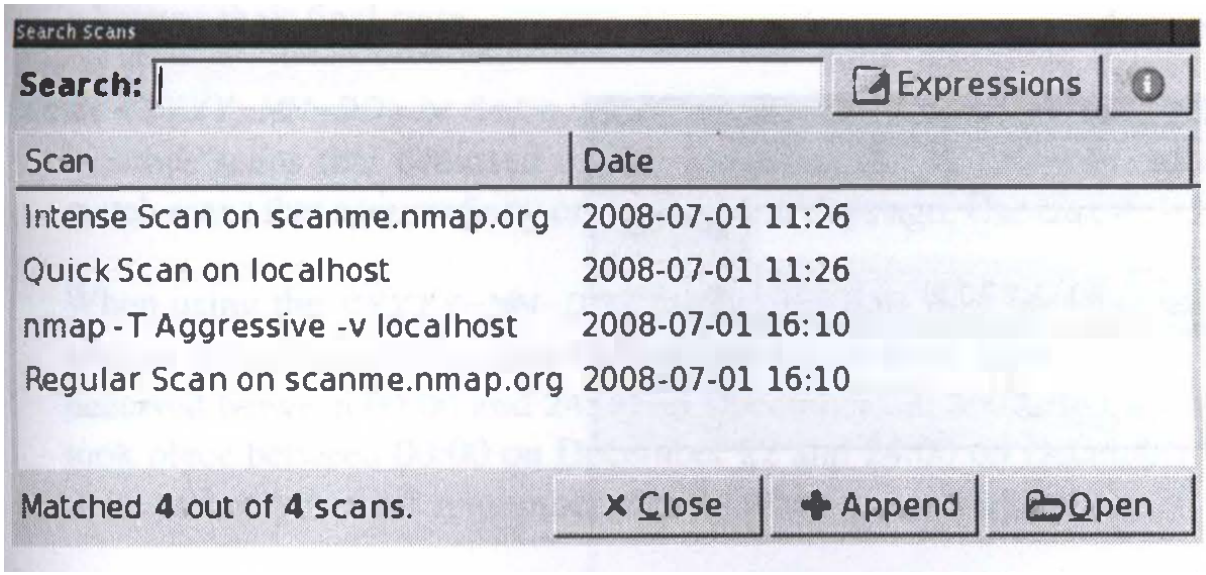
Чтобы создать новый профиль, используя другой профиль как шаблон, выберите профиль-шаблон, затем выберите «New Profile with Selected» в меню «Profile» или воспользуйтесь сочетанием клавиш `Ctrl+R`. Все параметры будут заданы на основе выбранного профиля, а имя и описание останутся пустыми, чтобы вы их заполнили. Любые изменения параметров повлияют только на вновь созданный профиль, но не на исходный профиль, из которого он был получен.

Чтобы выйти из редактора, не создавая производный профиль, воспользуйтесь кнопкой «Cancel».

12.8. Поиск сохранённых результатов

Zenmap позволяет искать в сохранённых файлах результатов сканирования и базе данных недавних сканирований. Чтобы начать поиск, выберите «Search Scan Results» в меню «Tools» или воспользуйтесь сочетанием клавиш `Ctrl+F`. Появится диалоговое окно поиска, показанное на рисунке 12.11.

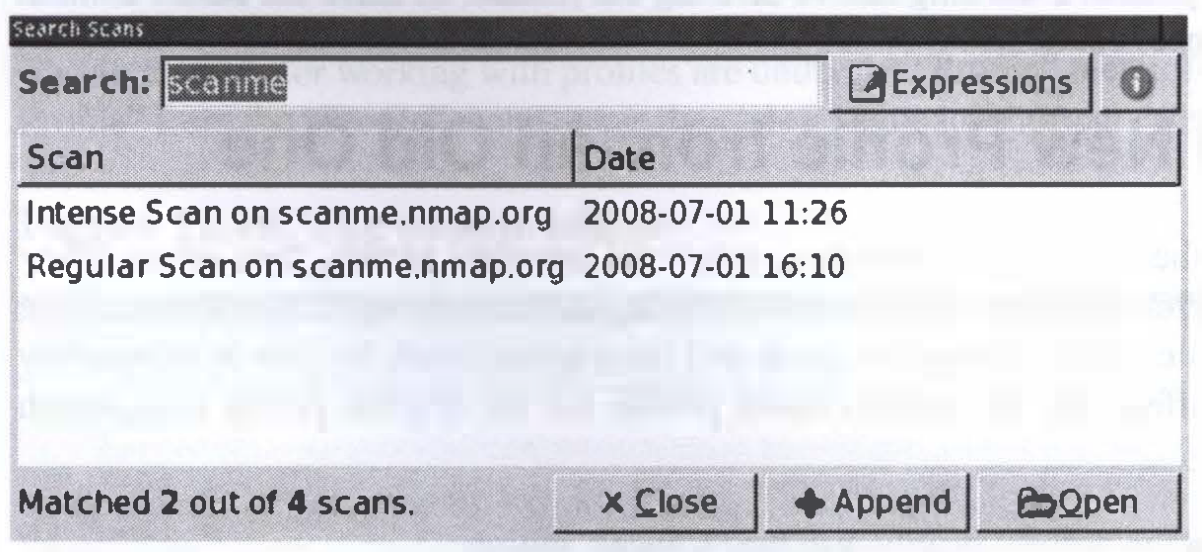
Рисунок 12.11. Диалоговое окно поиска



Изначально интерфейс поиска показывает все сканирования из базы данных недавних сканирований (о ней см. раздел 12.4.1 «База данных недавних сканирований»). Причина, по которой показываются все сканирования, проста: на поиск ещё не наложено никаких ограничений, поэтому возвращается каждый возможный результат.

При поиске можно задавать несколько критериев, однако простейший поиск - это поиск по ключевому слову. Просто введите в поле «Search» слово наподобие `scanme`, чтобы найти все сканирования, в выводе которых это слово встречается как имя узла, название операционной системы, профиль или что-либо ещё. Пример показан на рисунке 12.12.

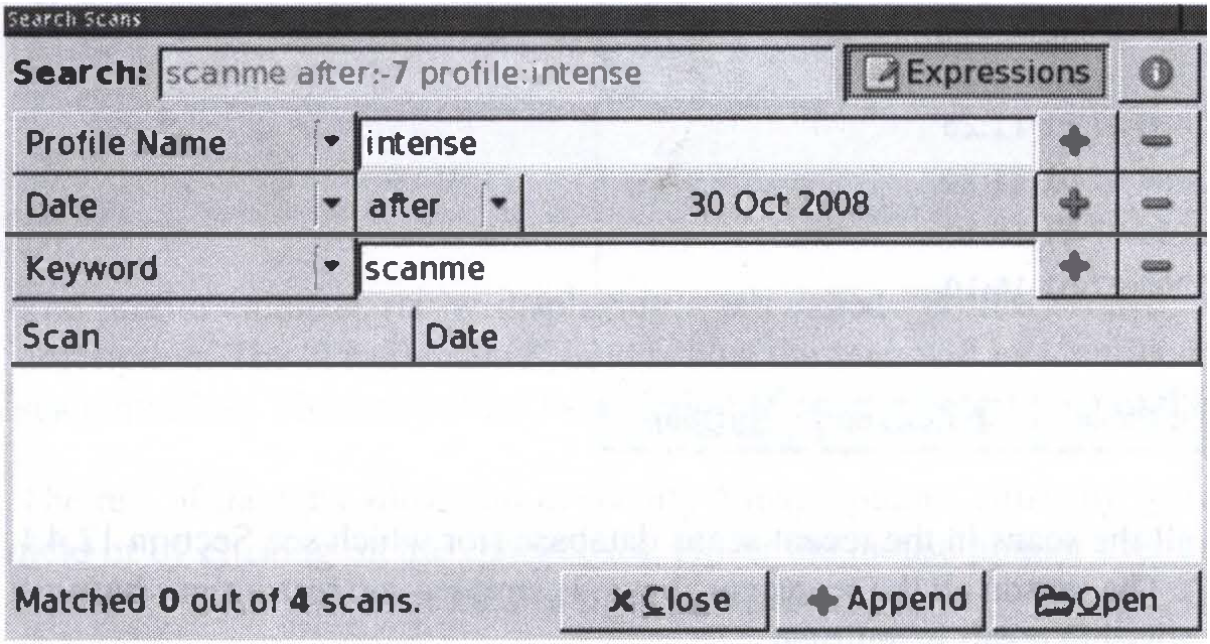
Рисунок 12.12. Поиск по ключевому слову



Поиск выполняется в реальном времени, по мере ввода. Найдя нужное сканирование, нажмите кнопку «Open» или дважды щёлкните имя сканирования.

Более сложные поисковые запросы можно строить с помощью интерфейса «Expressions». Нажмите кнопку «Expressions», и появится графическое представление текущего поиска. Измените поиск, выбирая значения в появившихся раскрывающихся списках. Нажмите «+», чтобы добавить критерий, и «-», чтобы удалить его. Снова нажмите кнопку «Expressions», чтобы скрыть критерии (они останутся в строке поиска). Пока выражения показаны, редактирование текста поискового запроса отключено. Пример более сложного поиска показан на рисунке 12.13.

Рисунок 12.13. Поиск с выражениями



Поисковые запросы объединяются операцией И: чтобы сканирование соответствовало запросу и появилось в списке результатов, истинными должны быть все критерии. Большинство поисковых запросов не учитывает регистр. (Единственный чувствительный к регистру критерий - `option:`.) По умолчанию поиск выполняется только по сканированиям из базы данных недавних сканирований. Для рекурсивного поиска файлов в каталоге используйте выражение «Include Directory».

Вы, вероятно, заметили, что при каждом выборе поискового выражения в поле поиска появляется его текстовое представление. В действительности поиском управляет именно строка в поле «Search»; интерфейс «Expressions» - лишь удобный способ её задавать. Узнав, какие поисковые строки соответствуют каким выражениям, можно пропустить интерфейс выражений и просто ввести строку поиска непосредственно.

Ниже приведён список всех текстовых критериев поиска, распознаваемых интерфейсом поиска. У большинства критериев есть краткая форма: `d:-5` равнозначно `date:-5`, а `op:80` - `open:80`. Краткая форма каждого критерия приведена в списке ниже.

• `<keyword>`. Слово без дополнительных обозначений соответствует чему угодно в сканировании. Например, `apache` найдёт все серверы Apache, а `linux` - все узлы Linux. При поиске по ключевому слову возможны ложные срабатывания, например если какой-либо узел случайно называется `apache` или `linux`.

• **Состояния портов.** Каждое возможное состояние порта также является критерием поиска. Вот эти критерии:

```
open:<ports>          (op: for short)
closed:<ports>        (cp: for short)
filtered:<ports>      (fp: for short)
unfiltered:<ports>    (ufp: for short)
open|filtered:<ports> (ofp: for short)
closed|filtered:<ports> (cfp: for short)
```

Используйте `open:80`, чтобы найти сканирования, в которых есть узел с открытым портом 80. Аргумент `<ports>` также может быть списком, разделённым запятыми. Кроме того, критерий `scanned:` (кратко `sp:`) находит сканирования, в которых сканировались указанные порты, независимо от их конечного состояния.

• `date:<YYYY-MM-DD>` **или** `date:-<n>` **(кратко** `d:`**).** Находит сканирования, выполненные в указанную дату в формате `<YYYY-MM-DD>`. Или используйте `date:-<n>`, чтобы найти сканирования,

выполненные в день <n> дней назад. Используйте date:-1, чтобы найти сканирования, выполненные вчера. При использовании формата <YYYY-MM-DD> после даты можно поставить один или несколько знаков ~, каждый из которых расширяет диапазон совпадающих дат на один день в обе стороны. date:2007-12-23 находит сканирования, выполненные с 00:00 до 24:00 23 декабря 2007 года. date:2007-12-23~ находит сканирования, выполненные с 00:00 22 декабря до 24:00 24 декабря. Такое «нечёткое» сопоставление дат полезно, когда вы не можете точно вспомнить, когда запускали сканирование.

• after:<YYYY-MM-DD> **или** after:-<n> **(кратко** a:**).** Находит сканирования, выполненные в указанную дату в формате <YYYY-MM-DD> или после неё. Или используйте after:-<n>, чтобы найти сканирования, выполненные за последние <n> дней. Например, after:-7 находит сканирования, выполненные за последнюю неделю.

• before:<YYYY-MM-DD> **или** before:-<n> **(кратко** b:**).** Находит сканирования, выполненные в указанную дату в формате <YYYY-MM-DD> или до неё. Или используйте before:-<n>, чтобы найти сканирования, выполненные в любое время ранее, чем <n> дней назад.

• target:<name> **(кратко** t:**).** Находит сканирования любых узлов с указанным именем. Это может быть либо имя, заданное при сканировании, либо имя обратного DNS любого узла.

• option:<option> **(кратко** o:**).** Находит сканирования, в которых использовался указанный параметр командной строки. Опускайте начальный - или --: option:A находит сканирования, в которых использовался параметр -A. Этот критерий ищет только буквальное совпадение. option:O не найдёт сканирования, в которых использовался -A, хотя -A подразумевает -O. Аналогично option:sU не найдёт сканирования, в которых использовался -sSU. Сопоставление параметров чувствительно к регистру.

• os:<string>**.** Находит сканирования узлов, в любой части описания ОС которых встречается указанная строка. os:windows в широком смысле вернёт сканирования узлов Microsoft Windows.

• service:<string> **(кратко** s:**).** Находит сканирования узлов, в любой части описания службы любого порта которых встречается указанная строка. service:ssh вернёт сканирования узлов, на которых работает SSH любого типа.

• profile:<name> **(кратко** pr:**).** Находит сканирования, в которых использовался профиль с указанным именем, например profile:"intense scan".

• inroute:<host> **(кратко** ir:**).** Находит сканирования, в выводе --traceroute которых указанный узел встречается как промежуточный маршрутизатор.

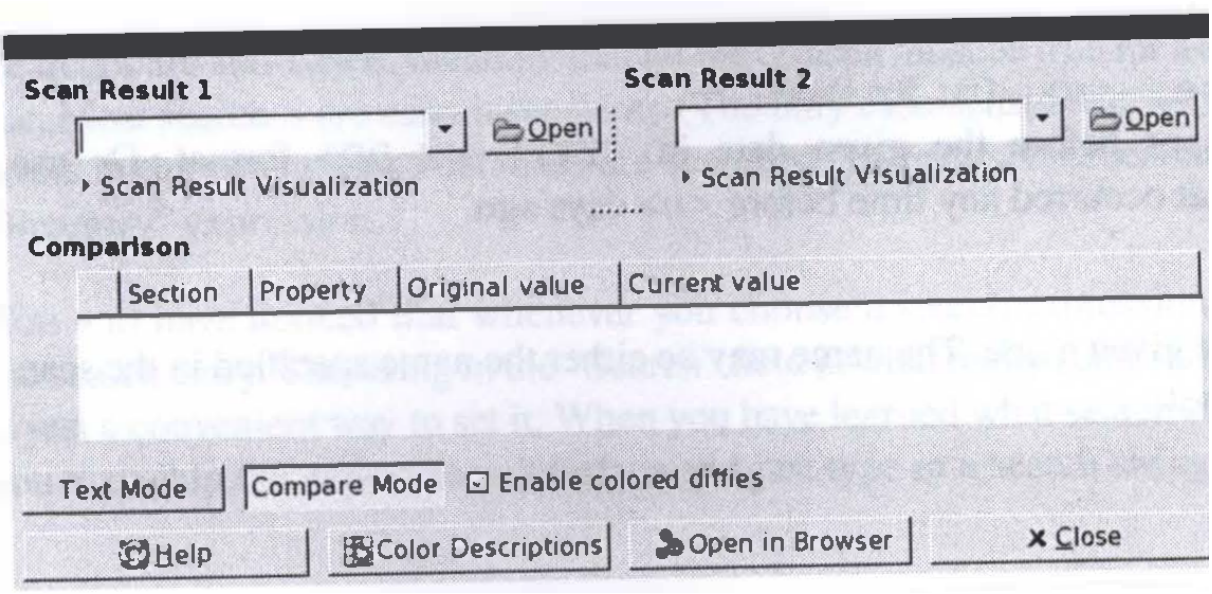
• dir:<directory>**.** dir: в действительности не является критерием поиска. Это способ выполнять поиск в каталоге файловой системы в дополнение к поиску в базе данных недавних сканирований. Каталоги рекурсивно просматриваются в поисках файлов с определёнными расширениями, по умолчанию только xml. Чтобы находить больше имён файлов, измените переменную file_extension в разделе [search] файла zenmap.conf согласно указаниям раздела 12.11.1 «Разделы zenmap.conf».

12.9. Сравнение результатов

Часто требуется дважды запустить одно и то же сканирование в разное время либо одновременно запустить два слегка различающихся сканирования и посмотреть, чем они различаются. Zenmap предоставляет интерфейс для сравнения результатов сканирования, показанный на рисунке 12.14. Откройте инструмент сравнения, выбрав «Compare Results» в меню «Tools» или воспользовавшись сочетанием клавиш Ctrl+D (вспомните diff). Zenmap поддерживает сравнение двух результатов

сканирования за один раз.

Рисунок 12.14. Инструмент сравнения



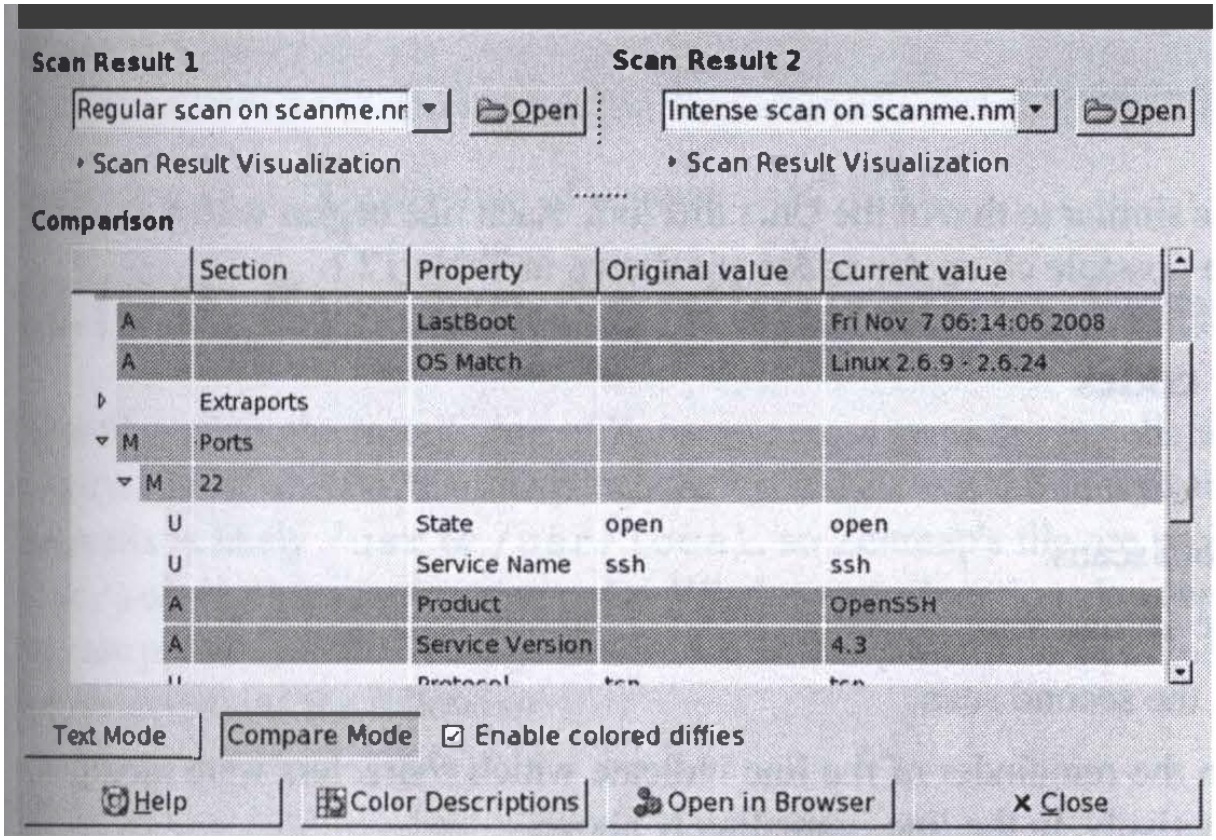
Первый шаг при выполнении сравнения - выбор двух сравниваемых сканирований. Раскрывающиеся списки под надписями «Scan Result 1» и «Scan Result 2» позволяют выбирать из открытых сканирований. Либо нажмите кнопки «Open», чтобы получить результаты сканирования из файла. Чтобы сравнить результаты из базы данных недавних сканирований, сначала необходимо открыть эти сканирования с помощью интерфейса поиска (см. раздел 12.8 «Поиск сохранённых результатов»).

Различие между Scan Result 1 и Scan Result 2 существенно. Сравнение всегда выполняется от Scan Result 1 к Scan Result 2, то есть показывает, чем Scan Result 2 отличается от Scan Result 1. Как только выбраны оба результата, сравнение выполняется немедленно.

12.9.1. Графическое сравнение

На рисунке 12.15 показано сравнение обычного и интенсивного сканирования одного и того же узла.

Рисунок 12.15. Графическое сравнение

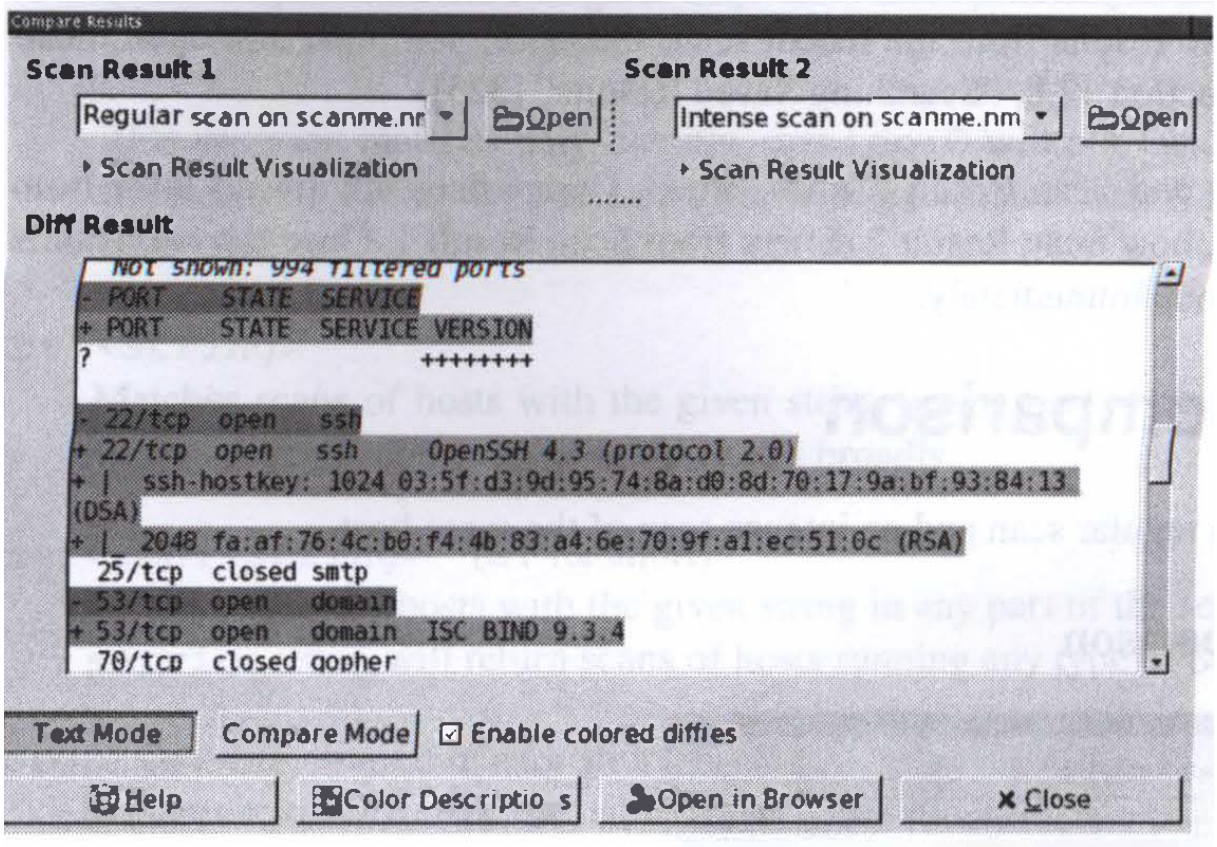


Различия и сходства двух сканирований показываются иерархически и с помощью цветов. У каждого цвета также есть буквенный код, описывающий, как изменилась (или не изменилась) эта часть сканирования. Коды таковы: `U` - без изменений, `A` - добавлено, `M` - изменено, `N` - отсутствует (или удалено). Цвета можно изменить, нажав кнопку «Color Descriptions».

12.9.2. Текстовое сравнение

Альтернативным представлением сравнения является текстовый режим, включаемый кнопкой «Text Mode». Текстовое сравнение тех же двух сканирований показано на рисунке 12.16. Преимущество вывода в текстовом режиме состоит в том, что его можно скопировать и вставить в файл или сообщение электронной почты.

Рисунок 12.16. Сравнение в текстовом режиме



Вывод сравнения в текстовом режиме подобен выводу Unix-инструмента `diff`. Каждая строка начинается с символа, указывающего её смысл. Возможные коды символов приведены в таблице 12.1.

Таблица 12.1. Коды символов текстового diff

Код — значение

• **(пробел)** — Строка одинакова в обоих сканированиях.
• + — Строка добавлена во втором сканировании.
• - — Строка удалена во втором сканировании.
• ? — Символы ^, + и - в остальной части строки указывают, какие символы соответственно изменены, добавлены или удалены в расположенной непосредственно выше строке.

HTML-представление текстовых различий можно просмотреть, нажав кнопку «Open in Browser». Это представление предназначено для сохранения в архивных целях или печати для отчёта.

12.10. Файлы, используемые Zenmap

Zenmap использует ряд конфигурационных и управляющих файлов и, разумеется, требует установленного Nmap. Место хранения файлов зависит от платформы и конфигурации Zenmap. Конфигурационные файлы делятся на две категории: системные и пользовательские.

12.10.1. Исполняемый файл nmap

Zenmap зависит от установленного исполняемого файла командной строки `nmap`. Сначала программа ищется во всех каталогах, указанных в переменной окружения `PATH`.

На некоторых платформах команда nmap обычно не устанавливается ни в один из каталогов PATH. Для удобства на этих платформах, если команда не найдена в PATH, дополнительно просматриваются следующие каталоги:

• В Mac OS X просматривается каталог /usr/local/bin.
• В Windows просматривается каталог, содержащий исполняемый файл Zenmap.

Чтобы использовать абсолютный путь к исполняемому файлу или если исполняемый файл установлен под именем, отличным от nmap, измените переменную nmap_command_path в разделе [paths] файла zenmap.conf. Например, если nmap установлен в /opt/bin, используйте:

```
[paths]
nmap_command_path = /opt/bin/nmap
```

А если у вас имеется самостоятельно скомпилированная версия Nmap с именем nmap-custom, используйте:

```
[paths]
nmap_command_path = nmap-custom
```

См. раздел 12.11 «Описание zenmap.conf».

12.10.2. Системные конфигурационные файлы

Эти файлы влияют на установку Zenmap в целом. В Unix и Mac OS X они находятся в <prefix>/share/zenmap, где <prefix> - префикс файловой системы, с которым был скомпилирован Zenmap. Скорее всего, префикс имеет значение /usr или /usr/local, поэтому файлы Zenmap, вероятно, находятся в /usr/share/zenmap или /usr/local/share/zenmap. В Windows расположение также зависит от места установки Zenmap. Вероятно, они находятся в C:\Program Files\Nmap\share\zenmap. Системный конфигурационный каталог Zenmap содержит следующее:

• config/. Файлы в config копируются в пользовательские конфигурационные каталоги. См. раздел 12.10.3 «Пользовательские конфигурационные файлы».
• docs/. Файлы в подкаталоге docs являются файлами документации Zenmap.
• locale/. Подкаталог содержит переводы текста, используемого Zenmap, на другие языки.
• misc/profile_editor.xml. Этот файл определяет, какие параметры представлены в редакторе профилей (см. раздел 12.7 «Редактор профилей»). Его можно с осторожностью редактировать, чтобы изменить редактор профилей для всей системы.
• misc/wizard.xml. Этот файл определяет, какие параметры представлены в мастере-конструкторе команд (см. раздел 12.6 «Мастер-конструктор команд Nmap»). Его можно с осторожностью редактировать, чтобы изменить мастер для всей системы.

12.10.3. Пользовательские конфигурационные файлы

Эти файлы влияют только на одного пользователя Zenmap. Некоторые из них копируются из подкаталога config системных файлов при первом запуске Zenmap. В Unix и Mac OS X пользовательские файлы находятся в <HOME>/.zenmap, где <HOME> - домашний каталог текущего пользователя. В Windows Vista они находятся в C:\Users\<USER>.zenmap, а в предыдущих версиях Windows - в C:\Documents and Settings\<USER>.zenmap, где <USER> - имя текущего пользователя.

• recent_scans.txt. Содержит список имён файлов недавно сохранённых сканирований. Эти сканирования показываются в меню «Scan». Чтобы сканирование появилось здесь, оно должно быть сохранено в файл. См. раздел 12.4 «Сохранение и загрузка результатов сканирования». Если

этого файла не существует, он создаётся при запуске Zenmap.
• scan_profile.usp. Этот файл содержит описания профилей сканирования, включая профили по умолчанию и созданные пользователем. Для внесения изменений в этот файл рекомендуется использовать редактор профилей (см. раздел 12.7 «Редактор профилей»). Этот файл копируется из системного конфигурационного каталога при первом запуске Zenmap.
• target_list.txt. Этот файл содержит список недавно сканировавшихся целей. Если он не существует, то создаётся при запуске Zenmap.
• zenmap.conf. Это основной конфигурационный файл Zenmap. Он содержит настройки экземпляра Zenmap конкретного пользователя и подробнее рассматривается в разделе 12.11 «Описание zenmap.conf».
• zenmap.db. Это база данных недавних сканирований, описанная в разделе 12.4.1 «База данных недавних сканирований». Если она ещё не существует, то создаётся.
• zenmap_version. Этот файл содержит версию Zenmap, с помощью которой был создан данный пользовательский конфигурационный каталог. Если вы подозреваете конфликт версий, может быть полезно сравнить номер версии в этом файле с одноимённым файлом в системном конфигурационном каталоге. При первом запуске Zenmap он просто копируется из системного конфигурационного каталога.

12.10.4. Файлы вывода

При каждом запуске сканирования Zenmap указывает Nmap поместить вывод XML во временный файл, чтобы Zenmap мог его разобрать. Обычно файл вывода XML удаляется после завершения сканирования. Однако если командная строка в Zenmap содержит параметр -oX или -oA, вывод XML записывается вместо этого в файл с указанным именем, и этот файл после завершения сканирования не удаляется. Иными словами, -oX и -oA работают так, как и следовало ожидать. -oG, -oN и -oS тоже работают, хотя Zenmap не использует создаваемые этими параметрами файлы вывода.

При обработке Zenmap этих имён файлов необходимо отметить одну важную особенность. Знаки процента (%) экранируются, чтобы они не интерпретировались как спецификаторы формата в стиле strftime (см. раздел 13.2.1 «Управление типом вывода»). Причина в том, что Zenmap должен точно знать, какое имя Nmap присвоит файлу вывода. Если в Zenmap ввести -oX scan-%T-%D.xml, вывод будет сохранён в файле scan-%T-%D.xml, а не scan-144840-121307.xml или в файле с другим именем, которое было бы основано на текущих времени и дате при непосредственном выполнении Nmap.

12.11. Описание zenmap.conf

zenmap.conf - пользовательский конфигурационный файл Zenmap. Это обычный текстовый файл, расположенный в пользовательском конфигурационном каталоге (см. раздел 12.10.3 «Пользовательские конфигурационные файлы»). Его синтаксис распознаётся модулем Python ConfigParser[2] и подобен синтаксису INI-файлов Windows. Разделы отделяются заголовками в квадратных скобках. Внутри разделов находятся строки, содержащие пары <name>=<value> или <name>: <value>. Ниже приведён фрагмент zenmap.conf.

```
[output_highlight]
enable_highlight = True

[paths]
nmap_command_path = nmap
[search]
```

```
search_db = 1
file_extension = xml
store_results = 1
directory =
save_time = 60;days
```

Некоторыми из этих настроек можно управлять из Zenmap, не редактируя конфигурационный файл непосредственно.

12.11.1. Разделы zenmap.conf

Логические значения `True`, `true` или `1` нормализуются в истину, а всё остальное - в ложь.

• `[paths]`. Раздел `[paths]` определяет важные пути, используемые Zenmap. В нём определена только одна переменная: `nmap_command_path`, путь к исполняемому файлу Nmap. Первое слово любой командной строки, выполняемой Zenmap, заменяется значением этой переменной. Значение по умолчанию `nmap` подходит для большинства систем. Примеры см. в разделе 12.10.1 «Исполняемый файл nmap».
• `[search]`. Раздел `[search]` определяет поведение средства поиска (см. раздел 12.8 «Поиск сохранённых результатов»). Имена в этом разделе соответствуют параметрам на вкладке «Search options» диалогового окна поиска. В нём определены следующие имена.
• `directory`. Каталог, в котором следует искать сохранённые файлы результатов сканирования.
• `file_extension`. Разделённый точками с запятой список расширений имён файлов, которые следует искать.

• `search_db`. Логическое значение, определяющее, следует ли искать в базе данных недавних сканирований.

• `store_results`. Логическое значение, определяющее, следует ли хранить результаты сканирования в базе данных недавних сканирований. См. раздел 12.4.1 «База данных недавних сканирований».

• `save_time`. Срок хранения результатов сканирования в базе данных недавних сканирований. Результаты старше этого срока удаляются при закрытии Zenmap. Формат: число и единица времени, разделённые точкой с запятой, например `60;days` или `1;years`.

• `[diff]`. Раздел `[diff]` определяет поведение инструмента сравнения (см. раздел 12.9 «Сравнение результатов»). В нём определены следующие имена.

• `diff_mode`. Определяет, показываются ли сравнения по умолчанию в графическом или текстовом режиме. Значением должно быть либо `compare` для графического режима, либо `text`.

• `colored_diff`. Логическое значение, определяющее, используются ли при сравнении цвета.

• `[diff_colors]`. Раздел `[diff_colors]` определяет цвета, используемые инструментом сравнения. В нём определены следующие имена: `unchanged`, `added`, `not_present` и `modified`; их значения определены в разделе 12.9 «Сравнение результатов». Значение каждого из них - список из трёх целых чисел в диапазоне 0-65535, представляющих красный, зелёный и синий цвета в формате `[<red>, <green>, <blue>]`. Например, `[65535, 0, 0]` задаёт красный цвет.

• `[output_highlight]`. Раздел `[output_highlight]` содержит одну логическую переменную `enable_highlight`, которая включает подсветку вывода при значении `True` и отключает её при значении `False`.

• `[date_highlight]`, `[hostname_highlight]`, `[ip_highlight]`, `[port_list_highlight]`, `[open_port_highlight]`, `[closed_port_highlight]`, `[filtered_port_highlight]`, `[details_highlight]`. Все эти разделы определяют характер подсветки вывода Nmap,

рассматриваемой в разделе «Вкладка Nmap Output». Их лучше всего редактировать из Zenmap. В каждом из этих разделов определены следующие имена.

• `regex`. Регулярное выражение, соответствующее нужной части вывода.

• `bold`. Логическое значение, определяющее, следует ли выделять эту подсветку полужирным шрифтом.

• `italic`. Логическое значение, определяющее, следует ли выделять эту подсветку курсивом.

• `underline`. Логическое значение, определяющее, следует ли подчёркивать эту подсветку.

• `text`. Цвет текста этой подсветки. Синтаксис - список из трёх целых чисел в диапазоне 0-65535, представляющих красный, зелёный и синий цвета в формате `[<red>, <green>, <blue>]`. Например, `[65535, 0, 0]` задаёт красную подсветку.

• `highlight`. Цвет фона этой подсветки. Синтаксис такой же, как у `text`.

12.12. Параметры командной строки

Поскольку Zenmap является графическим приложением, большая часть его возможностей доступна через графический интерфейс. Параметры командной строки Zenmap приведены здесь для полноты и потому, что иногда они полезны. В частности, полезно знать, что команда `zenmap <results file>` запускает Zenmap с уже открытыми результатами из `<results file>`.

12.12.1. Синтаксис вызова

```
zenmap [<options>] [<results file>]
```

12.12.2. Сводка параметров

• `-f, --file <results file>`. Открывает указанный файл результатов для просмотра. Файл результатов может быть файлом вывода Nmap XML (`.xml`, созданным `nmap -oX`) или файлом, ранее сохранённым Zenmap.
• `-h, --help`. Показывает справочное сообщение и завершает работу.
• `-n, --nmap <Nmap command line>`. Выполняет указанную команду Nmap в интерфейсе Zenmap. После `-n` или `--nmap` каждый оставшийся аргумент командной строки читается как часть выполняемой команды. Это означает, что `-n` или `--nmap` необходимо указывать последним, после всех остальных параметров. Обратите внимание: командная строка должна включать имя исполняемого файла `nmap`: `zenmap -n nmap -sS target`.
• `-p, --profile <profile>`. Запускает программу с выбранным указанным профилем. Имя профиля - просто строка: `"Regular scan"`. В сочетании с `-t` начинает сканирование указанной цели с указанным профилем.
• `-t, --target <target>`. Запускает программу с указанной целью. В сочетании с `-p` начинает сканирование указанной цели с указанным профилем.
• `-v, --verbose`. Увеличивает подробность вывода (Zenmap, не Nmap). Этот параметр можно указывать несколько раз, чтобы получать ещё более подробный вывод в окне консоли, из которого запущен Zenmap.

12.12.3. Вывод ошибок

Если Zenmap аварийно завершает работу, обычно он помогает отправить отчёт об ошибке с трассировкой стека. Установите переменную окружения `ZENMAP_DEVELOPMENT` (её значение не имеет значения), чтобы отключить автоматическую отправку отчётов об авариях и выводить ошибки в консоль. Попробуйте команду оболочки Bash `ZENMAP_DEVELOPMENT=1 zenmap -v -v`, чтобы получить полезный отладочный вывод.

В Windows стандартный поток ошибок перенаправляется в файл `zenmap.exe.log` в том же каталоге, где находится `zenmap.exe`, а не выводится в консоль.

12.13. История

Изначально Zenmap был создан на основе Umit[3] - графического интерфейса Nmap, разработанного во время спонсируемого Google проекта Nmap Summer of Code в 2005 и 2006 годах. Основным автором Umit был Adriano Monteiro Marques. Когда в 2007 году Umit был изменён и интегрирован в Nmap, его переименовали в Zenmap.

Сноски

[1] [назад] nmap.org
[2] [назад] docs.python.org
[3] [назад] umitproject.org

Глава 13. Форматы вывода Nmap

13.1. Введение

Распространённая проблема инструментов безопасности с открытым исходным кодом - запутанный и неорганизованный вывод. Они часто извергают множество строк несущественных отладочных сведений, вынуждая пользователей продираться через страницы вывода, пытаясь отделить важные результаты от шума. Авторы программ нередко прилагают мало усилий к эффективной организации и представлению результатов. Сообщения вывода могут быть трудны для понимания и плохо документированы. Это не должно слишком удивлять: писать хитроумный код, эксплуатирующий какую-либо слабость TCP/IP, обычно приятнее, чем документацию или пользовательский интерфейс. Поскольку авторам открытого исходного кода редко платят, они занимаются тем, что им нравится.

Рискуя обидеть своего друга Dan Kaminsky, я назову его сканер портов Scanrand[1] примером программы, которая явно разрабатывалась с гораздо большим вниманием к изящным техническим трюкам, чем к удобному пользовательскому интерфейсу. Образец вывода в примере 13.1 взят со страницы документации Scanrand.

Пример 13.1. Вывод Scanrand при сканировании локальной сети

```
scanrand 10.0.1.1-254:quick
 10.0.1.38:80    [01]   0.003s
10.0.1.110:443   [01]   0.017s
10.0.1.254:443   [01]   0.021s
 10.0.1.57:445   [01]   0.024s
 10.0.1.59:445   [01]   0.024s
 10.0.1.38:22    [01]   0.047s
10.0.1.110:22    [01]   0.058s
10.0.1.110:23    [01]   0.058s
10.0.1.254:22    [01]   0.077s
10.0.1.254:23    [01]   0.077s
 10.0.1.25:135   [01]   0.088s
 10.0.1.57:135   [01]   0.089s
 10.0.1.59:135   [01]   0.090s
 10.0.1.25:139   [01]   0.097s
 10.0.1.27:139   [01]   0.098s
 10.0.1.57:139   [01]   0.099s
 10.0.1.59:139   [01]   0.099s
 10.0.1.38:111   [01]   0.127s
 10.0.1.57:1025  [01]   0.147s
 10.0.1.59:1025  [01]   0.147s
 10.0.1.57:5000  [01]   0.156s
 10.0.1.59:5000  [01]   0.157s
 10.0.1.53:111   [01]   0.182s
```

Хотя свою задачу этот вывод решает, интерпретировать его трудно. Данные печатаются в порядке получения ответов, без возможности сортировать номера портов или хотя бы группировать вместе все открытые порты одного целевого узла. В начале каждой строки впустую расходуется много места, а сводка результатов не предоставляется.

Вывод Nmap тоже далёк от совершенства, хотя я и стараюсь изо всех сил сделать его читаемым, хорошо организованным и гибким. Учитывая множество способов использования Nmap людьми и другим программным обеспечением, ни один формат не способен угодить всем. Поэтому Nmap предлагает несколько форматов, включая интерактивный режим для непосредственного чтения людьми и XML для лёгкого разбора программами.

Помимо различных форматов вывода, Nmap предоставляет параметры управления подробностью вывода и отладочными сообщениями. Разные типы вывода можно направлять в стандартный поток вывода или в именованные файлы, которые Nmap может дополнять или перезаписывать. Файлы вывода можно также использовать для возобновления прерванных сканирований. В этой главе приведены полные сведения об этих параметрах и каждом формате вывода.

13.2. Флаги командной строки

Как и почти все остальные возможности Nmap, поведение вывода управляется флагами командной строки. Эти флаги сгруппированы по категориям и описаны в следующих разделах.

13.2.1. Управление типом вывода

Самое основное средство управления выводом - указание желаемого формата или форматов вывода. Nmap предлагает пять типов, кратко перечисленных ниже и полностью описанных в последующих разделах.

Форматы вывода, поддерживаемые Nmap

• **Интерактивный вывод.** Это вывод, который Nmap по умолчанию направляет в стандартный поток вывода (`stdout`). Поэтому специального параметра командной строки у него нет. Интерактивный режим предназначен для людей, непосредственно читающих результаты, и характеризуется таблицей представляющих интерес портов, показанной в десятках примеров по всей книге.

• **Обычный вывод (**`-oN`**).** Он очень похож на интерактивный вывод и направляется в выбранный вами файл. Однако он отличается от интерактивного вывода в нескольких отношениях, вытекающих из ожидания, что этот вывод будет анализироваться после завершения сканирования, а не интерактивно. Поэтому интерактивный вывод включает такие сообщения (в зависимости от уровня подробности, указанного с помощью `-v`), как оценки времени завершения сканирования и уведомления об открытых портах. Обычный вывод опускает их как ненужные после завершения сканирования и печати окончательной таблицы представляющих интерес портов. В первой строке этот тип вывода печатает использованную командную строку `nmap`, а также время и дату выполнения.

• **Вывод XML (**`-oX`**).** XML предлагает стабильный формат, легко разбираемый программным обеспечением. Бесплатные анализаторы XML доступны для всех основных языков программирования, включая C/C++, Perl, Python и Java. Почти во всех случаях, когда нетривиальное приложение взаимодействует с Nmap, предпочтительным форматом является XML. В этой главе также рассматривается преобразование результатов XML в другие форматы, такие как отчёты HTML и таблицы базы данных.

• **Вывод для grep (**`-oG`**).** Этим простым форматом легко манипулировать в командной строке с помощью простых Unix-инструментов, таких как `grep`, `awk`, `cut` и `diff`. Каждый узел перечисляется в одной строке, а символы табуляции, косой черты и запятой служат разделителями полей вывода. Хотя это удобно для быстрого осмысления результатов, для более серьёзных задач предпочтителен формат XML, поскольку он стабильнее и содержит больше сведений.

• **Вывод sCRiPt KiDDi3 (**`-oS`**).** Этот формат предоставлен для 133t haXXorZ!

Хотя интерактивный вывод используется по умолчанию и не имеет связанных с ним параметров командной строки, остальные четыре параметра формата используют одинаковый синтаксис. Они

принимают один аргумент - имя файла для сохранения результатов. Можно указать несколько форматов, но каждый формат разрешается указывать только один раз. Например, вы можете захотеть сохранить обычный вывод для собственного просмотра, а XML того же сканирования - для программного анализа. Это можно сделать с помощью параметров `-oX myscan.xml -oN myscan.nmap`. Хотя для краткости в этой главе используются простые имена наподобие `myscan.xml`, обычно рекомендуются более описательные имена. Выбор имён - вопрос личных предпочтений, однако я использую длинные имена, включающие дату сканирования и одно-два описывающих его слова, и помещаю их в каталог, названный по имени сканируемой компании. Для удобства можно указать `-oA <basename>`, чтобы одновременно сохранить результаты сканирования в обычном формате, XML и формате для grep. Они сохраняются соответственно в `<basename>.nmap`, `<basename>.xml` и `<basename>.gnmap`. Как и в большинстве программ, перед именами файлов можно поставить путь к каталогу, например `~/nmaplogs/foocorp/` в Unix или `c:\hacking\sco` в Windows.

Хотя эти параметры сохраняют результаты в файлы, Nmap по-прежнему печатает интерактивный вывод в `stdout` как обычно. Например, команда `nmap -oX myscan.xml target` печатает XML в `myscan.xml`, а стандартный вывод заполняет теми же интерактивными результатами, которые были бы напечатаны, если бы `-oX` вообще не был указан. Это можно изменить, передав символ дефиса в качестве аргумента одному из типов формата. Тогда Nmap отключает интерактивный вывод и вместо него печатает в стандартный поток вывода результаты в указанном формате. Поэтому команда `nmap -oX - target` отправит в `stdout` только вывод XML. Серьёзные ошибки всё равно могут печататься в обычный поток ошибок `stderr`.

Когда вы указываете имя файла флагу формата вывода, например `-oN`, по умолчанию этот файл перезаписывается. Если вы предпочитаете сохранить существующее содержимое файла и дописать новые результаты, укажите параметр `--append-output`. Тогда все имена файлов вывода, указанные при этом запуске Nmap, будут дополняться, а не перезаписываться. Это плохо работает с данными сканирования XML (`-oX`), поскольку получившийся файл, как правило, невозможно правильно разобрать, пока вы не исправите его вручную.

В отличие от некоторых аргументов Nmap, пробел между флагом параметра файла журнала (например, `-oX`) и именем файла или дефисом обязателен. Если опустить пробелы и передать такие аргументы, как `-oG-` или `-oXscan.xml`, средство обратной совместимости Nmap приведёт к созданию файлов обычного формата вывода с именами `G-` и `Xscan.xml` соответственно.

Все эти аргументы поддерживают преобразования имени файла в стиле `strftime`. `%H`, `%M`, `%S`, `%m`, `%d`, `%y` и `%Y` имеют в точности тот же смысл, что и в `strftime`. `%T` равнозначно `%H%M%S`, `%R` - `%H%M`, а `%D` - `%m%d%y`. Знак `%`, за которым следует любой другой символ, даёт просто этот символ (`%%` даёт знак процента). Поэтому `-oX 'scan-%T-%D.xml'` будет использовать XML-файл вида `scan-144840-121307.xml`.

13.2.2. Управление подробностью вывода

Решив, в каком формате или форматах следует сохранять результаты, можно решить, насколько подробными они должны быть. Первый параметр `-v` включает подробный режим уровня один. Укажите `-v` дважды для несколько большего эффекта. Уровни подробности выше двух бесполезны. Большинство изменений влияет только на интерактивный вывод, а некоторые также влияют на обычный вывод и вывод script kiddie. Остальные типы вывода предназначены для машинной обработки, поэтому в этих форматах Nmap по умолчанию может предоставлять значительную детализацию, не утомляя человека. Однако в других режимах есть несколько изменений, при которых размер вывода можно существенно уменьшить, опустив некоторые подробности. Например, строка комментария в выводе для grep, содержащая список всех просканированных

портов, печатается в подробном режиме, потому что она может быть довольно длинной. В следующем списке описаны основные изменения, получаемые при наличии хотя бы одного параметра `-v`.

• **Оценки времени завершения сканирования.** При сканированиях, занимающих больше одной-двух минут, в интерактивном режиме вывода время от времени появляются подобные обновления:

```
SYN Stealth Scan Timing: About 30.01% done; ETC: 16:04
(0:01:09 remaining)
```

Новые обновления выдаются, если оценки существенно изменились. Оценку времени завершения поддерживают все методы сканирования портов, кроме холостого сканирования и сканирования с отражением от FTP, а также сканирование версий.

• **Сообщение об открытых портах при их обнаружении.** Когда включён подробный режим, открытые порты печатаются в интерактивном режиме по мере их обнаружения. Они также по-прежнему сообщаются в окончательной таблице представляющих интерес портов. Это позволяет пользователям начать исследование открытых портов ещё до завершения Nmap. Уведомления об открытых портах выглядят так:

```
Discovered open port 53/tcp on 64.13.134.52
```

• **Дополнительные предупреждения.** Nmap всегда печатает предупреждения об очевидных ошибках и критических проблемах. При включении подробного режима порог снижается, позволяя печатать больше предупреждений. Таких предупреждений десятки: от чрезмерной потери пакетов или необычайно большой задержки у целей до портов, неожиданным образом отвечающих на пробы. Ограничение частоты не даёт этим предупреждениям затопить экран.

• **Дополнительные примечания.** В подробном режиме Nmap печатает множество дополнительных информационных примечаний. Например, он печатает время начала каждого сканирования портов вместе с количеством сканируемых узлов и портов. Позднее он печатает заключительную строку, сообщающую продолжительность сканирования и кратко подводящую итоги.

• **Дополнительные сведения об определении ОС.** В подробном режиме показываются результаты тестов предсказуемости порядковых номеров TCP ISN и IP ID. Они выполняются как побочный продукт определения ОС. При уровне подробности выше единицы в большем числе случаев показывается фактический отпечаток определения ОС.

• **При ping-сканировании печатаются неработающие узлы.** Во время ping-сканирования с включённым подробным режимом печатаются не только работающие, но и неработающие узлы.

• **Поздравления с днём рождения.** При запуске 1 сентября в подробном режиме Nmap поздравляет себя с днём рождения.

Изменения, которые обычно полезны только до завершения Nmap и печати отчёта, направляются лишь в интерактивный режим вывода. Если обычный вывод отправляется в файл с помощью `-oN`, этот файл не будет содержать уведомлений об открытых портах или оценок времени завершения, хотя они по-прежнему печатаются в `stdout`. Предполагается, что вы просмотрите файл после завершения Nmap и не захотите видеть много лишнего мусора, тогда как в стандартном выводе вы можете наблюдать за ходом выполнения Nmap и интересоваться текущим прогрессом. Если действительно нужно направить в файл всё, что печатается в `stdout`, воспользуйтесь перенаправлением потока вывода, предоставляемым оболочкой (например, `nmap -v scanme.nmap.org > scanoutput.nmap`).

Десятки небольших изменений, зависящих от подробности (в основном дополнительные сообщения), слишком многочисленны, чтобы рассматривать их все. Кроме того, они всегда могут измениться. Эффективный способ увидеть их все - распаковать архив с исходным кодом последней

версии Nmap и найти их командой наподобие `grep -A1 o.verbose *.cc`. Характерные фрагменты вывода показаны в примере 13.2.

Пример 13.2. Поиск условных конструкций подробного режима с помощью grep

```
if (o.debugging || o.verbose) {
  log_write(LOG_STDOUT, "Initiating Idlescan against %s\n",
            target->NameIP());
}

if (o.verbose)
  output_ports_to_machine_parseable_output(ports, o.TCPScan(),
                                           o.udpscan, o.ipprotscan);

if (o.debugging || o.verbose)
  gh_perror("recvfrom in get_rpc_results");

if (o.verbose && openport != (unsigned long) -1)
  log_write(LOG_STDOUT, "For OSScan assuming port %d is open, %d..."

if (o.verbose)
  log_write(LOG_NORMAL|LOG_SKID|LOG_STDOUT,
            "IP ID Sequence Generation: %s\n",...
```

Следующие два примера объединяют всё это. В примере 13.3 показан вывод обычного сканирования без параметра `-v`.

Пример 13.3. Интерактивный вывод без включённого подробного режима

```
# nmap -T4 -A -p- scanme.nmap.org

Starting Nmap ( http://nmap.org )
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 65529 filtered ports
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain  ISC BIND 9.3.4
70/tcp  closed gopher
80/tcp  open   http    Apache httpd 2.2.2 ((Fedora))
|_ HTML title: Go ahead and ScanMe!
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.17 - 2.6.21, Linux 2.6.23

TRACEROUTE (using port 22/tcp)
HOP RTT   ADDRESS
1   16.92 nodem-msfc-vl245-act-security-gw-1-113.ucsd.edu (132.239.1.113)
[... nine similar lines cut ...]
11  21.97 scanme.nmap.org (64.13.134.52)

OS and Service detection performed. Please report any incorrect results
at http://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 168.10 seconds
```

Пример 13.4 представляет вывод того же сканирования с включённым подробным режимом. Такие возможности, как дополнительные данные определения ОС, оценки времени завершения, уведомления об открытых портах и дополнительные информационные сообщения, легко заметить в последнем выводе. Эти дополнительные сведения часто полезны при интерактивном сканировании, поэтому при сканировании одной машины я всегда указываю `-v`, если только у меня нет веской причины этого не делать.

Пример 13.4. Интерактивный вывод с включённым подробным режимом

```
# nmap -v -T4 -A -p- scanme.nmap.org
Starting Nmap ( http://nmap.org )
Initiating Ping Scan at 00:12
Completed Ping Scan at 00:12, 0.02s elapsed (1 total hosts)
```

```
Initiating SYN Stealth Scan at 00:12
Scanning scanme.nmap.org (64.13.134.52) [65535 ports]
Discovered open port 80/tcp on 64.13.134.52
Discovered open port 53/tcp on 64.13.134.52
Discovered open port 22/tcp on 64.13.134.52
SYN Stealth Scan Timing: About 16.66% done; ETC: 00:15 (0:02:30 remaining)
Completed SYN Stealth Scan at 00:14, 125.13s elapsed (65535 total ports)
Scanning 3 services on scanme.nmap.org (64.13.134.52)
Completed Service scan at 00:14, 6.05s elapsed (3 services on 1 host)
Initiating OS detection (try #1) against scanme.nmap.org (64.13.134.52)
[Removed some verbose traceroute and parallel DNS related messages]
Initiating SCRIPT ENGINE at 00:14
Completed SCRIPT ENGINE at 00:14, 4.09s elapsed
Host scanme.nmap.org (64.13.134.52) appears to be up ... good.
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 65529 filtered ports
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain  ISC BIND 9.3.4
70/tcp  closed gopher
80/tcp  open   http    Apache httpd 2.2.2 ((Fedora))
|_ HTML title: Go ahead and ScanMe!
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.17 - 2.6.21, Linux 2.6.23
Uptime guess: 12.476 days (since Wed Jul 2 12:48:56 2008)
TCP Sequence Prediction: Difficulty=198 (Good luck!)
IP ID Sequence Generation: All zeros

TRACEROUTE (using port 22/tcp)
HOP RTT  ADDRESS
1   0.25 nodem-msfc-vl245-act-security-gw-1-113.ucsd.edu (132.239.1.113)
[... nine similar lines cut ...]
11 20.67 scanme.nmap.org (64.13.134.52)

OS and Service detection performed. Please report any incorrect results
at http://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 147.462 seconds
           Raw packets sent: 131128 (5.771MB) | Rcvd: 283637 (12.515MB)
```

13.2.3. Включение отладочного вывода

Когда даже подробный режим не предоставляет достаточно данных, можно включить отладку, которая затопит вас гораздо большим их количеством! Как и подробность (`-v`), отладка включается флагом командной строки (`-d`), а её уровень можно увеличить, указав флаг несколько раз. В качестве альтернативы уровень отладки можно задать, передав аргумент параметру `-d`. Например, `-d9` задаёт уровень девять. Это наивысший эффективный уровень, и он создаст тысячи строк, если только вы не выполняете очень простое сканирование с совсем небольшим количеством портов и целей.

Отладочный вывод полезен, когда в Nmap подозревается ошибка или вы просто не понимаете, что и почему делает Nmap. Поскольку эта возможность в основном предназначена для разработчиков, отладочные строки не всегда понятны сами по себе. Если вы не понимаете строку, остаётся лишь проигнорировать её, найти в исходном коде или попросить помощи в списке разработчиков (`nmap-dev`). Некоторые строки говорят сами за себя, но по мере повышения уровня отладки сообщения становятся всё менее понятными. В примере 13.5 показано несколько различных отладочных строк, полученных при сканировании Scanme с `-d5`.

Пример 13.5. Несколько характерных отладочных строк

```
Timeout vals: srtt: 27495 rttvar: 27495 to: 137475 delta -2753
```

```
            ==> srtt: 27150 rttvar: 21309 to: 112386
RCVD (15.3330s) TCP 64.13.134.52:25 > 132.239.1.115:50122 RA ttl=52
            id=0 iplen=40 seq=0 win=0 ack=4222318673
**TIMING STATS** (15.3350s): IP, probes active/freshportsleft/retry_stack/
                             outstanding/retranwait/onbench,
                             cwnd/ccthresh/delay, timeout/srtt/rttvar/
  Groupstats (1/1 incomplete): 83/*/*/*/*/* 82.80/75/* 100000/25254/4606
  64.13.134.52: 83/60836/0/777/316/4295 82.80/75/0 100000/26200/4223
Current sending rates: 711.88 packets / s, 31322.57 bytes / s.
Overall sending rates: 618.24 packets / s, 27202.62 bytes / s.
Discovered filtered port 10752/tcp on 64.13.134.52
Packet capture filter (device eth0): dst host 132.239.1.115 and
                                  (icmp or ((tcp or udp) and
                                  (src host 64.13.134.52)))
SCRIPT ENGINE: TCP 132.239.1.115:59045 > 64.13.134.52:53 | CLOSE
```

Полный пример здесь не приводится, потому что журналы отладки очень длинны. При сканировании Scanme без подробного режима было выведено 32 строки текста (пример 13.3 «Интерактивный вывод без включённого подробного режима»), а с ним - 61 строка (пример 13.4 «Интерактивный вывод с включённым подробным режимом»). То же сканирование с `-d` вместо `-v` заняло 113 строк. С `-d2` оно раздулось до 65 731 строки, а `-d5` вывел 396 879 строк! Параметр отладки неявно включает подробность, поэтому указывать оба не нужно.

Определение наилучшего уровня вывода для конкретной задачи отладки - вопрос проб и ошибок. Сначала я пробую низкий уровень, чтобы понять происходящее, а затем при необходимости повышаю его. По мере того как я узнаю больше, мне, возможно, удаётся точнее изолировать проблему или вопрос. Тогда я стараюсь упростить команду, чтобы компенсировать возросшее многословие более высокого уровня отладки.

Как `grep` помогает определить изменения и уровни, связанные с подробностью, так же он полезен при исследовании отладочного вывода. Рекомендую выполнить следующую команду из каталога `nmap-<VERSION>` в архиве исходного кода Nmap:

```
grep -A1 o.debugging *.cc
```

13.2.4. Обработка сообщений об ошибках и предупреждений

Предупреждения и ошибки, печатаемые Nmap, обычно выводятся только на экран (интерактивный вывод), оставляя файлы вывода обычного формата (как правило, указанные с помощью `-oN`) незахламлёнными. Если эти сообщения всё же требуется видеть в указанном файле обычного вывода, используйте параметр журналирования ошибок `--log-errors`. Он полезен, когда вы не наблюдаете за интерактивным выводом или хотите записывать ошибки во время отладки проблемы. Сообщения об ошибках и предупреждения по-прежнему будут появляться и в интерактивном режиме. Для большинства ошибок, связанных с неверными аргументами командной строки, это не сработает, потому что Nmap мог ещё не инициализировать файлы вывода. Кроме того, некоторые сообщения Nmap об ошибках и предупреждениях используют другую систему, которая пока не поддерживает этот параметр.

Альтернатива `--log-errors` - перенаправление интерактивного вывода (включая стандартный поток ошибок) в файл. В большинстве оболочек Unix это сделать легко. Например, tcsh использует форму `nmap <options> >& alloutput.nmap`. Bash использует немного иной синтаксис: `nmap <options> &> alloutput.nmap`. Синтаксис Windows `cmd.exe` для этого настолько запутан, что вместо него рекомендуется `--log-errors`. Например, можно выполнить `nmap --log-errors -oN alloutput.nmap <options>`.

13.2.5. Включение трассировки пакетов

Параметр `--packet-trace` заставляет Nmap печатать сводку о каждом отправленном и полученном им пакете. Это может быть чрезвычайно полезно для отладки или понимания поведения Nmap, что демонстрируют примеры по всей книге. В примере 13.6 показано простое ping-сканирование Scanme с включённой трассировкой пакетов.

Пример 13.6. Использование --packet-trace для детализации ping-сканирования Scanme

```
# nmap --packet-trace -n -sP scanme.nmap.org

Starting Nmap ( http://nmap.org )
SENT (0.0230s) ICMP 132.239.1.115 > 64.13.134.52 echo request
               (type=8/code=0) ttl=38 id=5420 iplen=28
SENT (0.0230s) TCP 132.239.1.115:43743 > 64.13.134.52:80 A ttl=57
               id=29415 iplen=40 seq=2799605278 win=2048 ack=2120834905
RCVD (0.0380s) TCP 64.13.134.52:80 > 132.239.1.115:43743 R ttl=52
               id=0 iplen=40 seq=2120834905 win=0
Host 64.13.134.52 appears to be up.
Nmap done: 1 IP address (1 host up) scanned in 0.04 seconds
```

Этот запуск Nmap показывает три дополнительные строки, вызванные трассировкой пакетов (каждая перенесена для удобства чтения). Каждая строка содержит несколько полей. Первое сообщает, отправлен пакет Nmap или получен им, и сокращается как `SENT` и `RCVD`. Следующее поле - счётчик времени, показывающий время, прошедшее с момента запуска Nmap. Время указано в секундах, и в данном случае Nmap потребовалась лишь крошечная доля секунды. Следующее поле - протокол: TCP, UDP или ICMP. Затем идут IP-адреса источника и назначения, разделённые направленной стрелкой. Для пакетов TCP или UDP после каждого IP-адреса ставится двоеточие и номер порта источника или назначения.

Остальная часть каждой строки зависит от протокола. Как видно, для ICMP, если возможно, указывается удобочитаемый тип (`echo request` в данном случае), за которым следуют значения типа и кода ICMP. Журнальные строки пакетов ICMP заканчиваются полями IP TTL, ID и длины пакета. После IP-адреса и порта назначения пакеты TCP используют немного иной формат. Сначала идёт список символов, представляющих установленные флаги TCP. Символы флагов - `SAFRPUEC`, что означает соответственно SYN, ACK, FIN, RST, PSH, URG, ECE и CWR. Последние два флага являются частью явного уведомления TCP о перегрузке, описанного в RFC 3168.

Поскольку трассировка пакетов может создавать тысячи строк вывода, полезно ограничивать интенсивность сканирования минимумом, который всё ещё соответствует вашей цели. Сканирование одного порта одной машины не похоронит вас под данными, тогда как вывод `--packet-trace` при сканировании целой сети может оказаться подавляющим. Трассировка пакетов автоматически включается, когда уровень отладки (`-d`) не меньше трёх.

Иногда `--packet-trace` предоставляет специализированные данные, которые Nmap иначе никогда не показывает. Например, пример 13.6 «Использование --packet-trace для детализации ping-сканирования Scanme» показывает пакеты ICMP и TCP, отправленные целевому узлу. Цель отвечает на эхо-запрос ICMP, что может быть ценной информацией, которую Nmap иначе не показывает. Возможно, целевой узел также ответил на TCP-пакет: получив один ответ на ping-сканирование, Nmap прекращает прослушивание, поскольку этого достаточно, чтобы определить, что узел работает.

13.2.6. Возобновление прерванных сканирований

Некоторые масштабные запуски Nmap занимают очень много времени - порядка нескольких дней. Такие сканирования не всегда завершаются. Ограничения могут запрещать запуск Nmap в рабочие часы, сеть может отключиться, машина с выполняющимся Nmap может подвергнуться запланированной или незапланированной перезагрузке, либо сам Nmap может аварийно

завершиться. Администратор, запустивший Nmap, также может отменить его по любой другой причине, нажав Ctrl+C. Перезапуск всего сканирования с самого начала может быть нежелателен. К счастью, если были сохранены журналы обычного (-oN) или предназначенного для grep (-oG) формата, пользователь может попросить Nmap возобновить сканирование с той цели, над которой он работал при прекращении выполнения. Просто укажите параметр --resume и передайте ему файл обычного/grepable-вывода как аргумент. Другие аргументы не допускаются, поскольку Nmap разбирает файл вывода, чтобы использовать те же параметры, которые были указаны прежде. Просто вызовите Nmap как nmap --resume <logfilename>. Nmap дополнит новыми результатами файлы данных, указанные при предыдущем запуске. Возобновление не поддерживает формат вывода XML, поскольку объединить два запуска в один допустимый XML-файл было бы трудно.

13.3. Интерактивный вывод

Интерактивный вывод - это то, что Nmap печатает в поток stdout, обычно появляющийся в окне терминала, из которого вы запустили Nmap. В других обстоятельствах stdout мог быть перенаправлен в файл либо результаты могло читать другое приложение, например Nessus или графический интерфейс Nmap. Если более крупное приложение интерпретирует результаты, а не печатает вывод Nmap непосредственно пользователю, уместнее использовать вывод XML, рассматриваемый в разделе 13.6 «Вывод XML (-oX)».

У этого формата лишь одна цель: представить результаты, ценные для читающего их человека. Не предпринимается никаких усилий, чтобы облегчить их машинный разбор или сохранять стабильный формат между версиями Nmap. Для этих целей существуют лучшие форматы. Самая трудная задача - решить, какие сведения достаточно ценны для печати. Печально опускать данные, нужные пользователю, но затопить его страницами преимущественно несущественного вывода может быть ещё хуже. Флаги подробности, отладки и трассировки пакетов позволяют сдвинуть этот баланс в соответствии с предпочтениями отдельных пользователей.

Этот формат вывода не требует здесь пространного описания, поскольку его уже показывают большинство примеров Nmap в книге. Чтобы понять интерактивный вывод Nmap для определённой возможности, обратитесь к посвящённому ей разделу книги. Типичные примеры интерактивного вывода приведены в примере 13.3 «Интерактивный вывод без включённого подробного режима» и примере 13.4 «Интерактивный вывод с включённым подробным режимом».

13.4. Обычный вывод (-oN)

Обычный вывод печатается в файл, когда параметр -oN указан с аргументом имени файла. Он похож на интерактивный вывод, за исключением того, что примечания, теряющие актуальность после завершения сканирования, удаляются. Предполагается, что файл будет прочитан после завершения Nmap, поэтому оценки времени завершения и новые уведомления об открытых портах избыточны по сравнению с фактическим временем завершения и упорядоченной таблицей портов. Поскольку вывод может храниться долго и просматриваться среди множества других журналов, Nmap печатает в первой строке время выполнения, аргументы командной строки и номер версии Nmap. Подобная строка в конце сканирования сообщает окончательные временные показатели и число узлов. Обе строки начинаются с символа решётки, обозначающего комментарии. Если вашему приложению приходится разбирать обычный вывод вместо форматов XML/grepable, обеспечьте, чтобы оно игнорировало нераспознанные комментарии, а не считало их ошибкой и не прекращало работу. Пример 13.7 - типичный пример обычного вывода. Обратите внимание: -oN -

использован, чтобы отключить интерактивный вывод и направить обычный вывод прямо в `stdout`.

Пример 13.7. Типичный пример обычного вывода

```
# nmap -T4 -A -p- -oN - scanme.nmap.org

# Nmap 4.68 scan initiated Tue Jul 15 07:27:26 2008 as: nmap -T4 -A -p- -oN - scanme.nmap.
    org
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 65529 filtered ports
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain  ISC BIND 9.3.4
70/tcp  closed gopher
80/tcp  open   http    Apache httpd 2.2.2 ((Fedora))
|_ HTML title: Go ahead and ScanMe!
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.17 - 2.6.21, Linux 2.6.23

TRACEROUTE (using port 22/tcp)
HOP RTT  ADDRESS
1   2.98 nodem-msfc-vl245-act-security-gw-1-113.ucsd.edu (132.239.1.113)
[... nine similar lines cut ...]
11 13.34 scanme.nmap.org (64.13.134.52)

OS and Service detection performed. Please report any incorrect results
at http://nmap.org/submit/ .
# Nmap done at Tue Jul 15 07:29:45 2008 -- 1 IP address (1 host up) scanned in 138.938
    seconds
```

13.5. Вывод \$crIpT kIddI3 OuTPuT (-oS)

Вывод script kiddie подобен интерактивному, за исключением того, что он подвергается постобработке, чтобы больше соответствовать 'l33t HaXXorZ! Прежде они смотрели на Nmap свысока из-за единообразного регистра букв и орфографии. Лучше всего этот вывод понять на примере, как в примере 13.8.

Пример 13.8. Типичный пример \$crIpT KiDDi3 OuTPuT

```
# nmap -T4 -A -oS - scanme.nmap.org

StaRting NMap ( httP://nmap.Org )
Int3restIng pOrtz On $CAnme.nmap.Org (64.13.134.52):
NOt ShOwn: 65529 FilterEd pOrt$
PORT    $TATE  $ERVIC3 V3R$IoN
22/tcP  Op3n   s$h     Open$$H 4.3 (prOtOcol 2.0)
25/TcP  closEd $mtp
53/tcp  op3n   dOma!n  I$C BIND 9.3.4
70/tcp  clo$ed G0ph3r
80/tcp  Op3n   htTP    4pach3 httpd 2.2.2 ((F3d0ra))
|_ HTML tITl3: gO aheAD And $canM3!
113/tcp cl0$Ed auTh
Device type: g3NeraL purp0$3
Running: L1Nux 2.6.X
oS detAilz: LinuX 2.6.17 - 2.6.21, L1nux 2.6.23
[Many lines cut for brevity]
NmAp doNe: 1 ip addre$z (1 H0$t up) $canneD iN 138.94 $ec0NdS
```

Некоторые люди, лишённые чувства юмора, воспринимают этот параметр слишком серьёзно и ругают меня за потакание скрипт-кидди. Это всего лишь шутка, высмеивающая скрипт-кидди: в действительности они не используют этот режим (надеюсь).

13.6. Вывод XML (-oX)

У XML, расширяемого языка разметки, есть как критики, так и множество рьяных сторонников. Я долго принадлежал к первым и лишь нехотя включил XML в Nmap после того, как добровольцы выполнили большую часть работы. С тех пор я научился ценить мощь и гибкость XML и даже написал эту книгу в формате DocBook XML. Я настоятельно рекомендую программистам взаимодействовать с Nmap через интерфейс XML, а не пытаться разбирать обычный, интерактивный или предназначенный для grep вывод. Этот формат содержит больше сведений, чем остальные, и достаточно расширяем, чтобы можно было добавлять новые возможности, не нарушая работу существующих использующих его программ. Его можно разбирать стандартными анализаторами XML, доступными для всех популярных языков программирования, обычно бесплатно. Редакторы, валидаторы, системы преобразования и многие другие приложения уже умеют работать с этим форматом. Обычный и интерактивный вывод, напротив, специфичны для Nmap и регулярно изменяются, поскольку я стремлюсь яснее представлять данные конечным пользователям. Вывод для grep также специфичен для Nmap, и его труднее расширять, чем XML. Он считается устаревшим, и многие возможности Nmap, такие как определение MAC-адреса, в этом формате вывода не представлены.

Пример вывода XML Nmap показан в примере 13.9. Пробельные символы скорректированы для удобства чтения. В данном случае XML был направлен в `stdout` благодаря конструкции `-oX -`. Одни программы, выполняющие Nmap, предпочитают читать вывод таким образом, тогда как другие указывают направить вывод в файл, а затем читают этот файл после завершения Nmap.

Пример 13.9. Пример вывода Nmap XML

```
# nmap -T4 -A -p- -oX - scanme.nmap.org
<?xml version="1.0" encoding="utf-8"?>
<?xml-stylesheet href="/usr/share/nmap/nmap.xsl" type="text/xsl"?>
<!-- Nmap 4.68 scan initiated Tue Jul 15 07:27:26 2008 as:
         nmap -T4 -A -p- -oX - scanme.nmap.org -->
<nmaprun scanner="nmap" args="nmap -T4 -A -p- -oX - scanme.nmap.org"
         start="1216106846" startstr="Tue Jul 15 07:27:26 2008"
         version="4.68" xmloutputversion="1.02">
  <scaninfo type="syn" protocol="tcp" numservices="65535" services="1-65535" />
  <verbose level="0" /> <debugging level="0" />
  <host starttime="1216106846" endtime="1216106985">
    <status state="up" reason="reset" />
    <address addr="64.13.134.52" addrtype="ipv4" />
    <hostnames><hostname name="scanme.nmap.org" type="PTR" /></hostnames>
    <ports><extraports state="filtered" count="65529">
      <extrareasons reason="no-responses" count="65529" /></extraports>
      <port protocol="tcp" portid="22">
        <state state="open" reason="syn-ack" reason_ttl="52" />
        <service name="ssh" product="OpenSSH" version="4.3"
              extrainfo="protocol 2.0" method="probed" conf="10" /> </port>
      <!-- Several port elements removed for brevity -->
      <port protocol="tcp" portid="80">
        <state state="open" reason="syn-ack" reason_ttl="52" />
        <service name="http" product="Apache httpd" version="2.2.2"
              extrainfo="(Fedora)" method="probed" conf="10" />
        <script id="HTML title" output="Go ahead and ScanMe!" /> </port>
      <port protocol="tcp" portid="113">
        <state state="closed" reason="reset" reason_ttl="52" />
        <service name="auth" method="table" conf="3" /> </port> </ports>
    <os>
      <portused state="open" proto="tcp" portid="22" />
      <portused state="closed" proto="tcp" portid="25" />
      <osclass type="general purpose" vendor="Linux" osfamily="Linux"
            osgen="2.6.X" accuracy="100" />
      <osmatch name="Linux 2.6.17 - 2.6.21" accuracy="100" line="11886" />
```

```
      <osmatch name="Linux 2.6.23" accuracy="100" line="13895" /> </os>
    <uptime seconds="1104050" lastboot="Wed Jul 2 12:48:55 2008" />
    <tcpsequence index="203" difficulty="Good luck!"
        values="31F88BFB,327D2AA6,329B817C,329D4191,321A15D3,32B3D917" />
    <ipidsequence class="All zeros" values="0,0,0,0,0,0" />
    <tcptssequence class="1000HZ"
        values="41CE58DD,41CE5941,41CE59A5,41CE5A09,41CE5A6D,41CE5AD5" />
    <trace port="22" proto="tcp">
      <hop ttl="1" rtt="2.98" ipaddr="132.239.1.113"
         host="nodem-msfc-vl245-act-security-gw-1-113.ucsd.edu" />
      <!-- Several hop elements removed for brevity -->
      <hop ttl="11" rtt="13.34" ipaddr="64.13.134.52"
         host="scanme.nmap.org" /> </trace>
    <times srtt="14359" rttvar="1215" to="100000" /> </host>
  <runstats><finished time="1216106985" timestr="Tue Jul 15 07:29:45 2008" />
    <hosts up="1" down="0" total="1" />
    <!-- Nmap done at Tue Jul 15 07:29:45 2008;
           1 IP address (1 host up) scanned in 138.938 seconds -->
  </runstats>
</nmaprun>
```

Ещё одно преимущество XML состоит в том, что его многословная природа делает его легче для чтения и понимания, чем другие форматы. Читатели, знакомые с Nmap в целом, вероятно, смогут понять большую часть XML-вывода из примера 13.9 «Пример вывода Nmap XML» без дополнительной документации. Формат вывода для grep, напротив, трудно расшифровать без собственного справочного руководства.

В примере XML-вывода есть несколько аспектов, которые могут быть непонятны сами по себе. Например, рассмотрите два элемента `port` в примере 13.10.

Пример 13.10. Элементы port в Nmap XML

```
<port protocol="tcp" portid="80">
  <state state="open" reason="syn-ack" reason_ttl="52" />
  <service name="http" product="Apache httpd" version="2.2.2"
        extrainfo="(Fedora)" method="probed" conf="10" />
  <script id="HTML title" output="Go ahead and ScanMe!" />
</port>
<port protocol="tcp" portid="113">
  <state state="closed" reason="reset" reason_ttl="52" />
  <service name="auth" method="table" conf="3" />
</port>
```

Протокол порта, ID (номер порта), состояние и имя службы те же, что показываются в таблице портов интерактивного вывода. Атрибуты службы `product`, `version` и `extrainfo` поступают от определения версий и объединяются в одно поле таблицы портов интерактивного вывода. Атрибуты `method` и `conf` не представлены ни в каких других типах вывода. Значением `method` может быть `table`, означающее, что имя службы было просто найдено в `nmap-services` по номеру порта и протоколу, или `probed`, означающее, что оно было определено системой определения версий. Атрибут `conf` измеряет уверенность Nmap в правильности имени службы. Значения находятся в диапазоне от единицы (наименьшая уверенность) до десяти. Для портов, определённых поиском в таблице, уровень уверенности Nmap составляет лишь три, тогда как в том, что порт 80 из примера 13.10 «Элементы port в Nmap XML» обслуживается Apache httpd, Nmap уверен в высокой степени (уровень 10), поскольку Nmap подключился к порту и обнаружил сервер, демонстрирующий протокол HTTP с баннерами Apache.

Ещё один аспект, который некоторые пользователи считают запутанным: атрибуты `/nmaprun/@start` и `/nmaprun/runstats/finished/@time` содержат временные отметки в Unix-времени - количестве секунд с 1 января 1970 года. Программам часто легче работать с такой формой. Для удобства людей версии 3.78 и новее включают эквивалентное календарное время в атрибутах `/nmaprun/@startstr` и `/nmaprun/runstats/finished/@endstr`.

Nmap включает определение типа документа (DTD), позволяющее анализаторам XML проверять допустимость XML-вывода Nmap. Хотя оно предназначено главным образом для программного использования, оно также может помочь людям интерпретировать XML-вывод Nmap. DTD определяет допустимые элементы формата и часто перечисляет атрибуты и значения, которые они могут принимать. Оно воспроизведено в приложении A «DTD XML-вывода Nmap».

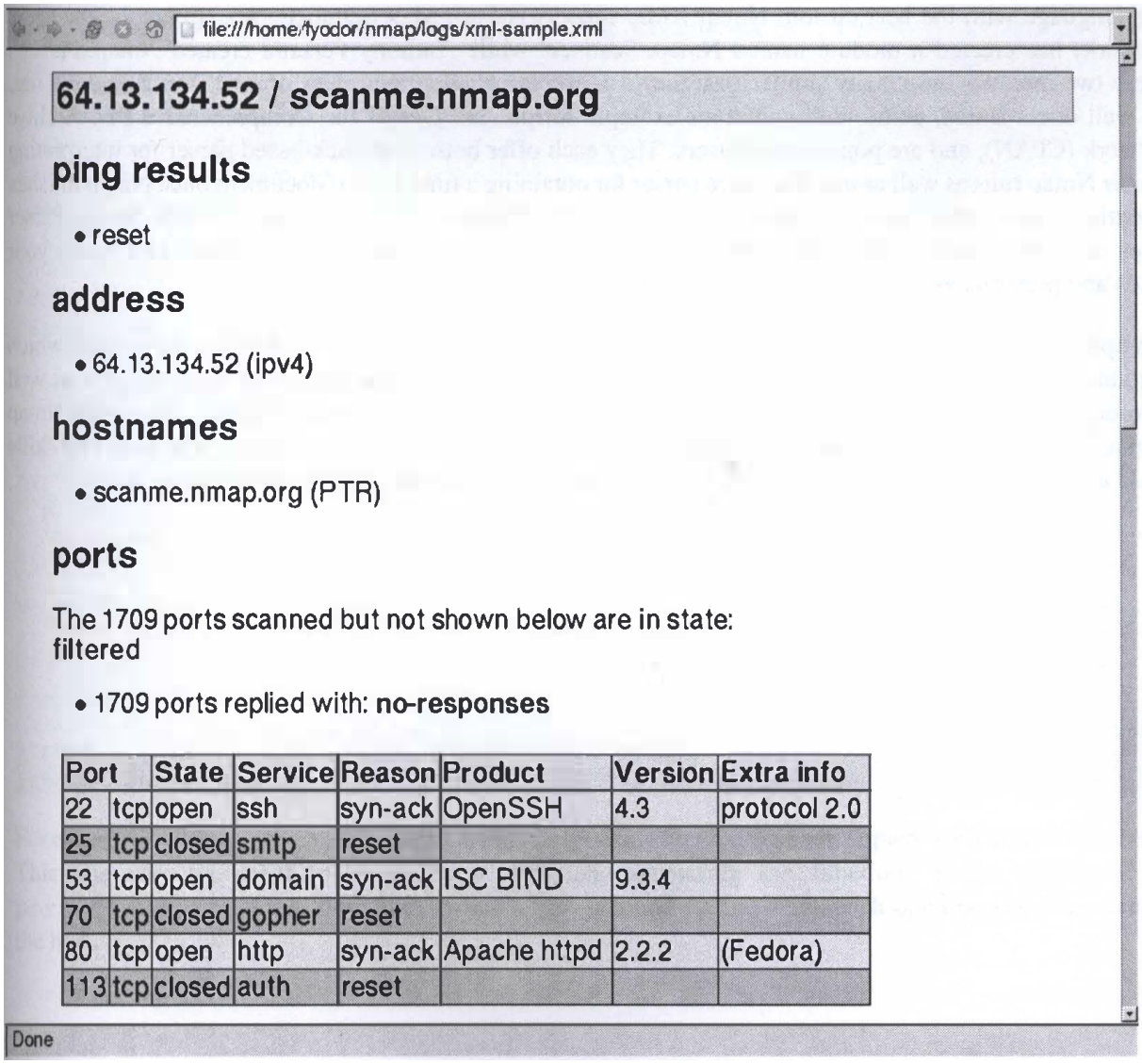
13.6.1. Использование XML-вывода

Формат Nmap XML можно использовать множеством мощных способов, хотя лишь немногие пользователи действительно пользуются его преимуществами. Полагаю, причина состоит в неопытности многих пользователей с XML в сочетании с недостатком практической, ориентированной на решения документации по использованию формата Nmap XML. В этой главе приведено несколько практических примеров, включая раздел 13.7 «Манипулирование XML-выводом с помощью Perl», раздел 13.8 «Вывод в базу данных» и раздел 13.9 «Создание отчётов HTML».

Ключевое преимущество XML состоит в том, что вам не нужно писать собственный анализатор, как приходится делать для специализированных типов вывода Nmap, таких как grepable- и интерактивный вывод. Подойдёт любой универсальный анализатор XML.

Наиболее знакомый людям анализатор XML находится в веб-браузере. И IE, и Mozilla/Firefox включают полноценные анализаторы, которые можно использовать для просмотра данных Nmap XML. Для этого достаточно ввести имя XML-файла или URL в адресную строку. На рисунке 13.1 приведён пример XML-вывода, отображённого веб-браузером. Как работает это автоматическое отображение и как сохранить постоянную копию отчёта HTML, рассматривается в разделе 13.9 «Создание отчётов HTML».

Рисунок 13.1. XML-вывод в веб-браузере



Разумеется, XML-вывод Nmap можно просматривать в любом текстовом редакторе или редакторе XML. Некоторые программы электронных таблиц, включая Microsoft Excel, способны непосредственно импортировать данные Nmap XML для просмотра. Эти XML-процессоры общего назначения имеют общее ограничение: они обрабатывают Nmap XML универсально, как любой другой XML-файл. Они не понимают относительную важность элементов и не знают, как организовать данные для более полезного представления. Использование специализированных XML-процессоров, осмысленно обрабатывающих XML-вывод Nmap, является предметом следующих разделов.

13.7. Манипулирование XML-выводом с помощью Perl

Универсальные анализаторы XML доступны для всех популярных языков программирования, часто бесплатно. Примерами служат библиотека libxml для C и анализатор Apache Xerces для Java и C++ (с привязками Perl и COM). Хотя этих анализаторов достаточно для обработки XML-вывода Nmap, разработчики создали специальные модули для нескольких языков, способные ещё больше упростить взаимодействие с Nmap XML.

Наилучшую специализированную поддержку Nmap XML имеет язык Perl. Max Schubert (ласково известный как Perldork) создал модуль Nmap::Scanner[2], а Anthony Persaud - Nmap::Parser[3]. У этих

двух модулей много общего: они могут сами выполнять Nmap или читать файл вывода, хорошо документированы, поставляются с многочисленными примерами сценариев, входят в Comprehensive Perl Archive Network (CPAN) и популярны среди пользователей. Каждый из них предлагает как анализатор на основе обратных вызовов для интерпретации данных во время работы Nmap, так и анализатор всего документа сразу для получения полностью разобранного документа после завершения Nmap. Их API немного различаются: Nmap::Scanner опирается на типобезопасные классы, тогда как Nmap::Parser - на более легковесные встроенные массивы Perl. Рекомендую рассмотреть оба и решить, какой лучше соответствует вашим потребностям и предпочтениям.

Пример 13.11 - простая демонстрация Nmap::Parser. Он взят из документации модуля (которая содержит и множество других примеров). Он выполняет быстрое сканирование, затем печатает общую статистику сканирования, а также сведения о каждом доступном целевом узле. Обратите внимание, насколько он читаем по сравнению со сценариями, использующими другие форматы вывода Nmap и заполненными логикой разбора и регулярными выражениями. Даже люди со слабыми навыками Perl могли бы использовать его как отправную точку для создания простых программ, автоматизирующих их задачи сканирования Nmap.

Пример 13.11. Образец кода Nmap::Parser

```
use Nmap::Parser;

      #PARSING
my $np = new Nmap::Parser;

$nmap_exe = '/usr/bin/nmap';
$np->parsescan($nmap_exe,'-sT -p1-1023', @ips);

#or

$np->parsefile('nmap_output.xml'); #using filenames

      #GETTING SCAN INFORMATION

print "Scan Information:\n";
$si = $np->get_scaninfo();
#get scan information by calling methods
print
'Number of services scanned: '.$si->num_of_services()."\n",
'Start Time: '.$si->start_time()."\n",
'Scan Types: ',(join ' ',$si->scan_types())."\n";

      #GETTING HOST INFORMATION

print "Hosts scanned:\n";
for my $host_obj ($np->get_host_objects()){
  print
  'Hostname  : '.$host_obj->hostname()."\n",
  'Address   : '.$host_obj->ipv4_addr()."\n",
  'OS match  : '.$host_obj->os_match()."\n",
  'Open Ports: '.(join ',',$host_obj->tcp_ports('open'))."\n";
       #... you get the idea...
}

#frees memory--helpful when dealing with memory intensive scripts
$np->clean();
```

Для сравнения в примере 13.12 приведён образец сценария Perl с использованием Nmap::Scanner, скопированный из его документации. В нём применяется событийный подход с обратными вызовами: регистрируются функции `scan_started` и `port_found`, печатающие уведомления в реальном времени, когда обнаруживается работающий узел и когда на нём обнаруживается каждый открытый порт.

Пример 13.12. Образец кода Nmap::Scanner

```
my $scanner = new Nmap::Scanner;
$scanner->register_scan_started_event(\&scan_started);
$scanner->register_port_found_event(\&port_found);
$scanner->scan('-sS -p 1-1024 -O --max-rtt-timeout 200ms somehost.org.net.it');

sub scan_started {
    my $self     = shift;
    my $host     = shift;

    my $hostname = $host->name();
    my $addresses = join(', ', map {$_->address()} $host->addresses());
    my $status = $host->status();

    print "$hostname ($addresses) is $status\n";
}

sub port_found {
    my $self     = shift;
    my $host     = shift;
    my $port     = shift;

    my $name = $host->name();
    my $addresses = join(', ', map {$_->addr()} $host->addresses());

    print "On host $name ($addresses), found ",
          $port->state()," port ",
          join('/',$port->protocol(),$port->portid()),"\n";
}
```

13.8. Вывод в базу данных

Распространённое желание - выводить результаты Nmap в базу данных для упрощения запросов и отслеживания. Это позволяет пользователям, от отдельного специалиста по тестированию на проникновение до международного предприятия, хранить все результаты сканирования и легко сравнивать их. Предприятие может ежедневно запускать крупные сканирования и планировать запросы, отправляющие администраторам по электронной почте сведения о вновь открытых портах или доступных машинах. Специалист по тестированию на проникновение может узнать о новой уязвимости и найти во всех старых результатах сканирования затронутое приложение, чтобы предупредить соответствующих клиентов. Исследователи могут сканировать миллионы IP-адресов и хранить результаты в базе данных для простых запросов в реальном времени.

Хотя эти цели похвальны, Nmap не предоставляет функции непосредственного вывода в базу данных. Существует не только слишком много различных типов баз данных, чтобы я мог поддерживать их все, но и потребности пользователей различаются настолько сильно, что ни одна схема базы данных не подходит всем. Потребности предприятия, специалиста по тестированию на проникновение и исследователя требуют разных структур таблиц.

Для проектов, достаточно крупных, чтобы требовать базы данных, я рекомендую сначала определить оптимальную схему БД, а затем написать простую программу или сценарий для надлежащего импорта данных Nmap XML. Благодаря широкой доступности анализаторов XML и модулей доступа к базам данных такие сценарии часто занимают лишь несколько минут. Perl нередко является хорошим выбором, поскольку предлагает мощный слой абстракции базы данных и специализированную поддержку Nmap XML. Раздел 13.7 «Манипулирование XML-выводом с помощью Perl» показывает, насколько легко сценарии Perl могут использовать данные Nmap XML.

Другой вариант - использовать нестандартное дополнение поддержки баз данных Nmap. Один из примеров - nmap-sql[4], добавляющий в сам Nmap возможность ведения журналов MySQL. Недостаток заключается в том, что сейчас он поддерживает лишь базу данных MySQL и его

приходится часто переносить на новые версии Nmap. С другой стороны, подход на основе XML гораздо реже перестаёт работать при выпуске новых версий Nmap.

Ещё один вариант - PBNJ[5], набор средств для наблюдения за изменениями сети во времени. Он сохраняет такие данные сканирования, как работающие узлы и открытые порты, в базе данных (SQLite, MySQL или Postgres). Он предоставляет гибкую систему запросов и предупреждений для доступа к этим данным или отображения изменений.

13.9. Создание отчётов HTML

В Nmap нет параметра для сохранения результатов сканирования в HTML, однако получить представление вывода Nmap XML в формате HTML несложно: достаточно открыть XML-файл в веб-браузере. Пример показан на рисунке 13.1 «Вывод XML в веб-браузере».

Как веб-браузер узнаёт, каким образом преобразовать XML в HTML? Файл вывода Nmap XML обычно содержит ссылку на таблицу стилей XSL[6] с именем `nmap.xsl`, которая описывает, как выполняется преобразование.

Инструкция обработки XML, указывающая, где находится таблица стилей, выглядит примерно так:

```
<?xml-stylesheet href="/usr/share/nmap/nmap.xsl" type="text/xsl"?>
```

Точное местоположение может отличаться в зависимости от платформы и способа настройки Nmap.

Такая ссылка на таблицу стилей прекрасно работает при просмотре результатов сканирования на той же машине, где оно было запущено, но не сработает, если XML-файл перенести на другую машину, на которой файл `nmap.xsl` находится в другом месте или вообще отсутствует. Чтобы сделать оформление XML переносимым, передайте Nmap параметр `--webxml`. Тогда инструкция обработки изменится следующим образом:

```
<?xml-stylesheet href="http://nmap.org/data/nmap.xsl" type="text/xsl"?>
```

Полученный XML-файл будет отображаться в виде HTML на любой подключённой к Интернету машине. Такое сетевое размещение чуть менее полезно, но из соображений конфиденциальности по умолчанию используется локальная копия `nmap.xsl`.

Чтобы использовать другую таблицу стилей, укажите параметр `--stylesheet <file>`. Обратите внимание, что `--webxml` - это псевдоним для `--stylesheet http://nmap.org/data/nmap.xsl`.

Чтобы полностью исключить таблицу стилей, используйте параметр `--no-stylesheet`. Тогда веб-браузеры будут показывать вывод как простое, неинтерпретированное дерево XML.

13.9.1. Сохранение постоянного отчёта HTML

Хотя веб-браузеры могут отображать Nmap XML в виде HTML, обычно они не позволяют без труда сохранить созданный HTML в файл. Для этого требуется автономный процессор XSLT. Ниже приведены команды, преобразующие файл вывода Nmap XML в HTML-файл с помощью распространённых процессоров XSLT.

• Saxon[7]

```
java -jar saxon.jar -a <nmap-output.xml> -o <nmap-output.html>
```

• Xalan[8]

```
Xalan -a <nmap-output.xml> -o <nmap-output.html> (using Xalan C++)
```

```
java -jar xalan.jar -IN <nmap-output.xml> -OUT <nmap-output.html> (using Xalan Java)
```

• xsltproc[9]

```
xsltproc <nmap-output.xml> -o <nmap-output.html>
```

13.10. Вывод для grep (-oG)

Этот формат вывода рассматривается последним, поскольку он признан устаревшим. Формат вывода XML гораздо мощнее и почти столь же удобен для опытных пользователей. XML - это стандарт, для которого доступны десятки превосходных анализаторов, тогда как вывод для grep - моя собственная простая разработка. XML расширяем и позволяет поддерживать новые возможности Nmap по мере их появления, а мне часто приходится исключать возможности из вывода для grep из-за отсутствия места для их размещения.

Тем не менее вывод для grep всё ещё весьма популярен. Это простой формат, в котором каждый узел перечисляется в одной строке и который легко искать и разбирать стандартными средствами Unix, такими как grep, awk, cut, sed, diff и Perl. Даже я обычно использую его для разовых проверок, выполняемых из командной строки. Чтобы найти все узлы с открытым портом SSH или работающие под Solaris, достаточно простого grep, переданного по каналу в awk либо cut для печати нужных полей. Одним из поклонников вывода для grep является МэдХэт (MadHat, madhat@unspecific.com), внёсший вклад в этот раздел.

В примере 13.13 показан типичный вывод для grep. Обычно каждый узел занимает лишь одну строку, но я разбил эту запись на семь строк, чтобы она поместилась на странице. Есть также три строки, начинающиеся со знака решётки (не считая строки команды Nmap). Это комментарии, описывающие время запуска Nmap, использованные параметры командной строки, время завершения и статистику. В одной из строк комментария перечислены просканированные номера портов. Я сократил её, чтобы не тратить десятки строк. Этот конкретный комментарий печатается только в подробном режиме (`-v`). Повышение уровня подробности выше единицы дополнительными `-v` уже не меняет вывод для grep. Время и даты заменены обозначением `[time]`, чтобы сократить длину строки.

Пример 13.13. Типичный пример вывода для grep

```
# nmap -oG - -T4 -A -v scanme.nmap.org
# Nmap 4.68 scan initiated [time] as: nmap -oG - -T4 -A -v scanme.nmap.org
# Ports scanned: TCP(1715;1-1027,1029-1033,...,65301) UDP(0;) PROTOCOLS(0;)
Host: 64.13.134.52 (scanme.nmap.org)    Ports: 22/open/tcp//ssh//OpenSSH 4.3 (protocol 2.
    0)/, 25/closed/tcp//smtp///, 53/open/tcp//domain//ISC BIND 9.3.4/, 70/closed/tcp//
    gopher///, 80/open/tcp//http//Apache httpd 2.2.2 ((Fedora))/, 113/closed/tcp//auth///
       Ignored State: filtered (1709)  OS: Linux 2.6.20-1 (Fedora Core 5)  Seq Index: 203
     IP ID Seq: All zeros
# Nmap done at [time] -- 1 IP address (1 host up) scanned in 34.96 seconds
```

В командной строке здесь указано, что вывод для grep следует направить в стандартный вывод посредством аргумента `-` для `-oG`. Также были запрошены агрессивная схема расчёта времени (`-T4`) и определение ОС и версий (`-A`). Строки комментариев не требуют пояснений, поэтому основная часть вывода для grep приходится на строку Host. Если бы я просканировал больше узлов, каждый из доступных имел бы собственную строку Host.

13.10.1. Поля вывода для grep

Строка узла разделена на поля, каждое из которых состоит из имени поля, следующего за ним двоеточия и пробела, а затем содержимого поля. Поля разделяются знаками табуляции (номер девять в ASCII, `'\t'`). В примере 13.13 «Типичный пример вывода для grep» показано шесть полей: Host, Ports, Ignored State, OS, Seq Index и IP ID. Раздел Status включается в сканирования со списком (`-sL`) и ping-сканирования (`-sP`), а раздел Protocols - в сканирования IP-протоколов (`-s0`). Точный набор приведённых полей зависит от использованных параметров Nmap. Например, обнаружение ОС приводит к появлению полей OS, Seq Index и IP ID. Поскольку они разделены табуляцией, поля можно просто разбить строкой Perl:

```
@fields = split("\t", $host_line);
```

В случае примера 13.13 «Типичный пример вывода для grep» массив `@fields` содержал бы шесть элементов, `$fields[0]` имел бы значение `Host: 64.13.134.52 (scanme.nmap.org)`, а `$fields[1]` содержал бы длинное поле Ports. Сценарии, разбирающие вывод для grep, должны игнорировать незнакомые им поля, поскольку для поддержки усовершенствований Nmap могут добавляться новые.

В следующих разделах описаны восемь возможных полей.

Поле Host

Пример: `Host: 64.13.134.52 (scanme.nmap.org)`

Поле Host всегда идёт первым и включается независимо от выбранных параметров Nmap. Оно содержит IP-адрес (адрес IPv6, если был указан `-6`), пробел, а затем имя обратной DNS в скобках. Если обратное имя недоступно, скобки будут пустыми.

Поле Ports

Пример: `Ports: 111/open/tcp//rpcbind (rpcbind V2)/(rpcbind:100000*2-2)/2 (rpc #100000)/, 113/closed/tcp//auth///`

Поле Ports намного сложнее всех остальных, что видно в примере 13.13 «Типичный пример вывода для grep». Оно содержит записи для каждого представляющего интерес порта (тех портов, которые вошли бы в таблицу портов обычного вывода Nmap). Записи портов разделяются запятой и пробелом. Каждая запись порта состоит из семи полей, разделённых прямой косой чертой (`/`). Это номер порта, состояние, протокол, владелец, служба, сведения SunRPC и сведения о версии. Некоторые подполя могут быть пустыми, особенно при простом сканировании портов без определения ОС или версий. Последовательные косые черты в примере 13.13 «Типичный пример вывода для grep» показывают пустые подполя. В Perl их можно разделить так:

```
($port, $state, $protocol, $owner, $service, $rpc_info, $version)
      = split('/', $ports);
```

Вместо этого сведения можно извлечь из командной строки такими командами:

```
cut -d/ -f<fieldnumbers>
awk -F/ '{print $<fieldnumber>}'
```

В других режимах вывода некоторые подполя могут содержать косую черту. Например, веб-сервер с поддержкой SSL отображался бы как `ssl/http`, а сведения о версии могли бы содержать строки наподобие `mod_ssl/2.8.12`. Поскольку косая черта служит разделителем подполей, это нарушило бы разбор. Чтобы избежать проблемы, косые черты, где бы они ни встретились в поле Port, заменяются символом вертикальной черты (`|`).

Анализаторы следует писать так, чтобы они допускали более семи разделённых косыми чертами подполей и просто игнорировали лишние, поскольку будущие усовершенствования Nmap могут потребовать новых подполей. В следующем списке описаны все семь определённых сейчас подполей Port.

• **Номер порта.** Это просто числовой номер порта TCP или UDP.

• **Состояние.** Здесь показывается то же состояние порта, которое появилось бы в таблице портов обычного вывода.

• **Протокол.** Это `tcp` или `udp`.

• **Владелец.** Раньше здесь указывалось имя пользователя, от имени которого работает удалённый сервер, по результатам запроса к серверу identd (`auth`) целевого узла. Ident-сканирование (`-I`) больше недоступно в Nmap, поэтому это поле всегда пусто. Данные ident всё ещё можно получить с помощью сценария NSE `identd-owners.nse`, хотя результаты в это поле не помещаются.

• **Служба.** Имя службы, полученное из `nmap-services` или (что надёжнее) посредством определения версии (`-sV`), если оно было запрошено и завершилось успешно. При включённом определении версий могут встречаться составные записи наподобие `ssl|http` и записи с вопросительным знаком в конце. Их смысл такой же, как в обычном выводе, что рассматривается в главе 7 «Определение версий служб и приложений».

• **Сведения SunRPC.** Если было запрошено определение версии (`-sV`) или RPC-сканирование (`-sR`) и обнаружилось, что порт использует протокол SunRPC, здесь приводятся номер программы RPC и принимаемые номера версий. Типичный пример - `(rpcbind:100000*2-2)`. Данные всегда возвращаются в скобках. Сначала идёт имя программы, затем двоеточие и номер программы, а после этого - звёздочка, за которой следуют разделённые дефисом младший и старший поддерживаемые номера версий. Таким образом, в этом примере rpcbind (программа с номером 100 000) прослушивает порт в ожидании запросов rpcbind версии 2.

• **Сведения о версии.** Если определение версии запрошено и завершается успешно, результаты приводятся здесь в том же формате, что и в интерактивном выводе. Для портов SunRPC данные RPC также печатаются здесь. Формат результатов RPC в этом столбце: `<младший номер версии>-<старший номер версии> (rpc #<номер программы rpc>)`. Если поддерживается только один номер версии, вместо диапазона печатается он один. Порт, для которого в подполе сведений SunRPC показано `(rpcbind:100000*2-2)`, в подполе сведений о версии будет иметь значение `2 (rpc #100000)`.

Поле Protocols

Пример: `Protocols: 1/open/icmp/, 2/open|filtered/igmp/`

При сканировании IP-протоколов (`-sO`) используется поле Protocols, а не Ports. Его содержимое весьма похоже на поле Ports, однако в нём только три подполя вместо семи. Они разделяются косыми чертами, как и в поле Ports. Любые косые черты, которые встретились бы в подполе, также заменяются вертикальными чертами (`|`), как это делается в поле Ports. Подполя - это номер протокола, состояние и имя протокола. Они соответствуют трём полям, показываемым в интерактивном выводе сканирования протоколов. Пример вывода для grep при сканировании IP-протоколов показан в примере 13.14. Строка Host, которая обычно целиком находится в одной строке, здесь перенесена для удобства чтения.

Пример 13.14. Вывод для grep при сканировании IP-протоколов

```
# nmap -v -oG - -sO localhost
# Nmap 4.68 scan initiated [time] as: nmap -v -oG - -sO localhost
```

```
# Ports scanned: TCP(0;) UDP(0;) PROTOCOLS(256;0-255)
Host: 127.0.0.1 (localhost)
      Protocols: 1/open/icmp/, 2/open|filtered/igmp/, 6/open/tcp/,
                 17/open/udp/, 136/open|filtered/udplite/, 255/open|filtered//
      Ignored State: closed (250)
# Nmap done at [time] -- 1 IP address (1 host up) scanned in 2.345 seconds
```

Поле Ignored State

Пример: `Ignored State: filtered (1658)`

Для экономии места Nmap может исключить из списка в поле Ports порты в одном из неоткрытых состояний. Nmap делает это и в интерактивном выводе. Постоянным пользователям Nmap знакомы строки наподобие «The 1658 ports scanned but not shown below are in state: filtered» («1658 просканированных портов, не показанных ниже, находятся в состоянии filtered»). В режиме вывода для grep это состояние приводится в поле Ignored State. За именем состояния следует пробел, а затем в скобках - число портов, обнаруженных в этом состоянии.

Поле OS

Пример: `OS: Linux 2.4.0 - 2.5.20`

Здесь перечисляются все точные совпадения ОС. Если совпадений несколько, они разделяются вертикальной чертой, как показано в примере 13.13 «Типичный пример вывода для grep». Приводятся лишь текстовые описания в свободной форме. Режим вывода для grep не сообщает классификацию по производителю, семейству ОС и типу устройства, показываемую в других режимах вывода.

Поле Seq Index

Пример: `Seq Index: 3004446`

Это число представляет собой оценку трудности атак с предсказанием начального порядкового номера TCP на удалённый узел. Они также известны как атаки с «слепой» подменой и позволяют злоумышленнику подделать полное TCP-соединение с удалённым узлом так, словно оно поступило с какого-либо другого IP-адреса. Это всегда может помочь злоумышленнику скрыть свои следы, а также способно привести к повышению привилегий в отношении таких служб, как rlogin, которые обычно предоставляют дополнительные привилегии доверенным IP-адресам. Значение Seq Index доступно, только когда запрошено определение ОС (`-O`) и соответствующая проверка завершилась успешно. В интерактивном выводе оно сообщается при запросе подробного режима (`-v`). Дополнительные сведения о вычислении и смысле этого значения приведены в главе 8 «Удалённое определение ОС».

Поле IP ID Seq

Пример: `IP ID Seq: All zeros`

Здесь просто описывается алгоритм создания IP ID удалённого узла. Поле доступно, только когда запрошено определение ОС (`-O`) и соответствующая проверка завершилась успешно. Оно также сообщается в интерактивном режиме и рассматривается в главе 8 «Удалённое определение ОС».

Поле Status

Пример: `Status: Up`

В режиме вывода для grep ping-сканирование и сканирование со списком содержат только два поля: Host и Status. Поле Status описывает целевой узел как Up, Down или Unknown. Сканирование со списком всегда относит цели к Unknown, поскольку не выполняет никаких проверок. Ping-сканирование помечает узел как up, если он отвечает хотя бы на один ping-запрос, и как down, если ответов не получено. Раньше оно также сообщало Smurf, если отправленные цели ping-запросы приводили к одному или нескольким ответам от других узлов, но теперь этого не происходит. Недоступные узлы показываются только при включении подробного режима параметром `-v`. В примере 13.15 показано ping-сканирование 100 случайных узлов, а в примере 13.16 - сканирование со списком пяти узлов.

Пример 13.15. Вывод ping-сканирования для grep

```
# nmap -sP -oG - -iR 100
# nmap [version] scan initiated [time] as: nmap -sP -oG - -iR 100
Host: 67.101.77.102 (h-67-101-77-102.nycmny83.covad.net)       Status: Up
Host: 219.93.164.197 ()  Status: Up
Host: 222.113.158.200 ()       Status: Up
Host: 66.130.155.190 (modemcable190.155-130-66.mc.videotron.ca) Status: Up
# Nmap done at [time] -- 100 IP addresses (4 hosts up) scanned in 13.22 seconds
```

Пример 13.16. Вывод сканирования со списком для grep

```
# nmap -sL -oG - -iR 5
# nmap [version] scan initiated [time] as: nmap -sL -oG - -iR 5
Host: 199.223.2.1 ()    Status: Unknown
Host: 191.222.112.87 ()  Status: Unknown
Host: 62.23.21.157 (host.157.21.23.62.rev.coltfrance.com)       Status: Unknown
Host: 138.217.47.127 (CPE-138-217-47-127.vic.bigpond.net.au)    Status: Unknown
Host: 8.118.0.91 ()     Status: Unknown
# Nmap done at [time] -- 5 IP addresses (0 hosts up) scanned in 1.797 seconds
```

13.10.2. Разбор вывода для grep в командной строке

Вывод для grep особенно хорош, когда нужно быстро собрать сведения, не тратясь на написание сценария для разбора XML-вывода. В примере 13.17 показан типичный случай. Требуется найти в сети размером с класс C все узлы с открытым портом 80. Nmap предписано сканировать у каждого узла только этот порт (пропустив этап ping) и вывести отчёт для grep в стандартный вывод. Результаты по каналу передаются простой команде awk, которая находит строки, содержащие `/open/`, и выводит для каждой совпавшей строки второе и третье поля. Эти поля - IP-адрес и имя узла (либо пустые скобки, если имя узла недоступно).

Пример 13.17. Разбор вывода для grep в командной строке

```
> nmap -p80 -PN -oG - 10.1.1.0/24 | awk '/open/{print $2 " " $3}'
10.1.1.72 (userA.corp.foocompany.biz)
10.1.1.73 (userB.corp.foocompany.biz)
10.1.1.75 (userC.corp.foocompany.biz)
10.1.1.149 (admin.corp.foocompany.biz)
10.1.1.152 (printer.corp.foocompany.biz)
10.1.1.160 (10-1-1-160.foocompany.biz)
10.1.1.161 (10-1-1-161.foocompany.biz)
10.1.1.201 (10-1-1-201.foocompany.biz)
10.1.1.254 (10-1-1-254.foocompany.biz)
```

Сноски

[1] [назад] sectools.org
[2] [назад] sourceforge.net
[3] [назад] nmapparser.wordpress.com
[4] [назад] http://sourceforge.net/projects/nmapsql
[5] [назад] http://pbnj.sourceforge.net/
[6] [назад] http://www.w3.org/Style/XSL/
[7] [назад] http://saxon.sourceforge.net/
[8] [назад] http://xalan.apache.org/
[9] [назад] http://xmlsoft.org/XSLT/

Глава 14. Изучение и настройка файлов данных Nmap

14.1. Введение

Для сканирования портов и других операций Nmap опирается на шесть файлов данных, имена всех которых начинаются с `nmap-`. Один из примеров - `nmap-services`, реестр имён портов с соответствующими номерами портов и протоколами. Остальные, поочерёдно описываемые в этой главе, - `nmap-service-probes` (база данных проб для определения версий), `nmap-rpc` (база соответствий имён программ SunRPC и номеров для прямого RPC-сканирования), `nmap-os-db` (база данных определения ОС), `nmap-mac-prefixes` (таблица соответствий префиксов MAC-адресов Ethernet (OUI) и производителей) и `nmap-protocols` (список IP-протоколов для сканирования протоколов). Кроме того, в этой главе рассматриваются некоторые файлы, связанные с написанием сценариев для движка сценариев Nmap. Дистрибутив исходного кода устанавливает эти файлы в `/usr/local/share/nmap/`, а официальные пакеты RPM для Linux помещают их в `/usr/share/nmap/`. Другие дистрибутивы могут устанавливать их в другом месте.

Последние версии этих файлов хранятся по адресу http://nmap.org/data/, хотя пользователям настоятельно рекомендуется обновиться до самой новой версии Nmap, а не брать более свежие файлы данных по отдельности. Нет никаких гарантий, что новые файлы будут работать со старыми версиями Nmap (хотя почти всегда они работают), а получившиеся версии Nmap-Франкенштейна способны внести путаницу в процесс отправки отпечатков операционных систем и служб.

Большинство пользователей никогда не меняют файлы данных, однако это может пригодиться опытным пользователям, которые захотят добавить отпечаток версии или назначение порта для нестандартной фоновой службы, работающей в их компании. В этом разделе описывается каждый файл и обычные способы его изменения. Затем рассматривается общий механизм замены файлов данных Nmap нестандартными версиями. Пара файлов не связана со сканированием портов напрямую, но для удобства все они рассматриваются здесь.

14.2. Список общеизвестных портов: nmap-services

Файл `nmap-services` представляет собой реестр имён портов с соответствующими номерами и протоколами. В каждой записи имеется число, отражающее вероятность обнаружить этот порт открытым. В большинстве строк также есть комментарий. Nmap игнорирует комментарии, но пользователи иногда ищут их в файле с помощью grep, когда Nmap сообщает об открытой службе незнакомого им типа. В примере 14.1 показан типичный фрагмент файла. Для удобства чтения добавлены пробелы, выравнивающие текст.

Пример 14.1. Фрагмент файла `nmap-services`

```
qotd        17/tcp   0.002346  # Quote of the Day
qotd        17/udp   0.009209  # Quote of the Day
msp         18/udp   0.000610  # Message Send Protocol
chargen     19/tcp   0.002559  # ttytst source Character Generator
chargen     19/udp   0.015865  # ttytst source Character Generator
ftp-data    20/tcp   0.001079  # File Transfer [Default Data]
ftp-data    20/udp   0.001878  # File Transfer [Default Data]
ftp         21/tcp   0.197667  # File Transfer [Control]
ftp         21/udp   0.004844  # File Transfer [Control]
```

```
ssh         22/tcp   0.182286  # Secure Shell Login
ssh         22/udp   0.003905  # Secure Shell Login
telnet      23/tcp   0.221265
telnet      23/udp   0.006211
priv-mail   24/tcp   0.001154  # any private mail system
priv-mail   24/udp   0.000329  # any private mail system
smtp        25/tcp   0.131314  # Simple Mail Transfer
smtp        25/udp   0.001285  # Simple Mail Transfer
```

Первоначально этот файл был основан на списке назначенных IANA портов по адресу http://www.iana.org/assignments/port-numbers, хотя за прошедшие годы в него добавили множество других портов. IANA не отслеживает трояны, червей и тому подобное, но их обнаружение важно для многих пользователей Nmap.

Грамматика этого файла довольно проста. В нём три столбца, разделённых пробельными символами. Первый - имя или сокращение службы, показываемое в столбце SERVICE вывода Nmap. Во втором столбце приводятся разделённые косой чертой номер порта и протокол. Такой синтаксис виден в столбце PORT вывода Nmap. Третий столбец - «частота порта», показатель того, как часто порт обнаруживался открытым при исследовательских сканированиях Интернета. Если она не указана, частота равна нулю. Nmap не учитывает ничего после третьего столбца, но в большинстве строк за ним идут пробельные символы, знак решётки (#) и комментарий. Строки могут быть пустыми или содержать только знак решётки с последующим комментарием.

Внимательные читатели заметят сходство структуры `nmap-services` и `/etc/services` (в Windows он обычно находится по адресу `C:\windows\system32\drivers\etc\services`). Это не совпадение. Формат сохранили, чтобы системные администраторы могли копировать любые нестандартные записи из собственного файла `/etc/services` или даже полностью заменить его своей версией. Формат `/etc/services` допускает третий столбец с псевдонимами службы. Это противоречило бы использованию третьего столбца для частоты порта, поэтому содержимое этого столбца игнорируется, если оно не является числом.

Пример 14.1 показывает, что порты UDP часто регистрируются для служб, работающих только по TCP, таких как SSH и FTP. Это унаследовано от IANA, которая склонна всегда регистрировать службы для обоих протоколов. Лишние записи не вредят, поскольку по умолчанию Nmap сканирует порты с наибольшими частотами, а низкочастотные порты просто пропускаются. Кроме того, хотя это может оказаться неожиданным, фрагмент показывает, что иногда UDP-аналоги популярных TCP-портов действительно бывают открыты.

Администраторы иногда изменяют этот файл, чтобы отразить нестандартные службы, работающие в их сети. Например, одна компания интерактивных услуг, которую я когда-то консультировал, запускала десятки различных нестандартных фоновых служб на портах с большими номерами. Благодаря этому Nmap может отображать результаты для таких портов с правильными именами, а не как `unknown`. Помните: если добавить записи без значения частоты порта, их частота будет считаться равной нулю, поэтому по умолчанию порт сканироваться не будет. Чтобы гарантировать сканирование всех именованных портов, используйте параметр наподобие `-p [1-65535]`.

Аналогично, некоторый зарегистрированный порт может часто указываться неверно для определённой организации. `nmap-services` может сопоставить сочетанию номера порта и протокола только одно имя службы, но иногда одним и тем же номером порта по умолчанию пользуются приложения нескольких разных типов. В таком случае я стараюсь выбрать для `nmap-services` самое популярное. Организации, которые обычно используют на таком номере порта другую службу, могут соответствующим образом изменить файл.

Службы, относящиеся только к одной организации, как правило, должны оставаться в её собственном `nmap-services`, но другие регистрации портов могут принести пользу всем. Если вы обнаружите, что порт по умолчанию для крупного червя, трояна, файлообменного приложения или другой службы отсутствует в последнем `nmap-services`, отправьте его мне

(<fyodor@insecure.org>) для включения в следующий выпуск. Это помогает всем пользователям, одновременно избавляя вас от необходимости сопровождать и обновлять собственную нестандартную версию nmap-services.

Ещё одна распространённая настройка - сократить nmap-services, оставив только самые распространённые и необходимые организации службы. Без спецификации портов Nmap не будет сканировать порты, отсутствующие в файле служб, поэтому таким образом можно ограничить число сканируемых портов, не передавая параметру -p длинный аргумент. Сокращённый файл обычно следует помещать в нестандартное место, доступное с помощью параметра --datadir или --servicedb, а не туда, где Nmap воспользуется им по умолчанию. Советы по настройке этих файлов, включая способы не позволить обновлениям Nmap стереть изменённые версии, приведены в разделе 14.9 «Использование настроенных файлов данных».

14.3. База данных сканирования версий: nmap-service-probes

Этот файл содержит пробы, которые система определения служб и версий Nmap (параметры -sV или -A) использует при опросе портов, чтобы определить, какая программа прослушивает порт. В примере 14.2 приведён типичный фрагмент.

Пример 14.2. Фрагмент файла nmap-service-probes

```
##############################NEXT PROBE##############################
# DNS Server status request: http://www.rfc-editor.org/rfc/rfc1035.txt
Probe UDP DNSStatusRequest q|\0\0\x10\0\0\0\0\0\0\0\0\0|
ports 53,135
match domain m|^\0\0\x90\x04\0\0\0\0\0\0\0\0|
# This one below came from 2 tested Windows XP boxes
match msrpc m|^\x04\x06\0\0\x10\0\0\0\0\0\0\0|
[...]
##############################NEXT PROBE##############################
Probe UDP Help q|help\r\n\r\n|
ports 7,13,37
match chargen m|@ABCDEFGHIJKLMNOPQRSTUVWXYZ|
match echo m|^help\r\n\r\n$|
match time m|^[\xc0-\xc5]....$|
```

Грамматика этого файла полностью описана в главе 7 «Определение версий служб и приложений». Хотя nmap-service-probes сложнее, чем nmap-services, польза от его совершенствования также может быть больше. Nmap можно научить действительно распознавать нестандартные службы компании, а не просто угадывать их по регистрации портов в nmap-services.

Кроме того, некоторые администраторы применяют определение версий к задачам, далеко выходящим за пределы его первоначального назначения. Короткая проба может заставить Nmap напечатать заголовок веб-страницы, распознать заражённые червями машины, найти открытые прокси и многое другое. Практический пример приведён в разделе 7.9 «РЕШЕНИЕ: настройка определения версий для нестандартных задач, таких как обнаружение открытых прокси».

14.4. Номера SunRPC: nmap-rpc

Как и nmap-services, файл nmap-rpc просто сопоставляет номера с именами. В данном случае номера программ SunRPC сопоставляются с именами использующих их программ. В примере 14.3 приведён типичный фрагмент.

Пример 14.3. Фрагмент файла nmap-rpc

```
rpcbind     100000  portmap sunrpc rpcbind
rstatd      100001  rstat rup perfmeter rstat_svc
rusersd     100002  rusers
nfs         100003  nfsprog nfsd
ypserv      100004  ypprog
mountd      100005  mount showmount
rpc.operd   100080  opermsg           # Sun Online-Backup
# DMFE/DAWS (Defense Automated Warning System)
#
Gqsrv       200034  gqsrv
Ppt         200035  ppt
Pmt         200036  pmt
```

Для Nmap важны только первые два разделённых пробелами столбца - имя и номер программы. Псевдонимы и комментарии, которые могут идти после них, не рассматриваются. Допускаются пустые строки и строки, начинающиеся с комментария со знаком решётки. Этот формат совпадает с используемым файлом `/etc/rpc` в Unix, поэтому при желании администраторы могут использовать вместо него этот файл.

`nmap-rpc` применяется только функцией RPC grinding в описаниях версий Nmap. Эта функция рассматривается в разделе 7.5.2 «RPC grinding».

Пользователи редко изменяют `nmap-rpc`. Если они всё же это делают, то обычно для добавления нестандартной службы либо общедоступной службы, отсутствующей в последнем `nmap-rpc`. Во втором случае сообщите мне по адресу <fyodor@insecure.org>, чтобы я мог добавить её в следующую версию. Как и с `nmap-services`, некоторые администраторы сокращают файл, удаляя малоизвестные программы RPC ради экономии времени сканирования. Действует то же предупреждение: укажите сокращённый `nmap-rpc` с помощью параметра `--datadir`, а не устанавливайте его туда, где он будет использоваться неявно.

14.5. База данных определения ОС Nmap: nmap-os-db

Файл данных `nmap-os-db` содержит сотни примеров того, как различные операционные системы отвечают на специализированные пробы Nmap для определения ОС. Он разделён на блоки, называемые отпечатками, и каждый отпечаток содержит имя операционной системы, её общую классификацию и данные ответов. В примере 14.4 приведён фрагмент файла с парой типичных отпечатков.

Пример 14.4. Фрагмент файла `nmap-os-db`

```
Fingerprint FreeBSD 7.0-CURRENT
Class FreeBSD | FreeBSD | 7.X | general purpose
SEQ(SP=101-10D%GCD=<7%ISR=108-112%TI=RD%II=RI%TS=20|21|22)
OPS(O1=M5B4NW8NNT11%O2=M578NW8NNT11%O3=M280NW8NNT11%O4=M5B4NW8NNT11%
    O5=M218NW8NNT11%O6=M109NNT11)
WIN(W1=FFFF%W2=FFFF%W3=FFFF%W4=FFFF%W5=FFFF%W6=FFFF)
ECN(R=Y%DF=Y%T=40%TG=40%W=FFFF%O=M5B4NW8%CC=N%Q=)
T1(R=Y%DF=Y%T=40%TG=40%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=N)
T3(R=Y%DF=Y%T=40%TG=40%W=FFFF%S=O%A=S+%F=AS%O=M109NW8NNT11%RD=0%Q=)
T4(R=Y%DF=Y%T=40%TG=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T5(R=Y%DF=Y%T=40%TG=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%TG=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%TG=40%W=0%S=Z%A=S%F=AR%O=%RD=0%Q=)
U1(DF=N%T=40%TG=40%TOS=0%IPL=38%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUL=G%RUD=G)
IE(DFI=S%T=40%TG=40%TOSI=S%CD=S%SI=S%DLI=S)

Fingerprint Linux 2.6.11 - 2.6.20
Class Linux | Linux | 2.6.X | general purpose
SEQ(SP=B9-CF%GCD=<7%ISR=C4-D7%TI=Z%II=I%TS=7)
OPS(O1=M5B4ST11NW1%O2=M5B4ST11NW1%O3=M5B4NNT11NW1%O4=M5B4ST11NW1%
```

```
    O5=M5B4ST11NW1%O6=M5B4ST11)
WIN(W1=16A0%W2=16A0%W3=16A0%W4=16A0%W5=16A0%W6=16A0)
ECN(R=Y%DF=Y%T=40%TG=40%W=16D0%O=M5B4NNSNW1%CC=N%Q=)
T1(R=Y%DF=Y%T=40%TG=40%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=N)
T3(R=Y%DF=Y%T=40%TG=40%W=16A0%S=O%A=S+%F=AS%O=M5B4ST11NW1%RD=0%Q=)
T4(R=Y%DF=Y%T=40%TG=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T5(R=Y%DF=Y%T=40%TG=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%TG=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%TG=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
U1(DF=N%T=40%TG=40%TOS=C0%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUL=G%RUD=G)
IE(DFI=N%T=40%TG=40%TOSI=S%CD=S%SI=S%DLI=S)
```

К базе данных ОС `nmap-os-db` обращаются, когда параметром `-O` запрошено удалённое определение ОС. Вкратце, Nmap отправляет целевой системе специальные пробы и сравнивает ответы с записями базы данных ОС. Если находится совпадение, запись базы данных, вероятно, описывает целевую систему. Процесс определения ОС полностью описан в главе 8 «Удалённое определение ОС». Подробное описание формата эталонного отпечатка см. в разделе 8.5.1 «Расшифровка формата отпечатка исследуемой системы».

Пользователи редко изменяют `nmap-os-db`. Добавление или изменение отпечатка - процесс умеренной сложности, а причин удалять отпечатки обычно вообще нет. Лучший способ получить обновлённую версию базы данных ОС - установить последний выпуск Nmap.

В базе данных ОС (пока) нет сведений обо всех когда-либо созданных сетевых операционных системах. База растёт благодаря вкладу пользователей Nmap. Если Nmap не может угадать ОС, но вы знаете, что это за система, отправьте отпечаток, следуя указаниям раздела 8.7.2 «Когда Nmap не находит совпадение и печатает отпечаток». Иногда отпечатки содержат ошибки или устаревают. Если вы заметите это, подумайте об отправке исправления, как описано в разделе 8.7.1 «Когда Nmap ошибается».

Совершенствование базы данных приносит пользу всем, а отправка ваших улучшений избавляет вас от необходимости сопровождать собственное ответвление файла.

14.6. Префиксы производителей MAC-адресов: nmap-mac-prefixes

Пользователи редко изменяют этот файл, который сопоставляет префиксы MAC-адресов с именами производителей. Далее он рассматривается полностью.

Каждое устройство Ethernet, ставшего преобладающим типом сетевого интерфейса, программируется с уникальным 48-битным идентификатором, известным как MAC-адрес. Этот адрес помещается в заголовки Ethernet, чтобы указать, какая машина в локальной сети отправила пакет и какой машине пакет предназначен. Люди обычно представляют его шестнадцатеричной строкой, например `00:60:1D:38:32:90`.

Чтобы обеспечить уникальность MAC-адресов в мире с тысячами производителей, IEEE назначает каждой компании, выпускающей устройства Ethernet, уникальный идентификатор организации (Organizationally Unique Identifier, OUI). Компания обязана использовать собственный OUI в качестве первых трёх байтов MAC-адресов выпускаемого оборудования. Например, OUI адреса `00:60:1D:38:32:90` - `00601D`. Остальные три байта компания может выбирать по своему усмотрению, пока они остаются уникальными. Самый простой подход - счётчик. Компании, назначившие все 16,8 миллиона возможных значений, могут получить дополнительные OUI. `nmap-mac-prefixes` сопоставляет каждый назначенный OUI с именем продающего их производителя. В примере 14.5 приведён типичный фрагмент.

Пример 14.5. Фрагмент файла `nmap-mac-prefixes`

```
006017 Tokimec
006018 Stellar ONE
006019 Roche Diagnostics
00601A Keithley Instruments
00601B Mesa Electronics
00601C Telxon
00601D Lucent Technologies
00601E Softlab
00601F Stallion Technologies
006020 Pivotal Networking
006021 DSC
006022 Vicom Systems
006023 Pericom Semiconductor
006024 Gradient Technologies
006025 Active Imaging PLC
006026 Viking Components
```

Первое значение - трёхбайтовый OUI в виде шести шестнадцатеричных цифр. За ним следует имя компании. Этот файл создаётся простым сценарием Perl из полного списка OUI, доступного по адресу http://standards.ieee.org/regauth/oui/oui.txt. IEEE также предлагает ответы на часто задаваемые вопросы об OUI по адресу http://standards.ieee.org/faqs/OUI.html.

Nmap может определить MAC-адрес узлов в локальной сети Ethernet, считывая заголовки из передаваемых данных. Он использует эту таблицу, чтобы по OUI найти и сообщить имя производителя. Это может пригодиться для примерного определения типа машины, с которой вы имеете дело. Устройство с OUI Cisco, Hewlett Packard или Sun, вероятно, является соответственно маршрутизатором, принтером или SPARCstation. Пример 14.5 «Фрагмент файла `nmap-mac-prefixes`» показывает, что устройство с адресом `00:60:1D:38:32:90` произведено Lucent. В действительности это беспроводная плата Lucent Orinoco в моём ноутбуке.

14.7. Список номеров IP-протоколов: nmap-protocols

Этот файл сопоставляет однобайтовый номер IP-протокола в заголовке IP с соответствующим именем протокола. В примере 14.6 приведён типичный фрагмент.

Пример 14.6. Фрагмент файла `nmap-protocols`

```
hopopt      0    HOPOPT     # IPv6 Hop-by-Hop Option
icmp        1    ICMP       # Internet Control Message
igmp        2    IGMP       # Internet Group Management
ggp         3    GGP        # Gateway-to-Gateway
ip          4    IP         # IP in IP (encapsulation)
st          5    ST         # Stream
tcp         6    TCP        # Transmission Control
cbt         7    CBT        # CBT
egp         8    EGP        # Exterior Gateway Protocol
...
chaos      16    CHAOS      # Chaos
udp        17    UDP        # User Datagram
```

Первые два поля - имя или сокращение протокола и его номер в десятичном формате. Nmap не принимает во внимание ничего после номера протокола. Файл используется для сканирования IP-протоколов, описанного в разделе 5.11 «Сканирование IP-протоколов (`-sO`)». Определено менее 140 протоколов, и пользователи почти никогда не изменяют этот файл. Исходные данные предоставляются IANA по адресу http://www.iana.org/assignments/protocol-numbers.

14.8. Файлы, связанные со сценариями

Сценарии, используемые движком сценариев Nmap, можно считать ещё одним видом файлов данных. Сценарии хранятся в подкаталоге `scripts` одного из каталогов, перечисленных в разделе 14.9 «Использование настроенных файлов данных». Имя каждого файла сценария оканчивается на `.nse`. Все подробности о сценариях см. в главе 9 «Движок сценариев Nmap».

Все файлы в каталоге сценариев, кроме одного, являются исполняемыми сценариями. Исключение - `script.db`. Этот файл представляет собой текстовый кэш сведений о том, к каким категориям относится каждый сценарий. Его не следует изменять напрямую; вместо этого используйте параметр `--script-updatedb`.

Каждый модуль расширения NSE (см. раздел 9.6 «Библиотеки NSE») хранится в одном из двух мест. Расширения, написанные только на Lua, находятся в подкаталоге `nselib` каталога данных Nmap, обычно того же, где расположен `scripts`. Там хранятся такие модули, как `shortport` и `stdnse`, в файлах с именами, оканчивающимися на `.lua`.

14.9. Использование настроенных файлов данных

Любой файл данных Nmap или все они могут быть заменены версиями, настроенными по желанию пользователя. Заменять их можно только целиком - нельзя задать изменения, которые во время выполнения будут объединены с исходными файлами. Когда Nmap ищет каждый файл, он последовательно ищет его по имени во многих каталогах и выбирает первый найденный. Это похоже на то, как оболочка Unix находит программы, которые вы просите выполнить, поочерёдно просматривая каталоги из `PATH`. В следующем списке приведён порядок поиска каталогов Nmap. Из него видно, что файл `nmap-services`, найденный в каталоге, указанном параметром `--datadir`, будет использован вместо файла из `~/.nmap/`, поскольку первый каталог просматривается раньше.

Порядок поиска каталогов файлов данных Nmap

1. Если указан параметр `--datadir`, проверить каталог, переданный ему в качестве аргумента.

2. Если задана переменная среды `NMAPDIR`, проверить этот каталог.

3. Если Nmap работает не в Windows, искать в `~/.nmap` пользователя, запустившего Nmap. Сначала проверяется домашний каталог реального идентификатора пользователя, а затем, если идентификаторы различаются, - эффективного UID.

4. Если Nmap работает в Windows, проверить каталог, в котором находится исполняемый файл Nmap.

5. Проверить скомпилированный в программу каталог `NMAPDATADIR`. В Windows это значение определено как `c:\nmap`, а в Unix - как `<$prefix>/share/nmap`. При стандартной сборке из исходного кода `<$prefix>` равен `/usr/local`, а в пакетах RPM для Linux - `/usr`. Значение `<$prefix>` можно изменить, передав `./configure` параметр `--prefix` при компиляции исходного кода.

6. В последнюю очередь проверяется текущий рабочий каталог оболочки (`.`). Он оставлен напоследок по тем же соображениям безопасности, по которым `.` не должен стоять первым в пути выполнения `PATH` оболочки. В системе с несколькими пользователями злоумышленник может поместить поддельные файлы данных в общий каталог, например `/tmp`. Эти файлы могут иметь неверный формат, из-за чего Nmap пожалуется и завершит работу, либо заставить Nmap пропускать важные порты. Если бы Nmap сначала проверял `.`, другие пользователи, которым случилось запустить Nmap в этом общем каталоге, получили бы поддельные версии. То же может произойти случайно, если непреднамеренно запустить Nmap в каталоге, где оказался файл с именем `nmap-services` (либо один из остальных). Пользователи, действительно желающие, чтобы

Nmap рано проверял текущий каталог, могут на свой страх и риск задать переменной среды NMAPDIR значение ..

Этот список показывает множество вариантов, доступных пользователю при выборе способа замены файла собственной настроенной версией. Обычно я рекомендую помещать настроенные файлы в специальный каталог, имя которого соответствует изменению. Например, сокращённый до ста самых распространённых портов файл nmap-services можно поместить в ~/nmap-fewports. Затем укажите этот каталог параметром --datadir. Так настроенные файлы гарантированно будут применяться только намеренно. Поскольку форматы вывода Nmap в файл включают использованную командную строку Nmap, при последующем просмотре журналов вы будете знать, какие файлы применялись.

Другой вариант - просто изменить оригинал в NMAPDATADIR. Это редко рекомендуется, поскольку при следующем обновлении Nmap изменённый файл, скорее всего, будет перезаписан. Кроме того, это затрудняет использование исходных файлов, если вы подозреваете, что замены вызывают проблему. Также становится трудно сравнить свою версию с оригиналом, чтобы вспомнить внесённые изменения.

Третий вариант - поместить настроенные файлы в каталог Unix ~/.nmap. Разумеется, добавлять следует только изменённые файлы. Остальные, как обычно, будут извлекаться из NMAPDATADIR. Это очень удобно, поскольку Nmap будет неявно использовать настроенные файлы при каждом запуске. Но это может оказаться и недостатком. Пользователи иногда забывают о существовании файлов. Когда они обновляют Nmap до версии с более новыми файлами данных, старые копии из ~/.nmap продолжают использоваться, снижая качество результатов.

Другой вариант - задать переменной среды NMAPDIR каталог с файлами. Это может пригодиться при проверке новой версии Nmap. Предположим, вы получили Nmap версии 4.68, заметили огромный список изменений и решили проверить её, прежде чем заменять текущую заведомо рабочую версию. Можно скомпилировать её в ~/src/nmap-4.68, но при выполнении оттуда Nmap попытается прочитать файлы данных из /usr/local/share/nmap. Это старые версии, поскольку Nmap 4.68 ещё не установлен. Просто задайте NMAPDIR значение ~/src/nmap-4.68, проверяйте сколько душе угодно, а затем выполните make install. Недостаток постоянного использования NMAPDIR состоит в том, что имя каталога не записывается в файлы вывода Nmap, в отличие от использования --datadir.

Глава 15. Справочное руководство Nmap

Имя

nmap - средство исследования сети и сканер безопасности/портов

Синтаксис

```
nmap [<Тип сканирования>...] [<Параметры>] {<спецификация цели>}
```

15.1. Описание

Nmap («Network Mapper», «Сетевой картограф») - средство с открытым исходным кодом для исследования сетей и аудита безопасности. Оно создано для быстрого сканирования крупных сетей, хотя прекрасно работает и с отдельными узлами. Nmap новаторскими способами использует необработанные IP-пакеты, чтобы определить, какие узлы доступны в сети, какие службы (имя и версия приложения) предлагают эти узлы, под управлением каких операционных систем (и версий ОС) они работают, какие виды фильтров пакетов и брандмауэров используются, а также десятки других характеристик. Хотя Nmap обычно применяется для аудита безопасности, многие системные и сетевые администраторы считают его полезным для повседневных задач, таких как инвентаризация сети, управление графиками обновления служб и наблюдение за временем работы узлов или служб.

Вывод Nmap представляет собой список просканированных целей с дополнительными сведениями о каждой из них в зависимости от использованных параметров. Главная среди этих сведений - «таблица представляющих интерес портов». В ней перечислены номер порта и протокол, имя службы и состояние. Состояние бывает `open`, `filtered`, `closed` или `unfiltered`. `open` означает, что приложение на целевой машине прослушивает соединения или пакеты на этом порту. `filtered` означает, что брандмауэр, фильтр или иное сетевое препятствие блокирует порт, поэтому Nmap не может определить, открыт он или закрыт. Порты `closed` не прослушиваются никакими приложениями, хотя могут открыться в любое время. Порты относят к `unfiltered`, когда они отвечают на пробы Nmap, но Nmap не может определить, открыты они или закрыты. Nmap сообщает сочетания состояний `open|filtered` и `closed|filtered`, когда не может определить, какое из двух состояний описывает порт. Таблица портов может также содержать подробные сведения о версиях программного обеспечения, если было запрошено определение версий. Когда запрошено сканирование IP-протоколов (`-sO`), Nmap приводит сведения о поддерживаемых IP-протоколах, а не о прослушиваемых портах.

Помимо таблицы представляющих интерес портов, Nmap может предоставить дополнительные сведения о целях, включая обратные DNS-имена, предположения об операционной системе, типы устройств и MAC-адреса.

Типичное сканирование Nmap показано в примере 15.1. Единственные аргументы Nmap в этом примере - `-A`, включающий определение ОС и версий, сканирование сценариями и трассировку маршрута; `-T4` для более быстрого выполнения; а затем два имени целевых узлов.

Пример 15.1. Характерное сканирование Nmap

```
# nmap -A -T4 scanme.nmap.org

Starting Nmap ( http://nmap.org
Interesting ports on scanme.nmap.org (64.13.134.52):
Not shown: 994 filtered ports
PORT    STATE  SERVICE  VERSION
22/tcp  open   ssh      OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain   ISC BIND 9.3.4
70/tcp  closed gopher
80/tcp  open   http     Apache httpd 2.2.2 ((Fedora))
|_ HTML title: Go ahead and ScanMe!
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.20-1 (Fedora Core 5)

TRACEROUTE (using port 80/tcp)
HOP RTT   ADDRESS
[Cut first seven hops for brevity]
8   10.59 so-4-2-0.mpr3.paol.us.above.net (64.125.28.142)
9   11.00 metro0.sv.svcolo.com (208.185.168.173)
10  9.93  scanme.nmap.org (64.13.134.52)

Nmap done: 1 IP address (1 host up) scanned in 17.00 seconds
```

Самую новую версию Nmap можно получить по адресу http://nmap.org. Последняя версия страницы руководства доступна по адресу http://nmap.org/book/man.html.

15.2. Краткий перечень параметров

Этот краткий перечень параметров печатается при запуске Nmap без аргументов, а последняя версия всегда доступна по адресу http://nmap.org/data/nmap.usage.txt. Он помогает вспомнить самые распространённые параметры, но не заменяет подробную документацию в остальной части этого руководства. Некоторые малоизвестные параметры здесь даже не представлены.

```
Nmap 4.76 ( http://nmap.org )
Usage: nmap [Scan Type(s)] [Options] {target specification}
TARGET SPECIFICATION:
  Can pass hostnames, IP addresses, networks, etc.
  Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254
  -iL <inputfilename>: Input from list of hosts/networks
  -iR <num hosts>: Choose random targets
  --exclude <host1[,host2][,host3],...>: Exclude hosts/networks
  --excludefile <exclude_file>: Exclude list from file
HOST DISCOVERY:
  -sL: List Scan - simply list targets to scan
  -sP: Ping Scan - go no further than determining if host is online
  -PN: Treat all hosts as online -- skip host discovery
  -PS/PA/PU [portlist]: TCP SYN/ACK or UDP discovery to given ports

-PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes
-PO [protocol list]: IP Protocol Ping
-n/-R: Never do DNS resolution/Always resolve [default: sometimes]
--dns-servers <serv1[,serv2],...>: Specify custom DNS servers
--system-dns: Use OS's DNS resolver
SCAN TECHNIQUES:
  -sS/sT/sA/sW/sM: TCP SYN/Connect()/ACK/Window/Maimon scans
  -sU: UDP Scan
  -sN/sF/sX: TCP Null, FIN, and Xmas scans
  --scanflags <flags>: Customize TCP scan flags
  -sI <zombie host[:probeport]>: Idle scan
  -sO: IP protocol scan
  -b <FTP relay host>: FTP bounce scan
```

```
  --traceroute: Trace hop path to each host
  --reason: Display the reason a port is in a particular state
PORT SPECIFICATION AND SCAN ORDER:
  -p <port ranges>: Only scan specified ports
    Ex: -p22; -p1-65535; -p U:53,111,137,T:21-25,80,139,8080
  -F: Fast mode - Scan fewer ports than the default scan
  -r: Scan ports consecutively - don't randomize
  --top-ports <number>: Scan <number> most common ports
  --port-ratio <ratio>: Scan ports more common than <ratio>
SERVICE/VERSION DETECTION:
  -sV: Probe open ports to determine service/version info
  --version-intensity <level>: Set from 0 (light) to 9 (try all probes)
  --version-light: Limit to most likely probes (intensity 2)
  --version-all: Try every single probe (intensity 9)
  --version-trace: Show detailed version scan activity (for debugging)
SCRIPT SCAN:
  -sC: equivalent to --script=default
  --script=<Lua scripts>: <Lua scripts> is a comma separated list of
           directories, script-files or script-categories
  --script-args=<n1=v1,[n2=v2,...]>: provide arguments to scripts
  --script-trace: Show all data sent and received
  --script-updatedb: Update the script database.
OS DETECTION:
  -O: Enable OS detection
  --osscan-limit: Limit OS detection to promising targets
  --osscan-guess: Guess OS more aggressively
TIMING AND PERFORMANCE:
  Options which take <time> are in milliseconds, unless you append 's'
  (seconds), 'm' (minutes), or 'h' (hours) to the value (e.g. 30m).
  -T[0-5]: Set timing template (higher is faster)
  --min-hostgroup/max-hostgroup <size>: Parallel host scan group sizes
  --min-parallelism/max-parallelism <time>: Probe parallelization
  --min-rtt-timeout/max-rtt-timeout/initial-rtt-timeout <time>: Specifies
      probe round trip time.
  --max-retries <tries>: Caps number of port scan probe retransmissions.
  --host-timeout <time>: Give up on target after this long

  --scan-delay/--max-scan-delay <time>: Adjust delay between probes
  --min-rate <number>: Send packets no slower than <number> per second
  --max-rate <number>: Send packets no faster than <number> per second
FIREWALL/IDS EVASION AND SPOOFING:
  -f; --mtu <val>: fragment packets (optionally w/given MTU)
  -D <decoy1,decoy2[,ME],...>: Cloak a scan with decoys
  -S <IP_Address>: Spoof source address
  -e <iface>: Use specified interface
  -g/--source-port <portnum>: Use given port number
  --data-length <num>: Append random data to sent packets
  --ip-options <options>: Send packets with specified ip options
  --ttl <val>: Set IP time-to-live field
  --spoof-mac <mac address/prefix/vendor name>: Spoof your MAC address
  --badsum: Send packets with a bogus TCP/UDP checksum
OUTPUT:
  -oN/-oX/-oS/-oG <file>: Output scan in normal, XML, s|<rIpt kIddi3,
     and Grepable format, respectively, to the given filename.
  -oA <basename>: Output in the three major formats at once
  -v: Increase verbosity level (use twice or more for greater effect)
  -d[level]: Set or increase debugging level (Up to 9 is meaningful)
  --open: Only show open (or possibly open) ports
  --packet-trace: Show all packets sent and received
  --iflist: Print host interfaces and routes (for debugging)
  --log-errors: Log errors/warnings to the normal-format output file
  --append-output: Append to rather than clobber specified output files
  --resume <filename>: Resume an aborted scan
  --stylesheet <path/URL>: XSL stylesheet to transform XML output to HTML
  --webxml: Reference stylesheet from Nmap.Org for more portable XML
  --no-stylesheet: Prevent associating of XSL stylesheet w/XML output
MISC:
  -6: Enable IPv6 scanning
  -A: Enables OS detection and Version detection, Script scanning and Traceroute
  --datadir <dirname>: Specify custom Nmap data file location
  --send-eth/--send-ip: Send using raw ethernet frames or IP packets
```

```
  --privileged: Assume that the user is fully privileged
  --unprivileged: Assume the user lacks raw socket privileges
  -V: Print version number
  -h: Print this help summary page.
EXAMPLES:
  nmap -v -A scanme.nmap.org
  nmap -v -sP 192.168.0.0/16 10.0.0.0/8
  nmap -v -iR 10000 -PN -p 80
SEE THE MAN PAGE FOR MANY MORE OPTIONS, DESCRIPTIONS, AND EXAMPLES
```

15.3. Спецификация целей

Всё, что в командной строке Nmap не является параметром (или аргументом параметра), рассматривается как спецификация целевого узла. Простейший случай - указать для сканирования IP-адрес или имя целевого узла.

Иногда требуется просканировать целую сеть соседних узлов. Для этого Nmap поддерживает адресацию в стиле CIDR. К IP-адресу или имени узла можно добавить `/<число битов>`, и Nmap просканирует все IP-адреса, у которых первые `<число битов>` совпадают с указанным эталонным IP-адресом или именем узла. Например, `192.168.10.0/24` просканирует 256 узлов от `192.168.10.0` (в двоичном виде: `11000000 10101000 00001010 00000000`) до `192.168.10.255` (в двоичном виде: `11000000 10101000 00001010 11111111`) включительно. `192.168.10.40/24` сделает в точности то же самое. Поскольку узел `scanme.nmap.org` имеет IP-адрес `64.13.134.52`, спецификация `scanme.nmap.org/16` просканирует 65 536 IP-адресов от `64.13.0.0` до `64.13.255.255`. Наименьшее допустимое значение - `/0`, сканирующее весь Интернет. Наибольшее значение - `/32`, при котором сканируется только названный узел или IP-адрес, поскольку зафиксированы все биты адреса.

Запись CIDR кратка, но не всегда достаточно гибка. Например, может понадобиться просканировать `192.168.0.0/16`, пропустив все IP-адреса, оканчивающиеся на `.0` или `.255`, поскольку они часто являются широковещательными. Nmap поддерживает это посредством адресации диапазонами октетов. Вместо обычного IP-адреса для каждого октета можно указать разделённый запятыми список чисел или диапазонов. Например, `192.168.0-255.1-254` пропустит все адреса диапазона, оканчивающиеся на `.0` или `.255`. Диапазоны не обязаны ограничиваться последними октетами: спецификатор `0-255.0-255.13.37` выполнит сканирование во всём Интернете всех IP-адресов, оканчивающихся на `13.37`. Такая широкая выборка может пригодиться для обзоров и исследований Интернета.

Адреса IPv6 можно задавать только полностью определённым адресом IPv6 или именем узла. CIDR и диапазоны октетов для IPv6 не поддерживаются, поскольку редко бывают полезны.

Nmap принимает в командной строке несколько спецификаций узлов, и они не обязаны быть одного типа. Команда `nmap scanme.nmap.org 192.168.0.0/16 10.0.0,1,3-7.0-255` делает именно то, чего от неё можно ожидать.

Хотя цели обычно указываются в командной строке, для управления выбором целей также доступны следующие параметры:

• `-iL <входной файл>` **(ввод из списка).** Читает спецификации целей из `<входного файла>`. Передавать в командной строке огромный список узлов зачастую неудобно, хотя такая потребность возникает часто. Например, DHCP-сервер может экспортировать список из 10 000 текущих аренд, которые требуется просканировать. Или, возможно, нужно просканировать все IP-адреса, кроме занятых узлами с несанкционированными статическими IP-адресами. Просто создайте список узлов для сканирования и передайте это имя файла Nmap как аргумент параметра `-iL`. Записи могут иметь любой формат, принимаемый Nmap в командной строке (IP-адрес, имя узла, CIDR, IPv6 или

диапазоны октетов). Каждая запись должна отделяться одним или несколькими пробелами, знаками табуляции или переводами строки. В качестве имени файла можно указать дефис (`-`), если нужно, чтобы Nmap читал узлы из стандартного ввода, а не из настоящего файла.

• `-iR <число узлов>` **(выбор случайных целей).** Для обзоров всего Интернета и других исследований может понадобиться выбирать цели случайным образом. Аргумент `<число узлов>` сообщает Nmap, сколько IP-адресов следует создать. Нежелательные IP-адреса, например относящиеся к определённым частным, многоадресным или нераспределённым диапазонам, автоматически пропускаются. Для бесконечного сканирования можно указать аргумент `0`. Помните, что некоторые сетевые администраторы возмущаются несанкционированным сканированием своих сетей и могут пожаловаться. Используйте этот параметр на свой страх и риск! Если однажды дождливым днём вам станет совсем скучно, попробуйте команду `nmap -sS -PS80 -iR 0 -p 80`, чтобы найти случайные веб-серверы для просмотра.

• `--exclude <узел 1>[,<узел 2>[,...]]` **(исключить узлы/сети).** Задаёт разделённый запятыми список целей, которые необходимо исключить из сканирования, даже если они входят в указанный вами общий диапазон сети. Переданный список использует обычный синтаксис Nmap, поэтому может включать имена узлов, сетевые блоки CIDR, диапазоны октетов и так далее. Это бывает полезно, когда сеть, которую требуется просканировать, содержит неприкосновенные критически важные серверы, системы, которые, как известно, неблагоприятно реагируют на сканирование портов, или подсети, администрируемые другими людьми.

• `--excludefile <файл исключений>` **(исключить список из файла).** Предоставляет ту же функциональность, что и параметр `--exclude`, за исключением того, что исключаемые цели передаются не в командной строке, а в `<файле исключений>` с разделением переводами строки, пробелами или знаками табуляции.

15.4. Обнаружение узлов

Один из самых первых шагов любой операции сетевой разведки - сократить (иногда огромный) набор диапазонов IP-адресов до списка активных или представляющих интерес узлов. Сканировать каждый порт каждого отдельного IP-адреса медленно и обычно не требуется. Разумеется, то, что делает узел интересным, в большой степени зависит от целей сканирования. Сетевых администраторов могут интересовать только узлы, на которых работает определённая служба, тогда как аудиторам безопасности может быть важно каждое устройство с IP-адресом. Администратору может быть достаточно одного ICMP ping для обнаружения узлов во внутренней сети, тогда как внешний специалист по тестированию на проникновение может применять разнообразный набор из десятков проб, пытаясь обойти ограничения брандмауэра.

Поскольку потребности в обнаружении узлов столь разнообразны, Nmap предлагает широкий выбор параметров настройки применяемых методов. Обнаружение узлов иногда называют ping-сканированием, но оно выходит далеко за пределы простых пакетов эхо-запроса ICMP, связанных с повсеместно распространённым средством ping. Пользователи могут полностью пропустить этап ping посредством сканирования со списком (`-sL`) или отключения ping (`-PN`) либо обратиться к сети с произвольными сочетаниями многопортовых проб TCP SYN/ACK, UDP и ICMP. Цель этих проб - вызвать ответы, показывающие, что IP-адрес действительно активен (используется узлом или сетевым устройством). Во многих сетях в любой момент времени активно лишь небольшое число IP-адресов. Особенно часто это встречается в частном адресном пространстве, таком как `10.0.0.0/8`. В этой сети 16 миллионов IP-адресов, но я видел, как её использовали компании с менее чем тысячей машин. Обнаружение узлов позволяет найти эти машины в редко заселённом море IP-адресов.

Если параметры обнаружения узлов не заданы, Nmap отправляет каждой целевой машине пакет TCP ACK, предназначенный порту 80, и эхо-запрос ICMP. Исключение состоит в том, что для всех целей в локальной сети Ethernet применяется ARP-сканирование. Для непривилегированных пользователей оболочки Unix вместо ACK посредством системного вызова `connect` отправляется пакет SYN. Эти значения по умолчанию эквивалентны параметрам `-PA -PE`. Такое обнаружение узлов часто достаточно при сканировании локальных сетей, но для аудита безопасности рекомендуется более полный набор проб обнаружения.

Параметры `-P*` (выбирающие типы ping) можно сочетать. Вероятность прохождения через строгие брандмауэры можно повысить, отправляя пробы многих типов с различными портами и флагами TCP и кодами ICMP. Также обратите внимание, что обнаружение ARP (`-PR`) по умолчанию выполняется для целей в локальной сети Ethernet, даже если указаны другие параметры `-P*`, поскольку оно почти всегда быстрее и эффективнее.

По умолчанию Nmap обнаруживает узлы, а затем выполняет сканирование портов каждого узла, который определён как работающий. Это верно, даже если указать нестандартные типы обнаружения узлов, например UDP-пробы (`-PU`). Чтобы узнать, как выполнить только обнаружение узлов, прочитайте описание параметра `-sP`, либо используйте `-PN`, чтобы пропустить обнаружение узлов и просканировать порты всех целевых узлов. Обнаружением узлов управляют следующие параметры:

• `-sL` **(сканирование со списком).** Сканирование со списком - вырожденная форма обнаружения узлов, которая просто перечисляет каждый узел указанных сетей, не отправляя никаких пакетов целевым узлам. По умолчанию Nmap всё же выполняет обратное разрешение DNS для узлов, чтобы узнать их имена. Зачастую удивительно, сколько полезных сведений раскрывают простые имена узлов.

Например, `fw.chi` - имя чикагского брандмауэра одной компании. В конце Nmap также сообщает общее число IP-адресов. Сканирование со списком хорошо подходит для проверки здравого смысла и позволяет убедиться, что IP-адреса целей указаны правильно. Если у узлов имеются незнакомые вам доменные имена, стоит провести дальнейшее расследование, чтобы не просканировать сеть другой компании.

Поскольку задача состоит всего лишь в печати списка целевых узлов, с этим режимом нельзя сочетать параметры функций более высокого уровня, таких как сканирование портов, определение ОС или ping-сканирование. Если нужно отключить ping-сканирование, продолжая выполнять подобные функции более высокого уровня, прочитайте о параметре `-PN`.

• `-sP` **(ping-сканирование).** Этот параметр предписывает Nmap выполнить только ping-сканирование (обнаружение узлов), а затем напечатать доступные узлы, ответившие на сканирование. Если запрошены трассировка маршрута и сценарии NSE для узлов, они также выполняются, но дальнейшие проверки (такие как сканирование портов или определение ОС) не производятся. По умолчанию это на один шаг навязчивее сканирования со списком и часто может применяться для тех же целей. Оно позволяет провести лёгкую разведку целевой сети, не привлекая особого внимания. Сведения о количестве работающих узлов ценнее для злоумышленников, чем предоставляемый сканированием со списком перечень всех отдельных IP-адресов и имён узлов.

Системные администраторы также часто считают этот параметр полезным. С его помощью легко подсчитать доступные машины в сети или следить за доступностью сервера. Это часто называют ping sweep, и данный способ надёжнее отправки ping на широковещательный адрес, поскольку многие узлы не отвечают на широковещательные запросы.

По умолчанию параметр `-sP` отправляет эхо-запрос ICMP и пакет TCP ACK на порт 80. При выполнении непривилегированным пользователем на порт 80 цели отправляется только пакет SYN

(посредством вызова connect). Когда привилегированный пользователь пытается сканировать цели в локальной сети Ethernet, применяются запросы ARP, если не указан параметр --send-ip. Для большей гибкости -sP можно сочетать с любыми типами проб обнаружения (параметрами -P*, кроме -PN). Если используются какие-либо из этих параметров типов проб и номеров портов, стандартные пробы (ACK и эхо-запрос) переопределяются. При наличии строгих брандмауэров между исходным узлом, где работает Nmap, и целевой сетью рекомендуется применять эти расширенные методы. В противном случае узлы могут быть пропущены, когда брандмауэр отбрасывает пробы или ответы на них.

• -PN **(без ping).** Этот параметр полностью пропускает этап обнаружения Nmap. Обычно Nmap использует этот этап, чтобы определить активные машины для более тяжёлого сканирования. По умолчанию Nmap выполняет тяжёлые проверки, такие как сканирование портов, определение версий или определение ОС, только для узлов, признанных работающими. Отключение обнаружения узлов с помощью -PN заставляет Nmap попытаться выполнить запрошенные функции сканирования для каждого указанного целевого IP-адреса. Поэтому, если в командной строке указано целевое адресное пространство размером с класс B (/16), сканируются все 65 536 IP-адресов. Нормальное обнаружение узлов пропускается, как при сканировании со списком, но вместо остановки и печати списка целей Nmap продолжает выполнять запрошенные функции так, словно каждый целевой IP-адрес активен. Для машин в локальной сети Ethernet ARP-сканирование всё равно выполняется (если не указан --send-ip), поскольку Nmap нужны MAC-адреса для дальнейшего сканирования целевых узлов. Раньше этот флаг назывался P0 (с нулём), но его переименовали, чтобы избежать путаницы с флагом PO (с буквой O) для ping по протоколу.

• -PS <список портов> **(ping TCP SYN).** Этот параметр отправляет пустой пакет TCP с установленным флагом SYN. Порт назначения по умолчанию - 80 (его можно настроить во время компиляции, изменив DEFAULT_TCP_PROBE_PORT_SPEC в nmap.h). В качестве параметра можно указать другие порты. Синтаксис такой же, как для -p, за исключением того, что спецификаторы типа порта наподобие T: не допускаются. Примеры: -PS22 и -PS22-25,80,113,1050,35000. Обратите внимание, что между -PS и списком портов не должно быть пробела. Если указано несколько проб, они будут отправлены параллельно.

Флаг SYN сообщает удалённой системе, что вы пытаетесь установить соединение. Обычно порт назначения будет закрыт и в ответ отправится пакет RST (сброс). Если порт окажется открытым, цель выполнит второй шаг трёхэтапного рукопожатия TCP, ответив пакетом SYN/ACK. Затем машина, на которой работает Nmap, разрывает зарождающееся соединение, отвечая RST вместо отправки пакета ACK, который завершил бы трёхэтапное рукопожатие и установил полноценное соединение. Пакет RST отправляет ядро машины, где работает Nmap, в ответ на неожиданный SYN/ACK, а не сам Nmap.

Для Nmap неважно, открыт порт или закрыт. И ответ RST, и рассмотренный выше SYN/ACK сообщают Nmap, что узел доступен и отвечает.

На машинах Unix отправлять и получать необработанные пакеты TCP обычно может только привилегированный пользователь root. Для непривилегированных пользователей автоматически применяется обходной путь: для каждого целевого порта инициируется системный вызов connect. В результате целевому узлу отправляется пакет SYN в попытке установить соединение. Если connect быстро завершается успешно либо с ошибкой ECONNREFUSED, нижележащий стек TCP должен был получить SYN/ACK или RST, и узел помечается как доступный. Если попытка соединения остаётся в ожидании до истечения времени, узел помечается как недоступный. Этот обходной путь также применяется для соединений IPv6, поскольку Nmap пока не поддерживает создание необработанных пакетов IPv6.

• -PA <список портов> **(ping TCP ACK).** Ping TCP ACK весьма похож на только что рассмотренный ping SYN. Различие, как нетрудно догадаться, состоит в том, что вместо флага SYN

устанавливается флаг TCP ACK. Такой пакет ACK якобы подтверждает данные в установленном TCP-соединении, но никакого соединения не существует. Поэтому удалённые узлы всегда должны отвечать пакетом RST, попутно раскрывая своё существование.

Параметр `-PA` использует тот же стандартный порт, что и проба SYN (80), и также может принимать список портов назначения в том же формате. Если его пытается использовать непривилегированный пользователь или указана цель IPv6, применяется рассмотренный выше обходной путь с `connect`. Этот обход несовершенен, поскольку `connect` в действительности отправляет пакет SYN, а не ACK.

Пробы ping SYN и ACK предлагаются вместе, чтобы максимально повысить вероятность обхода брандмауэров. Многие администраторы настраивают маршрутизаторы и другие простые брандмауэры так, чтобы блокировать входящие пакеты SYN, кроме предназначенных общедоступным службам, таким как веб-сайт или почтовый сервер компании. Это предотвращает другие входящие соединения с организацией, одновременно позволяя пользователям беспрепятственно устанавливать исходящие соединения с Интернетом. Такой подход без отслеживания состояния требует мало ресурсов брандмауэра или маршрутизатора и широко поддерживается аппаратными и программными фильтрами. Брандмауэр Linux Netfilter/iptables предлагает удобный параметр `--syn` для реализации этого подхода без отслеживания состояния. При наличии подобных правил пробы ping SYN (`-PS`), отправленные закрытым целевым портам, скорее всего, будут заблокированы. В таких случаях проба ACK показывает себя блестяще, проходя прямо сквозь эти правила.

Другой распространённый вид брандмауэра использует правила с отслеживанием состояния, которые отбрасывают неожиданные пакеты. Изначально эта возможность встречалась главным образом в дорогих брандмауэрах, однако с годами стала гораздо более распространённой. Система Linux Netfilter/iptables поддерживает её посредством параметра `--state`, который классифицирует пакеты по состоянию соединения. Проба SYN с большей вероятностью сработает против такой системы, поскольку неожиданные пакеты ACK обычно распознаются как поддельные и отбрасываются. Решение этой дилеммы - отправлять и пробы SYN, и пробы ACK, указав `-PS` и `-PA`.

• `-PU <список портов>` **(ping UDP).** Ещё один параметр обнаружения узлов - ping UDP, который отправляет указанным портам пустой пакет UDP (если не задан `--data-length`). Список портов имеет тот же формат, что и у рассмотренных выше параметров `-PS` и `-PA`. Если порты не указаны, по умолчанию используется 31338. Это значение можно настроить во время компиляции, изменив `DEFAULT_UDP_PROBE_PORT_SPEC` в `nmap.h`. По умолчанию используется крайне необычный порт, поскольку для данного типа сканирования зачастую нежелательно отправлять пакеты открытым портам.

Достигнув закрытого порта целевой машины, UDP-проба должна вызвать ответный ICMP-пакет «порт недоступен». Для Nmap это означает, что машина работает и доступна. Многие другие виды ошибок ICMP, такие как недоступность узла или сети либо превышение TTL, указывают на неработающий или недоступный узел. Отсутствие ответа трактуется так же. Если достигнут открытый порт, большинство служб просто игнорируют пустой пакет и не возвращают никакого ответа. Именно поэтому стандартный порт пробы - 31338, который с очень малой вероятностью окажется занят. Некоторые службы, такие как протокол Character Generator (`chargen`), ответят на пустой пакет UDP и тем самым раскроют Nmap доступность машины.

Главное преимущество этого типа сканирования состоит в обходе брандмауэров и фильтров, проверяющих только TCP. Например, когда-то у меня был беспроводной широкополосный маршрутизатор Linksys BEFW11S4. Внешний интерфейс этого устройства по умолчанию фильтровал все порты TCP, но UDP-пробы всё равно вызывали сообщения «порт недоступен» и тем самым выдавали устройство.

• `-PE; -PP; -PM` **(типы ping ICMP).** Помимо рассмотренных выше необычных типов обнаружения узлов TCP и UDP, Nmap может отправлять стандартные пакеты, посылаемые повсеместно распространённой программой ping. Nmap отправляет целевым IP-адресам пакет ICMP типа 8 (эхо-запрос), ожидая от доступных узлов ответ типа 0 (эхо-ответ). К сожалению для исследователей сетей, многие узлы и брандмауэры теперь блокируют эти пакеты, вместо того чтобы отвечать, как требует RFC 1122. Поэтому сканирования только с ICMP редко бывают достаточно надёжны для неизвестных целей в Интернете. Но для системных администраторов, наблюдающих за внутренней сетью, они могут стать практичным и эффективным подходом. Чтобы включить эхо-запросы, используйте параметр `-PE`.

Хотя эхо-запрос является стандартным запросом ping ICMP, Nmap им не ограничивается. Стандарт ICMP (RFC 792) также определяет пакеты запроса временной метки, запроса сведений и запроса маски адреса с кодами 13, 15 и 17 соответственно. Хотя заявленное назначение этих запросов - получение таких сведений, как маски адресов и текущее время, их легко использовать для обнаружения узлов. Ответившая система работает и доступна. Сейчас Nmap не реализует пакеты запроса сведений, поскольку они не получили широкой поддержки. RFC 1122 настаивает: «узел НЕ ДОЛЖЕН реализовывать эти сообщения». Запросы временной метки и маски адреса можно отправлять параметрами `-PP` и `-PM` соответственно. Ответ с временной меткой (код ICMP 14) или ответ с маской адреса (код 18) раскрывает доступность узла. Эти два запроса могут быть полезны, когда администраторы специально блокируют пакеты эхо-запросов, забывая, что для той же цели можно использовать другие запросы ICMP.

• `-PO <список протоколов>` **(ping IP-протокола).** Самый новый параметр обнаружения узлов - ping IP-протокола, отправляющий IP-пакеты с указанным номером протокола в заголовке IP. Список протоколов имеет тот же формат, что и списки портов у рассмотренных выше параметров обнаружения узлов TCP и UDP. Если протоколы не указаны, по умолчанию отправляется несколько IP-пакетов для ICMP (протокол 1), IGMP (протокол 2) и IP-in-IP (протокол 4). Протоколы по умолчанию можно настроить во время компиляции, изменив `DEFAULT_PROTO_PROBE_PORT_SPEC` в `nmap.h`. Обратите внимание, что для ICMP, IGMP, TCP (протокол

6. и UDP (протокол 17) пакеты отправляются с правильными заголовками протоколов, тогда как пакеты остальных протоколов не содержат дополнительных данных после заголовка IP (если не указан параметр `--data-length`).

Этот метод обнаружения узлов ищет либо ответы по тому же протоколу, что и проба, либо сообщения ICMP «протокол недоступен», означающие, что указанный протокол не поддерживается на целевом узле. Ответ любого типа означает, что целевой узел работает.

• `-PR` **(ping ARP).** Один из самых распространённых сценариев использования Nmap - сканирование локальной сети Ethernet. В большинстве локальных сетей, особенно использующих частные диапазоны адресов, определённые RFC 1918, подавляющее большинство IP-адресов в любой момент не используется. Когда Nmap пытается отправить необработанный IP-пакет, например эхо-запрос ICMP, операционная система должна определить аппаратный адрес назначения (ARP), соответствующий целевому IP-адресу, чтобы правильно адресовать кадр Ethernet. Это часто медленно и проблематично, поскольку операционные системы не создавались в расчёте на необходимость за короткое время выполнить миллионы запросов ARP к недоступным узлам.

При ARP-сканировании запросами ARP управляют Nmap и его оптимизированные алгоритмы. Получив ответ, Nmap уже не нужно заботиться о пакетах ping на основе IP, поскольку он знает, что узел работает. Благодаря этому ARP-сканирование намного быстрее и надёжнее сканирований на основе IP. Поэтому оно выполняется по умолчанию при сканировании узлов Ethernet, которые Nmap определяет как находящиеся в локальной сети Ethernet. Даже если указаны другие типы ping (например, `-PE` или `-PS`), для всех целей в той же локальной сети Nmap вместо них использует ARP. Если вы совершенно не хотите выполнять ARP-сканирование, укажите `--send-ip`.

• `--traceroute` **(трассировка пути к узлу).** Трассировки маршрута выполняются после сканирования с использованием сведений из его результатов для определения порта и протокола, которые с наибольшей вероятностью достигнут цели. Это работает со всеми типами сканирования, кроме сканирования с соединением (`-sT`) и сканирования через незанятый узел (`-sI`). Все трассировки используют динамическую модель расчёта времени Nmap и выполняются параллельно.

Трассировка маршрута работает, отправляя пакеты с малым TTL (временем жизни) в попытке вызвать сообщения ICMP Time Exceeded от промежуточных переходов между сканером и целевым узлом. Стандартные реализации traceroute начинают с TTL, равного 1, и увеличивают его, пока не будет достигнут целевой узел. Трассировка Nmap начинает с большого TTL, а затем уменьшает его до нуля. Обратное направление позволяет Nmap применять хитроумные алгоритмы кэширования, чтобы ускорить трассировки нескольких узлов. В среднем Nmap отправляет на 5-10 пакетов меньше для каждого узла в зависимости от состояния сети. Если сканируется одна подсеть (например, `192.168.0.0/24`), Nmap, возможно, придётся отправить большинству узлов только по одному пакету.

• `-n` **(без разрешения DNS).** Предписывает Nmap никогда не выполнять обратное разрешение DNS для обнаруженных активных IP-адресов. Поскольку DNS может работать медленно даже со встроенным в Nmap параллельным упрощённым преобразователем, этот параметр способен резко сократить время сканирования.

• `-R` **(разрешение DNS для всех целей).** Предписывает Nmap всегда выполнять обратное разрешение DNS целевых IP-адресов. Обычно обратное разрешение DNS производится только для отвечающих (работающих) узлов.

• `--system-dns` **(использовать системный преобразователь DNS).** По умолчанию Nmap разрешает IP-адреса, отправляя запросы непосредственно настроенным на вашем узле серверам имён и затем прослушивая ответы. Для повышения производительности параллельно выполняется множество запросов (часто десятки). Укажите этот параметр, чтобы вместо этого использовать системный преобразователь (по одному IP-адресу посредством вызова `getnameinfo`). Это медленнее и редко бывает полезно, если только вы не обнаружили ошибку в параллельном преобразователе Nmap (сообщите нам, если обнаружите). При сканировании IPv6 всегда используется системный преобразователь.

• `--dns-servers <сервер 1>[,<сервер 2>[,...]]` **(серверы для запросов обратного DNS).** По умолчанию Nmap определяет серверы DNS (для разрешения rDNS) из файла `resolv.conf` (Unix) или реестра (Win32). Вместо этого данным параметром можно указать другие серверы. Этот параметр не учитывается, если используется `--system-dns` или сканирование IPv6. Несколько серверов DNS часто работают быстрее, особенно если выбрать полномочные серверы для целевого пространства IP-адресов. Этот параметр также способен повысить скрытность, поскольку запросы можно перенаправлять практически через любой рекурсивный сервер DNS в Интернете.

Этот параметр полезен и при сканировании частных сетей. Иногда правильные сведения rDNS предоставляют только несколько серверов имён, причём вы можете даже не знать, где они находятся. Можно просканировать сеть на наличие порта 53 (возможно, с определением версий), а затем поочерёдно выполнять сканирование со списком Nmap (`-sL`), указывая каждый сервер имён посредством `--dns-servers`, пока не найдётся работающий.

15.5. Основы сканирования портов

Хотя с годами функциональность Nmap росла, программа начиналась как эффективный сканер портов, и это остаётся её основной функцией. Простая команда `nmap <цель>` сканирует более 1660 портов TCP узла `<цель>`. Хотя многие сканеры портов традиционно объединяли все порты в состояния open или closed, Nmap намного более детален. Он делит порты на шесть состояний: `open`, `closed`, `filtered`, `unfiltered`, `open|filtered` и `closed|filtered`.

Эти состояния не являются неотъемлемыми свойствами самого порта, а описывают то, каким его видит Nmap. Например, сканирование Nmap из той же сети, где находится цель, может показать порт 135/tcp как `open`, тогда как одновременное сканирование с теми же параметрами через Интернет может показать этот порт как `filtered`.

Шесть состояний портов, распознаваемых Nmap

• `open`. Приложение активно принимает TCP-соединения или UDP-пакеты на этом порту. Их обнаружение часто является основной целью сканирования портов. Люди, заботящиеся о безопасности, знают, что каждый открытый порт - путь для атаки. Злоумышленники и специалисты по тестированию на проникновение хотят использовать открытые порты, тогда как администраторы пытаются закрыть или защитить их брандмауэрами, не мешая законным пользователям. Открытые порты интересны и для сканирований, не связанных с безопасностью, поскольку показывают службы, доступные для использования в сети.

• `closed`. Закрытый порт доступен (он получает пакеты проб Nmap и отвечает на них), но никакое приложение его не прослушивает. Такие порты могут помочь показать, что по IP-адресу работает узел (обнаружение узла, или ping-сканирование), а также при определении ОС. Поскольку закрытые порты достижимы, возможно, стоит просканировать их позднее на случай, если некоторые откроются. Администраторы могут подумать о блокировке таких портов брандмауэром. Тогда они будут выглядеть как `filtered`, что рассматривается далее.

• `filtered`. Nmap не может определить, открыт порт или нет, поскольку фильтрация пакетов мешает пробам достичь порта. Фильтрацию может выполнять отдельное устройство-брандмауэр, правила маршрутизатора или программный брандмауэр узла. Эти порты раздражают злоумышленников, поскольку предоставляют очень мало сведений. Иногда они отвечают сообщениями об ошибке ICMP, например тип 3, код 13 (назначение недоступно: связь административно запрещена), но гораздо чаще фильтры просто отбрасывают пробы без ответа. Это вынуждает Nmap повторять попытку несколько раз на случай, если проба была отброшена из-за перегрузки сети, а не фильтрации. Сканирование резко замедляется.

• `unfiltered`. Состояние unfiltered означает, что порт доступен, но Nmap не может определить, открыт он или закрыт. Только сканирование ACK, применяемое для картографирования наборов правил брандмауэра, относит порты к этому состоянию. Сканирование портов unfiltered другими типами сканирования, такими как Window, SYN или FIN, может помочь установить, открыт ли порт.

• `open|filtered`. Nmap помещает порты в это состояние, когда не может определить, открыт порт или фильтруется. Это происходит при типах сканирования, в которых открытые порты не отвечают. Отсутствие ответа может также означать, что фильтр пакетов отбросил пробу или вызванный ею ответ. Поэтому Nmap не знает наверняка, открыт порт или фильтруется. Так классифицируют порты сканирования UDP, IP-протоколов, FIN, NULL и Xmas.

• `closed|filtered`. Это состояние применяется, когда Nmap не может определить, закрыт порт или фильтруется. Оно используется только при сканировании IP ID через незанятый узел.

15.6. Методы сканирования портов

Будучи новичком в ремонте автомобилей, я могу часами пытаться приспособить свои примитивные инструменты (молоток, клейкую ленту, гаечный ключ и так далее) к текущей задаче. Когда я терплю сокрушительную неудачу и буксирую свою развалюху к настоящему механику, он неизменно роется в огромном ящике с инструментами, пока не достаёт идеальное приспособление, с которым работа кажется совсем простой. Искусство сканирования портов устроено сходным образом. Специалисты понимают десятки методов сканирования и выбирают подходящий метод (или сочетание) для конкретной задачи. Неопытные пользователи и скрипт-кидди, напротив, пытаются решить любую проблему стандартным сканированием SYN. Поскольку Nmap бесплатен, единственное препятствие на пути к мастерству сканирования портов - знания. Это определённо лучше автомобильного мира, где может потребоваться большое умение, чтобы понять, что нужен стяжной компрессор пружины стойки, а затем всё равно придётся заплатить за него тысячи долларов.

Большинство типов сканирования доступно только привилегированным пользователям. Причина в том, что они отправляют и получают необработанные пакеты, для чего в Unix-системах требуются права `root`. В Windows рекомендуется использовать учётную запись администратора, хотя иногда Nmap работает на этой платформе и у непривилегированных пользователей, если WinPcap уже загружен в ОС. Требование прав `root` было серьёзным ограничением при выпуске Nmap в 1997 году, поскольку многим пользователям были доступны лишь общие учётные записи оболочки. Теперь мир изменился. Компьютеры подешевели, намного больше людей имеют постоянно включённый прямой доступ в Интернет, а настольные Unix-системы (включая Linux и Mac OS X) получили широкое распространение. Теперь доступна версия Nmap для Windows, позволяющая запускать программу на ещё большем числе настольных систем. По всем этим причинам пользователям реже требуется запускать Nmap из ограниченных общих учётных записей оболочки. К счастью, привилегированные параметры делают Nmap намного мощнее и гибче.

Хотя Nmap старается выдавать точные результаты, помните, что все его выводы основаны на пакетах, возвращённых целевыми машинами (или стоящими перед ними брандмауэрами). Такие узлы могут быть ненадёжны и отправлять ответы, предназначенные запутать Nmap или ввести его в заблуждение. Гораздо чаще встречаются узлы, не соответствующие RFC и не отвечающие на пробы Nmap должным образом. Сканирования FIN, NULL и Xmas особенно восприимчивы к этой проблеме. Такие вопросы относятся к конкретным типам сканирования и поэтому рассматриваются в описаниях отдельных типов.

В этом разделе описывается около дюжины методов сканирования портов, поддерживаемых Nmap. Одновременно можно использовать только один метод, за исключением того, что сканирование UDP (`-sU`) разрешается сочетать с любым одним типом сканирования TCP. Для облегчения запоминания параметры типов сканирования портов имеют вид `-s<C>`, где `<C>` - заметный символ в названии сканирования, обычно первый. Единственное исключение - устаревшее сканирование с ретрансляцией FTP (`-b`). По умолчанию Nmap выполняет сканирование SYN, хотя заменяет его сканированием с соединением, если у пользователя нет необходимых прав для отправки необработанных пакетов (для этого в Unix требуются права `root`) или указаны цели IPv6. Из перечисленных в этом разделе сканирований непривилегированные пользователи могут выполнять только сканирование с соединением и сканирование с ретрансляцией FTP.

• `-sS` **(сканирование TCP SYN).** Сканирование SYN является стандартным и самым популярным вариантом по веским причинам. Его можно выполнять быстро, сканируя тысячи портов в секунду в быстрой сети, не затруднённой ограничивающими брандмауэрами. Сканирование SYN сравнительно ненавязчиво и скрытно, поскольку никогда не завершает TCP-соединения. Оно работает с любым соответствующим стандарту стеком TCP, а не зависит от особенностей отдельных платформ, как сканирования Nmap FIN/NULL/Xmas, Maimon и через незанятый узел. Оно также позволяет чётко и надёжно различать состояния `open`, `closed` и `filtered`.

Этот метод часто называют полуоткрытым сканированием, поскольку полноценное TCP-соединение не открывается. Вы отправляете пакет SYN, словно собираетесь открыть

настоящее соединение, и затем ожидаете ответа. SYN/ACK означает, что порт прослушивается (open), тогда как RST (сброс) указывает на отсутствие прослушивателя. Если после нескольких повторных передач ответ не получен, порт помечается как filtered. Порт также помечается filtered при получении ошибки ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13).

• -sT **(сканирование с соединением TCP).** Сканирование с соединением TCP является стандартным типом сканирования TCP, когда сканирование SYN невозможно. Это происходит, когда у пользователя нет прав на необработанные пакеты или сканируются сети IPv6. Вместо создания необработанных пакетов, как при большинстве других типов сканирования, Nmap просит нижележащую операционную систему установить соединение с целевой машиной и портом, выполняя системный вызов connect. Это тот же высокоуровневый системный вызов, который веб-браузеры, клиенты P2P и большинство других сетевых приложений используют для установки соединения. Он входит в программный интерфейс, известный как API сокетов Беркли. Вместо чтения необработанных ответных пакетов из сети Nmap использует этот API, чтобы получить сведения о состоянии каждой попытки соединения.

Когда сканирование SYN доступно, обычно лучше выбрать его. У Nmap меньше контроля над высокоуровневым вызовом connect, чем над необработанными пакетами, поэтому он менее эффективен. Системный вызов завершает соединения с открытыми целевыми портами, вместо того чтобы выполнять полуоткрытый сброс, как сканирование SYN. Это не только требует больше времени и пакетов для получения тех же сведений, но и повышает вероятность регистрации соединения целевыми машинами. Приличная IDS обнаружит и то и другое, но на большинстве машин такой системы сигнализации нет. Многие службы обычной Unix-системы добавят запись в syslog, а иногда и загадочное сообщение об ошибке, когда Nmap соединится и затем закроет соединение, не отправив данных. Совсем никудышные службы при этом аварийно завершаются, хотя такое встречается редко. Администратор, увидевшая в журналах множество попыток соединения из одной системы, должна понять, что её просканировали методом connect.

• -sU **(сканирование UDP).** Хотя большинство популярных служб Интернета работают по протоколу TCP, службы UDP широко распространены. DNS, SNMP и DHCP (зарегистрированные порты 53, 161/162 и 67/68) - три самых распространённых примера. Поскольку сканирование UDP обычно медленнее и сложнее TCP, некоторые аудиторы безопасности игнорируют эти порты. Это ошибка: уязвимые службы UDP встречаются довольно часто, и злоумышленники определённо не игнорируют протокол целиком. К счастью, Nmap может помочь провести инвентаризацию портов UDP.

Сканирование UDP включается параметром -sU. Его можно сочетать с типом сканирования TCP, например сканированием SYN (-sS), чтобы проверить оба протокола за один запуск.

Сканирование UDP работает, отправляя пустой (без данных) заголовок UDP каждому целевому порту. Если возвращается ошибка ICMP «порт недоступен» (тип 3, код 3), порт имеет состояние closed. Другие ошибки ICMP «недоступно» (тип 3, коды 1, 2, 9, 10 или 13) помечают порт как filtered. Иногда служба отвечает пакетом UDP, доказывая, что порт open. Если после повторных передач ответ не получен, порт классифицируется как open|filtered. Это означает, что порт может быть открыт или связь блокируют фильтры пакетов. Определение версий (-sV) может помочь отличить действительно открытые порты от фильтруемых.

Серьёзная трудность сканирования UDP - выполнить его быстро. Открытые и фильтруемые порты редко отправляют какие-либо ответы, заставляя Nmap дожидаться истечения времени, а затем повторять передачу на случай, если проба или ответ были потеряны. Закрытые порты часто представляют ещё большую проблему. Обычно они возвращают ошибку ICMP «порт недоступен». Но в отличие от пакетов RST, отправляемых закрытыми портами TCP в ответ на сканирование SYN или connect, многие узлы по умолчанию ограничивают частоту сообщений ICMP «порт недоступен». Особенно строго это делают Linux и Solaris. Например, ядро Linux 2.4.20 ограничивает сообщения

«назначение недоступно» одним в секунду (в `net/ipv4/icmp.c`).

Nmap обнаруживает ограничение частоты и соответственно замедляется, чтобы не наводнять сеть бесполезными пакетами, которые целевая машина отбросит. К сожалению, ограничение в стиле Linux до одного пакета в секунду заставляет сканирование 65 536 портов занимать более 18 часов. Ускорить сканирование UDP можно, параллельно сканируя больше узлов, сначала быстро проверяя только популярные порты, выполняя сканирование из-за брандмауэра и применяя `--host-timeout`, чтобы пропускать медленные узлы.

• `-sN`; `-sF`; `-sX` **(сканирования TCP NULL, FIN и Xmas).** Эти три типа сканирования (с параметром `--scanflags`, описанным в следующем разделе, возможно ещё больше вариантов) используют тонкую лазейку в RFC TCP, чтобы различать открытые и закрытые порты. На странице 65 RFC 793 сказано, что если состояние порта назначения CLOSED, входящий сегмент без RST вызывает отправку ответного RST. Затем на следующей странице рассматриваются пакеты, отправленные открытым портам без установленных битов SYN, RST или ACK: «вы вряд ли сюда попадёте, но если попадёте - отбросьте сегмент и вернитесь».

При сканировании систем, соответствующих этому тексту RFC, любой пакет без битов SYN, RST или ACK приведёт к возврату RST, если порт закрыт, и полному отсутствию ответа, если порт открыт. Пока ни один из этих трёх битов не включён, допустимо любое сочетание остальных трёх (FIN, PSH и URG). Nmap использует это в трёх типах сканирования:

• **Сканирование Null (`-sN`).** Не устанавливает ни одного бита (заголовок флагов TCP равен 0).

• **Сканирование FIN (`-sF`).** Устанавливает только бит TCP FIN.

• **Сканирование Xmas (`-sX`).** Устанавливает флаги FIN, PSH и URG, зажигая пакет, словно рождественскую ёлку.

Эти три типа сканирования ведут себя совершенно одинаково, за исключением флагов TCP, установленных в пакетах проб. Получение пакета RST означает, что порт `closed`, а отсутствие ответа - что он `open|filtered`. Порт помечается `filtered`, если получена ошибка ICMP «недоступно» (тип 3, код 1, 2, 3, 9, 10 или 13).

Главное преимущество этих типов сканирования - возможность проскользнуть через некоторые брандмауэры без отслеживания состояния и маршрутизаторы с фильтрацией пакетов. Ещё одно преимущество состоит в том, что эти типы сканирования немного скрытнее даже сканирования SYN. Однако не рассчитывайте на это: большинство современных продуктов IDS можно настроить на их обнаружение. Главный недостаток заключается в том, что не все системы буквально соблюдают RFC 793. Некоторые системы отправляют RST в ответ на пробы независимо от того, открыт порт или нет. Из-за этого все порты помечаются `closed`. Среди основных операционных систем так поступают Microsoft Windows, многие устройства Cisco, BSDI и IBM OS/400. Однако это сканирование работает против большинства систем на основе Unix. Другой недостаток этих сканирований - невозможность отличить открытые порты от некоторых фильтруемых, поэтому ответ имеет вид `open|filtered`.

• `-sA` **(сканирование TCP ACK).** Это сканирование отличается от рассмотренных до сих пор тем, что никогда не определяет порты `open` (и даже `open|filtered`). Оно применяется для картографирования наборов правил брандмауэра, определения того, отслеживают ли они состояние, и выявления фильтруемых портов.

В пакете пробы сканирования ACK установлен только флаг ACK (если не используется `--scanflags`). При сканировании нефильтруемых систем и открытые, и закрытые порты возвращают пакет RST. Затем Nmap помечает их как `unfiltered`: они достижимы пакетом ACK, но остаётся неизвестным, открыты они или закрыты. Порты, которые не отвечают или возвращают определённые сообщения об ошибке ICMP (тип 3, код 1, 2, 3, 9, 10 или 13), помечаются как `filtered`.

• `-sW` **(сканирование TCP Window).** Сканирование Window в точности совпадает со сканированием ACK, за исключением того, что использует деталь реализации некоторых систем для различения открытых и закрытых портов, а не всегда печатает `unfiltered` при возврате RST. Для этого оно исследует поле TCP Window возвращённых пакетов RST. В некоторых системах открытые порты используют положительный размер окна (даже для пакетов RST), а закрытые - нулевой. Поэтому вместо того, чтобы при получении RST всегда перечислять порт как `unfiltered`, сканирование Window указывает `open` или `closed`, если значение TCP Window в этом сбросе соответственно положительно или равно нулю.

Это сканирование зависит от детали реализации меньшинства систем в Интернете, поэтому ему не всегда можно доверять. Не поддерживающие его системы обычно возвращают все порты как `closed`. Разумеется, возможно, что на машине действительно нет открытых портов. Если большинство просканированных портов закрыты, но несколько распространённых номеров портов (например, 22, 25, 53) имеют состояние `filtered`, система, скорее всего, подвержена этому методу. Иногда системы даже демонстрируют прямо противоположное поведение. Если сканирование показывает 1000 открытых портов и три закрытых или фильтруемых, эти три вполне могут быть действительно открытыми.

• `-sM` **(сканирование TCP Maimon).** Сканирование Maimon названо в честь его первооткрывателя Уриэля Маймона (Uriel Maimon). Он описал метод в выпуске 49 журнала Phrack (ноябрь 1996 года). Nmap, включавший этот метод, вышел двумя выпусками позже. Метод полностью совпадает со сканированиями NULL, FIN и Xmas, за исключением того, что проба имеет флаги FIN/ACK. Согласно RFC 793 (TCP), в ответ на такую пробу должен создаваться пакет RST независимо от того, открыт порт или закрыт. Однако Уриэль заметил, что многие системы, производные от BSD, просто отбрасывают пакет, если порт открыт.

• `--scanflags` **(нестандартное сканирование TCP).** Действительно опытные пользователи Nmap не обязаны ограничиваться готовыми типами сканирования. Параметр `--scanflags` позволяет разработать собственное сканирование, указав произвольные флаги TCP. Дайте волю воображению, попутно обходя системы обнаружения вторжений, производители которых всего лишь пролистали страницу руководства Nmap и добавили конкретные правила!

Аргументом `--scanflags` может быть числовое значение флагов, например 9 (PSH и FIN), но символьные имена использовать проще. Просто соедините любое сочетание `URG`, `ACK`, `PSH`, `RST`, `SYN` и `FIN`. Например, `--scanflags URGACKPSHRSTSYNFIN` устанавливает всё, хотя это не очень полезно для сканирования. Порядок указания флагов не имеет значения.

Помимо желаемых флагов можно задать тип сканирования TCP (например, `-sA` или `-sF`). Базовый тип сообщает Nmap, как интерпретировать ответы. Например, при сканировании SYN отсутствие ответа означает порт `filtered`, а при сканировании FIN он рассматривается как `open|filtered`. Nmap будет действовать так же, как для базового типа сканирования, но применит указанные вами флаги TCP. Если базовый тип не задан, используется сканирование SYN.

• `-sI <узел-зомби>[:<порт пробы>]` **(сканирование через незанятый узел).** Этот сложный метод позволяет выполнить действительно слепое сканирование портов TCP цели (то есть цели не отправляются пакеты с вашего настоящего IP-адреса). Вместо этого уникальная атака по побочному каналу использует предсказуемую последовательность создания идентификаторов IP-фрагментов на узле-зомби, чтобы получить сведения об открытых портах цели. Системы IDS покажут, что сканирование исходит от указанной вами машины-зомби (которая должна работать и отвечать определённым критериям). Все подробности этого увлекательного типа сканирования приведены в разделе 5.10 «Сканирование через незанятый узел (`-sI`)».

Помимо исключительной скрытности (благодаря слепому характеру), этот тип сканирования позволяет картографировать отношения доверия между машинами на основе IP. Список портов

показывает открытые порты с точки зрения узла-зомби. Поэтому можно попробовать сканировать цель с помощью различных зомби, которым, по вашему мнению, могут доверять (посредством правил маршрутизатора или фильтра пакетов).

К узлу-зомби можно добавить двоеточие с номером порта, если требуется проверить определённый порт зомби на изменения IP ID. В противном случае Nmap использует порт, применяемый по умолчанию для ping TCP (80).

• `-sO` **(сканирование IP-протоколов).** Сканирование IP-протоколов позволяет определить, какие IP-протоколы (TCP, ICMP, IGMP и так далее) поддерживаются целевыми машинами. Технически это не сканирование портов, поскольку перебираются номера IP-протоколов, а не номера портов TCP или UDP. Тем не менее оно всё равно использует параметр `-p` для выбора сканируемых номеров протоколов, сообщает результаты в обычном формате таблицы портов и даже применяет тот же нижележащий движок сканирования, что и настоящие методы сканирования портов. Поэтому оно достаточно близко к сканированию портов и рассматривается здесь.

Помимо собственной полезности, сканирование протоколов демонстрирует силу программного обеспечения с открытым исходным кодом. Хотя основная идея довольно проста, мне не приходило в голову добавить её, и я не получал запросов на такую функцию. Затем летом 2000 года Герхард Ригер (Gerhard Rieger) придумал идею, написал превосходную реализующую её заплату и отправил её в список рассылки `nmap-hackers`. Я включил заплату в дерево Nmap и на следующий день выпустил новую версию. Мало у какого коммерческого программного обеспечения есть пользователи, достаточно увлечённые, чтобы проектировать и вносить собственные улучшения!

Сканирование протоколов работает сходно со сканированием UDP. Вместо перебора поля номера порта UDP-пакета оно отправляет заголовки IP-пакетов и перебирает восьмибитное поле IP-протокола. Заголовки обычно пусты, не содержат данных и даже правильного заголовка заявленного протокола. Три исключения - TCP, UDP и ICMP. Для них включается правильный заголовок протокола, поскольку иначе некоторые системы не отправляют пакеты и поскольку в Nmap уже есть функции для их создания. Вместо сообщений ICMP «порт недоступен» сканирование протоколов ожидает сообщения ICMP «протокол недоступен». Если Nmap получает от целевого узла любой ответ по любому протоколу, этот протокол помечается как `open`. Ошибка ICMP «протокол недоступен» (тип 3, код 2) приводит к пометке протокола как `closed`. Другие ошибки ICMP «недоступно» (тип 3, код 1, 3, 9, 10 или 13) приводят к пометке протокола как `filtered` (хотя одновременно доказывают, что ICMP открыт). Если после повторных передач ответ не получен, протокол помечается как `open|filtered`.

• `-b <узел ретрансляции FTP>` **(сканирование с ретрансляцией FTP).** Интересная особенность протокола FTP (RFC 959) - поддержка так называемых прокси-соединений FTP. Она позволяет пользователю соединиться с одним FTP-сервером, а затем запросить отправку файлов третьему серверу. Такая возможность созрела для злоупотреблений на многих уровнях, поэтому большинство серверов прекратило её поддерживать. Одно из допускаемых ею злоупотреблений - заставить FTP-сервер сканировать порты других узлов. Просто поочерёдно просите FTP-сервер отправить файл каждому представляющему интерес порту целевого узла. Сообщение об ошибке опишет, открыт порт или нет. Это хороший способ обойти брандмауэры, поскольку FTP-серверы организации часто размещаются там, где имеют больший доступ к другим внутренним узлам, чем любой посторонний узел Интернета. Nmap поддерживает сканирование с ретрансляцией FTP параметром `-b`. Он принимает аргумент вида `<имя пользователя>:<пароль>@<сервер>:<порт>`. `<сервер>` - имя или IP-адрес уязвимого FTP-сервера. Как в обычном URL, часть `<имя пользователя>:<пароль>` можно опустить: тогда применяются учётные данные анонимного входа (`user: anonymous password:-wwwuser@`). Номер порта (и предшествующее двоеточие) также можно опустить; тогда используется стандартный FTP-порт (21) на `<сервере>`.

Эта уязвимость была широко распространена в 1997 году, когда вышел Nmap, но в основном уже исправлена. Уязвимые серверы всё ещё встречаются, поэтому этот способ стоит попробовать, когда всё остальное не помогло. Если ваша цель - обойти брандмауэр, просканируйте целевую сеть на открытый порт 21 (или даже на любые службы FTP, если сканируете все порты с определением версий), а затем попробуйте сканирование с ретрансляцией через каждую из них. Nmap сообщит, уязвим узел или нет. Если вы всего лишь пытаетесь замести следы, не нужно (и в действительности не следует) ограничиваться узлами целевой сети. Прежде чем сканировать случайные адреса Интернета в поисках уязвимых FTP-серверов, подумайте о том, что системные администраторы могут не оценить подобное злоупотребление их серверами.

15.7. Спецификация портов и порядок сканирования

Помимо всех рассмотренных выше методов сканирования, Nmap предлагает параметры, определяющие, какие порты сканируются и будет ли порядок сканирования случайным или последовательным. По умолчанию Nmap сканирует все порты до 1024 включительно, а также порты с большими номерами, перечисленные в файле `nmap-services` для сканируемых протоколов.

• `-p <диапазоны портов>` **(сканировать только указанные порты).** Этот параметр задаёт порты для сканирования и переопределяет стандартный набор. Можно указывать отдельные номера портов и диапазоны, разделённые дефисом (например, `1-1023`). Начальное и/или конечное значение диапазона разрешается опустить, и тогда Nmap использует соответственно 1 и 65535. Поэтому для сканирования портов от 1 до 65535 можно указать `-p-`. Порт ноль разрешается сканировать, если указать его явно. При сканировании IP-протоколов (`-sO`) этот параметр задаёт номера протоколов для сканирования (0-255).

При одновременном сканировании портов TCP и UDP конкретный протокол можно указать, поместив перед номерами портов `T:` или `U:`. Квалификатор действует до следующего квалификатора. Например, аргумент `-p U:53,111,137,T:21-25,80,139,8080` просканирует порты UDP 53, 111 и 137, а также перечисленные порты TCP. Обратите внимание: чтобы сканировать и UDP, и TCP, нужно указать `-sU` и хотя бы один тип сканирования TCP (например, `-sS`, `-sF` или `-sT`). Если квалификатор протокола не указан, номера портов добавляются во все списки протоколов.

Порты также можно указывать по именам из файла `nmap-services`. В именах разрешается применять подстановочные знаки `*` и `?`. Например, чтобы просканировать FTP и все порты, имена которых начинаются с `http`, используйте `-p ftp,http*`. Помните о раскрытии шаблонов оболочкой и, если сомневаетесь, заключайте аргумент `-p` в кавычки.

Диапазоны портов можно окружать квадратными скобками, чтобы указать только те порты внутри диапазона, которые присутствуют в `nmap-services`. Например, следующая команда просканирует все порты из `nmap-services`, равные 1024 или меньшие: `-p [-1024]`. Помните о раскрытии шаблонов оболочкой и, если сомневаетесь, заключайте аргумент `-p` в кавычки.

• `-F` **(быстрое сканирование с ограниченным набором портов).** Указывает, что следует сканировать меньше портов, чем по умолчанию. Обычно Nmap сканирует 1000 самых распространённых портов для каждого сканируемого протокола. С `-F` это число уменьшается до 100.

Чтобы знать, какие порты наиболее распространены, Nmap нужен файл `nmap-services` со сведениями о частоте (подробнее о частотах портов см. раздел 14.2 «Список общеизвестных портов: `nmap-services`»). Если сведения о частоте портов недоступны, возможно из-за использования нестандартного `nmap-services`, параметр `-F` означает сканирование только портов,

названных в файле служб (обычно Nmap сканирует все именованные порты и порты 1-1024).

• `-r` **(не выбирать порты случайно).** По умолчанию Nmap перемешивает порядок сканирования портов (за исключением того, что некоторые часто доступные порты ради эффективности перемещаются ближе к началу). Обычно такое перемешивание желательно, но для последовательного сканирования портов можно указать `-r`.

• `--port-ratio <десятичное число от 0 до 1>`**.** Сканирует все порты в файле `nmap-services`, отношение которых превышает число, указанное аргументом (только новый формат `nmap-services`).

• `--top-ports <целое число не меньше 1>`**.** Сканирует N портов с наибольшим отношением, найденных в файле `nmap-services` (только новый формат `nmap-services`).

15.8. Определение служб и версий

Направьте Nmap на удалённую машину, и он может сообщить, что открыты порты `25/tcp`, `80/tcp` и `53/udp`. Используя базу `nmap-services` примерно из 2200 общеизвестных служб, Nmap сообщит, что эти порты, вероятно, соответствуют почтовому серверу (SMTP), веб-серверу (HTTP) и серверу имён (DNS). Такое сопоставление обычно точно: подавляющее большинство фоновых служб, прослушивающих порт TCP 25, действительно являются почтовыми серверами. Однако не следует ставить на это безопасность! Люди могут запускать и запускают службы на необычных портах.

Даже если Nmap прав и на гипотетическом сервере выше работают серверы SMTP, HTTP и DNS, это не так много сведений. При оценке уязвимостей (и даже простой инвентаризации сети) своей компании или клиентов действительно нужно знать, какие именно почтовые серверы и серверы DNS и каких версий работают. Точный номер версии очень помогает определить, каким эксплойтам подвержен сервер. Определение версий помогает получить эти сведения.

После обнаружения портов TCP и/или UDP одним из других методов сканирования определение версий опрашивает эти порты, чтобы подробнее выяснить, что в действительности работает. База данных `nmap-service-probes` содержит пробы для опроса различных служб и выражения сопоставления для распознавания и разбора ответов. Nmap пытается определить протокол службы (например, FTP, SSH, Telnet, HTTP), имя приложения (например, ISC BIND, Apache httpd, Solaris telnetd), номер версии, имя узла, тип устройства (например, принтер, маршрутизатор), семейство ОС (например, Windows, Linux), а иногда и прочие подробности: открыт ли X-сервер для соединений, версию протокола SSH или имя пользователя KaZaA. Разумеется, большинство служб не предоставляет все эти сведения. Если Nmap скомпилирован с поддержкой OpenSSL, он соединяется с серверами SSL, чтобы определить службу, прослушивающую за уровнем шифрования. При обнаружении служб RPC автоматически применяется RPC grinder Nmap (`-sR`) для определения программы RPC и номеров версий. Некоторые порты UDP остаются в состоянии `open|filtered`, когда сканирование портов UDP не смогло определить, открыт порт или фильтруется. Определение версий пытается вызвать ответ от этих портов (как и от открытых) и при успехе меняет состояние на `open`. С портами TCP `open|filtered` поступают так же. Обратите внимание, что параметр Nmap `-A`, помимо прочего, включает определение версий. Подробно определение версий описано в главе 7 «Определение версий служб и приложений».

Когда Nmap получает ответы от службы, но не может сопоставить их с базой данных, он печатает специальный отпечаток и URL, по которому его можно отправить, если вы точно знаете, что работает на порту. Уделите пару минут отправке, чтобы ваша находка принесла пользу всем. Благодаря таким отправкам в Nmap имеется около 3000 шаблонов сопоставления для более чем 350 протоколов, включая SMTP, FTP, HTTP и другие.

Определение версий включается и управляется следующими параметрами:

• `-sV` **(определение версий).** Включает рассмотренное выше определение версий. Вместо него можно использовать `-A`, который, помимо прочего, включает определение версий.

• `--allports` **(не исключать никакие порты из определения версий).** По умолчанию определение версий Nmap пропускает порт TCP 9100, поскольку некоторые принтеры просто печатают всё, что отправлено этому порту, что приводит к десяткам страниц запросов HTTP GET, двоичных запросов сеанса SSL и так далее. Такое поведение можно изменить, исправив или удалив директиву `Exclude` в `nmap-service-probes`, либо указать `--allports`, чтобы сканировать все порты независимо от директив `Exclude`.

• `--version-intensity <интенсивность>` **(задать интенсивность сканирования версий).** При сканировании версий (`-sV`) Nmap отправляет ряд проб, каждой из которых присвоено значение редкости от одного до девяти. Пробы с меньшими номерами эффективны против широкого круга распространённых служб, тогда как пробы с большими номерами полезны редко. Уровень интенсивности определяет, какие пробы следует применять. Чем больше число, тем вероятнее правильное распознавание службы. Однако сканирования высокой интенсивности занимают больше времени. Интенсивность должна находиться между 0 и 9. Значение по умолчанию - 7. Если проба зарегистрирована для целевого порта посредством директивы `ports` файла `nmap-service-probes`, она выполняется независимо от уровня интенсивности. Это гарантирует, что пробы DNS всегда будут выполняться для любого открытого порта 53, проба SSL - для 443 и так далее.

• `--version-light` **(включить лёгкий режим).** Удобный псевдоним для `--version-intensity 2`. Лёгкий режим значительно ускоряет сканирование версий, но немного снижает вероятность распознать службы.

• `--version-all` **(попробовать каждую отдельную пробу).** Псевдоним для `--version-intensity 9`, гарантирующий попытку каждой отдельной пробы для каждого порта.

• `--version-trace` **(трассировать работу сканирования версий).** Заставляет Nmap печатать обширные отладочные сведения о работе сканирования версий. Это подмножество сведений, получаемых с `--packet-trace`.

• `-sR` **(сканирование RPC).** Этот метод работает совместно с различными методами сканирования портов Nmap. Он берёт все порты TCP, обнаруженные открытыми, и наводняет их командами NULL программ SunRPC в попытке определить, являются ли они портами RPC, и если да, то какую программу и номер версии обслуживают. Таким образом, фактически получаются те же сведения, что и с `rpcinfo -p`, даже если portmapper цели находится за брандмауэром (или защищён TCP wrappers). В настоящее время ложные цели не работают со сканированием RPC. Оно автоматически включается как часть сканирования версий (`-sV`), если вы его запрашиваете. Поскольку определение версий включает этот метод и намного полнее, `-sR` требуется редко.

15.9. Определение ОС

Одна из самых известных возможностей Nmap - удалённое определение ОС по отпечатку стека TCP/IP. Nmap отправляет удалённому узлу ряд пакетов TCP и UDP и исследует практически каждый бит ответов. Выполнив десятки проверок, таких как выборка TCP ISN, поддержка и порядок параметров TCP, выборка IP ID и проверка начального размера окна, Nmap сравнивает результаты со своей базой `nmap-os-db`, содержащей более тысячи известных отпечатков ОС, и при совпадении печатает подробные сведения об ОС. Каждый отпечаток включает текстовое описание ОС в свободной форме и классификацию с именем производителя (например, Sun), нижележащей ОС

(например, Solaris), поколением ОС (например, 10) и типом устройства (универсальное, маршрутизатор, коммутатор, игровая приставка и так далее).

Если Nmap не может угадать ОС машины, а условия благоприятны (например, найден хотя бы один открытый и один закрытый порт), Nmap предоставляет URL, по которому можно отправить отпечаток, если вы (наверняка) знаете работающую на машине ОС. Тем самым вы пополняете набор известных Nmap операционных систем, повышая точность для всех.

Определение ОС включает некоторые другие проверки, использующие сведения, которые всё равно собираются в ходе процесса. Одна из них - классификация предсказуемости последовательности TCP. Она приблизительно измеряет трудность установления поддельного TCP-соединения с удалённым узлом. Это полезно для эксплуатации отношений доверия на основе исходного IP-адреса (rlogin, фильтры брандмауэра и так далее) или сокрытия источника атаки. Подмена такого рода теперь выполняется редко, но многие машины всё ещё уязвимы. Фактическое число сложности основано на статистической выборке и может колебаться. Обычно лучше использовать словесную классификацию, например «worthy challenge» или «trivial joke». Она сообщается в обычном выводе только в подробном режиме (`-v`). Если подробный режим включён вместе с `-O`, также сообщается последовательность создания IP ID. Большинство машин относятся к классу «incremental»: они увеличивают поле ID в заголовке IP для каждого отправляемого пакета. Это делает их уязвимыми для нескольких сложных атак по сбору сведений и подмене.

Ещё одни дополнительные сведения, включаемые определением ОС, - предположение о времени работы цели. Для этого применяется параметр временной метки TCP (RFC 1323), чтобы угадать, когда машина перезагружалась последний раз. Предположение может быть неточным, если счётчик временных меток не инициализируется нулём либо переполняется и возвращается к началу, поэтому оно печатается только в подробном режиме.

Определение ОС рассматривается в главе 8 «Удалённое определение ОС».

Определение ОС включается и управляется следующими параметрами:

• `-O` **(включить определение ОС).** Включает рассмотренное выше определение ОС. Вместо него можно использовать `-A`, который включает определение ОС вместе с другими функциями.

• `--osscan-limit` **(ограничить определение ОС перспективными целями).** Определение ОС намного эффективнее, если найден хотя бы один открытый и один закрытый порт TCP. Установите этот параметр, и Nmap даже не будет пытаться определять ОС узлов, которые не соответствуют этому критерию. Это может сэкономить значительное время, особенно при сканировании множества узлов с `-PN`. Параметр действует, только когда определение ОС запрошено посредством `-O` или `-A`.

• `--osscan-guess`; `--fuzzy` **(угадывать результаты определения ОС).** Когда Nmap не удаётся обнаружить точное совпадение ОС, он иногда предлагает в качестве возможных вариантов близкие совпадения. По умолчанию для этого совпадение должно быть очень близким. Любой из этих равнозначных параметров заставляет Nmap угадывать агрессивнее. Nmap всё равно сообщит, что напечатано неточное совпадение, и покажет уровень уверенности (в процентах) для каждого предположения.

• `--max-os-tries` **(задать максимальное число попыток определения ОС для цели).** Когда Nmap определяет ОС цели и не находит точного совпадения, он обычно повторяет попытку. По умолчанию Nmap пытается пять раз, если условия благоприятны для отправки отпечатка ОС, и дважды, если условия не столь хороши. Меньшее значение `--max-os-tries` (например, 1) ускоряет Nmap, но исключает повторные попытки, которые могли бы распознать ОС. Вместо этого можно задать большое значение, разрешив ещё больше попыток при благоприятных условиях. Так поступают редко, кроме случаев создания лучших отпечатков для отправки и включения в базу ОС Nmap.

15.10. Движок сценариев Nmap (NSE)

Движок сценариев Nmap (NSE) - одна из самых мощных и гибких возможностей Nmap. Он позволяет пользователям писать (и распространять) простые сценарии (на языке программирования Lua[1]), автоматизирующие широкий круг сетевых задач. Эти сценарии выполняются параллельно со скоростью и эффективностью, ожидаемыми от Nmap. Пользователи могут опираться на растущий и разнообразный набор сценариев, распространяемых с Nmap, или писать собственные для нестандартных потребностей.

Среди задач, которые мы имели в виду при создании системы, были обнаружение сети, более сложное определение версий и обнаружение уязвимостей. NSE можно использовать даже для эксплуатации уязвимостей.

Чтобы отразить эти различные применения и упростить выбор запускаемых сценариев, каждый сценарий содержит поле, связывающее его с одной или несколькими категориями. В настоящее время определены категории `safe`, `intrusive`, `malware`, `version`, `discovery`, `vuln`, `auth` и `default`. Все они описаны в разделе 9.2.1 «Категории сценариев».

Движок сценариев Nmap подробно описан в главе 9 «Движок сценариев Nmap» и управляется следующими параметрами:

• `-sC`. Выполняет сканирование сценариями со стандартным набором сценариев. Эквивалентен `--script=default`. Некоторые сценарии этой категории считаются навязчивыми, и их не следует запускать против целевой сети без разрешения.

• `--script <категории сценариев>|<каталог>|<имя файла>|all`. Выполняет сканирование сценариями (как `-sC`), используя вместо стандартного набора разделённый запятыми список категорий сценариев, отдельных сценариев или каталогов со сценариями. Сначала Nmap пытается интерпретировать аргументы как категории, а затем (если это не удалось) как файлы или каталоги. Сценарий или каталог сценариев можно указать абсолютным либо относительным путём. Абсолютные пути используются как есть. Относительные пути до обнаружения просматриваются в следующих местах: `--datadir/`; `$NMAPDIR/`; `~/.nmap/` (не проверяется в Win32); `NMAPDATADIR/` или `./`. В каждом из них также проверяется подкаталог `scripts/`.

Если каталог указан и найден, Nmap загружает из него все сценарии NSE (все файлы с именами, оканчивающимися на `.nse`). Имена файлов без расширения `.nse` игнорируются. Nmap не ищет сценарии рекурсивно в подкаталогах. Если указаны отдельные имена файлов, расширение файла не обязано быть `.nse`.

По умолчанию сценарии Nmap хранятся в подкаталоге `scripts` каталога данных Nmap (см. главу 14 «Изучение и настройка файлов данных Nmap»). Для эффективности сценарии индексируются в базе данных `scripts/script.db`, где перечислены категории, к которым относится каждый сценарий. Чтобы выполнить все сценарии в базе сценариев Nmap, передайте аргумент `all`.

Вредоносные сценарии не запускаются в песочнице и потому способны повредить систему или нарушить конфиденциальность. Никогда не запускайте сценарии третьих сторон, если не доверяете авторам или не провели тщательный аудит сценариев самостоятельно.

• `--script-args <имя1>=<значение1>,<имя2>={<имя3>=<значение3>},<имя4>=<значение4>`. Позволяет передавать аргументы сценариям NSE. Аргументы передаются парами `имя=значение`. Переданный аргумент обрабатывается и сохраняется в таблице Lua, доступной всем сценариям. Имена воспринимаются как строки (они должны быть буквенно-цифровыми значениями) и используются как ключи таблицы аргументов. Значения являются строками либо самостоятельными таблицами (окружёнными { и }). Например, можно передать разделённые

запятыми аргументы: `user=bar,pass=foo,whois={whodb=nofollow+ripe}`. Строковые аргументы потенциально используются несколькими сценариями; вложенные таблицы обычно применяются только одним сценарием. В сценариях, принимающих вложенную таблицу, она обычно называется по имени сценария (как `whois` в этом примере).

• `--script-trace`**.** Делает то же, что `--packet-trace`, но на один уровень ISO выше. Если указан этот параметр, печатаются все входящие и исходящие сообщения сценария. Показанные сведения включают протокол связи, источник, цель и переданные данные. Если более 5% всех переданных данных непечатаемы, трассировка выводится в формате шестнадцатеричного дампа. Указание `--packet-trace` также включает трассировку сценариев.

• `--script-updatedb`**.** Обновляет базу сценариев `scripts/script.db`, которую Nmap использует для определения доступных стандартных сценариев и категорий. Обновлять базу требуется только после добавления или удаления сценариев NSE из стандартного каталога `scripts` либо изменения категорий сценария. Обычно этот параметр используется отдельно: `nmap --script-updatedb`.

15.11. Расчёт времени и производительность

Одним из моих главных приоритетов при разработке Nmap всегда была производительность. Стандартное сканирование (`nmap <имя узла>`) узла в моей локальной сети занимает пятую долю секунды. Этого едва хватает, чтобы моргнуть, но время накапливается при сканировании сотен или тысяч узлов. Более того, некоторые параметры сканирования, такие как сканирование UDP и определение версий, способны значительно увеличить время сканирования. То же могут сделать некоторые конфигурации брандмауэров, особенно ограничение частоты ответов. Хотя Nmap применяет параллелизм и множество сложных алгоритмов для ускорения сканирований, окончательный контроль над работой Nmap остаётся у пользователя. Опытные пользователи тщательно составляют команды Nmap, чтобы получить только нужные им сведения и уложиться в ограничения времени.

К методам сокращения времени сканирования относятся исключение некритических проверок и обновление до последней версии Nmap (улучшения производительности вносятся часто). Оптимизация параметров расчёта времени также может дать существенный эффект. Эти параметры перечислены ниже.

Некоторые параметры принимают значение времени. По умолчанию оно задаётся в миллисекундах, однако к значению можно добавить `s`, `m` или `h`, чтобы указать секунды, минуты либо часы. Поэтому аргументы `--host-timeout 900000`, `900s` и `15m` равнозначны.

• `--min-hostgroup <число узлов>`; `--max-hostgroup <число узлов>` **(изменить размеры групп параллельного сканирования).** Nmap способен параллельно сканировать порты или версии нескольких узлов. Для этого он делит целевое пространство IP-адресов на группы и сканирует по одной группе за раз. В целом большие группы эффективнее. Недостаток состоит в том, что результаты узлов нельзя предоставить до завершения всей группы. Поэтому, если Nmap начал с группы размером 50, пользователь не получит никаких отчётов (кроме обновлений, предлагаемых в подробном режиме), пока не будут завершены первые 50 узлов.

По умолчанию Nmap применяет компромиссный подход к этому противоречию. Сначала размер группы может составлять всего пять, чтобы первые результаты появились быстро, а затем увеличивается вплоть до 1024. Точные стандартные числа зависят от заданных параметров. Ради эффективности Nmap использует большие группы для UDP или сканирований TCP с малым числом портов.

Если максимальный размер группы задан параметром `--max-hostgroup`, Nmap никогда его не превысит. Укажите минимальный размер посредством `--min-hostgroup`, и Nmap постарается удерживать размер групп выше этого уровня. Nmap может потребоваться использовать меньшие группы, чем задано, если на данном интерфейсе осталось недостаточно целевых узлов для выполнения указанного минимума. Можно задать оба параметра, чтобы удерживать размер группы в определённом диапазоне, хотя это требуется редко.

Эти параметры не действуют на этапе обнаружения узлов. Это относится и к простому ping-сканированию (`-sP`). Для повышения скорости и точности обнаружение узлов всегда работает с большими группами узлов.

Основное применение этих параметров - задать большой минимальный размер группы, чтобы полное сканирование выполнялось быстрее. Распространённый выбор - 256, чтобы сканировать сеть блоками размером с класс C. При сканировании множества портов превышение этого числа вряд ли существенно поможет. Для сканирований всего нескольких номеров портов полезным может оказаться размер группы узлов 2048 и больше.

• `--min-parallelism <число проб>`; `--max-parallelism <число проб>` **(изменить параллелизм проб).** Эти параметры управляют общим числом незавершённых проб для группы узлов. Они применяются при сканировании портов и обнаружении узлов. По умолчанию Nmap рассчитывает постоянно изменяющееся идеальное значение параллелизма на основе производительности сети. Если пакеты отбрасываются, Nmap замедляется и допускает меньше незавершённых проб. Идеальное число проб постепенно растёт по мере того, как сеть доказывает свою надёжность. Эти параметры задают минимальную или максимальную границу переменной. По умолчанию идеальный параллелизм может снизиться до единицы, если сеть ненадёжна, и вырасти до нескольких сотен в идеальных условиях.

Чаще всего `--min-parallelism` задают числом больше единицы, чтобы ускорить сканирование плохо работающих узлов или сетей. Играть с этим параметром рискованно: слишком большое значение может повлиять на точность. Кроме того, это уменьшает способность Nmap динамически управлять параллелизмом в зависимости от состояния сети. Значение десять может быть разумным, хотя я изменяю его только в крайнем случае.

Параметр `--max-parallelism` иногда устанавливают равным единице, чтобы Nmap не отправлял узлам больше одной пробы одновременно. Это может быть полезно в сочетании с `--scan-delay` (рассматривается ниже), хотя последний обычно и сам достаточно хорошо справляется с задачей.

• `--min-rtt-timeout <время>`, `--max-rtt-timeout <время>`, `--initial-rtt-timeout <время>` **(изменить время ожидания проб).** Nmap поддерживает текущее значение времени ожидания, определяющее, сколько он ждёт ответа на пробу, прежде чем отказаться или передать пробу повторно. Оно рассчитывается по времени ответа предыдущих проб. Точная формула приведена в разделе 5.13 «Код и алгоритмы сканирования». Если задержка сети оказывается значительной и изменчивой, это время ожидания может вырасти до нескольких секунд. Оно также начинается с осторожного (большого) уровня и может некоторое время оставаться таким, когда Nmap сканирует не отвечающие узлы.

Значения `--max-rtt-timeout` и `--initial-rtt-timeout` ниже стандартных могут значительно сократить время сканирования. Особенно это верно для сканирований без ping (`-PN`) и сильно фильтруемых сетей. Однако не будьте слишком агрессивны. Сканирование может занять больше времени, если задать настолько малое значение, что множество проб будут истекать и передаваться повторно, пока ответ ещё находится в пути.

Если все узлы находятся в локальной сети, 100 миллисекунд - разумное агрессивное значение `--max-rtt-timeout`. Если задействована маршрутизация, сначала отправьте узлу сети ping средством ICMP ping или нестандартным генератором пакетов вроде hping2, который с большей

вероятностью пройдёт через брандмауэр. Посмотрите максимальное время двустороннего прохождения примерно десяти пакетов. Возможно, его стоит удвоить для `--initial-rtt-timeout` и утроить либо учетверить для `--max-rtt-timeout`. Обычно я не устанавливаю максимальный RTT ниже 100 мс независимо от времени ping. Также я не превышаю 1000 мс.

`--min-rtt-timeout` - редко используемый параметр, который может пригодиться, когда сеть настолько ненадёжна, что даже стандартное значение Nmap слишком агрессивно. Поскольку Nmap снижает время ожидания до минимума, только когда сеть кажется надёжной, такая потребность необычна и о ней следует сообщить как об ошибке в список рассылки `nmap-dev`.

• `--max-retries <число попыток>` **(задать максимальное число повторных передач проб сканирования портов).** Когда Nmap не получает ответа на пробу сканирования порта, это может означать, что порт фильтруется. Возможно также, что проба или ответ просто потерялись в сети. Ещё одна возможность - на целевом узле включено ограничение частоты, временно заблокировавшее ответ. Поэтому Nmap пытается снова, повторно передавая исходную пробу. Обнаружив низкую надёжность сети, Nmap может повторить попытку гораздо больше раз, прежде чем отказаться от порта. Это повышает точность, но также увеличивает время сканирования. Когда производительность критична, сканирование можно ускорить, ограничив разрешённое число повторных передач. Можно даже указать `--max-retries 0`, чтобы полностью запретить повторные передачи, хотя это рекомендуется только в ситуациях наподобие неформальных обзоров, где допустимы отдельные пропущенные порты и узлы.

По умолчанию (без шаблона `-T`) разрешено десять повторных передач. Если сеть выглядит надёжной, а целевые узлы не ограничивают частоту, Nmap обычно выполняет только одну повторную передачу. Поэтому снижение `--max-retries` до небольшого значения, например трёх, даже не влияет на большинство сканирований целей. Такие значения способны значительно ускорить сканирование медленных узлов (с ограничением частоты). Обычно при раннем отказе Nmap от портов теряется часть сведений, хотя это может быть предпочтительнее, чем позволить истечь `--host-timeout` и потерять все сведения о цели.

• `--host-timeout <время>` **(отказаться от медленных целевых узлов).** Некоторые узлы просто требуют много времени для сканирования. Причиной может быть плохо работающее или ненадёжное сетевое оборудование либо программное обеспечение, ограничение частоты пакетов или строгий брандмауэр. Несколько процентов самых медленных просканированных узлов способны поглотить большую часть времени сканирования. Иногда лучше принять потери и сначала пропустить эти узлы. Укажите в `--host-timeout` максимальное время, которое готовы ждать. Например, `30m` гарантирует, что Nmap не потратит на один узел больше получаса. Учтите, что в течение этих получаса Nmap может одновременно сканировать другие узлы, поэтому это не полная потеря. Узел с истёкшим временем пропускается. Для него не печатаются таблица портов и результаты определения ОС или версий.

• `--scan-delay <время>`; `--max-scan-delay <время>` **(изменить задержку между пробами).** Этот параметр заставляет Nmap ждать как минимум указанное время между каждой пробой, отправляемой определённому узлу. Это особенно полезно при ограничении частоты. Машины Solaris (среди многих других) обычно отвечают на пакеты проб сканирования UDP только одним сообщением ICMP в секунду. Всё, что Nmap отправит сверх этого, будет потрачено впустую. Значение `--scan-delay 1s` удержит Nmap на такой низкой скорости. Nmap пытается обнаружить ограничение частоты и соответственно изменить задержку сканирования, но явное указание не помешает, если уже известно, какая скорость подходит лучше всего.

Когда Nmap увеличивает задержку сканирования для приспособления к ограничению частоты, сканирование резко замедляется. Параметр `--max-scan-delay` задаёт наибольшую задержку, разрешённую Nmap. Низкое значение `--max-scan-delay` может ускорить Nmap, но это рискованно. Слишком малое значение способно привести к бесполезным повторным передачам пакетов и

возможным пропущенным портам, когда цель применяет строгое ограничение частоты.

Ещё одно применение `--scan-delay` - обход пороговых систем обнаружения и предотвращения вторжений (IDS/IPS). Этот метод используется в разделе «Практический пример: обход стандартных правил Snort 2.2.0», чтобы победить стандартный детектор сканирования портов в IDS Snort. Большинство других систем обнаружения вторжений можно победить тем же способом.

• `--min-rate <число>; --max-rate <число>` **(непосредственно управлять скоростью сканирования).** Динамический расчёт времени Nmap хорошо находит подходящую скорость сканирования. Однако иногда вам может быть известна подходящая скорость сканирования сети либо необходимо гарантировать завершение сканирования к определённому времени. Или, возможно, нужно не позволить Nmap сканировать слишком быстро. Параметры `--min-rate` и `--max-rate` предназначены для таких ситуаций.

Когда указан `--min-rate`, Nmap старается отправлять пакеты с заданной или большей скоростью. Аргумент - положительное вещественное число, представляющее скорость пакетов в пакетах в секунду. Например, `--min-rate 300` означает, что Nmap постарается удерживать скорость отправки на уровне не ниже 300 пакетов в секунду. Задание минимальной скорости не мешает Nmap работать быстрее, если позволяют условия.

Аналогично `--max-rate` ограничивает скорость отправки сканирования заданным максимумом. Например, используйте `--max-rate 100`, чтобы в быстрой сети ограничить отправку 100 пакетами в секунду. Используйте `--max-rate 0.1` для медленного сканирования со скоростью один пакет каждые десять секунд. Применяйте `--min-rate` и `--max-rate` вместе, чтобы удерживать скорость в определённом диапазоне.

Эти два параметра являются глобальными: они действуют на всё сканирование, а не на отдельные узлы. Они влияют только на сканирование портов и обнаружение узлов. Другие функции, такие как определение ОС, реализуют собственный расчёт времени.

Есть два условия, при которых фактическая скорость сканирования может опуститься ниже запрошенного минимума. Первое - минимальная скорость выше максимальной, с которой Nmap способен отправлять, а она зависит от оборудования. В таком случае Nmap просто будет отправлять пакеты с максимально возможной скоростью, но помните, что столь высокие скорости, вероятно, приведут к потере точности. Второй случай - Nmap нечего отправлять, например в конце сканирования, когда последние пробы уже отправлены и Nmap ожидает истечения их времени или ответа. Снижение скорости сканирования в конце либо между группами узлов нормально. Скорость отправки может временно превысить максимум, компенсируя непредсказуемые задержки, но в среднем останется на максимальном уровне или ниже.

Минимальную скорость следует задавать осторожно. Сканирование быстрее, чем поддерживает сеть, способно снизить точность. В некоторых случаях большая скорость заставляет сканирование занимать больше времени, чем при меньшей. Причина в том, что адаптивные алгоритмы повторной передачи Nmap обнаружат перегрузку сети, вызванную чрезмерной скоростью сканирования, и увеличат число повторных передач ради повышения точности. Поэтому, хотя пакеты отправляются быстрее, всего их отправляется больше. Если необходимо задать верхний предел общего времени сканирования, ограничьте число повторных передач параметром `--max-retries`.

• `--defeat-rst-ratelimit`**.** Многие узлы давно применяют ограничение частоты, чтобы уменьшить число отправляемых сообщений об ошибке ICMP (например, ошибок «порт недоступен»). Теперь некоторые системы вводят сходные ограничения частоты для создаваемых ими пакетов RST (сброс). Это может резко замедлить Nmap, который изменяет расчёт времени в соответствии с ограничениями частоты. Можно предписать Nmap игнорировать эти ограничения (для таких сканирований портов, как SYN, которые не рассматривают не отвечающие порты как открытые), указав `--defeat-rst-ratelimit`.

Использование этого параметра способно снизить точность: некоторые порты будут выглядеть не отвечающими, поскольку Nmap недостаточно долго ждал ограниченного по частоте ответа RST. При сканировании SYN отсутствие ответа приводит к пометке порта как `filtered`, а не `closed`, что наблюдается при получении пакетов RST. Этот параметр полезен, когда вас интересуют только открытые порты и различение закрытых и фильтруемых портов не стоит дополнительного времени.

• `-T paranoid|sneaky|polite|normal|aggressive|insane` **(задать шаблон расчёта времени).** Хотя рассмотренные в предыдущем разделе детальные средства управления расчётом времени мощны и эффективны, некоторые люди считают их запутанными. Более того, выбор подходящих значений иногда занимает больше времени, чем сканирование, которое требуется оптимизировать. Поэтому Nmap предлагает более простой подход с шестью шаблонами расчёта времени. Их можно указать параметром `-T` и номером (0-5) либо именем. Имена шаблонов: `paranoid` (0), `sneaky` (1), `polite` (2), `normal` (3), `aggressive` (4) и `insane` (5). Первые два предназначены для обхода IDS. Режим polite замедляет сканирование, чтобы потреблять меньше пропускной способности и ресурсов целевой машины. Режим normal используется по умолчанию, поэтому `-T3` ничего не меняет. Режим aggressive ускоряет сканирование, предполагая, что сеть достаточно быстра и надёжна. Наконец, insane предполагает исключительно быструю сеть либо готовность пожертвовать некоторой точностью ради скорости.

Эти шаблоны позволяют пользователю указать желаемую агрессивность, оставляя Nmap выбор точных значений времени. Шаблоны также вносят некоторые небольшие изменения скорости, для которых сейчас нет детальных параметров. Например, `-T4` запрещает динамической задержке сканирования превышать 10 мс для портов TCP, а `-T5` ограничивает её 5 мс. Шаблоны можно сочетать с детальными средствами управления; указанные вами детальные параметры имеют приоритет над стандартным значением шаблона расчёта времени для этого параметра. Я рекомендую использовать `-T4` при сканировании достаточно современных и надёжных сетей. Сохраняйте этот параметр и при добавлении детальных средств управления, чтобы пользоваться включаемыми им дополнительными мелкими оптимизациями.

При приличном широкополосном соединении или Ethernet я рекомендую всегда использовать `-T4`. Некоторым нравится `-T5`, хотя для меня он слишком агрессивен. Люди иногда указывают `-T2`, считая, что так меньше вероятность аварийного завершения узлов, либо потому что вообще считают себя вежливыми. Часто они не понимают, насколько медленным в действительности является `-T polite`. Сканирование может занять в десять раз больше времени, чем стандартное. Аварийные завершения машин и проблемы с пропускной способностью при стандартных параметрах расчёта времени (`-T3`) редки, поэтому осторожным сканерам я обычно рекомендую именно их. Исключение определения версий намного эффективнее снижает эти проблемы, чем игра со значениями времени.

Хотя `-T0` и `-T1` могут пригодиться для обхода предупреждений IDS, сканирование тысяч машин или портов займёт с ними исключительно много времени. Для такого долгого сканирования может быть предпочтительнее задать точные нужные значения времени вместо готовых значений `-T0` и `-T1`.

Основные эффекты `T0` - последовательное выполнение сканирования только одного порта за раз и ожидание пяти минут между отправкой каждой пробы. `T1` и `T2` похожи, но ожидают между пробами соответственно лишь 15 и 0,4 секунды. `T3` - стандартное поведение Nmap, включающее параллелизм. `-T4` эквивалентен `--max-rtt-timeout 1250 --initial-rtt-timeout 500 --max-retries 6` и устанавливает максимальную задержку сканирования TCP в 10 миллисекунд. `T5` эквивалентен `--max-rtt-timeout 300 --min-rtt-timeout 50 --initial-rtt-timeout 250 --max-retries 2 --host-timeout 15m`, а также устанавливает максимальную задержку сканирования TCP в 5 мс.

15.12. Обход брандмауэров/IDS и подмена

Многие первопроходцы Интернета представляли себе глобальную открытую сеть с универсальным пространством IP-адресов, позволяющим устанавливать виртуальные соединения между любыми двумя узлами. Это позволяет узлам выступать настоящими равноправными участниками, предоставляя друг другу сведения и получая их. Люди могли бы с работы обращаться ко всем домашним системам, изменять настройки климат-контроля или отпирать двери для пришедших раньше гостей. Это видение универсальной связанности было подавлено нехваткой адресного пространства и опасениями безопасности. В начале 1990-х организации стали развёртывать брандмауэры специально для сокращения связанности. Огромные сети отгораживались от нефильтруемого Интернета прокси приложений, преобразованием сетевых адресов и фильтрами пакетов. Неограниченный поток сведений уступил место строгому регулированию разрешённых каналов связи и проходящего по ним содержимого.

Сетевые препятствия наподобие брандмауэров способны сделать картографирование сети чрезвычайно сложным. Проще не станет, поскольку подавление случайной разведки часто является одной из главных целей установки этих устройств. Тем не менее Nmap предлагает множество функций, помогающих понять такие сложные сети и проверить, что фильтры работают как задумано. Он даже поддерживает механизмы обхода плохо реализованной защиты. Один из лучших способов понять состояние безопасности своей сети - попытаться победить её. Поставьте себя на место злоумышленника и примените методы этого раздела против своих сетей. Запустите сканирование с ретрансляцией FTP, сканирование через незанятый узел, атаку с фрагментацией или попробуйте создать туннель через один из собственных прокси.

Помимо ограничения сетевой активности компании всё чаще наблюдают за трафиком с помощью систем обнаружения вторжений (IDS). Все основные IDS поставляются с правилами для обнаружения сканирований Nmap, поскольку сканирования иногда предшествуют атакам. Многие из этих продуктов недавно превратились в системы предотвращения вторжений (IPS), которые активно блокируют трафик, признанный вредоносным. К сожалению для сетевых администраторов и производителей IDS, надёжно обнаруживать злые намерения посредством анализа данных пакетов сложно. Злоумышленники с терпением, навыками и помощью некоторых параметров Nmap обычно могут незаметно пройти мимо IDS. Тем временем администраторы вынуждены справляться с большим числом ложноположительных результатов, когда невинная активность ошибочно диагностируется и вызывает предупреждение или блокируется.

Иногда люди предлагают не включать в Nmap функции обхода правил брандмауэров или скрытого прохождения мимо IDS. Они утверждают, что злоумышленники с той же вероятностью применят эти функции во вред, с какой администраторы используют их для повышения безопасности. Проблема такой логики в том, что злоумышленники всё равно применяли бы эти методы, просто нашли бы другие средства или встроили функциональность в Nmap с помощью заплаты. При этом администраторам стало бы намного труднее выполнять свою работу. Развёртывание только современных исправленных FTP-серверов - намного более мощная защита, чем попытка не допустить распространение средств, реализующих атаку FTP bounce.

В обнаружении и преодолении брандмауэров и IDS нет волшебной пули (или параметра Nmap). Для этого нужны навыки и опыт. Учебное руководство выходит за рамки этого справочника, который лишь перечисляет соответствующие параметры и описывает их действие.

• `-f` **(фрагментировать пакеты);** `--mtu` **(использовать указанный MTU).** Параметр `-f` заставляет запрошенное сканирование (включая ping-сканирования) использовать крошечные фрагментированные IP-пакеты. Идея состоит в разделении заголовка TCP между несколькими пакетами, чтобы фильтрам пакетов, системам обнаружения вторжений и другим помехам было труднее обнаружить ваши действия. Будьте осторожны! Некоторые программы плохо

обрабатывают такие крошечные пакеты. Старый снифер Sniffit немедленно завершался с ошибкой сегментации при получении первого фрагмента. Укажите параметр один раз, и Nmap разделит пакеты на части размером не более восьми байтов после заголовка IP. Поэтому 20-байтовый заголовок TCP будет разделён на три пакета: два с восемью байтами заголовка TCP и один с последними четырьмя. Разумеется, у каждого фрагмента также есть заголовок IP. Укажите `-f` снова, чтобы использовать по 16 байтов в каждом фрагменте (и уменьшить число фрагментов). Либо задайте собственный размер смещения параметром `--mtu`. Не указывайте `-f` вместе с `--mtu`. Смещение должно быть кратно восьми. Хотя фрагментированные пакеты не пройдут через фильтры пакетов и брандмауэры, помещающие в очередь все фрагменты IP, такие как параметр `CONFIG_IP_ALWAYS_DEFRAG` ядра Linux, некоторые сети не могут позволить себе вызываемые этим потери производительности и оставляют его отключённым. Другие не могут включить его, поскольку фрагменты могут идти в сеть разными маршрутами. Некоторые исходные системы дефрагментируют исходящие пакеты в ядре. Один из примеров - Linux с модулем отслеживания соединений iptables. Выполните сканирование при работающем снифере вроде Wireshark, чтобы убедиться, что отправленные пакеты фрагментированы. Если проблемы создаёт ОС узла, попробуйте `--send-eth`, чтобы обойти уровень IP и отправлять необработанные кадры Ethernet.

Фрагментация поддерживается только для функций Nmap с необработанными пакетами, включая сканирование портов TCP и UDP (кроме сканирования connect и FTP bounce) и определение ОС. Такие функции, как определение версий и движок сценариев Nmap, обычно не поддерживают фрагментацию, поскольку для связи с целевыми службами полагаются на стек TCP вашего узла.

• `-D <ложная цель 1>[,<ложная цель 2>][,ME][,...]` **(скрыть сканирование ложными целями).** Выполняет сканирование с ложными целями, из-за чего удалённому узлу кажется, что указанные в качестве ложных целей узлы тоже сканируют целевую сеть. Поэтому их IDS может сообщить о 5-10 сканированиях портов с уникальных IP-адресов, но не определит, какой IP-адрес сканировал их, а какие были невинными ложными целями. Хотя этот способ можно победить трассировкой маршрута, отбрасыванием ответов и другими активными механизмами, в целом он эффективно скрывает ваш IP-адрес.

Разделяйте ложные узлы запятыми; при желании одну из ложных целей можно обозначить `ME`, указав позицию своего настоящего IP-адреса. Если поместить `ME` на шестую позицию или дальше, некоторые распространённые детекторы сканирования портов (например, превосходный Scanlogd Solar Designer) вряд ли вообще покажут ваш IP-адрес. Если `ME` не используется, Nmap поместит вас в случайную позицию. Можно также использовать `RND`, чтобы создать случайный незарезервированный IP-адрес, или `RND:<число>`, чтобы создать `<число>` адресов.

Обратите внимание: узлы, используемые в качестве ложных целей, должны работать, иначе можно случайно устроить SYN-флуд целям. Кроме того, определить сканирующий узел будет довольно легко, если в сети в действительности работает только один. Возможно, стоит применять IP-адреса вместо имён, чтобы сети ложных целей не видели вас в журналах серверов имён.

Ложные цели применяются и при первоначальном ping-сканировании (с ICMP, SYN, ACK или чем-либо ещё), и на этапе фактического сканирования портов. Они также используются при удалённом определении ОС (`-0`). Ложные цели не работают с определением версий или сканированием с соединением TCP. Когда действует задержка сканирования, она выдерживается между пакетами поддельных проб, а не между отдельными пробами. Поскольку ложные цели отправляются одним пакетом одновременно, они могут временно нарушать пределы управления перегрузкой.

Стоит отметить, что слишком много ложных целей способны замедлить сканирование и даже снизить его точность. Кроме того, некоторые интернет-провайдеры фильтруют поддельные пакеты, но многие вообще не ограничивают поддельные IP-пакеты.

• `-S <IP-адрес>` **(подделать исходный адрес).** При некоторых обстоятельствах Nmap не может определить ваш исходный адрес (и сообщит об этом). В такой ситуации передайте `-S` IP-адрес интерфейса, через который требуется отправлять пакеты.

Другое возможное применение этого флага - подделать сканирование, заставив цели думать, что их сканирует кто-то другой. Представьте компанию, которую конкурент постоянно сканирует по портам! Для такого применения обычно требуются параметры `-e` и `-PN`. Учтите, что ответные пакеты обычно не возвращаются вам (они будут адресованы подделываемому IP-адресу), поэтому Nmap не создаст полезных отчётов.

• `-e <интерфейс>` **(использовать указанный интерфейс).** Сообщает Nmap, через какой интерфейс отправлять и получать пакеты. Nmap должен уметь определять это автоматически, но сообщит, если не сможет.

• `--source-port <номер порта>; -g <номер порта>` **(подделать номер исходного порта).** Удивительно распространённая ошибка конфигурации - доверять трафику только по номеру исходного порта. Легко понять, как это происходит. Администратор устанавливает новенький блестящий брандмауэр и тут же получает поток жалоб от неблагодарных пользователей, чьи приложения перестали работать. В частности, DNS может сломаться, поскольку ответы UDP DNS от внешних серверов больше не способны войти в сеть. FTP - другой распространённый пример. При активной передаче FTP удалённый сервер пытается установить обратное соединение с клиентом для передачи запрошенного файла.

Безопасные решения этих проблем существуют, часто в виде прокси уровня приложений или модулей брандмауэра, разбирающих протоколы. К сожалению, есть и более простые небезопасные решения. Заметив, что ответы DNS приходят с порта 53, а активный FTP - с порта 20, многие администраторы попали в ловушку простого разрешения входящего трафика с этих портов. Они часто полагают, что злоумышленник не заметит и не использует такие дыры в брандмауэре. В других случаях администраторы считают это краткосрочной временной мерой до реализации более безопасного решения. А затем забывают об обновлении безопасности.

В эту ловушку попадают не только перегруженные сетевые администраторы. Множество продуктов поставлялось с такими небезопасными правилами. Виновна была даже Microsoft. Фильтры IPsec, поставлявшиеся с Windows 2000 и Windows XP, содержат неявное правило, разрешающее весь трафик TCP или UDP с порта 88 (Kerberos). В другом общеизвестном случае версии персонального брандмауэра Zone Alarm вплоть до 2.1.25 разрешали любые входящие пакеты UDP с исходным портом 53 (DNS) или 67 (DHCP).

Nmap предлагает равнозначные параметры `-g` и `--source-port` для эксплуатации этих слабостей. Просто укажите номер порта, и Nmap будет по возможности отправлять пакеты с этого порта. Для правильной работы некоторых проверок определения ОС Nmap обязан использовать другие номера портов, а запросы DNS игнорируют флаг `--source-port`, поскольку Nmap поручает их обработку системным библиотекам. Большинство сканирований TCP, включая SYN, полностью поддерживает параметр, как и сканирование UDP.

• `--data-length <число>` **(добавить случайные данные к отправляемым пакетам).** Обычно Nmap отправляет минималистичные пакеты, содержащие только заголовок. Поэтому пакеты TCP обычно имеют размер 40 байтов, а эхо-запросы ICMP - всего 28. Этот параметр предписывает Nmap добавлять указанное число случайных байтов к большинству отправляемых пакетов. Пакеты определения ОС (`-O`) не затрагиваются, поскольку точность требует единообразия проб, но большинство пакетов ping и сканирования портов поддерживает параметр. Он немного замедляет работу, но способен сделать сканирование чуть менее заметным.

• `--ip-options <S|R [маршрут]|L [маршрут]|T|U ...>; --ip-options <шестнадцатеричная строка>` **(отправлять пакеты с указанными параметрами IP).** Протокол IP предлагает несколько

параметров, которые можно помещать в заголовки пакетов. В отличие от повсеместно распространённых параметров TCP, параметры IP встречаются редко из-за соображений практичности и безопасности. В действительности многие маршрутизаторы Интернета блокируют самые опасные параметры, такие как маршрутизация от источника. Тем не менее в некоторых случаях параметры полезны для определения и изменения сетевого маршрута к целевым машинам. Например, параметр записи маршрута может позволить определить путь к цели, даже когда более традиционные подходы в стиле traceroute терпят неудачу. Либо, если пакеты отбрасывает определённый брандмауэр, можно задать другой маршрут посредством строгой или свободной маршрутизации от источника.

Самый мощный способ указать параметры IP - просто передать значения аргументом `--ip-options`. Перед каждым шестнадцатеричным числом ставьте `\x`, а затем две цифры. Некоторые символы можно повторять, поставив после них звёздочку и требуемое число повторений. Например, `\x01\x07\x04\x00*36\x01` - шестнадцатеричная строка, содержащая 36 байтов NUL.

Nmap также предлагает сокращённый механизм задания параметров. Просто передайте букву `R`, `T` или `U`, чтобы запросить соответственно запись маршрута, запись временных меток или обе функции вместе. Свободную или строгую маршрутизацию от источника можно указать буквой `L` или `S`, после которой идут пробел и разделённый пробелами список IP-адресов.

Чтобы увидеть параметры в отправленных и полученных пакетах, укажите `--packet-trace`. Дополнительные сведения и примеры применения параметров IP с Nmap см. по адресу http://seclists.org/nmap-dev/2006/q3/0052.html.

• `--ttl <значение>` **(задать поле времени жизни IP).** Устанавливает в отправляемых пакетах поле времени жизни IPv4 в указанное значение.

• `--randomize-hosts` **(перемешать порядок целевых узлов).** Предписывает Nmap перемешивать каждую группу численностью до 16 384 узлов перед сканированием. Это может сделать сканирования менее очевидными для различных систем наблюдения за сетью, особенно в сочетании с медленными параметрами расчёта времени. Если требуется перемешивать группы большего размера, увеличьте `PING_GROUP_SZ` в `nmap.h` и перекомпилируйте. Другой способ - создать список целевых IP-адресов сканированием со списком (`-sL -n -oN <имя файла>`), перемешать его сценарием Perl, а затем передать весь список Nmap посредством `-iL`.

• `--spoof-mac <MAC-адрес, префикс или имя производителя>` **(подделать MAC-адрес).** Просит Nmap использовать указанный MAC-адрес для всех отправляемых необработанных кадров Ethernet. Этот параметр подразумевает `--send-eth`, чтобы Nmap действительно отправлял пакеты уровня Ethernet. MAC можно задать в нескольких форматах. Если это просто число `0`, Nmap выбирает для сеанса совершенно случайный MAC-адрес. Если заданная строка состоит из чётного числа шестнадцатеричных цифр (пары при желании разделяются двоеточиями), Nmap использует их как MAC. Если предоставлено меньше 12 шестнадцатеричных цифр, Nmap заполняет остаток шести байтов случайными значениями. Если аргумент не является нулём или шестнадцатеричной строкой, Nmap ищет в `nmap-mac-prefixes` имя производителя, содержащее указанную строку (без учёта регистра). При совпадении Nmap использует OUI производителя (трёхбайтовый префикс), а оставшиеся три байта заполняет случайно. Допустимые примеры аргумента `--spoof-mac`: `Apple`, `0`, `01:02:03:04:05:06`, `deadbeefcafe`, `0020F2` и `Cisco`. Параметр влияет только на сканирования с необработанными пакетами, такие как SYN или определение ОС, а не на ориентированные на соединения функции вроде определения версий или движка сценариев Nmap.

• `--badsum` **(отправлять пакеты с неверными контрольными суммами TCP/UDP).** Просит Nmap использовать неверную контрольную сумму TCP или UDP для пакетов, отправляемых целевым узлам. Поскольку практически все стеки IP узлов правильно отбрасывают такие пакеты, любые полученные ответы, вероятно, поступают от брандмауэра или IDS, которые не стали проверять

контрольную сумму. Дополнительные сведения о методе см. по адресу http://nmap.org/p60-12.html.

15.13. Вывод

Любое средство безопасности полезно лишь настолько, насколько полезен создаваемый им вывод. Сложные проверки и алгоритмы малоценны, если их результаты не представлены организованно и понятно. Учитывая множество способов применения Nmap людьми и другими программами, ни один формат не устроит всех. Поэтому Nmap предлагает несколько форматов, включая интерактивный режим для непосредственного чтения людьми и XML для удобного разбора программами.

Помимо различных форматов вывода Nmap предоставляет параметры управления подробностью вывода и отладочными сообщениями. Типы вывода можно направлять в стандартный вывод или именованные файлы, которые Nmap способен дополнять либо перезаписывать. Файлы вывода также можно использовать для возобновления прерванных сканирований.

Nmap предоставляет вывод в пяти форматах. Стандартный называется интерактивным выводом и направляется в стандартный вывод (stdout). Существует также обычный вывод, похожий на интерактивный, но показывающий меньше сведений о ходе выполнения и предупреждений, поскольку его предполагается анализировать после завершения сканирования, а не интерактивно.

Вывод XML - один из важнейших типов вывода: его можно преобразовать в HTML, легко разобрать программами, такими как графические интерфейсы Nmap, или импортировать в базы данных.

Два оставшихся типа - простой вывод для grep, включающий большинство сведений о целевом узле в одной строке, и sCRiPt KiDDi3 OutPUt для пользователей, считающих себя k-r4d.

Хотя интерактивный вывод является стандартным и не имеет связанных параметров командной строки, остальные четыре варианта формата используют одинаковый синтаксис. Они принимают один аргумент - имя файла для сохранения результатов. Можно указать несколько форматов, но каждый формат разрешено указать только один раз. Например, можно сохранить обычный вывод для собственного просмотра и одновременно XML того же сканирования для программного анализа. Это можно сделать параметрами `-oX myscan.xml -oN myscan.nmap`. Хотя для краткости в этой главе используются простые имена наподобие `myscan.xml`, обычно рекомендуются более описательные. Выбор имён - дело личных предпочтений; я использую длинные имена, включающие дату сканирования и одно-два описывающих его слова, и помещаю их в каталог с именем сканируемой компании.

Хотя эти параметры сохраняют результаты в файлы, Nmap, как обычно, продолжает печатать интерактивный вывод в stdout. Например, команда `nmap -oX myscan.xml target` печатает XML в `myscan.xml`, а стандартный вывод заполняет теми же интерактивными результатами, которые были бы напечатаны, если бы `-oX` вообще не указывался. Это можно изменить, передав в качестве аргумента одного из типов формата символ дефиса. Тогда Nmap отключит интерактивный вывод и вместо него напечатает в стандартный поток вывода результаты в указанном формате. Поэтому команда `nmap -oX - target` отправит в stdout только вывод XML. Серьёзные ошибки всё равно могут печататься в стандартный поток ошибок stderr.

В отличие от некоторых аргументов Nmap пробел между флагом параметра файла журнала (например, `-oX`) и именем файла или дефисом обязателен. Если пропустить пробелы и передать аргументы наподобие `-oG-` или `-oXscan.xml`, средство обратной совместимости Nmap создаст файлы обычного формата с именами `G-` и `Xscan.xml` соответственно.

Все эти аргументы поддерживают в имени файла преобразования в стиле `strftime`. `%H`, `%M`, `%S`, `%m`, `%d`, `%y` и `%Y` в точности соответствуют `strftime`. `%T` равен `%H%M%S`, `%R` - `%H%M`, а `%D` - `%m%d%y`. Знак `%` с

любым другим символом просто даёт этот символ (%% создаёт знак процента). Поэтому `-oX 'scan-%T-%D.xml'` использует XML-файл вида `scan-144840-121307.xml`.

Nmap также предлагает параметры управления подробностью сканирования и добавлением данных в файлы вывода вместо их перезаписи. Все они описаны ниже.

Форматы вывода Nmap

• `-oN <спецификация файла>` **(обычный вывод).** Просит направить обычный вывод в указанное имя файла. Как рассмотрено выше, он немного отличается от интерактивного вывода.

• `-oX <спецификация файла>` **(вывод XML).** Просит направить вывод XML в указанное имя файла. Nmap включает определение типа документа (DTD), позволяющее анализаторам XML проверять вывод Nmap XML. Хотя оно предназначено главным образом для программного использования, DTD также помогает людям понимать вывод Nmap XML. DTD определяет допустимые элементы формата и часто перечисляет возможные атрибуты и их значения. Последняя версия всегда доступна по адресу http://nmap.org/data/nmap.dtd.

XML предлагает стабильный формат, который легко разбирается программным обеспечением. Бесплатные анализаторы XML доступны для всех основных языков программирования, включая C/C++, Perl, Python и Java. Для большинства этих языков написаны даже привязки, специально обрабатывающие вывод и выполнение Nmap. Примеры - Nmap::Scanner[2] и Nmap::Parser[3] в Perl CPAN. Почти во всех случаях взаимодействия нетривиального приложения с Nmap предпочтительным форматом является XML.

Вывод XML ссылается на таблицу стилей XSL, которую можно использовать для форматирования результатов как HTML. Самый простой способ - просто загрузить вывод XML в веб-браузер, например Firefox или IE. По умолчанию это сработает только на машине, где запускался Nmap (или настроенной сходным образом), из-за жёстко заданного пути файловой системы `nmap.xsl`. Используйте `--webxml` или `--stylesheet`, чтобы создавать переносимые XML-файлы, отображающиеся как HTML на любой машине с подключением к Интернету.

• `-oS <спецификация файла>` **(ScRipT KIddI3 oUTpuT).** Вывод для скрипт-кидди похож на интерактивный, но подвергается последующей обработке, чтобы лучше подходить 133t HaXXorZ, которые прежде свысока смотрели на Nmap из-за последовательного использования регистра и правописания. Людям без чувства юмора следует учесть, что этот параметр высмеивает скрипт-кидди, прежде чем ругать меня за якобы «помощь им».

• `-oG <спецификация файла>` **(вывод для grep).** Этот формат вывода рассматривается последним, поскольку признан устаревшим. Формат вывода XML намного мощнее и почти столь же удобен для опытных пользователей. XML - стандарт, для которого доступны десятки превосходных анализаторов, тогда как вывод для grep - моя собственная простая разработка. XML расширяем для поддержки новых функций Nmap по мере их выпуска, тогда как мне часто приходится исключать эти функции из вывода для grep из-за отсутствия места для них.

Тем не менее вывод для grep всё ещё довольно популярен. Это простой формат, в котором каждый узел перечислен в одной строке и который легко искать и разбирать стандартными средствами Unix, такими как grep, awk, cut, sed, diff и Perl. Даже я обычно использую его для разовых проверок в командной строке. Чтобы найти все узлы с открытым портом SSH или работающие под Solaris, достаточно простого grep для определения узлов, переданного по каналу команде awk либо cut для печати нужных полей.

Вывод для grep состоит из комментариев (строк, начинающихся со знака решётки (#)) и строк целей. Строка цели включает сочетание шести именованных полей, разделённых табуляциями и сопровождаемых двоеточием. Поля: Host, Ports, Protocols, Ignored State, OS, Seq Index, IP ID и Status.

Обычно самое важное из этих полей - Ports, предоставляющее подробности о каждом представляющем интерес порте. Это разделённый запятыми список записей портов. Каждая запись представляет один интересующий порт и состоит из семи подполей, разделённых косыми чертами (/). Эти подполя: Port number, State, Protocol, Owner, Service, SunRPC info и Version info.

Как и в случае вывода XML, эта страница руководства не позволяет описать формат целиком. Более подробное рассмотрение формата вывода Nmap для grep приведено в разделе 13.10 «Вывод для grep (-oG)».

• -oA <основное имя> **(выводить во всех форматах).** Для удобства можно указать -oA <основное имя>, чтобы одновременно сохранить результаты сканирования в обычном формате, XML и формате для grep. Они сохраняются соответственно в <основное имя>.nmap, <основное имя>.xml и <основное имя>.gnmap. Как и в большинстве программ, к именам файлов можно добавить путь каталога, например ~/nmaplogs/foocorp/ в Unix или c:\hacking\sco в Windows.

Параметры подробности и отладки

• -v **(увеличить уровень подробности).** Повышает уровень подробности, заставляя Nmap печатать больше сведений о выполняемом сканировании. Открытые порты показываются сразу после обнаружения, а оценки времени завершения предоставляются, когда Nmap считает, что сканирование займёт больше нескольких минут. Используйте параметр дважды или больше для ещё большей подробности.

Большинство изменений влияет только на интерактивный вывод, а некоторые также на обычный вывод и вывод для скрипт-кидди. Остальные типы вывода предназначены для машинной обработки, поэтому Nmap может по умолчанию предоставлять в них существенные подробности, не утомляя человека. Однако в других режимах есть несколько изменений, позволяющих значительно сократить размер вывода, исключив некоторые подробности. Например, строка комментария вывода для grep со списком всех просканированных портов печатается только в подробном режиме, поскольку может быть довольно длинной.

• -d [уровень] **(увеличить или задать уровень отладки).** Когда даже подробный режим не предоставляет достаточно данных, вам доступна отладка, способная завалить вас куда большим объёмом сведений! Как и параметр подробности (-v), отладка включается флагом командной строки (-d), а уровень отладки можно повышать, указывая его несколько раз. Кроме того, уровень отладки можно задать, передав аргумент параметру -d. Например, -d9 задаёт девятый уровень. Это наивысший действующий уровень, и он выдаст тысячи строк, если только вы не запускаете очень простое сканирование с совсем небольшим числом портов и целей.

Отладочный вывод полезен, когда подозревается ошибка в Nmap или если вы просто не понимаете, что и почему делает Nmap. Поскольку эта возможность главным образом предназначена для разработчиков, строки отладки не всегда говорят сами за себя. Вам может встретиться нечто вроде: Timeout vals: srtt: -1 rttvar: -1 to: 1000000 delta 14987 ==> srtt: 14987 rttvar: 14987 to: 100000. Если вы не понимаете строку, остаётся лишь проигнорировать её, отыскать её в исходном коде или обратиться за помощью в список рассылки разработчиков (nmap-dev). Некоторые строки понятны сами по себе, однако с повышением уровня отладки сообщения становятся всё более загадочными.

• --reason **(причины состояний узла и порта).** Показывает причину, по которой каждому порту назначено определённое состояние и каждый узел считается работающим или неработающим. Этот параметр выводит тип пакета, определившего состояние порта или узла. Например, пакет RST от закрытого порта или эхо-ответ от работающего узла. Сведения, которые может предоставить Nmap, зависят от типа сканирования или проверки доступности. SYN-сканирование и SYN-пинг (-sS и -PS) дают очень подробные сведения, но сканирование с TCP connect (-sT) ограничено реализацией системного вызова connect. Эта возможность автоматически включается

параметром отладки (`-d`), а результаты сохраняются в файлах журнала XML, даже если данный параметр не указан.

• `--packet-trace` **(трассировать отправленные и полученные пакеты и данные).** Заставляет Nmap печатать сводку каждого отправленного или полученного пакета. Это часто применяется для отладки, но также служит ценным способом для новых пользователей в точности понять, что Nmap делает за кулисами. Чтобы не печатать тысячи строк, можно указать ограниченное число сканируемых портов, например `-p20-30`. Если вас интересует только происходящее в подсистеме определения версий, используйте вместо этого `--version-trace`. Если вас интересует только трассировка сценариев, укажите `--script-trace`. С `--packet-trace` вы получаете всё перечисленное.

• `--open` **(показывать только открытые или, возможно, открытые порты).** Иногда вас интересуют только порты, к которым действительно можно подключиться (открытые), и вы не хотите загромождать результаты портами в состояниях `closed`, `filtered` и `closed|filtered`. Обычно вывод настраивают после сканирования средствами вроде grep, awk и Perl, но эта возможность была добавлена из-за огромного числа просьб. Укажите `--open`, чтобы видеть только порты в состояниях `open`, `open|filtered` и `unfiltered`. Эти три состояния обрабатываются как обычно, а значит, `open|filtered` и `unfiltered` могут сворачиваться в количества, если таких портов подавляющее большинство.

• `--iflist` **(перечислить интерфейсы и маршруты).** Печатает список интерфейсов и системные маршруты в том виде, как их определил Nmap. Это полезно для отладки проблем маршрутизации или неверного определения типа устройства (например, когда Nmap считает PPP-соединение Ethernet-соединением).

• `--log-errors` **(записывать ошибки и предупреждения в файл вывода обычного режима).** Предупреждения и ошибки, печатаемые Nmap, обычно выводятся только на экран (интерактивный вывод), оставляя незагромождёнными файлы вывода обычного формата (как правило, заданные с помощью `-oN`). Когда вы всё же хотите видеть эти сообщения в указанном файле обычного вывода, добавьте этот параметр. Он полезен, когда вы не наблюдаете за интерактивным выводом или хотите записывать ошибки во время отладки проблемы.

Сообщения об ошибках и предупреждения всё равно будут также появляться в интерактивном режиме. Это не сработает для большинства ошибок, связанных с неверными аргументами командной строки, поскольку Nmap, возможно, ещё не инициализировал файлы вывода. Кроме того, некоторые сообщения Nmap об ошибках и предупреждениях используют другую систему, которая пока не поддерживает этот параметр.

Альтернатива `--log-errors` - перенаправить интерактивный вывод (включая стандартный поток ошибок) в файл. Большинство оболочек Unix позволяет легко применить этот подход, хотя в Windows он может оказаться затруднительным.

Прочие параметры вывода

• `--append-output` **(дописывать в файлы вывода, а не затирать их).** Когда вы задаёте имя файла флагу формата вывода, такому как `-oX` или `-oN`, этот файл по умолчанию перезаписывается. Если вы предпочитаете сохранить существующее содержимое файла и дописать новые результаты, укажите параметр `--append-output`. Тогда данные будут дописываться, а не затирать содержимое всех имён файлов вывода, заданных в этом запуске Nmap. Это не очень хорошо работает с данными сканирования XML (`-oX`), поскольку итоговый файл обычно не удастся правильно разобрать, пока вы не исправите его вручную.

• `--resume <имя файла>` **(возобновить прерванное сканирование).** Некоторые обширные запуски Nmap занимают очень много времени - порядка нескольких дней. Такие сканирования не всегда завершаются. Ограничения могут запрещать запуск Nmap в рабочее время, сеть может

отключиться, машина, на которой работает Nmap, может подвергнуться запланированной или незапланированной перезагрузке, либо может аварийно завершиться сам Nmap. Администратор, запустивший Nmap, может отменить работу и по любой другой причине, нажав ctrl-C. Начинать всё сканирование заново может быть нежелательно. К счастью, если были сохранены обычные (-oN) или пригодные для grep (-oG) журналы, пользователь может попросить Nmap возобновить сканирование с той цели, на которой выполнение прекратилось. Просто укажите параметр --resume и передайте ему в качестве аргумента файл обычного либо пригодного для grep вывода. Другие аргументы не допускаются, поскольку Nmap разбирает файл вывода, чтобы использовать те же аргументы, которые были заданы прежде. Просто вызовите Nmap так: nmap --resume <имя_файла_журнала>. Nmap будет дописывать новые результаты в файлы данных, заданные в предыдущем запуске. Возобновление не поддерживает формат вывода XML, поскольку объединить два запуска в один допустимый файл XML было бы сложно.

• --stylesheet <путь или URL> **(задать таблицу стилей XSL для преобразования вывода XML).** Nmap поставляется с таблицей стилей XSL по имени nmap.xsl для просмотра или преобразования вывода XML в HTML. Вывод XML включает директиву xml-stylesheet, указывающую на nmap.xsl там, где она была первоначально установлена Nmap (либо в текущем рабочем каталоге в Windows). Просто загрузите вывод XML Nmap в современном веб-браузере, и он должен получить nmap.xsl из файловой системы и использовать её для отображения результатов. Если вы хотите использовать другую таблицу стилей, задайте её как аргумент --stylesheet. Необходимо передать полный путь или URL. Один из распространённых вариантов вызова: --stylesheet http://nmap.org/data/nmap.xsl. Это предписывает браузеру загрузить последнюю версию таблицы стилей с Nmap.Org. Параметр --webxml делает то же самое, требуя меньше набора и запоминания. Загрузка XSL с Nmap.Org упрощает просмотр результатов на машине, где нет установленного Nmap (а значит, и nmap.xsl). Поэтому URL часто полезнее, однако по соображениям конфиденциальности по умолчанию используется расположение nmap.xsl в локальной файловой системе.

• --webxml **(загрузить таблицу стилей с Nmap.Org).** Этот параметр для удобства является всего лишь псевдонимом для --stylesheet http://nmap.org/data/nmap.xsl.

• --no-stylesheet **(исключить объявление таблицы стилей XSL из XML).** Укажите этот параметр, чтобы Nmap не связывал никакую таблицу стилей XSL со своим выводом XML. Директива xml-stylesheet не включается.

15.14. Прочие параметры

В этом разделе описаны некоторые важные (и не столь важные) параметры, которым на самом деле не нашлось места где-либо ещё.

• -6 **(включить сканирование IPv6).** С 2002 года Nmap предлагает поддержку IPv6 для своих наиболее популярных возможностей. В частности, IPv6 поддерживают проверка доступности (только TCP), сканирование с установлением соединения и определение версий. Синтаксис команды остаётся обычным, за исключением того, что также добавляется параметр -6. Разумеется, если вместо имени узла указывается адрес, необходимо использовать синтаксис IPv6. Адрес может выглядеть как 3ffe:7501:4819:2000:210:f3ff:fe03:14d0, поэтому рекомендуется использовать имена узлов. Вывод выглядит как обычно; единственным признаком IPv6 будет адрес IPv6 в строке «интересующих портов».

Хотя IPv6 и не захватил мир штурмом, в некоторых странах (обычно азиатских) он используется довольно широко, и большинство современных операционных систем его поддерживает. Чтобы применять Nmap с IPv6, и источник, и цель сканирования должны быть настроены для IPv6. Если

ваш интернет-провайдер (как и большинство из них) не выделяет вам адреса IPv6, широко доступны бесплатные туннельные брокеры, которые прекрасно работают с Nmap. Я пользуюсь бесплатной службой туннельного брокера IPv6 по адресу http://www.tunnelbroker.net. Другие туннельные брокеры перечислены в Википедии[4]. Ещё один популярный бесплатный подход - туннели 6to4.

• `-A` **(параметры агрессивного сканирования).** Этот параметр включает дополнительные расширенные и агрессивные параметры. Я ещё не решил точно, что именно обозначает эта буква. Сейчас он включает определение ОС (`-O`), сканирование версий (`-sV`), сканирование сценариями (`-sC`) и трассировку маршрута (`--traceroute`). В будущем могут быть добавлены новые возможности. Смысл в том, чтобы включить полный набор параметров сканирования и не заставлять людей запоминать большое число флагов. Однако, поскольку сканирование с набором сценариев по умолчанию считается навязчивым, не следует применять `-A` к целевым сетям без разрешения. Этот параметр включает только возможности, но не параметры времени (такие как `-T4`) или подробности (`-v`), которые вам также могут понадобиться.

• `--datadir <имя каталога>` **(задать нестандартное расположение файлов данных Nmap).** Во время работы Nmap получает некоторые специальные данные из файлов `nmap-service-probes`, `nmap-services`, `nmap-protocols`, `nmap-rpc`, `nmap-mac-prefixes` и `nmap-os-db`. Если расположение какого-либо из этих файлов было задано (параметрами `--servicedb` или `--versiondb`), для этого файла используется указанное расположение. После этого Nmap ищет эти файлы в каталоге, заданном параметром `--datadir` (если он указан). Не найденные там файлы ищутся в каталоге, заданном переменной окружения `NMAPDIR`. Далее проверяется `~/.nmap` для реального и эффективного UID (только в системах POSIX) либо расположение исполняемого файла Nmap (только Win32), а затем расположение, заданное при компиляции, например `/usr/local/share/nmap` или `/usr/share/nmap`. В крайнем случае Nmap ищет в текущем каталоге.

• `--servicedb <файл служб>` **(задать нестандартный файл служб).** Предписывает Nmap использовать указанный файл служб вместо файла данных `nmap-services`, поставляемого с Nmap. Применение этого параметра также включает быстрое сканирование (`-F`). Дополнительные сведения о файлах данных Nmap приведены в описании `--datadir`.

• `--versiondb <файл проб служб>` **(задать нестандартный файл проб служб).** Предписывает Nmap использовать указанный файл проб служб вместо файла данных `nmap-service-probes`, поставляемого с Nmap. Дополнительные сведения о файлах данных Nmap приведены в описании `--datadir`.

• `--send-eth` **(использовать непосредственную отправку кадров Ethernet).** Предписывает Nmap отправлять пакеты непосредственно на уровне Ethernet (канальном), а не на более высоком уровне IP (сетевом). По умолчанию Nmap выбирает вариант, который обычно лучше всего подходит для платформы, на которой он работает. Непосредственный доступ к сокетам (уровень IP) обычно наиболее эффективен на машинах Unix, тогда как для работы в Windows необходимы кадры Ethernet, поскольку Microsoft отключила поддержку непосредственного доступа к сокетам. Несмотря на этот параметр, Nmap по-прежнему использует необработанные IP-пакеты в Unix, когда иного выбора нет (например, при соединениях не через Ethernet).

• `--send-ip` **(отправлять на уровне необработанных IP-пакетов).** Предписывает Nmap отправлять пакеты через сокеты необработанных IP-пакетов, а не посылать кадры Ethernet более низкого уровня. Это дополнение к рассмотренному выше параметру `--send-eth`.

• `--privileged` **(считать, что пользователь обладает всеми привилегиями).** Предписывает Nmap просто считать, что у него достаточно привилегий для отправки через сокеты необработанных пакетов, перехвата пакетов и подобных операций, которые обычно требуют привилегий root в системах Unix. По умолчанию Nmap завершает работу, если запрошены такие операции, но `geteuid` не равен нулю. `--privileged` полезен с возможностями ядра Linux и

подобными системами, которые могут быть настроены так, чтобы разрешать непривилегированным пользователям сканирование необработанными пакетами. Обязательно указывайте флаг этого параметра перед всеми флагами параметров, требующих привилегий (SYN-сканирование, определение ОС и т. д.). В качестве равнозначной альтернативы `--privileged` можно задать переменную окружения `NMAP_PRIVILEGED`.

• `--unprivileged` **(считать, что у пользователя нет привилегий для сокетов необработанных пакетов).** Этот параметр противоположен `--privileged`. Он предписывает Nmap считать, что у пользователя нет привилегий для сетевых сокетов необработанных пакетов и перехвата пакетов. Это полезно для тестирования, отладки или когда сетевая функциональность необработанных пакетов вашей операционной системы каким-либо образом нарушена. В качестве равнозначной альтернативы `--unprivileged` можно задать переменную окружения `NMAP_UNPRIVILEGED`.

• `--release-memory` **(освободить память перед завершением работы).** Этот параметр полезен только для отладки утечек памяти. Он заставляет Nmap освобождать выделенную память непосредственно перед завершением работы, чтобы фактические утечки памяти было легче заметить. Обычно Nmap пропускает это действие, поскольку операционная система всё равно выполняет его при завершении процесса.

• `--interactive` **(запустить в интерактивном режиме).** Запускает Nmap в интерактивном режиме, который предлагает интерактивное приглашение Nmap и позволяет легко запускать несколько сканирований (синхронно или в фоновом режиме). Это полезно для тех, кто сканирует из многопользовательских систем, поскольку они часто хотят проверить свою безопасность, не сообщая всем остальным пользователям системы, какие именно системы сканируют. Чтобы включить этот режим, используйте `--interactive`, а затем введите `h` для получения справки. Этот параметр применяется редко, поскольку полноценные оболочки обычно привычнее и обладают более полным набором возможностей. Он включает оператор «восклицательный знак» (`!`) для выполнения команд оболочки - это одна из многих причин не устанавливать Nmap с setuid root.

• `-V`; `--version` **(напечатать номер версии).** Печатает номер версии Nmap и завершает работу.

• `-h`; `--help` **(напечатать страницу со сводкой справки).** Печатает короткий экран справки с наиболее распространёнными флагами команд. Такой же результат даёт запуск Nmap без аргументов.

15.15. Взаимодействие во время работы

Во время выполнения Nmap перехватываются все нажатия клавиш. Это позволяет взаимодействовать с программой, не прерывая и не перезапуская её. Некоторые специальные клавиши изменяют параметры, а любые другие печатают сообщение о состоянии сканирования. Принято, что строчные буквы увеличивают объём вывода, а прописные уменьшают его. Для получения справки также можно нажать `?`.

• `v` / `V`. Увеличить / уменьшить уровень подробности.

• `d` / `D`. Увеличить / уменьшить уровень отладки.

• `p` / `P`. Включить / выключить трассировку пакетов.

• `?`. Напечатать экран справки по взаимодействию во время работы.

• **Любая другая клавиша.** Напечатать сообщение о состоянии наподобие следующего:

```
Stats: 0:00:08 elapsed; 111 hosts completed (5 up), 5 undergoing Service Scan

Service scan Timing: About 28.00% done; ETC: 16:18 (0:00:15 remaining)
```

15.16. Примеры

Ниже приведены примеры использования Nmap - от простых и повседневных до немного более сложных и необычных. Чтобы сделать их конкретнее, используются некоторые реальные IP-адреса и доменные имена. Вместо них следует подставить адреса или имена из собственной сети. Хотя я не считаю, что сканирование портов других сетей незаконно или должно быть незаконным, некоторые сетевые администраторы не одобряют сканирование своих сетей без приглашения и могут пожаловаться. Лучше всего сначала получить разрешение.

Для целей тестирования вам разрешено сканировать узел `scanme.nmap.org`. Это разрешение распространяется только на сканирование с помощью Nmap, но не на проверку эксплойтов или атаки типа «отказ в обслуживании». Для экономии пропускной способности, пожалуйста, не запускайте более двенадцати сканирований этого узла в сутки. Если этой бесплатной службой цели для сканирования будут злоупотреблять, её отключат, а Nmap сообщит: `Failed to resolve given hostname/IP: scanme.nmap.org`. Эти разрешения также распространяются на узлы `scanme2.nmap.org`, `scanme3.nmap.org` и так далее, хотя сейчас таких узлов не существует.

```
nmap -v scanme.nmap.org
```

Этот параметр сканирует все зарезервированные TCP-порты машины `scanme.nmap.org`. Параметр `-v` включает подробный режим.

```
nmap -sS -O scanme.nmap.org/24
```

Запускает скрытое SYN-сканирование каждой работающей машины из 255 машин сети «класса C», в которой находится Scanme. Также предпринимается попытка определить операционную систему на каждом включённом и работающем узле. Для этого требуются привилегии root из-за SYN-сканирования и определения ОС.

```
nmap -sV -p 22,53,110,143,4564 198.116.0-255.1-127
```

Запускает перечисление узлов и TCP-сканирование первой половины каждой из 255 возможных восьмибитных подсетей в адресном пространстве класса B `198.116`. При этом проверяется, работают ли системы с SSH, DNS, POP3 или IMAP на их стандартных портах либо с чем-либо на порту 4564. Для каждого из этих портов, найденного открытым, применяется определение версии, чтобы установить, какое приложение работает.

```
nmap -v -iR 100000 -PN -p 80
```

Предписывает Nmap выбрать случайным образом 100 000 узлов и просканировать их на наличие веб-серверов (порт 80). Перечисление узлов отключается с помощью `-PN`, поскольку предварительная отправка пары проб, чтобы определить, работает ли узел, расточительна, когда вы в любом случае проверяете только один порт на каждом целевом узле.

```
nmap -PN -p80 -oX logs/pb-port80scan.xml -oG logs/pb-port80scan.gnmap 216.163.128.20/20
```

Эта команда сканирует 4096 IP-адресов на наличие веб-серверов (не проверяя их доступность) и сохраняет вывод в пригодном для grep формате и формате XML.

15.17. Ошибки

Как и его автор, Nmap не идеален. Но вы можете помочь сделать его лучше, отправляя отчёты об ошибках или даже создавая исправления. Если Nmap ведёт себя не так, как вы ожидаете, сначала

обновитесь до последней версии, доступной на http://nmap.org. Если проблема сохраняется, проведите исследование, чтобы определить, не была ли она уже обнаружена и устранена. Попробуйте поискать сообщение об ошибке на нашей странице поиска по адресу http://insecure.org/search.html или в Google. Также попробуйте просмотреть архивы `nmap-dev` по адресу http://seclists.org/. Прочитайте и всю эту страницу руководства. Если всё это ничего не дало, отправьте отчёт об ошибке по адресу nmap-dev@insecure.org. Пожалуйста, включите всё, что вы узнали о проблеме, а также сведения о том, какую версию Nmap вы запускаете и под какой версией операционной системы она работает. На сообщения о проблемах и вопросы по использованию Nmap, отправленные по адресу nmap-dev@insecure.org, гораздо вероятнее ответят, чем на письма непосредственно Фёдору. Если перед отправкой сообщения вы подпишетесь на список `nmap-dev`, оно минует модерацию и пройдёт быстрее. Подписаться можно по адресу http://cgi.insecure.org/mailman/listinfo/nmap-dev.

Исправления кода для устранения ошибок ещё лучше отчётов об ошибках. Основные инструкции по созданию файлов исправлений с вашими изменениями доступны по адресу http://nmap.org/data/HACKING. Исправления можно отправлять в `nmap-dev` (рекомендуется) или непосредственно Фёдору.

15.18. Автор

Fyodor fyodor@insecure.org (http://insecure.org) За многие годы сотни людей внесли ценный вклад в Nmap. Подробности приведены в файле `CHANGELOG`, который распространяется с Nmap и также доступен по адресу http://nmap.org/changelog.

15.19. Юридические уведомления

15.19.1. Авторские права и лицензирование Nmap

Сканер безопасности Nmap защищён авторским правом (C) 1996-2008 Insecure.Com LLC. Nmap также является зарегистрированной торговой маркой Insecure.Com LLC. Эта программа является свободным программным обеспечением; вы можете распространять и (или) изменять её на условиях Стандартной общественной лицензии GNU, опубликованной Фондом свободного программного обеспечения; версии 2 с пояснениями и исключениями, описанными ниже. Это гарантирует ваше право использовать, изменять и распространять данное программное обеспечение при соблюдении определённых условий. Если вы хотите встроить технологию Nmap в проприетарное программное обеспечение, мы продаём альтернативные лицензии (обращайтесь по адресу sales@insecure.com). Десятки поставщиков программного обеспечения уже лицензируют такие технологии Nmap, как обнаружение узлов, сканирование портов, определение ОС и определение версий.

Обратите внимание, что GPL налагает важные ограничения на «производные работы», но не даёт подробного определения этого термина. Во избежание недоразумений мы считаем приложение «производной работой» для целей этой лицензии, если оно выполняет любое из следующих действий:

• Включает исходный код из Nmap.

• Читает или включает защищённые авторским правом файлы данных Nmap, такие как `nmap-os-db` или `nmap-service-probes`.

• Выполняет Nmap и разбирает результаты (в отличие от типичных оболочек или приложений с меню выполнения, которые просто показывают необработанный вывод Nmap и потому не являются производными работами).

• Встраивает, включает или объединяет Nmap с проприетарным установщиком исполняемых файлов, например созданным InstallShield.

• Связывается с библиотекой или выполняет программу, которая делает что-либо из перечисленного выше.

Следует считать, что термин «Nmap» также включает любые части или производные работы Nmap. Этот список не исчерпывающий, а предназначен лишь для пояснения нашего толкования производных работ на нескольких распространённых примерах. Эти ограничения действуют только тогда, когда вы фактически распространяете Nmap. Например, ничто не мешает вам написать и продавать проприетарный интерфейс для Nmap. Просто распространяйте его отдельно и направляйте людей на http://nmap.org для загрузки Nmap.

Мы не считаем это дополнительными ограничениями сверх GPL, а лишь разъяснением того, как мы толкуем «производные работы» применительно к нашему продукту Nmap, лицензированному по GPL. Это подобно тому, как Линус Торвальдс объявил своё толкование применимости «производных работ» к модулям ядра Linux. Наше толкование относится только к Nmap - мы не говорим от имени каких-либо других продуктов GPL.

Если у вас есть вопросы об ограничениях лицензии GPL при использовании Nmap в работах не под GPL, мы будем рады помочь. Как упоминалось выше, мы также предлагаем альтернативную лицензию для интеграции Nmap в проприетарные приложения и устройства. Такие договоры проданы многим поставщикам средств безопасности и обычно включают бессрочную лицензию, а также приоритетную поддержку и обновления и помогают финансировать дальнейшую разработку технологии Nmap. За дополнительными сведениями обращайтесь по адресу sales@insecure.com.

В качестве особого исключения из условий GPL Insecure.Com LLC предоставляет разрешение связывать код этой программы с любой версией библиотеки OpenSSL, распространяемой по лицензии, идентичной приведённой во включённом файле `COPYING.OpenSSL`, и распространять связанные комбинации, включающие обе части. Для всего используемого кода, кроме OpenSSL, вы обязаны во всех отношениях соблюдать GNU GPL. Если вы изменяете этот файл, то можете распространить это исключение на свою версию файла, но не обязаны этого делать.

Если вы получили эти файлы вместе с письменным лицензионным соглашением или договором, устанавливающим условия, отличные от приведённых выше, это альтернативное лицензионное соглашение имеет преимущественную силу перед данными комментариями.

15.19.2. Лицензия Creative Commons для этого руководства Nmap

Справочное руководство Nmap защищено авторским правом (C) 2005-2008 Insecure.Com LLC. Настоящим оно предоставляется на условиях версии 2.5 лицензии Creative Commons Attribution[5]. Это позволяет вам распространять и изменять работу по своему желанию при условии указания первоначального источника. Кроме того, вы можете решить рассматривать этот документ как подпадающий под ту же лицензию, что и сам Nmap (рассмотренную выше).

15.19.3. Доступность исходного кода и вклад сообщества

Исходный код этого программного обеспечения предоставляется потому, что мы считаем: пользователи имеют право точно знать, что программа собирается сделать, прежде чем запускать её. Это также позволяет вам проверять программное обеспечение на наличие дыр в безопасности (пока ни одной не найдено).

Исходный код также позволяет переносить Nmap на новые платформы, исправлять ошибки и добавлять новые возможности. Настоятельно рекомендуется отправлять свои изменения по адресу fyodor@insecure.org для возможного включения в основной дистрибутив. Отправляя эти изменения Фёдору или в один из списков рассылки разработчиков Insecure.Org, вы, как предполагается, предоставляете Фёдору и Insecure.Com LLC неограниченное неисключительное право повторно использовать, изменять и перелицензировать код. Nmap всегда будет доступен с открытым исходным кодом, но это важно, поскольку невозможность перелицензировать код вызвала разрушительные проблемы у других проектов свободного программного обеспечения (таких как KDE и NASM). Кроме того, время от времени мы перелицензируем код третьим сторонам, как рассматривалось выше. Если вы хотите задать особые лицензионные условия для своего вклада, просто сообщите о них при отправке.

15.19.4. Отсутствие гарантии

Эта программа распространяется в надежде, что она будет полезной, но БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ; даже без подразумеваемой гарантии ТОВАРНОЙ ПРИГОДНОСТИ или ПРИГОДНОСТИ ДЛЯ ОПРЕДЕЛЁННОЙ ЦЕЛИ. Дополнительные сведения приведены в Стандартной общественной лицензии GNU версии 2.0 по адресу http://www.gnu.org/licenses/gpl-2.0.html либо в файле `COPYING`, включённом в Nmap.

Также следует отметить, что Nmap иногда приводил к аварийному завершению плохо написанных приложений, стеков TCP/IP и даже операционных систем. Хотя это случается крайне редко, важно помнить о такой возможности. Nmap никогда не следует запускать против критически важных систем, если вы не готовы к простою. Здесь мы признаём, что Nmap может вызвать аварийное завершение ваших систем или сетей, и отказываемся от всякой ответственности за любой ущерб или проблемы, которые может причинить Nmap.

15.19.5. Ненадлежащее использование

Из-за небольшого риска аварийных завершений, а также потому, что некоторые «чёрные шляпы» любят использовать Nmap для разведки перед атакой на системы, отдельные администраторы расстраиваются и могут пожаловаться, когда их систему сканируют. Поэтому часто рекомендуется запрашивать разрешение даже перед лёгким сканированием сети.

По соображениям безопасности Nmap никогда не следует устанавливать с особыми привилегиями (например, suid root).

15.19.6. Программное обеспечение третьих сторон

Этот продукт включает программное обеспечение, разработанное Apache Software Foundation. Вместе с Nmap распространяется изменённая версия переносимой библиотеки перехвата пакетов Libpcap. Версия Nmap для Windows вместо неё использует производную от Libpcap библиотеку WinPcap. Поддержку регулярных выражений обеспечивает библиотека, являющаяся программным обеспечением с открытым исходным кодом и написанная Филипом Хейзелом. Некоторые функции

работы с необработанными сетевыми пакетами используют сетевую библиотеку Libdnet, написанную Дугом Сонгом. Изменённая версия распространяется с Nmap. Nmap может при необходимости связываться с набором криптографических средств OpenSSL для поддержки определения версий SSL. Движок сценариев Nmap использует встроенную версию языка программирования Lua. Всё стороннее программное обеспечение, описанное в этом абзаце, можно свободно распространять на условиях лицензий в стиле BSD.

15.19.7. Классификация экспортного контроля США

Экспортный контроль США: Insecure.Com LLC считает, что Nmap подпадает под номер классификации экспортного контроля США (ECCN) 5D992. Эта категория называется «программное обеспечение информационной безопасности, не контролируемое по 5D002». Единственное ограничение этой классификации - AT (борьба с терроризмом), которое распространяется почти на все товары и запрещает экспорт в несколько стран-изгоев, таких как Иран и Северная Корея. Поэтому экспорт Nmap не требует специальной лицензии, разрешения или иного одобрения государственных органов.

Сноски

[1] [назад] http://lua.org
[2] [назад] http://sourceforge.net/projects/nmap-scanner/
[3] [назад] http://nmapparser.wordpress.com/
[4] [назад] http://en.wikipedia.org/wiki/List_of_IPv6_tunnel_brokers
[5] [назад] http://creativecommons.org/licenses/by/2.5/

Приложение A. DTD вывода Nmap в формате XML

A.1. Назначение

Это определение типа документа (DTD) используется анализаторами XML для проверки вывода Nmap в формате XML. Последняя версия всегда доступна по адресу http://nmap.org/data/nmap.dtd. Хотя оно предназначено прежде всего для программного использования, здесь оно включено из-за своей ценности для людей, помогающей интерпретировать вывод Nmap в формате XML. DTD определяет допустимые элементы формата и часто перечисляет атрибуты и значения, которые они могут принимать. Использование DTD подробнее рассматривается в разделе 13.6 «Вывод XML (-oX)».

A.2. Полное DTD

```
<!--
    nmap.dtd
    This is the DTD for Nmap's XML output (-oX) format.
    $Id: nmap.dtd 11010 2008-11-10 19:05:12Z david $

    Originally written by:
    William McVey <wam@cisco.com> <wam+nmap@wamber.net>

    Now maintained by Fyodor <fyodor@insecure.org> as part of Nmap.

    To validate using this file, simply add a DOCTYPE line similar to:
    <!DOCTYPE nmaprun SYSTEM "nmap.dtd">
    to the nmap output immediately below the prologue (the first line).  This
    should allow you to run a validating parser against the output (so long
    as the DTD is in your parser's DTD search path).

    Bugs:
    Most of the elements are "locked" into the specific order that nmap
    generates, when there really is no need for a specific ordering.
    This is primarily because I don't know the xml DTD construct to
    specify "one each of this list of elements, in any order".  If there
    is a construct similar to SGML's '&' operator, please let me know.

    Portions Copyright (c) 2001-2008 Insecure.Com LLC
    Portions Copyright (c) 2001 by Cisco systems, Inc.

    Permission to use, copy, modify, and distribute modified and
    unmodified copies of this software for any purpose and without fee is
    hereby granted, provided that (a) this copyright and permission notice
    appear on all copies of the software and supporting documentation, (b)
    the name of Cisco Systems, Inc. not be used in advertising or
    publicity pertaining to distribution of the program without specific
    prior permission, and (c) notice be given in supporting documentation
    that use, modification, copying and distribution is by permission of

    Cisco Systems, Inc.

    Cisco Systems, Inc. makes no representations about the suitability
    of this software for any purpose.  THIS SOFTWARE IS PROVIDED ``AS
    IS'' AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
    WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
    FITNESS FOR A PARTICULAR PURPOSE.
-->
<!-- parameter entities to specify common "types" used elsewhere in the DTD -->
```

```
<!ENTITY % attr_alpha "CDATA" >
<!ENTITY % attr_numeric "CDATA" >
<!ENTITY % attr_ipaddr "CDATA" >
<!ENTITY % attr_percent "CDATA" >
<!ENTITY % attr_type "(ipv4 | ipv6 | mac)" >

<!ENTITY % host_states "(up|down|unknown|skipped)" >

<!-- see: nmap.c:statenum2str for list of port states -->
<!-- Maybe they should be enumerated as in scan_types below , but I -->
<!-- don't know how to escape states like open|filtered -->
<!ENTITY % port_states "CDATA" >

<!ENTITY % hostname_types "(PTR)" >

<!-- see output.c:output_xml_scaninfo_records for scan types -->
<!ENTITY % scan_types "(syn|ack|bounce|connect|null|xmas|window|maimon|fin|udp|ipproto)" >

<!-- <!ENTITY % ip_versions "(ipv4)" > -->

<!ENTITY % port_protocols "(ip|tcp|udp)" >

<!-- I don't know exactly what these are, but the values were enumerated via:
     grep "conf=" *
-->
<!ENTITY % service_confs  "( 0 | 3 | 5 | 10)" >

<!-- This element was started in nmap.c:nmap_main().
     It represents to the topmost element of the output document.
-->
<!ELEMENT nmaprun      (scaninfo*, verbose, debugging,
                        ((taskbegin, taskprogress*, taskend) | host | output)*,
                        runstats) >
<!ATTLIST nmaprun
            scanner        (nmap)        #REQUIRED
            args        CDATA        #IMPLIED
            start        %attr_numeric;    #IMPLIED
            startstr    CDATA            #IMPLIED
            version        CDATA        #REQUIRED

            profile_name    CDATA        #IMPLIED
            xmloutputversion CDATA        #REQUIRED
>

<!-- this element is written in output.c:doscaninfo() -->
<!ELEMENT scaninfo    EMPTY >
<!ATTLIST scaninfo
            type        %scan_types;     #REQUIRED
            scanflags    CDATA         #IMPLIED
            protocol    %port_protocols; #REQUIRED
            numservices    %attr_numeric;     #REQUIRED
            services    CDATA         #REQUIRED
>

<!-- these elements are written in nmap.c:nmap_main() -->
<!ELEMENT verbose    EMPTY >
<!ATTLIST verbose    level        %attr_numeric;    #IMPLIED >

<!ELEMENT debugging     EMPTY >
<!ATTLIST debugging    level        %attr_numeric;    #IMPLIED >

<!-- this element is written in timing.c:beginOrEndTask() -->
<!ELEMENT taskbegin    EMPTY >
<!ATTLIST taskbegin
            task        CDATA        #REQUIRED
            time        %attr_numeric;    #REQUIRED
            extrainfo    CDATA        #IMPLIED
>

<!-- this element is written in timing.c:printStats() -->
<!ELEMENT taskprogress    EMPTY >
```

```
<!ATTLIST taskprogress
            task        CDATA        #REQUIRED
            time        %attr_numeric;    #REQUIRED
            percent        %attr_percent;    #REQUIRED
            remaining    %attr_numeric;    #REQUIRED
            etc        %attr_numeric;    #REQUIRED
>

<!-- this element is written in timing.c:beginOrEndTask() -->
<!ELEMENT taskend    EMPTY >
<!ATTLIST taskend
            task        CDATA        #REQUIRED
            time        %attr_numeric;    #REQUIRED
            extrainfo    CDATA        #IMPLIED
>

<!--

     this element is started in nmap.c:nmap_main() and filled by
     output.c:write_host_status(), output.c:printportoutput(), and
     output.c:printosscanoutput()
-->
<!ELEMENT host        ( status, address , (address | hostnames |
                          smurf | ports | os | distance | uptime |
                          tcpsequence | ipidsequence | tcptssequence |
                          hostscript | trace)*, times ) >
<!ATTLIST host
            starttime    %attr_numeric;    #IMPLIED
            endtime        %attr_numeric;    #IMPLIED
            comment        CDATA        #IMPLIED
>

<!-- these elements are written by output.c:write_xml_initial_hostinfo() -->
<!ELEMENT status    EMPTY >
<!ATTLIST status    state        %host_states;    #REQUIRED
          reason    CDATA        #REQUIRED
 >

<!ELEMENT address    EMPTY >
<!ATTLIST address
            addr        %attr_ipaddr;    #REQUIRED
            addrtype    %attr_type;    "ipv4"
            vendor        CDATA        #IMPLIED
>

<!ELEMENT hostnames    (hostname)* >
<!ELEMENT hostname    EMPTY >
<!ATTLIST hostname
            name        CDATA        #IMPLIED
            type        %hostname_types; #IMPLIED
>

<!-- this element is written by output.c:write_host_status() -->
<!ELEMENT smurf        EMPTY >
<!ATTLIST smurf        responses    %attr_numeric;    #REQUIRED >

<!-- these elements are written by output.c:printportoutput() -->

<!ELEMENT ports        (extraports* , port*) >

<!ELEMENT extraports    (extrareasons)* >
<!ATTLIST extraports
            state        %port_states;    #REQUIRED
            count        %attr_numeric;  #REQUIRED
>

<!ELEMENT extrareasons EMPTY >
<!ATTLIST extrareasons
            reason        CDATA    #REQUIRED
            count        CDATA    #REQUIRED
>
```

```
<!ELEMENT port        (state , owner? , service?, script*) >
<!ATTLIST port
            protocol    %port_protocols;    #REQUIRED
            portid        %attr_numeric;    #REQUIRED
>

<!ELEMENT state        EMPTY >
<!ATTLIST state
        state        %port_states;    #REQUIRED
        reason        CDATA    #REQUIRED
        reason_ttl    CDATA    #REQUIRED
        reason_ip    CDATA    #IMPLIED
>

<!ELEMENT owner        EMPTY >
<!ATTLIST owner        name        CDATA        #REQUIRED >

<!ELEMENT service    EMPTY >
<!ATTLIST service
            name        CDATA        #REQUIRED
            conf        %service_confs;    #REQUIRED
                        method          (table|detection|probed) #REQUIRED
                        version         CDATA           #IMPLIED
                        product         CDATA           #IMPLIED
                        extrainfo       CDATA           #IMPLIED
            tunnel        (ssl)        #IMPLIED
            proto        (rpc)        #IMPLIED
            rpcnum        %attr_numeric;    #IMPLIED
            lowver        %attr_numeric;    #IMPLIED
            highver        %attr_numeric;    #IMPLIED
                        hostname        CDATA           #IMPLIED
                        ostype          CDATA           #IMPLIED
                        devicetype      CDATA           #IMPLIED
                        servicefp       CDATA           #IMPLIED
>

<!ELEMENT script    EMPTY >
<!ATTLIST script
    id    CDATA    #REQUIRED
    output    CDATA    #REQUIRED
>

<!ELEMENT os        ( portused* , osclass*, osmatch*, osfingerprint* ) >

<!ELEMENT portused    EMPTY >
<!ATTLIST portused
            state         %port_states;    #REQUIRED
            proto         %port_protocols; #REQUIRED
            portid         %attr_numeric;    #REQUIRED
>
<!ELEMENT osclass      EMPTY >
<!ATTLIST osclass
                        vendor          CDATA           #REQUIRED
                        osgen           CDATA           #IMPLIED
                        type            CDATA           #IMPLIED
                        accuracy        CDATA           #REQUIRED
                        osfamily        CDATA           #REQUIRED
>

<!ELEMENT osmatch    EMPTY >
<!ATTLIST osmatch
            name        CDATA        #REQUIRED
            accuracy    %attr_numeric;    #REQUIRED
            line        %attr_numeric;    #REQUIRED
>

<!ELEMENT osfingerprint    EMPTY >
<!ATTLIST osfingerprint
            fingerprint    CDATA        #REQUIRED
>
<!ELEMENT distance    EMPTY >
```

```
<!ATTLIST distance
            value        %attr_numeric;    #REQUIRED
>

<!ELEMENT uptime    EMPTY >
<!ATTLIST uptime
            seconds        %attr_numeric;    #REQUIRED
            lastboot    CDATA        #IMPLIED
>

<!ELEMENT tcpsequence    EMPTY >
<!ATTLIST tcpsequence
            index        %attr_numeric;    #REQUIRED
            difficulty    CDATA        #REQUIRED
            values        CDATA        #REQUIRED
>

<!ELEMENT ipidsequence    EMPTY >
<!ATTLIST ipidsequence

            class        CDATA        #REQUIRED
            values        CDATA        #REQUIRED
>

<!ELEMENT tcptssequence    EMPTY >
<!ATTLIST tcptssequence
            class        CDATA        #REQUIRED
            values        CDATA        #IMPLIED
>

<!ELEMENT trace (hop*, error?) >
<!ATTLIST trace
      proto   CDATA   #REQUIRED
      port    CDATA   #REQUIRED
>

<!ELEMENT hop EMPTY>
<!ATTLIST hop
      ttl     CDATA   #REQUIRED
      rtt     CDATA   #IMPLIED
      ipaddr  CDATA   #IMPLIED
      host    CDATA   #IMPLIED
>

<!ELEMENT error EMPTY>
<!ATTLIST error
     errorstr   CDATA #IMPLIED
>

<!ELEMENT times EMPTY>
<!ATTLIST times
    srtt    CDATA    #REQUIRED
    rttvar    CDATA    #REQUIRED
    to    CDATA    #REQUIRED
>

<!-- For embedding another type of output (screen output) like Zenmap does. -->
<!ELEMENT output (#PCDATA)>
<!ATTLIST output type  (interactive)  #IMPLIED>

<!-- these elements are generated in output.c:printfinaloutput() -->
<!ELEMENT runstats    (finished, hosts) >

<!ELEMENT finished    EMPTY >
<!ATTLIST finished    time        %attr_numeric;    #REQUIRED
                        timestr        CDATA            #IMPLIED

>

<!ELEMENT hosts        EMPTY >
<!ATTLIST hosts
```

```
            up        %attr_numeric;    "0"
            down        %attr_numeric;    "0"
            total        %attr_numeric;    #REQUIRED
>

<!ELEMENT hostscript ( script+ )>
```